DevOps Interview Question Bank

529 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based

Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Linux, Shell Scripting & Networking	22	21	43	86
Docker & Containers	17	13	9	39
Kubernetes	31	30	28	89
CI/CD, Jenkins, GitOps & Ansible	38	23	18	79
Terraform & Infrastructure as Code	15	20	13	48
AWS	18	36	27	81
GCP, Azure & DevSecOps	12	12	8	32
Monitoring, Observability, SRE & Scale Design	9	15	8	32
Behavioral & Project Experience	9	0	10	19
Shell Scripting: 24 Real Interview Scripts	7	8	9	24
All topics	178	178	173	**529**

Table of Contents

Linux, Shell Scripting & Networking **5**

Basic 5

Advanced 21

Scenario-Based 39

Docker & Containers **75**

Basic 75

Advanced 89

Scenario-Based 101

Kubernetes **110**

Basic 110

Advanced 141

Scenario-Based 173

CI/CD, Jenkins, GitOps & Ansible **204**

Basic 204

Advanced 237

Scenario-Based 266

Terraform & Infrastructure as Code **284**

Basic 284

Advanced 297

Scenario-Based 320

AWS **336**

Basic 336

Advanced 350

Scenario-Based 378

GCP, Azure & DevSecOps **405**

Basic 405

Advanced 416

Scenario-Based 429

Monitoring, Observability, SRE & Scale Design **439**

Basic 439
Advanced 446
Scenario-Based 458

Behavioral & Project Experience **467**

Basic 467
Scenario-Based 473

Shell Scripting: 24 Real Interview Scripts **480**

Basic 480
Advanced 487
Scenario-Based 497

Linux, Shell Scripting & Networking

Basic

Q1. How do you create a new user in Linux, and where is the default home directory created?

> **In simple words:** Creating a user is like giving a new employee an ID card plus a personal locker. The ID card is the account, the locker is the home folder where their personal stuff lives.

Detailed answer

Use `useradd` (low-level) or `adduser` (friendly, interactive wrapper on Debian/Ubuntu).

- `useradd -m -s /bin/bash priya` -> `-m` creates the home directory, `-s` sets the login shell.
- Default home location is **/home/<username>** (for root it is `/root`). The default comes from `/etc/default/useradd` (`HOME=/home`) and `/etc/login.defs`.
- New home folders are pre-filled with files copied from **/etc/skel** (like `.bashrc`).
- Set password with `passwd priya`; add to a group like sudo with `usermod -aG sudo priya` (on RHEL/Amazon Linux the admin group is `wheel`).
- User details are stored in `/etc/passwd`, hashed passwords in `/etc/shadow`.

Note: plain `useradd` on some distros does NOT create the home folder unless you pass `-m`.

Example:

```
sudo useradd -m -s /bin/bash -c "Priya DevOps" priya
sudo passwd priya
sudo usermod -aG sudo priya
grep priya /etc/passwd
# priya:x:1001:1001:Priya DevOps:/home/priya:/bin/bash
ls -la /home/priya          # .bashrc .profile copied from /etc/skel
```

> ***What to say to the interviewer:*** *I use* `useradd -m -s /bin/bash <user>` *or* `adduser` *on Ubuntu, then* `passwd` *and* `usermod -aG` *for groups. The home directory is created under /home/<user> by default, seeded from /etc/skel, and the defaults come from /etc/default/useradd and /etc/login.defs.*

Q2. How do you search for files containing a specific string, and print only the file names?

(also asked as: find all files containing a specific sentence and print only the file names)

> **In simple words:** It is like asking a librarian "which books mention the word X?" and wanting just the list of book titles, not every page where it appears.

Detailed answer

`grep` is the tool. Key flags:

- `-r` / `-R` -> search recursively through folders.

- `-l` -> print only the **file names** that match (not the matching lines).
- `-i` -> ignore case; `-n` -> show line numbers; `-w` -> whole word only.
- `-F` -> treat the pattern as a fixed string (safe for sentences with dots or brackets).
- `--include="*.log"` -> limit to certain file types.
- For a full sentence, wrap it in quotes so the shell passes it as one argument.

You can also combine `find` + `grep` when you need filters like age or size.

Example:

```
# File names only, recursive, exact sentence
grep -rlF "Connection refused by upstream" /var/log/

# Case-insensitive, only .conf files
grep -rli --include="*.conf" "max_connections" /etc/

# With find: only files modified in last 2 days
find /var/log -type f -mtime -2 -exec grep -lF "OutOfMemory" {} +

# Count of matching files
grep -rlF "ERROR" /app/logs | wc -l
```

> ***What to say to the interviewer:*** *I use* `grep -rl "text" /path` `-r` *for recursive and* `-l` *to print only file names; for a full sentence I add* `-F` *so it is matched literally. If I need extra filters like file age or type, I combine it with* `find ... -exec grep -l`.

Q3. How would you validate command-line arguments in a shell script?

> **In simple words:** Before a chef starts cooking, they check the ingredients are actually on the counter. A script should also check it was given the right inputs before doing anything risky.

Detailed answer

Useful building blocks:

- `$#` -> number of arguments; `$1`, `$2` -> individual arguments; `$@` -> all arguments.
- Check count: `if [ $# -ne 2 ]; then usage; fi`.
- Check empty: `[ -z "$1" ]`; check file exists: `[ -f "$file" ]`; directory: `[ -d "$dir" ]`.
- Check a number with a regex: `[[ "$port" =~ ^[0-9]+$ ]]`.
- Use `getopts` for flags like `-e prod -v`.
- Print a `usage` message and `exit 1` (non-zero = failure) so CI pipelines notice the error.
- Add `set -euo pipefail` at the top so the script stops on errors and unset variables.

Example:

```
#!/usr/bin/env bash
set -euo pipefail

usage() { echo "Usage: $0 <env: dev|stage|prod> <port>"; exit 1; }

[ $# -eq 2 ] || usage
ENV="$1"; PORT="$2"

case "$ENV" in
  dev|stage|prod) ;;
  *) echo "Invalid env: $ENV"; usage ;;
esac

[[ "$PORT" =~ ^[0-9]+$ ]] && [ "$PORT" -le 65535 ] || { echo "Invalid port"; exit 2; }

echo "Deploying to $ENV on port $PORT"
```

What to say to the interviewer: *I check* `$#` *for the argument count, validate each value with tests like* `-z`, `-f` *or a regex, and use* `getopts` *for flags. On bad input I print a usage message and exit with a non-zero code, and I start scripts with* `set -euo pipefail` *so they fail fast.*

Q4. How do Linux file permissions work (chmod, chown)?

In simple words: Every file has a lock with three keyholders: the owner, the owner's team (group), and everyone else. Each can be allowed to read, write, or run it.

Detailed answer

`ls -l` shows something like `-rwxr-x---`:

- 1st char: type (`-` file, `d` directory, `l` link).
- Next 3 groups: **user / group / others**, each with `r` (read=4), `w` (write=2), `x` (execute=1).
- `chmod 750 deploy.sh` -> owner rwx (7), group r-x (5), others nothing (0).
- `chmod +x script.sh` -> make it executable.
- `chown appuser:appgroup file` -> change owner and group.
- On directories, `x` means "can enter the folder".
- Special bits: setuid, setgid, sticky bit (`/tmp` uses sticky so users cannot delete each other's files).
- Private keys must be `600` or SSH will refuse them.

Example:

```
ls -l deploy.sh
# -rw-r--r-- 1 ubuntu ubuntu 512 Sep 20 10:00 deploy.sh
chmod 750 deploy.sh
sudo chown jenkins:devops deploy.sh
chmod 600 ~/.ssh/id_rsa
ls -ld /tmp     # drwxrwxrwt  <- 't' = sticky bit
```

What to say to the interviewer: *Permissions are read/write/execute for user, group and others, shown by* `ls -l` *and set with* `chmod` *either symbolically or in octal like 755 or 600. I use* `chown` *to change ownership, and I follow least privilege, for example SSH keys at 600 and scripts at 750.*

Q5. How do you manage services and read their logs (systemd, systemctl, journalctl)?

In simple words: systemd is the building manager that starts all the machines (services) when the building opens and restarts them if they break. journalctl is the manager's logbook.

Detailed answer

- `systemctl status nginx` -> is it running, recent log lines, PID.
- `systemctl start|stop|restart|reload nginx`.
- `systemctl enable nginx` -> start automatically on boot.
- `systemctl list-units --failed` -> what is broken right now.
- `journalctl -u nginx -f` -> follow logs for one service.
- `journalctl -u nginx --since "1 hour ago"`; `journalctl -p err -b` -> errors since last boot.
- `journalctl --disk-usage` and `journalctl --vacuum-size=500M` -> control log size.
- Unit files live in `/etc/systemd/system/`; after editing run `systemctl daemon-reload`.
- `Restart=on-failure` in a unit file gives auto-healing.

Example:

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My App
After=network.target

[Service]
User=appuser
ExecStart=/opt/myapp/bin/start.sh
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload && sudo systemctl enable --now myapp
journalctl -u myapp -f
```

What to say to the interviewer: *I use* `systemctl` *to start, stop, enable and check services, and* `journalctl -u <service>` *to read their logs with filters like* `--since` *and* `-p err`*. For custom apps I write a unit file with* `Restart=on-failure` *so systemd auto-restarts it.*

Q6. What is cron and how do you schedule a job?

In simple words: cron is an alarm clock for the server. You tell it "at 2 AM every night, run this script" and it does it without anyone logging in.

Detailed answer

- Edit with `crontab -e` (per user) or put files in `/etc/cron.d/`.
- Format: `minute hour day-of-month month day-of-week command`.

- `*/5 * * * *` -> every 5 minutes; `0 2 * * *` -> 2:00 AM daily; `0 9 * * 1-5` -> 9 AM weekdays.
- cron runs with a **minimal PATH/environment** -> use full paths inside scripts.
- Always redirect output to a log: `>> /var/log/job.log 2>&1`, otherwise failures are silent.
- Check it ran: `grep CRON /var/log/syslog` (Ubuntu) or `journalctl -u cron`.
- Use `flock` to avoid two copies running at the same time.
- Modern alternatives: systemd timers, Kubernetes CronJobs.

Example:

```
crontab -e
# Clean logs older than 7 days every night at 2 AM
0 2 * * * /usr/bin/find /var/log/myapp -name "*.log" -mtime +7 -delete >> /var/log/cleanup.log 2>&1
# Health check every 5 min, never overlapping
*/5 * * * * /usr/bin/flock -n /tmp/hc.lock /opt/scripts/healthcheck.sh >> /var/log/hc.log 2>&1

crontab -l      # list jobs
```

> ***What to say to the interviewer:*** *cron schedules recurring jobs using five time fields, edited via* `crontab -e`*. I always use absolute paths, redirect output to a log file, and use* `flock` *to prevent overlapping runs; in Kubernetes the equivalent is a CronJob.*

Q7. Which commands do you use to check CPU, memory and disk on a Linux server?

> **In simple words:** These are the dashboard gauges of a car: speed (CPU), fuel (memory), and boot space (disk).

Detailed answer

Need	Command	What to look at
CPU/processes	`top`, `htop`	%CPU, load average, `wa` (IO wait), `st` (steal)
Load	`uptime`	load average vs number of cores (`nproc`)
Memory	`free -h`	the **available** column, not "free"
Disk space	`df -h`	Use% per filesystem
Inodes	`df -i`	many tiny files can fill inodes
Folder sizes	`du -sh /var/*` then sort -h	largest directories
Disk IO	`iostat -x 1`, `iotop`	%util, await
Per-process history	`vmstat 1`, `sar`	trends over time

Linux uses spare RAM as file cache, so low "free" memory is normal; watch "available" and swap usage.

Example:

```
nproc; uptime
free -h
df -h
du -xh / --max-depth=1 2>/dev/null | sort -h | tail
ps aux --sort=-%cpu | head -5
ps aux --sort=-%mem | head -5
```

What to say to the interviewer: *I start with* `top`/`htop` *and* `uptime` *for CPU and load,* `free -h` *for memory looking at the available column, and* `df -h` *plus* `du -sh` *for disk. For deeper issues I use* `iostat`, `vmstat` *and* `ps --sort` *to find the heaviest processes.*

Q8. How do you check running processes in Linux?

In simple words: Processes are the workers currently busy in the office. `ps` is a snapshot of the attendance sheet, `top` is a live CCTV view, and `pgrep` is calling out one worker by name.

Detailed answer

- `ps aux` -> snapshot of all processes (user, PID, %CPU, %MEM, command).
- `ps -ef --forest` -> shows parent/child tree (who started whom).
- `top` / `htop` -> live, sortable view (press `P` for CPU, `M` for memory in top).
- `pgrep -a nginx` -> find PIDs by name; `pidof nginx` similar.
- `systemctl status nginx` -> service state plus its main PID and child processes.
- `ps -o pid,ppid,stat,etime,cmd -p <PID>` -> details like uptime (`etime`) and state (`R` running, `S` sleeping, `D` stuck on IO, `Z` zombie).
- `lsof -p <PID>` -> files and sockets the process has open; `ss -tnp` -> its network connections.
- Stop it: `kill <PID>` (graceful SIGTERM), `kill -9 <PID>` only as last resort (SIGKILL, no cleanup).

Example:

```
ps aux --sort=-%cpu | head -5
ps -ef --forest | grep -A3 nginx
pgrep -a java
# 2143 java -Xmx2g -jar /opt/app/app.jar
ps -o pid,stat,etime,%mem,cmd -p 2143
ps aux | awk '$8 ~ /Z/'        # zombie processes
```

What to say to the interviewer: *I use* `ps aux` *or* `ps -ef --forest` *for a snapshot,* `top`/`htop` *for live usage, and* `pgrep` *or* `systemctl status` *to find a specific process. For deeper checks I look at process state and uptime with* `ps -o`*, and open files or sockets with* `lsof` *and* `ss -p`*.*

Q9. What command finds files larger than 100MB?

(also asked as: How do you find large unused files across partitions? / Which directory is too big?)

In simple words: Like walking through a warehouse with a scale and putting a sticker on every box heavier than 100 kg, so you know exactly what is taking up the space.

Detailed answer

- `find / -xdev -type f -size +100M` -> all files over 100MB; `-xdev` stays on one filesystem (skips /proc and mounted volumes).
- Add `-exec ls -lh {} +` or `-printf` to show sizes, and `sort` to rank them.
- `du -ah /var | sort -rh | head -20` -> largest files AND folders under a path.
- `ncdu /` -> interactive, easy browsing of disk usage.
- Combine with age: `-mtime +30` for big files not modified in 30 days (safe cleanup candidates).
- "Unused" = not read recently: `-atime +90` (last access). Many servers mount with `relatime`, so atime is only roughly accurate - treat it as a hint, not proof.
- To scan every partition, run `find` once per mount point from `df -h` output, each with `-xdev`, so network mounts (NFS) are not crawled by accident.
- Redirect errors with `2>/dev/null` to hide "Permission denied" noise.

Example:

```
sudo find / -xdev -type f -size +100M -printf '%s\t%p\n' 2>/dev/null \
  | sort -rn | head -10 | awk '{printf "%.1f MB\t%s\n", $1/1048576, $2}'
# 2310.4 MB  /var/log/myapp/app.log
# 845.2 MB   /opt/exports/dump-2026-08.sql

sudo du -ah /var 2>/dev/null | sort -rh | head -20
find /opt/exports -type f -size +100M -mtime +30 -ls
```

What to say to the interviewer: *I use* `find / -xdev -type f -size +100M`*, usually with* `-printf` *and* `sort` *to rank by size, and* `du -ah | sort -rh | head` *to see the biggest folders too. Adding* `-mtime +30` *gives me old, large files that are the safest candidates for cleanup.*

Q10. Which commands do you use for basic network troubleshooting, and how do you check connectivity between two servers?

(also asked as: How do you check network connectivity between two servers? / A server cannot reach a specific external IP - how do you troubleshoot?)

In simple words: Think of sending a parcel. `dig` checks the address book, `ping` checks the road exists, `traceroute` shows each town on the way, `ss` shows which doors in the building are open, and `curl` actually knocks and waits for a reply.

Detailed answer

- `ss -tulnp` (or older `netstat -tulnp`) -> which ports are listening and which process owns them.
- `dig api.example.com` / `nslookup` -> DNS resolution.
- `ping host` -> basic reachability (ICMP may be blocked in cloud - not proof of failure).
- `traceroute` / `mtr` -> where along the path packets are lost or slow.
- `curl -v https://host/health` -> full HTTP/TLS conversation.
- `nc -zv host 5432` or `telnet host 5432` -> is a TCP port reachable.
- `ip a`, `ip route` -> interfaces and routes.

- `tcpdump -i eth0 port 443` -> capture packets for deep debugging.

Server A -> Server B checklist: resolve the name (`dig`), test the exact port from A (`nc -zv B 5432`), confirm B is listening on the right interface (`ss -tlnp` on B - `127.0.0.1` means local only, `0.0.0.0` means all), then check the firewalls in between: OS firewall (`ufw status`, `iptables -L`), cloud **Security Groups** (inbound on B, outbound on A), **NACLs** (stateless - need return ephemeral ports too), and route tables/peering. Timeout = blocked somewhere; refused = reached B but nothing listening.

Cannot reach one specific external IP (others work): test the IP directly to rule out DNS (`curl -v https://<ip>` vs the name), check the route it takes (`ip route get <ip>` - a VPN or peering route may send it the wrong way), `traceroute`/`mtr` to see where it stops, the local firewall (`iptables -L -n`, `nft list ruleset`), outbound Security Group/NACL rules and the NAT gateway for private subnets, whether the remote side allow-lists your source IP (your NAT/Elastic IP), and **MTU** issues (small pings work but big packets hang: `ping -M do -s 1472 <ip>`).

Example:

```
ss -tulnp | grep 8080
dig +short db.internal.example.com
nc -zv 10.0.2.15 5432
# Connection to 10.0.2.15 5432 port [tcp/postgresql] succeeded!
curl -sv -o /dev/null https://api.example.com/health
mtr -rwc 20 api.example.com
```

> ***What to say to the interviewer:*** *I go layer by layer -* `dig` *for DNS,* `ping`*/*`mtr` *for reachability,* `nc -zv` *to test the TCP port,* `ss -tulnp` *to confirm the service is listening, and* `curl -v` *to see the HTTP and TLS response. If still unclear I capture traffic with* `tcpdump`*.*

Q11. How much Linux do you use in DevOps work, and which distributions have you worked with?

> **In simple words:** Linux is the ground almost every DevOps building stands on - servers, containers and Kubernetes nodes all run on it, so a DevOps engineer lives in the Linux terminal daily.

Detailed answer

Speak from real usage:

- **Distros:** Ubuntu (22.04/24.04 LTS) for servers like monitoring boxes, Amazon Linux 2023 on AWS EC2/EKS nodes, sometimes RHEL/CentOS/Rocky, and Alpine or Debian-slim as container base images.
- **Daily tasks:** SSH/SSM into servers, reading logs (`journalctl`, `tail -f`, `grep`), checking resources (`top`, `df`, `free`), managing services with `systemctl`, package management (`apt`, `dnf/yum`), cron jobs, file permissions, writing Bash automation, networking checks (`curl`, `dig`, `ss`).
- **Examples you can mention:** cleaning a nearly full disk on a monitoring server with `journalctl --vacuum`, tuning a service to fix memory pressure, writing health-check scripts.

Key differences worth knowing: Debian family uses `apt` and `.deb`; RHEL family uses `dnf`/`yum` and `.rpm`.

Example:

```
cat /etc/os-release
# PRETTY_NAME="Ubuntu 22.04.4 LTS"
uname -r          # kernel version
apt update && apt list --upgradable   # Ubuntu
dnf check-update                      # Amazon Linux 2023 / RHEL
```

> ***What to say to the interviewer:*** *I use Linux every day - mainly Ubuntu LTS and Amazon Linux on AWS, plus Alpine and Debian-slim in containers. My daily work includes log analysis, service management with systemd, resource troubleshooting, package management, cron and Bash automation.*

Q12. How does DNS work?

Also asked as: What is DNS caching?

> **In simple words:** DNS is the phone book of the internet. You know the name "google.com", DNS looks up the phone number (IP address) so your computer can call it.

Detailed answer

Flow when you type `api.example.com`:

1. Check the local cache (browser, OS, `/etc/hosts`).
2. Ask the configured **recursive resolver** (from `/etc/resolv.conf`, e.g. VPC resolver, 8.8.8.8).
3. Resolver asks a **root server** -> "ask the .com servers".
4. .com **TLD server** -> "ask example.com's authoritative name servers".
5. **Authoritative server** (e.g. Route 53) returns the record.
6. Answer is cached for the record's **TTL**.

Common record types: `A` (IPv4), `AAAA` (IPv6), `CNAME` (alias to another name), `MX` (mail), `TXT` (verification), `NS` (name servers). Low TTL = faster changes but more queries. Many "network outages" are actually DNS issues - stale cache, wrong record, or resolver overload.

Example:

```
dig api.example.com +short
dig api.example.com +trace          # see root -> TLD -> authoritative
dig @8.8.8.8 api.example.com        # ask a specific resolver
cat /etc/resolv.conf
```

DNS caching: every layer keeps answers for the record's **TTL**, so repeat lookups are fast and don't hit the authoritative servers again.

- **Layers:** the browser or application (the JVM caches DNS, sometimes forever, so set `networkaddress.cache.ttl`), the OS stub resolver (`systemd-resolved`, `nscd`), the recursive resolver (ISP, 8.8.8.8, the AWS VPC resolver), and in Kubernetes CoreDNS's cache plus NodeLocal DNSCache.
- **Negative caching:** a failed lookup (NXDOMAIN) is also cached, so a record you just created may seem "missing" for a while.
- **Why it matters in DevOps:** lower the TTL (for example to 60s) a day **before** a migration or failover so clients pick up the new IP quickly, then raise it again afterwards.

```
resolvectl statistics           # cache hits and misses (systemd-resolved)
sudo resolvectl flush-caches    # flush the local cache
dig +noall +answer example.com  # the TTL is the second column and counts down on repeat queries
```

> ***What to say to the interviewer:*** *DNS translates names into IP addresses: the client asks a recursive resolver, which walks from root to TLD to the authoritative server and caches the answer for the TTL. I debug it with* `dig`*, including* `+trace` *and querying specific resolvers.*

Q13. Explain TCP/IP basics: TCP vs UDP, ports and the three-way handshake.

> **In simple words:** TCP is a registered courier - it confirms delivery and re-sends lost parcels. UDP is dropping flyers in letterboxes - faster, but no guarantee each one arrives.

Detailed answer

- **IP** gives each machine an address and routes packets between networks.
- **Ports** identify the application on that machine (22 SSH, 80 HTTP, 443 HTTPS, 5432 PostgreSQL, 3306 MySQL, 6379 Redis, 53 DNS).
- **TCP**: connection-based, ordered, reliable (retransmits). Used by HTTP, SSH, databases.
- **UDP**: connectionless, no guarantee, low overhead. Used by DNS, video streaming, VoIP, NTP.
- **Three-way handshake**: client sends `SYN` -> server replies `SYN-ACK` -> client sends `ACK` -> connection open. Closing uses `FIN`/`ACK`.
- Every handshake costs one round trip, which is why connection reuse (keep-alive, pooling) reduces latency.
- `Connection refused` = host reachable but nothing listening; `timeout` = packet dropped (firewall/security group/routing).

Example:

```
sudo tcpdump -i eth0 -nn port 443 -c 3
# 10.0.1.5.50412 > 52.1.2.3.443: Flags [S]    <- SYN
# 52.1.2.3.443 > 10.0.1.5.50412: Flags [S.]   <- SYN-ACK
# 10.0.1.5.50412 > 52.1.2.3.443: Flags [.]    <- ACK
```

> ***What to say to the interviewer:*** *TCP is reliable and connection-oriented with a SYN, SYN-ACK, ACK handshake, while UDP is faster but best-effort. A "connection refused" usually means nothing is listening on the port, while a timeout usually points to a firewall, security group or routing problem.*

Q14. How does TLS/HTTPS work?

> **In simple words:** TLS is like sending a letter in a locked box. The website first shows an ID card (certificate) signed by a trusted authority, then you both agree on a secret key so nobody in between can read the letter.

Detailed answer

1. Client connects and says which TLS versions/ciphers it supports and the hostname (SNI).
2. Server sends its **certificate** (public key + domain name, signed by a Certificate Authority).
3. Client verifies: trusted CA chain, not expired, domain matches.

4. Both sides use key exchange (e.g. ECDHE) to create a shared **session key**.
5. All data is encrypted with that fast symmetric key.

DevOps-relevant points:

- Certificates expire -> automate renewal (Let's Encrypt/cert-manager, AWS ACM) and alert before expiry.
- **TLS termination** often happens at the load balancer/Ingress.
- Common errors: expired cert, missing intermediate chain, hostname mismatch, old TLS version.
- **mTLS** = both sides show certificates (used in service meshes).

Example:

```
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null \
  | openssl x509 -noout -subject -issuer -dates
# notAfter=Dec 30 23:59:59 2026 GMT
curl -vI https://api.example.com 2>&1 | grep -E "SSL|expire|subject"
```

> ***What to say to the interviewer:*** *In TLS the server presents a CA-signed certificate, the client validates it, and both negotiate a symmetric session key so traffic is encrypted. Operationally I automate certificate renewal, monitor expiry, and debug with* `openssl s_client` *and* `curl -v`*.*

Q15. What is the difference between a load balancer and a reverse proxy?

(also asked as: How would you load balance 3-4 servers, which type is preferred and why?)

> **In simple words:** A reverse proxy is a receptionist at the front desk. Visitors never walk straight into the offices; the receptionist checks IDs, answers common questions and passes messages on, even when there is only one office. A load balancer is a traffic warden who spreads visitors across several identical counters so no one counter gets overloaded.

Detailed answer

- **Reverse proxy**: sits in front of one or more servers and talks to clients on their behalf. Jobs: **TLS termination**, caching, compression, header rewrites, path-based routing (`/api` to one app, `/` to another), authentication, rate limiting and WAF. It is useful even with a **single** backend.
- **Load balancer**: spreads traffic across **multiple** backends, with **health checks** (unhealthy servers are removed) and algorithms such as round robin, least connections and IP/hash-based stickiness. It works at **L4** (TCP/UDP: sees IPs and ports) or **L7** (HTTP: sees URLs, headers, cookies).
- They overlap a lot: nginx, HAProxy and Envoy do both. An **AWS ALB** is an L7 load balancer that also acts as a reverse proxy (TLS, path/host rules). An **AWS NLB** is an L4 load balancer that forwards connections and is not an HTTP proxy.

Aspect	Reverse proxy	Load balancer
Main goal	shield and enhance backends	distribute load, high availability
Needs many backends?	no, one is fine	yes, that is the point
OSI layer	usually L7 (HTTP)	L4 or L7

Aspect	Reverse proxy	Load balancer
Typical features	TLS, cache, compression, rewrites, auth, WAF	health checks, algorithms, stickiness, failover
Examples	nginx, Apache mod_proxy, Envoy, Traefik	AWS ALB/NLB, HAProxy, nginx upstream, F5

Load balancing 3-4 servers - which type and why?

- Algorithms: **round robin** (default, each server in turn), **weighted** round robin (`weight=3` for a bigger server), **least connections** (`least_conn`, send to the least busy), **ip_hash** / **hash** (same client or key -> same server, for stickiness or cache locality), and **random two choices** (`random two least_conn`: pick 2 at random, use the less loaded one - good with many LB instances).
- **L4 vs L7**: L4 (TCP, e.g. NLB, HAProxy `mode tcp`) is faster and protocol-agnostic - use it for databases, MQTT, gRPC passthrough or huge throughput. L7 (HTTP, e.g. ALB, nginx) can route by path/host, terminate TLS, retry and see HTTP errors.
- **Health checks** are what make it a load balancer and not just a splitter: remove failed servers automatically (passive `max_fails` in open-source nginx, active checks in HAProxy/ALB).
- Preferred for HTTP APIs: **L7 with least connections**, because request durations vary - round robin can pile slow requests onto one server, while least_conn follows the real load, and L7 gives health checks on real HTTP status plus TLS and routing in one place. Keep the app stateless so no stickiness is needed.

Example:

```
upstream app_backend {
    least_conn;                                 # load balancing algorithm
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
}
server {
    listen 443 ssl;                             # reverse proxy: TLS termination
    server_name app.example.com;
    ssl_certificate     /etc/nginx/tls/app.crt;
    ssl_certificate_key /etc/nginx/tls/app.key;
    gzip on;
    location /api/ {
        proxy_pass http://app_backend;          # load balanced across 2 servers
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

What to say to the interviewer: *A reverse proxy sits in front of servers and handles TLS termination, caching, compression, routing and security, even with a single backend. A load balancer spreads traffic across many backends with health checks and algorithms at L4 or L7. In practice nginx, HAProxy and an AWS ALB do both, while an NLB is a pure L4 load balancer.*

Q16. How do you find files modified in the last 10 minutes?

In simple words: Like asking "who touched anything in this room in the last 10 minutes?" - useful right after an incident or a deployment to see exactly what changed.

Detailed answer

- `find /path -type f -mmin -10` -> modified in the last 10 minutes (`-mmin` = minutes, `-mtime` = days).
- `-mmin +10` means MORE than 10 minutes ago; `-10` means LESS than 10 minutes ago.
- `-cmin -10` -> metadata changed (permissions/owner) in the last 10 minutes.
- `-amin -10` -> accessed recently (depends on `relatime` mount option).
- `-newer reference.file` -> changed after another file, handy with a marker made by `touch` before a deploy.
- Add `-xdev` and `2>/dev/null` when scanning `/`, and exclude `/proc` and `/sys`.

Example:

```
sudo find /etc /opt/app -type f -mmin -10 -ls
sudo find / -xdev -type f -mmin -10 2>/dev/null | grep -v -E "^/(proc|sys)"
touch /tmp/before_deploy; ./deploy.sh
find /opt/app -newer /tmp/before_deploy -type f
```

> ***What to say to the interviewer:*** *I use* `find /path -type f -mmin -10`*, where the minus sign means "less than 10 minutes ago", and* `-cmin` *if I also care about permission or ownership changes. For deployments I touch a marker file first and use* `find -newer` *to list exactly what changed.*

Q17. What is Linux patching, why does it matter, and what types of patches are there?

(also asked as: What are the ways to apply patches? / Security patch vs kernel patch?)

> **In simple words:** Patching is like the regular service of a car - fixing known faults (bugs) and fitting new locks (security fixes) before a thief finds the weak one.

Detailed answer

Patching = updating OS packages to newer, fixed versions. It matters because most breaches exploit **known, already-fixed** vulnerabilities (CVEs); it is also needed for compliance (PCI, ISO 27001, SOC 2) and stability.

Type	What it fixes	Reboot?
Security patch	a CVE in a package (openssl, openssh, sudo)	usually no - restart the service
Bug-fix / feature update	normal package updates	usually no
Kernel patch	the Linux kernel itself	**yes** (or live patch)

Ways to apply: package manager by hand (`apt`, `dnf/yum`), unattended/automatic updates (`unattended-upgrades`, `dnf-automatic`) for security-only, config management at scale (Ansible), cloud tools (**AWS SSM Patch Manager**), live patching for kernels, or immutable infra - bake a new patched AMI/image and replace servers.

Example:

```
sudo apt list --upgradable 2>/dev/null | grep -i security    # Ubuntu
sudo dnf updateinfo list security                            # RHEL / Amazon Linux
sudo dnf upgrade --security -y
```

What to say to the interviewer: *Patching means updating packages to close known vulnerabilities and bugs, which matters because most attacks use CVEs that already have fixes. Security and normal package patches usually only need a service restart, while kernel patches need a reboot or live patching, and at scale I apply them with Ansible or SSM Patch Manager, or by rebaking images.*

Q18. How do you check a package version, check available updates, and update all packages?

In simple words: Before servicing the car you read the current part numbers, check which new parts are available, then fit them - and write down the old numbers in case you need to go back.

Detailed answer

Task	Debian/Ubuntu (apt)	RHEL/Amazon Linux (dnf/yum)
Installed version	`dpkg -l openssl` or `apt policy openssl`	`rpm -q openssl` or `dnf info openssl`
Refresh repo metadata	`apt update`	`dnf makecache`
List available updates	`apt list --upgradable`	`dnf check-update` (or `yum check-update`)
Update one package	`apt install --only-upgrade openssl`	`dnf update openssl`
Update everything	`apt upgrade` (or `full-upgrade`)	`dnf upgrade` (or `yum update`)
History	`/var/log/apt/history.log`	`dnf history`

Note: it is `dpkg -l` (lowercase L for "list"), not `dpkg -1`. `apt upgrade` never removes packages; `full-upgrade` may add or remove packages to resolve dependencies. Always note current versions before patching so a rollback is possible.

Example:

```
dpkg -l | grep -E "openssl|openssh-server"
# ii  openssl  3.0.2-0ubuntu1.15  amd64  Secure Sockets Layer toolkit
sudo apt update && apt list --upgradable
sudo apt upgrade -y

rpm -q openssl
sudo dnf check-update
sudo dnf upgrade -y
```

What to say to the interviewer: *I check versions with* `dpkg -l` *or* `apt policy` *on Ubuntu and* `rpm -q` *on RHEL, list pending updates with* `apt list --upgradable` *or* `dnf check-update`*, and apply them with* `apt upgrade` *or* `dnf upgrade`*. I record the before-versions and patch history so I can roll back if needed.*

Q19. How do you find the OS version on Linux?

In simple words: Like checking the model and year written inside a car's door frame before ordering spare parts - you need to know the exact version before installing or patching anything.

Detailed answer

- `cat /etc/os-release` -> works on almost every modern distro (NAME, VERSION_ID, PRETTY_NAME). Best first choice, and easy to use in scripts (`. /etc/os-release; echo $ID $VERSION_ID`).
- `hostnamectl` -> OS, kernel and architecture in one view (systemd systems).
- `lsb_release -a` -> Debian/Ubuntu (may need the `lsb-release` package).
- `cat /etc/redhat-release` or `/etc/system-release` -> RHEL, CentOS, Rocky, Amazon Linux.
- `uname -r` -> **kernel** version only (not the distro); `uname -a` for everything including architecture.

Example:

```
cat /etc/os-release | grep -E "^(NAME|VERSION_ID)="
# NAME="Amazon Linux"
# VERSION_ID="2023"
hostnamectl | grep -E "Operating System|Kernel"
uname -r            # 6.1.100-1.amzn2023.x86_64
. /etc/os-release && [ "$ID" = "ubuntu" ] && sudo apt update || sudo dnf makecache
```

What to say to the interviewer: *I use* `cat /etc/os-release` *because it works on nearly every distro and is easy to parse in scripts, or* `hostnamectl` *for a quick summary.* `uname -r` *gives the kernel version, which is different from the distribution version.*

Q20. How do you search for a specific file inside subfolders?

In simple words: `find` walks through every drawer of the cabinet looking for the file right now; `locate` checks a pre-printed index, which is instant but may be a day old.

Detailed answer

- `find /path -type f -name "app.conf"` -> exact name, recursive through all subfolders.
- `-iname` -> ignore case; wildcards need quotes: `-name "*.log"`.
- `-type d` for directories; `-maxdepth 3` to limit how deep it goes.
- `2>/dev/null` hides "Permission denied" noise; `-xdev` stays on one filesystem.
- `locate app.conf` -> very fast, searches a database updated daily; run `sudo updatedb` to refresh it (package `mlocate`/`plocate`).
- `fd app.conf` -> a modern, faster find alternative, if installed.
- Act on results: `-exec ls -l {} +` or `-delete` (careful).

Example:

```
find /etc -type f -name "nginx.conf" 2>/dev/null
find / -xdev -iname "*docker-compose*.yml" 2>/dev/null
find /opt -maxdepth 3 -type d -name "releases"
sudo updatedb && locate -i jenkins.war
```

> ***What to say to the interviewer:*** *I use* `find /path -type f -name "file"`*, with* `-iname` *for case-insensitive search and* `2>/dev/null` *to hide permission errors. For a quick system-wide lookup I use* `locate`*, remembering its database has to be refreshed with* `updatedb`*.*

Q21. How do you check which ports are open or listening on a Linux server?

> **In simple words:** `ss` tells you which doors in the building are unlocked from the inside. `nmap` from another machine tells you which doors you can actually reach from the street, after the security guards (firewall, Security Group) have had their say.

Detailed answer

- `ss -tulnp` -> TCP/UDP, listening sockets, numeric ports, and the owning process (needs sudo for `-p`). This is the modern replacement for `netstat -tulnp`.
- `sudo lsof -i :8080` -> which process owns a specific port.
- `ss -tn state established '( dport = :5432 )'` -> active connections to a port.
- Bind address matters: `127.0.0.1:8080` is reachable only locally, `0.0.0.0:8080` or `[::]:8080` on all interfaces.
- **Listening is not the same as reachable:** test from outside with `nc -zv host 8080` or `nmap -p 8080 host`, then check firewalld/ufw/iptables and the cloud Security Group/NACL.

Example:

```
sudo ss -tulnp
# Netid State  Local Address:Port  Process
# tcp   LISTEN 0.0.0.0:22          users:(("sshd",pid=812,fd=3))
# tcp   LISTEN 127.0.0.1:5432      users:(("postgres",pid=990,fd=6))   <- local only
# tcp   LISTEN *:80                users:(("nginx",pid=1204,fd=6))

sudo lsof -i :80
nc -zv 10.0.2.15 80            # from another host: is it reachable?
sudo firewall-cmd --list-ports || sudo ufw status
```

> ***What to say to the interviewer:*** *I use* `ss -tulnp` *to see listening ports, the bind address and the owning process, and* `lsof -i :port` *for one specific port. I remember that listening on 127.0.0.1 means local only, and that a listening port may still be blocked, so I test from another host with* `nc -zv` *or nmap and then check the host firewall and the Security Group.*

Q22. What is the difference between curl and wget?

> **In simple words:** curl is a Swiss-army knife for talking to any web API and seeing exactly what comes back. wget is a download manager: point it at a file or a whole site and it fetches everything, even resuming if the line drops.

Detailed answer

Feature	curl	wget
Main purpose	Transfer data, test and call APIs	Download files and mirror sites
Default output	Prints to stdout	Saves to a file
HTTP methods and headers	Any method (`-X`), headers `-H`, data `-d`	Mainly GET/POST
Recursive download	No	Yes (`-r`, `--mirror`)
Resume download	`-C -`	`-c`
Protocols	Many (HTTP, FTP, SFTP, SMTP, LDAP ...)	HTTP, HTTPS, FTP
Library	libcurl, used by many apps	Standalone tool

- Use **curl** for health checks, REST APIs, debugging TLS or headers (`-v`, `-I`) and timing (`-w`).
- Use **wget** to download installers or artifacts, resume big downloads, or run background downloads (`-b`).

Example:

```
curl -sS -o /dev/null -w '%{http_code} %{time_total}s\n' https://api.example.com/health
curl -X POST -H 'Content-Type: application/json' -d '{"name":"test"}' https://api.example.com/items
curl -vI https://example.com            # headers + TLS handshake details

wget -c https://releases.example.com/app-1.4.2.tar.gz     # resume if interrupted
wget -q -O - https://example.com/install.sh | head        # print to stdout like curl
```

> ***What to say to the interviewer:*** *curl is my tool for APIs and troubleshooting: any HTTP method, custom headers, verbose TLS output and timing, with output to stdout. wget is better for downloading files, resuming large downloads and mirroring sites recursively. In scripts I use curl with* `-fsS` *and a timeout for health checks.*

Advanced

Q23. Write a shell script to verify whether SSH connectivity to a server (or list of servers) is working.

> **In simple words:** Like a security guard walking down a corridor and trying each door with the key - noting which open, which are locked, and which are not answering at all.

Detailed answer

Good script qualities: non-interactive (`BatchMode=yes` so it never waits for a password), a short `ConnectTimeout`, a separate TCP port check to tell "network down" from "auth failed", clear exit codes, and a summary for cron/CI.

Example:

```
#!/usr/bin/env bash
# Usage: ./check_ssh.sh servers.txt [user]
set -uo pipefail
FILE="${1:?Usage: $0 <servers_file> [user]}"
USER_NAME="${2:-ubuntu}"
FAILED=0

while read -r HOST; do
  [ -z "$HOST" ] && continue
  if ! nc -z -w 3 "$HOST" 22 2>/dev/null; then
    echo "[DOWN]  $HOST - port 22 not reachable (network/SG/firewall)"
    FAILED=$((FAILED+1)); continue
  fi
  if ssh -n -o BatchMode=yes -o ConnectTimeout=5 \
         -o StrictHostKeyChecking=accept-new \
         "$USER_NAME@$HOST" 'hostname' >/dev/null 2>&1; then
    echo "[OK]    $HOST"
  else
    echo "[AUTH]  $HOST - port open but SSH login failed (key/user/permissions)"
    FAILED=$((FAILED+1))
  fi
done < "$FILE"

echo "Failed: $FAILED"
exit $(( FAILED > 0 ? 1 : 0 ))
```

Note `ssh -n` stops ssh from eating the rest of the input file inside the while loop - a classic bug.

> ***What to say to the interviewer:*** *My script loops over hosts, first checks port 22 with* `nc -z` *to separate network issues from authentication issues, then runs* `ssh -o BatchMode=yes -o ConnectTimeout=5 user@host hostname`*. It reports OK/DOWN/AUTH per host and exits non-zero if any fail, so it works in cron or a pipeline.*

Q24. How can you access a Windows C: drive from an Ubuntu system (dual boot or mounted disk)?

> **In simple words:** The Windows disk is like a filing cabinet in a different language. Ubuntu needs the translator (NTFS driver) and then you just open the drawer at a folder you choose.

Detailed answer

1. Find the partition: `lsblk -f` or `sudo fdisk -l` -> look for the `ntfs` partition (e.g. `/dev/sda3` or `/dev/nvme0n1p3`).
2. Create a mount point: `sudo mkdir -p /mnt/windows`.
3. Mount it: `sudo mount -t ntfs3 /dev/sda3 /mnt/windows` (kernel 5.15+ `ntfs3` driver) or `-t ntfs-3g` (install `ntfs-3g`). Use `-o ro` for read-only to be safe.
4. Make it permanent via `/etc/fstab` using the UUID from `blkid`.

Common gotcha: if Windows was shut down with **Fast Startup / hibernation**, the disk is marked "in use" and mounts read-only or fails. Fix by disabling Fast Startup in Windows or doing a full shutdown; `ntfsfix` can clear the dirty flag only as a last resort. BitLocker-encrypted drives need `dislocker` and the recovery key.

Example:

```
lsblk -f
# nvme0n1p3 ntfs  Windows  1A2B3C4D5E6F7788
sudo mkdir -p /mnt/windows
sudo mount -t ntfs3 -o ro /dev/nvme0n1p3 /mnt/windows
ls /mnt/windows/Users

# /etc/fstab (permanent)
UUID=1A2B3C4D5E6F7788  /mnt/windows  ntfs3  defaults,uid=1000,gid=1000,nofail  0  0
```

What to say to the interviewer: *I identify the NTFS partition with* `lsblk -f`*, create a mount point and mount it with the* `ntfs3` *or* `ntfs-3g` *driver, and add it to /etc/fstab by UUID for persistence. If it mounts read-only, it is usually Windows Fast Startup or hibernation, which must be disabled first.*

Q25. How do you troubleshoot network latency step by step?

In simple words: A slow pizza delivery could be slow ordering (DNS), traffic on the road (network), a slow door check (TLS), or a slow kitchen (application). You time each stage to find the real culprit.

Detailed answer

1. **Measure precisely where time goes** with `curl -w`: DNS lookup, TCP connect, TLS handshake, time to first byte (TTFB), total.
2. **DNS slow** -> resolver issue, low TTL, too many search domains (`ndots` in Kubernetes).
3. **Connect slow** -> network distance, packet loss, congested NAT/firewall; check with `mtr` for loss per hop.
4. **TLS slow** -> no session reuse, huge certificate chains, CPU-starved terminator.
5. **TTFB slow but connect fast** -> the server/app/database is slow, not the network.
6. Check the host: `ss -s` (connection counts, TIME_WAIT), retransmits via `netstat -s | grep -i retrans`, NIC errors with `ip -s link`.
7. Compare from different locations (same subnet vs internet) to isolate the segment.
8. Look at p95/p99 not just averages - tail latency hides behind a good average.

Example:

```
curl -o /dev/null -s -w \
 "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} \
ttfb:%{time_starttransfer} total:%{time_total}\n" https://api.example.com/health
# dns:0.004 connect:0.021 tls:0.058 ttfb:1.842 total:1.845   <- app is slow, not network

mtr -rwc 50 api.example.com
netstat -s | grep -i retrans
```

What to say to the interviewer: *I break the request into DNS, TCP connect, TLS and time-to-first-byte using* `curl -w`*, which immediately tells me whether it is a network or an application problem. Then I use* `mtr` *for packet loss, retransmit counters and connection stats on the host, and always look at p95/p99 rather than averages.*

Q26. How do you approach troubleshooting and debugging automation (shell) scripts?

In simple words: Debugging a script is like replaying a recipe step by step with a narrator saying out loud what is being done, so you catch the exact step where it went wrong.

Detailed answer

- Run with trace: `bash -x script.sh` or put `set -x` around a suspect block - prints every command with variables expanded.
- Use strict mode `set -euo pipefail` so errors are not silently ignored.
- Add a `trap` on ERR to print the failing line number.
- Lint with **shellcheck** - catches unquoted variables, wrong tests, etc. (add it to CI).
- Log with timestamps to a file; separate stdout and stderr.
- Reproduce in the same environment: cron and CI have a different `PATH`, user and working directory than your shell.
- Check exit codes (`echo $?`) of each external command.
- Make scripts idempotent (safe to re-run) and test with a `--dry-run` flag first.

Example:

```
#!/usr/bin/env bash
set -euo pipefail
trap 'echo "ERROR at line $LINENO: $BASH_COMMAND (exit $?)" >&2' ERR
log() { echo "$(date '+%F %T') $*" | tee -a /var/log/deploy.log; }

log "Starting backup"
set -x
tar -czf "/backup/app-$(date +%F).tar.gz" /opt/app
set +x
log "Done"
```

```
shellcheck deploy.sh
# SC2086: Double quote to prevent globbing and word splitting.
```

What to say to the interviewer: *I run the script with* `bash -x`*, use* `set -euo pipefail` *and an ERR trap to print the failing line, and lint it with shellcheck in CI. I also reproduce in the same context it runs in, like cron or the CI agent, because PATH and user differences cause most "works on my machine" script failures.*

Q27. Can you share an example of a complex automation script you have written?

In simple words: Interviewers want a short story: what boring or risky manual task existed, what your script did automatically, and what it saved.

Detailed answer

Use the structure **Problem -> What the script does -> Safety features -> Result**. A good Bash example: an automated disk cleanup and alert script for monitoring servers.

- Problem: small monitoring servers repeatedly hit 90%+ disk from journal logs and old exports.

- Script: checks usage with `df`, if above threshold vacuums journald, deletes rotated logs older than N days, removes old temp exports, re-checks usage and posts a message to a Teams/Slack webhook with before/after numbers.
- Safety: dry-run mode, only touches whitelisted paths, `flock` to prevent overlaps, logs every action.
- Result: no more manual 2 AM cleanups; alert only when cleanup is not enough.

Example:

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD=85; WEBHOOK="${WEBHOOK_URL:?}"; DRY="${DRY_RUN:-false}"
usage() { df --output=pcent / | tail -1 | tr -dc '0-9'; }
run() { [ "$DRY" = true ] && echo "DRY: $*" || eval "$@"; }

BEFORE=$(usage)
[ "$BEFORE" -lt "$THRESHOLD" ] && exit 0

run "journalctl --vacuum-size=300M"
run "find /var/log -name '*.gz' -mtime +7 -delete"
run "find /opt/exports -type f -mtime +14 -delete"
AFTER=$(usage)

MSG="$(hostname): disk ${BEFORE}% -> ${AFTER}%"
[ "$AFTER" -ge "$THRESHOLD" ] && MSG="$MSG - STILL HIGH, manual action needed"
curl -s -H 'Content-Type: application/json' -d "{\"text\":\"$MSG\"}" "$WEBHOOK"
```

> ***What to say to the interviewer:*** *One example is a disk-cleanup automation for our monitoring servers: it checks disk usage, vacuums journald and deletes old rotated logs and exports from whitelisted paths, re-measures, and posts before/after numbers to a Teams webhook. It has a dry-run mode and flock protection, and it removed a recurring manual task while only alerting us when cleanup was not enough.*

Q28. Write a shell script to find prime numbers (e.g. all primes up to N).

> **In simple words:** A prime number can only be divided evenly by 1 and itself. The script is like testing each number with a set of keys (divisors) - if any key fits, it is not prime. You only need to try keys up to the square root, because any bigger key would have a smaller partner key you already tried.

Detailed answer

Why interviewers ask it: it is not really about maths. It checks whether you can write working Bash with **argument validation**, **loops** (`for`/`while`), **conditionals** (`if`, `[[ ]]`), **arithmetic** (`$(( ))`, `(( ))`, modulo `%`), functions and return codes - the same building blocks used in real automation scripts.

Key points to say while writing it:

- Validate input: exactly one argument, and it must be a positive integer (regex `^[0-9]+$`).
- Numbers below 2 are not prime; 2 is the only even prime.
- Only test odd divisors from 3 up to sqrt(n) -> check `i * i <= n` instead of calling a sqrt tool (Bash has integer maths only).
- Use a function that returns 0 (prime) or 1 (not prime) so it reads cleanly in an `if`.

- Bash is fine for small N; for very large ranges a Sieve of Eratosthenes or another language is faster - mentioning this shows judgement.

Example:

```
#!/usr/bin/env bash
# Usage: ./primes.sh <N>   -> prints all primes from 2 to N
set -euo pipefail

usage() { echo "Usage: $0 <positive integer N>" >&2; exit 1; }

[ $# -eq 1 ] || usage
N="$1"
[[ "$N" =~ ^[0-9]+$ ]] || { echo "Error: '$N' is not a positive integer" >&2; exit 2; }

is_prime() {
  local n=$1 i
  (( n < 2 )) && return 1
  (( n == 2 )) && return 0
  (( n % 2 == 0 )) && return 1
  for (( i = 3; i * i <= n; i += 2 )); do
    (( n % i == 0 )) && return 1
  done
  return 0
}

primes=()
for (( num = 2; num <= N; num++ )); do
  if is_prime "$num"; then primes+=("$num"); fi
done

echo "Primes up to $N: ${primes[*]:-none}"
echo "Count: ${#primes[@]}"
```

```
$ ./primes.sh 30
Primes up to 30: 2 3 5 7 11 13 17 19 23 29
Count: 10

$ ./primes.sh abc
Error: 'abc' is not a positive integer

$ ./primes.sh
Usage: ./primes.sh <positive integer N>
```

What to say to the interviewer: *I validate that exactly one positive integer is passed, then loop from 2 to N and test each number with an is_prime function that rejects numbers below 2 and even numbers, and checks odd divisors only while i times i is less than or equal to n. It demonstrates Bash loops, conditionals, arithmetic with double parentheses and return codes, and for very large N I would mention a sieve or a faster language.*

Q29. How do you extend a partition or filesystem without unmounting it?

In simple words: Like adding extra rooms to a house while people are still living in it - first make the plot bigger (disk/volume), then move the wall (partition), then tell the house it now has more floor space (filesystem).

Detailed answer

Both ext4 and XFS can be **grown online** (while mounted). Shrinking XFS is impossible, and shrinking ext4 needs an unmount.

Plain partition (common on EC2):

1. Grow the disk: modify the EBS volume size in AWS (no downtime).
2. `lsblk` -> confirm the disk is bigger than the partition.
3. `sudo growpart /dev/nvme0n1 1` -> grow partition 1.
4. Grow the filesystem: `resize2fs` for ext4, `xfs_growfs /mountpoint` for XFS (check with `df -Th`).

LVM:

1. Add space: grow the disk + `pvresize`, or add a new disk with `pvcreate` + `vgextend`.
2. `lvextend -r -L +20G /dev/vg0/data` -> `-r` also resizes the filesystem in the same step.

Take a snapshot first - a resize is safe but not undoable.

Example:

```
lsblk; df -Th /
sudo growpart /dev/nvme0n1 1
sudo resize2fs /dev/nvme0n1p1        # ext4
sudo xfs_growfs /                    # XFS (Amazon Linux default)

# LVM
sudo pvresize /dev/nvme1n1
sudo lvextend -r -L +20G /dev/vg_data/lv_data
```

Also asked as: Add 50GB to /opt using LVM without any downtime.

Full chain for "add 50GB to /opt with LVM, no downtime":

1. `df -hT /opt` -> find the LV (e.g. `/dev/mapper/vg_app-opt`) and the filesystem type.
2. `vgs` -> is there at least 50G in the `VFree` column of the volume group?
3. If not: attach a new disk, then `pvcreate /dev/nvme2n1` and `vgextend vg_app /dev/nvme2n1` (or grow the existing disk and run `pvresize`).
4. `lvextend -r -L +50G /dev/vg_app/opt` -> `-r` grows the filesystem too. Or do it in two steps: `lvextend -L +50G ...`, then `xfs_growfs /opt` (XFS, uses the mount point) or `resize2fs /dev/vg_app/opt` (ext4).
5. `df -h /opt` to verify. Everything stays mounted and no application restart is needed.

> ***What to say to the interviewer:*** *For a normal partition I grow the EBS volume, run* `growpart`*, then* `resize2fs` *for ext4 or* `xfs_growfs` *for XFS, all while mounted. With LVM I extend the physical volume or volume group and run* `lvextend -r`*, which grows the logical volume and filesystem together; I always snapshot first.*

Q30. What are the steps to add a new disk to a Linux server?

> **In simple words:** A new hard disk is like an empty new cupboard. You divide it into shelves (partition), fit the shelving system (filesystem), place it in a room people can reach (mount point), and write it into the house plan so it is still there after a restart (fstab).

Detailed answer

1. Attach the disk (EBS attach, or physically) and find it: `lsblk` (new device e.g. `/dev/nvme1n1` with no mount point).
2. Optional partition: `parted /dev/nvme1n1 mklabel gpt mkpart primary 0% 100%` (or use the whole disk / LVM).
3. Create a filesystem: `mkfs.xfs` or `mkfs.ext4`.
4. Create a mount point and mount: `mkdir /data && mount /dev/nvme1n1p1 /data`.
5. Make it permanent in `/etc/fstab` using the **UUID** (device names like nvme1n1 can change after reboot), with `nofail` so a missing disk does not stop boot.
6. Test the fstab entry with `mount -a` BEFORE rebooting - a bad fstab line can leave the server unbootable.
7. Set ownership/permissions for the app user.

Example:

```
lsblk
sudo parted -s /dev/nvme1n1 mklabel gpt mkpart primary xfs 0% 100%
sudo mkfs.xfs /dev/nvme1n1p1
sudo mkdir -p /data
sudo blkid /dev/nvme1n1p1        # UUID="3f2c..."
echo 'UUID=3f2c-...  /data  xfs  defaults,nofail  0 2' | sudo tee -a /etc/fstab
sudo mount -a && df -h /data
```

> ***What to say to the interviewer:*** *I identify the new disk with* `lsblk`*, partition it with parted, create an XFS or ext4 filesystem, mount it on a directory, and add it to /etc/fstab by UUID with* `nofail`*. I always validate with* `mount -a` *before any reboot, because a broken fstab entry can stop the server booting.*

Q31. How do you create and mount a swap file?

> **In simple words:** Swap is an overflow parking lot on disk. When RAM (the main car park) is full, less-used data is parked there - slower to fetch, but better than the OOM killer throwing things out.

Detailed answer

1. Create the file: `fallocate -l 2G /swapfile` (or `dd` if fallocate is not supported, e.g. on some XFS setups).
2. Lock it down: `chmod 600 /swapfile` (swap can contain secrets from memory).
3. Format as swap: `mkswap /swapfile`.
4. Enable: `swapon /swapfile`; verify with `swapon --show` / `free -h`.
5. Persist: add `/swapfile none swap sw 0 0` to `/etc/fstab`.
6. Tune `vm.swappiness` (e.g. 10) so the kernel prefers RAM and only swaps under pressure.

Swap is a safety buffer, not a fix for too little RAM - heavy swapping makes everything slow. Kubernetes nodes traditionally run with swap off (kubelet requires it unless swap support is explicitly configured).

Example:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swap.conf && sudo sysctl --system
free -h
```

What to say to the interviewer: *I create a file with fallocate, set it to 600, run mkswap and swapon, add it to /etc/fstab, and lower swappiness so it is only used under pressure. It is a buffer against OOM kills on small servers, not a replacement for right-sizing, and I keep swap off on Kubernetes nodes.*

Q32. How do you find which processes are writing to a file in real time?

In simple words: A shared notebook keeps filling up and you want to know who is writing in it. You can look at who is holding it right now (lsof/fuser), watch it live (inotifywait), or install a CCTV camera that records every writer (auditd).

Detailed answer

- `lsof /var/log/app.log` -> processes that currently have the file open (the `FD` column ends in `w` or `u` for write).
- `fuser -v /var/log/app.log` -> PIDs using the file (also works on mount points: `fuser -vm /data`).
- `inotifywait -m /var/log/app.log` (package `inotify-tools`) -> live stream of modify/open/close events. It shows WHAT happens, not WHO.
- `auditctl -w /etc/nginx/nginx.conf -p wa -k nginx_conf` -> audit rule recording writes and attribute changes WITH the PID, user and command; read with `ausearch -k nginx_conf`. Best for "who keeps changing this config?"
- For a known process, `strace -f -e trace=write -p <PID>` shows its writes live.

Example:

```
sudo lsof /var/log/app.log
# COMMAND  PID  USER  FD  TYPE ... NAME
# java    2143  app   12w  REG ... /var/log/app.log
sudo fuser -v /var/log/app.log
inotifywait -m -e modify,close_write /etc/nginx/nginx.conf
sudo auditctl -w /etc/nginx/nginx.conf -p wa -k nginx_conf
sudo ausearch -k nginx_conf -i | tail
```

What to say to the interviewer: *For who has it open right now I use* `lsof` *or* `fuser`*; to watch changes live I use* `inotifywait`*; and to record exactly which user and process modify it over time I add an auditd watch rule and query it with* `ausearch`*. For one known process,* `strace -e trace=write` *shows its writes in real time.*

Q33. What is the fastest way to copy a huge file (or many files) across servers?

(also asked as: How do you transfer a large file securely between servers?)

In simple words: Moving house - you can carry boxes one by one (scp), use a van that only moves boxes that changed and resumes if it stops (rsync), pack everything tightly first (compression), or hire several vans at once (parallel).

Detailed answer

Method	When to use
`rsync -avP`	default choice: resumable (`--partial`), progress, only sends changes
`scp` / `sftp`	simple one-off copy, not resumable
tar piped over ssh	millions of small files - one stream instead of per-file overhead
compression (`rsync -z`, `zstd`)	slow network + compressible data; skip it for already-compressed files or fast LANs (CPU becomes the bottleneck)
parallel streams (`parallel` + rsync, `bbcp`)	high-latency, high-bandwidth links where one TCP stream cannot fill the pipe
On AWS: via **S3**	`aws s3 cp` uses multipart parallel uploads, then download on the other side; also works across regions/accounts

Tips: use a light SSH cipher only if allowed, run inside `tmux`, verify with `sha256sum` on both sides.

Making it secure: always go over SSH (`scp`, `rsync -e ssh`, `sftp`) with key-based auth - never plain FTP/rsync daemon on untrusted networks. Use a dedicated low-privilege transfer user, and a jump host (`-J bastion`) instead of opening port 22 widely. On S3, use a private bucket with SSE-KMS encryption, a VPC endpoint and short-lived pre-signed URLs or IAM roles. Sensitive files can be encrypted before sending (`gpg -c` or `age`). Finally, prove integrity: compare `sha256sum` on both ends.

Example:

```
rsync -avP --partial /data/dump.sql.gz user@10.0.2.20:/backup/
tar -cf - /data/images | zstd -T0 | ssh user@10.0.2.20 "zstd -d | tar -xf - -C /restore"
aws s3 cp /data/dump.sql.gz s3://my-transfer-bucket/     # multipart, parallel
sha256sum /data/dump.sql.gz      # compare with the destination
```

What to say to the interviewer: *My default is* `rsync -avP` *because it is resumable and only sends differences; for millions of small files I stream* `tar` *over ssh, and I only compress when the network is slower than the CPU. On AWS I often copy through S3 because multipart uploads are parallel, and I always verify with checksums.*

Q34. How do you analyse disk I/O performance on Linux?

In simple words: The disk is a single checkout counter. You check how busy the cashier is (%util), how long each customer waits (await), how long the queue is, and who is buying the most (iotop).

Detailed answer

- `iostat -xz 1` -> per device: `r/s`, `w/s`, `rkB/s`, `wkB/s`, `await` (ms per request, including queue), `aqu-sz` (queue length), `%util`. High await + high queue = the disk is the bottleneck. (%util near 100% is less meaningful on SSD/NVMe, which serve requests in parallel.)

- `vmstat 1` -> wa column (CPU waiting on IO) and b (processes blocked on IO).
- `iotop -o` / `pidstat -d 1` -> which process is doing the IO.
- `top` -> processes in `D` state are stuck waiting on IO.
- Benchmark capability with `fio` (on a test file, never over live data).
- On AWS also check EBS limits: volume IOPS/throughput (gp3 baseline 3000 IOPS / 125 MB/s), burst balance on gp2, and the instance's EBS bandwidth limit in CloudWatch.

Example:

```
iostat -xz 1 3
# Device  r/s   w/s   wkB/s   await  aqu-sz  %util
# nvme1n1 12.0  950.0 60800.0  38.2   36.1   99.8   <- saturated
sudo iotop -o
pidstat -d 1 5
fio --name=randwrite --filename=/data/fio.test --rw=randwrite --bs=4k --size=1G \
    --iodepth=32 --ioengine=libaio --direct=1 --runtime=30 --time_based
```

> ***What to say to the interviewer:*** *I use* `iostat -x` *to read await, queue size and utilisation per device,* `vmstat` *for IO wait and blocked processes, and* `iotop` *or* `pidstat -d` *to find the process causing it. On AWS I compare against the EBS volume's IOPS and throughput limits in CloudWatch and use* `fio` *on a test file to benchmark.*

Q35. How do you roll back a bad patch or package update?

> **In simple words:** Before repainting a wall you take a photo and keep the old paint tin. If the new colour is wrong you repaint with the old tin (downgrade) or, if it is a disaster, restore the whole room from the photo (snapshot).

Detailed answer

Options, from lightest to strongest:

1. **RHEL/Amazon Linux:** `dnf history` (or `yum history`) lists every transaction -> `dnf history undo <ID>` reverts that transaction, or `dnf downgrade pkg`.
2. **Ubuntu/Debian:** no built-in undo. Find the old version in `/var/log/apt/history.log`, then `apt install pkg=<old-version>` (if still in the repo or apt cache), and `apt-mark hold pkg` so it is not re-upgraded.
3. **Kernel:** boot the previous kernel from GRUB (kept installed by default) and set it as default.
4. **Whole server:** restore the **EBS snapshot / AMI** taken before patching - the most reliable rollback.
5. **Immutable infra:** redeploy the previous AMI/image version.

That is why the rule is: snapshot or AMI before patching production, and patch staging first.

Example:

```
sudo dnf history
# ID | Command line  | Date and time    | Action(s) | Altered
#  42 | upgrade -y    | 2026-09-20 02:00 | Upgrade   |   37
sudo dnf history undo 42

grep -A3 "Upgrade:" /var/log/apt/history.log | grep nginx
sudo apt install nginx=1.18.0-6ubuntu14.4
sudo apt-mark hold nginx
```

What to say to the interviewer: *On RHEL-based systems I use* `dnf history undo` *for the transaction, on Ubuntu I reinstall the exact previous version with* `apt install pkg=version` *and hold it, and for kernels I boot the older kernel from GRUB. The safest rollback is the EBS snapshot or AMI I take before every production patch.*

Q36. How do you apply a kernel patch, reboot safely, and keep the old kernel available?

In simple words: Changing the engine of a car. You keep the old engine in the garage, so if the new one will not start, you drop the old one back in and drive away.

Detailed answer

1. Snapshot/AMI first; drain the node or take it out of the load balancer.
2. Install the new kernel: `apt install linux-image-<ver>` / `apt full-upgrade`, or `dnf update kernel`. Package managers install kernels **side by side** - the old one stays.
3. Check the new kernel is registered in GRUB: `grubby --default-kernel` (RHEL) or `/boot/grub/grub.cfg` (Ubuntu); keep at least 2 kernels (`installonly_limit=3` in dnf.conf; do not `autoremove` the previous one right away).
4. Reboot in the maintenance window; verify with `uname -r` and service health checks.
5. If it fails: pick the old kernel from the GRUB menu (EC2 serial console helps here), then set it as the default (`grubby --set-default` or `GRUB_DEFAULT` + `update-grub`).
6. Check whether a reboot is needed at all: `/var/run/reboot-required` (Ubuntu), `needs-restarting -r` (RHEL).

Example:

```
uname -r
sudo dnf update kernel -y
sudo grubby --info=ALL | grep -E "^kernel"
sudo needs-restarting -r
sudo reboot
uname -r                                        # verify new kernel
sudo grubby --set-default /boot/vmlinuz-6.1.100-1.amzn2023.x86_64   # roll back if needed
```

What to say to the interviewer: *I snapshot and drain the node, install the new kernel, which sits alongside the old one, confirm GRUB has both, reboot in a maintenance window and verify with* `uname -r` *and health checks. If it misbehaves I boot the previous kernel from GRUB and set it as default, and I keep at least two kernels installed.*

Q37. What is live kernel patching?

> **In simple words:** Fixing a tyre while the car is still driving. You fix the dangerous hole now without stopping, and change the whole tyre at the next pit stop.

Detailed answer

Live patching applies **security fixes to the running kernel in memory** without a reboot, by redirecting vulnerable functions to patched versions.

- Tools: **kpatch** (Red Hat), **kGraft** (SUSE, merged into upstream livepatch), **Canonical Livepatch** (Ubuntu Pro), **Ksplice** (Oracle).
- Good for: critical CVEs on servers where reboots are expensive (databases, stateful nodes), buying time until the next maintenance window.
- Limits: only for selected security fixes, not full kernel upgrades; the fix is lost on reboot unless the new kernel package is also installed; you still need a real reboot eventually.
- In cloud/Kubernetes environments, rolling node replacement (new AMI) often makes live patching unnecessary.

Example:

```
# Ubuntu Pro
sudo pro enable livepatch
canonical-livepatch status --verbose
# RHEL
sudo dnf install kpatch-patch-$(uname -r | sed 's/\.x86_64//;s/\./_/g')
kpatch list
```

> ***What to say to the interviewer:*** *Live patching - with kpatch, Canonical Livepatch, kGraft or Ksplice - patches critical kernel vulnerabilities in memory without a reboot. I use it to protect hard-to-reboot servers until a planned maintenance window, but it does not replace installing the new kernel and rebooting eventually.*

Q38. How do you patch a fleet of servers in production?

> **In simple words:** Vaccinating a whole city: you test the vaccine on a small group first, do it neighbourhood by neighbourhood at quiet times, keep a record of who got it, and stop immediately if the first group has problems.

Detailed answer

1. **Inventory and compliance**: know which servers need which patches (SSM Patch Manager compliance, Inspector findings, CVE feeds).
2. **Tooling**: **AWS SSM Patch Manager** (patch baselines, patch groups by tag, maintenance windows, compliance reports) or **Ansible** playbooks (`serial:` batches) for non-AWS/hybrid.
3. **Order**: dev -> staging -> a **canary batch** in prod (e.g. 10%) -> the rest in batches, never all at once or both nodes of an HA pair together.
4. **Safety**: pre-patch snapshots/AMIs, drain from load balancer first, health checks after each batch, automatic stop if failures exceed a threshold (`max_fail_percentage` in Ansible, error threshold in SSM).
5. **Maintenance windows** and change approval; communicate with teams.

6. **Immutable alternative**: rebake a patched AMI (Packer / EC2 Image Builder), roll it out through the Auto Scaling Group instance refresh, or replace EKS node groups - no in-place patching at all.
7. Report compliance afterwards.

Example:

```
# Ansible: rolling patch, 20% at a time, stop if >10% fail
- hosts: web
  serial: "20%"
  max_fail_percentage: 10
  become: true
  tasks:
    - name: Upgrade all packages
      ansible.builtin.dnf: { name: "*", state: latest }
    - name: Reboot if needed
      ansible.builtin.reboot: { reboot_timeout: 600 }
    - name: Health check
      ansible.builtin.uri: { url: "http://localhost/health", status_code: 200 }
```

> ***What to say to the interviewer:*** *I patch in stages - dev, staging, then a prod canary batch and the rest in rolling batches during maintenance windows - using SSM Patch Manager or Ansible with serial batches, pre-patch snapshots, load balancer draining and a failure threshold that stops the rollout. For immutable workloads I instead rebake a patched AMI and roll it out with an ASG instance refresh or node group replacement.*

Q39. In Nginx, how do you make a client stick to a specific server (sticky sessions)?

> **In simple words:** A regular customer always goes to the same bank teller because that teller has their file on the desk. Stickiness keeps sending them to that teller - but it is better to keep the file in a shared cabinet (Redis) so any teller can help.

Detailed answer

Options in open-source nginx:

- `ip_hash`: hashes the client IP (first 3 octets for IPv4) -> same server. Simple, but many users behind one corporate NAT or mobile carrier all land on one server, and it breaks when client IPs change.
- `hash $cookie_sessionid consistent;` (or any key like `$request_uri`): hash a session cookie or header. `consistent` uses ketama consistent hashing, so adding or removing a server only moves about 1/N of the clients.
- `sticky cookie` directive: nginx itself inserts a cookie naming the server - available only in **NGINX Plus** (commercial). HAProxy (`cookie SERVERID insert`) and AWS ALB (target group stickiness) provide it for free.

Why stickiness is a last resort: load becomes uneven, scaling in/out and deployments break sessions, and a failed server logs its users out. The better design is **stateless app servers with the session externalised** in Redis/Memcached/a DB or a signed token (JWT), so any server can serve any request. Use stickiness only for legacy apps or cache locality.

Example:

```
upstream app_sticky {
    hash $cookie_JSESSIONID consistent;   # or: ip_hash;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.13:8080 max_fails=3 fail_timeout=30s;
}
# NGINX Plus only:
# upstream app { server 10.0.1.11:8080; server 10.0.1.12:8080;
#                sticky cookie srv_id expires=1h path=/; }
```

What to say to the interviewer: *In open-source nginx I use* `ip_hash` *or, better,* `hash $cookie_<session> consistent` *so the same session always maps to the same server; the* `sticky cookie` *directive exists only in NGINX Plus. I prefer to avoid stickiness altogether by storing sessions in Redis or using tokens, because sticky sessions cause uneven load and lost sessions when a server fails.*

Q40. In Nginx, with sticky sessions or ip_hash, what happens if a server goes down?

In simple words: If your usual bank teller goes home sick, the manager sends you to another teller - but that teller does not have your file, so you must start again, unless the files are kept in a shared cabinet.

Detailed answer

- Open-source nginx uses **passive health checks**: when requests to a server fail (connection errors, timeouts, and optionally 5xx via `proxy_next_upstream`) `max_fails` times within `fail_timeout`, nginx marks it **unavailable for** `fail_timeout` seconds, then tries it again.
- With `ip_hash`/`hash`, clients mapped to the dead server are **remapped to other servers**. Their in-memory session is on the dead server, so they **lose their session** (logged out, cart empty) - unless sessions are shared (Redis) or replicated.
- The first request that hits the failure can still error or be slow (it has to time out), then `proxy_next_upstream` retries on another server for idempotent requests.
- `consistent` hashing limits the damage: only the failed server's clients move; with plain modulo hashing, removing a server reshuffles almost everyone.
- `down` is a manual flag for planned maintenance. With `ip_hash` it keeps the hash mapping for the other servers (clients of the down server are spread elsewhere), so use `down` instead of deleting the line.
- **Active health checks** (`health_check` in NGINX Plus, HAProxy, ALB) probe a URL in the background and remove a server BEFORE user requests fail, and put it back when it recovers.

Example:

```
upstream app {
    ip_hash;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
    server 10.0.1.13:8080 down;          # planned maintenance
}
location / {
    proxy_pass http://app;
    proxy_next_upstream error timeout http_502 http_503;
    proxy_next_upstream_tries 2;
    proxy_connect_timeout 2s;
}
```

What to say to the interviewer: *After* `max_fails` *failures within* `fail_timeout`*, nginx marks the server unavailable and remaps its clients to the remaining servers, so those users lose their session unless it is stored centrally in something like Redis. Consistent hashing means only that server's clients move,* down *is for planned maintenance, and active health checks in NGINX Plus, HAProxy or an ALB remove the server before users see errors.*

Q41. How does header management work in Nginx?

In simple words: Headers are the notes written on the outside of an envelope. As a reverse proxy, nginx rewrites the notes going to the backend (who really sent it, which address it was for) and adds security stickers on the replies going back to the browser.

Detailed answer

Request headers to the backend (`proxy_set_header`):

- `Host $host` -> otherwise nginx sends the upstream name, which breaks virtual hosts.
- `X-Real-IP $remote_addr` and `X-Forwarded-For $proxy_add_x_forwarded_for` -> the real client IP; without them the app only sees nginx's IP.
- `X-Forwarded-Proto $scheme` -> tells the app the original request was HTTPS (for correct redirects and secure cookies).

Response headers to the client (`add_header`):

- Security headers: `Strict-Transport-Security` (HSTS), `X-Content-Type-Options nosniff`, `X-Frame-Options`, `Content-Security-Policy`.
- Add `always`, otherwise the header is only sent on 2xx/3xx responses, not on errors.
- **Inheritance gotcha**: `add_header` (and `proxy_set_header`) directives are inherited from the parent block ONLY if the child block defines none of its own. One `add_header` inside a `location` silently drops all the server-level ones -> repeat them or use an `include` snippet.
- `proxy_hide_header X-Powered-By;` -> remove backend headers that leak versions; `server_tokens off;` hides the nginx version.

Other gotchas:

- `underscores_in_headers off` is the default, so headers like `api_key` are silently DROPPED. Use dashes or turn it on.
- Large cookies or JWTs -> `400 Request Header Or Cookie Too Large` -> raise `large_client_header_buffers 4 16k;`. On the response side, big upstream headers need `proxy_buffer_size`.

Example:

```
server {
    listen 443 ssl;
    server_name app.example.com;
    server_tokens off;
    large_client_header_buffers 4 16k;
    include /etc/nginx/snippets/security-headers.conf;   # reusable add_header block

    location /api/ {
        proxy_pass http://app_backend;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_hide_header X-Powered-By;
        include /etc/nginx/snippets/security-headers.conf;  # re-include: add_header here
        add_header Cache-Control "no-store" always;         # would drop parent headers
    }
}
# /etc/nginx/snippets/security-headers.conf
# add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
# add_header X-Content-Type-Options "nosniff" always;
# add_header X-Frame-Options "DENY" always;
```

> ***What to say to the interviewer:*** *I use* `proxy_set_header` *to pass Host, X-Real-IP, X-Forwarded-For and X-Forwarded-Proto so the backend knows the real client and scheme, and* `add_header ... always` *for security headers like HSTS and nosniff. The classic gotchas are that a child block with its own add_header drops all inherited ones, that headers containing underscores are dropped by default, and that large cookies need* `large_client_header_buffers`*.*

Q42. How do you identify and manage parent-child process relationships in Linux?

> **In simple words:** Processes form a family tree. Every process has a parent that created it. When a child finishes, the parent must sign off the paperwork (wait), otherwise the child becomes a zombie; if the parent dies first, the child is adopted by the grandparent of all processes (init/systemd).

Detailed answer

- Every process has a **PID** and a **PPID** (parent's PID). New processes are made with **fork()** (copy of the parent) followed by **exec()** (replace with a new program) - that is how a shell runs every command.
- See the tree: `ps -ef --forest`, `ps -o pid,ppid,pgid,sid,stat,cmd`, `pstree -p`, or `grep PPid /proc/<PID>/status`.
- **Orphans**: if a parent dies first, the child is re-parented to PID 1 (systemd/init) or to a **subreaper** (for example a systemd user manager or a container init like tini), which later reaps it.
- **Zombies**: a child that exited but whose parent has not called `wait()`; the parent receives `SIGCHLD` and should reap it (see Q58).
- **Process groups and sessions**: a pipeline shares a process group (PGID). `kill --<PGID>` signals the whole group - useful to stop a script AND all its children. `pkill -P <PPID>` kills the children of a parent.

- Detaching: `nohup cmd &` ignores hangup when you log out; `setsid cmd` starts a new session with no controlling terminal; for real services use systemd, which tracks all children in a cgroup (`systemctl status` shows the tree, `systemctl stop` kills them all).

Example:

```
pstree -p $(pgrep -o nginx)
# nginx(1201)-+-nginx(1202)
#             `-nginx(1203)
ps -o pid,ppid,pgid,stat,cmd --forest -g $(ps -o sid= -p $$)
grep -E "PPid|State" /proc/1202/status
kill -TERM -- -$(ps -o pgid= -p 4400 | tr -d ' ')   # stop the whole process group
pkill -TERM -P 4400                                  # stop only the children of 4400
setsid ./long_job.sh > job.log 2>&1 < /dev/null &
```

What to say to the interviewer: *I look at PID and PPID with* `ps -ef --forest`*,* `ps -o pid,ppid,pgid,stat` *or* `pstree -p`*, knowing that processes are created by fork and exec, orphans are adopted by PID 1 or a subreaper, and zombies exist until the parent reaps them after SIGCHLD. To manage them I signal a whole process group with* `kill -- -PGID`*, use* `pkill -P` *for the children, and prefer systemd, which tracks every child in a cgroup.*

Q43. Write a script to convert human-readable file sizes (e.g. 1.5K, 20M, 3G) to bytes.

In simple words: Like converting "1.5 kg" and "300 g" into grams so you can compare and add them. The script reads the unit letter, multiplies by the right number, and rejects nonsense like "5X".

Detailed answer

Know the two standards: **IEC** (binary) K = 1024, M = 1024^2, G = 1024^3 (what `ls -h`, `du -h` and `df -h` use), and **SI** (decimal) k = 1000, M = 1000^2 (what disk vendors and `df -H` use). Say which one you assume.

- Easiest: GNU `numfmt --from=iec 1.5K` -> 1536 (use `--from=si` for 1000-based).
- Pure Bash cannot do decimals (integer maths only), so use `awk` for the arithmetic.
- Handle: optional decimals, upper/lowercase units, optional `B`/`iB` suffix, no unit = bytes, invalid input -> error and non-zero exit.

Example:

```
#!/usr/bin/env bash
# Usage: ./to_bytes.sh 1.5K 20M 3G 512
set -uo pipefail

to_bytes() {
  local input="${1^^}"                       # uppercase: 1.5k -> 1.5K
  input="${input%IB}"; input="${input%B}"    # accept 1.5KiB, 20MB, 512B
  if [[ ! "$input" =~ ^([0-9]+(\.[0-9]+)?)([KMGT]?)$ ]]; then
    echo "invalid size: $1" >&2; return 1
  fi
  local num="${BASH_REMATCH[1]}" unit="${BASH_REMATCH[3]}" power=0
  case "$unit" in K) power=1 ;; M) power=2 ;; G) power=3 ;; T) power=4 ;; esac
  awk -v n="$num" -v p="$power" 'BEGIN { printf "%.0f\n", n * (1024 ^ p) }'
}

[ $# -ge 1 ] || { echo "Usage: $0 <size>..." >&2; exit 1; }
rc=0
for s in "$@"; do
  if b=$(to_bytes "$s"); then printf "%-8s = %s bytes\n" "$s" "$b"; else rc=2; fi
done
exit $rc
```

```
$ ./to_bytes.sh 1.5K 20M 3g 512 2T 5X
1.5K     = 1536 bytes
20M      = 20971520 bytes
3g       = 3221225472 bytes
512      = 512 bytes
2T       = 2199023255552 bytes
invalid size: 5X

$ numfmt --from=iec 1.5K 20M 3G       # one-liner alternative
1536
20971520
3221225472
$ numfmt --from=si 20M                # decimal units
20000000
```

What to say to the interviewer: *The quick answer is* `numfmt --from=iec`*, or* `--from=si` *for 1000-based units. In a script I uppercase the input, strip an optional B or iB, validate it with a regex capturing the number and a K, M, G or T unit, and do the multiplication in awk because Bash has no floating point, returning an error for invalid input. I also state whether I assume 1024-based IEC or 1000-based SI units.*

Scenario-Based

Q44. An Ubuntu OS upgrade failed midway. How do you troubleshoot and recover?

In simple words: It is like a house renovation stopped halfway - first make sure the house still stands, then finish or undo the half-done work carefully, and next time take photos (backups) before starting.

Detailed answer

1. **Before anything, snapshot** (EBS/VM snapshot) if the machine is still up - so recovery attempts are reversible.

2. If it still boots: finish the interrupted package work:

- `sudo dpkg --configure -a` -> configure half-installed packages.
- `sudo apt --fix-broken install` (`apt -f install`).
- `sudo apt update && sudo apt full-upgrade`.
- Check `/var/log/dist-upgrade/` and `/var/log/apt/term.log` for the actual error.

3. Common causes: disk full on `/` or `/boot` (old kernels), held packages, third-party PPAs, network interruption, dpkg lock left behind.

4. If it does **not** boot: at GRUB choose **Advanced options** -> an older kernel or recovery mode -> root shell -> remount rw (`mount -o remount,rw /`) and run the dpkg/apt fixes.

5. If GRUB itself is broken: boot a live USB (or on AWS, detach the root volume and attach it to a rescue instance), `chroot` into the disk, fix packages, `update-grub`/`grub-install`.

6. Last resort: restore from snapshot/backup, or rebuild (immutable infra makes this easy).

7. Prevention: test the upgrade on staging first, take a snapshot, run in `tmux`/`screen` so an SSH drop does not kill it, ensure free space.

Example:

```
sudo dpkg --configure -a
sudo apt --fix-broken install
sudo apt update && sudo apt full-upgrade
df -h / /boot
tail -50 /var/log/dist-upgrade/main.log

# Rescue via chroot (live USB or rescue EC2 with the volume attached)
sudo mount /dev/xvdf1 /mnt
for d in dev proc sys run; do sudo mount --bind /$d /mnt/$d; done
sudo chroot /mnt
dpkg --configure -a && apt -f install && update-grub
```

> ***What to say to the interviewer:*** *If the system still boots I run* `dpkg --configure -a` *and* `apt --fix-broken install`*, then read the dist-upgrade logs for the real cause, often a full disk or a third-party repo. If it will not boot I use an older kernel or recovery mode, or a live/rescue instance with chroot to repair, and restore from snapshot as a last resort; going forward I always snapshot and test upgrades on staging, running them in tmux.*

Q45. A Linux node shows high CPU usage even though all the pods on it appear healthy. What could be the reasons?

(also asked as: A service is consuming 100% CPU - how do you troubleshoot it? / High CPU usage is degrading performance - what do you do?)

> **In simple words:** The shop assistants (pods) look fine, but the building is still overheating. Something else is using the power - the building's own machinery, a hidden process, or even the neighbour sharing your electricity line (noisy neighbour on a VM).

Detailed answer

Step-by-step:

1. `top`/`htop` on the node -> sort by CPU; is it a pod process or a **system process** (kubelet, containerd, dockerd, journald, antivirus/security agents, log shippers)?

2. Read the CPU breakdown in `top`:

- `us` high -> user processes (app or agent).
- `sy` high -> kernel work: too many syscalls, context switching, network interrupts, iptables with thousands of rules (kube-proxy).
- `wa` high -> waiting on disk IO, not real compute.
- `st` high -> **steal time**, the hypervisor gives your CPU to other VMs, or a burstable T-instance ran out of CPU credits.
- `si` high -> softirq, heavy network packet processing.

3. Pods with **no CPU limits** or `BestEffort` pods may burst and consume the node, while still passing health checks.
4. Zombie/runaway processes, cron jobs, or leftover containers not managed by Kubernetes (`crictl ps -a`).
5. Kubelet itself busy: image garbage collection, too many pods, frequent probes, log rotation.
6. Security incident: crypto-miner - check unknown processes and outbound connections.
7. Compare `kubectl top pods` vs node `top` - the gap is the non-pod usage.

If one known service is at 100% CPU: find the PID (`top`, `pidstat`), then the hot thread (`top -H -p <PID>`); for Java run `jstack <PID>` and match the thread id in hex; use `strace -c -p <PID>` to see if it is spinning on syscalls, and `perf top -p <PID>` for hot functions. Check its logs and recent deploys/config changes (infinite retry loop, bad regex, GC thrash). Short-term: restart it or cap it (`systemctl set-property svc CPUQuota=50%`); long-term: fix the code and add CPU alerts.

Example:

```
kubectl top node ip-10-0-1-23
kubectl top pods -A --sort-by=cpu | head
# On the node:
top -o %CPU          # check us / sy / wa / st values
pidstat -u 2 5       # per-process CPU over time
crictl stats         # per-container CPU
mpstat -P ALL 1 3    # per-core; one core at 100% = single-threaded hog
```

> ***What to say to the interviewer:*** *I compare* `kubectl top` *with* `top` *on the node to see whether the load is from pods or from system components like kubelet, containerd or agents. Then I read the CPU breakdown - high system time, IO wait or steal time each point to different causes like iptables, disk bottlenecks or a noisy neighbour or exhausted T-instance credits - and I also check for pods without limits and unknown processes.*

Q46. Disk utilization on production servers reached 95%. What immediate and long-term actions would you take?

(also asked as: The / partition is full - how do you find and delete large files safely? / What is using up /var? / You got a low disk space alert - how do you clean up safely?)

> **In simple words:** Your cupboard is almost full. Right now you throw out the obvious junk so the door closes; later you buy a bigger cupboard, set a rule for old stuff, and put a sensor that warns you at 80%.

Detailed answer

Immediate (stop the bleeding, safely):

1. Find what is big: `df -h`, then `du -xh --max-depth=1 / | sort -h`, and `ncdu` if installed. Also `df -i` for inodes.
2. Check **deleted-but-open files**: `lsof +L1` - a deleted log still held by a process keeps using space until the process restarts or you truncate it.
3. Safe quick wins: `journalctl --vacuum-size=500M`, remove old rotated logs (`*.gz`), `apt clean`, old kernels (`apt autoremove`), `docker system prune` (careful on prod), old core dumps and temp exports.
4. Truncate a live huge log instead of deleting it: `truncate -s 0 app.log`.
5. Never delete database files or anything you do not understand - move/compress first.
6. If still critical: grow the volume online (EBS modify -> `growpart` -> `resize2fs`/`xfs_growfs`).

Long-term:

- Log rotation (`logrotate`, journald `SystemMaxUse`), ship logs to central storage (Loki/ELK/CloudWatch).
- Alert at 80% and 90% with trend-based "disk full in X hours" alerts.
- Separate volumes for `/var/log` or data; lifecycle cleanup cron; capacity planning.
- Write an RCA: what grew and why.

Example:

```
df -h; df -i
sudo du -xh --max-depth=1 /var | sort -h | tail
sudo lsof +L1 | awk '$7 > 100000000'        # deleted files still open, >100MB
sudo journalctl --vacuum-size=500M
sudo truncate -s 0 /var/log/myapp/app.log
# Grow EBS after modifying volume size in AWS:
sudo growpart /dev/nvme0n1 1 && sudo resize2fs /dev/nvme0n1p1
```

> ***What to say to the interviewer:*** *Immediately I find the biggest consumers with* `du`*, check for deleted-but-open files with* `lsof +L1`*, and free space safely by vacuuming journald, removing old rotated logs, and truncating huge live logs, or grow the EBS volume online if needed. Long term I fix log rotation, ship logs centrally, add 80/90 percent and trend alerts, and document the root cause.*

Q47. One of your EC2/Linux nodes keeps going out of memory. How would you diagnose and prevent it?

(also asked as: How do you locate and stop a process consuming high memory? / A server shows consistently high memory usage - what do you check?)

> **In simple words:** A room keeps overflowing with guests and the bouncer (OOM killer) keeps throwing someone out. You check the guest list to see who is bringing too many friends, then set a limit per guest or get a bigger room.

Detailed answer

Diagnose:

1. Confirm OOM kills: `dmesg -T | grep -i -E "killed process|out of memory"` or `journalctl -k | grep -i oom`. The log names the killed process and its memory.
2. Current usage: `free -h` (watch **available**), `ps aux --sort=-%mem | head`, `smem` for real per-process usage.
3. Trend over time: CloudWatch (needs the **CloudWatch agent** - memory is not a default EC2 metric) or Prometheus node-exporter. A steady climb = **memory leak**; sudden spikes = a batch job/traffic burst.
4. Check what changed: new deployment, config change (e.g. JVM `-Xmx`, PHP-FPM/Gunicorn worker counts too high), cache growth.
5. On Kubernetes nodes: pods without memory limits, too many pods, and missing `kube-reserved`/`system-reserved` so system daemons get starved.

Consistently high memory (no OOM kills): first check it is real. Linux fills spare RAM with page cache (`buff/cache` in `free -h`), which is released on demand, so only "available" dropping and swap in use (`vmstat si/so`) mean real pressure. Then trend it: flat = normal working set (maybe right-size); a steady climb = leak.

Locate and stop a memory hog (right now): `ps aux --sort=-%mem | head` or `top` then `M` -> note the PID and owner. If it is a managed service, restart it properly (`systemctl restart svc`) rather than killing it. Otherwise `kill <PID>` (SIGTERM, lets it clean up), wait, and use `kill -9` only if it ignores that. Capture evidence first if possible (heap dump, `pmap -x <PID>`), because a restart hides the leak.

Prevent:

- Right-size the instance or tune the app (worker counts, heap size).
- Set memory limits (systemd `MemoryMax=`, container/pod limits) so one process cannot eat the node.
- Add swap only as a buffer on non-latency-critical servers (not usually on K8s nodes).
- Alert on memory above 85% and on OOM events; fix leaks with the dev team using heap dumps.

Example:

```
dmesg -T | grep -i "killed process"
# [Tue Sep 22 03:14:07 2026] Out of memory: Killed process 2143 (java) total-vm:6.1GB, anon-rss:3.8G
    B
free -h
ps aux --sort=-%mem | head -5
# Cap a service with systemd
sudo systemctl set-property myapp.service MemoryMax=1500M
```

> ***What to say to the interviewer:*** *I confirm the OOM kill in* `dmesg` *or the kernel journal to see which process was killed, then check per-process memory and the trend over time with the CloudWatch agent or node-exporter to tell a leak from a spike. To prevent it, I right-size or tune the app's workers and heap, set memory limits via systemd or pod limits, reserve memory for system daemons on K8s nodes, and alert on high memory and OOM events.*

Q48. Latency spikes suddenly in production during off-peak hours. Where do you start debugging?

In simple words: If the roads are jammed at 3 AM when nobody is driving, it is not rush hour - it is probably road maintenance. Off-peak slowness usually comes from scheduled background work, not users.

Detailed answer

Low traffic + high latency points to **scheduled or internal** causes. Order of checks:

1. **Correlate the time**: does it happen at the same minute every day/hour? That strongly suggests cron jobs, backups, batch/ETL jobs, log rotation, DB maintenance (vacuum, index rebuild), snapshots, antivirus scans, or cache expiry.
2. **Scope it**: all endpoints or one? All hosts or one AZ/node? Use p95/p99 per endpoint and per instance.
3. **Scale-in side effects**: autoscaling removed capacity at night, so the few remaining instances or cold caches struggle; scaled-to-zero services have **cold starts**; JIT/connection pools re-warm.
4. **Host level** at that minute: `sar`/`iostat` for IO wait (backups hammering disk), CPU steal or exhausted burst credits on T-instances, GC pauses in logs.
5. **Network/DNS**: DNS TTL expiry, connection pools idle-timed out by NAT/LB (350s NAT idle timeout on AWS) -> new handshakes; certificate or token refresh.
6. **Dependencies**: DB slow query log, third-party API maintenance windows.
7. Use traces to see which span grows.

Example:

```
crontab -l; ls /etc/cron.d/; systemctl list-timers        # scheduled jobs
sar -u -d -f /var/log/sysstat/sa22 -s 02:50:00 -e 03:20:00  # CPU/disk at spike time
grep -i "pause" /var/log/myapp/gc.log | tail
curl -o /dev/null -s -w "connect:%{time_connect} ttfb:%{time_starttransfer}\n" https://api/x
```

What to say to the interviewer: *Because traffic is low, I suspect internal causes first, so I correlate the spike time with cron jobs, backups, batch jobs, DB maintenance and autoscaling scale-in or cold starts. Then I scope it by endpoint and host using p99 metrics and traces, and check host IO wait, CPU steal or credits, GC pauses, and idle connection timeouts at that exact window.*

Q49. A user cannot log in to a Linux server. How do you troubleshoot?

In simple words: Someone cannot enter the office. Is their ID card expired, blocked, the wrong card, is the door itself broken, or is their desk (home folder) missing?

Detailed answer

1. Get the exact error and method (SSH key, password, console).
2. Read the auth log: `/var/log/auth.log` (Ubuntu) or `/var/log/secure` (RHEL), or `journalctl -u sshd`.
3. Account state: `id user`, `chage -l user` (password expired?), `passwd -S user` (L = locked), `faillock --user user` or `pam_tally2` (locked after failed attempts).

4. Shell: `/etc/passwd` shell set to `/sbin/nologin` or `/bin/false`?
5. SSH rules: `AllowUsers`/`AllowGroups`/`DenyUsers` in `/etc/ssh/sshd_config`; key in `~/.ssh/authorized_keys` with correct permissions.
6. Home directory missing or wrong owner.
7. Disk full (`df -h`) or too many processes can also block logins.

Example:

```
sudo tail -f /var/log/auth.log
passwd -S priya;  chage -l priya
sudo faillock --user priya --reset
getent passwd priya          # check shell and home
sudo usermod -s /bin/bash priya
```

> ***What to say to the interviewer:*** *I read the auth log first because it usually names the reason, then check whether the account is locked or expired with* `passwd -S`*,* `chage -l` *and faillock, whether the shell is nologin, and sshd AllowUsers rules and key permissions. I also check the home directory and disk space.*

Q50. A script is executable by one user but not another, or a user cannot access a directory even though they are in the correct group. Why?

(also asked as: A user cannot access a directory even though they are in the correct group)

> **In simple words:** Two people have the same door key, but one is not on the building's allowed list, or the corridor leading to the door is locked for them.

Detailed answer

Check in this order:

- File permissions: `ls -l script.sh` -> maybe `rwxr-x---`, so "others" cannot execute. The second user is not the owner or in the group.
- Parent directories: every directory in the path needs `x` for that user (`namei -l /opt/scripts/script.sh` shows each level).
- Interpreter: the shebang (`#!/usr/bin/env bash`) must be executable by them.
- Mount options: a filesystem mounted `noexec` (like `/tmp` on hardened servers) blocks execution for everyone running it from there.
- ACLs: `getfacl script.sh` may explicitly deny a user.
- SELinux/AppArmor contexts: `ls -Z`.
- Inside the script: it may read files or use sudo that only user A is allowed.
- `PATH` differences: user B runs a different copy.
- Group just added? Group membership is read at login, so the user's current session still has the OLD groups - `id` in their shell will not show it. Log out and in again, or run `newgrp devops` for that shell.
- Directory needs the right bits: `r` to list, `x` to enter, `w+x` to create or delete files inside.
- NFS/EFS shares: the server checks numeric UIDs/GIDs, so a GID mismatch between servers or `root_squash` blocks access.

- Immutable attribute: `lsattr` shows `i`, and then even root cannot modify it (`chattr -i` to remove).

Example:

```
ls -l /opt/scripts/deploy.sh      # -rwxr-x--- root devops
namei -l /opt/scripts/deploy.sh
getfacl /opt/scripts/deploy.sh
id userb                          # not in devops group
sudo usermod -aG devops userb     # fix (user must log in again)
```

> ***What to say to the interviewer:*** *I compare the file's owner, group and mode with the second user's groups using* `ls -l` *and* `id`*, check every parent directory with* `namei -l`*, and look at ACLs, noexec mounts and SELinux contexts. Usually the fix is adding the user to the right group or adjusting group permissions, not chmod 777.*

Q51. You cannot SSH into a server. How do you troubleshoot?

(also asked as: SSH is not working)

> **In simple words:** You cannot enter a house. Either the road to it is blocked (network), nobody opens the door (sshd down), or the door opens but your key does not fit (authentication).

Detailed answer

1. Read the error: `Connection timed out` -> network/firewall; `Connection refused` -> host reachable, sshd not listening; `Permission denied (publickey)` -> auth problem; `Host key verification failed` -> host key changed (rebuilt server).
2. Verbose client: `ssh -vvv user@host` shows the exact stage that fails.
3. Network: `nc -zv host 22`, Security Group/NACL inbound 22 from your IP, route tables, public IP or bastion path.
4. Auth: right user (`ec2-user` vs `ubuntu`), right key, key file mode `600`, server `~/.ssh` 700 and `authorized_keys` 600, correct owner.
5. Server side, via another path (EC2 **Serial Console**, **SSM Session Manager**, console): `systemctl status sshd`, `sshd -t` (config syntax), `/var/log/auth.log`, disk full, `fail2ban` ban.
6. Last resort on AWS: stop instance, attach the root volume to a rescue instance, fix config/keys.

Example:

```
ssh -vvv -i key.pem ec2-user@10.0.1.5
nc -zv 10.0.1.5 22
chmod 600 key.pem
aws ssm start-session --target i-0abc123     # no SSH needed
sudo sshd -t && sudo systemctl restart sshd
```

> ***What to say to the interviewer:*** *I read the exact error with* `ssh -vvv` *to classify it as network, sshd or authentication. Then I check port 22 and security groups for timeouts, sshd status and config for refusals, and user, key and permissions for auth failures, using SSM Session Manager or the serial console to get in if SSH is fully broken.*

Q52. Ping to a server works, but SSH using its hostname fails. Why?

In simple words: You can reach the house by its exact GPS coordinates, but using its name sends you to a different house, or the house is open for waving (ping) but its door (port 22) is locked.

Detailed answer

- Did ping use the **IP** and SSH the **name**? Then it is DNS: `dig host`, `getent hosts host` - the name may resolve to a different/old IP, IPv6 (AAAA) first, or a wrong `/etc/hosts` entry.
- If ping by name also works: ping is ICMP, SSH is TCP 22 - the firewall/Security Group may allow ICMP but not port 22. Test `nc -zv host 22`.
- `~/.ssh/config` may override the hostname, user, port or ProxyJump for that name.
- `known_hosts` mismatch for that name ("REMOTE HOST IDENTIFICATION HAS CHANGED") if the name now points to a rebuilt server.
- sshd running on a non-default port.
- Slow login rather than failure: `UseDNS yes` on the server doing reverse lookups.

Example:

```
getent hosts app01.internal; dig +short app01.internal
ssh -G app01.internal | grep -E "^(hostname|port|user) "
nc -zv app01.internal 22
ssh-keygen -R app01.internal         # remove stale host key
```

What to say to the interviewer: *Ping only proves ICMP to some IP works, so I check whether the hostname resolves to the same IP with* `getent hosts` *or dig, whether port 22 is actually open with* `nc -zv`*, and whether ~/.ssh/config or a stale known_hosts entry is interfering.*

Q53. A log file shows junk characters when you open it. What could be wrong?

In simple words: It is like opening a zipped parcel and trying to read the packing foam - the content is fine, you are just looking at it in the wrong form.

Detailed answer

- It is **compressed**: rotated logs like `app.log.1.gz` -> use `zcat`, `zless`, `zgrep`. Check with `file app.log`.
- It is **binary**: systemd journal files, `wtmp`/`btmp` (use `last`, `lastb`), or database files.
- **Encoding** mismatch: UTF-16 from a Windows app, or a different charset -> `file -i`, `iconv -f UTF-16 -t UTF-8`.
- **ANSI color codes** from apps (`^[[32m`) -> `less -R` or strip with sed.
- **NUL bytes** at the start after log rotation with `copytruncate` while the app keeps writing at its old offset -> the file becomes "sparse" with zeros. Fix: make the app open with append mode or reopen logs on rotation (signal/postrotate) instead of copytruncate.
- Disk/filesystem corruption (rare) -> check `dmesg`.

Example:

```
file /var/log/app.log.2.gz          # gzip compressed data
zgrep "ERROR" /var/log/app.log.2.gz
file -i app.log                     # charset=utf-16le
iconv -f UTF-16LE -t UTF-8 app.log > app-utf8.log
less -R colored.log
tr -d '\000' < app.log | less       # strip NUL bytes
```

What to say to the interviewer: *I run* `file` *on it first: it is often a gzip-rotated log to read with zcat or zgrep, a binary file like journal or wtmp, a UTF-16 encoding to convert with iconv, or ANSI colour codes. NUL bytes at the start usually mean logrotate copytruncate while the app keeps its old offset.*

Q54. A cron job is not running. How do you debug it?

(also asked as: A cron job did not run last night - what do you check?)

In simple words: The alarm clock did not ring. Is the clock plugged in (cron service), was the time set right, and when it rang did the person actually know how to do the task without their usual tools (environment)?

Detailed answer

1. Is cron running? `systemctl status cron` (Ubuntu) / `crond` (RHEL).
2. Did it trigger? `grep CRON /var/log/syslog` or `journalctl -u cron` -> if the line is there, cron ran it and the **script** failed.
3. Syntax: check the 5 time fields (crontab.guru logic), and in `/etc/crontab` or `/etc/cron.d/` there is an extra **user** field. Files in `/etc/cron.d` must not have dots in the name and need a trailing newline.
4. Environment: cron has minimal `PATH` and no profile -> use absolute paths, set `PATH=` at the top, `cd` to the right directory.
5. `%` is special in crontab (means newline) -> escape as `\%` (common with `date +%F`).
6. Permissions: script executable? `/etc/cron.allow` / `cron.deny`? Correct user's crontab?
7. Capture output: `>> /tmp/job.log 2>&1` and test with `env -i /bin/sh -c '/path/script.sh'` to mimic cron.
8. Timezone of the server vs what you expected (cron uses system time).

Example:

```
systemctl status cron
grep CRON /var/log/syslog | tail
crontab -l
# wrong:  0 2 * * * backup.sh
# right:
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
0 3 * * * /usr/bin/tar czf /backup/app-$(date +\%F).tgz /opt/app
```

What to say to the interviewer: *I confirm the cron service runs and check syslog or journalctl to see whether the job was triggered; if it was, the script is failing, usually because of cron's minimal PATH, relative paths, or an unescaped % sign. I redirect output to a log file and reproduce with* `env -i` *to mimic cron's environment.*

Q55. yum/dnf or apt is failing. How do you troubleshoot it?

In simple words: The shop you order parts from is not delivering. Maybe you cannot reach the shop (network/proxy), the catalogue is outdated (metadata), the shop's ID is not trusted (GPG key), or another order is still being processed (lock).

Detailed answer

1. Read the full error - it usually names the repo or package.
2. Network: can you reach the repo? `curl -I <repo-url>`, DNS, proxy settings, NAT gateway / VPC endpoint for private instances.
3. Lock: another apt/dnf or `unattended-upgrades` is running -> wait; `ps aux | grep -E "apt|dpkg"`. Only remove stale lock files if no process holds them.
4. Stale metadata: `apt clean && apt update` / `dnf clean all && dnf makecache`.
5. GPG/key errors: import the repo key (expired keys are common with third-party repos).
6. Broken or interrupted installs: `dpkg --configure -a`, `apt --fix-broken install`; `dnf check`, `package-cleanup --problems`.
7. Bad or dead third-party repo: disable it (`/etc/apt/sources.list.d/`, `dnf config-manager --set-disabled`).
8. Disk full on `/var` or `/boot`; clock wrong (TLS/GPG validity errors).

Example:

```
sudo apt update 2>&1 | tail
sudo lsof /var/lib/dpkg/lock-frontend
sudo dpkg --configure -a && sudo apt --fix-broken install
sudo dnf clean all && sudo dnf makecache
curl -I https://archive.ubuntu.com/ubuntu/
df -h /var /boot; timedatectl
```

What to say to the interviewer: *I read the exact error, then check repo reachability, DNS and proxy, whether another package process holds the lock, and refresh metadata with clean and update. Other common causes are expired GPG keys, dead third-party repos, interrupted installs needing* `dpkg --configure -a`*, a full /var or /boot, and a wrong system clock.*

Q56. A user is in the sudoers file but sudo still fails. Why?

In simple words: Your name is on the VIP list, but the list has a typo, a later line overrides you, or the bouncer is checking a different list.

Detailed answer

- Syntax error in `/etc/sudoers` or a file in `/etc/sudoers.d/` -> sudo may refuse to work at all. Always edit with `visudo` and check with `visudo -c`.
- Rule order: sudoers uses the **last matching rule** - a later `%group` line may restrict the user.
- Group membership added but the user did not log out/in -> `id` in their session will not show the new group.
- Rule limited to certain commands or hosts, or `!` negations.
- Files in `/etc/sudoers.d/` ignored if their name contains a `.` or `~`, or have wrong permissions (should be `0440`, owned by root).

- Password expired, account locked, or `requiretty` settings.
- Messages: check `/var/log/auth.log` or `/var/log/secure` - "user NOT in sudoers" vs "command not allowed".

Example:

```
sudo visudo -c
sudo -l -U priya              # what is this user actually allowed?
id priya
ls -l /etc/sudoers.d/         # -r--r----- root root  90-priya
echo 'priya ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx' | sudo tee /etc/sudoers.d/90-priya
sudo chmod 440 /etc/sudoers.d/90-priya
```

> ***What to say to the interviewer:*** *I check syntax with* `visudo -c`*, see the user's effective rules with* `sudo -l -U user`*, and confirm with* `id` *that a newly added group is active in their session. Common causes are a later rule overriding theirs, a sudoers.d file with a dot in its name or wrong permissions, and command-restricted rules; the auth log shows which one.*

Q57. A server hit a kernel panic. How do you troubleshoot and recover?

> **In simple words:** The building's power system tripped completely. First get power back (reboot / older kernel), then read the black box recorder (crash dump, logs) to see what caused the trip.

Detailed answer

Recover:

1. Reboot. If it panics again, pick an **older kernel** in GRUB or recovery mode (on AWS use the EC2 Serial Console, or attach the root volume to a rescue instance).
2. Common triggers: new kernel or driver/module after patching, bad fstab or missing root device, corrupted initramfs, hardware/memory faults, out-of-memory with `panic_on_oom`.
3. Fixes: boot the old kernel and remove/hold the bad one, rebuild initramfs (`dracut -f` / `update-initramfs -u`), fix fstab, fsck the root disk.

Find the root cause:

- Screenshot/serial console output (EC2 "Get system log" / "Get instance screenshot").
- `journalctl -k -b -1` (previous boot kernel log) if persistent journald is enabled.
- **kdump** captures a crash dump in `/var/crash`, analysed with the `crash` tool.
- Recent changes: new kernel, modules, hardware events.

Example:

```
aws ec2 get-console-output --instance-id i-0abc123 --latest --output text | tail -40
journalctl -k -b -1 | tail -50
ls /var/crash/
sudo dracut -f            # RHEL: rebuild initramfs
sudo update-initramfs -u  # Ubuntu
sudo systemctl enable --now kdump
```

> ***What to say to the interviewer:*** *I bring it back first, booting the previous kernel or using recovery mode or a rescue instance, then find the cause from the console output, the previous boot's kernel log and a kdump crash dump. The usual culprits are a new kernel or driver, a broken initramfs or fstab, or hardware, and I enable kdump so the next panic leaves evidence.*

Q58. How do you find and handle zombie processes?

In simple words: A zombie is a worker who has finished and gone home, but their name is still on the attendance sheet because the manager (parent process) never signed them out.

Detailed answer

- A zombie (Z state, `<defunct>`) has already exited; it uses no CPU or memory, only a PID table entry. It exists because the **parent** did not call `wait()` to collect its exit status.
- You **cannot kill a zombie** - it is already dead. `kill -9` on it does nothing.
- Fix the parent: send it `SIGCHLD` (`kill -s SIGCHLD <PPID>`) to prompt it to reap; if that fails, restart/kill the parent -> the zombies are adopted by init/systemd (PID 1), which reaps them.
- A few zombies are harmless; thousands can exhaust the PID limit (`pid_max`) and stop new processes starting.
- In containers, if your app is PID 1 and does not reap children, use `tini` or `docker run --init` (or `shareProcessNamespace` patterns) - a common cause.
- Long-term: fix the parent's code to reap children.

Example:

```
ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/'
#  4521  4400  Z   [worker] <defunct>
ps -o pid,cmd -p 4400                 # the parent
sudo kill -s SIGCHLD 4400
sudo systemctl restart myapp          # if the parent will not reap
```

What to say to the interviewer: *I list them with* `ps` *filtering for Z state and find the parent PID, because a zombie is already dead and cannot be killed; the parent has not reaped it. I signal the parent with SIGCHLD or restart it so PID 1 adopts and reaps them, and in containers I use tini or* `--init` *as PID 1.*

Q59. An application fails with "Too many open files". How do you fix it?

In simple words: Every process has a limited number of drawer handles it can hold at once. The app is holding too many - either it needs a bigger allowance or it is forgetting to let go (leak).

Detailed answer

1. Confirm the limit and usage: `cat /proc/<PID>/limits | grep "open files"` and `ls /proc/<PID>/fd | wc -l` (or `lsof -p PID | wc -l`).
2. Leak or real need? If the count keeps rising with steady traffic, it is a **file/socket leak** (connections not closed). Look at `lsof -p PID` - many sockets in `CLOSE_WAIT` point to the app not closing connections.
3. Raise the limit correctly:

- systemd services: `LimitNOFILE=65536` in the unit (or a drop-in), then `daemon-reload` and restart. (`/etc/security/limits.conf` does NOT apply to systemd services.)
- Login sessions: `/etc/security/limits.conf` (`nofile`), then re-login; check with `ulimit -n`.
- System-wide max: `fs.file-max` / `fs.nr_open` via sysctl (rarely the actual problem).

- Containers: `--ulimit nofile=65536:65536`.

4. Monitor fd usage so it alerts before the limit.

Example:

```
PID=$(pgrep -f myapp | head -1)
grep "open files" /proc/$PID/limits      # Max open files 1024 1024
ls /proc/$PID/fd | wc -l                 # 1021
lsof -p $PID | awk '{print $NF}' | sort | uniq -c | sort -rn | head
sudo systemctl edit myapp                # add: [Service] LimitNOFILE=65536
sudo systemctl daemon-reload && sudo systemctl restart myapp
```

> ***What to say to the interviewer:*** *I compare the process's current descriptor count in /proc/PID/fd with its limit in /proc/PID/limits and check with lsof whether it is a leak, such as sockets piling up in CLOSE_WAIT. If the app genuinely needs more, I raise LimitNOFILE in the systemd unit, since limits.conf does not apply to services, and add monitoring on descriptor usage.*

Q60. A user's home directory is missing. How do you restore it?

> **In simple words:** An employee's locker was removed. You give them a new empty locker with the standard starter kit, and if their old belongings matter you fetch them from the storage backup.

Detailed answer

1. Confirm: `getent passwd user` shows the expected home path; check whether it was moved or is on a mount that did not come up (NFS/EFS home, `/home` on a separate disk -> `df -h`, `mount -a`).
2. Recreate a fresh one: `mkhomedir_helper user` (or `mkdir` + copy `/etc/skel`), set owner `user:user` and mode `700`/`750`.
3. Restore data from backup/snapshot (EBS snapshot -> new volume -> mount and copy with `rsync -a` to keep permissions).
4. Restore `~/.ssh/authorized_keys` so SSH key login works again (700 on `.ssh`, 600 on the file).
5. If it happens for many users, enable `pam_mkhomedir` so homes are auto-created on first login (common with LDAP/AD users).
6. Investigate why: `userdel -r`, cleanup script, failed mount - check auth logs and bash history.

Example:

```
getent passwd priya            # priya:x:1001:1001::/home/priya:/bin/bash
sudo mkhomedir_helper priya    # or:
sudo cp -r /etc/skel /home/priya && sudo chown -R priya:priya /home/priya && sudo chmod 750 /home/pr
    iya
sudo rsync -a /mnt/snapshot/home/priya/ /home/priya/
```

> ***What to say to the interviewer:*** *I first check whether it is truly deleted or just on a mount that failed, then recreate it with mkhomedir_helper or /etc/skel with the right ownership and permissions, and restore data and SSH keys from a backup or snapshot using rsync -a. For directory-based users I enable pam_mkhomedir, and I find out what removed it.*

Q61. The server time is wrong. How do you fix it using NTP/chrony?

In simple words: If a server's watch is wrong, it arrives late to every meeting - certificates look expired, logs do not line up, and logins with time-based tokens fail. NTP keeps checking the watch against an atomic clock.

Detailed answer

1. Check: `timedatectl` -> "System clock synchronized: no" / "NTP service: inactive"? Also check the **timezone** - maybe the time is right but the zone is wrong.
2. Use **chrony** (default on RHEL/Amazon Linux, common on Ubuntu) or systemd-timesyncd.
3. Configure servers in `/etc/chrony.conf`: on AWS use the **Amazon Time Sync Service** `169.254.169.123` (no internet needed); otherwise `pool.ntp.org` or company NTP.
4. Make sure UDP 123 outbound is allowed if using external servers.
5. Large offsets: chrony slews (slowly adjusts) by default; `chronyc makestep` jumps immediately (careful with running databases/apps).
6. Verify with `chronyc tracking` and `chronyc sources -v`.
7. Monitor clock offset (node-exporter `node_timex_offset_seconds`).

Example:

```
timedatectl
sudo timedatectl set-timezone Asia/Kolkata
echo "server 169.254.169.123 prefer iburst" | sudo tee -a /etc/chrony.conf
sudo systemctl enable --now chronyd
sudo chronyc makestep
chronyc tracking | grep -E "System time|Leap"
chronyc sources -v
```

What to say to the interviewer: *I check* `timedatectl` *for sync status and timezone, configure chrony with a reliable source - on AWS the Amazon Time Sync Service at 169.254.169.123 - and restart it, using* `chronyc makestep` *for a large offset. I verify with* `chronyc tracking` *and monitor clock drift, because wrong time breaks TLS, tokens, clusters and log correlation.*

Q62. You deleted large files but disk space was not freed. Why? What happens when a file is deleted while still in use?

(also asked as: What happens when a file is deleted but still in use?)

In simple words: Deleting a file removes its name from the index, but if someone is still reading the book, the library cannot recycle the paper until they give it back.

Detailed answer

- On Linux, `rm` removes the **directory entry** (the name/link). The data blocks and inode are freed only when the link count is 0 **and** no process has the file open.
- A running process (e.g. an app writing to its log) keeps writing to the deleted file, invisible to `ls` and `du` but still using space shown by `df`.
- Find them: `lsof +L1` or `lsof | grep deleted`.
- Fix: restart/reload the process (logrotate `postrotate` does this), or truncate through its file descriptor without restart: `: > /proc/<PID>/fd/<FD>`.

- Prevention: truncate live logs (`truncate -s 0`) instead of `rm`, and use proper logrotate.
- Bonus: while the process runs, the deleted file can still be **recovered** by copying `/proc/<PID>/fd/<FD>`.

Example:

```
sudo lsof +L1
# COMMAND  PID USER  FD   TYPE SIZE/OFF    NLINK NAME
# java    2143 app   12w  REG  8589934592  0     /var/log/app.log (deleted)
sudo sh -c ': > /proc/2143/fd/12'     # free space without restart
sudo systemctl restart myapp           # or restart the process
```

What to say to the interviewer: *Deleting a file only removes its name; the space is freed when no process holds it open, so an app still writing to a deleted log keeps consuming space that df shows but du does not. I find those with `lsof +L1` and either restart the process or truncate it through /proc/PID/fd, and prevent it by truncating instead of deleting live logs.*

Q63. df and du show different disk usage. Why?

In simple words: df reads the building's official meter, du walks around counting boxes it can see. If some boxes are hidden or thrown out but still held by someone, the two numbers disagree.

Detailed answer

Main reasons, most common first:

1. **Deleted-but-open files** -> counted by `df`, invisible to `du`. Check `lsof +L1` (see previous question).
2. **Files hidden under a mount point** -> data written to `/data` before a disk was mounted on `/data` sits underneath, hidden. Check with a bind mount: `mount --bind / /mnt/root && du -sh /mnt/root/data`.
3. **Reserved blocks** -> ext4 reserves ~5% for root, so df "Used + Avail" is less than "Size". Can reduce with `tune2fs -m 1` on data disks.
4. Filesystem metadata/journal overhead.
5. `du` run without `-x`, or without permission to read some folders, or double-counting across mounts.
6. Sparse files and hard links count differently (`du --apparent-size`).

Example:

```
df -h /; sudo du -xsh /
sudo lsof +L1 | head
sudo mkdir -p /mnt/rootfs && sudo mount --bind / /mnt/rootfs
sudo du -sh /mnt/rootfs/data           # data hidden under the /data mount?
sudo umount /mnt/rootfs
sudo tune2fs -l /dev/nvme0n1p1 | grep "Reserved block count"
```

What to say to the interviewer: *df reads filesystem-level usage while du sums the files it can see, so the gap is usually deleted files still held open, which I find with `lsof +L1`, or data hidden underneath a mount point, which I reveal with a bind mount. Reserved blocks on ext4 and metadata overhead explain smaller differences.*

Q64. The system load is very high. How do you analyse it?

In simple words: Load average is the length of the queue at the counters. A long queue can mean the cashiers are all busy (CPU) or everyone is waiting for the stockroom (disk IO) - the fix is different.

Detailed answer

1. `uptime` -> 1, 5, 15-minute load averages. Compare to CPU count (`nproc`): load 8 on 8 cores is full, on 2 cores it is overloaded. Rising 1-min vs 15-min = getting worse.
2. On Linux load = runnable processes (**R**) + processes in uninterruptible sleep (**D**, usually IO). So high load does not always mean high CPU.
3. `top` / `vmstat 1`:

- High `us`/`sy` and many in `r` column -> CPU-bound -> find the process (`ps --sort=-%cpu`, `pidstat`).
- High `wa` and many in `b` column / `D` state -> IO-bound -> `iostat -x`, `iotop` (slow disk, EBS limits, NFS hang).

4. Hung NFS mounts can put many processes in `D` state and send load through the roof with idle CPU.
5. Memory pressure -> swapping (`si`/`so` in vmstat) also raises load.
6. Correlate with time (cron, backups, deploys) and fix: scale, tune, move the job, or fix the storage.

Example:

```
uptime; nproc
# load average: 24.10, 18.40, 9.02    (4 cores -> overloaded, getting worse)
vmstat 1 5          # r, b, wa, si/so columns
ps -eo stat,pid,cmd | awk '$1 ~ /^D/'   # stuck in IO wait
iostat -xz 1 3
```

What to say to the interviewer: *I compare the load average with the number of cores and remember Linux load counts both runnable and IO-blocked processes. Then vmstat and top tell me whether it is CPU-bound, IO-bound, a hung NFS mount with processes in D state, or swapping, and I drill down with pidstat, iostat or iotop and correlate it with scheduled jobs.*

Q65. An NFS mount is not working. How do you troubleshoot it?

In simple words: You are trying to use a shared cupboard in another building. The road may be closed (network/ports), the other building may not have given you permission (exports), or your door key type does not match (NFS version).

Detailed answer

1. Error message: `mount.nfs: access denied`, `timed out`, `No such file or directory`, or a hang.
2. Network: `ping`/`nc -zv server 2049` (NFSv4 only needs TCP 2049; v3 also needs 111 rpcbind and mountd ports). On AWS EFS: the mount target's Security Group must allow **2049 from the client**.
3. Server exports: `showmount -e server` (v3) and `/etc/exports` - is the client IP/subnet allowed? Run `exportfs -ra` after changes.

4. Client packages: `nfs-utils` (RHEL) / `nfs-common` (Ubuntu); `amazon-efs-utils` for EFS.
5. Version mismatch: force `-o vers=4.1`.
6. Permission issues after mounting: UID/GID mismatch between client and server, `root_squash`.
7. Hung (stale) mount: `umount -f -l /mnt/nfs`; use `_netdev,nofail` in fstab so boot does not hang.
8. Logs: `dmesg`, `journalctl` on client; `/var/log/messages` on server.

Example:

```
nc -zv 10.0.3.10 2049
showmount -e 10.0.3.10
sudo mount -t nfs4 -o vers=4.1,timeo=600,retrans=2 10.0.3.10:/exports/data /mnt/data
dmesg | tail
# /etc/fstab
10.0.3.10:/exports/data  /mnt/data  nfs4  vers=4.1,_netdev,nofail  0 0
```

> ***What to say to the interviewer:*** *I read the mount error and test port 2049 - on EFS that means the mount target security group - then check the server's exports allow my client and that the client has nfs-utils. After that I look at NFS version mismatches, UID/GID and root_squash permission issues, and use* `_netdev,nofail` *in fstab so a failed share does not block boot.*

Q66. How do you recover a deleted file on Linux?

(also asked as: An important config file was deleted and there is no backup - what do you do?)

> **In simple words:** Linux has no recycle bin. You can grab the file back if someone is still holding it, fetch a copy from the backup cupboard, or try forensic tools that search the floor for the scraps - which get swept away quickly.

Detailed answer

In order of success rate:

1. **Still open by a process?** `lsof +L1 | grep filename` -> copy it back from the file descriptor: `cp /proc/<PID>/fd/<FD> /restore/file`. Works perfectly and instantly.
2. **Backups/snapshots**: EBS snapshot -> create volume -> mount -> copy; AWS Backup; S3 versioning; LVM snapshots; git history for config/code.
3. **Undelete tools** (last resort): `extundelete` / `ext4magic` for ext4, `xfs_undelete` for XFS, `testdisk`/`photorec` for raw carving. Immediately **stop writing** to that filesystem - remount read-only or unmount - because new writes overwrite freed blocks. Recover to a DIFFERENT disk.
4. Prevention: backups with tested restores, `alias rm='rm -i'` for humans, trash tools, and config under version control.

Deleted config file with no backup:

- If the service is still running, its config is often still loaded in memory - do NOT restart it yet. Check `/proc/<PID>/fd` in case the file is open, and dump the running config if the app supports it (`nginx -T`, `haproxy -c`, database `SHOW VARIABLES`).

- Package default: find the owning package (`dpkg -S /etc/nginx/nginx.conf`, `rpm -qf`) and reinstall it - `apt-get install --reinstall nginx` (add `-o Dpkg::Options::="--force-confmiss"` so a missing conffile is recreated) or `dnf reinstall nginx`. `rpm -V nginx` shows which files are missing or changed. Then re-apply your customisations.
- Other copies: the same file on a sibling server, `.bak`/`.rpmsave`/`.dpkg-old` files, the AMI/snapshot, **etckeeper** (git history of /etc).
- Real fix: keep configs in **config management** (Ansible/Git), so re-running the playbook IS the restore.

Example:

```
sudo lsof +L1 | grep report.csv
# python 3310 app 5r REG ... /data/report.csv (deleted)
sudo cp /proc/3310/fd/5 /tmp/report.csv
# Last resort, filesystem unmounted:
sudo umount /data
sudo extundelete /dev/nvme1n1p1 --restore-file data/report.csv --output-dir /recovery
```

> ***What to say to the interviewer:*** *If a process still has it open I copy it back from /proc/PID/fd immediately; otherwise I restore from a snapshot or backup. As a last resort I unmount or remount the filesystem read-only to avoid overwriting blocks and try extundelete or testdisk onto a different disk, and prevention is backups with regularly tested restores.*

Q67. A filesystem went read-only. How do you remount it read-write without a reboot?

> **In simple words:** The notebook locked itself to "view only" because it noticed a torn page. You can unlock it, but you should first check why it locked, or you may tear more pages.

Detailed answer

- The kernel remounts a filesystem read-only when it detects **errors** (`errors=remount-ro` in fstab) - disk/EBS I/O errors or corruption. So first read `dmesg` for EXT4-fs/XFS errors.
- If the cause is resolved (e.g. temporary storage glitch, or it was mounted ro on purpose/in recovery mode):
- `mount -o remount,rw /mountpoint` -> switch back to read-write without unmounting.
- If there are real filesystem errors, remount-rw is risky -> schedule an fsck (see next question) or fail over.
- Also check the disk is not simply full, which gives "No space left" rather than read-only.
- In recovery mode/single-user the root is mounted ro by default -> `mount -o remount,rw /` is the normal first step.

Example:

```
dmesg -T | grep -i -E "ext4-fs error|xfs|remount|i/o error" | tail
mount | grep " / "            # (ro,relatime,...)
sudo mount -o remount,rw /
touch /tmp/rwtest && echo OK
```

What to say to the interviewer: *I use* `mount -o remount,rw /mountpoint`*, but only after checking dmesg for why the kernel made it read-only, because usually it detected I/O errors or corruption. If there are real errors I plan an fsck or failover instead of forcing it writable.*

Q68. How do you fix a corrupted filesystem with fsck?

In simple words: fsck is a doctor that checks the filing system's index. You never operate on a patient who is running around - the filesystem must be unmounted (resting) first.

Detailed answer

- **Never run fsck on a mounted read-write filesystem** - it can cause more corruption.
- Snapshot the volume first (AWS: EBS snapshot) so the repair can be undone.
- Data disk: stop apps, `umount /data`, then `fsck -f /dev/xxx` (ext4 `e2fsck -fy`) or for XFS `xfs_repair /dev/xxx` (XFS does not use fsck; if the log is dirty, mount/unmount first to replay it, use `xfs_repair -L` only as a last resort because it discards the log).
- Root filesystem: boot into recovery/rescue mode, a live USB, or on AWS attach the root volume to a **rescue instance** and repair it there. Or force a check on next boot (`fsck.mode=force` kernel parameter, or `touch /forcefsck` on older systems).
- Check `lost+found` afterwards for recovered fragments.
- Find the cause: failing disk (SMART, EBS status checks), unclean shutdowns.

Example:

```
sudo umount /data
sudo e2fsck -fy /dev/nvme1n1p1     # ext4
sudo xfs_repair /dev/nvme1n1p1     # XFS
sudo mount /data && ls /data/lost+found
```

What to say to the interviewer: *I take a snapshot, unmount the filesystem - or for root use rescue mode or attach the volume to a rescue instance - and run* `e2fsck -f` *for ext4 or* `xfs_repair` *for XFS, never on a mounted filesystem. Afterwards I check lost+found and investigate the underlying disk or shutdown issue.*

Q69. The disk shows free space but you get "No space left on device". Why? (inode exhaustion)

In simple words: A car park with lots of empty space but no parking tickets left. Each file needs a ticket (inode); millions of tiny files use up the tickets before the space.

Detailed answer

- Every file uses one **inode**; ext4 fixes the inode count at format time. Millions of tiny files (sessions, cache, mail queue, temp files, small log fragments, container layers) can exhaust inodes while `df -h` shows free GB.
- Confirm: `df -i` -> `IUse%` at 100%.
- Find the folder with most files: `du --inodes -x / | sort -n | tail` (or `find /var -xdev -type f | cut -d/ -f2-3 | sort | uniq -c | sort -n | tail`).

- Fix: delete the tiny files (e.g. old PHP sessions, cache, `/var/spool`), using `find -delete` - `rm *` fails with "argument list too long".
- Long-term: fix the app's cleanup, add cron cleanup, alert on inode usage, or format data disks with more inodes (`mkfs.ext4 -i 4096`) or use XFS, which allocates inodes dynamically.

Example:

```
df -i /
# Filesystem     Inodes  IUsed IFree IUse% Mounted on
# /dev/nvme0n1p1 1310720 1310720   0  100% /
sudo du --inodes -x / 2>/dev/null | sort -n | tail -5
sudo find /var/app/cache -type f -mtime +2 -delete
```

> ***What to say to the interviewer:*** *That is usually inode exhaustion, which I confirm with* `df -i`*; millions of tiny files use up all inodes while there is still free space. I find the directory with the most files using* `du --inodes`*, clean them up with* `find -delete`*, then fix the app's cleanup, alert on inode usage, or use XFS or more inodes for that disk.*

Q70. Apache is running, but the website is not accessible. How do you troubleshoot it?

> **In simple words:** The shop is open and the lights are on, but customers cannot get in. Maybe the front door is on the wrong street (port/interface), a guard blocks the entrance (firewall/SELinux), the signboard points elsewhere (DNS/vhost), or the shelves are locked (file permissions).

Detailed answer

Work from the inside out:

1. Is it really healthy? `systemctl status httpd` (RHEL) / `apache2` (Ubuntu), and `apachectl configtest` for syntax.
2. Is it listening where you think? `ss -tlnp | grep -E ':80|:443'` -> `127.0.0.1:80` means local only; check `Listen` directives.
3. Test locally: `curl -I http://localhost` -> works locally but not remotely = network/firewall. Fails locally = Apache/app config.
4. Test by external IP from outside, then by domain name -> works by IP but not name = **DNS**.
5. Firewalls: `firewall-cmd --list-all` / `ufw status`, then cloud Security Group/NACL/load balancer target health.
6. **SELinux** (RHEL): `getenforce`; wrong file context on a custom DocumentRoot (`restorecon -Rv`), or `httpd_can_network_connect` off when proxying to a backend.
7. vhost/DocumentRoot: correct `ServerName`, `<Directory>` has `Require all granted`, files readable and parent directories have `x` for the apache user.
8. Logs tell the story: `error_log` (403 permission denied, AH00035) and `access_log` (are requests even arriving?).
9. HTTPS only failing: certificate path, expired cert, port 443 not open.

Example:

```
sudo apachectl configtest              # Syntax OK
sudo ss -tlnp | grep httpd
curl -I http://localhost
sudo firewall-cmd --add-service={http,https} --permanent && sudo firewall-cmd --reload
sudo tail -f /var/log/httpd/error_log
getenforce; sudo restorecon -Rv /var/www/mysite
sudo setsebool -P httpd_can_network_connect on
```

What to say to the interviewer: *I verify the config with* `apachectl configtest`*, confirm with* `ss` *that it listens on the right interface and port, then compare curl on localhost with curl from outside to split an Apache problem from a network one. After that I check firewalld or ufw, security groups, SELinux contexts and booleans, vhost and DocumentRoot permissions, DNS, and the error and access logs.*

Q71. Automate an application deployment with a shell script, with backup, health check and automatic rollback.

In simple words: Like changing a shop's display overnight: keep the old display packed next to you, set up the new one, switch the lights on and check it looks right - if not, put the old one back before customers arrive.

Detailed answer

Design used by real tools (Capistrano-style):

- Each release goes into its own timestamped folder: `/opt/app/releases/20260924_101500`.
- A symlink `/opt/app/current` points to the live release -> switching is **atomic** (`ln -sfn`), and rollback is just pointing back.
- Keep the last N releases for quick rollback; delete older ones.
- Health check in a retry loop after restart; on failure, switch the symlink back and restart.
- `set -euo pipefail` + `trap` so a failed step never leaves things half-done silently.

Example:

```
#!/usr/bin/env bash
# Usage: ./deploy.sh /tmp/app-build.tar.gz
set -euo pipefail
ARTIFACT="${1:?Usage: $0 <artifact.tar.gz>}"
APP=/opt/app; SERVICE=myapp; HEALTH=http://localhost:8080/health; KEEP=5
NEW="$APP/releases/$(date +%Y%m%d_%H%M%S)"
PREV="$(readlink -f "$APP/current" || true)"

log() { echo "$(date '+%F %T') $*"; }
rollback() {
  log "Deployment FAILED - rolling back to $PREV"
  [ -n "$PREV" ] && ln -sfn "$PREV" "$APP/current" && systemctl restart "$SERVICE"
  exit 1
}
trap rollback ERR

[ -f "$ARTIFACT" ] || { log "Artifact not found"; exit 2; }
mkdir -p "$NEW" && tar -xzf "$ARTIFACT" -C "$NEW"
log "Switching current -> $NEW"
ln -sfn "$NEW" "$APP/current"
systemctl restart "$SERVICE"

for i in $(seq 1 10); do
  if curl -fsS --max-time 3 "$HEALTH" >/dev/null; then
    log "Healthy after ${i} checks"; trap - ERR
    ls -1dt "$APP"/releases/* | tail -n +$((KEEP+1)) | xargs -r rm -rf
    exit 0
  fi
  sleep 3
done
false     # health never passed -> triggers ERR trap -> rollback
```

What to say to the interviewer: *My script extracts each build into a timestamped release directory, atomically switches a* `current` *symlink, restarts the service with systemctl and polls the health endpoint in a loop. An ERR trap under* `set -euo pipefail` *points the symlink back to the previous release and restarts it if any step or the health check fails, and old releases are pruned to keep the last five.*

Q72. An application fails intermittently. How do you use Linux command-line tools to analyse its logs?

In simple words: Intermittent failures are like a car that stalls "sometimes". You go through the trip log, count how often and at what times it happened, and look at what else happened at the same minute.

Detailed answer

1. Narrow the time window: `journalctl -u myapp --since "2026-09-24 10:00" --until "10:30"`.
2. Find errors with context: `grep -E "ERROR|Exception|timeout" -C 5 app.log` (lines before/after show the cause).
3. Count and group to find patterns:

- `grep -c` for totals; `sort | uniq -c | sort -rn` for the most common messages.
- `awk` to count HTTP status codes per minute from access logs -> shows spikes.

4. Include rotated logs: `zgrep` on `.gz` files.

5. Watch live while reproducing: `tail -F` (follows through rotation) or `less +F`.
6. **Correlate timestamps** with other sources: `dmesg -T` (OOM kills, disk errors), system journal, cron times, deploy times, database logs.
7. Check if failures cluster on one host, one endpoint or one upstream.

Example:

```
journalctl -u myapp --since "1 hour ago" -p err
grep -E "ERROR|Timeout" -C 3 /var/log/myapp/app.log | less
# Top error messages
grep ERROR app.log | sed 's/^[^]]*] //' | sort | uniq -c | sort -rn | head
# 5xx per minute from nginx access log ($9 = status, $4 = [time)
awk '$9 ~ /^5/ {print substr($4,2,17)}' /var/log/nginx/access.log | uniq -c | sort -rn | head
zgrep -h "Timeout" /var/log/myapp/app.log*.gz | wc -l
dmesg -T | grep -i "killed process"
tail -F /var/log/myapp/app.log | grep --line-buffered ERROR
```

> ***What to say to the interviewer:*** *I narrow the time window with* `journalctl --since`*, grep errors with context lines, and use* `sort | uniq -c` *and awk to count error types and 5xx per minute, which reveals patterns. I include rotated logs with zgrep, follow live with* `tail -F` *while reproducing, and correlate the timestamps with dmesg OOM events, cron and deployments.*

Q73. A critical service fails to start after a reboot. How do you troubleshoot it?

> **In simple words:** After a power cut, one machine will not start. Maybe it was never set to start automatically, it tried to start before the electricity or water it needs was ready, or someone left something blocking it.

Detailed answer

1. Was it enabled? `systemctl is-enabled myapp` -> if `disabled`, it never tried: `systemctl enable myapp`.
2. What failed? `systemctl --failed`, `systemctl status myapp`, and `journalctl -b -u myapp` (this boot only).
3. **Ordering/dependencies**: it started before what it needs. Network apps should use `After=network-online.target` + `Wants=network-online.target`; add `After=`/`Requires=` for databases or mounts (`RequiresMountsFor=/data`).
4. **Mounts not ready**: a data disk or NFS missing from fstab, or it failed -> app cannot find its files. Use `nofail`/`_netdev` so a bad mount does not break boot.
5. **Port already in use**: another service grabbed it (`ss -tlnp`).
6. Permissions / env files: `EnvironmentFile=` missing, `/run` or `/tmp` directories not recreated (use `RuntimeDirectory=`).
7. SELinux denials: `ausearch -m avc -ts boot`.
8. Unit edited but not reloaded -> `systemctl daemon-reload`.
9. Add `Restart=on-failure` so a transient issue heals itself.

Example:

```
systemctl is-enabled myapp
systemctl --failed
journalctl -b -u myapp --no-pager | tail -30
systemctl list-dependencies myapp
sudo systemctl edit myapp
# [Unit]
# After=network-online.target postgresql.service
# Wants=network-online.target
# RequiresMountsFor=/data
sudo systemctl daemon-reload && sudo systemctl restart myapp
```

What to say to the interviewer: *I check whether the unit is enabled, then read* `journalctl -b -u` *and* `systemctl --failed` *for the actual error. The common causes after a reboot are ordering problems, fixed with* `After=network-online.target` *or* `RequiresMountsFor`*, a mount that did not come up, a port already taken, missing environment or runtime files, or SELinux, and I add* `Restart=on-failure` *for resilience.*

Q74. How would you tune the kernel with sysctl for a high-traffic application?

In simple words: The kernel ships with settings for an average house. A busy stadium needs wider gates and a longer waiting area - but you only widen the gates that are actually crowded, and you measure before and after.

Detailed answer

Common knobs for many concurrent connections:

Setting	What it controls	Typical high-traffic value
`net.core.somaxconn`	max accept queue per listening socket	4096-65535 (app backlog must match)
`net.ipv4.tcp_max_syn_backlog`	half-open connection (SYN) queue	8192+
`net.core.netdev_max_backlog`	packets queued from the NIC	16384+
`net.ipv4.ip_local_port_range`	ports for OUTGOING connections (proxies)	`1024 65535`
`net.ipv4.tcp_tw_reuse`	reuse TIME_WAIT sockets for outgoing	1
`fs.file-max` + `LimitNOFILE`	total and per-process open files/sockets	e.g. 2M and 65536
`vm.swappiness`	how eagerly to swap	1-10 for latency-sensitive apps

Rules:

- **Measure first**: `ss -s`, `netstat -s | grep -i -E "overflow|drop"` (listen queue overflows), TIME_WAIT counts, fd usage. Tune only what the data shows is a bottleneck.
- Persist in `/etc/sysctl.d/99-app.conf`, apply with `sysctl --system`; change one thing at a time; load test and compare.
- Avoid cargo-cult tuning copied from blogs: `tcp_tw_recycle` was removed from Linux (it broke clients behind NAT), and huge buffers can increase latency.
- In Kubernetes, many of these are namespaced and set per pod (`securityContext.sysctls`), others only on the node.

Example:

```
ss -s
netstat -s | grep -i "listen"          # "times the listen queue of a socket overflowed"
cat <<'EOF' | sudo tee /etc/sysctl.d/99-webapp.conf
net.core.somaxconn = 8192
net.ipv4.tcp_max_syn_backlog = 8192
net.core.netdev_max_backlog = 16384
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_tw_reuse = 1
vm.swappiness = 10
EOF
sudo sysctl --system
```

> ***What to say to the interviewer:*** *I first measure with* `ss -s` *and* `netstat -s` *for listen queue overflows, TIME_WAIT and file descriptor usage, and only then tune the relevant values such as somaxconn, tcp_max_syn_backlog, ip_local_port_range, tcp_tw_reuse and file limits. I persist them in /etc/sysctl.d, change one thing at a time with load tests before and after, and avoid copying blog settings like the removed tcp_tw_recycle.*

Q75. How do you deploy updates with zero downtime on a plain Linux server (no Kubernetes)?

> **In simple words:** Changing the cashier at a checkout without closing it: the new cashier sits down at the next counter, the manager starts sending new customers there, and the old cashier finishes serving the people already in line before leaving.

Detailed answer

Techniques (often combined):

1. **Two or more instances behind a reverse proxy** (nginx/HAProxy) or a load balancer: take instance A out of rotation (drain), update and health-check it, put it back, then do B - a **rolling restart**.
2. **Connection draining**: let in-flight requests finish before stopping (LB deregistration delay, HAProxy `drain` state, app handling SIGTERM gracefully).
3. **Graceful reload of the proxy**: `nginx -t && nginx -s reload` (or `systemctl reload nginx`) starts new workers with the new config while old workers finish their requests - no dropped connections.
4. **Atomic symlink switch + graceful app reload**: deploy to a new release folder, switch `current` symlink, then graceful reload (many app servers like gunicorn, php-fpm or puma support reloading workers on a signal without closing the listening socket).
5. **Blue-green on one host**: run the new version on another port, health-check it, flip the nginx upstream, reload nginx, keep the old one for instant rollback.
6. **systemd socket activation**: systemd holds the listening socket, so connections queue briefly during a restart instead of being refused.
7. Database changes must be backward compatible (expand/contract) so old and new versions can run together.

Example:

```
upstream app {
    server 127.0.0.1:8081;       # blue (current)
    server 127.0.0.1:8082 down;  # green (new) - flip 'down' after health check
}
```

```
curl -fsS http://127.0.0.1:8082/health && \
  sudo sed -i 's/8081;/8081 down;/; s/8082 down;/8082;/' /etc/nginx/conf.d/app.conf && \
  sudo nginx -t && sudo nginx -s reload
```

> ***What to say to the interviewer:*** *I run at least two app instances behind nginx or HAProxy and do a rolling update: drain one, deploy and health-check it, return it to the pool, then the next, with the proxy itself changed only by graceful reloads. On a single host I use blue-green on two ports or an atomic symlink switch plus the app server's graceful reload, and I keep database changes backward compatible so both versions can run together.*

Q76. Design a 3-tier architecture (web, app, DB), explain the request flow, and debug a failure layer by layer with Linux commands.

> **In simple words:** A restaurant: the waiter at the front (web tier) takes the order, the kitchen (app tier) cooks it, and the storeroom (database) holds the ingredients. When a customer complains, you walk the same path the order took and check each station in turn, instead of guessing.

Detailed answer

Design:

```
 User --DNS--> app.example.com
   |
   v   HTTPS 443
[ Load balancer / nginx (web tier) ]   public subnet, TLS termination, static files
   |            |            (health checks, least_conn)
   v HTTP 8080  v
[ App server 1 ]  [ App server 2 ]     private subnet, stateless, sessions in Redis
   |            |
   v TCP 5432   v
[ DB primary ] ---replication---> [ DB replica ]   private DB subnet, Multi-AZ
```

- Each tier sits in its own subnet with security groups allowing only the tier above it (LB -> app 8080, app -> DB 5432).
- Web tier scales horizontally, app tier is stateless behind the LB, DB has a primary plus a replica for failover and reads.
- Flow: DNS resolves -> TCP + TLS to the LB -> LB proxies to a healthy app server -> app queries the DB -> response travels back the same way.

Debug, layer by layer (what to run and why):

1. **Scope first** - all users or some, all pages or one endpoint, since when, what changed (deploy, config, certificate)?
2. **DNS** - `dig +short app.example.com` -> does the name resolve, to the right IP (stale record, wrong TTL)?
3. **Connectivity** - `nc -zv <lb-ip> 443` and `curl -v https://app.example.com/health` -> is the port reachable, and what status comes back (timeout = network/SG; 502/504 = backend problem)?

4. **TLS** - `openssl s_client -connect app.example.com:443 -servername app.example.com` -> expired certificate, broken chain, wrong hostname?
5. **Web tier** - `nginx -t` (config valid?), `ss -tlnp` (listening on 443?), `tail -f /var/log/nginx/error.log` ("upstream timed out" or "connection refused" tells you the app tier is the problem), `access.log` for status codes and `$upstream_response_time`.
6. **App tier** - `systemctl status myapp` and `journalctl -u myapp -f` (crashing, exceptions?), `curl localhost:8080/health` on the box (bypasses the LB), `top` (CPU/memory saturated?), `lsof -p <PID> | wc -l` (fd leak), `strace -p <PID>` (stuck waiting on a socket = waiting on the DB?), `ss -tnp | grep 5432` (connection pool full?).
7. **DB tier** - `nc -zv db 5432`; connections vs `max_connections` (`SELECT count(*) FROM pg_stat_activity;`), long or blocked queries (`pg_stat_activity`, slow query log), locks, replication lag; on the host `iostat -x 1` (disk saturated?), `df -h` (disk full stops writes).
8. **Network in between** - `mtr` (loss or latency per hop), `ss -s` and `netstat -s | grep -i retrans` (retransmits), `tcpdump -i eth0 port 5432` (are packets leaving and replies coming back?), MTU (`ping -M do -s 1472 <ip>` - big packets hang while small ones work), security groups/NACLs.
9. **Fix and verify** - roll back the change or scale/restart the failing tier, confirm with the same checks and dashboards, then write the RCA.

Example:

```
dig +short app.example.com
curl -sv -o /dev/null -w "%{http_code} %{time_total}\n" https://app.example.com/health
openssl s_client -connect app.example.com:443 -servername app.example.com </dev/null 2>/dev/null \
  | openssl x509 -noout -dates
sudo nginx -t && sudo tail -20 /var/log/nginx/error.log
# upstream timed out (110) while reading response header from upstream: 10.0.2.11:8080
ssh app1 'curl -s localhost:8080/health; journalctl -u myapp -n 30 --no-pager'
ssh app1 'ss -tnp | grep -c ":5432"'               # 100 = pool exhausted?
psql -h db -c "SELECT state, count(*) FROM pg_stat_activity GROUP BY state;"
mtr -rwc 30 10.0.3.10; netstat -s | grep -i retrans
```

What to say to the interviewer: *I design it as a load balancer or nginx web tier in a public subnet, stateless app servers in a private subnet, and a primary database with a replica in a DB subnet, each tier only reachable from the one above. To debug I follow the request path: DNS with dig, reachability and status with curl and nc, TLS with openssl, then the nginx error log to see whether upstream failed, the app's journal, resources and strace, the database's connections, slow queries and disk, and finally mtr, retransmits, tcpdump and MTU for the network in between.*

Q77. You are locked out of a server over SSH and have no root access. How do you recover?

In simple words: You lost the key to your flat. Instead of breaking the door, you use the building manager's master access (the cloud console), or you take the lock out, fix it on a workbench, and put it back.

Detailed answer

On AWS / cloud (in order of convenience):

1. **SSM Session Manager** -> if the SSM agent and instance role are there, you get a shell with no SSH at all: `aws ssm start-session --target i-0abc`.
2. **EC2 Instance Connect** or the **EC2 Serial Console** (Nitro instances) -> console access even when networking or sshd is broken.
3. **Rescue-volume method:** stop the instance -> detach the root EBS volume -> attach it to a helper instance in the same AZ -> mount it -> fix `home/ec2-user/.ssh/authorized_keys`, `etc/ssh/sshd_config` or `etc/sudoers.d/` -> unmount -> reattach as the root device -> start.
4. **User data:** on some AMIs, cloud-init user data set to run on every boot can re-inject a public key.

On-prem / VM: use the hypervisor or iLO/iDRAC console, boot to single-user mode (`systemd.unit=rescue.target`, or `rd.break` on RHEL to reset the root password), then fix the problem.

Afterwards: find out why it happened (a bad sshd_config, full disk, changed key, fail2ban ban, SG change), and enable SSM so it never needs this again.

Example:

```
# Rescue method on a helper instance (root volume attached as /dev/xvdf)
sudo lsblk
sudo mkdir -p /mnt/rescue && sudo mount /dev/xvdf1 /mnt/rescue
cat ~/new_key.pub | sudo tee -a /mnt/rescue/home/ec2-user/.ssh/authorized_keys
sudo chroot /mnt/rescue sshd -t -f /etc/ssh/sshd_config   # validate config
sudo umount /mnt/rescue
```

> ***What to say to the interviewer:*** *On AWS I first try SSM Session Manager or the EC2 serial console, which don't depend on SSH. If neither is available, I stop the instance, attach its root volume to a helper instance, fix authorized_keys, sshd_config or sudoers, and reattach it. On-prem I use the out-of-band console and rescue mode. Then I fix the root cause and make sure SSM is enabled everywhere.*

Q78. Write a shell script that checks whether a service is running, restarts it if not, and logs the event.

> **In simple words:** A night watchman who checks that the lights are on. If they're off, he flips the switch, writes it in the logbook, and calls someone if the switch won't work.

Detailed answer

- Use `systemctl is-active --quiet` to check each service.
- Restart, then **verify** that the restart actually worked; don't assume it did.
- Log to syslog/journald with `logger`, so logs are centralized and rotated.
- Return a non-zero exit code if anything is still down, so cron or monitoring can alert.
- Note: `Restart=on-failure` in the systemd unit usually makes this script unnecessary. Use the script for extra checks or alerting.

Example:

```
#!/usr/bin/env bash
# svc_guard.sh - restart services that are down, log it, and report failures
set -uo pipefail
SERVICES=("${@:-nginx}")
failed=0

for svc in "${SERVICES[@]}"; do
  if systemctl is-active --quiet "$svc"; then
    continue
  fi
  logger -t svc_guard -p user.warning "$svc is down - restarting"
  if systemctl restart "$svc" && sleep 3 && systemctl is-active --quiet "$svc"; then
    logger -t svc_guard "$svc restarted successfully"
  else
    logger -t svc_guard -p user.err "$svc FAILED to restart: $(systemctl is-failed "$svc")"
    failed=1
  fi
done
exit "$failed"
```

```
# run every 2 minutes; view the history with: journalctl -t svc_guard
*/2 * * * * root /opt/scripts/svc_guard.sh nginx docker
```

```
# Usually the better fix - let systemd do it:
# /etc/systemd/system/nginx.service.d/restart.conf
[Service]
Restart=on-failure
RestartSec=5
```

> ***What to say to the interviewer:*** *The script loops over the services, uses* `systemctl is-active` *to check each one, restarts any that are down, verifies the restart worked, logs everything with logger into journald, and exits non-zero if a service is still down so monitoring alerts. I'd point out that* `Restart=on-failure` *in the systemd unit handles most of this natively, so the script is mainly for alerting.*

Q79. Networking: a remote user cannot connect to the company VPN. How do you troubleshoot?

> **In simple words:** Check the problem from the user's side outward: is their internet working, can they reach the VPN gate, does the gate recognise their ID, and once inside, do they get a map (routes and DNS)?

Detailed answer

1. **Scope:** is it only this user or everyone? If everyone, look at the VPN gateway itself (service down, certificate expired, licence limit).
2. **User's internet:** can they browse? Hotel or corporate Wi-Fi often blocks UDP 500/4500 (IPsec) or 1194 (OpenVPN), so try a TCP/443 profile.
3. **Reachability:** `nslookup vpn.company.com`, `nc -vz vpn.company.com 443`, or check the UDP ports.
4. **Authentication:** wrong password, expired account or certificate, MFA problem, clock skew (certificate and TOTP validity). Check the VPN server and IdP logs for the user.

5. **Connected but nothing works:** no route pushed (split tunnel), overlapping subnets (the home LAN is 192.168.1.0/24, the same as the office), DNS not pushed, MTU issues (large packets drop, so lower the MTU or set MSS clamping).
6. **AWS Client VPN / Site-to-Site:** authorization rules, route table entries for the target subnet, the SG on the endpoint's association, tunnel status in CloudWatch.

Example:

```
nslookup vpn.company.com
nc -vz vpn.company.com 443
# after connecting:
ip route | grep tun0          # were routes pushed?
ping -M do -s 1372 10.10.0.10 # MTU test; lower the size if it fails
resolvectl status | grep -A2 tun0
```

> ***What to say to the interviewer:*** *I first check whether it's one user or everyone. Then I check the user's own internet and whether VPN ports are blocked on their network, then authentication (certificate expiry, MFA, clock skew) in the server logs. If they connect but can't reach anything, I look at the pushed routes, overlapping home and office subnets, DNS and MTU. On AWS Client VPN I also check the authorization rules and route table.*

Q80. Networking: users report slow access between two branch offices over a WAN link (or a remote office cannot reach the main server). How do you troubleshoot?

Also asked as: A user in a remote office cannot access the company's main server. How would you diagnose it?

> **In simple words:** Traffic between offices travels on a motorway you share with everyone. It can be slow because the road is full (bandwidth), too long (latency), full of potholes (packet loss), or the trucks are too tall for the bridges (MTU).

Detailed answer

1. **Scope:** one user, one app, or the whole office? All the time or only at peak hours?
2. **Can't reach at all:** test step by step: local gateway -> WAN router -> remote side (`traceroute`/`mtr`). Check that the VPN/MPLS tunnel is up, the routes for the server subnet exist on both ends, firewall/ACL/SG rules, and DNS (does the name resolve to the internal IP?).
3. **Slow:** measure latency and loss with `mtr` over several minutes. Measure throughput with `iperf3` against link capacity, and look at interface utilization and errors (SNMP/NetFlow).
4. **Common causes:** a saturated link (backups or updates at the wrong time), no QoS for critical traffic, packet loss on the ISP, MTU/fragmentation over the tunnel, a chatty application (SMB/database) that suffers from high round-trip time, duplex mismatch or interface errors.
5. **Fixes:** QoS, schedule bulk transfers off-peak, increase bandwidth or add a second link, WAN optimisation or caching, MSS clamping on the tunnel, and move chatty apps closer to users. In the cloud, consider Direct Connect instead of internet VPN.

Example:

```
mtr -rwc 100 10.20.0.15           # latency and loss per hop over 100 probes
iperf3 -c 10.20.0.15 -t 30        # throughput (iperf3 -s running on the far side)
ping -M do -s 1400 10.20.0.15     # MTU through the tunnel
ip -s link show eth0              # errors and drops on the interface
```

What to say to the interviewer: *I scope it first: which users, which apps, and when. If it's unreachable, I trace hop by hop and check the tunnel, routes on both sides, firewall rules and DNS. If it's slow, I use mtr for latency and loss, iperf3 for throughput against the link's capacity, and interface counters for errors. The usual culprits are a saturated link, no QoS, ISP packet loss or MTU problems over the tunnel.*

Q81. Networking: the network has intermittent connectivity issues. What could cause it, and how do you troubleshoot?

In simple words: A light that flickers is harder to fix than one that's off, because you have to catch it in the act. So you set up continuous recording and look for a pattern in time, place or load.

Detailed answer

Likely sources:

- Physical: a bad cable or SFP, a flapping interface, Wi-Fi interference, a duplex mismatch.
- Capacity: link saturation at peaks, NAT port exhaustion (e.g. NAT Gateway `ErrorPortAllocation`), conntrack table full (`nf_conntrack: table full` in dmesg).
- Addressing: duplicate IP / ARP conflicts, DHCP lease problems.
- Routing: a route flapping between two paths, BGP session resets, asymmetric routing through a stateful firewall.
- DNS: one of several resolvers failing, so only some lookups fail.
- Cloud: instance bandwidth or PPS limits, health checks toggling targets, spot interruptions.

Approach:

1. Get timestamps and a pattern: which hosts, when, and which destinations.
2. Run continuous tests (`mtr`, `ping -D` with timestamps, synthetic checks) and correlate with metrics: interface errors, drops, CPU on network devices, NAT/conntrack counters.
3. Capture during an event (`tcpdump` on both ends) to see retransmits, resets or ARP storms.
4. Change one thing at a time and confirm the pattern disappears.

Example:

```
ping -D -i 0.5 10.0.1.20 | tee ping.log         # timestamped, catch the drop window
ip -s link show eth0                            # rx/tx errors, dropped
dmesg -T | grep -Ei 'conntrack|link is down|duplex'
arping -D -I eth0 10.0.1.20                     # duplicate IP detection
ethtool -S eth0 | grep -i allowance             # AWS ENA bandwidth/PPS limits hit
```

What to say to the interviewer: *Intermittent problems need data over time, so I collect timestamps and run continuous pings or mtr to find a pattern. Then I correlate with interface errors, NAT and conntrack exhaustion, ARP or duplicate-IP conflicts, route flaps, DNS resolvers and cloud bandwidth limits, and capture packets during an event. Once I have a hypothesis, I change one thing at a time and confirm the drops stop.*

Q82. Networking: traffic is not being forwarded between two subnets. How do you troubleshoot the routing?

In simple words: Two neighbourhoods and one junction between them. Each house must know "for the other neighbourhood, go via the junction", the junction must be willing to pass traffic, and the reply must be able to come back the same way.

Detailed answer

1. **Hosts:** is the default gateway correct on both sides (`ip route`)? Is the subnet mask right? A wrong mask makes a host think the other subnet is local.
2. **Router or Linux box:** does the routing table have both subnets (connected or static)? On Linux, is IP forwarding on (`sysctl net.ipv4.ip_forward=1`)?
3. **Return path:** traffic may go out fine but the reply has no route back. Asymmetric routing through a firewall also drops it.
4. **Filters:** ACLs on router interfaces, the host firewall, and in AWS the NACLs (stateless, so both directions and the ephemeral ports 1024-65535 must be allowed) and Security Groups.
5. **AWS specifics:** subnets in the same VPC route via the `local` route automatically, so if they can't talk it's the SG or NACL. Between VPCs, check the peering/TGW routes in BOTH route tables, TGW route table associations, and that the CIDRs don't overlap. For an EC2 acting as a router or NAT, disable source/destination check.
6. **Verify:** `traceroute` shows where packets stop; `tcpdump` on the router shows whether packets arrive and leave.

Example:

```
ip route get 10.0.2.25                # which route/interface would be used
sysctl net.ipv4.ip_forward            # must be 1 on a Linux router
sudo tcpdump -ni eth1 host 10.0.2.25  # does traffic leave the router?
aws ec2 describe-route-tables --filters Name=association.subnet-id,Values=subnet-0abc
```

What to say to the interviewer: *I check the hosts' gateway and subnet mask, then the router's routing table and IP forwarding, then the return path, because routing failures are often one-way. Then I check ACLs and firewalls. In AWS, subnets in one VPC always have a local route, so it's usually NACLs or Security Groups, while between VPCs I check both route tables and CIDR overlap. traceroute and tcpdump show exactly where packets stop.*

Q83. Networking: you need to add a new subnet to an existing network. What are the steps?

In simple words: Adding a new street to a town: pick house numbers that nobody else is using, connect the street to the road map, and put up the right gates.

Detailed answer

1. **Plan the CIDR:** pick a free range inside the VPC or network that doesn't overlap existing subnets, on-prem ranges, peered VPCs or VPN clients. Size it for growth: a /24 gives 256 addresses, and AWS reserves 5 per subnet, so 251 are usable. EKS pods using VPC CNI need large subnets.

2. **Subnetting math:** 10.0.0.0/16 split into /20s gives 16 subnets of 4096 addresses: 10.0.0.0/20, 10.0.16.0/20, 10.0.32.0/20, and so on.
3. **Create it** in the right AZ (AWS) or VLAN/interface (on-prem). A VLAN is layer 2 separation; a subnet is the layer 3 range you put on it.
4. **Routing:** associate the right route table: public (0.0.0.0/0 -> IGW) or private (0.0.0.0/0 -> NAT). Add routes on peering, TGW, VPN or on-prem routers so other networks can reach it.
5. **Security:** NACLs and SGs, and update firewall rules and allow-lists that reference CIDRs.
6. **Services:** DHCP scope or auto-assign public IP setting, DNS, VPC endpoints, tags. Then do it all in Terraform, and test connectivity from both directions.

Example:

```
# Quick subnet math
ipcalc 10.0.48.0/20        # Network, Broadcast, HostMin/HostMax, Hosts/Net: 4094

aws ec2 create-subnet --vpc-id vpc-0abc --cidr-block 10.0.48.0/20 \
  --availability-zone ap-south-1c \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=private-1c}]'
aws ec2 associate-route-table --route-table-id rtb-0priv --subnet-id subnet-0new
```

> ***What to say to the interviewer:*** *I pick a non-overlapping CIDR sized for growth, remembering AWS reserves five addresses per subnet. I create the subnet in the right AZ, associate the public or private route table, and add routes on peering, TGW or VPN so other networks can reach it. Then I update NACLs, Security Groups and allow-lists, do it all in Terraform, and test from both directions.*

Q84. Networking: you notice abnormal traffic patterns on an interface. How do you investigate and what do you recommend?

In simple words: The water meter suddenly spins much faster than usual. You find out which tap is running, whether it's a guest having a long shower (legitimate spike) or a burst pipe (attack or leak).

Detailed answer

1. **Quantify:** the baseline vs now, inbound vs outbound, bytes vs packets (a high PPS with small packets suggests a DDoS or scan).
2. **Top talkers:** which source/destination IPs and ports? Use NetFlow/sFlow, **VPC Flow Logs** (query with Athena or CloudWatch Logs Insights), `iftop` or `nethogs` on the host.
3. **Classify:** legitimate (a deploy, backup, new customer, crawler), misconfiguration (a retry storm, a logging loop, traffic crossing AZs or going through NAT), or malicious (DDoS, port scans, crypto-mining, data exfiltration to unknown IPs).
4. **Deep dive:** `tcpdump` a sample, check GuardDuty findings, and check the process on the host that opened those connections.
5. **Respond:** block at the right layer (NACL/WAF rate rule/Shield for DDoS, SG for a host), isolate a compromised host, rotate its credentials, and fix the misconfiguration.
6. **Recommend:** baselines and anomaly alerts on flow metrics, egress filtering, WAF rate limiting, and GuardDuty/IDS.

Example:

```
-- Athena on VPC Flow Logs: top talkers in the last hour
SELECT srcaddr, dstaddr, dstport, SUM(bytes)/1048576 AS mb
FROM vpc_flow_logs
WHERE start > to_unixtime(now() - interval '1' hour)
GROUP BY srcaddr, dstaddr, dstport
ORDER BY mb DESC LIMIT 20;
```

```
sudo iftop -i eth0 -P           # live per-connection bandwidth
sudo nethogs eth0               # which process is using it
sudo tcpdump -ni eth0 -c 200 'not port 22'
```

> ***What to say to the interviewer:*** *I compare against the baseline, then find the top talkers with flow logs or NetFlow and iftop on the host, and classify it: a legitimate spike, a misconfiguration like a retry loop, or malicious activity like a DDoS or exfiltration. I confirm with tcpdump and GuardDuty, block or isolate at the right layer, and recommend baselines with anomaly alerts, egress filtering and WAF rate limits.*

Q85. Networking: how do you implement redundancy for critical network devices and links?

> **In simple words:** Never have only one bridge into town. Build two, on different routes, and have a traffic officer who instantly sends cars to the second bridge if the first one closes.

Detailed answer

Options (on-prem):

- **Redundant devices:** two routers/firewalls sharing a virtual gateway IP with **VRRP/HSRP** (active/standby), or firewall HA pairs.
- **Redundant links:** dual ISPs, link aggregation (LACP) for switch uplinks, and diverse physical paths.
- **Dynamic routing** (BGP/OSPF) so traffic reroutes automatically when a path fails.
- **Power and hardware:** dual PSUs, separate power feeds, spares on site.

Options (cloud/hybrid):

- Spread across **multiple AZs**: NAT Gateway per AZ, load balancers across AZs, subnets in each AZ.
- **Site-to-Site VPN** has two tunnels per connection, so configure both. Use two customer gateways for device redundancy.
- **Direct Connect** with a second DX at a different location, or DX + VPN as backup, with BGP deciding the path.
- Route 53 health-check failover for public endpoints.

Recommendation: active/standby pairs with VRRP for gateways, two diverse links with BGP failover, and in AWS multi-AZ everything plus both VPN tunnels. Test failover regularly, because untested redundancy often fails when you need it.

Example:

```
           ISP-A ----\                      /---- AZ-a: NAT GW, app, DB primary
 Office  [FW1]==VRRP==[FW2] ==BGP== AWS TGW ==
           ISP-B ----/  (2 VPN tunnels + DX)  \---- AZ-b: NAT GW, app, DB standby
```

What to say to the interviewer: *I remove single points of failure at each layer: device pairs with VRRP or HSRP, two diverse links or ISPs with BGP for automatic failover, and dual power. In AWS that means multi-AZ subnets and a NAT gateway per AZ, both VPN tunnels up, and Direct Connect backed by a VPN. Most importantly I test failover regularly.*

Q86. Networking: a router is not learning routes from its neighbours (BGP/OSPF). How do you troubleshoot?

In simple words: Two neighbours only share gossip (routes) once they have properly introduced themselves. First check that they're actually talking, then that they agreed on the rules, then that nobody is filtering what's being said.

Detailed answer

1. **Is the neighbour relationship up?** BGP: the state should be `Established`; if it's stuck in `Idle`/`Active`/`Connect`, it's a connectivity or config issue. OSPF: the state should be `Full`.
2. **Connectivity:** can the routers ping each other on the peering IPs? Is TCP 179 (BGP) allowed through ACLs and firewalls? Is OSPF (IP protocol 89) or multicast allowed?
3. **Parameters must match:** BGP neighbour IP and **ASN**, MD5 password, eBGP multihop/TTL. OSPF area ID, hello/dead timers, subnet mask, authentication, MTU (mismatched MTU gets stuck in ExStart), network type.
4. **Established but no routes:** inbound/outbound **filters** (prefix-lists, route-maps), prefixes not advertised (a missing `network` statement or redistribution), max-prefix limit hit, next-hop unreachable, or a better route already in the table (administrative distance).
5. **AWS angle:** for Site-to-Site VPN and Direct Connect, check the tunnel/BGP status in the console or CloudWatch (`TunnelState`), the ASN on the customer gateway vs the VGW/TGW, the advertised prefixes (limit of 100 for VPN), and route propagation enabled on the VPC route tables.

Example:

```
router# show ip bgp summary          # neighbour state, prefixes received
router# show ip bgp neighbors 169.254.10.1 advertised-routes
router# show ip ospf neighbor        # FULL / EXSTART / INIT
router# show ip route bgp
```

```
aws ec2 describe-vpn-connections --query 'VpnConnections[].VgwTelemetry'
aws ec2 describe-route-tables --query 'RouteTables[].PropagatingVgws'
```

What to say to the interviewer: *First I check whether the adjacency is up: BGP Established or OSPF Full. If not, I check connectivity, TCP 179 or OSPF traffic through firewalls, and matching parameters like ASN, passwords, areas, timers and MTU. If the session is up but no routes arrive, it's usually filters, prefixes not advertised, or max-prefix limits. For AWS VPN or Direct Connect, I also check the tunnel telemetry, the ASNs and route propagation on the VPC route tables.*

Docker & Containers

Basic

Q1. What is a Docker image and what is a Docker container?

> **In simple words:** An image is like a recipe card or a frozen ready-meal packet. A container is the actual meal you heat up and eat. One packet design, many meals.

Detailed answer

- A **Docker image** is a read-only, layered package that holds everything an app needs: OS base files, runtime, libraries, your code and the default start command. It is built from a Dockerfile and stored in a registry (Docker Hub, ECR).
- A **Docker container** is a running (or stopped) instance of that image. Docker adds a thin writable layer on top of the image, gives it its own process space, network and filesystem view.
- You can start 10 containers from 1 image. Deleting a container does not delete the image.
- Anything written inside the container's writable layer is lost when the container is removed, unless you use a volume.

Example:

```
docker pull nginx:1.27            # download the image
docker images                     # list images
docker run -d --name web1 -p 8080:80 nginx:1.27   # container 1
docker run -d --name web2 -p 8081:80 nginx:1.27   # container 2, same image
docker ps                         # list running containers
```

> ***What to say to the interviewer:*** *An image is the immutable, versioned blueprint built from a Dockerfile; a container is a running instance of that image with its own writable layer, network and processes. One image can run as many containers, and removing a container never touches the image.*

Q2. How do containerisation technologies like Docker and Kubernetes simplify application deployment and management?

Also asked as: What is the difference between Docker and Kubernetes, and how do they complement each other in production?

> **In simple words:** Shipping containers made global trade easy because every crane and ship handles the same box shape. Docker puts every app in the same "box", and Kubernetes is the port that stacks, moves and replaces those boxes automatically.

Detailed answer

Docker solves "it works on my machine":

- App + dependencies are packed into one image, so dev, test and prod run exactly the same thing.
- Starts in seconds (no full OS boot like a VM), uses fewer resources.
- Versioned images (`app:1.4.2`) make rollback as simple as running the old tag.

Kubernetes solves "running many containers on many servers":

- Scheduling: decides which node each container runs on.
- Self-healing: restarts crashed containers, replaces pods on failed nodes.
- Scaling: HPA adds or removes pods based on load.
- Rolling updates and rollbacks with zero downtime.
- Service discovery and load balancing via Services and Ingress.

Docker vs Kubernetes - not competitors, they complement each other:

Point	Docker	Kubernetes
Job	Build images and run containers	Orchestrate containers across many nodes
Scope	One host	A cluster of many nodes
Failure handling	Restart policy on that host only	Reschedules pods to healthy nodes
Scaling	Manual (`docker run` more copies)	HPA / Cluster Autoscaler, declarative replicas
Networking	Bridge networks, `-p` port mapping	Services, DNS, Ingress, NetworkPolicy
Updates	Stop old, start new yourself	Rolling updates and rollback built in

Runtime detail interviewers like: Kubernetes no longer talks to the Docker daemon - the dockershim was removed in Kubernetes 1.24, so nodes use a CRI runtime such as **containerd** or **CRI-O**. Docker-built images are standard **OCI images**, so they still run unchanged. In practice: Docker (or buildx) builds and pushes the image in CI, Kubernetes runs it in production.

Example:

```
# Docker: package once
docker build -t myorg/orders:1.4.2 .
docker push myorg/orders:1.4.2

# Kubernetes: run 3 copies, update, roll back
kubectl create deployment orders --image=myorg/orders:1.4.2 --replicas=3
kubectl set image deployment/orders orders=myorg/orders:1.4.3
kubectl rollout undo deployment/orders
```

> ***What to say to the interviewer:*** *Docker gives a consistent, portable, versioned package so the same artifact runs everywhere. Kubernetes then orchestrates those containers at scale - scheduling, self-healing, autoscaling, rolling updates and service discovery - so we manage desired state instead of individual servers. Since Kubernetes 1.24 nodes run containers through containerd or CRI-O rather than the Docker daemon, but Docker-built OCI images run unchanged, so Docker builds in CI and Kubernetes runs in production.*

Q3. What is a Dockerfile? Explain its purpose and structure, and how you write one.

Also asked as: What is the structure of your Dockerfile? How do you create a Dockerfile?

In simple words: A Dockerfile is a step-by-step instruction sheet: "start with this base, copy these files, install these things, and when switched on, run this." Docker follows it top to bottom to cook the

image.

Detailed answer

A Dockerfile is a plain text file of instructions. Each instruction creates a layer, and layers are cached, so order matters: put things that change rarely at the top and your source code near the bottom.

Common instructions, in typical order:

- `FROM` - base image (always first).
- `ARG` / `ENV` - build-time and runtime variables.
- `WORKDIR` - working folder inside the image.
- `COPY` - copy dependency files first, then source code.
- `RUN` - run commands at build time (install packages).
- `EXPOSE` - documents the port the app listens on.
- `USER` - switch to a non-root user.
- `HEALTHCHECK` - optional health command.
- `ENTRYPOINT` / `CMD` - what runs when the container starts.

Example:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

```
docker build -t myapp:1.0 .
docker run -d -p 3000:3000 myapp:1.0
```

What to say to the interviewer: *A Dockerfile is the build recipe for an image - base image, working directory, dependency install, code copy, exposed port, user and start command. I order it so rarely changing layers like dependency installs come before the source copy, which keeps builds fast through layer caching.*

Q4. What is WORKDIR in a Dockerfile? What does it do?

In simple words: It is like doing "cd into this folder" before you start working. Every later step happens inside that folder.

Detailed answer

- `WORKDIR /app` sets the current directory for all following `RUN`, `CMD`, `ENTRYPOINT`, `COPY` and `ADD` instructions.
- If the folder does not exist, Docker creates it.
- It is also the default directory when the container starts (and when you `docker exec` into it).

- Prefer `WORKDIR` over `RUN cd /app && ...`, because `cd` in one `RUN` does not carry over to the next `RUN` - each RUN is a fresh shell.
- You can use it multiple times; relative paths build on the previous one.

Example:

```
FROM alpine:3.20
WORKDIR /app          # creates /app and moves into it
COPY . .              # copies into /app
RUN pwd               # prints /app
WORKDIR logs          # now /app/logs
CMD ["sh", "-c", "pwd"]   # container prints /app/logs
```

> ***What to say to the interviewer:*** *WORKDIR sets the working directory for every later instruction and for the running container, creating it if needed. I use it instead of RUN cd because each RUN starts a new shell, so a cd would not persist.*

Q5. How do you define a startup command in Docker? What is the difference between CMD and ENTRYPOINT?

> **In simple words:** ENTRYPOINT is the fixed job of the machine (a coffee machine always makes coffee). CMD is the default setting (medium size) that the user can easily change when pressing the button.

Detailed answer

Both define what runs when the container starts.

Point	CMD	ENTRYPOINT
Purpose	Default command or default arguments	Main executable that always runs
Override at `docker run`	Replaced by any args after image name	Only with `--entrypoint` flag
Used together	Provides default args to ENTRYPOINT	Receives CMD as its arguments
Typical use	Simple apps, easy to override	Wrapper scripts, CLI-style images

- Always use the **exec form** (JSON array) `["node","server.js"]` so your app runs as PID 1 and receives stop signals (SIGTERM). The shell form `CMD node server.js` runs under `/bin/sh -c` and may not get signals, causing slow, ungraceful shutdowns.
- Only the last CMD / ENTRYPOINT in a Dockerfile counts.

Example:

```
FROM alpine:3.20
ENTRYPOINT ["ping"]
CMD ["-c", "3", "google.com"]
```

```
docker run myping                # runs: ping -c 3 google.com
docker run myping -c 1 8.8.8.8   # runs: ping -c 1 8.8.8.8  (CMD replaced)
docker run --entrypoint sh -it myping   # ENTRYPOINT overridden
```

A real-world pair from the official nginx style - the binary is fixed, the flags are the default:

```
FROM nginx:1.27-alpine
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
# docker run mynginx -t        -> runs "nginx -t" (config test) instead of the server
```

What to say to the interviewer: *ENTRYPOINT defines the fixed executable and CMD supplies default arguments that users can override at docker run. I combine them - ENTRYPOINT for the binary or an entrypoint script, CMD for default flags - and always use the exec form so the process gets signals as PID 1.*

Q6. In a Dockerfile, what is the practical difference between COPY and ADD? When would you prefer one over the other?

In simple words: COPY is a plain photocopy. ADD is a photocopy machine with extra tricks - it can unzip boxes and fetch things from the internet. Extra tricks mean surprises, so use the plain one unless you need the trick.

Detailed answer

Point	COPY	ADD
Copy local files/folders	Yes	Yes
Auto-extract local tar (.tar, .tar.gz)	No	Yes
Download from URL	No	Yes (not auto-extracted)
Predictability	High	Lower (magic behaviour)
Best practice	Default choice	Only for local tar extraction

- Prefer **COPY** - clear and predictable, recommended by Docker.
- Use **ADD** only when you want a local tarball unpacked into the image.
- Instead of `ADD <url>`, use `RUN curl ... && verify checksum && extract && cleanup` in one layer, so the downloaded archive does not bloat the image and you can verify it.

Example:

```
COPY package.json ./                # simple copy
COPY --chown=node:node src/ ./src/   # copy with ownership
ADD rootfs.tar.gz /                 # auto-extracts into /
RUN curl -fsSL -o /tmp/tool.tgz https://example.com/tool.tgz \
 && tar -xzf /tmp/tool.tgz -C /usr/local/bin \
 && rm /tmp/tool.tgz
```

What to say to the interviewer: *COPY just copies local files; ADD also extracts local tar archives and can fetch URLs. I use COPY by default and ADD only for local tar extraction; for remote files I use curl in a RUN step so I can verify and clean up in the same layer.*

Q7. How does Docker networking work?

In simple words: Docker builds a small private office network on your server. Containers on the same office network can call each other by name; to let outsiders in, you open a door (publish a port) on the building.

Detailed answer

Docker creates virtual networks using Linux network namespaces, virtual ethernet pairs (veth) and iptables rules.

Network drivers:

- **bridge** (default) - private network on the host (`docker0`, 172.17.0.0/16). Containers get private IPs; outbound traffic is NATed through the host.
- **user-defined bridge** - same as bridge but with built-in DNS, so containers reach each other by container name. Recommended for multi-container apps.
- **host** - container shares the host network stack directly; no isolation, no port mapping, fastest.
- **none** - no networking.
- **overlay** - spans multiple hosts (Docker Swarm).
- **macvlan** - container gets its own MAC/IP on the physical LAN.

Port publishing `-p 8080:80` adds a NAT rule: host port 8080 -> container port 80.

Example:

```
docker network create appnet
docker run -d --name db  --network appnet -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --name api --network appnet -p 8080:3000 myorg/api:1.0
# inside api, the DB is reachable as host "db" on port 5432
docker exec api getent hosts db
docker network inspect appnet
```

> ***What to say to the interviewer:*** *By default containers join a bridge network with private IPs and NAT to the outside; publishing a port with -p maps a host port to the container. For multi-container apps I use a user-defined bridge so containers find each other by name through Docker's embedded DNS, and there are host, none, overlay and macvlan drivers for other cases.*

Q8. What is a multi-stage Docker build, why is it used, and what is its advantage over a single-stage build?

> **In simple words:** You build furniture in a workshop full of saws and glue, but you only deliver the finished table to the customer - not the whole workshop. Multi-stage builds ship only the finished product.

Detailed answer

A multi-stage Dockerfile has more than one `FROM`. Early stages have compilers, build tools and dev dependencies; the final stage copies only the built output with `COPY --from=<stage>`.

Advantages over single-stage:

- **Much smaller images** (for example 1.1 GB -> 150 MB), so faster pulls, faster scaling, less registry cost.
- **Better security** - no compilers, package managers, source code or build secrets in the final image, so fewer CVEs and smaller attack surface.
- **One Dockerfile** replaces separate "build" and "runtime" Dockerfiles or shell scripts.
- **Better caching** - stages build in parallel with BuildKit and unused stages are skipped.
- Can target a stage for testing: `docker build --target test .`

Example:

```
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /app

FROM gcr.io/distroless/static-debian12
COPY --from=build /app /app
ENTRYPOINT ["/app"]
# final image ~10 MB instead of ~900 MB
```

> ***What to say to the interviewer:*** *A multi-stage build uses several FROM stages - a heavy build stage and a minimal runtime stage - and copies only the compiled artifact forward. That gives much smaller, more secure images with no build tools or source inside, all from a single Dockerfile.*

Q9. What are the stages or steps of a Docker image build? Explain build context, layers and cache.

> **In simple words:** You hand the chef a box of ingredients (build context), the chef follows the recipe line by line (instructions), puts each finished step on its own tray (layer), and remembers trays already made so it does not redo them next time (cache).

Detailed answer

What happens when you run `docker build -t app:1.0 .`:

1. **Build context is sent** - the `.` folder (minus `.dockerignore` entries) is packed and sent to the Docker daemon / BuildKit. A huge context = slow builds.
2. **Dockerfile is parsed** - `FROM` pulls the base image if not present.
3. **Each instruction runs in order** - `RUN`, `COPY`, `ADD` each produce a new read-only **layer** (a filesystem diff). `ENV`, `CMD`, `EXPOSE`, `LABEL` only change metadata.
4. **Cache check per step** - if the instruction text (and for COPY/ADD, the file checksums) is identical to a previous build, Docker reuses the cached layer. The first step that changes invalidates the cache for **every step after it**.
5. **Image is assembled** - layers stacked + config (entrypoint, env, labels) -> image ID, then tagged.
6. **Push (optional)** - only layers the registry does not already have are uploaded.

That is why you copy dependency files and install them before copying source code.

Example:

```
docker build --progress=plain -t app:1.0 .
#  => [internal] load build context         transferring context: 2.1MB
#  => [2/5] WORKDIR /app                     CACHED
#  => [3/5] COPY package*.json ./            CACHED
#  => [4/5] RUN npm ci                       CACHED
#  => [5/5] COPY . .                         0.3s   <- only this step reran
docker history app:1.0
```

> ***What to say to the interviewer:*** *The build sends the context to the daemon, pulls the base image, then executes each instruction in order, with RUN, COPY and ADD each creating a cached layer. Once a step changes, all later steps rebuild, so I order the Dockerfile from least to most frequently*

changing and keep the context small with .dockerignore.

Q10. What is the docker command to build an image, and which flags are commonly used?

In simple words: "docker build" is the "cook now" button. The flags are the settings: what to name the dish, which recipe card to use, which ingredients to pass in, whether to reuse leftovers, and which kitchen (CPU type) it is cooked for. The dot at the end means "use the ingredients in this folder".

Detailed answer

Basic form: `docker build -t name:tag -f Dockerfile .`

- The trailing `.` is the **build context** - the folder whose files are sent to the builder and that `COPY`/`ADD` can see (filtered by `.dockerignore`). It is not the Dockerfile location.

Flag	What it does
`-t name:tag`	Name and tag the image (repeat for multiple tags)
`-f path/Dockerfile`	Use a Dockerfile other than `./Dockerfile`
`--build-arg KEY=val`	Pass a value to an `ARG`
`--no-cache`	Ignore the layer cache, rebuild every step
`--pull`	Always pull the newest version of the base image
`--target build`	Stop at a named stage of a multi-stage build
`--platform linux/amd64,linux/arm64`	Build for other CPU architectures (multi-arch with `buildx`)
`--secret id=npmrc,src=~/.npmrc`	Mount a build-time secret without storing it in a layer
`--label key=value`	Add metadata (git SHA, team)
`--progress=plain`	Full step-by-step output, useful for debugging slow builds
`--cache-from` / `--cache-to`	Use or export a remote build cache (CI speed-up)
`--push` (buildx)	Push straight to the registry after building

Example:

```
# Everyday build
docker build -t myorg/api:1.4.2 .

# Different Dockerfile, build args, fresh base, label
docker build -f docker/Dockerfile.prod \
  --build-arg APP_VERSION=1.4.2 --pull \
  --label org.opencontainers.image.revision=$(git rev-parse HEAD) \
  -t myorg/api:1.4.2 .

# Build only the test stage
docker build --target test -t myorg/api:test .

# CI: multi-arch, secret, remote cache, push
docker buildx build --platform linux/amd64,linux/arm64 \
  --secret id=npmrc,src=$HOME/.npmrc \
  --cache-from type=registry,ref=$REG/api:buildcache \
  --cache-to type=registry,ref=$REG/api:buildcache,mode=max \
  --progress=plain -t $REG/api:1.4.2 --push .
```

What to say to the interviewer: *The command is docker build -t name:tag -f Dockerfile followed by the build context, usually a dot for the current folder. Common flags are --build-arg, --no-cache and --pull for clean rebuilds, --target for a multi-stage stage, --secret for build-time secrets, --label for metadata, and with buildx --platform for multi-arch plus --cache-from and --cache-to for fast CI builds.*

Q11. How do you pass environment variables during docker build? What is the difference between ARG and ENV, and how do you handle secrets at build time?

In simple words: ARG is a sticky note you use only while cooking and then throw away. ENV is a label printed on the final box that everyone who opens it can read. Secrets should be neither - they are handed over through a private window and never stored.

Detailed answer

Point	ARG	ENV
Available during build	Yes	Yes
Available in running container	No	Yes
Set from CLI	`--build-arg NAME=val`	`-e NAME=val` at `docker run`
Stored in image	Value visible in `docker history`	Stored in image config
Good for	Versions, build flags	Runtime config (`NODE_ENV`, `PORT`)

- An `ARG` declared before `FROM` is only usable in `FROM`; redeclare it after `FROM` to use it in steps.
- To make a build arg available at runtime: `ENV APP_VERSION=$APP_VERSION`.
- **Never pass secrets with ARG or ENV** - they end up in history or config. Use BuildKit secrets: the secret is mounted only for that one RUN step and never written into a layer.

Example:

```
# syntax=docker/dockerfile:1
ARG NODE_VERSION=20
FROM node:${NODE_VERSION}-alpine
ARG APP_VERSION=dev
ENV APP_VERSION=${APP_VERSION} NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci --omit=dev
COPY . .
CMD ["node", "server.js"]
```

```
docker build \
  --build-arg NODE_VERSION=20 --build-arg APP_VERSION=1.4.2 \
  --secret id=npmrc,src=$HOME/.npmrc \
  -t app:1.4.2 .
docker run -e PORT=8080 app:1.4.2
```

What to say to the interviewer: *ARG is a build-time-only variable passed with --build-arg, ENV is baked into the image and available at runtime, and I can copy an ARG into an ENV if the app needs it. Secrets never go through either - I use BuildKit --secret mounts so they exist only during that RUN step and never land in a layer.*

Q12. Which container registry do you use, and how do you choose one?

> **In simple words:** A registry is the warehouse where finished image boxes are stored and picked up from. You pick the warehouse closest to your trucks, with good locks and a clean-out policy.

Detailed answer

Common options:

Registry	Best when	Notes
AWS ECR	Workloads on AWS (EKS, ECS, EC2)	IAM auth, private, scan on push, lifecycle policies, cross-region replication
Docker Hub	Public/open-source images	Pull rate limits for anonymous/free users
GHCR (GitHub)	Code and CI on GitHub	Auth with GITHUB_TOKEN, per-repo permissions
GCP Artifact Registry / Azure ACR	GCP / Azure workloads	Same idea as ECR in their cloud
Harbor / Nexus / JFrog	Self-hosted, on-prem, multi-cloud	Built-in scanning, replication, RBAC

Selection criteria: close to the runtime (fast, no egress cost), IAM/SSO integration, private by default, vulnerability scanning, immutable tags, retention policies, replication for DR.

Typical answer for an AWS shop: **ECR** for private app images, with a pull-through cache for Docker Hub base images to avoid rate limits.

Example:

```
aws ecr create-repository --repository-name orders \
  --image-tag-mutability IMMUTABLE \
  --image-scanning-configuration scanOnPush=true
aws ecr get-login-password | docker login --username AWS --password-stdin \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com
docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders:1.4.2
```

> ***What to say to the interviewer:*** *We run on AWS, so we use private ECR repositories - one per service - with IAM-based auth, immutable tags, scan on push and lifecycle policies. Docker Hub is only for public base images, ideally through an ECR pull-through cache to avoid rate limits.*

Q13. What are Docker labels and why would you use them?

> **In simple words:** Labels are sticky tags on a box - "made on this date, by this team, from this code version". They don't change what is inside, but make boxes easy to find, sort and audit.

Detailed answer

- A label is a key-value metadata pair on an image, container, volume or network.
- Set in the Dockerfile with `LABEL`, or at runtime with `docker run --label`.
- **Traceability** - git commit, build number, version, source repo (standard OCI keys like `org.opencontainers.image.revision`).
- **Ownership** - team, contact, cost centre.
- **Filtering and cleanup** - `docker ps --filter label=...`, prune only certain images.

- **Tool config** - Traefik routing rules, Prometheus/log shipper discovery, Watchtower.
- Labels do not affect runtime behaviour and are not for secrets.

Example:

```
ARG GIT_SHA
LABEL org.opencontainers.image.source="https://github.com/myorg/orders" \
      org.opencontainers.image.revision="${GIT_SHA}" \
      org.opencontainers.image.version="1.4.2" \
      team="payments"
```

```
docker build --build-arg GIT_SHA=$(git rev-parse HEAD) -t orders:1.4.2 .
docker inspect orders:1.4.2 --format '{{json .Config.Labels}}'
docker ps --filter "label=team=payments"
docker image prune -a --filter "label=env=dev"
```

> ***What to say to the interviewer:*** *Labels are key-value metadata on images and containers. I use them to record the git SHA, version and owning team for traceability, to filter and clean up resources, and for tools like Traefik or monitoring agents that discover containers by label.*

Q14. What are Docker restart policies (no, on-failure, always, unless-stopped)?

> **In simple words:** It's the instruction to the building's security guard for when a machine switches off: leave it off, restart it only if it broke, always restart it, or restart it unless a human deliberately turned it off.

Detailed answer

Policy	Restarts when	Survives daemon/host reboot	Typical use
no (default)	Never	No	One-off tasks, debugging
`on-failure[:N]`	Exit code is non-zero (optional max N retries)	Yes, if it had failed	Batch jobs, workers
`always`	Any exit; also after daemon restart even if you stopped it manually	Yes	Critical services
`unless-stopped`	Any exit, except when you stopped it manually	Yes, unless manually stopped	Most long-running services

- Docker adds an increasing delay between restarts (backoff) to avoid tight crash loops.
- A restart policy hides crashes - always monitor the `RestartCount` and alert on it.
- In Kubernetes the pod `restartPolicy` and controllers take over this job.

Example:

```
docker run -d --name api --restart unless-stopped myorg/api:1.4.2
docker run -d --name job --restart on-failure:5 myorg/worker:2.0
docker update --restart always api            # change on a running container
docker inspect api --format '{{.HostConfig.RestartPolicy.Name}} restarts={{.RestartCount}}'
```

> ***What to say to the interviewer:*** *Docker has four restart policies - no, on-failure with an optional retry count, always, and unless-stopped. I use unless-stopped for long-running services and on-failure for jobs, and I alert on restart count so a restart policy never silently hides a crash loop.*

Q15. What is Docker Swarm and how is it different from Kubernetes?

In simple words: Both are managers for a fleet of containers. Swarm is a small, simple family-run bus company - easy to start, fewer rules. Kubernetes is a big city transport authority - more setup, but it handles huge scale, many routes and every kind of emergency.

Detailed answer

- **Docker Swarm** is Docker's built-in orchestrator. `docker swarm init` turns Docker hosts into a cluster of manager and worker nodes; you deploy **services** (with replicas) and **stacks** (Compose files) and Swarm keeps them running, load-balances them and does rolling updates.
- **Kubernetes** is a separate, much richer orchestration platform (API server, etcd, scheduler, controllers) and is the industry standard, with managed versions like EKS, GKE and AKS.

Point	Docker Swarm	Kubernetes
Setup	One command, built into Docker	More components; usually managed (EKS/GKE/AKS)
Learning curve	Low - uses Compose files	Steep - many object types
Unit of deployment	Service / task	Pod, Deployment, StatefulSet, etc.
Autoscaling	No built-in autoscaling	HPA, VPA, Cluster Autoscaler, KEDA
Networking	Overlay network + routing mesh	CNI plugins, Services, Ingress, NetworkPolicy
Storage	Basic volumes	PV/PVC, StorageClasses, CSI drivers
Config and secrets	Docker configs and secrets	ConfigMaps, Secrets, external secret operators
Ecosystem	Small, limited active development	Huge: Helm, Argo CD, Prometheus, service mesh
Best for	Small teams, simple apps, few hosts	Production at scale, microservices, multi-team

Example:

```
docker swarm init --advertise-addr 10.0.1.10
docker swarm join-token worker            # command to run on worker nodes
docker service create --name web --replicas 3 -p 80:80 nginx:1.27
docker service scale web=5
docker service update --image nginx:1.27.1 --update-order start-first web
docker stack deploy -c docker-compose.yml shop
```

What to say to the interviewer: *Docker Swarm is Docker's native orchestrator - very easy to set up and uses Compose files, but it has no built-in autoscaling and a small ecosystem. Kubernetes needs more setup but gives autoscaling, rich networking and storage, RBAC and a huge ecosystem, which is why production microservice platforms standardise on it.*

Q16. What are the common Docker commands you use daily?

In simple words: Like a mechanic's most-used tools on the belt - a small set of commands covers 90 percent of daily work: build, run, look inside, read logs, clean up.

Detailed answer

Group	Command	What it does
Images	`docker build -t app:1.0 .`	Build an image from a Dockerfile
Images	`docker images`	List local images
Images	`docker pull` / `docker push`	Download / upload an image
Images	`docker tag app:1.0 $REG/app:1.0`	Add a registry tag
Images	`docker history app:1.0`	Show layers and their sizes
Containers	`docker run -d --name web -p 8080:80 nginx`	Start a container
Containers	`docker ps` / `docker ps -a`	Running / all containers
Containers	`docker stop` / `start` / `restart web`	Control lifecycle
Containers	`docker rm -f web`	Remove a container
Debug	`docker logs -f --tail 100 web`	Follow logs
Debug	`docker exec -it web sh`	Shell inside a running container
Debug	`docker inspect web`	Full JSON: exit code, IP, mounts, env
Debug	`docker stats` / `docker top web`	Live CPU and memory / processes
Debug	`docker events`	Real-time daemon events (die, oom)
Cleanup	`docker system df`	Disk usage by Docker
Cleanup	`docker system prune`	Remove stopped containers, dangling images, unused networks
Cleanup	`docker image prune -a` / `docker builder prune`	Unused images / build cache
Networks	`docker network ls` / `docker network create appnet`	List / create networks
Networks	`docker network inspect appnet`	Which containers and IPs are on it
Volumes	`docker volume ls` / `docker volume create data`	List / create volumes
Volumes	`docker volume inspect data`	Mountpoint on the host
Compose	`docker compose up -d` / `docker compose down`	Start / stop a multi-container stack
Compose	`docker compose ps` / `docker compose logs -f api`	Status / logs for a service
Compose	`docker compose build` / `docker compose pull`	Rebuild / pull service images

Example:

```
docker build -t myorg/api:1.4.2 .
docker run -d --name api --network appnet -p 8080:3000 myorg/api:1.4.2
docker logs -f --tail 50 api
docker exec -it api sh
docker inspect api --format '{{.State.Status}} {{.State.ExitCode}}'
docker system df && docker image prune -a -f --filter "until=168h"
```

> ***What to say to the interviewer:*** *Day to day I use docker build, tag and push for images; run, ps, stop and rm for containers; logs, exec, inspect, stats and events for debugging; system df and prune for cleanup; plus docker compose up, logs and down for multi-container stacks.*

Q17. How do volumes work in Docker? When do you use bind mounts vs named volumes (and tmpfs)?

> **In simple words:** A container is like a hotel room - when you check out, everything left inside is thrown away. A volume is your own locker that survives after checkout. A bind mount is a window into a specific folder of the host's house. tmpfs is a whiteboard in the room - fast, but wiped when you leave.

Detailed answer

Data written into the container's writable layer is lost when the container is removed and is slow for heavy writes. Mounts keep data outside the container:

Type	Where data lives	Managed by	Best for	Watch out
Named volume	`/var/lib/docker/volumes/<name>/_data`	Docker	Databases, app data in prod	Must back up separately
Bind mount	Any host path you choose	You	Dev live code reload, config files, host logs	Depends on host folder layout and permissions
tmpfs	Host memory only	Docker	Temp files, secrets scratch, read-only containers	Lost on stop, uses RAM

- Named volumes are portable, can use volume drivers (NFS, EBS plugins) and are pre-filled with the image's files at that path on first use.
- A bind mount hides whatever the image had at that path.
- Add `:ro` / `readonly` for config you don't want the app to change.

`-v` vs `--mount`: `-v` is short and silently creates a missing host folder; `--mount` is explicit key=value and errors if the source is missing - clearer for production scripts.

Permissions / UID gotcha: Files are owned by numeric UIDs, not names. If the container runs as UID 1000 but the host folder is owned by root, the app gets `Permission denied`. Fix: `chown` the host folder to the container UID, run the container with `--user $(id -u):$(id -g)` for bind mounts, or set ownership in the Dockerfile (`COPY --chown`, `RUN chown`) before the volume is first created.

Example:

```
# Named volume
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16
docker run -d --name db2 --mount type=volume,src=pgdata2,dst=/var/lib/postgresql/data postgres:16

# Bind mount (read-only config) and tmpfs
docker run -d --name web \
  --mount type=bind,src=/etc/myapp/nginx.conf,dst=/etc/nginx/nginx.conf,readonly \
  --mount type=tmpfs,dst=/tmp,tmpfs-size=64m nginx:1.27

# Back up and restore a volume through a throwaway container
docker run --rm -v pgdata:/data:ro -v $PWD:/backup alpine tar czf /backup/pgdata.tgz -C /data .
docker run --rm -v pgdata:/data -v $PWD:/backup alpine tar xzf /backup/pgdata.tgz -C /data

# UID gotcha fix for a bind mount
sudo chown -R 1000:1000 /srv/app-data
docker volume inspect pgdata --format '{{.Mountpoint}}'
```

For databases, a file-level tar is only safe with the DB stopped - otherwise use the DB's own dump tool (`pg_dump`) or storage snapshots.

> ***What to say to the interviewer:*** *Named volumes are Docker-managed storage that outlives the container, which I use for databases and app data; bind mounts map a specific host path, which I use for config files and local development; tmpfs is in-memory for temporary or sensitive scratch data. I prefer --mount in production because it is explicit, back volumes up through a throwaway container or DB dumps, and handle the UID mismatch by chowning to the container user.*

Advanced

Q18. Write a sample multi-stage Dockerfile you would actually use in a real project (Node.js app).

> **In simple words:** Stage 1 installs everything and builds the app, stage 2 keeps only production packages, and the last stage is a clean, small box with just what is needed to run.

Detailed answer

Real-project points shown below:

- Pin base image versions, never `latest`.
- Copy `package*.json` first so `npm ci` is cached unless dependencies change.
- Separate build stage (dev dependencies, TypeScript compile) from runtime.
- Runtime runs as non-root `node` user, with `NODE_ENV=production`.
- Exec form CMD so SIGTERM reaches Node for graceful shutdown.
- A `.dockerignore` keeps `node_modules`, `.git` and `.env` out of the build context.

Example:

```
# syntax=docker/dockerfile:1
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci

FROM deps AS build
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:20-alpine AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist
COPY --chown=node:node package.json ./
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

```
# .dockerignore
node_modules
.git
.env
*.log
coverage
```

> ***What to say to the interviewer:*** *I use a deps stage to cache npm ci, a build stage that compiles and prunes dev dependencies, and a slim alpine runtime stage that copies only dist and production node_modules, runs as the non-root node user and uses exec-form CMD. Combined with a .dockerignore, this typically cuts the image by 60-80 percent.*

Q19. Create a Dockerfile for running a database container (MySQL or PostgreSQL).

> **In simple words:** The official database image is a ready-made kitchen. You just add your own recipe book (init scripts) and settings, and keep the food (data) in a separate fridge (volume) so it survives if the kitchen is rebuilt.

Detailed answer

Key points:

- Build on the official image (`postgres:16`, `mysql:8.4`) - do not install a DB from scratch.
- Scripts placed in `/docker-entrypoint-initdb.d/` run **only on first start** when the data directory is empty.
- Never bake passwords into the image; pass them at runtime (env var, or better `*_FILE` pointing to a secret file).
- Always mount a **volume** for the data directory, otherwise data is lost when the container is removed.
- Add a healthcheck so dependent services wait until the DB is ready.

Example:

```
FROM postgres:16-alpine
ENV POSTGRES_DB=appdb \
    POSTGRES_USER=appuser
COPY init/01-schema.sql /docker-entrypoint-initdb.d/
COPY config/postgresql.conf /etc/postgresql/postgresql.conf
HEALTHCHECK --interval=10s --timeout=5s --retries=5 \
  CMD pg_isready -U appuser -d appdb || exit 1
EXPOSE 5432
CMD ["postgres", "-c", "config_file=/etc/postgresql/postgresql.conf"]
```

```
docker build -t mydb:1.0 .
docker run -d --name db \
  -e POSTGRES_PASSWORD_FILE=/run/secrets/db_pass \
  -v /secure/db_pass:/run/secrets/db_pass:ro \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 mydb:1.0
```

What to say to the interviewer: *I extend the official Postgres or MySQL image, add init SQL into docker-entrypoint-initdb.d, a custom config and a pg_isready healthcheck, and pass the password at runtime via a secret file rather than baking it in. Data always goes to a named volume; in production I would still prefer a managed DB like RDS.*

Q20. Write a Dockerfile for a monitoring tool like Nagios.

In simple words: Take a clean Ubuntu box, install the Nagios package and its web server from the official package store, add your own list of servers to watch, open the web page door (port 80), and add a self-check so Docker knows it is alive.

Detailed answer

Design points:

- Build on a pinned OS base (`ubuntu:24.04`) and install `nagios4` from the distro repository - no compiling from source. (For a quick start, the widely used community image `jasonrivers/nagios` is an alternative.)
- Nagios Core has two parts: the **nagios4 daemon** (runs the checks) and the **web UI** (Apache + CGI). The container starts both: Nagios in the background, Apache in the foreground as the main process.
- Host and service definitions go into `/etc/nagios4/conf.d/` - copy them in, or better, mount them so config changes do not need a rebuild.
- Volumes for state and logs (`/var/lib/nagios4`, `/var/log/nagios4`).
- The web login password comes from a runtime secret, never baked into the image.
- Validate the config at startup (`nagios4 -v`) so a bad config fails fast.

Example:

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update \
 && apt-get install -y --no-install-recommends nagios4 monitoring-plugins \
      apache2 apache2-utils curl procps \
 && a2enmod cgi rewrite \
 && rm -rf /var/lib/apt/lists/*
COPY conf.d/ /etc/nagios4/conf.d/
COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
RUN chmod +x /usr/local/bin/docker-entrypoint.sh
VOLUME ["/var/lib/nagios4", "/var/log/nagios4"]
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD pgrep -x nagios4 >/dev/null && curl -s -o /dev/null http://localhost/nagios4/ || exit 1
ENTRYPOINT ["/usr/local/bin/docker-entrypoint.sh"]
```

```
#!/bin/sh
# docker-entrypoint.sh
set -e
if [ -f /run/secrets/nagios_pass ]; then
  htpasswd -bc /etc/nagios4/htpasswd.users nagiosadmin "$(cat /run/secrets/nagios_pass)"
fi
/usr/sbin/nagios4 -v /etc/nagios4/nagios.cfg      # fail fast on bad config
/usr/sbin/nagios4 -d /etc/nagios4/nagios.cfg      # daemon in background
exec apachectl -D FOREGROUND                      # web UI as the main process
```

```
docker build -t mynagios:1.0 .
docker run -d --name nagios -p 8080:80 \
  -v $PWD/conf.d:/etc/nagios4/conf.d:ro \
  -v nagios-data:/var/lib/nagios4 \
  -v /secure/nagios_pass:/run/secrets/nagios_pass:ro mynagios:1.0
# UI: http://<host>:8080/nagios4/
```

> ***What to say to the interviewer:*** *I base it on a pinned Ubuntu image, install nagios4 with Apache from the distro repo, copy or mount the host and service definitions into conf.d, expose port 80 and add a healthcheck that confirms both the daemon and the web UI. An entrypoint script validates the config, starts Nagios in the background and runs Apache in the foreground, with the admin password injected at runtime.*

Q21. Besides multi-stage builds and slim base images, how else can you reduce Docker image size?

Also asked as: How do you optimize a Docker image for production?

> **In simple words:** Packing a suitcase: don't pack what you won't use, throw away the packaging, and don't leave the rubbish inside the bag.

Detailed answer

1. `.dockerignore` - exclude `.git`, `node_modules`, test data, logs, local builds.
2. **Clean up in the same RUN layer** - files deleted in a later layer still exist in the earlier layer.
3. **Skip extras** - `apt-get install --no-install-recommends`, `pip install --no-cache-dir`, `apk add --no-cache`, `npm ci --omit=dev`.
4. **Combine RUN steps** to reduce layers and leftover caches.
5. **Distroless or scratch** final images for compiled apps.

6. **Remove build secrets and caches** - use BuildKit `--mount=type=cache` so caches never land in the image.
7. **Copy only the needed artifact**, not the whole repo.
8. **Analyse layers** with `docker history` and the `dive` tool to find big layers.
9. **Strip binaries / static linking** (Go `-ldflags="-s -w"`).
10. **Squash / fewer COPY of large assets**; serve big static files from S3/CDN instead of the image.

Example:

```
# BAD - cache stays in an earlier layer
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# GOOD - one layer, cleaned up
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl ca-certificates \
 && rm -rf /var/lib/apt/lists/*
```

```
docker history --no-trunc myapp:1.0 | head
dive myapp:1.0
```

> ***What to say to the interviewer:*** *Beyond multi-stage and slim bases, I use a strict .dockerignore, install with no-recommends and no-cache flags, clean up in the same RUN layer, use distroless or scratch where possible and copy only the final artifact. Then I verify with docker history or dive to find any fat layer.*

Q22. How do you manage and push Docker images when you have multiple services?

Also asked as: How do you connect Jenkins and push Docker images to AWS ECR (Docker side)?

> **In simple words:** Like a warehouse where every product has its own shelf (repository) and every batch gets a unique label (tag), so you always know exactly which version is shipped where.

Detailed answer

- **One repository per service** in the registry (ECR): `orders`, `payments`, `users`.
- **Immutable, traceable tags**: git commit SHA and/or semantic version (`1.4.2`, `sha-3f9c1ab`). Avoid relying on `latest` in prod. Turn on ECR tag immutability.
- **Build once, promote everywhere** - the same image digest moves dev -> staging -> prod; only config changes.
- **CI loops** over changed services only (monorepo path filters) and builds them in parallel.
- **Authenticate with short-lived tokens** (`aws ecr get-login-password` using an IAM role) - no static keys.
- **Scan on push** (ECR scanning or Trivy in CI) and add lifecycle policies to clean old images.
- Use `docker buildx` for multi-arch (amd64/arm64) when needed.

Example:

```
ACCOUNT=123456789012; REGION=ap-south-1
REG=$ACCOUNT.dkr.ecr.$REGION.amazonaws.com
TAG=$(git rev-parse --short HEAD)

aws ecr get-login-password --region $REGION | docker login --username AWS --password-stdin $REG

for svc in orders payments users; do
  docker build -t $REG/$svc:$TAG -t $REG/$svc:1.4.2 ./services/$svc
  docker push $REG/$svc:$TAG
  docker push $REG/$svc:1.4.2
done
```

> ***What to say to the interviewer:*** *Each service gets its own ECR repository and every build is tagged with the git SHA plus a version, with tag immutability on. CI builds only changed services, logs in with short-lived tokens from an IAM role, scans on push and promotes the same image digest across environments.*

Q23. Do you have an archival or cleanup process for Docker images in AWS ECR?

> **In simple words:** A cupboard that throws out old clothes automatically - keep the last few outfits and the special ones, bin the rest - so it never overflows or costs more rent.

Detailed answer

Yes - with **ECR lifecycle policies**. Rules are evaluated by priority and expire images automatically.

Typical policy:

- Keep the last N (for example 20) release-tagged images (`v*` or `prod-*`).
- Delete untagged images (left behind when a tag moves) after 1-7 days.
- Delete dev/feature-branch images older than 14-30 days.

Also:

- Use **tag immutability** so a tag cannot be silently overwritten.
- For long-term audit needs, keep release images in a separate "archive" repository (or replicate to another region/account) with a longer retention rule.
- Test with the lifecycle policy **preview** before applying.
- Clean up locally on build agents too: `docker image prune -a --filter "until=168h"`.

Example:

```
{
  "rules": [
    { "rulePriority": 1, "description": "Keep last 20 releases",
      "selection": { "tagStatus": "tagged", "tagPrefixList": ["v"],
                     "countType": "imageCountMoreThan", "countNumber": 20 },
      "action": { "type": "expire" } },
    { "rulePriority": 2, "description": "Expire untagged after 7 days",
      "selection": { "tagStatus": "untagged", "countType": "sinceImagePushed",
                     "countUnit": "days", "countNumber": 7 },
      "action": { "type": "expire" } }
  ]
}
```

```
aws ecr put-lifecycle-policy --repository-name orders \
  --lifecycle-policy-text file://policy.json
```

> ***What to say to the interviewer:*** *Yes, we use ECR lifecycle policies - keep the last 20 release tags, expire untagged images after a week and feature-branch images after a few weeks - with tag immutability enabled. Anything that must be kept long-term goes to a separate archive repository with its own retention.*

Q24. How do you secure Docker containers and images?

Also asked as: Container security - non-root, distroless, read-only filesystems. How do you optimize (harden) a Docker image for production? / What type of Docker images should be used in production?

> **In simple words:** Lock the house (non-root, read-only), keep few doors and windows (minimal image), and have an inspector check it for known weak locks before anyone moves in (image scanning).

Detailed answer

Build time:

- Minimal, pinned base images (distroless, alpine, slim), ideally pinned by digest.
- **Scan images** in CI and at the registry (details in the image-scanning question below).
- No secrets in images or layers (use BuildKit `--mount=type=secret`).
- Lint Dockerfiles with `hadolint`; sign images (cosign) and generate an SBOM.

Run time:

- Run as **non-root** (`USER 1000`).
- **Read-only root filesystem**, with tmpfs for scratch folders.
- Drop Linux capabilities (`--cap-drop ALL`), `--security-opt no-new-privileges`.
- Never use `--privileged` or mount `/var/run/docker.sock` into app containers.
- Set CPU/memory limits; keep Docker engine and host patched.

Example:

```
trivy image --severity HIGH,CRITICAL --exit-code 1 --ignore-unfixed myapp:1.0
grype myapp:1.0 --fail-on high

docker run -d --name api \
  --user 1000:1000 --read-only --tmpfs /tmp \
  --cap-drop ALL --security-opt no-new-privileges \
  --memory 512m --cpus 1 myapp:1.0
```

Which images to use in production:

- Start from **official, verified-publisher, or an internal golden base image**, never random images from Docker Hub.
- Keep it **minimal:** distroless, `-slim`, alpine (watch musl compatibility) or Ubuntu chiseled. Fewer packages means fewer CVEs and a smaller attack surface.
- **Pin versions:** `node:20.11-slim@sha256:...`, never `latest`, so builds are reproducible.
- Use **multi-stage builds** so compilers and build tools stay out of the final image.

- Run as **non-root**, with no secrets baked in, scanned (Trivy) and signed (cosign), with an SBOM and provenance labels.
- **Rebuild regularly** (for example weekly) so base-image security patches get in, even when the code didn't change.

> ***What to say to the interviewer:*** *I use minimal pinned images, scan every build with Trivy and fail the pipeline on fixable high or critical CVEs, and keep secrets out of layers. At runtime containers run as non-root with a read-only filesystem, dropped capabilities, no-new-privileges and resource limits - never privileged.*

Q25. How do you pass database credentials (or other secrets) to a containerised application securely?

> **In simple words:** Don't write the house key on the front door (the image). Hand it to the person only when they arrive, from a locked key box (secrets manager).

Detailed answer

What NOT to do:

- Hardcode in code, `ENV` in the Dockerfile, or `ARG` (visible in `docker history`).
- Commit `.env` files to git.

Better options, from okay to best:

1. **Runtime env vars** from a protected source (`--env-file` not in git). Simple, but visible in `docker inspect`.
2. **Secret files mounted read-only** (Docker secrets, Compose secrets, Kubernetes Secrets as volumes) and use `*_FILE` variables.
3. **External secret manager** - AWS Secrets Manager / SSM Parameter Store / Vault. The app (or an init step) fetches the secret at startup using its IAM role, so no long-lived credentials exist anywhere. Supports rotation.
4. In Kubernetes: External Secrets Operator or Secrets Store CSI driver syncing from Secrets Manager.
5. For build-time secrets (private package tokens) use BuildKit `--mount=type=secret` so they never land in a layer.

Example:

```
# Docker Compose secret mounted as a file
services:
  app:
    image: myorg/app:1.0
    environment:
      DB_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
secrets:
  db_password:
    file: ./secrets/db_password.txt   # not committed to git
```

```
# Fetch from AWS Secrets Manager at startup using the instance/task IAM role
export DB_PASSWORD=$(aws secretsmanager get-secret-value \
  --secret-id prod/app/db --query SecretString --output text | jq -r .password)
```

What to say to the interviewer: *I never bake credentials into the image or git. In production the secret lives in AWS Secrets Manager or Vault and is injected at runtime - fetched via the workload's IAM role or mounted as a read-only file - which also enables rotation without rebuilding the image.*

Q26. How do you scan Docker images for vulnerabilities, both during the build and at the registry level?

Also asked as: What methods do you use to check container vulnerabilities? Scanning with Trivy or Grype.

In simple words: Two checkpoints - a security check at the factory gate before the box leaves (CI scan), and a warehouse inspector who keeps re-checking stored boxes as new weaknesses are discovered (registry scan).

Detailed answer

1. During the build (CI gate - shift left):

- Run **Trivy**, **Grype** or **Docker Scout** right after `docker build`, before `docker push`.
- Fail the pipeline on **CRITICAL/HIGH fixable** CVEs (`--ignore-unfixed` avoids blocking on issues with no patch).
- Also scan the Dockerfile/IaC for misconfig (`trivy config`, `hadolint`) and generate an SBOM.
- Keep a reviewed `.trivyignore` with expiry dates for accepted risks.

2. At the registry (continuous):

- **ECR basic scanning** - scan on push (Clair-based CVE database).
- **ECR enhanced scanning** via **Amazon Inspector** - continuous re-scanning when new CVEs are published, OS + language packages, findings to Security Hub/EventBridge -> alerts.
- Docker Hub / Harbor / GHCR offer similar scanning.

3. At deploy/runtime: admission policy (Kyverno/OPA) to block unscanned or unsigned images; rebuild base images regularly to pick up patches.

Example:

```
docker build -t $REG/orders:$TAG .
trivy image --severity CRITICAL,HIGH --ignore-unfixed --exit-code 1 $REG/orders:$TAG
grype $REG/orders:$TAG --fail-on critical
docker scout cves $REG/orders:$TAG --only-severity critical,high
docker push $REG/orders:$TAG

aws ecr put-registry-scanning-configuration --scan-type ENHANCED \
  --rules '[{"scanFrequency":"CONTINUOUS_SCAN","repositoryFilters":[{"filter":"*","filterType":"WILD
    CARD"}]}]'
aws ecr describe-image-scan-findings --repository-name orders --image-id imageTag=$TAG
```

What to say to the interviewer: *I scan twice - in CI with Trivy or Grype right after the build, failing the pipeline on fixable critical and high CVEs before the image is pushed, and in the registry with ECR enhanced scanning through Inspector, which keeps re-scanning stored images as new CVEs appear and raises alerts. Base images are rebuilt regularly so fixes actually land.*

Q27. How do you monitor Docker containers in production?

In simple words: Give every container a fitness tracker (metrics), a diary that is collected centrally (logs), and a heartbeat check (health check), then set alarms when something looks wrong.

Detailed answer

- **Metrics**: **cAdvisor** exposes per-container CPU, memory, network, disk I/O; **node-exporter** for the host; **Prometheus** scrapes both; **Grafana** dashboards. `docker stats` for a quick live view.
- **Logs**: containers log to stdout/stderr; a **log driver** (json-file with rotation, awslogs, fluentd) or an agent like **Fluent Bit / Promtail** ships them to Loki, ELK or CloudWatch.
- **Health**: HEALTHCHECK in the Dockerfile -> status shows `healthy/unhealthy`; external uptime checks on the endpoint.
- **Events**: `docker events` for die/oom/restart.
- **Alerts** (Alertmanager -> Teams/Slack/pager): container down, restart count increasing, OOM kills, memory near limit, CPU throttling, unhealthy status, host disk above 80 percent.

Example:

```
docker run -d --name cadvisor -p 8081:8080 \
  -v /:/rootfs:ro -v /var/run:/var/run:ro -v /sys:/sys:ro \
  -v /var/lib/docker/:/var/lib/docker:ro gcr.io/cadvisor/cadvisor:v0.49.1
```

```
# Prometheus alert: container memory above 90 percent of its limit
- alert: ContainerMemoryHigh
  expr: container_memory_working_set_bytes{name!=""}
        / container_spec_memory_limit_bytes{name!=""} > 0.9
  for: 5m
  labels: { severity: warning }
```

What to say to the interviewer: *I use cAdvisor and node-exporter scraped by Prometheus with Grafana dashboards for metrics, Fluent Bit or Promtail shipping stdout logs to Loki or CloudWatch, and Dockerfile HEALTHCHECKs. Alertmanager notifies on container down, rising restart count, OOM kills, memory near limit and host disk usage.*

Q28. How do you handle logging for containers running in production?

In simple words: Every container just talks out loud (stdout). Docker writes it into a notebook that is torn out and replaced when full (rotation), and a courier (Fluent Bit) carries copies to a central library (Loki/ELK/CloudWatch) where anyone can search them - but nobody should ever say passwords out loud.

Detailed answer

1. Log to stdout/stderr, not files inside the container. Docker captures both streams; files inside the container are lost with it and fill the writable layer.

2. Pick a logging driver:

Driver	Where logs go	Note
`json-file`	Local JSON files (default)	No rotation unless configured
`local`	Local, compressed, rotated by default	Good default for single hosts

Driver	Where logs go	Note
`journald`	systemd journal	`journalctl CONTAINER_NAME=api`
`awslogs`	CloudWatch Logs	Needs IAM role on the host
`fluentd`	Fluentd / Fluent Bit endpoint	Central pipelines
`gelf`	Graylog / Logstash	UDP or TCP

`docker logs` works directly with json-file, local and journald; for remote drivers Docker keeps a local dual-logging cache (Docker 20.10+).

3. Rotate logs - default json-file grows forever and can fill `/var/lib/docker`. Set `max-size` and `max-file` in `/etc/docker/daemon.json` (applies to new containers).

4. Ship centrally - an agent (Fluent Bit, Vector, Promtail) tails the container log files and sends them to Loki, ELK/OpenSearch or CloudWatch, adding labels like container, service, env.

5. Structured JSON logs with timestamp, level, service and a `trace_id` so one request can be followed across services and linked to traces.

6. Never log secrets (passwords, tokens, card numbers); mask them in the app and add redaction filters in the shipper.

7. Blocking vs non-blocking mode - default `mode=blocking`: if the log destination is slow, the app's writes to stdout block and the app slows down. `mode=non-blocking` with `max-buffer-size` keeps the app fast, but logs can be dropped when the buffer fills. Use non-blocking for remote drivers on latency-critical apps.

For metrics and alerting on containers, see the monitoring question above - logs explain why, metrics tell you when.

Example:

```
# /etc/docker/daemon.json  (restart docker; affects newly created containers)
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "5", "mode": "non-blocking",
                "max-buffer-size": "4m" }
}
```

```
docker run -d --name api \
  --log-driver awslogs \
  --log-opt awslogs-region=ap-south-1 \
  --log-opt awslogs-group=/prod/api \
  --log-opt mode=non-blocking --log-opt max-buffer-size=4m \
  myorg/api:1.4.2

docker inspect api --format '{{.HostConfig.LogConfig.Type}}'
```

```
{"ts":"2026-09-24T10:15:02Z","level":"error","service":"orders",
 "trace_id":"4bf92f3577b34da6","msg":"payment timeout","duration_ms":3012}
```

> ***What to say to the interviewer:*** *Apps log structured JSON with a trace_id to stdout, Docker's json-file or local driver has max-size and max-file rotation in daemon.json so disks never fill, and Fluent Bit or Promtail ships the logs to Loki, ELK or CloudWatch. I keep secrets out of logs, and for*

remote drivers on latency-sensitive services I use non-blocking mode with a buffer, accepting that some logs may drop under pressure.

Q29. How do you do a zero-downtime update of a service using plain Docker (without Kubernetes)?

In simple words: Open the new shop counter next door, check it works, then move the "Open" sign to it and only then close the old counter. Customers never find a closed door.

Detailed answer

Plain `docker stop` + `docker run` always causes a gap, so use **blue-green behind a reverse proxy** (Nginx, Traefik, HAProxy):

1. Current version (**blue**) runs behind Nginx on port 8081.
2. Start the new version (**green**) on port 8082 with the new image.
3. Wait until green passes its health check.
4. Switch the Nginx upstream to green and `nginx -s reload` (graceful - existing connections finish).
5. Watch errors/latency for a few minutes.
6. Stop blue (keep the image for instant rollback). Rollback = switch upstream back.

Alternatives: Docker Swarm `docker service update --update-order start-first`, or Traefik with labels which discovers containers automatically. The app must handle SIGTERM gracefully and be stateless (sessions in Redis, not memory).

Example:

```
docker run -d --name app-green --network web -p 8082:3000 myorg/app:1.5.0
until curl -fs http://localhost:8082/health; do sleep 2; done

sed -i 's/127.0.0.1:8081/127.0.0.1:8082/' /etc/nginx/conf.d/app.conf
nginx -t && nginx -s reload

sleep 300 && docker stop app-blue     # after verification
```

What to say to the interviewer: *With plain Docker I use blue-green behind Nginx or Traefik - start the new container alongside the old one, wait for its health check, switch the proxy upstream with a graceful reload, then retire the old one while keeping it for fast rollback. Docker Swarm's start-first update order gives the same result natively.*

Q30. How do you move containers from one server to another?

In simple words: You don't move a running kitchen - you send the recipe (image) to the new place, carry the food store (volume data) separately, and start cooking there.

Detailed answer

Containers are disposable - you move the **image**, the **config** and the **data**, then start a fresh container.

1. **Image** - best: push to a registry and pull on the new host. No registry / air-gapped: `docker save` -> copy the tar -> `docker load`.

2. **Config** - `docker run` flags, env files, Compose file (keep them in git). `docker inspect` on the old host shows ports, env, mounts.
3. **Data** - volumes are NOT part of the image or `docker commit`. Back up the volume (tar through a helper container, or DB dump) and restore on the new host.
4. Start on the new host, verify, switch DNS/load balancer, then remove the old one.

Avoid `docker export/import` (flattens the filesystem and loses image metadata like CMD, ENV) and avoid `docker commit` as a migration method (unreproducible images).

Example:

```
# Image via registry
docker push $REG/app:1.4.2          # old host
docker pull $REG/app:1.4.2          # new host

# Image without registry
docker save myorg/app:1.4.2 | gzip > app.tar.gz
scp app.tar.gz user@newhost:/tmp/ && ssh user@newhost "gunzip -c /tmp/app.tar.gz | docker load"

# Volume data
docker run --rm -v appdata:/data -v $PWD:/backup alpine tar czf /backup/appdata.tgz -C /data .
docker run --rm -v appdata:/data -v $PWD:/backup alpine tar xzf /backup/appdata.tgz -C /data
```

> ***What to say to the interviewer:*** *I treat containers as disposable - the image goes through a registry (or docker save and load if there is no registry), the run configuration comes from a Compose file in git, and volume data is backed up and restored separately since it is not part of the image. Then I start it on the new host, verify, and switch traffic.*

Scenario-Based

Q31. A Docker container keeps restarting (or suddenly stops / crashes) in production. How do you find the root cause?

Also asked as: If a running container suddenly stops, how do you troubleshoot? A container has crashed or exited - how do you figure out what went wrong?

> **In simple words:** Like a car that keeps stalling: check the dashboard warning (exit code), read the trip log (container logs), and check whether it ran out of fuel (memory) or was told to stop by someone else.

Detailed answer

1. **Status and restart count**: `docker ps -a` - look at STATUS and how often it restarts.
2. **Exit code and reason**: `docker inspect` -> `State.ExitCode`, `State.OOMKilled`, `State.Error`.

- `0` = process finished (wrong CMD, app not long-running).
- `1` = app error (config, missing env, crash).
- `125/126/127` = docker run error / not executable / command not found.
- `137` = SIGKILL, usually **OOM killed** or forced kill.
- `139` = segfault. `143` = SIGTERM (normal stop).

3. **Logs**: `docker logs --tail 200 <c>` and `--previous` style checks around the crash time.
4. **Events**: `docker events` or `journalctl -u docker` - who stopped it, OOM, health check failures.
5. **Resources**: `docker stats`, host `dmesg | grep -i oom`, disk full (`df -h`).
6. **Dependencies**: DB/API reachable? DNS? Wrong env or missing secret?
7. **Health check** failing -> restart policy or orchestrator kills it.
8. **Reproduce**: run the same image with `--entrypoint sh` to inspect files, config and permissions.
9. **Fix and prevent**: raise memory limit or fix leak, fix config, add proper health check, alert on restart count.

Example:

```
docker ps -a --filter name=api
docker inspect api --format 'Exit={{.State.ExitCode}} OOM={{.State.OOMKilled}} Err={{.State.Error}}'
# Exit=137 OOM=true Err=
docker logs --since 30m api | tail -50
docker events --since 1h --filter container=api
dmesg -T | grep -i -E "killed process|out of memory"
docker run --rm -it --entrypoint sh myorg/api:1.4.2
```

> ***What to say to the interviewer:*** *I start with docker ps -a and docker inspect to get the exit code and OOMKilled flag, then read docker logs and docker events around the crash. Exit 137 with OOMKilled points to memory, exit 1 to an app or config error, 127 to a bad command; I confirm by running the image with a shell, fix the cause and add alerts on restart count.*

Q32. Your Docker image size has grown from 300MB to 3GB. How would you investigate and optimize it?

> **In simple words:** Your suitcase suddenly weighs ten times more. Open it layer by layer to find what got packed by mistake, then repack properly.

Detailed answer

Investigate first (find what changed):

1. `docker history <image>` - see size per layer; the fat layer shows which instruction caused it.
2. `dive <image>` - browse files added per layer and wasted space.
3. `git diff` the Dockerfile and `.dockerignore` between the 300MB and 3GB versions.

Common culprits:

- `.dockerignore` missing or broken -> `.git`, `node_modules`, datasets, logs copied in.
- Base image changed (alpine -> full ubuntu, or a `-dev` / CUDA image).
- Build tools and caches left in (apt lists, pip/npm cache, compilers).
- Files deleted in a later layer (still stored in the earlier one).
- Large artifacts, dumps or test files copied with `COPY . .`.

Fix:

- Restore/strengthen `.dockerignore`; copy only needed paths.
- Multi-stage build, slim/distroless runtime.
- Clean up in the same RUN; `--no-install-recommends`, `--no-cache-dir`.
- Move big static data to S3 or a volume.
- Add a CI guard that fails the build if the image exceeds a size budget.

Example:

```
docker images myapp
docker history --format "{{.Size}}\t{{.CreatedBy}}" myapp:bad | sort -h | tail -5
# 2.6GB   COPY . .     <- culprit: whole repo incl. data/ and .git
cat .dockerignore
dive myapp:bad

# CI size guard
SIZE=$(docker image inspect myapp:$TAG --format '{{.Size}}')
[ "$SIZE" -gt 524288000 ] && echo "Image > 500MB" && exit 1
```

> ***What to say to the interviewer:*** *I use docker history and dive to find the exact layer that grew, then diff the Dockerfile and .dockerignore against the last good version - usually it is a broken .dockerignore, a heavier base image or leftover build caches. I fix it with multi-stage builds, targeted COPY and same-layer cleanup, and add a size budget check in CI.*

Q33. Your Docker image build time has shot up to 25 minutes. How do you optimize it?

> **In simple words:** If you rewash every dish every time you cook one new dish, dinner takes forever. Reuse what is already clean (cache) and only redo what changed.

Detailed answer

1. **Measure**: build with `--progress=plain` to see which step is slow.
2. **Fix layer order**: copy dependency manifests first, install, then `COPY . .`. If source is copied before the install, every code change reinstalls all dependencies.
3. `.dockerignore`: a huge build context (GBs of `.git`, data) is uploaded every build.
4. **Enable BuildKit** and cache mounts (`--mount=type=cache`) for npm/pip/maven/go caches.
5. **Remote cache in CI**: ephemeral CI agents start empty, so use `--cache-from` / `--cache-to` with a registry cache (ECR) or the CI cache.
6. **Multi-stage parallelism**: independent stages build in parallel with BuildKit.
7. **Pre-built base image** with heavy OS packages, rebuilt weekly instead of every build.
8. **Pin versions** so cache is stable; avoid `apt-get update` busting cache unnecessarily.
9. **Faster runners** / local registry mirror to avoid slow pulls and rate limits.
10. Build only changed services in a monorepo.

Example:

```
# syntax=docker/dockerfile:1
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci
COPY . .
RUN npm run build
```

```
docker buildx build \
  --cache-from type=registry,ref=$REG/app:buildcache \
  --cache-to   type=registry,ref=$REG/app:buildcache,mode=max \
  -t $REG/app:$TAG --push .
```

> ***What to say to the interviewer:*** *I first profile the build with plain progress to find the slow step, then fix layer ordering so dependency installs are cached, trim the build context with .dockerignore, and enable BuildKit cache mounts. In CI I add a registry-backed cache with cache-from and cache-to, which usually brings builds from 25 minutes down to a few minutes.*

Q34. You are building a Docker image but requirements.txt (or package.json) keeps installing an old dependency version. How do you fix it?

> **In simple words:** Docker remembered an old shopping trip and gave you the same old bag instead of going to the shop again. You need to tell it the list changed or force a fresh trip.

Detailed answer

Likely causes:

1. **Layer cache** - the install step was cached. If the file really changed, the `COPY requirements.txt` layer should bust the cache; if it did not, the file used is not the one you edited.
2. **Unpinned versions** (`requests>=2.0`) - with a cached layer you keep the old resolved version; without cache you may get a random new one.
3. **Wrong file copied** - build context path, `.dockerignore`, or a stale file in the repo; `COPY` happens after the install step.
4. **Old base image** already has the package installed (pull a fresh base).
5. **Private mirror / proxy** serving a stale index.
6. CI uses a stale remote cache or an old image tag instead of the new build.

Fix:

- Pin exact versions (`requests==2.32.3`) or use a lock file.
- Copy the dependency file just before the install step.
- Rebuild without cache: `docker build --no-cache` or `--pull` for a fresh base.
- Verify inside the image what version got installed.

Example:

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
docker build --no-cache --pull -t app:fix .
docker run --rm app:fix pip show requests | grep Version
docker run --rm app:fix cat requirements.txt     # confirm the right file got in
```

What to say to the interviewer: *It is almost always a cached layer or unpinned versions, so I pin exact versions with a lock file, make sure the dependency file is copied right before the install step, and rebuild with --no-cache --pull. Then I verify the installed version inside the image before pushing.*

Q35. Docker containers are consuming high CPU on the host. How do you identify the root cause?

In simple words: The electricity bill is huge - go room to room with a meter to find which appliance is burning power, then check why it is running non-stop.

Detailed answer

1. **Which container?** `docker stats --no-stream` - CPU %, memory, per container.
2. **Which process inside?** `docker top <c>` or `docker exec <c> top`; on host, `top`/`htop` and map PID to container with `docker inspect`.
3. **Is it real work or a problem?**

- Traffic spike -> legit, scale out.
- Infinite loop, busy retry loop, crash-restart loop (restarts cost CPU).
- Memory near limit -> heavy GC (Java/Node) shows as CPU.
- Crypto-miner in a compromised image (unknown processes!).

4. **Logs** for errors or retry storms: `docker logs --since 10m`.
5. **Host-level**: `iowait`, steal time on a VM, noisy neighbour.
6. **Profile the app** (thread dump, `perf`, language profiler).
7. **Contain**: set `--cpus` limits so one container cannot starve others; restart or scale; fix code.

Example:

```
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
docker top api
docker exec -it api top -b -n 1 | head -15
docker inspect api --format '{{.RestartCount}}'
docker update --cpus 1.5 api          # cap CPU live
```

What to say to the interviewer: *I use docker stats to find the container, then docker top or top inside it to find the process, and check logs, restart count and memory pressure to decide if it is real load, a loop, GC thrashing or something malicious. I contain it with CPU limits or scaling, then profile and fix the root cause.*

Q36. Docker shows "no space left on device". How do you resolve it?

Also asked as: If Docker shows "no space available", how do you fix it?

In simple words: The storeroom is full of old boxes nobody uses - old images, stopped containers, leftover build scraps and giant log files. Clear them out and set a rule so it doesn't fill again.

Detailed answer

1. **Confirm what is full**: `df -h` (disk) and `df -i` (inodes). Docker data lives in `/var/lib/docker`.
2. **See Docker usage**: `docker system df -v`.
3. **Clean safely**:

- Stopped containers: `docker container prune`

- Dangling/unused images: `docker image prune -a` (removes images not used by any container)

- Build cache: `docker builder prune`

- Unused volumes: `docker volume prune` - careful, this can delete data.

4. **Huge container logs**: `/var/lib/docker/containers/*/*-json.log` can be GBs; truncate, then set log rotation.
5. **Long-term**:

- Log rotation in `/etc/docker/daemon.json`.

- Scheduled prune job on build agents.

- Move Docker data-root to a bigger disk, or grow the EBS volume.

- Disk usage alerts at 80 percent.

Example:

```
df -h /var/lib/docker
docker system df -v
docker container prune -f
docker image prune -a -f --filter "until=168h"
docker builder prune -f
du -sh /var/lib/docker/containers/*/*-json.log | sort -h | tail -3
truncate -s 0 /var/lib/docker/containers/<id>/<id>-json.log
```

```
# /etc/docker/daemon.json
{ "log-driver": "json-file", "log-opts": { "max-size": "50m", "max-file": "3" } }
```

> ***What to say to the interviewer:*** *I confirm with df and docker system df, then prune stopped containers, unused images and build cache, and truncate oversized container logs - being careful with volume prune since it can delete data. Long term I set log rotation in daemon.json, schedule pruning on build agents and alert on disk usage before it reaches 100 percent.*

Q37. The Docker daemon becomes unavailable on a production server. How would you recover?

> **In simple words:** The engine of the machine stopped. First check whether it is out of fuel (disk), overheated (memory), or has a broken setting, then restart it carefully so the running work isn't lost.

Detailed answer

1. **Confirm**: `docker ps` -> "Cannot connect to the Docker daemon". Check `systemctl status docker` and `containerd`.
2. **Read why**: `journalctl -u docker -n 200 --no-pager`.
3. **Common causes and fixes**:

- Disk full on `/var/lib/docker` -> free space (logs, old images), then start.
- Bad `/etc/docker/daemon.json` (JSON typo after a change) -> validate/fix.
- containerd crashed -> `systemctl restart containerd`.
- Socket permission / stale `docker.pid`.
- Host OOM or kernel/storage-driver error.

4. **Restart**: `systemctl restart docker`. With `"live-restore": true`, running containers keep running while the daemon restarts.
5. **Verify**: containers with `--restart unless-stopped` come back; check app health.
6. **If the host is broken**: drain it from the load balancer and replace it (immutable infra) instead of fixing live.
7. **Prevent**: enable live-restore, monitor the docker service and disk, log rotation, test daemon.json changes.

Example:

```
sudo systemctl status docker containerd
sudo journalctl -u docker --since "30 min ago" --no-pager | tail -40
df -h /var/lib/docker
jq . /etc/docker/daemon.json                  # validate JSON
sudo systemctl restart containerd docker
docker ps
```

```
# /etc/docker/daemon.json
{ "live-restore": true, "log-opts": { "max-size": "50m", "max-file": "3" } }
```

> ***What to say to the interviewer:*** *I check systemctl status and journalctl for docker and containerd to find the cause - usually disk full, a bad daemon.json or containerd failure - fix it and restart the service, then verify containers came back via restart policies. For prevention I enable live-restore so containers survive daemon restarts, monitor the service and disk, and replace unhealthy hosts behind the load balancer.*

Q38. A containerised app runs perfectly on a developer machine but crashes in production. What is your debug checklist?

> **In simple words:** The dish tastes great at home but fails in the restaurant. Same recipe? Same ingredients? Same oven settings? Check every difference between the two kitchens.

Detailed answer

1. **Same image?** Compare image digest, not just tag. `latest` or a locally built image may differ from what CI pushed.
2. **CPU architecture**: built on an ARM Mac (arm64) but prod is amd64 -> `exec format error`. Build with `--platform` / buildx multi-arch.
3. **Config and secrets**: missing env vars, wrong `.env`, secrets not injected in prod.
4. **Network and dependencies**: prod DB/API endpoints, security groups, DNS, TLS certificates, proxy.
5. **Resource limits**: prod memory/CPU limit lower -> OOMKilled (exit 137).
6. **Permissions**: prod runs as non-root or read-only filesystem; app writes to a folder it cannot.

7. **Volumes and files**: dev bind-mounts local files that are not in the image.
8. **Versions**: base image, runtime and dependency versions not pinned.
9. **Logs and exit code**: `docker logs`, `docker inspect` in prod.
10. **Reproduce**: run the exact prod image locally with prod-like env and limits.

Example:

```
docker inspect --format '{{index .RepoDigests 0}}' myapp:1.4.2      # compare dev vs prod
docker image inspect myapp:1.4.2 --format '{{.Architecture}}'      # arm64 vs amd64?
docker logs --tail 100 myapp
docker buildx build --platform linux/amd64,linux/arm64 -t $REG/myapp:1.4.2 --push .
docker run --rm --memory 256m --read-only --user 1000 --env-file prod.env myapp:1.4.2
```

> ***What to say to the interviewer:*** *I first confirm it is the same image digest and architecture, then compare everything environmental - env vars and secrets, network access, resource limits, user permissions and mounted files - using the prod logs and exit code. Then I reproduce by running the exact prod image locally with prod-like limits and config.*

Q39. A container is running but not reachable on its port. How do you debug it?

> **In simple words:** The shop is open inside, but customers can't get in. Check: is there a door to the street (port mapping)? Is the shopkeeper listening at the front counter or only in the back room (0.0.0.0 vs 127.0.0.1)? Is a guard blocking the street (firewall / security group)?

Detailed answer

Work from inside out:

1. **Is the app listening inside the container?** `docker exec <c> netstat -tlnp` (or `ss -tlnp`) and `curl localhost:<port>` from inside.
2. **Bound to the right address?** If the app listens on `127.0.0.1`, it is only reachable from inside the container. It must bind to `0.0.0.0`. This is the most common cause.
3. **Is the port published?** `docker ps` / `docker port <c>` - EXPOSE alone does NOT publish. Needs `-p hostPort:containerPort`, and the container port must match the app's real port.
4. **From the host**: `curl localhost:<hostPort>`, `nc -zv localhost <hostPort>`. Port already used by another process? `ss -tlnp | grep <port>`.
5. **Network mode**: with `--network host` there is no `-p` mapping; with `--network none` nothing works. Container-to-container calls should use the container name on a user-defined network, not `localhost`.
6. **Host firewall / iptables**: `ufw`, `firewalld`, or iptables DOCKER chain rules.
7. **Cloud layer**: Security Group / NACL inbound rule for the host port, load balancer target health.
8. **Logs**: `docker logs` - maybe the app failed to start its listener.

Example:

```
docker ps --format "table {{.Names}}\t{{.Ports}}"
# api   3000/tcp            <- NOT published (no 0.0.0.0:8080->3000/tcp)
docker port api
docker exec api ss -tlnp
# LISTEN 0 511 127.0.0.1:3000   <- bound to loopback only, must be 0.0.0.0
curl -v http://localhost:8080/health
nc -zv <server-ip> 8080
sudo iptables -t nat -L DOCKER -n | grep 8080

docker run -d --name api -p 8080:3000 -e HOST=0.0.0.0 myorg/api:1.4.2
```

What to say to the interviewer: *I go inside out - check the app is listening inside the container and bound to 0.0.0.0 rather than 127.0.0.1, confirm the port is actually published with docker port and maps to the right container port, test with curl and nc from the host, and then check the host firewall and cloud security group. The two most common causes are a loopback bind and a missing or mismatched -p mapping.*

Kubernetes

Basic

Q1. Explain Kubernetes: what is it, how does it work, and what does each control-plane component do?

Also asked as: What are the core components of Kubernetes, and which matter most? Explain Kubernetes architecture. How does Kubernetes work? / How do worker nodes communicate with the control plane?

> **In simple words:** Kubernetes is the manager of a big restaurant kitchen. You tell the manager "I always want 3 cooks making pizza", and the manager keeps checking and hires or replaces cooks so there are always exactly 3.

Detailed answer

Kubernetes (K8s) is a container orchestrator. You describe the **desired state** in YAML (for example "3 replicas of my app, image v2"), and Kubernetes runs **control loops** that keep comparing desired state with actual state and fixing the difference.

A cluster has two parts:

- **Control plane (the brain)**
- **Worker nodes (the muscle)** that actually run your Pods

Component	Where	Job in plain words
kube-apiserver	Control plane	The front door. Every kubectl command, controller and kubelet talks only to it. Validates and stores objects.
etcd	Control plane	The cluster's database (key-value). Stores all desired and current state.
kube-scheduler	Control plane	Picks which node a new Pod should run on (based on CPU/memory requests, affinity, taints).
kube-controller-manager	Control plane	Runs controllers (Deployment, ReplicaSet, Node, Job...) that fix drift between desired and actual state.
cloud-controller-manager	Control plane	Talks to AWS/GCP/Azure to create load balancers, attach volumes, etc.
kubelet	Every node	Agent that starts/stops containers for Pods assigned to its node and reports health.
kube-proxy	Every node	Programs iptables/IPVS rules so Service IPs route to Pod IPs.
Container runtime	Every node	containerd/CRI-O actually runs the containers.

Which matter most? The **API server** (nothing works without it) and **etcd** (lose it without a backup and you lose the cluster's state) are the most critical; on the node side, **kubelet** (no kubelet = node NotReady) and **CoreDNS** (no DNS = services cannot find each other).

Flow of `kubectl apply -f deploy.yaml`: kubectl -> API server -> saved in etcd -> Deployment controller creates a ReplicaSet -> ReplicaSet controller creates Pods -> scheduler assigns each Pod to a node -> kubelet on that node pulls the image and starts the container -> kube-proxy makes it reachable via the Service.

Example:

```
kubectl get componentstatuses        # older clusters
kubectl get pods -n kube-system      # coredns, kube-proxy, aws-node (CNI) ...
kubectl get nodes -o wide
```

How worker nodes talk to the control plane:

- Everything goes through the **API server**. Nodes never talk to etcd, the scheduler or the controller manager directly.
- **Node -> control plane:** the kubelet and kube-proxy open outbound TLS connections to the API server (port 443 or 6443) using the node's credentials (client certificate or, on EKS, an IAM-based token). The kubelet **watches** for pods assigned to its node and reports node and pod status back. kube-proxy watches Services and EndpointSlices.
- **Control plane -> node:** the API server calls the kubelet on **port 10250** for `kubectl logs`, `exec`, `port-forward` and metrics. In EKS this path uses cross-account ENIs placed in your subnets; some setups use the Konnectivity proxy instead.
- The node's **lease** object in `kube-node-lease` is its heartbeat. If renewals stop for about 40 seconds, the node controller marks the node NotReady.

```
kubectl get --raw /api/v1/nodes/ip-10-0-1-23/proxy/healthz   # API server -> kubelet 10250
kubectl get lease -n kube-node-lease                          # node heartbeats
```

> ***What to say to the interviewer:*** *Kubernetes is a declarative container orchestrator: I declare desired state and control loops keep reality matching it. The control plane has the API server as the single entry point, etcd as the state store, the scheduler for placement and the controller manager for reconciliation; each worker node runs kubelet, kube-proxy and a container runtime. On EKS, AWS manages the control plane and we manage the node groups.*

Q2. Which component watches your deployment.yaml and keeps the Pods running exactly as defined?

Also asked as: Walk me through what the controller manager does during a Deployment - the reconciliation logic, not rollout status.

> **In simple words:** You hand the order to the front desk (API server), it is written in the notebook (etcd), and a supervisor (controller manager) keeps walking around making sure reality matches the notebook, all day, forever.

Detailed answer

No single component "watches the file". The YAML file is only sent once (`kubectl apply`). After that:

1. **kube-apiserver** validates it and stores the Deployment object in **etcd**.
2. **kube-controller-manager** runs the **Deployment controller**, which watches Deployment objects (via the API server) and creates/updates a **ReplicaSet**.

3. The **ReplicaSet controller** watches ReplicaSets and Pods; if there are fewer Pods than `replicas` (a Pod crashed, a node died), it creates new Pod objects. If there are too many, it deletes some.
4. **kube-scheduler** assigns each new Pod to a node.
5. **kubelet** on that node starts the containers and restarts them if they die (per `restartPolicy` and liveness probes).

This "observe -> compare -> act" cycle is called the **reconciliation loop**. It is why deleting a Pod manually just makes a new one appear.

How the reconciliation actually works inside the controller manager:

- Controllers do not poll; they use **informers** that **watch** the API server and keep a local cache of objects. Every add/update/delete event puts the object's key (`namespace/name`) onto a **work queue**.
- A worker takes a key, reads the **desired state** (Deployment spec) and the **actual state** (ReplicaSets and Pods from the cache) and computes the difference. The logic is **level-based**: it looks at the current state, not at the history of events, so a missed event is fixed on the next sync.
- **Deployment controller:** hashes the Pod template into a `pod-template-hash` label. If a ReplicaSet with that hash exists, it just scales it; if the template changed, it creates a new ReplicaSet with the new hash, then scales new up and old down within `maxSurge`/`maxUnavailable`. It writes progress into Deployment `status` (replicas, updatedReplicas, availableReplicas, conditions `Progressing`/`Available`).
- **ReplicaSet controller:** counts Pods matching its selector (including the hash label), creates or deletes Pods to match `replicas`, and sets **ownerReferences** so the Pods belong to it (garbage collector deletes them when the owner is deleted).
- **Scheduler** binds Pending Pods to nodes; **kubelet** watches Pods bound to its node, runs containers and reports status back. Those status updates generate new events, and the loop continues.
- If something fails (API error, conflict), the key is **re-queued with back-off**, so the system keeps converging.

Example:

```
kubectl delete pod web-7c9b-abcde
kubectl get pods -w          # a new web-7c9b-xxxxx appears within seconds
kubectl get events --sort-by=.lastTimestamp | tail
# ... ReplicaSet web-7c9b  SuccessfulCreate  Created pod: web-7c9b-xxxxx
```

> ***What to say to the interviewer:*** *The API server stores the Deployment in etcd, and the controllers inside kube-controller-manager, specifically the Deployment and ReplicaSet controllers, keep reconciling actual pods against that desired state using informers, work queues and a level-based diff, with the pod-template-hash deciding whether to scale an existing ReplicaSet or create a new one. The scheduler places new pods and kubelet keeps containers running on the node, so a deleted or crashed pod is replaced automatically.*

Q3. What is etcd? Can Kubernetes function without etcd?

> **In simple words:** etcd is the kitchen manager's notebook. If the notebook is lost, the cooks already cooking keep cooking, but nobody can take new orders or remember what the kitchen is supposed to look like.

Detailed answer

etcd is a distributed, consistent key-value store (uses the Raft consensus algorithm). It holds every object: Deployments, Pods, Secrets, ConfigMaps, node status.

No, Kubernetes cannot function properly without etcd. The API server is stateless and reads/writes everything from etcd. If etcd is down:

- Already running Pods usually **keep running** (kubelet and containers do not need etcd moment to moment).
- But you **cannot** create/update/delete anything, scheduling stops, controllers cannot heal failed Pods, and `kubectl` returns errors.

Production etcd runs as an **odd number of members (3 or 5)** so it keeps quorum when one member fails (3 members tolerate 1 failure, 5 tolerate 2). On EKS/GKE/AKS the provider runs etcd for you.

Example:

```
# self-managed cluster: check etcd health
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key endpoint health
```

> ***What to say to the interviewer:*** *etcd is the single source of truth for cluster state, so Kubernetes cannot really work without it. Running workloads keep running for a while, but the API becomes unusable and nothing can be scheduled or healed. That is why etcd runs as 3 or 5 members for quorum and is backed up with regular snapshots.*

Q4. What are Kubernetes objects? What are the common manifest files/objects and what does each do?

Also asked as: How do you create a Kubernetes "class" (object/manifest) in your project? Which objects have you used in your project?

> **In simple words:** Objects are the "order forms" you give Kubernetes. Each form type asks for something different: a running app, a network address, a config file, a disk.

Detailed answer

A Kubernetes object is a persistent record of intent stored in etcd. Every object has `apiVersion`, `kind`, `metadata` (name, namespace, labels) and usually `spec` (what you want) and `status` (what exists).

Object	What it does (one line)
Pod	Smallest unit: one or more containers sharing network (IP) and volumes

Object	What it does (one line)
Deployment	Runs stateless pods via ReplicaSets; rolling updates and rollback
StatefulSet	Pods with stable names, stable DNS and their own PVCs (databases, Kafka)
DaemonSet	One pod on every (or selected) node: log agents, node-exporter, CNI
Job	Runs pods until a task completes successfully (migration, batch)
CronJob	Creates Jobs on a schedule (backups, reports)
Service	Stable IP/DNS name + load balancing to pods selected by labels
Ingress	HTTP/HTTPS host and path routing from outside to Services (needs a controller)
ConfigMap	Non-sensitive configuration as key-values or files
Secret	Sensitive data; **base64-encoded, NOT encrypted by default** - enable KMS encryption at rest and RBAC
PersistentVolumeClaim	A pod's request for storage; bound to a PersistentVolume
HorizontalPodAutoscaler	Scales replica count on CPU/memory/custom metrics
ServiceAccount	Identity for pods (used with RBAC and IRSA)
Role	Namespaced set of permissions (verbs on resources)
ClusterRole	Cluster-wide set of permissions
RoleBinding	Grants a Role/ClusterRole to users, groups or ServiceAccounts in a namespace
NetworkPolicy	Firewall rules for which pods may talk to which
Namespace	Logical partition for teams/apps (RBAC, quotas)
PodDisruptionBudget	Limits how many pods can be voluntarily evicted at once

Project example: each microservice had a Deployment + Service + ConfigMap + Secret + HPA; Kafka/Redis-like stateful pieces used StatefulSets with PVCs; the log/metrics agents (Promtail, node-exporter) ran as DaemonSets so one copy runs on every node; nightly jobs used CronJobs.

Example:

```
kubectl api-resources | head
kubectl get deploy,svc,cm,secret,hpa -n my-app
```

> ***What to say to the interviewer:*** *Objects are the declarative records Kubernetes stores in etcd, each with a spec for desired state and a status for actual state; the everyday set is Pod, Deployment, StatefulSet, DaemonSet, Job, Service, Ingress, ConfigMap, Secret, PVC, HPA and the RBAC objects. In my projects every service had a Deployment, Service, ConfigMap, Secret and HPA, monitoring agents ran as DaemonSets, and stateful components used StatefulSets with PVCs.*

Q5. How do you create a Kubernetes object, and what command do you use to get inside a Pod?

Also asked as: How do you write YAML manifests? What is the command to enter a pod?

> **In simple words:** You can either give Kubernetes spoken instructions one at a time (imperative) or hand it a written plan it keeps following (declarative). Getting into a Pod is like opening a remote terminal into that little computer.

Detailed answer

Creating objects - two styles:

Style	Commands	When
Imperative	`kubectl create deployment`, `kubectl run`, `kubectl expose`, `kubectl scale`	Quick tests, learning, generating YAML
Declarative	`kubectl apply -f file.yaml` (or Helm / Argo CD)	Production: files live in Git, repeatable, reviewable

Every manifest needs four top-level fields: `apiVersion`, `kind`, `metadata` (name, namespace, labels) and `spec`. Use `kubectl explain deployment.spec.template.spec.containers` to look up fields instead of memorizing them, and validate with `kubectl apply --dry-run=server -f`.

`kubectl create -f` fails if the object already exists; `kubectl apply -f` creates or updates it. A handy trick is to generate YAML imperatively and then manage it declaratively.

Getting into a Pod: `kubectl exec -it <pod> -- /bin/sh` (or `bash` if the image has it). Use `-c <container>` for multi-container Pods. Distroless images have no shell, so use `kubectl debug` to attach a temporary debug container.

Example:

```
kubectl create deployment web --image=nginx --dry-run=client -o yaml > web.yaml
kubectl apply -f web.yaml
kubectl apply -f k8s/                 # whole folder
kubectl diff -f web.yaml              # preview changes

kubectl exec -it web-7c9b-abcde -- /bin/sh
kubectl exec -it web-7c9b-abcde -c sidecar -- sh
kubectl exec web-7c9b-abcde -- env | grep DB_
kubectl debug -it web-7c9b-abcde --image=busybox:1.36 --target=web
```

> ***What to say to the interviewer:*** *For production I create objects declaratively with kubectl apply -f from Git, usually through Helm or Argo CD, and use imperative commands only for quick tests or to generate YAML with dry-run. To get inside a pod I use kubectl exec -it pod -- sh, adding -c for a specific container, or kubectl debug when the image has no shell.*

Q6. What is a Deployment, and how is it different from a ReplicaSet?

Also asked as: What is Kubernetes Deployment and ReplicaSet?

> **In simple words:** A ReplicaSet is a supervisor who only counts heads: "there must be 3 workers". A Deployment is the HR manager above him who also handles upgrades: it brings in new workers (v2)

and retires old ones (v1) gradually.

Detailed answer

- **ReplicaSet:** makes sure N identical Pods (matching a label selector) are always running. It knows nothing about versions or upgrades.
- **Deployment:** manages ReplicaSets. When you change the Pod template (new image), it creates a **new ReplicaSet** and scales it up while scaling the old one down (rolling update). It keeps old ReplicaSets for **rollback** (`revisionHistoryLimit`).

Feature	ReplicaSet	Deployment
Keeps N pods running	Yes	Yes (via ReplicaSet)
Rolling update	No	Yes
Rollback / history	No	Yes
Pause / resume rollout	No	Yes

You almost never create ReplicaSets directly; you create Deployments.

Example:

```
kubectl create deployment web --image=nginx:1.25 --replicas=3
kubectl set image deployment/web nginx=nginx:1.27
kubectl get rs           # web-6d4f... (old, 0 pods)  web-7c9b... (new, 3 pods)
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
```

What to say to the interviewer: *A ReplicaSet only guarantees a number of identical pods. A Deployment sits on top, creates a new ReplicaSet for every template change, rolls pods over gradually and keeps revision history so I can roll back with kubectl rollout undo. In practice we always use Deployments.*

Q7. What is the difference between a Deployment and a StatefulSet?

In simple words: Deployment Pods are like identical taxis - any taxi will do and they are interchangeable. StatefulSet Pods are like named employees with their own desk and locker - "mongo-0" always comes back as mongo-0 with the same locker (disk).

Detailed answer

Feature	Deployment	StatefulSet
Pod names	Random (`web-7c9b-x2k4`)	Stable, ordered (`mongo-0`, `mongo-1`)
Network identity	Shared Service IP	Stable DNS per pod via headless Service (`mongo-0.mongo`)
Storage	Usually none or shared PVC	`volumeClaimTemplates`: each pod gets its **own** PVC that follows it
Start/stop order	Parallel, any order	Ordered (0, 1, 2 up; reverse down) by default
Rolling update	Any pod, surge allowed	One by one, highest ordinal first; supports `partition`
Use for	Stateless apps: APIs, web, workers	Databases, Kafka, ZooKeeper, Redis, Elasticsearch

If a StatefulSet pod is deleted, it comes back with the **same name and the same PVC**, so its data survives. Deleting the StatefulSet does **not** delete the PVCs by default (protects data).

Example:

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: mongo}
spec:
  serviceName: mongo            # headless Service (clusterIP: None)
  replicas: 3
  selector: {matchLabels: {app: mongo}}
  template:
    metadata: {labels: {app: mongo}}
    spec:
      containers:
      - name: mongo
        image: mongo:7
        volumeMounts: [{name: data, mountPath: /data/db}]
  volumeClaimTemplates:
  - metadata: {name: data}
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: gp3
      resources: {requests: {storage: 20Gi}}
```

> ***What to say to the interviewer:*** *Deployments are for stateless, interchangeable pods, while StatefulSets give each pod a stable name, stable DNS through a headless Service and its own persistent volume, with ordered startup and updates. I use StatefulSets for databases, Kafka or Redis, and Deployments for APIs and web services.*

Q8. How do you create an Nginx Deployment and expose it through a Service?

> **In simple words:** The Deployment starts the nginx "shops"; the Service puts up one permanent signboard and phone number that forwards customers to whichever shop is open.

Detailed answer

Example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits:   {memory: 256Mi}
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: ClusterIP          # NodePort or LoadBalancer to expose outside
  selector:
    app: nginx             # must match pod labels
  ports:
  - port: 80
    targetPort: 80
```

```
kubectl apply -f nginx.yaml
kubectl get pods -l app=nginx
kubectl get endpoints nginx-svc      # should list 3 pod IPs
kubectl port-forward svc/nginx-svc 8080:80   # test: curl localhost:8080

# imperative quick way
kubectl create deployment nginx --image=nginx --replicas=3
kubectl expose deployment nginx --port=80 --type=LoadBalancer
```

> ***What to say to the interviewer:*** *I write a Deployment with replicas, a label like app=nginx and containerPort 80, then a Service whose selector matches that label. I verify with kubectl get endpoints to confirm pod IPs are attached, and choose ClusterIP for internal, LoadBalancer or Ingress for external traffic.*

Q9. Why are Services necessary, and how does a Service know which Pods to route traffic to?

> **In simple words:** Pods are like food trucks that keep moving and changing their phone numbers. A Service is the fixed call-center number that always knows where the trucks currently are.

Detailed answer

Why Services: Pods are temporary. When a Pod restarts or scales, it gets a **new IP**. Clients cannot hard-code Pod IPs. A Service gives:

- A **stable virtual IP (ClusterIP)** and **DNS name** (`my-svc.my-ns.svc.cluster.local`)
- **Load balancing** across all healthy Pods
- Service discovery for other apps

How it finds Pods: through **labels and selectors**.

1. The Service has `selector: app: web`.
2. The EndpointSlice controller continuously finds Pods with label `app=web` that are **Ready** (readiness probe passing).
3. It writes their IPs into an **EndpointSlice**.
4. kube-proxy on every node programs iptables/IPVS rules: traffic to the ClusterIP -> one of those Pod IPs.

If a Pod fails its readiness probe, it is removed from endpoints automatically, so no traffic reaches it.

Example:

```
kubectl get svc web -o jsonpath='{.spec.selector}'   # {"app":"web"}
kubectl get pods -l app=web -o wide
kubectl get endpointslices -l kubernetes.io/service-name=web
# empty endpoints = selector/label mismatch or pods not Ready
```

> ***What to say to the interviewer:*** *Pods are ephemeral and their IPs change, so a Service provides a stable IP, DNS name and load balancing. It selects pods by label selector; the endpoint controller keeps only Ready matching pods in the EndpointSlice and kube-proxy routes the ClusterIP to them. An empty endpoints list is my first check when a Service does not work.*

Q10. What is the difference between ClusterIP, NodePort and LoadBalancer? How do you expose services in Kubernetes?

Also asked as: How do you expose an application externally? What are the Service types and how does Ingress work?

> **In simple words:** ClusterIP is an internal office extension number, NodePort is a side door on every building with a fixed door number, and LoadBalancer is a proper public reception desk with its own address.

Detailed answer

Type	Reachable from	How	Typical use
ClusterIP (default)	Inside cluster only	Virtual IP + DNS	Service-to-service calls, DBs
NodePort	Outside via `NodeIP:30000-32767`	Opens same port on every node	Testing, on-prem, behind own LB
LoadBalancer	Internet / VPC	Cloud creates an LB (AWS NLB/CLB) pointing to nodes/pods	Expose one service directly
ExternalName	Inside	DNS CNAME to external host	Point to external DB/API
Headless (`clusterIP: None`)	Inside	DNS returns pod IPs directly	StatefulSets (Kafka, MongoDB)

Each type builds on the previous: LoadBalancer creates a NodePort, which creates a ClusterIP.

Ingress is the usual production way: one load balancer + an Ingress controller (NGINX, AWS Load Balancer Controller -> ALB) routes by host/path to many ClusterIP services, with TLS. Cheaper than one LoadBalancer per service.

Example:

```
kubectl expose deploy web --port=80 --type=NodePort
kubectl get svc web        # 80:31234/TCP -> curl http://<node-ip>:31234

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
spec:
  ingressClassName: alb
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web, port: {number: 80}}}
```

> ***What to say to the interviewer:*** *ClusterIP is internal-only, NodePort opens a static port on every node, and LoadBalancer asks the cloud to provision an external load balancer. In production I keep services as ClusterIP and expose them through an Ingress controller with host and path routing and TLS, so many services share one load balancer.*

Q11. What is the difference between an Ingress and a LoadBalancer Service?

> **In simple words:** A LoadBalancer Service is giving every shop in a mall its own street entrance and security guard (expensive). An Ingress is one main mall entrance with a receptionist who reads your ticket and sends you to the right shop.

Detailed answer

	LoadBalancer Service	Ingress
OSI layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP/HTTPS)
Routing	Just forwards a port to pods	By host (`api.x.com`) and path (`/orders`)
Load balancers created	One per Service	One shared LB for many Services
TLS	Usually terminated at app or NLB	Central TLS termination (cert-manager, ACM)
Extras	None	Redirects, rewrites, auth, rate limits, WAF (ALB)
Cost	Grows with number of Services	One LB cost
Needs a controller	No (cloud controller does it)	Yes (ingress-nginx, AWS Load Balancer Controller)

Use a **LoadBalancer Service** for non-HTTP traffic (MQTT, raw TCP, gRPC on NLB, databases for internal clients) or when you need an NLB with static IPs. Use **Ingress** for HTTP web apps and

APIs. The newer **Gateway API** is the successor to Ingress with richer routing.

Example:

```
# L4: one NLB just for this service
apiVersion: v1
kind: Service
metadata:
  name: mqtt
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
spec:
  type: LoadBalancer
  selector: {app: emqx}
  ports: [{port: 1883, targetPort: 1883}]

kubectl get ingress -A     # many hosts/paths, same ADDRESS (one ALB)
```

> ***What to say to the interviewer:*** *A LoadBalancer Service is layer 4 and creates one cloud load balancer per service, while Ingress is layer 7 and lets one load balancer route many hostnames and paths to different services with central TLS. I use Ingress for HTTP APIs to save cost and get path routing, and LoadBalancer or NLB for TCP protocols like MQTT.*

Q12. Why do we use namespaces in Kubernetes?

> **In simple words:** Namespaces are separate floors in the same office building. Each team gets its own floor with its own access cards and its own electricity budget, but they share the building.

Detailed answer

A namespace is a **virtual partition** inside one cluster. Uses:

- **Organization / isolation:** separate teams, apps or environments (`hes`, `mdms`, `monitoring`, `dev`, `staging`). Same object names can exist in different namespaces.
- **Access control:** RBAC Roles/RoleBindings are namespaced, so team A can deploy only in its namespace.
- **Resource limits:** `ResourceQuota` caps total CPU/memory/pod count per namespace; `LimitRange` sets default requests/limits for pods that forget them.
- **Network policy:** NetworkPolicies are applied per namespace, e.g. deny traffic from other namespaces.
- **Cleanup:** `kubectl delete ns feature-x` removes everything in it.

Namespaces are **not** a hard security boundary (same nodes, same kernel); for strong isolation use separate node groups or clusters. Some objects are cluster-wide: Nodes, PersistentVolumes, StorageClasses, ClusterRoles, Namespaces themselves.

Cross-namespace calls use the full name: `svc-name.other-ns.svc.cluster.local`.

Example:

```
kubectl create ns team-a
kubectl apply -n team-a -f - <<EOF
apiVersion: v1
kind: ResourceQuota
metadata: {name: quota}
spec:
  hard: {requests.cpu: "4", requests.memory: 8Gi, pods: "30"}
EOF
kubectl get pods -n team-a
kubectl config set-context --current --namespace=team-a
```

> ***What to say to the interviewer:*** *Namespaces logically divide one cluster by team, app or environment, and they are the unit for RBAC, ResourceQuota, LimitRange and NetworkPolicy. They are not a hard security boundary, so for strict isolation I combine them with separate node groups or separate clusters.*

Q13. What are ConfigMaps and Secrets?

> **In simple words:** A ConfigMap is a notice board with settings anyone can read. A Secret is a locked drawer for passwords. Both let you change settings without rebuilding the app image.

Detailed answer

- **ConfigMap:** non-sensitive config (URLs, feature flags, config files).
- **Secret:** sensitive data (passwords, tokens, TLS certs). Stored **base64-encoded, which is NOT encryption**. You must enable encryption at rest (KMS on EKS) and restrict access with RBAC.

Both can be consumed as **environment variables** or **mounted files**.

Important behavior: env vars are read only at container start, so changing a ConfigMap does **not** update env vars in running Pods (you must restart/roll the Deployment). Mounted files are updated automatically after a short delay (except `subPath` mounts).

Example:

```
kubectl create configmap app-cfg --from-literal=LOG_LEVEL=info
kubectl create secret generic db-cred --from-literal=DB_PASS='S3cr3t!'

spec:
  containers:
  - name: app
    image: myapp:1.0
    envFrom:
    - configMapRef: {name: app-cfg}
    env:
    - name: DB_PASS
      valueFrom:
        secretKeyRef: {name: db-cred, key: DB_PASS}
    volumeMounts:
    - {name: cfg, mountPath: /etc/app}
  volumes:
  - name: cfg
    configMap: {name: app-cfg}

echo 'UzNjcjN0IQ==' | base64 -d   # anyone with read access can decode
```

What to say to the interviewer: *ConfigMaps hold non-sensitive config and Secrets hold sensitive values; both are injected as env vars or mounted files so config is decoupled from the image. Secrets are only base64-encoded, so I enable KMS encryption at rest, lock them down with RBAC, and in production prefer syncing from AWS Secrets Manager or Vault using External Secrets Operator.*

Q14. What is the rolling update strategy?

In simple words: Changing the tyres of a bus one at a time while it is still running, so passengers never have to get off.

Detailed answer

Rolling update (default Deployment strategy) replaces old Pods with new ones **gradually**, so the app stays available. Two knobs:

- `maxSurge`: how many **extra** Pods above desired count can exist during rollout (e.g. 25% or 1).
- `maxUnavailable`: how many Pods can be **down** during rollout. Set `0` for zero downtime.

A new Pod only counts as available when its **readiness probe** passes (plus `minReadySeconds`), so a broken version stops the rollout instead of taking down the app. The other strategy is `Recreate` (kill all, then start new: downtime, used when two versions cannot run together).

Example:

```
spec:
  replicas: 4
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

```
kubectl set image deploy/web web=myapp:2.0
kubectl rollout status deploy/web
kubectl rollout undo deploy/web          # if something is wrong
```

What to say to the interviewer: *A rolling update creates new pods and removes old ones in small batches controlled by maxSurge and maxUnavailable. With maxUnavailable 0 and a proper readiness probe, capacity never drops and a bad version simply stalls the rollout, and I can revert with kubectl rollout undo.*

Q15. How do you auto-scale pods? Explain Horizontal Pod Autoscaling (HPA) and the other types of scaling.

Also asked as: What types of scaling exist in Kubernetes? How do you autoscale pods on CPU plus custom metrics in EKS? How do you implement HPA in AKS?

In simple words: When the queue at a ticket counter gets long, the manager opens more counters; when it gets quiet, he closes some.

Detailed answer

Types of scaling:

Type	What it changes	Trigger
Manual	Replica count	`kubectl scale deploy web --replicas=5`
HPA	Number of pods	CPU, memory, custom/external metrics
VPA	CPU/memory requests of each pod (bigger/smaller pods)	Historical usage; restarts pods to apply
KEDA	Number of pods, including to zero	Events: Kafka lag, SQS depth, cron, Prometheus
Cluster Autoscaler / Karpenter	Number of nodes	Pending pods that do not fit / empty nodes

Do not run HPA and VPA on the same CPU/memory metric for one workload - they fight each other.

HPA automatically changes the **number of Pod replicas** of a Deployment/StatefulSet based on metrics.

How it works:

1. **metrics-server** collects CPU/memory from kubelets every ~15s.
2. The HPA controller checks every 15s and calculates: `desiredReplicas = ceil(currentReplicas * currentMetric / targetMetric)`.
3. It scales within `minReplicas` and `maxReplicas`, with a **scale-down stabilization window** (default 5 min) to avoid flapping.

Requirements: Pods **must have resource requests** set, because CPU utilization % is calculated against the request.

CPU plus custom metrics (e.g. on EKS):

- `metrics-server` serves the **resource metrics API** (CPU/memory) - install it as an EKS add-on.
- **prometheus-adapter** serves the **custom metrics API** (`custom.metrics.k8s.io`) from Prometheus queries, e.g. requests per second per pod -> use a `Pods` metric.
- **KEDA** serves the **external metrics API** for things outside the cluster (SQS depth, Kafka lag, CloudWatch).
- With several metrics in one HPA, it computes replicas for each and uses the **highest**.

AKS specifics: metrics-server is **built in**; the same `autoscaling/v2` HPA YAML works; event-driven scaling via the **AKS KEDA add-on** (`az aks update --enable-keda`); node scaling with the **cluster autoscaler enabled per node pool** (`az aks nodepool update --enable-cluster-autoscaler --min-count 2 --max-count 10`).

HPA adds Pods; if nodes are full, **Cluster Autoscaler or Karpenter** adds nodes. VPA (Vertical) changes the size of Pods instead of the count.

Example:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: web}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: web}
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 70}
  - type: Pods                                # custom metric via prometheus-adapter
    pods:
      metric: {name: http_requests_per_second}
      target: {type: AverageValue, averageValue: "100"}
```

```
kubectl get hpa web     # TARGETS 85%/70%, 140/100  REPLICAS 4 -> scaling up
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1 | head -c 300
```

> ***What to say to the interviewer:*** *Kubernetes offers manual scaling, HPA for pod count, VPA for pod size, KEDA for event-driven scaling and Cluster Autoscaler or Karpenter for nodes. HPA changes replica count based on metrics like CPU against the pod's request, using the formula current replicas times current over target, and custom metrics come from prometheus-adapter. It needs metrics-server and resource requests, and I pair it with Cluster Autoscaler or Karpenter so new pods have nodes to land on. For event-driven metrics like Kafka lag I use KEDA.*

Q16. What are liveness, readiness and startup probes? How are they different and how do you use them?

Also asked as: What are the different types of probes in a Deployment YAML? What is a startup probe? How does Kubernetes decide a pod is ready?

> **In simple words:** Readiness asks "are you ready to serve customers?" (if no, stop sending people). Liveness asks "are you alive?" (if no, restart you). Startup asks "have you finished waking up?" (until yes, do not run the other checks).

Detailed answer

Probe	Question	On failure	Use for
Readiness	Can it take traffic now?	Removed from Service endpoints (not restarted)	Warm-up, dependency down, overloaded
Liveness	Is it stuck/dead?	Container is **restarted**	Deadlocks, hung process
Startup	Has it finished starting?	Restarted after failureThreshold x period	Slow-starting apps (Java, big caches)

While the startup probe has not succeeded, liveness and readiness are disabled, so a slow app is not killed during boot.

How Kubernetes decides a pod is **Ready**: the pod condition `Ready=True` only when **all containers are running and their readiness probes pass** (plus any custom `readinessGates`). A container with no readiness probe is considered ready as soon as it starts. Only Ready pods are added to Service endpoints, and a Deployment counts a pod as available after it stays Ready for `minReadySeconds`. Probe types: `httpGet`, `tcpSocket`, `exec`, `grpc`.

Good practice:

- Liveness should check only the process itself, **not** downstream DBs (otherwise a DB blip restarts every pod).
- Readiness can check critical dependencies.
- Do not make liveness too aggressive; it causes CrashLoopBackOff under load.

Example:

```
startupProbe:
  httpGet: {path: /healthz, port: 8080}
  failureThreshold: 30
  periodSeconds: 5          # up to 150s to start
readinessProbe:
  httpGet: {path: /ready, port: 8080}
  periodSeconds: 5
  failureThreshold: 3
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```

> ***What to say to the interviewer:*** *Readiness controls traffic, liveness controls restarts, and startup protects slow-booting apps by delaying the other two. I keep liveness light and dependency-free, put dependency checks in readiness, and use a startup probe instead of a huge initialDelaySeconds.*

Q17. Explain the Kubernetes Pod lifecycle.

> **In simple words:** A pod's life is like a flight: waiting at the gate (Pending), boarding checks (init containers), flying (Running), landing normally (Succeeded) or crashing (Failed). When it is time to land, the crew gets a warning (SIGTERM) and some time to finish before the doors are forced open (SIGKILL).

Detailed answer

Pod phases (`status.phase`):

Phase	Meaning
Pending	Accepted, but not all containers are running yet: waiting for scheduling, image pull, volume attach
Running	Bound to a node, at least one container running (or starting/restarting)
Succeeded	All containers exited with 0 and will not restart (Jobs)
Failed	All containers stopped and at least one exited non-zero / was killed
Unknown	Node not reporting, state cannot be obtained

Note: `CrashLoopBackOff`, `ImagePullBackOff`, `OOMKilled` are **container state reasons**, not phases. Container states are **Waiting**, **Running**, **Terminated**.

Startup order:

1. Scheduled to a node -> volumes mounted, images pulled.

2. **Init containers** run one by one, each must succeed (e.g. wait for DB, run migrations, fetch config).
3. App containers start; **postStart** hook runs; **startup probe**, then readiness/liveness probes.
4. Ready -> added to Service endpoints.

Graceful termination:

1. Pod marked Terminating and removed from endpoints (in parallel).
2. **preStop** hook runs (e.g. `sleep 10`).
3. **SIGTERM** sent to the container's main process (PID 1).
4. Kubernetes waits up to `terminationGracePeriodSeconds` (default **30s**, counting the preStop time).
5. Still running -> **SIGKILL** (exit code 137).

`restartPolicy`: Always (Deployments), OnFailure / Never (Jobs).

Example:

```
spec:
  terminationGracePeriodSeconds: 40
  initContainers:
  - name: wait-db
    image: busybox:1.36
    command: ["sh", "-c", "until nc -z postgres 5432; do sleep 2; done"]
  containers:
  - name: app
    image: myapp:1.0
    lifecycle:
      preStop: {exec: {command: ["sh", "-c", "sleep 10"]}}

kubectl get pod app-xyz -o jsonpath='{.status.phase}'
kubectl get pod app-xyz -o jsonpath='{.status.containerStatuses[0].state}'
```

> ***What to say to the interviewer:*** *A pod moves through Pending, Running, then Succeeded or Failed, with Unknown when the node stops reporting; init containers run to completion first, then app containers with hooks and probes. On deletion it is removed from endpoints, preStop runs, SIGTERM is sent, and after terminationGracePeriodSeconds, 30 by default, it gets SIGKILL, so apps must handle SIGTERM gracefully.*

Q18. What are PV and PVC? Which comes first, and why?

> **In simple words:** A PV is an actual storage locker; a PVC is a request slip saying "I need a 10 GB locker". Either the building manager pre-builds lockers (static), or a locker factory (StorageClass) builds one the moment a slip arrives (dynamic).

Detailed answer

- **PersistentVolume (PV):** a piece of real storage in the cluster (EBS volume, EFS, NFS, disk). Cluster-scoped, managed by admins or created automatically.
- **PersistentVolumeClaim (PVC):** a namespaced **request** by a user/pod for storage: size, access mode, StorageClass. The pod mounts the PVC, not the PV.
- **StorageClass:** describes how to create storage dynamically (provisioner `ebs.csi.aws.com`, type gp3, reclaim policy, `allowVolumeExpansion`, `volumeBindingMode`).

Which comes first?

- **Static provisioning:** admin creates the **PV first**, then a PVC claims it (binding by size, access mode, class, selector).
- **Dynamic provisioning (most common in cloud):** you create the **PVC first**; the StorageClass provisioner creates the PV automatically and binds it. With `WaitForFirstConsumer`, the volume is created only when a pod is scheduled, so it lands in the pod's AZ.

Binding is 1-to-1. Access modes: RWO (one node), ROX, RWX (many nodes, e.g. EFS), RWOP. Reclaim policy: **Delete** (volume removed with PVC) or **Retain** (keep data for manual recovery - use for important data).

Example:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: {name: gp3}
provisioner: ebs.csi.aws.com
parameters: {type: gp3}
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources: {requests: {storage: 20Gi}}

kubectl get pvc,pv       # STATUS Bound
```

> ***What to say to the interviewer:*** *A PV is the actual storage and a PVC is a namespaced request for it that pods mount. With static provisioning the admin creates the PV first; in the cloud we normally use dynamic provisioning where the PVC comes first and a StorageClass such as gp3 on the EBS CSI driver creates and binds the PV, ideally with WaitForFirstConsumer and Retain for important data.*

Q19. What is RBAC in Kubernetes, what is it used for, and how do you configure it?

> **In simple words:** RBAC is the building's access-card system. A Role says which doors exist on a card ("can open deployment and pod doors on floor prod"); a RoleBinding hands that card to a person or robot.

Detailed answer

RBAC (Role-Based Access Control) decides **who can do what on which resources**. Used for least privilege for developers, CI/CD, and applications.

Building blocks:

Object	Scope	Purpose
Role	Namespace	List of allowed verbs on resources in one namespace
ClusterRole	Cluster	Same, cluster-wide or for cluster-scoped resources (nodes, PVs); can be reused in namespaces
RoleBinding	Namespace	Gives a Role/ClusterRole to subjects inside one namespace
ClusterRoleBinding	Cluster	Gives a ClusterRole everywhere
Subjects	-	User, Group (from IAM/OIDC), ServiceAccount (for pods and CI)

Verbs: get, list, watch, create, update, patch, delete. RBAC is **additive only** (no deny rules); anything not granted is denied.

Best practices: namespace-scoped Roles, groups instead of individual users, no wildcard `*`, avoid giving `secrets` list/get and `pods/exec` broadly, separate ServiceAccount per app, audit with `kubectl auth can-i`.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: dev-read-deploy, namespace: team-a}
rules:
- apiGroups: ["", "apps"]
  resources: ["pods", "pods/log", "deployments"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: devs-read, namespace: team-a}
subjects:
- {kind: Group, name: team-a-devs, apiGroup: rbac.authorization.k8s.io}
roleRef: {kind: Role, name: dev-read-deploy, apiGroup: rbac.authorization.k8s.io}

kubectl auth can-i delete pods -n team-a --as-group=team-a-devs --as=test   # no
kubectl auth can-i --list --as=system:serviceaccount:team-a:api -n team-a
```

> ***What to say to the interviewer:*** *RBAC controls who can perform which verbs on which resources: Roles and ClusterRoles define permissions, and RoleBindings or ClusterRoleBindings grant them to users, groups or ServiceAccounts. I keep it least-privilege and namespace-scoped, map IAM or SSO groups to Kubernetes groups, and verify with kubectl auth can-i.*

Q20. What does Kubernetes administration mean to you?

> **In simple words:** A Kubernetes admin is the building manager, not a tenant. Tenants (developers) run their shops; the manager keeps the lifts, power, security, water and fire exits working, plans for growth, and handles emergencies.

Detailed answer

It is owning the **platform** so application teams can deploy safely:

Area	What it covers
Cluster lifecycle	Provisioning with IaC (Terraform/eksctl), version upgrades one minor at a time, version skew between control plane, kubelet and kubectl, deprecated APIs
Node management	Node groups / Karpenter, AMI updates, cordon/drain, taints and labels, capacity planning
Access & RBAC	SSO/IAM mapping (access entries), namespaced Roles, ServiceAccounts, IRSA/Pod Identity, least privilege
Networking	CNI (IPs, subnets), Ingress controllers, load balancers, CoreDNS, NetworkPolicies, TLS (cert-manager)
Storage	StorageClasses, CSI drivers (EBS/EFS), PVC expansion, reclaim policies, snapshots
Add-ons	CNI, CoreDNS, kube-proxy, metrics-server, CSI, autoscaler, External Secrets - kept on compatible versions
Security	Pod Security Admission, Gatekeeper/Kyverno policies, image scanning, secret encryption (KMS), audit logs
Observability	Prometheus/Grafana metrics, Loki/ELK logs, tracing, alerting with runbooks
Backup & DR	etcd snapshots (self-managed), Velero, GitOps rebuild, restore drills
Capacity & cost	Requests/limits, ResourceQuota/LimitRange, rightsizing, Spot, autoscaling
Incident response	On-call, troubleshooting nodes/pods/network, postmortems

Example:

```
kubectl get nodes -o wide ; kubectl top nodes
kubectl get pods -A | grep -vE "Running|Completed"
kubectl version ; aws eks list-addons --cluster-name prod-eks
kubectl auth can-i --list --as=system:serviceaccount:prod:api -n prod
kubectl get resourcequota -A ; kubectl get pdb -A
velero backup get
```

What to say to the interviewer: *To me Kubernetes administration means owning the platform end to end: cluster provisioning and upgrades, node management, RBAC and identity, networking, ingress and DNS, storage classes, add-ons, security policies, observability, backup and DR, capacity and cost, and incident response. The goal is that application teams can deploy self-service and safely without worrying about the cluster underneath.*

Q21. Rapid-fire: Kubernetes concept checks (common trick questions)

In simple words: These are short "gotcha" questions where the popular answer is slightly wrong. Knowing the precise answer shows real understanding.

Detailed answer

Question	Correct answer	Why
Two nodes, one has pods, one is empty. Where does a new pod go?	Usually the emptier node - it is not random	Scheduler first **filters** (enough requested CPU/memory, taints, affinity, ports), then **scores**; default scoring (LeastAllocated, BalancedAllocation, topology spread) favours less-allocated nodes. It uses **requests**, not actual usage
A container is OOMKilled. Is the container or the whole pod restarted?	Only that **container** is restarted (per `restartPolicy`, default Always)	The pod object, its IP and volumes stay; pod is only replaced if it is evicted (node memory pressure) or deleted
Can env vars / ConfigMap values update without recreating the pod?	Env vars: **no**, need a restart. ConfigMap mounted as a volume: **eventually** (kubelet sync, up to about a minute plus cache TTL), never for `subPath` mounts, and the app must re-read the file	Immutable ConfigMaps never change; common pattern is a checksum annotation or Reloader to trigger a rollout
Once a pod is created, is it stable?	**No**, pods are ephemeral	Node failure, eviction, preemption, drains and rollouts replace them - that is why controllers and Services exist
Does a ClusterIP Service load-balance TCP?	**Yes, per connection (L4)**, not per request	kube-proxy iptables picks a random endpoint, IPVS supports algorithms; long-lived connections (gRPC/HTTP2 keep-alive) stick to one pod - use L7 balancing or a service mesh
How do you collect app logs, and can they be lost?	App writes to **stdout/stderr**; a **DaemonSet agent** (Fluent Bit, Fluentd, Promtail/Alloy) ships them to Loki/ELK/CloudWatch. Prometheus is for **metrics, not logs**	Node-local logs are lost when the pod is deleted or the node dies; `kubectl logs` only keeps current and previous container, within rotation limits
Liveness probe passes - is the app fine?	**No**, it only proves what the probe checks (often just "process responds")	Dependencies and correctness can be broken; use readiness probes, SLO metrics and synthetic checks
How does the app scale with traffic?	**HPA** for pods + **Cluster Autoscaler/Karpenter** for nodes; **KEDA** for event-driven	HPA alone cannot help if no node has room
Is `kubectl exec -it pod -- bash` "logging into the pod"?	Not exactly: it starts a **new process inside one container's namespaces** via API server -> kubelet -> container runtime	Needs a shell in the image (distroless has none -> use `kubectl debug` ephemeral container); use `-c` to pick the container
A container keeps exiting and restarting. What do you do?	`kubectl logs --previous`, `kubectl describe` (Last State, exit code), events, probes, resources	Exit 137 = killed (OOM/SIGKILL), 1 = app error, 0 = main process finished (wrong command for a long-running pod)

Example:

```
kubectl describe pod api-x | grep -A5 "Last State"
kubectl get pod api-x -o jsonpath='{.status.containerStatuses[0].restartCount}'
kubectl debug -it api-x --image=busybox:1.36 --target=api
```

What to say to the interviewer: *These checks are about precision: the scheduler scores nodes by requests and usually picks the less-allocated one, an OOM kill restarts only the container, env vars need a restart while mounted ConfigMaps update eventually, ClusterIP balances per connection not per request, logs go through a DaemonSet shipper and not Prometheus, and a passing liveness probe does not prove the app is healthy.*

Q22. What is the NodePort port range?

In simple words: Kubernetes reserves a block of high-numbered side doors on every node for NodePort services, so they never clash with normal ports like 22 or 443.

Detailed answer

- Default NodePort range: **30000-32767** (TCP/UDP).
- If you do not specify `nodePort`, Kubernetes picks a free one from that range; you can set one explicitly within the range.
- The same port is opened on **every node**, and traffic to any node's IP on that port reaches the Service (even if no pod runs on that node, unless `externalTrafficPolicy: Local`).
- The range can be changed with the API server flag `--service-node-port-range` (not possible on managed EKS/GKE control planes).
- Remember to allow the range in Security Groups/firewalls if clients connect directly.

Example:

```
apiVersion: v1
kind: Service
metadata: {name: web-np}
spec:
  type: NodePort
  selector: {app: web}
  ports:
  - {port: 80, targetPort: 8080, nodePort: 30080}

curl http://<any-node-ip>:30080
```

What to say to the interviewer: *The default NodePort range is 30000 to 32767, configurable with the API server's service-node-port-range flag on self-managed clusters. The port opens on every node, so firewalls or security groups must allow it, and externalTrafficPolicy Local keeps the client IP and avoids an extra hop.*

Q23. What is an Ingress, and how do you expose applications on an on-premises Kubernetes cluster? Do LoadBalancer Services work on-prem?

In simple words: In the cloud, asking for a LoadBalancer is like calling a taxi company - one arrives. On-prem there is no taxi company, so you either run your own taxi service (MetalLB) or use a side door (NodePort) behind your own gatekeeper (HAProxy/F5).

Detailed answer

Ingress is an API object with HTTP/HTTPS routing rules (host, path, TLS). It does nothing alone: an **Ingress controller** (ingress-nginx, Traefik, HAProxy, AWS LB Controller) reads the rules and routes traffic to ClusterIP Services.

On-prem, LoadBalancer Services do not work out of the box - there is no cloud-controller to create an LB, so the Service stays `EXTERNAL-IP <pending>`. Options:

1. **MetalLB** (or **kube-vip**): assigns IPs from a pool you give it on your LAN and announces them via L2 (ARP) or BGP. Then `type: LoadBalancer` works like in the cloud.
2. **NodePort + external load balancer:** HAProxy/NGINX/F5 in front of all nodes on the NodePort (e.g. `node1:30080`, `node2:30080`).
3. **Ingress controller on hostNetwork / DaemonSet** listening on 80/443 of edge nodes, with DNS or a VIP (keepalived) pointing at them.
4. Then create **Ingress** resources for each app, TLS via cert-manager.

Typical design: MetalLB gives one IP to the ingress-nginx controller Service, and all apps are exposed through Ingress rules behind it.

Example:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata: {name: lan-pool, namespace: metallb-system}
spec: {addresses: ["192.168.10.240-192.168.10.250"]}
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata: {name: l2, namespace: metallb-system}

kubectl get svc -n ingress-nginx ingress-nginx-controller
# TYPE LoadBalancer  EXTERNAL-IP 192.168.10.240   (instead of <pending>)
```

> ***What to say to the interviewer:*** *An Ingress defines host and path routing that an ingress controller like NGINX implements. On-prem, LoadBalancer Services stay pending because there is no cloud provider, so I install MetalLB or kube-vip to hand out LAN IPs, or put HAProxy or F5 in front of NodePorts, and then expose apps through an ingress controller with cert-manager for TLS.*

Q24. Can we define multiple ports in a Kubernetes Deployment? How do you verify they are correctly defined and working?

> **In simple words:** One shop can have a front door for customers and a back door for deliveries. You just have to label each door clearly.

Detailed answer

Yes. A container can list multiple `containerPorts`, and a Service can expose multiple ports. When a Service has more than one port, **each port must have a name**.

Note: `containerPort` is mostly informational; the app must actually listen on that port. Verification means checking the app, not just the YAML.

Example:

```
containers:
- name: app
  image: myapp:1.0
  ports:
  - {name: http, containerPort: 8080}
  - {name: metrics, containerPort: 9090}
---
kind: Service
spec:
  selector: {app: myapp}
  ports:
  - {name: http, port: 80, targetPort: http}
  - {name: metrics, port: 9090, targetPort: metrics}
```

Verify:

```
kubectl get pod <pod> -o jsonpath='{.spec.containers[*].ports}'
kubectl describe svc myapp              # both ports + Endpoints for each
kubectl get endpoints myapp             # 10.0.1.5:8080,10.0.1.5:9090
kubectl exec <pod> -- ss -lntp          # is the app really listening on both?
kubectl port-forward pod/<pod> 8080 9090
curl localhost:8080/health ; curl localhost:9090/metrics
```

> ***What to say to the interviewer:*** *Yes, a container can declare multiple named ports and the Service must name each port when there is more than one. I verify with describe svc and get endpoints, then exec ss -lnt inside the pod and port-forward plus curl to prove the app actually listens on each port.*

Q25. The team asks you to pass an extra startup parameter to an app deployed from a Docker image. Where do you define the startup command in Kubernetes?

> **In simple words:** The Docker image comes with a default "how to start" instruction. In Kubernetes you can overwrite it in the Pod spec without rebuilding the image.

Detailed answer

In the container spec:

- `command` overrides the image's **ENTRYPOINT**
- `args` overrides the image's **CMD**

To add a parameter while keeping the image's entrypoint, usually set only `args`. Changing `command`/`args` in a Deployment changes the Pod template, so a rolling update happens automatically.

Dockerfile	Kubernetes field
ENTRYPOINT	command
CMD	args

Example:

```
containers:
- name: api
  image: myapi:1.4
  command: ["java"]                              # optional: replaces ENTRYPOINT
  args: ["-Xmx512m", "-jar", "/app/api.jar", "--log-level=debug"]
  env:
  - name: EXTRA_OPTS
    value: "--feature-x=true"
```

```
kubectl edit deploy api        # or change YAML/Helm values and apply
kubectl get pod <pod> -o jsonpath='{.spec.containers[0].args}'
```

> ***What to say to the interviewer:*** *I define it in the container spec: command overrides the Docker ENTRYPOINT and args overrides CMD. For an extra flag I usually add it to args, apply through Git or Helm, and the Deployment rolls out new pods automatically.*

Q26. What is CoreDNS, and what role does it play in Kubernetes? How does DNS resolution work inside a pod?

Also asked as: How does DNS resolution work inside a pod, and what do you check when a service is not reachable by name?

> **In simple words:** CoreDNS is the cluster's phone directory. Apps ask "what is the number for payment-service?" and CoreDNS answers with the Service IP.

Detailed answer

CoreDNS is the cluster DNS server (runs as a Deployment in `kube-system`, exposed by the `kube-dns` Service with a fixed ClusterIP, e.g. `10.100.0.10` on EKS). Every Pod's `/etc/resolv.conf` points to it.

Resolution inside a pod, step by step:

1. kubelet writes the pod's `/etc/resolv.conf` (with the default `dnsPolicy: ClusterFirst`): `nameserver <kube-dns ClusterIP>`, `search <ns>.svc.cluster.local svc.cluster.local cluster.local` (+ VPC domain), `options ndots:5`.
2. The app looks up `payments`. Because the name has fewer than 5 dots, the resolver first tries each **search domain**: `payments.<ns>.svc.cluster.local` -> found if the Service is in the same namespace.
3. The query goes to the kube-dns ClusterIP; kube-proxy DNATs it to one CoreDNS pod.
4. CoreDNS's **kubernetes plugin** answers from its watch of Services/EndpointSlices (ClusterIP for normal Services, pod IPs for headless).
5. Names outside `cluster.local` go to the **forward plugin** -> upstream (node's `/etc/resolv.conf`, i.e. the VPC resolver on AWS).

Corefile plugins you should know: `kubernetes` (cluster records), `forward` (upstream), `cache` (caches answers, e.g. 30s), `loop` (detects forwarding loops and stops CoreDNS - common crash cause when the node resolv.conf points back to itself), `errors`, `health`/`ready` (probes), `prometheus` (metrics on :9153), `reload`, `loadbalance`.

It resolves:

- Services: `my-svc.my-namespace.svc.cluster.local` -> ClusterIP
- Headless Services / StatefulSet pods: `mongo-0.mongo.db.svc.cluster.local` -> pod IP
- External names: forwards to upstream DNS (VPC resolver on AWS)

Within the same namespace, a Pod can just call `http://my-svc`. Because of `ndots:5`, short external names cause several extra lookups; high DNS load is a common source of latency, fixed by scaling CoreDNS or using NodeLocal DNSCache.

If CoreDNS is down, service-to-service calls fail with "could not resolve host", even though Pods and Services are healthy.

Example:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl run dnstest --rm -it --image=busybox:1.36 -- nslookup my-svc.my-ns
kubectl exec <pod> -- cat /etc/resolv.conf
# nameserver 10.100.0.10
# search prod.svc.cluster.local svc.cluster.local cluster.local ap-south-1.compute.internal
# options ndots:5
kubectl get svc kube-dns -n kube-system
kubectl get cm coredns -n kube-system -o yaml
#   .:53 { errors; health; ready
#          kubernetes cluster.local in-addr.arpa ip6.arpa { pods insecure; fallthrough in-addr.arpa
    ip6.arpa }
#          prometheus :9153; forward . /etc/resolv.conf; cache 30; loop; reload; loadbalance }
kubectl logs -n kube-system -l k8s-app=kube-dns
```

When a service is not reachable by name, check: short name vs `svc.ns` vs FQDN, the pod's resolv.conf and dnsPolicy, CoreDNS pods/logs, whether the Service exists with endpoints, and NetworkPolicy allowing port 53 (see the scenario questions on cross-namespace discovery and cluster-wide DNS failures).

> ***What to say to the interviewer:*** *CoreDNS is the cluster DNS that turns Service names into ClusterIPs and forwards external names upstream. Inside a pod, resolv.conf points to the kube-dns ClusterIP with namespace search domains and ndots:5, CoreDNS's kubernetes plugin answers cluster names and the forward plugin sends the rest upstream, with cache and loop plugins in the Corefile. When names fail I check the FQDN, resolv.conf, CoreDNS pods and logs and a busybox nslookup test.*

Q27. What is Helm and why is it important? Explain the Helm chart folder structure and the commands you use to deploy.

Also asked as: What is the folder structure of a Helm chart? Which Helm commands do you use?

> **In simple words:** Helm is like an app store and installer for Kubernetes. Instead of copying 10 YAML files and editing them by hand for each environment, you install one package and pass a small settings file.

Detailed answer

Helm is the package manager for Kubernetes.

- **Chart:** a package of templated YAML (Deployment, Service, Ingress, HPA...).
- **values.yaml:** the inputs (image tag, replicas, resources) - one file per environment.
- **Release:** an installed instance of a chart in a cluster, with **revision history**.

Why it matters:

- One chart, many environments (`values-dev.yaml`, `values-prod.yaml`) - DRY.
- Versioned releases and one-command **rollback**.
- Install third-party tools easily (ingress-nginx, Prometheus, cert-manager).
- Works well with Argo CD/Jenkins for GitOps/CI-CD.

Chart folder structure:

```
myapp/
  Chart.yaml          # chart name, version (chart), appVersion (app), dependencies
  values.yaml         # default inputs: image, replicas, resources, ingress...
  values-prod.yaml    # (our convention) per-environment overrides
  charts/             # dependency sub-charts (e.g. redis) pulled by helm dependency update
  templates/
    deployment.yaml   # uses {{ .Values.image.tag }}, {{ .Release.Name }}
    service.yaml
    ingress.yaml
    hpa.yaml
    configmap.yaml
    _helpers.tpl      # reusable named templates: names, labels (not rendered itself)
    NOTES.txt         # message printed after install
  .helmignore
```

Commands I use:

Command	Purpose
`helm lint ./myapp`	Check chart for errors
`helm template myapp ./myapp -f values-prod.yaml`	Render YAML locally to review
`helm install` / `helm upgrade --install`	First install / idempotent install-or-upgrade (used in CI)
`helm diff upgrade ...` (plugin)	Show what will change before upgrading
`helm list -n prod`, `helm history myapp -n prod`	See releases and revisions
`helm rollback myapp 3 -n prod`	Go back to revision 3
`helm uninstall myapp -n prod`	Remove release

Example:

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm create myapp                          # scaffold a chart
helm lint ./myapp
helm template myapp ./myapp -f values-prod.yaml   # render and review YAML
helm upgrade --install myapp ./myapp -n prod --create-namespace \
  -f values-prod.yaml --set image.tag=1.4.2 --atomic --timeout 5m
helm history myapp -n prod
helm rollback myapp 3 -n prod

# templates/deployment.yaml snippet
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
replicas: {{ .Values.replicaCount }}
```

> ***What to say to the interviewer:*** *Helm packages Kubernetes manifests into versioned, templated charts: Chart.yaml holds metadata, values.yaml the inputs, templates the YAML with placeholders*

and _helpers.tpl reusable snippets. In CI I run helm lint and helm template, then helm upgrade --install with an environment values file and --atomic so a failed upgrade rolls back automatically, and helm rollback for manual reverts.

Q28. What is KEDA, and how does it help with event-driven autoscaling?

In simple words: Normal HPA looks at how tired the workers are (CPU). KEDA looks at how many parcels are waiting at the door (queue length) and hires workers before they get tired - and can send everyone home when there are zero parcels.

Detailed answer

KEDA (Kubernetes Event-Driven Autoscaling) scales workloads based on **external event sources**: Kafka consumer lag, SQS queue depth, RabbitMQ, Redis lists, Prometheus queries, cron schedules and 60+ others.

How it works:

- You create a `ScaledObject` pointing at a Deployment and a trigger.
- KEDA acts as a **metrics adapter** and creates/manages an HPA under the hood.
- It can **scale to zero** when there are no events and back to 1 when events arrive (plain HPA minimum is 1).

Great for consumers/workers where CPU is a poor signal (a consumer can be idle on CPU while lag grows).

Example:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: {name: order-consumer}
spec:
  scaleTargetRef: {name: order-consumer}
  minReplicaCount: 0
  maxReplicaCount: 30
  triggers:
  - type: kafka
    metadata:
      bootstrapServers: kafka:9092
      consumerGroup: orders
      topic: orders
      lagThreshold: "100"      # ~1 pod per 100 messages of lag
```

What to say to the interviewer: *KEDA extends HPA to scale on event sources such as Kafka lag, SQS depth or Prometheus queries, and it can scale to zero. I use it for queue consumers where CPU does not reflect backlog, and cap maxReplicaCount at the topic's partition count because extra consumers would sit idle.*

Q29. How many nodes are there in your Kubernetes cluster? How do you decide?

In simple words: There is no magic number - it is like asking how many buses a school needs. It depends on how many students, how big the buses are, and whether you keep a spare.

Detailed answer

Answer honestly with your numbers and the reasoning. A typical answer structure:

- **Control plane:** managed by AWS in EKS (we do not see those nodes); self-managed clusters use 3 control-plane nodes for etcd quorum.
- **Worker nodes:** spread across at least **3 AZs** so one AZ failure does not take down the app; minimum 2-3 per node group for HA.
- Separate **node groups** for different workloads (general apps, memory-heavy, isolated workloads) using taints/labels.
- Count is **not fixed** - Cluster Autoscaler/Karpenter scales between min and max.

Sizing logic: sum of Pod requests + ~20-30% headroom + DaemonSet overhead + the per-node Pod limit (on EKS the VPC CNI limits Pods per node by ENIs/IPs).

Example:

```
kubectl get nodes -L topology.kubernetes.io/zone,eks.amazonaws.com/nodegroup
# NAME                STATUS  ZONE         NODEGROUP
# ip-10-0-1-12...     Ready   ap-south-1a  app-ng
# ip-10-0-2-40...     Ready   ap-south-1b  app-ng
# ip-10-0-3-77...     Ready   ap-south-1c  pull-backend-ng
kubectl top nodes
```

> ***What to say to the interviewer:*** *Our EKS control plane is managed by AWS, and worker nodes run in managed node groups spread over three AZs, autoscaling roughly between a minimum and maximum rather than a fixed number. We size from total pod requests plus headroom, and keep separate node groups for workloads that need isolation.*

Q30. How do you deploy an application in a Kubernetes (EKS) cluster? Explain the end-to-end flow.

> **In simple words:** Developer writes code -> a factory (CI) packs it into a box (image) -> the box goes to a warehouse (ECR) -> the delivery order (manifest/Helm) tells Kubernetes to use the new box -> Kubernetes swaps old boxes for new ones gradually.

Detailed answer

Typical flow:

1. Developer pushes code / merges PR to Git.
2. CI (Jenkins/GitHub Actions) runs tests, builds Docker image, scans it (Trivy), tags it with version/commit SHA.
3. Pushes image to **ECR**.
4. CD updates the image tag in Helm values or manifests:

- Push-based: Jenkins runs `helm upgrade` / `kubectl apply` using a kubeconfig/IAM role.
- GitOps: commit the new tag to a config repo; **Argo CD** syncs it to the cluster.

5. API server stores the new spec; Deployment creates a new ReplicaSet; rolling update starts.
6. Scheduler places Pods; kubelet pulls image from ECR (node IAM role permits it).
7. Readiness probe passes -> Pod added to Service endpoints -> Ingress/ALB sends traffic.
8. Verify: rollout status, dashboards, error rates; rollback if bad.

Example:

```
docker build -t 1234.dkr.ecr.ap-south-1.amazonaws.com/api:1.4.2 .
docker push 1234.dkr.ecr.ap-south-1.amazonaws.com/api:1.4.2
aws eks update-kubeconfig --name prod-eks --region ap-south-1
helm upgrade --install api ./charts/api -n prod --set image.tag=1.4.2 --atomic
kubectl rollout status deploy/api -n prod
```

What to say to the interviewer: *CI builds, tests, scans and pushes a versioned image to ECR; CD updates the image tag in Helm values and either Jenkins runs helm upgrade or Argo CD syncs from Git. Kubernetes then performs a rolling update gated by readiness probes, and I verify with rollout status and metrics, rolling back with helm rollback if error rates rise.*

Q31. When deploying to Kubernetes, do you only update Docker images, or also replicas, storage and CPU allocation?

In simple words: Upgrading a shop is not just new stock (image). Sometimes you add staff (replicas), a bigger storeroom (storage) or more power (CPU/memory).

Detailed answer

All of them, but through the same Git/Helm process, and each behaves differently:

Change	How	Effect
Image tag	values.yaml `image.tag`	Rolling update
Replicas	`replicas` or better HPA min/max	No restart, just add/remove pods (do not hard-set replicas if HPA manages them)
CPU/memory requests/limits	`resources`	Rolling update (pods recreated); resize from real usage (VPA recommendations, Grafana)
Storage	PVC `spec.resources.requests.storage`	Can only **grow**, and only if StorageClass has `allowVolumeExpansion: true`
Config	ConfigMap/Secret	Needs pod restart unless mounted as files (use checksum annotation)

Example:

```
# values-prod.yaml
image: {tag: "1.4.2"}
autoscaling: {minReplicas: 3, maxReplicas: 15}
resources:
  requests: {cpu: 250m, memory: 512Mi}
  limits: {memory: 1Gi}

kubectl patch pvc data-mongo-0 -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'
```

What to say to the interviewer: *Besides images, we tune replicas or HPA bounds, CPU and memory requests based on observed usage, and expand PVCs when storage grows. All changes go through Helm values in Git so they are reviewed and versioned, and I remember resource changes restart pods while PVCs can only be expanded, not shrunk.*

Advanced

Q32. What are master (control-plane) and worker nodes, and how do you create and manage Kubernetes clusters (eksctl / Terraform)?

Also asked as: How do you create an EKS cluster using Terraform, and how do the Kubernetes core components come into play?

> **In simple words:** The master is the head office that makes decisions; worker nodes are the factories that do the actual work. On EKS, AWS runs the head office for you - you only build and manage the factories.

Detailed answer

- **Master / control-plane nodes:** run API server, etcd, scheduler, controller manager. They decide and store state; normally no app Pods run there.
- **Worker nodes:** run kubelet, kube-proxy, container runtime, CNI - and your application Pods.

On **EKS** the control plane is fully managed (multi-AZ, AWS patches it, you pay per cluster-hour). You manage worker capacity as **managed node groups**, self-managed groups, **Fargate** profiles or **Karpenter**-provisioned nodes.

Ways to create a cluster:

- **eksctl:** quickest; one YAML/command, good for labs.
- **Terraform** (production): usually the `terraform-aws-modules/eks` module - VPC + subnets, IAM roles (cluster role, node role), the EKS cluster, node groups, add-ons (vpc-cni, coredns, kube-proxy, ebs-csi), access entries. State in S3 with locking, reviewed via `plan` in CI.

What happens when Terraform creates EKS: AWS brings up the API server and etcd in its own account, exposes an API endpoint, and injects ENIs in your subnets; when node groups join, their kubelets register with the API server; add-ons deploy CoreDNS and kube-proxy as Pods.

Day-2 management: version upgrades, add-on upgrades, node AMI rotation, access (RBAC/access entries), autoscaling, monitoring.

Example:

```
eksctl create cluster --name demo --region ap-south-1 \
  --nodegroup-name ng1 --node-type t3.medium --nodes 2 --nodes-min 2 --nodes-max 4

module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  version         = "~> 20.0"
  cluster_name    = "prod-eks"
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
  eks_managed_node_groups = {
    app = { instance_types = ["m6i.large"], min_size = 3, max_size = 10, desired_size = 3 }
  }
}

aws eks update-kubeconfig --name prod-eks --region ap-south-1
kubectl get nodes
```

What to say to the interviewer: *The control plane runs the API server, etcd, scheduler and controllers, and worker nodes run kubelet, kube-proxy and the app pods. On EKS AWS manages the control plane; I create the cluster, VPC, IAM roles, node groups and add-ons with Terraform modules, use eksctl for quick labs, and handle upgrades and node rotation as day-2 operations.*

Q33. Explain Amazon EKS architecture. What subnets are used in EKS?

Also asked as: What is a subnet and which types of subnets are used in Kubernetes/EKS?

In simple words: A subnet is a street inside your private city (VPC). Public streets face the highway (internet); private streets are behind a gate. You put the front desk (load balancer) on the public street and the workers (nodes/pods) on private streets.

Detailed answer

A **subnet** is a range of IPs inside a VPC, tied to one AZ. **Public subnet** = route table has a route to an Internet Gateway. **Private subnet** = no direct internet route; outbound via NAT Gateway or VPC endpoints.

EKS architecture:

- **Control plane** in an AWS-managed VPC, spread over 3 AZs; exposes a public and/or private API endpoint.
- **Cross-account ENIs** placed in your subnets so the control plane can talk to kubelets.
- **Worker nodes** in **private subnets** across 2-3 AZs.
- **VPC CNI:** every Pod gets a **real VPC IP** from the subnet (so subnets must be sized big, e.g. /19; or use prefix delegation / custom networking).
- **Public subnets** hold internet-facing ALB/NLB and NAT Gateways. Tag them `kubernetes.io/role/elb=1`; private ones `kubernetes.io/role/internal-elb=1` so the Load Balancer Controller finds them.
- IAM: cluster role, node role (ECR pull, CNI), IRSA/Pod Identity for apps.
- Add-ons: vpc-cni, CoreDNS, kube-proxy, EBS CSI driver, AWS Load Balancer Controller.

Example:

```
VPC 10.0.0.0/16
  public-a 10.0.0.0/24   -> ALB, NAT GW
  public-b 10.0.1.0/24   -> ALB, NAT GW
  private-a 10.0.32.0/19 -> nodes + pod IPs
  private-b 10.0.64.0/19 -> nodes + pod IPs
  private-c 10.0.96.0/19 -> nodes + pod IPs

aws eks describe-cluster --name prod-eks \
  --query 'cluster.resourcesVpcConfig.{subnets:subnetIds,public:endpointPublicAccess}'
```

What to say to the interviewer: *EKS runs a managed multi-AZ control plane and connects to our VPC with ENIs; worker nodes live in private subnets across at least two AZs, while public subnets host internet-facing load balancers and NAT gateways. Because the VPC CNI gives every pod a VPC IP, I size private subnets generously and tag subnets so the load balancer controller can discover them.*

Q34. How does authentication work in EKS, and how do pods get AWS permissions and secrets securely?

Also asked as: How do you store secrets securely in EKS?

In simple words: To enter the building (cluster), you show your AWS ID card; a guest list (access entries / aws-auth) says which room you can enter (RBAC). Pods get their own temporary badges (IRSA / Pod Identity) instead of sharing a master key.

Detailed answer

Humans and CI -> cluster:

1. `kubectl` calls `aws eks get-token` (built into kubeconfig; older setups used `aws-iam-authenticator`) which creates a signed, short-lived token from your IAM identity.
2. EKS verifies the IAM identity (authentication).
3. It maps the IAM user/role to a Kubernetes user/group via:

- the legacy **aws-auth ConfigMap** in kube-system (easy to break, one typo can lock everyone out), or
- the newer **EKS access entries** (API-managed, with access policies like AmazonEKSClusterAdminPolicy) - preferred.

4. Kubernetes **RBAC** decides what that user/group can do (authorization).

Pods -> AWS services:

- **IRSA** (IAM Roles for Service Accounts): OIDC provider + annotate ServiceAccount with a role ARN; pod gets temporary credentials.
- **EKS Pod Identity:** newer, simpler (agent add-on + association, no OIDC setup per cluster).
- Never put AWS access keys in Secrets or images; avoid relying on the node role for apps.

Secrets: enable **KMS envelope encryption** for Secrets, restrict `get secrets` via RBAC, and keep the source of truth in **AWS Secrets Manager / SSM Parameter Store**, synced by **External Secrets Operator** or the Secrets Store CSI driver using IRSA.

Example:

```
aws eks create-access-entry --cluster-name prod-eks \
  --principal-arn arn:aws:iam::111122223333:role/DevOpsRole
aws eks associate-access-policy --cluster-name prod-eks \
  --principal-arn arn:aws:iam::111122223333:role/DevOpsRole \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy \
  --access-scope type=cluster

apiVersion: v1
kind: ServiceAccount
metadata:
  name: orders
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::111122223333:role/orders-s3-read
```

What to say to the interviewer: *kubectl authenticates with a short-lived token from aws eks get-token, EKS maps the IAM principal to Kubernetes identities through access entries or the older aws-auth ConfigMap, and RBAC authorizes actions. Pods get AWS permissions through IRSA or EKS Pod Identity rather than static keys, and secrets stay in Secrets Manager synced by External Secrets with KMS encryption enabled on the cluster.*

Q35. How do Jenkins and a Kubernetes cluster communicate with each other?

Also asked as: How does Jenkins deploy to Kubernetes, and how do you run Jenkins agents on Kubernetes?

In simple words: Jenkins needs a visitor pass to the cluster. You give it a limited pass (a service account or an IAM role) that only opens the doors it needs, instead of the admin master key.

Detailed answer

Two directions:

1. **Jenkins deploys to Kubernetes:**

- Jenkins agent has `kubectl`/`helm` and a **kubeconfig**.

- On EKS: Jenkins runs with an **IAM role** (EC2 instance profile or IRSA if Jenkins is in-cluster); `aws eks update-kubeconfig` generates the kubeconfig, and the role is mapped via access entry to a namespaced RBAC role.

- Non-EKS: a Kubernetes **ServiceAccount** with a Role/RoleBinding limited to the target namespace; its token/kubeconfig stored in **Jenkins Credentials** (never in the Jenkinsfile).

2. **Kubernetes runs Jenkins agents:** the **Kubernetes plugin** creates an ephemeral agent Pod per build and deletes it after, so agents scale on demand.

Alternative (GitOps): Jenkins does not touch the cluster at all - it only pushes a new image tag to Git and **Argo CD** (inside the cluster) pulls the change. This removes cluster credentials from CI.

Example:

```
stage('Deploy') {
  steps {
    withCredentials([file(credentialsId: 'kubeconfig-prod', variable: 'KUBECONFIG')]) {
      sh 'helm upgrade --install api charts/api -n prod --set image.tag=${GIT_COMMIT}'
    }
  }
}

# RBAC for the deployer SA (namespace-scoped)
kubectl create role deployer -n prod --verb=get,list,watch,create,update,patch \
  --resource=deployments,services,configmaps
kubectl create rolebinding jenkins-deployer -n prod --role=deployer \
  --serviceaccount=cicd:jenkins
```

What to say to the interviewer: *Jenkins talks to the cluster API with a kubeconfig backed by either an IAM role mapped through EKS access entries or a namespace-scoped ServiceAccount stored in Jenkins credentials, never an admin kubeconfig. The Kubernetes plugin lets Jenkins run ephemeral agent pods, and in GitOps setups Jenkins just updates Git and Argo CD applies the change from inside the cluster.*

Q36. You have Docker images for a frontend, a backend and a database. How do you deploy them on Kubernetes with YAML?

In simple words: Three tenants move into the building: the frontend gets the ground-floor shop with a public door (Ingress), the backend gets an internal office (ClusterIP), and the database gets a vault with its own locked storage (StatefulSet + PVC).

Detailed answer

Plan:

- **Namespace** `shop`.
- **Secret** for DB password, **ConfigMap** for non-secret settings (DB host, API URL).
- **Database:** StatefulSet + headless Service + `volumeClaimTemplates` (PVC). In production on AWS, a managed DB (RDS) is often better.
- **Backend:** Deployment (replicas 2+, probes, resources) + ClusterIP Service; reads DB host from ConfigMap, password from Secret.
- **Frontend:** Deployment + ClusterIP Service.
- **Ingress:** `/` -> frontend, `/api` -> backend, TLS.

Example:

```
apiVersion: v1
kind: Secret
metadata: {name: db-secret, namespace: shop}
type: Opaque
stringData: {POSTGRES_PASSWORD: "change-me"}
---
apiVersion: v1
kind: ConfigMap
metadata: {name: app-config, namespace: shop}
data: {DB_HOST: "postgres.shop.svc.cluster.local", DB_NAME: "shop"}
---
apiVersion: v1
kind: Service
metadata: {name: postgres, namespace: shop}
spec: {clusterIP: None, selector: {app: postgres}, ports: [{port: 5432}]}
---
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: postgres, namespace: shop}
spec:
  serviceName: postgres
  replicas: 1
  selector: {matchLabels: {app: postgres}}
  template:
    metadata: {labels: {app: postgres}}
    spec:
      containers:
      - name: postgres
        image: postgres:16
        envFrom: [{secretRef: {name: db-secret}}]
        ports: [{containerPort: 5432}]
        volumeMounts: [{name: data, mountPath: /var/lib/postgresql/data}]
  volumeClaimTemplates:
  - metadata: {name: data}
    spec: {accessModes: [ReadWriteOnce], resources: {requests: {storage: 10Gi}}}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: backend, namespace: shop}
spec:
  replicas: 2
  selector: {matchLabels: {app: backend}}
  template:
    metadata: {labels: {app: backend}}
    spec:
      containers:
      - name: backend
        image: 1234.dkr.ecr.ap-south-1.amazonaws.com/backend:1.0.0
        ports: [{containerPort: 8080}]
        envFrom: [{configMapRef: {name: app-config}}]
```

```
        env:
        - name: DB_PASSWORD
          valueFrom: {secretKeyRef: {name: db-secret, key: POSTGRES_PASSWORD}}
        readinessProbe: {httpGet: {path: /health, port: 8080}}
        resources: {requests: {cpu: 200m, memory: 256Mi}, limits: {memory: 512Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: backend, namespace: shop}
spec: {selector: {app: backend}, ports: [{port: 80, targetPort: 8080}]}
---
# frontend Deployment + Service: same pattern, image frontend:1.0.0, port 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: shop, namespace: shop}
spec:
  ingressClassName: nginx
  rules:
  - host: shop.example.com
    http:
      paths:
      - {path: /api, pathType: Prefix, backend: {service: {name: backend, port: {number: 80}}}}
      - {path: /, pathType: Prefix, backend: {service: {name: frontend, port: {number: 80}}}}
```

What to say to the interviewer: *I put the database in a StatefulSet with a headless Service and a PVC, the backend and frontend in Deployments with ClusterIP Services, and inject config from a ConfigMap and the password from a Secret. An Ingress routes slash to the frontend and slash-api to the backend with TLS, and in real AWS production I would usually move the database to RDS.*

Q37. You have 6 Pods and 3 Nodes. How do you ensure exactly 2 Pods run on each Node?

In simple words: You want 6 guests at 3 tables with exactly 2 per table. You set a rule: "no table may have more than 1 guest more than any other table".

Detailed answer

Best option: **topologySpreadConstraints** with `maxSkew: 1` across `kubernetes.io/hostname` and `whenUnsatisfiable: DoNotSchedule`. With 6 replicas and 3 nodes, skew 1 forces a 2-2-2 layout.

Other tools:

- **Pod anti-affinity** only gives "at most 1 per node" (required) or "prefer spreading" (preferred) - it cannot say "exactly 2".
- **DaemonSet** gives exactly 1 per node, not 2.
- Resource sizing hack (requests so only 2 fit per node) works but is fragile.

Caveats: the scheduler only enforces this at **scheduling time**; later imbalance (after a node failure and recovery) is not fixed automatically - use the **descheduler** to rebalance. Also add `nodeSelector`/taints if only those 3 nodes should be used.

Example:

```
spec:
  replicas: 6
  template:
    metadata: {labels: {app: web}}
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector: {matchLabels: {app: web}}

kubectl get pods -l app=web -o wide --no-headers | awk '{print $7}' | sort | uniq -c
#   2 node-a
#   2 node-b
#   2 node-c
```

> ***What to say to the interviewer:*** *I use topologySpreadConstraints with maxSkew 1 on the hostname topology key and DoNotSchedule, which forces a 2-2-2 split for 6 replicas on 3 nodes. Anti-affinity cannot express exactly two per node, and because spreading is only enforced at scheduling time I would add the descheduler to rebalance after node failures.*

Q38. How would you ensure even Pod distribution across Nodes and Availability Zones?

Also asked as: How do you use multi-zone pod affinity/anti-affinity so a node or zone failure does not impact regional SLAs?

> **In simple words:** Do not keep all your eggs in one basket, and do not keep all your baskets in one room. Spread pods across nodes, and nodes across zones.

Detailed answer

Layered approach:

1. **Nodes in 3 AZs** (node groups spanning subnets in each AZ).
2. **Zone spread:** `topologySpreadConstraints` on `topology.kubernetes.io/zone` with `maxSkew: 1`, `DoNotSchedule` for critical services.
3. **Node spread:** a second constraint on `kubernetes.io/hostname` with `ScheduleAnyway` (soft) so it does not block scaling.
4. **Pod anti-affinity** (required) for replicas that must never share a node (e.g. 3 Kafka brokers, DB replicas).
5. **Pod affinity** to co-locate chatty pairs (app + its cache) in the same zone to cut latency and cross-AZ data cost.
6. **PodDisruptionBudget** so drains/upgrades never take down more than 1 replica.
7. Capacity: enough headroom in each zone to absorb one zone's pods (N+1 zones).

Watch-outs: PVCs on EBS are zone-bound - a StatefulSet pod can only restart in its volume's AZ; use `WaitForFirstConsumer` StorageClass. Cluster Autoscaler node groups should be one per AZ (or use Karpenter) so scale-up happens in the right zone.

Example:

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector: {matchLabels: {app: stream-edge}}
- maxSkew: 1
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: ScheduleAnyway
  labelSelector: {matchLabels: {app: stream-edge}}
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector: {matchLabels: {app: kafka}}
      topologyKey: kubernetes.io/hostname
```

What to say to the interviewer: *I spread replicas across zones with a hard topology spread constraint and across nodes with a soft one, use required anti-affinity for replicas that must never share a node, and add PodDisruptionBudgets. I also keep enough per-zone headroom, remember EBS volumes are zone-bound, and configure autoscaling per zone so a zone loss is absorbed without breaking SLAs.*

Q39. How do you optimize resource requests and limits in a production cluster?

In simple words: Requests are the seats you reserve on a train; limits are the maximum luggage allowed. Reserve too many seats and the train runs half empty (waste); reserve too few and people get kicked off (evictions, throttling).

Detailed answer

- **Request** = guaranteed amount; used by the **scheduler** to place Pods and by HPA for utilization %.
- **Limit** = hard cap. CPU over limit -> **throttled** (slow). Memory over limit -> **OOMKilled**.

Approach:

1. **Measure** real usage for 1-2 weeks (Prometheus/Grafana: `container_cpu_usage_seconds_total`, `container_memory_working_set_bytes`, p95/p99).
2. **Set requests near p90-p95** of normal usage; memory **limit = request or a bit above** (memory cannot be throttled, only killed).
3. **CPU limits:** often omit or set generous, because CPU limits cause throttling latency even when the node is idle; keep requests accurate instead.
4. Use **VPA in recommendation mode** or Goldilocks/Kubecost for suggestions.
5. **QoS classes:** Guaranteed (request = limit) for critical pods -> last to be evicted; BestEffort (no requests) -> first to be evicted. Never run prod pods as BestEffort.
6. Enforce defaults with **LimitRange** and caps with **ResourceQuota** per namespace.
7. Check node packing (`kubectl describe node` allocated %) and rightsize node types; cluster autoscaler removes underused nodes.

Example:

```
resources:
  requests: {cpu: 300m, memory: 512Mi}
  limits:   {memory: 768Mi}        # no CPU limit to avoid throttling

# PromQL: CPU throttling ratio
sum(rate(container_cpu_cfs_throttled_periods_total{pod=~"api.*"}[5m]))
 / sum(rate(container_cpu_cfs_periods_total{pod=~"api.*"}[5m]))

kubectl top pods -n prod --sort-by=memory
kubectl describe node <node> | grep -A5 "Allocated resources"
```

What to say to the interviewer: *I base requests on observed p90 to p95 usage from Prometheus, set memory limits close to requests, and avoid tight CPU limits because they cause throttling. I use VPA recommendations, QoS Guaranteed for critical pods, LimitRange defaults and ResourceQuotas, and review node allocation so autoscaling can remove wasted capacity.*

Q40. How do you implement auto-scaling when traffic fluctuates heavily? How do you automate node scaling in EKS?

Also asked as: How do you design auto-scaling for very large concurrent traffic without over-provisioning? How do you automate ECS/EKS node scaling?

In simple words: Two levels of hiring: HPA hires more workers (pods) quickly; Cluster Autoscaler/Karpenter builds more rooms (nodes) when workers have no place to sit. For a known big event, you hire in advance.

Detailed answer

1. **Pod level - HPA / KEDA:**

- Scale on the metric that leads load: requests per second or concurrent sessions (via Prometheus Adapter/KEDA), not just CPU.

- Tune `behavior`: fast scale-up (e.g. +100% every 15s), slow scale-down (stabilization 5-10 min) to avoid flapping.

2. **Node level:**

- **Cluster Autoscaler:** adds nodes to ASGs when Pods are Pending; removes underused nodes.

- **Karpenter** (preferred on EKS): watches Pending pods and launches right-sized instances directly in seconds, supports mixed instance types and Spot, consolidates empty nodes.

3. **Kill cold start:**

- **Overprovisioning pods:** low-priority "pause" pods reserve spare capacity; real pods preempt them instantly while new nodes come up.

- Pre-scale before known events (scheduled scaling / raise `minReplicas` via CronJob or KEDA cron trigger).

- Small, pre-pulled images; fast startup probes.

4. **Protect:** PDBs, requests set correctly, max limits to avoid runaway cost, Spot for stateless + On-Demand base.

Example:

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies: [{type: Percent, value: 100, periodSeconds: 15}]
  scaleDown:
    stabilizationWindowSeconds: 600
    policies: [{type: Percent, value: 10, periodSeconds: 60}]

apiVersion: karpenter.sh/v1
kind: NodePool
metadata: {name: default}
spec:
  template:
    spec:
      requirements:
      - {key: karpenter.sh/capacity-type, operator: In, values: [spot, on-demand]}
      nodeClassRef: {group: karpenter.k8s.aws, kind: EC2NodeClass, name: default}
  limits: {cpu: "1000"}
  disruption: {consolidationPolicy: WhenEmptyOrUnderutilized}
```

What to say to the interviewer: *I scale pods with HPA or KEDA on leading metrics like requests per second, with fast scale-up and slow scale-down, and scale nodes with Karpenter or Cluster Autoscaler. To avoid cold starts I keep low-priority overprovisioning pods as buffer and pre-scale before known events, with PDBs and max limits to protect availability and cost.*

Q41. How would you monitor HPA scaling decisions in real time and detect if metrics-server is lagging?

In simple words: HPA is a driver looking at the speedometer (metrics). If the speedometer is stuck, the driver makes wrong decisions. You watch both the driver's actions and whether the speedometer is fresh.

Detailed answer

Watch HPA decisions:

- `kubectl get hpa -w` and `kubectl describe hpa` (Conditions: `AbleToScale`, `ScalingActive`, `ScalingLimited`, plus events "New size: 8; reason: cpu above target").
- **kube-state-metrics** exposes `kube_horizontalpodautoscaler_status_current_replicas`, `..._desired_replicas`, `..._spec_max_replicas`. Graph current vs desired vs max in Grafana; alert when desired == max for 10 min (capped) or desired != current for too long.

Detect metrics-server lag / failure:

- `kubectl top pods` failing or old; `kubectl get apiservice v1beta1.metrics.k8s.io` shows `False (FailedDiscoveryCheck)`.
- HPA shows `<unknown>/70%` targets and `FailedGetResourceMetric` events.
- metrics-server logs (kubelet scrape errors, TLS issues), its own CPU/memory throttling; scrape interval (`--metric-resolution`, default 15s).
- Compare HPA-seen value with Prometheus-seen value; alert on staleness, e.g. `time() - timestamp(...)`.
- For custom metrics: check Prometheus Adapter / KEDA operator logs and `kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1`.

Example:

```
kubectl describe hpa api -n prod | sed -n '/Conditions/,$p'
kubectl get apiservice v1beta1.metrics.k8s.io
kubectl get --raw "/apis/metrics.k8s.io/v1beta1/namespaces/prod/pods" | head -c 400
kubectl logs -n kube-system deploy/metrics-server

# Alert: HPA stuck at max
kube_horizontalpodautoscaler_status_current_replicas
  >= kube_horizontalpodautoscaler_spec_max_replicas
```

What to say to the interviewer: *I track HPA current, desired and max replicas from kube-state-metrics in Grafana with alerts when it is pinned at max, and read describe hpa conditions and events for the reason behind each decision. For metrics-server health I check the metrics APIService status, unknown targets on the HPA, metrics-server logs and compare values against Prometheus to spot stale data.*

Q42. What are the differences between GKE Ingress and the NGINX Ingress Controller?

In simple words: GKE Ingress hires Google's own reception at the city gate (Google Cloud Load Balancer). NGINX Ingress puts your own receptionist inside your building (pods), behind a simple door.

Detailed answer

	GKE Ingress (gce class)	NGINX Ingress Controller
Who routes	Google Cloud HTTP(S) Load Balancer (global, outside cluster)	NGINX pods inside the cluster
LB created	External/internal Application LB per Ingress	One L4 LoadBalancer Service in front of NGINX pods
Features	Cloud Armor (WAF), Cloud CDN, IAP, Google-managed certs, global anycast IP	Rewrites, regex paths, rate limiting, auth, canary annotations, custom snippets
Backend	Container-native via NEGs (straight to pods)	kube-proxy/Service to pods
Portability	GCP only	Any cloud/on-prem
Config change speed	Slower (LB provisioning minutes)	Near instant
Cost	Per forwarding rule/LB	Nodes running NGINX + one L4 LB

Choose GKE Ingress for global reach, Cloud Armor/CDN integration and fully managed operation; NGINX for advanced HTTP features, portability and many Ingresses sharing one LB. The same comparison applies on AWS: AWS Load Balancer Controller (ALB) vs ingress-nginx.

Example:

```
metadata:
  annotations:
    kubernetes.io/ingress.class: "gce"            # GKE external LB
    # or with spec.ingressClassName: nginx and:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/limit-rps: "20"
```

What to say to the interviewer: *GKE Ingress provisions a Google Cloud load balancer outside the cluster with native Cloud Armor, CDN and managed certificates, while NGINX Ingress runs as pods inside the cluster behind a single L4 load balancer and offers richer HTTP features and portability. I pick GKE Ingress for global, managed edge security and NGINX when I need rewrites, rate limits or cloud-neutral config.*

Q43. How do you enforce rules in Kubernetes to control which pods can talk to each other?

In simple words: By default every room in the building has open doors. NetworkPolicies are door locks: "only the backend may enter the database room".

Detailed answer

Use **NetworkPolicy**. By default all pods can talk to all pods. A NetworkPolicy selects pods (by label) and allows only listed ingress/egress traffic; once a pod is selected by any policy, anything not allowed is denied.

Steps:

1. Make sure the **CNI enforces** NetworkPolicy (Calico, Cilium, EKS VPC CNI with network policy enabled, GKE Dataplane V2). With a CNI that does not enforce it, policies silently do nothing.
2. Apply **default deny** per namespace.
3. Allow specific flows (frontend -> backend:8080, backend -> db:5432).
4. Allow **DNS egress** to kube-dns (UDP/TCP 53) or everything breaks.
5. Test with a temporary pod (`curl`, `nc`).

For L7 rules (HTTP methods/paths) or mTLS identity, use Cilium policies or a service mesh (Istio AuthorizationPolicy).

Example:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny, namespace: shop}
spec: {podSelector: {}, policyTypes: [Ingress, Egress]}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: db-allow-backend, namespace: shop}
spec:
  podSelector: {matchLabels: {app: postgres}}
  policyTypes: [Ingress]
  ingress:
  - from: [{podSelector: {matchLabels: {app: backend}}}]
    ports: [{protocol: TCP, port: 5432}]
---
# allow DNS for all pods
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - to: [{namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: kube-system}}}]
    ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
```

What to say to the interviewer: *I use NetworkPolicies on a CNI that enforces them, start with a default-deny per namespace, then explicitly allow required flows such as backend to database on 5432 and DNS egress to kube-dns. For layer 7 rules or identity-based control I add Cilium policies or an Istio AuthorizationPolicy with mTLS.*

Q44. How would you secure a Kubernetes cluster at the network, pod and identity/access levels?

Also asked as: How would you secure a Kubernetes / GKE cluster?

In simple words: Secure a building in layers: fence and gates (network), rules for how each tenant behaves (pod security), and ID cards with only the rooms you need (identity and access).

Detailed answer

Network level

- Private cluster / private API endpoint (or restrict public endpoint CIDRs); nodes in private subnets.
- NetworkPolicies default-deny; Security Groups (SG for pods on EKS).
- TLS at Ingress + WAF (AWS WAF / Cloud Armor); mTLS between services with a service mesh.
- Restrict egress (egress policies, NAT/firewall allow-lists).

Pod / workload level

- **Pod Security Standards** (`restricted` via Pod Security Admission labels on namespaces).
- `runAsNonRoot`, `readOnlyRootFilesystem`, drop all capabilities, no `privileged`, no hostPath/hostNetwork.
- Signed, scanned images (Trivy in CI, ECR scanning), allow only trusted registries, pin by digest.
- Policy engines **OPA Gatekeeper / Kyverno** to enforce rules (no latest tag, must have limits).
- Resource limits to prevent noisy-neighbor DoS; runtime detection (Falco, GuardDuty EKS Runtime).

Identity & access level

- RBAC least privilege; namespace-scoped Roles; no one uses cluster-admin daily.
- EKS access entries / GKE IAM; short-lived tokens; SSO.
- Pods use **IRSA / Pod Identity / GKE Workload Identity**, not node credentials; `automountServiceAccountToken: false` where not needed.
- Secrets encrypted with KMS, external secret stores.
- **Audit logging** enabled to CloudWatch / Cloud Logging; regular version upgrades and CIS benchmark (kube-bench).

Example:

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted

securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities: {drop: ["ALL"]}
  seccompProfile: {type: RuntimeDefault}

kubectl auth can-i --list --as=system:serviceaccount:prod:api -n prod
```

> ***What to say to the interviewer:*** *At the network level I use private endpoints, private subnets, default-deny NetworkPolicies, WAF and TLS or mTLS. At the pod level I enforce the restricted Pod Security Standard, non-root read-only containers, scanned images and Gatekeeper or Kyverno policies. For identity I apply least-privilege RBAC, IRSA or Workload Identity for pods, KMS-encrypted secrets and audit logging.*

Q45. How do you secure secrets in Kubernetes?

Also asked as: How do you manage secrets across 50+ services without exposing Vault access?

> **In simple words:** A Kubernetes Secret by default is a letter in a see-through envelope (base64). Securing it means a locked safe (KMS encryption), a short list of key holders (RBAC), and ideally keeping the real valuables in a bank (Secrets Manager / Vault).

Detailed answer

Kubernetes Secrets are only **base64-encoded** and stored in etcd. Layers of protection:

1. **Encryption at rest:** EKS KMS envelope encryption (GKE: application-layer secrets encryption with Cloud KMS).
2. **RBAC:** only the workloads/people that need them can `get`/`list` secrets; note that anyone who can create pods in a namespace can read its secrets through the pod.
3. **External source of truth:** AWS Secrets Manager / SSM, HashiCorp Vault, GCP Secret Manager.

- **External Secrets Operator** syncs them into K8s Secrets on a refresh interval.
- **Secrets Store CSI Driver** mounts them as files directly from the store (can avoid a K8s Secret object).
- Vault Agent injector sidecar for dynamic secrets.

4. **Git:** never commit plain Secrets; use **SOPS** or **Sealed Secrets** if they must live in Git (GitOps).
5. **Access from pods** via IRSA/Workload Identity (no static keys).
6. Rotation + audit logging (who read which secret).
7. Prefer mounting as files over env vars (env vars leak into crash dumps, `kubectl describe`, child processes).

Example:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: {name: db-cred, namespace: prod}
spec:
  refreshInterval: 1h
  secretStoreRef: {name: aws-secrets-manager, kind: ClusterSecretStore}
  target: {name: db-cred}
  data:
  - secretKey: DB_PASSWORD
    remoteRef: {key: prod/orders/db, property: password}
```

50+ services without handing out Vault tokens:

- Every service gets its own **ServiceAccount**. Vault's **Kubernetes auth method** maps that SA (plus namespace) to a Vault role, which maps to a policy allowing only that service's paths.
- Secrets are delivered by the **Vault Agent Injector** sidecar (annotations) or the **Secrets Store CSI driver**. The app just reads a file, and never holds a long-lived or broad Vault token.
- Use **dynamic secrets** for databases (Vault creates a DB user per pod with a short TTL), so a leak expires quickly.
- An alternative is the **External Secrets Operator** with a SecretStore per namespace, if you prefer native K8s Secrets synced from Vault or AWS Secrets Manager.
- Turn on Vault audit logs, and template policies so onboarding service 51 is a PR, not a manual task.

```
metadata:
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "meter-api"                      # bound to SA meter-api
    vault.hashicorp.com/agent-inject-secret-db: "database/creds/meter-api"   # dynamic
```

> ***What to say to the interviewer:*** *Native Secrets are just base64, so I enable KMS encryption at rest, restrict access with RBAC, and keep the real values in AWS Secrets Manager or Vault synced by External Secrets Operator or the CSI driver using IRSA. For GitOps I use Sealed Secrets or SOPS, and I rotate secrets and audit who accesses them.*

Q46. How do you manage secrets across multi-region Kubernetes clusters?

> **In simple words:** Keep one master copy of each key in a central safe that automatically makes copies in each city's branch vault, and let each city's workers fetch from their local vault.

Detailed answer

Design:

- **Single source of truth:** AWS Secrets Manager with **multi-region replica secrets** (primary in region A, read replicas in B/C; same name, replicated automatically, each encrypted with a regional KMS key). Alternatively Vault with performance replication.
- **Per-cluster sync:** each cluster runs **External Secrets Operator** with a `ClusterSecretStore` pointing to its **local region** endpoint, authenticated by **IRSA / Pod Identity** (per-cluster IAM role scoped to specific secret ARNs/paths like `prod/orders/*`).
- **Consistent naming:** `env/app/secret` so the same ExternalSecret manifest (via Helm/Argo CD ApplicationSet) works in every region.

- **Rotation:** rotate once at the primary (Lambda rotation); replicas update; ESO `refreshInterval` pulls new values; apps reload (Reloader or checksum annotation roll).
- **Failover:** if the primary region is down, replicas can be **promoted** to standalone.
- Audit with CloudTrail in each region; alert on access-denied spikes.

Example:

```
aws secretsmanager replicate-secret-to-regions --secret-id prod/orders/db \
  --add-replica-regions Region=ap-southeast-1 Region=eu-west-1

apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata: {name: aws-sm}
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-southeast-1          # local region per cluster
      auth:
        jwt: {serviceAccountRef: {name: external-secrets, namespace: external-secrets}}
```

What to say to the interviewer: *I keep secrets in AWS Secrets Manager with multi-region replicas and run External Secrets Operator in every cluster, each reading from its local region using a scoped IRSA role. Rotation happens once at the primary, replicas propagate, ESO refreshes the Kubernetes Secret and Reloader restarts the pods; replicas can be promoted if the primary region fails.*

Q47. How do you handle database credential rotation in Kubernetes without downtime?

In simple words: Change the lock on a door while people are still using it: first make the new key work too, hand out new keys, then retire the old key.

Detailed answer

Steps:

1. Store credentials in **AWS Secrets Manager** (or Vault) - not hard-coded.
2. Enable **automatic rotation** (Secrets Manager rotation Lambda for RDS). Use the **alternating users** strategy (two DB users, rotate one while the other is active) or keep AWSPREVIOUS valid briefly, so existing connections do not break.
3. **Sync to K8s:** External Secrets Operator with a short `refreshInterval`, or Secrets Store CSI driver with rotation enabled.
4. **App picks up new value:**

- If mounted as a file, the app can re-read it on auth failure.
- If env var, pods must restart: use **Stakater Reloader** (restarts Deployment when Secret changes) or a checksum annotation in Helm -> rolling restart, zero downtime.

5. Connection pools must reconnect gracefully on auth errors.
6. Even better: **Vault dynamic secrets** (short-lived credentials generated per pod) or **RDS IAM authentication** (token instead of password).
7. Monitor: alert on DB auth failures after rotation.

Example:

```
aws secretsmanager rotate-secret --secret-id prod/orders/db --rotation-lambda-arn <arn> \
  --rotation-rules AutomaticallyAfterDays=30

metadata:
  annotations:
    secret.reloader.stakater.com/reload: "db-cred"    # Reloader rolls pods on change
```

> ***What to say to the interviewer:*** *Credentials live in Secrets Manager with automatic rotation using an alternating-users strategy, External Secrets Operator syncs the new value into Kubernetes and Reloader performs a rolling restart so pods pick it up with no downtime. Where possible I prefer RDS IAM auth or Vault dynamic secrets so there is no long-lived password at all.*

Q48. What is your strategy for backup and restore in a Kubernetes cluster? How do you handle disaster recovery for stateful apps on containers?

Also asked as: What is your approach to disaster recovery for stateful apps running on containers?

> **In simple words:** Back up two things: the building's blueprint (Kubernetes objects) and the contents of the vaults (persistent data). A backup only counts if you have practised rebuilding from it.

Detailed answer

What to back up:

- **Cluster objects:** ideally already in **Git** (GitOps) + Helm charts, so a cluster can be rebuilt with Terraform + Argo CD.
- **etcd snapshots** (self-managed clusters only; EKS/GKE manage etcd).
- **Persistent volumes / data:** EBS snapshots, application-level dumps.
- **Velero:** backs up namespaces/resources to S3 and volumes via snapshots or file-level (Kopia), on a schedule, with restore to the same or another cluster (also used for migrations).

Stateful DR strategy:

- Prefer **managed data services** (RDS Multi-AZ, cross-region read replica, automated backups/PITR) over DBs in pods.
- For DBs in K8s: app-consistent backups (`pg_dump`, `mongodump`, operator backups like CloudNativePG/Percona to S3), plus volume snapshots.
- Copy backups **cross-region/cross-account**, S3 versioning + Object Lock.
- Define **RPO/RTO**; choose backup & restore, pilot light, warm standby, or active-active accordingly.
- **Test restores regularly** (monthly restore drill into a scratch namespace, verify checksums / row counts).

Example:

```
velero install --provider aws --bucket k8s-backups --backup-location-config region=ap-south-1 \
  --snapshot-location-config region=ap-south-1 --plugins velero/velero-plugin-for-aws:v1.10.0
velero schedule create daily-prod --schedule="0 1 * * *" --include-namespaces prod --ttl 720h
velero backup get
velero restore create --from-backup daily-prod-20260920010000 --namespace-mappings prod:prod-restore
    -test

# self-managed etcd
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db ...
```

> ***What to say to the interviewer:*** *Manifests live in Git so the cluster can be rebuilt with Terraform and Argo CD, Velero takes scheduled backups of namespaces and volume snapshots to S3, and databases get application-consistent backups copied cross-region. I prefer managed databases for state, define RPO and RTO per service, and run regular restore drills because an untested backup is not a backup.*

Q49. How do you migrate data from one MongoDB pod to another? How do you manage stateful workloads in Kubernetes?

> **In simple words:** Moving house for a database: either copy the boxes while the house is closed (dump/restore), or build the new house as an extension that keeps itself in sync and then just move in (replica set).

Detailed answer

Stateful workloads need StatefulSets (stable identity), PVCs (persistent storage), headless Services (stable DNS), anti-affinity, PDBs, and ideally an **operator** (MongoDB Community/Percona Operator) that handles replication, failover, backups.

Migration options:

1. **Replica set method (near zero downtime - preferred):**

- Add the new pod (new StatefulSet / new cluster) as a **secondary** member: `rs.add("mongo-new-0.mongo-new:27017")`.
- Wait for initial sync (`rs.status()` shows SECONDARY, low lag).
- `rs.stepDown()` to make the new member primary, update app connection string (or keep it through the Service), then `rs.remove()` old members.

2. **mongodump / mongorestore (simple, needs write freeze):**

- Stop writes or accept a maintenance window, dump from old pod, restore into new pod, switch connection string.

3. **Volume-level:** snapshot the PVC (VolumeSnapshot / EBS snapshot) and create a new PVC from it for the new pod (same data, same engine version).

Always: take a backup first, verify counts/checksums after, keep the old copy until verified.

Example:

```
kubectl exec -it mongo-0 -n db -- mongosh --eval 'rs.add("mongo-new-0.mongo-new.db.svc:27017")'
kubectl exec -it mongo-0 -n db -- mongosh --eval 'rs.status().members.map(m=>[m.name,m.stateStr])'

# dump and restore
kubectl exec mongo-0 -n db -- mongodump --archive --gzip > dump.gz
kubectl exec -i mongo-new-0 -n db -- mongorestore --archive --gzip < dump.gz
```

What to say to the interviewer: *Stateful apps run as StatefulSets with PVCs, headless Services, anti-affinity and ideally an operator. To migrate MongoDB with near-zero downtime I add the new pod as a replica set secondary, wait for sync, step down the old primary and remove old members; for simpler cases I use mongodump and mongorestore in a maintenance window or restore from a PVC snapshot, always verifying data before deleting the old copy.*

Q50. Have you upgraded Kubernetes clusters? How do you upgrade a cluster (EKS) safely?

In simple words: Upgrade the brain first, then replace the workers one by one while the factory keeps running - and check the recipe book (APIs) is still valid for the new version.

Detailed answer

Steps for EKS (one minor version at a time, e.g. 1.29 -> 1.30):

1. **Prepare:** read release notes; find **deprecated/removed APIs** (`kubent`, `pluto`, EKS upgrade insights); check add-on and Helm chart compatibility (ingress controller, cert-manager, CSI, Karpenter).
2. **Test** in dev/staging first.
3. **Upgrade control plane** (Terraform `cluster_version` change or `eksctl upgrade cluster`). AWS does it in-place with no API downtime.
4. **Upgrade add-ons:** vpc-cni, CoreDNS, kube-proxy, EBS CSI to versions matching the new K8s.
5. **Upgrade nodes:** managed node group rolling update (new AMI) - it cordons and drains nodes respecting **PDBs**; or blue/green node groups (create new group, drain old); Karpenter drift replaces nodes.
6. **Validate:** nodes Ready, pods running, dashboards, error rates, smoke tests.

Rollback: control plane cannot be downgraded, so testing first and blue/green node groups (or a blue/green cluster for big jumps) is the safety net. Node versions may lag the control plane within the supported skew.

Example:

```
kubent                                  # find deprecated APIs
eksctl upgrade cluster --name prod-eks --version 1.30 --approve
aws eks update-addon --cluster-name prod-eks --addon-name coredns --addon-version <ver>
eksctl upgrade nodegroup --cluster prod-eks --name app-ng --kubernetes-version 1.30
kubectl get nodes -o wide     # VERSION column shows v1.30.x
```

What to say to the interviewer: *I upgrade one minor version at a time after scanning for deprecated APIs and testing in staging: control plane first, then add-ons like VPC CNI, CoreDNS and kube-proxy, then node groups with a rolling or blue-green node replacement that respects PDBs. Because the control plane cannot be downgraded, preparation and validation matter most.*

Q51. How do you perform zero-downtime deployments in Kubernetes?

Also asked as: You need to implement zero-downtime deployment for a service in Kubernetes. Walk me through your plan. / How do you design zero-downtime deployments for STATEFUL apps on Kubernetes?

In simple words: Open new checkout counters and only close the old ones after the new ones are actually serving customers - and let the old cashier finish the customer already at the counter.

Detailed answer

Checklist:

1. **Rolling update** with `maxUnavailable: 0`, `maxSurge: 1` (or 25%).
2. **Readiness probe** that truly reflects "can serve traffic"; `minReadySeconds` to catch instant crashes.
3. **Graceful shutdown:**

- App handles **SIGTERM** (stop accepting, finish in-flight requests).

- `preStop` hook `sleep 5-15` so load balancers/kube-proxy remove the pod from endpoints before it stops.

- `terminationGracePeriodSeconds` longer than the longest request.

4. **Enough replicas** (>=2) spread across nodes/AZs + **PDB** so drains do not take all replicas.
5. **Backward-compatible changes:** DB migrations in expand/contract style so old and new versions both work during rollout.
6. For AWS ALB ingress: use IP target mode, set deregistration delay, pod readiness gates.
7. **Progressive delivery** for risky releases: canary/blue-green with Argo Rollouts, automatic rollback on metrics.
8. Verify with `rollout status` + error-rate dashboards; `rollout undo` or `helm rollback` if needed.

Example:

```
spec:
  strategy: {type: RollingUpdate, rollingUpdate: {maxSurge: 1, maxUnavailable: 0}}
  minReadySeconds: 10
  template:
    spec:
      terminationGracePeriodSeconds: 45
      containers:
      - name: api
        readinessProbe: {httpGet: {path: /ready, port: 8080}, periodSeconds: 5}
        lifecycle:
          preStop: {exec: {command: ["sh", "-c", "sleep 10"]}}
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: api-pdb}
spec: {minAvailable: 2, selector: {matchLabels: {app: api}}}
```

Stateful apps (databases, Kafka, Redis, apps with sessions):

- **StatefulSet rolling updates** go one pod at a time in reverse order. Use `updateStrategy.rollingUpdate.partition` for a canary: only pods with an ordinal >= partition get the new version.

- **Readiness probes that mean "caught up":** for a replica, ready only when replication lag is low, not just when the port is open.

- **PodDisruptionBudget** (`maxUnavailable: 1`), so drains and upgrades never take down the quorum.

- **preStop hook + terminationGracePeriodSeconds** to hand over leadership or drain connections before SIGTERM kills the process.
- **Migration sequencing:** apply expand-only, backward-compatible schema changes first, roll the app, then contract in a later release.
- Prefer **operators** (CloudNativePG, Strimzi for Kafka, Redis operators) that handle switchover and failover properly.

```
apiVersion: apps/v1
kind: StatefulSet
spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate: { partition: 2 }     # only pod-2 (and above) gets the new version first
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: db
          lifecycle:
            preStop: { exec: { command: ["/scripts/step-down-and-drain.sh"] } }
          readinessProbe:
            exec: { command: ["/scripts/replica-lag-ok.sh"] }
```

> ***What to say to the interviewer:*** *I use a rolling update with maxUnavailable 0, accurate readiness probes, a preStop sleep plus SIGTERM handling and enough grace period so in-flight requests finish, and PDBs with multiple replicas across zones. Database changes are backward compatible, risky releases go through canary with automatic rollback, and I watch rollout status and error rates to revert quickly.*

Q52. How would you do blue-green (and canary) deployments in a Kubernetes / EKS cluster?

Also asked as: How do you handle blue/green deployment for EKS? What would that look like in practice?

> **In simple words:** Blue-green is building a complete second stage behind the curtain, testing it, then turning the spotlight to it in one move - and turning it back instantly if something is wrong. Canary is letting a few audience members try the new show first.

Detailed answer

Blue-green with plain Kubernetes:

1. `app-blue` Deployment (v1, label `version: blue`) is live; Service selector points to `version: blue`.
2. Deploy `app-green` (v2, `version: green`) at full capacity alongside it.
3. Test green via a separate test Service/host (`green.internal`).
4. **Switch:** patch the Service selector to `version: green` - all traffic moves at once.
5. **Rollback:** patch selector back to `blue` (seconds). Keep blue running until confident, then scale it down.

Gradual traffic shifting (canary style):

- **Istio VirtualService weights:** 90/10 -> 50/50 -> 100/0 between subsets; rollback = set weights back to 100% old, no pod restarts.

- **NGINX Ingress** canary annotations (`canary-weight: "10"`) or **ALB weighted target groups**.
- **Argo Rollouts / Flagger** automate steps and run **analysis** (Prometheus error rate, latency) and auto-rollback.

Watch-outs: double capacity during blue-green (cost), DB schema must work for both versions, sticky sessions/long connections, and consumers (Kafka workers) should not both process in parallel unexpectedly.

Example:

```
kubectl patch svc app -p '{"spec":{"selector":{"app":"app","version":"green"}}}'
kubectl patch svc app -p '{"spec":{"selector":{"app":"app","version":"blue"}}}'   # rollback

apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: {name: app}
spec:
  hosts: [app]
  http:
  - route:
    - {destination: {host: app, subset: blue},  weight: 90}
    - {destination: {host: app, subset: green}, weight: 10}   # then 50, then 100
```

> ***What to say to the interviewer:*** *For blue-green I run blue and green Deployments side by side, test green on an internal route, then switch the Service selector or ingress target to green and roll back by switching it back. For gradual shifts I use Istio VirtualService weights such as 10, 50 then 100 percent, or Argo Rollouts with Prometheus analysis for automatic rollback, keeping database changes compatible with both versions.*

Q53. How would you set up an automated rollback strategy in Kubernetes for failed deployments?

Also asked as: Explain how you would handle a failed rollout during a deployment.

> **In simple words:** Give the deployment a safety net and a referee: if the new version does not become healthy in time, or the error rate goes up, the referee automatically puts the old version back.

Detailed answer

Layers of automation:

1. **Kubernetes-native detection:** `progressDeadlineSeconds` marks rollout `Failed` (ProgressDeadlineExceeded) if new pods do not become ready; readiness probes stop bad pods getting traffic; old ReplicaSet stays serving because `maxUnavailable: 0`.
2. **CI/CD gate:** `kubectl rollout status --timeout` or `helm upgrade --atomic --timeout 5m` -> on failure Helm **automatically rolls back** to the previous release and the pipeline fails.
3. **Metric-based rollback (progressive delivery): Argo Rollouts** or Flagger run canary steps with an `AnalysisTemplate` querying Prometheus (5xx rate, p99 latency); if the analysis fails, it aborts and returns traffic to stable automatically.
4. **GitOps:** Argo CD - revert the Git commit to roll back; the cluster follows.

5. Post-rollback: alert the team, keep the failed ReplicaSet/pods logs for RCA, block re-deploy until fixed.

Example:

```
helm upgrade --install api charts/api -n prod --set image.tag=$TAG --atomic --timeout 5m

apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata: {name: error-rate}
spec:
  metrics:
  - name: error-rate
    interval: 1m
    failureLimit: 1
    successCondition: result[0] < 0.02
    provider:
      prometheus:
        address: http://prometheus.monitoring:9090
        query: |
          sum(rate(http_requests_total{app="api",status=~"5.."}[2m]))
          / sum(rate(http_requests_total{app="api"}[2m]))
```

> ***What to say to the interviewer:*** *I rely on progressDeadlineSeconds and readiness probes so a bad version never takes traffic, and deploy with helm upgrade --atomic so a failed rollout is reverted automatically by the pipeline. For metric-driven safety I use Argo Rollouts canaries with Prometheus analysis on error rate and latency, which abort and restore the stable version on their own.*

Q54. What are PodDisruptionBudgets and why do you need them?

> **In simple words:** A PDB is a rule for maintenance crews: "you may take workers off the line for repairs, but at least 2 must always stay working".

Detailed answer

A **PodDisruptionBudget** limits how many pods of an app can be down at the same time due to **voluntary disruptions**: node drains, cluster upgrades, Cluster Autoscaler/Karpenter consolidation, `kubectl drain`.

- `minAvailable: 2` or `maxUnavailable: 1` (number or %).
- The eviction API refuses evictions that would break the budget; the drain waits.
- It does **not** protect against involuntary disruptions (node crash, OOM kill).

Gotchas:

- A PDB with `maxUnavailable: 0` or `minAvailable` = replicas blocks drains forever -> node upgrades hang.
- Single-replica apps with a strict PDB block maintenance; either run 2+ replicas or allow 1 unavailable.

Example:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: api-pdb, namespace: prod}
spec:
  maxUnavailable: 1
  selector: {matchLabels: {app: api}}

kubectl get pdb -n prod
# NAME     MIN AVAILABLE  MAX UNAVAILABLE  ALLOWED DISRUPTIONS
# api-pdb  N/A            1                1
```

What to say to the interviewer: *A PodDisruptionBudget caps how many replicas can be evicted at once during voluntary disruptions like drains, upgrades or autoscaler consolidation. I set maxUnavailable 1 for most services with at least two replicas, and avoid budgets that allow zero disruptions because they block node upgrades.*

Q55. What does the pod status ContainerStatusUnknown mean, and when does it occur?

In simple words: The manager lost contact with a worker's walkie-talkie. He does not know if the worker is still working, has left, or has collapsed - so the status is "unknown".

Detailed answer

`ContainerStatusUnknown` means the kubelet **could not determine the container's state**, usually because the container disappeared and its status could not be retrieved (it shows with reason "The container could not be located when the pod was terminated", exit code 137).

Common causes:

- **Node problems:** node became NotReady / lost contact with control plane, kubelet restarted or crashed.
- **Eviction under resource pressure** (disk/memory) where the container was killed and cleaned up.
- **Container runtime issues** (containerd restarted, runtime state lost).
- Node was terminated (Spot interruption, scale-down) while pods were running.

What to do:

1. `kubectl describe pod` and `kubectl get events` - look for Evicted, NodeNotReady, OOM.
2. Check node: `kubectl describe node` conditions (MemoryPressure, DiskPressure), kubelet/containerd logs.
3. Usually the controller has already created a replacement; delete the stale pod (`kubectl delete pod`).
4. Fix root cause: resource requests/limits, disk cleanup, node health, Spot handling.

Example:

```
kubectl get pods -A | grep ContainerStatusUnknown
kubectl describe pod <pod> | sed -n '/State/,/Events/p'
kubectl get pods -A --field-selector=status.phase=Failed -o name | xargs kubectl delete
journalctl -u kubelet --since "1 hour ago"      # on the node
```

What to say to the interviewer: *ContainerStatusUnknown means kubelet lost track of the container's state, typically after node pressure evictions, a kubelet or containerd restart, or a node going away. I check pod events and node conditions, clean up the stale pod since the controller usually replaced it, and fix the underlying node or resource issue.*

Q56. How would you configure readiness/liveness probes to catch buffering or lag issues in stream-processing microservices before users notice?

In simple words: Do not just ask the worker "are you breathing?" - ask "are you keeping up with the conveyor belt?" If he is falling behind, stop giving him new boxes before the pile hits customers.

Detailed answer

Plain "process is up" checks miss slowness. Design:

- **Readiness = "can I serve well right now?":** expose `/ready` that fails if internal lag exceeds a threshold, e.g. consumer lag > N messages, internal buffer/queue depth > X%, segment processing time > target, dependency (Kafka/Redis) unreachable. Failing readiness removes the pod from the load balancer so users go to healthy pods; it is **not restarted**.
- **Liveness = "am I stuck?":** a **watchdog** timestamp updated by the processing loop; fail only if no progress for, say, 60s (deadlock). Do **not** tie liveness to lag, or a traffic spike restarts all pods and makes it worse.
- **Startup probe** covers warm-up (loading codecs, caches) so pods are not killed at boot.
- Tuned timing: `periodSeconds: 5`, `failureThreshold: 2-3` for readiness (fast reaction), longer for liveness.
- Probes are only a safety net: also export **metrics** (lag, buffer ratio, rebuffer rate, p99 latency) to Prometheus with alerts and HPA/KEDA scaling on lag, plus synthetic playback checks.

Example:

```
readinessProbe:
  httpGet: {path: /ready, port: 8080}     # 503 if lag > 2s or buffer > 80%
  periodSeconds: 5
  failureThreshold: 2
  timeoutSeconds: 1
livenessProbe:
  httpGet: {path: /live, port: 8080}      # 500 only if loop idle > 60s
  periodSeconds: 10
  failureThreshold: 6
startupProbe:
  httpGet: {path: /live, port: 8080}
  periodSeconds: 5
  failureThreshold: 24
```

What to say to the interviewer: *I make readiness reflect quality of service, failing when processing lag or buffer depth crosses a threshold so the pod is taken out of rotation, while liveness only detects a stuck processing loop via a heartbeat so load spikes do not trigger restart storms. A startup probe covers warm-up, and lag metrics also drive alerts and KEDA scaling.*

Q57. How does data flow in a Kubernetes cluster? Explain the Kubernetes networking model.

In simple words: Every pod gets its own phone number and can call any other pod directly, with no switchboard operator changing numbers in between. Services are shared hotline numbers that forward to whichever pod is free, and the Ingress is the reception desk for calls from outside.

Detailed answer

The networking model (rules every CNI must follow):

- Every pod gets its **own IP**; containers in a pod share it (talk over `localhost`).
- **Pod-to-pod without NAT**, across all nodes: the IP a pod sees for itself is the IP others use.
- Nodes can reach all pods and vice versa.

How packets move:

1. **Same node:** pod eth0 -> **veth pair** -> node bridge / routing table -> other pod's veth. Never leaves the node.
2. **Across nodes:** handled by the **CNI plugin**:

- **Overlay** (Flannel VXLAN, Calico IPIP/VXLAN): packet is encapsulated with the node IPs, sent across, decapsulated. Pod IPs come from a separate range.
- **Native routing** (AWS VPC CNI, Azure CNI, GKE VPC-native): pods get **real VPC IPs** on node ENIs, so the VPC routes them directly - no encapsulation, but subnets must have enough IPs.

3. **Pod -> Service (east-west):** app resolves `svc.ns` via **CoreDNS** -> gets the ClusterIP (a virtual IP that exists on no interface) -> **kube-proxy** rules (iptables or IPVS) on the source node **DNAT** it to a Ready endpoint pod IP -> normal pod-to-pod path. **Cilium** can replace kube-proxy with eBPF.
4. **External -> app (north-south):** client -> DNS (Route 53) -> cloud load balancer (ALB/NLB) -> ingress controller pods (or directly to pod IPs in IP target mode) -> Service -> pod.
5. **Pod -> internet (egress):** pod -> node -> SNAT to node IP -> NAT Gateway -> internet.

Example:

```
                Client
                  |  DNS: app.example.com -> ALB
                  v
          +------------------+
          |  Cloud LB (ALB)  |
          +--------+---------+
                   v
          +------------------+      Ingress rules: host/path
          | Ingress ctrl pod |
          +--------+---------+
                   |  Service "api" ClusterIP 10.100.5.20
                   |  kube-proxy DNAT -> endpoint pod IP
      +------------+-------------+
      v                          v
 Node A                     Node B
 [api pod 10.0.1.15]        [api pod 10.0.2.33]
   | veth                       | veth
 [bridge/routes]            [bridge/routes]
   +------- CNI (VPC routing or VXLAN overlay) -------+
 api pod -> "db.data" -> CoreDNS -> ClusterIP -> db pod IP

kubectl get pods -o wide              # pod IPs and nodes
kubectl get endpointslices -l kubernetes.io/service-name=api
sudo iptables -t nat -L KUBE-SERVICES | grep api      # on a node (iptables mode)
```

For troubleshooting along this path, see the scenario questions on Ingress, request-flow 503s and pod-to-pod connectivity.

> ***What to say to the interviewer:*** *Kubernetes uses a flat network where every pod has its own IP and can reach any other pod without NAT; same-node traffic goes over veth pairs and the bridge, while cross-node traffic is carried by the CNI either as an overlay like VXLAN or with native VPC IPs as in the AWS VPC CNI. Services are virtual IPs that kube-proxy DNATs to Ready pod endpoints, CoreDNS maps names to them, and external traffic flows DNS, load balancer, ingress controller, Service, pod.*

Q58. What happens if a node with local storage gets scaled down by the autoscaler?

> **In simple words:** If you keep your files in a hotel room's drawer (local disk) and the hotel removes that room to save money, the files are gone. Keep important things in a bank locker (network storage) that you can take to any room.

Detailed answer

- Data in **emptyDir**, **hostPath** or **local PersistentVolumes** lives on that node's disk. When the node is removed, **that data is lost**. A pod using a local PV also cannot move to another node (the PV is tied to that node).
- **Cluster Autoscaler** by default **skips scaling down** nodes that run pods with local storage (emptyDir/hostPath), unless the pod is annotated `cluster-autoscaler.kubernetes.io/safe-to-evict: "true"` (or the flag `--skip-nodes-with-local-storage=false` is set). Pods without a controller also block scale-down.
- **Karpenter** consolidation **does** drain and replace nodes; it respects PDBs and the pod annotation `karpenter.sh/do-not-disrupt: "true"`, which blocks voluntary disruption.

- Spot interruptions and node failures ignore all of this - the node just disappears.

How to design for it:

- Keep durable data on **network volumes** (EBS via PVC for single-writer, EFS for shared) with **StatefulSets**; the volume reattaches to the pod on another node (same AZ for EBS).
- Use emptyDir only for **scratch/cache** data that can be rebuilt.
- **PDBs** to limit simultaneous evictions; graceful shutdown to flush data.
- **Backups** (Velero, app-level) for anything important.
- If you must use local NVMe (high-performance DBs), rely on app-level replication (Kafka, Cassandra) and use the do-not-disrupt / safe-to-evict annotations deliberately.

Example:

```
metadata:
  annotations:
    karpenter.sh/do-not-disrupt: "true"                       # Karpenter
    cluster-autoscaler.kubernetes.io/safe-to-evict: "false"   # Cluster Autoscaler
spec:
  volumes:
  - name: scratch
    emptyDir: {sizeLimit: 2Gi}        # lost when pod/node goes away
```

> ***What to say to the interviewer:*** *Local data in emptyDir, hostPath or local PVs is lost when the node is removed; Cluster Autoscaler skips such nodes by default unless pods are marked safe-to-evict, while Karpenter consolidation drains them unless pods carry the do-not-disrupt annotation. I keep durable data on EBS or EFS through StatefulSets and PVCs, use local disk only for rebuildable scratch data, and add PDBs and backups.*

Q59. How do you enforce runtime security in Kubernetes?

> **In simple words:** Security at the gate (admission) stops dangerous tenants from moving in; security guards inside (runtime detection) watch for someone doing something suspicious after they have moved in.

Detailed answer

1. Before the pod runs (admission):

- **PodSecurityPolicy was removed in Kubernetes 1.25.** The built-in replacement is **Pod Security Admission** enforcing the **Pod Security Standards** (privileged / baseline / restricted) per namespace via labels.
- **OPA Gatekeeper or Kyverno** for custom policies: only images from our ECR, no `latest` tag, must have limits, no hostPath, required labels.
- **Image signing verification** (cosign/Sigstore with Kyverno or Gatekeeper, AWS Signer + Ratify) so only images built by our pipeline run; vulnerability scanning in CI and in the registry.

2. How the container runs (hardening via securityContext):

- `runAsNonRoot`, fixed non-zero UID, `allowPrivilegeEscalation: false`
- `readOnlyRootFilesystem: true` (writable emptyDir only where needed)
- `capabilities: drop: ["ALL"]`, no `privileged`, no hostNetwork/hostPID
- `seccompProfile: RuntimeDefault`; **AppArmor/SELinux** profiles

- Minimal/distroless images; no service account token unless needed.

3. While it runs (detection and response):

- **Falco** (eBPF-based rules: shell spawned in container, write to /etc, unexpected network connection) or **GuardDuty EKS Runtime Monitoring**, Sysdig, Aqua.
- Alerts to on-call; automated response (kill pod, isolate with NetworkPolicy).
- Audit logs + NetworkPolicies to limit lateral movement.

Example:

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: {name: disallow-latest-tag}
spec:
  validationFailureAction: Enforce
  rules:
  - name: require-pinned-tag
    match: {any: [{resources: {kinds: [Pod]}}]}
    validate:
      message: "Image tag latest is not allowed"
      pattern: {spec: {containers: [{image: "!*:latest"}]}}

# Falco rule idea: alert on "Terminal shell in container"
```

What to say to the interviewer: *PodSecurityPolicy was removed in 1.25, so I enforce the restricted Pod Security Standard with Pod Security Admission, plus Gatekeeper or Kyverno for custom rules and signed image verification. Containers run non-root with read-only filesystems, all capabilities dropped and seccomp RuntimeDefault, and at runtime Falco or GuardDuty EKS Runtime Monitoring detects suspicious behaviour like shells in containers.*

Q60. HPA vs VPA vs Karpenter: when would you NOT use each, and how would you simulate HPA behaviour in staging?

In simple words: HPA hires more workers, VPA gives each worker a bigger desk, Karpenter builds more office floors. Each is wrong for some jobs - you do not hire ten managers for a one-manager role, and you do not rebuild the office every hour.

Detailed answer

Tool	What it does	When NOT to use it
HPA	Changes number of pods	Singletons or leader-based/stateful apps that cannot safely run multiple copies; when the bottleneck is elsewhere (DB), so more pods just add load; when CPU is a poor signal (use KEDA or custom metrics)
VPA	Changes CPU/memory requests (recreates pods to apply)	Spiky stateless services needing instant reaction (VPA is slow and restarts pods); together with HPA on the same CPU/memory metric (they fight); restart-sensitive apps. Use `updateMode: "Off"` (recommend only) instead
Karpenter	Adds/removes nodes for pending pods, consolidates	It does not scale pods at all; very steady workloads where consolidation churn (draining nodes) adds disruption without savings; strict stateful/local-storage pods (use do-not-disrupt)

A good combination: **HPA** on replicas, **VPA in recommendation mode** to right-size requests, **Karpenter** for nodes.

Simulate HPA in staging:

1. Same HPA spec and same requests/limits as prod (HPA math depends on requests).
2. Generate load with **k6, hey or Locust** (ramp-up, spike and soak patterns).
3. Watch `kubectl get hpa -w`, `kubectl describe hpa` events, pod counts, latency and node scale-up time.
4. Tune the `behavior` block (scale-up speed, stabilization windows) and repeat; record time-to-scale.

Example:

```
hey -z 5m -c 200 http://api.staging.svc.cluster.local/orders
k6 run --vus 500 --duration 10m spike-test.js
kubectl get hpa api -n staging -w
# NAME  REFERENCE      TARGETS   MINPODS MAXPODS REPLICAS
# api   Deployment/api 180%/70%  3       20      3 -> 8 -> 14

apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: {name: api-vpa}
spec:
  targetRef: {apiVersion: apps/v1, kind: Deployment, name: api}
  updatePolicy: {updateMode: "Off"}      # recommendations only
```

> ***What to say to the interviewer:*** *I avoid HPA for singletons or stateful leaders, avoid VPA for spiky services and never combine HPA and VPA on the same CPU or memory metric, and I remember Karpenter only scales nodes and can add churn for steady workloads. My usual combination is HPA for replicas, VPA in recommendation-only mode for right-sizing and Karpenter for nodes, and I validate HPA in staging with k6 or hey while watching kubectl get hpa -w and tuning the behavior block.*

Q61. How would you implement autoscaling for a multi-tenant workload, including a strategy for tenant isolation?

Also asked as: How do you isolate traffic between environments (dev/test/prod) in a shared AKS cluster?

In simple words: Many families share one apartment building. Each family gets its own flat (namespace) with a fixed electricity quota, VIP families can get their own floor, and the building adds floors when needed - but no family may use up everyone else's electricity.

Detailed answer

Isolation (so one tenant cannot hurt others):

- **Namespace per tenant** (or per environment) with RBAC.
- **ResourceQuota** caps total CPU/memory/pods per tenant - this also implicitly caps how far that tenant's HPAs can scale. **LimitRange** sets default requests/limits so nothing runs BestEffort.
- **PriorityClasses**: premium tenants and system components preempt lower tiers under pressure.
- **Dedicated node pools** for noisy or premium tenants: taints + tolerations and nodeSelector/affinity; **Karpenter NodePools per tenant tier** (for example `premium` On-Demand, `standard` Spot) with their own limits.
- **NetworkPolicy** default-deny between tenant namespaces.
- **Hard isolation** when required (compliance, untrusted code): **vCluster** (virtual control plane per tenant) or **cluster per tenant**.

Environments in a shared AKS cluster: default-deny NetworkPolicy per environment namespace (Azure Network Policy, Calico or Cilium), separate ingress classes/controllers per environment, separate node pools with taints per environment, and Azure RBAC for Kubernetes scoped per namespace. Real advice: **prod should be its own cluster** (blast radius, upgrades, compliance); share only dev/test.

Scaling:

- **HPA per tenant Deployment** on CPU or per-tenant custom metrics (requests per second with a `tenant` label via prometheus-adapter).
- **KEDA** for queue-based tenants (per-tenant queue depth / Kafka lag).
- **VPA in recommend-only mode** to right-size requests; do not mix HPA and VPA on the same metric.
- Cluster level: **Karpenter / Cluster Autoscaler** adds nodes in the right pool.
- **Fairness:** quotas + max replicas per tenant + priority so a single tenant's spike cannot starve others; per-tenant rate limiting at the ingress/API gateway.
- Observe per-tenant usage and cost (Kubecost labels) for chargeback.

Example:

```
apiVersion: v1
kind: ResourceQuota
metadata: {name: tenant-a-quota, namespace: tenant-a}
spec:
  hard: {requests.cpu: "20", requests.memory: 40Gi, limits.memory: 60Gi, pods: "100"}
---
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: {name: premium}
spec:
  template:
    spec:
      taints: [{key: tier, value: premium, effect: NoSchedule}]
      requirements:
      - {key: karpenter.sh/capacity-type, operator: In, values: [on-demand]}
      nodeClassRef: {group: karpenter.k8s.aws, kind: EC2NodeClass, name: default}
  limits: {cpu: "200"}
---
# tenant-a pods (premium)
tolerations: [{key: tier, operator: Equal, value: premium, effect: NoSchedule}]
nodeSelector: {karpenter.sh/nodepool: premium}
priorityClassName: tenant-premium
```

What to say to the interviewer: *I isolate tenants with a namespace each, ResourceQuota and LimitRange, PriorityClasses, default-deny NetworkPolicies and dedicated tainted node pools or Karpenter NodePools per tier, moving to vCluster or a cluster per tenant when hard isolation is required, and for environments I keep prod in its own cluster. Each tenant scales with its own HPA or KEDA trigger on per-tenant metrics, VPA runs in recommendation mode, Karpenter adds nodes, and quotas plus max replicas keep one tenant from starving the others.*

Scenario-Based

Q62. What common Kubernetes errors have you faced, and how did you fix them?

In simple words: Every pod error name is a clue that tells you which stage of the pod's life went wrong: could not be placed, could not get its image, could not start, or started and then died.

Detailed answer

Error / status	What it means	First checks	Typical fixes
Pending	Scheduler cannot place the pod	`kubectl describe pod` events: Insufficient cpu/memory, taints, node affinity, unbound PVC, pod limit per node	Lower requests, add nodes (autoscaler), fix selectors/tolerations, fix StorageClass/AZ
ImagePullBackOff / ErrImagePull	Node cannot pull the image	Events: not found, unauthorized, timeout	Fix tag/repo name, ECR permissions on node role, imagePullSecrets, NAT/VPC endpoints for ECR, rate limits (Docker Hub)
CreateContainerConfigError	Pod spec references something missing	Events: `secret "db-cred" not found`, `configmap ... not found`, missing key	Create the Secret/ConfigMap/key, fix names, check namespace
CrashLoopBackOff	Container starts then keeps exiting	`kubectl logs --previous`, exit code, describe	Fix app config/env, dependencies, command/args, probes
OOMKilled (exit 137)	Container exceeded memory limit	describe -> Last State: OOMKilled; memory graph	Raise memory limit, fix leak, tune JVM/heap (`-XX:MaxRAMPercentage`)
Error / exit 1	App crashed with an error	logs --previous	Fix code/config
RunContainerError / exec format error	Wrong command or wrong CPU architecture	logs, image arch	Fix entrypoint, build multi-arch (amd64/arm64)
Evicted	Node under memory/disk pressure	node conditions, `kubectl get pods --field-selector=status.phase=Failed`	Set requests, clean disk, bigger nodes, log rotation
Terminating forever	Finalizers or unreachable node	describe, finalizers	Fix node, remove stuck finalizer carefully, `--force --grace-period=0` last resort

Example:

```
kubectl get pods -A | grep -vE "Running|Completed"
kubectl describe pod <pod> -n <ns> | sed -n '/Events/,$p'
kubectl logs <pod> -n <ns> --previous
kubectl get events -n <ns> --sort-by=.lastTimestamp | tail -20
```

> ***What to say to the interviewer:*** *The status tells me which stage failed: Pending is scheduling, ImagePullBackOff is registry access or tag, CreateContainerConfigError is a missing Secret or ConfigMap, CrashLoopBackOff is the app exiting, and OOMKilled is memory limit. I always start with describe pod events and logs --previous, then fix the specific cause such as requests, IAM pull permissions, missing config or memory limits.*

Q63. Rapid-fire: What happens if...?

In simple words: Knowing how Kubernetes behaves when each piece breaks lets you guess the cause from the symptom in seconds.

Detailed answer

Situation	What you see	What to do
A pod keeps crashing repeatedly	CrashLoopBackOff; kubelet restarts it with exponential back-off 10s, 20s, 40s... capped at 5 minutes (reset after 10 min of healthy running)	`logs --previous`, exit code, describe events; fix config/dependency/probe
CoreDNS fails	Pods and Services look healthy but apps get "could not resolve host"; calls by IP still work	Check kube-system CoreDNS pods/logs, scale it, check NetworkPolicy on port 53, NodeLocal DNSCache
A node runs out of memory	Node condition MemoryPressure, new pods not scheduled there, kubelet evicts pods (BestEffort first, then Burstable over their requests); containers over their own limit get OOMKilled	Set proper requests/limits, fix leaks, bigger/more nodes, keep system reserved memory
Service selector matches no pods	Service exists but `kubectl get endpoints` is empty; clients get connection refused or timeout	Fix label/selector mismatch or make pods Ready; check targetPort
An unready app gets traffic	No readiness probe, so pod is added to endpoints as soon as the container starts; users get errors during startup/deploys	Add readiness (and startup) probes
etcd fails / loses quorum	API server errors, kubectl fails, nothing can be created, scaled or healed; **already running pods keep running** and serve traffic	Restore quorum (fix/replace member) or restore from snapshot; on EKS/GKE the provider handles etcd
Rolling deploy with bad config	With `maxUnavailable: 0`, new pods fail readiness, old pods keep serving; rollout stalls and after `progressDeadlineSeconds` (default 600s) is marked failed; no probe = bad pods take traffic	`kubectl rollout undo`, fix config; always use readiness probes
Kubelet loses contact with API server	Node goes NotReady/Unknown after the grace period (~40-50s); node is tainted; after the default toleration of 300s (5 min) pods are evicted and Deployments recreate them elsewhere; containers may still run on the isolated node	Fix kubelet/network; StatefulSet pods need node confirmed dead before replacement
HPA scales too aggressively	Replica count spikes, costs jump, node autoscaler adds many nodes, possibly hits ResourceQuota (pods fail to create)	Tune `behavior.scaleUp` policies, `stabilizationWindowSeconds`, sane `maxReplicas`, better metric
Container exits 0 but app is broken	Deployment (restartPolicy Always) restarts it anyway -> may still loop as CrashLoopBackOff "Completed"; if process stays up but app is broken, nothing happens without probes	Liveness probe for hung app, readiness for broken dependency; check why main process exits

Example:

```
kubectl get endpoints <svc>                  # selector problem?
kubectl describe node <node> | grep -A8 Conditions
kubectl get deploy <app> -o jsonpath='{.status.conditions[?(@.type=="Progressing")]}'
kubectl describe hpa <app>
```

> ***What to say to the interviewer:*** *I match symptoms to components: empty endpoints means a selector or readiness problem, name resolution failures with healthy pods point to CoreDNS, an etcd failure freezes the API but does not kill running pods, and a node losing its API connection has its pods evicted after about five minutes. Readiness probes plus maxUnavailable 0 are what save a bad rolling deploy.*

Q64. A pod is stuck in CrashLoopBackOff even though the image is valid. How do you debug and fix it?

Also asked as: Your pod is stuck in CrashLoopBackOff, what is your next step? Half your pods are in CrashLoopBackOff and the team says it works on my machine - you are on call, what do you do live?

> **In simple words:** The car (image) is fine, but it keeps stalling as soon as it starts - maybe no fuel (missing secret), road blocked (database unreachable), or the inspector (liveness probe) keeps switching it off too early.

Detailed answer

CrashLoopBackOff = the container starts, exits, and kubelet keeps restarting it with increasing delay (up to 5 min). The image pulls fine, so the problem is **runtime**.

Live triage order:

1. **Scope first:** which pods/nodes, since when, all replicas or some? Only new ReplicaSet pods -> recent deploy. Only one node -> node issue. Half the pods -> maybe the new version during a rollout, or pods on a specific node group/arch.
2. **What changed?** `kubectl rollout history`, recent Helm/Argo CD sync, ConfigMap/Secret edits.
3. **Logs of the crashed attempt:** `kubectl logs <pod> --previous` (current logs are often empty).
4. **Exit code and reason:** `kubectl describe pod` -> Last State. 1 = app error, 137 = OOMKilled/SIGKILL, 139 = segfault, 143 = SIGTERM, 127 = command not found.
5. **Config/env:** missing `DB_PASSWORD` env from Secret, wrong key name, wrong DB host (localhost vs Service name) - compare with the working environment (`kubectl exec env`, diff values files).
6. **Dependencies:** DB/Kafka reachable? Test from a debug pod (`nc -zv postgres 5432`); NetworkPolicy, SG.
7. **Probes:** liveness too aggressive (short timeout, no startup probe) kills a slow-starting app - events show "Liveness probe failed".
8. **Resources:** OOMKilled -> raise memory limit or fix heap.
9. **Platform differences:** image built for arm64 but nodes amd64 (`exec format error`), `readOnlyRootFilesystem`/non-root breaking writes, missing volume mounts.
10. **Mitigate first if prod is hurting:** `kubectl rollout undo`, then fix calmly.

Example:

```
kubectl get pods -o wide -l app=api           # which nodes, which ReplicaSet hash
kubectl logs api-6f9c-abcde --previous
kubectl describe pod api-6f9c-abcde | grep -A6 "Last State"
#   Last State: Terminated  Reason: Error  Exit Code: 1
kubectl exec -it api-6f9c-xyz -- env | grep DB_  # compare with healthy pod
kubectl get secret db-cred -o jsonpath='{.data}' | head -c 200
kubectl debug -it api-6f9c-abcde --image=nicolaka/netshoot --target=api
kubectl rollout undo deploy/api
```

What to say to the interviewer: *I scope the problem first, check what changed, and read kubectl logs --previous and the exit code from describe pod. With a valid image the usual culprits are missing env or secrets like DB_PASSWORD, an unreachable database, an over-aggressive liveness probe, OOMKilled or an architecture mismatch; if users are affected I roll back first with kubectl rollout undo and then fix the root cause.*

Q65. A containerized app works on a developer machine (or locally) but fails or crashes in Kubernetes / production. What is your debug checklist?

In simple words: The dish tastes fine in your home kitchen but not in the restaurant. Same recipe, different kitchen: different ingredients (config), different stove (CPU architecture), smaller pots (memory limits), stricter health inspector (probes, security rules).

Detailed answer

Checklist - compare local vs cluster:

1. **Same image?** Local may use a locally-built or `latest` tag; cluster pulls a different digest. Pin tags/digests.
2. **Env and config:** missing env vars, Secrets, ConfigMaps; `.env` file exists only on the laptop.
3. **Networking/DNS:** app connects to `localhost:5432` locally; in K8s it must be `postgres.db.svc.cluster.local`. NetworkPolicies, SGs, egress to external APIs.
4. **Resources:** laptop has 16 GB, pod limit 256Mi -> OOMKilled; CPU limit -> slow startup -> liveness kills it.
5. **Probes:** wrong path/port, too short timeouts.
6. **Security context:** runs as non-root / read-only root filesystem in K8s; app tries to write to `/app/tmp` -> permission denied. Mount an `emptyDir`.
7. **Architecture:** built on Apple Silicon (arm64), nodes are amd64 -> `exec format error`. Build multi-arch with buildx.
8. **Filesystem/volumes:** files expected in a path that is not mounted; case-sensitive paths.
9. **Time/locale, certificates:** missing CA certs in slim image for HTTPS calls.
10. **Signals:** app ignores SIGTERM or runs as a shell script child, so it gets killed on each rollout.

Example:

```
kubectl logs <pod> --previous
kubectl describe pod <pod>                    # OOMKilled? probe failures? mount errors?
kubectl get pod <pod> -o jsonpath='{.spec.containers[0].image}'
docker manifest inspect myrepo/app:1.2 | grep architecture
kubectl exec -it <pod> -- sh -c 'id; env | sort; ls -ld /app'
docker buildx build --platform linux/amd64,linux/arm64 -t myrepo/app:1.2 --push .
```

What to say to the interviewer: *I diff the two environments: exact image digest, env vars and secrets, service DNS names instead of localhost, memory and CPU limits, probes, security context like non-root and read-only filesystems, and CPU architecture. logs --previous and describe pod usually point to one of these within minutes.*

Q66. A deployment rollout is stuck and new pods are not becoming ready. What steps do you follow?

In simple words: New workers were hired but they never pass the "ready to work" check, so the manager will not let the old workers go home - the handover is frozen.

Detailed answer

1. **See the rollout state:** `kubectl rollout status` and Deployment conditions (`ProgressDeadlineExceeded`?).
2. **Which pods and what state:** new ReplicaSet pods Pending, CrashLoopBackOff, or Running but not Ready?
3. **Pending:** insufficient CPU/memory (maxSurge needs extra capacity), node selector/taints, PVC not bound, ResourceQuota exceeded (check ReplicaSet events: `exceeded quota`).
4. **Running but 0/1 Ready:** readiness probe failing - wrong path/port, app not listening, dependency down, probe timeout too short. `kubectl describe` shows `Readiness probe failed: HTTP probe failed with statuscode: 503`.
5. **Crashing:** logs --previous (see CrashLoopBackOff answer).
6. **Image:** ImagePullBackOff - wrong tag.
7. **Is the Deployment paused?** `kubectl rollout resume`.
8. **Decide:** if production impact or deadline -> `kubectl rollout undo` (or `helm rollback`). Old pods are still serving if `maxUnavailable: 0`.
9. **Prevent:** `progressDeadlineSeconds`, Helm `--atomic`, test probes in staging, canary.

Example:

```
kubectl rollout status deploy/api --timeout=120s
kubectl get rs -l app=api                        # new RS: DESIRED 1, READY 0
kubectl get pods -l app=api
kubectl describe pod <new-pod> | sed -n '/Conditions/,$p'
kubectl describe rs <new-rs> | tail              # quota / admission errors
kubectl rollout undo deploy/api
kubectl rollout history deploy/api
```

What to say to the interviewer: *I check rollout status and the new ReplicaSet's pods: Pending points to capacity, quota or scheduling constraints, Running but not Ready points to the readiness probe or a dependency, and crashes send me to logs --previous. If production is impacted I roll back immediately, since old pods are still serving, and then fix the probe, config or capacity issue.*

Q67. A kubectl rollout restart did not trigger new pods as expected. What could be wrong?

In simple words: You pressed the restart button, but either you pressed it on the wrong machine, the machine was paused, or the new parts could not be installed, so the old ones stayed.

Detailed answer

`kubectl rollout restart` works by adding/updating the annotation `kubectl.kubernetes.io/restartedAt` in the Pod template, which triggers a normal rolling update. Possible causes:

1. **Wrong target:** wrong namespace/context/cluster, or wrong resource name (restarted a different Deployment).
2. **Deployment is paused** (`spec.paused: true`) - template changes are recorded but not rolled out.
3. **New pods cannot become Ready/scheduled** - they are Pending (no capacity, quota) or failing readiness; with `maxUnavailable: 0` the old pods remain, so it looks like nothing happened.
4. **GitOps/Helm overwrote it:** Argo CD with self-heal reverts the annotation immediately (drift), or an operator manages the Deployment and resets the template.
5. **Resource is not a Deployment/StatefulSet/DaemonSet** (plain pods or a Job cannot be rolled out-restarted).
6. **StatefulSet with `OnDelete` update strategy** or a `partition` set - pods are not replaced automatically.
7. **Old kubectl version** (restart was added in 1.15) or RBAC denies `patch`.
8. Pods were restarted but you are looking at pod age of other pods, or the rollout is just slow (large `minReadySeconds`).

Example:

```
kubectl config current-context; kubectl get deploy api -n prod
kubectl rollout restart deploy/api -n prod
kubectl get deploy api -n prod -o jsonpath='{.spec.paused}{"\n"}'
kubectl get deploy api -n prod -o jsonpath='{.spec.template.metadata.annotations}'
kubectl rollout status deploy/api -n prod
kubectl get rs -n prod -l app=api
kubectl get sts db -o jsonpath='{.spec.updateStrategy}'
```

What to say to the interviewer: *rollout restart just patches a restartedAt annotation into the pod template, so I check whether I hit the right context and namespace, whether the Deployment is paused, whether new pods are stuck Pending or not Ready, and whether Argo CD self-heal or an operator reverted the change. For StatefulSets I also check for an OnDelete strategy or partition.*

Q68. A pod is failing because of a ConfigMap change. How would you roll back and prevent this next time?

In simple words: Someone changed the recipe card on the kitchen wall and now every new dish is wrong. Put the old card back quickly, then make a rule that recipe changes are tested and versioned like everything else.

Detailed answer

Roll back now:

1. Identify the change: `kubectl describe cm`, Git history / Argo CD diff, audit logs.
2. **Restore the previous ConfigMap** (from Git: revert the commit and let Argo CD sync, or `kubectl apply -f` the old version, or `helm rollback` the release that contains it).
3. **Restart pods** so they pick it up (env vars are read only at start): `kubectl rollout restart deploy/<app>`. Note: `kubectl rollout undo` alone does **not** restore ConfigMap content, because ConfigMaps are not part of Deployment revisions.
4. Verify pods Ready and errors gone.

Prevent:

- Manage ConfigMaps in **Git + Helm/Argo CD**, reviewed PRs, no `kubectl edit` in prod.
- **Immutable, versioned ConfigMaps:** name includes a hash (`app-config-7f9c2`, Kustomize `configMapGenerator`), so a change creates a new ConfigMap and a new Deployment revision -> `rollout undo` then really rolls back config too.
- Helm **checksum annotation** so config changes trigger a proper rolling update gated by readiness probes (bad config stops at the first pod instead of silently breaking all pods on next restart).
- Validate config in CI (schema, `helm template`, app `--validate-config`), test in staging, canary.
- `immutable: true` on ConfigMaps to prevent in-place edits.

Example:

```
# templates/deployment.yaml
spec:
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}

# kustomization.yaml
configMapGenerator:
- name: app-config
  files: [app.properties]        # generates app-config-<hash>

git revert <commit> && git push        # Argo CD syncs old config
kubectl rollout restart deploy/api -n prod
```

> ***What to say to the interviewer:*** *I restore the previous ConfigMap from Git or helm rollback and restart the Deployment, remembering rollout undo alone does not revert ConfigMap data. To prevent it, config lives in Git with review, and I use hashed or immutable ConfigMaps or a Helm checksum annotation so every config change becomes a normal rolling update gated by readiness probes and easy to roll back.*

Q69. The dev team pushed a bad Helm chart and the rollback hook fails. What is your live rollback strategy?

In simple words: The emergency exit door (rollback hook) is jammed. You do not stand there pushing it - you use the side door (skip hooks or roll the Deployment back directly), get everyone out, and fix the door later.

Detailed answer

Step by step:

1. **See release state:** `helm history <rel>` and `helm status` - find the last good revision and whether the release is `failed` or stuck in `pending-upgrade` / `pending-rollback`.
2. **Rollback without hooks:** `helm rollback <rel> <good-rev> --no-hooks --wait --timeout 5m`. This skips the broken pre/post-rollback hook Job.
3. **Stuck in pending state** ("another operation is in progress"):

- First try `helm rollback` to the last good revision.

- Last resort: the release state is stored as Secrets `sh.helm.release.v1.<rel>.v<N>` in the release namespace; delete (after backing up) the Secret of the stuck pending revision, so Helm sees the last deployed revision again, then roll back/upgrade.

4. **Fastest app-level escape:** if Helm itself is broken, `kubectl rollout undo deploy/<app>` restores the previous ReplicaSet in seconds and stops user impact; reconcile Helm afterwards (next `helm upgrade` or rollback aligns it).
5. **Clean up failed hook Jobs** (`kubectl delete job <hook-job>`) that block reruns.
6. **GitOps:** if Argo CD manages it, **revert the Git commit** (or roll back the Application to the previous synced revision; pause auto-sync first) - otherwise Argo CD will re-apply the bad chart.
7. Verify: pods Ready, error rate, then communicate.

Prevent: `helm lint`, `helm template` + kubeconform, `helm diff` in CI; `--atomic --wait --timeout`; `helm.sh/hook-delete-policy: before-hook-creation,hook-succeeded`; hooks idempotent; deploy to staging first; canary.

Example:

```
helm history api -n prod
# REV  STATUS           CHART       DESCRIPTION
# 11   superseded       api-1.8.0   Upgrade complete
# 12   pending-upgrade  api-1.9.0   Preparing upgrade
helm rollback api 11 -n prod --no-hooks --wait --timeout 5m

kubectl get secret -n prod -l owner=helm,name=api
kubectl get secret sh.helm.release.v1.api.v12 -n prod -o yaml > backup-v12.yaml
kubectl delete secret sh.helm.release.v1.api.v12 -n prod     # last resort

kubectl rollout undo deploy/api -n prod                    # fastest app-level escape
```

What to say to the interviewer: *I check helm history, then run helm rollback to the last good revision with --no-hooks so the broken hook is skipped; if the release is stuck in pending-upgrade I roll back or, as a last resort, remove that revision's sh.helm.release secret after backing it up. If Helm is unusable, kubectl rollout undo restores service in seconds, and under Argo CD I revert the Git commit; prevention is lint, template and diff in CI plus --atomic and proper hook delete policies.*

Q70. Users cannot access an application exposed via Ingress (or the Ingress stopped working). How do you debug it?

Also asked as: Your service is not accessible externally - where do you start troubleshooting? If an Ingress stops working, how would you fix it?

In simple words: Follow the path of a letter: street address (DNS) -> building gate (load balancer) -> receptionist (Ingress controller) -> office (Service) -> person (Pod). Find the first place where the letter stops.

Detailed answer

Work outside-in (or inside-out - both fine, just be systematic):

1. **DNS:** `dig app.example.com` - does it resolve to the current LB address? (LB recreated -> new DNS name; Route 53 record stale.)
2. **Load balancer:** exists? listeners 80/443? **target group health** - unhealthy targets mean health-check path/port or SG problem. Security Group allows client traffic; subnets tagged correctly.
3. **TLS:** certificate valid and matches host (`curl -vk`); ACM/cert-manager renewal failed?
4. **Ingress controller:** pods running? `kubectl logs` of ingress-nginx / aws-load-balancer-controller - errors reconciling the Ingress (bad annotation, missing subnets, IAM permissions)? `ingressClassName` correct?
5. **Ingress rules:** `kubectl describe ingress` - host/path correct, backend service and port exist, pathType, rewrite rules.
6. **Service:** `kubectl get endpoints` - empty = selector mismatch or pods not Ready; `targetPort` matches container port.
7. **Pods:** Ready? app listening on that port? `kubectl port-forward` to pod and Service to test, bypassing ingress.
8. **Network policy** blocking traffic from ingress controller namespace; WAF rules blocking (403).
9. Check what changed recently (deploy, cert, annotation, controller upgrade).

Example:

```
dig +short app.example.com
curl -sv https://app.example.com/health -o /dev/null
kubectl get ingress -n shop; kubectl describe ingress shop -n shop
kubectl logs -n kube-system deploy/aws-load-balancer-controller | tail -50
kubectl get svc,endpoints backend -n shop
kubectl port-forward svc/backend 8080:80 -n shop; curl localhost:8080/health
aws elbv2 describe-target-health --target-group-arn <tg-arn>
```

What to say to the interviewer: *I trace the request path hop by hop: DNS resolution, load balancer listeners and target health, TLS, ingress controller logs and the Ingress rules, then Service endpoints and finally the pods with port-forward. Empty endpoints, wrong targetPort, unhealthy targets from a bad health-check path, and controller errors from annotations or IAM are the most common causes.*

Q71. Kubernetes pods are healthy, but users are receiving errors (for example 503). How do you troubleshoot the complete request flow?

Also asked as: You are paged for 503 errors but all pods look healthy - where do you look next: metrics, probes, Ingress?

> **In simple words:** Every worker says "I am fine", yet customers are still turned away at the door. The problem is somewhere on the path between the door and the workers, or the workers are "fine" only because the health check is too easy.

Detailed answer

Request flow: **Client -> DNS -> CDN/WAF -> Load balancer -> Ingress controller -> Service -> Pod -> downstream (DB/cache/APIs).**

1. **Where are errors generated?** Look at who returns the 503: LB metrics (ELB 5xx vs Target 5xx), ingress controller access logs (`upstream connect error`, `no live upstreams`), or the app itself.
2. **Endpoints/readiness:** `kubectl get endpoints` - maybe only a few pods are Ready; readiness flapping; probes too shallow (pod "healthy" but app returns 503 for real requests).
3. **LB target health:** targets draining/unhealthy; target type (instance vs IP); `externalTrafficPolicy: Local` nodes without pods fail health checks.
4. **Capacity:** HPA already at `maxReplicas`? pods saturated (CPU throttling, thread/connection pool exhausted), ingress `max connections`/rate limits, app queue full -> 503.
5. **Timeouts and keep-alive:** app keep-alive timeout shorter than LB/ingress idle timeout -> intermittent 502/503 when the app closes a connection the proxy reuses. Set app keep-alive > LB idle timeout.
6. **Deploys/drains:** errors only during rollouts -> missing preStop/graceful shutdown; PDB missing during node drains/Karpenter consolidation.
7. **Downstream:** DB max connections, Redis latency, third-party API failing - app returns 503 when a dependency is down. Traces (Tempo/Jaeger) show the slow span.
8. **Network:** NetworkPolicy, DNS latency, conntrack table full, SNAT/NAT port exhaustion.
9. **Segment the errors:** by path, pod, node, AZ, version, client region - a pattern points to the cause.

Example:

```
kubectl get endpoints api -n prod -o wide
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller | grep -E ' 50[234] ' | tail
kubectl get hpa -n prod; kubectl top pods -n prod
kubectl get events -n prod --sort-by=.lastTimestamp | tail
# PromQL: error rate by pod
sum by (pod) (rate(http_requests_total{status=~"5.."}[5m]))
aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB \
  --metric-name HTTPCode_ELB_5XX_Count ...
```

> ***What to say to the interviewer:*** *I follow the request through DNS, load balancer, ingress, Service and pods, first identifying which layer generates the 503 from LB metrics and ingress logs. Then I check endpoints and readiness, LB target health, HPA at max or saturated pods, keep-alive and timeout mismatches, graceful shutdown during deploys and downstream dependencies, segmenting errors by pod, node and zone to find the pattern.*

Q72. A Service is running but application requests are timing out. How do you troubleshoot the networking?

In simple words: The phone number exists and the line rings, but nobody answers or the call drops. Either the call is not reaching the right person, or the person is too slow to pick up.

Detailed answer

Timeouts (not immediate refusal) usually mean packets are dropped or the app is too slow. Steps:

1. **Endpoints:** `kubectl get endpoints` - pods listed? If empty, clients may hang/refuse.
2. **Bypass layers to localize:**

- Call the **pod IP:port** directly from another pod -> works? then Service/kube-proxy layer.
- Call the **ClusterIP** -> works? then DNS or client side.
- Call the **DNS name** -> slow? DNS issue.

3. **Port mapping:** Service `port` -> `targetPort` must match the port the app listens on (`ss -lnt` in the pod). App bound to `127.0.0.1` instead of `0.0.0.0` is a classic.
4. **NetworkPolicy:** a default-deny drops packets silently -> timeout.
5. **kube-proxy/CNI:** kube-proxy pods healthy, iptables/IPVS rules present; CNI (aws-node) errors, IP exhaustion in subnet.
6. **Security Groups / NACLs** (EKS SG for pods, node SG between nodes), cross-node traffic blocked.
7. **App slowness:** CPU throttling, thread pool exhaustion, DB slow queries; check latency metrics and traces - "network timeout" is often a slow app.
8. **Conntrack table full** (`nf_conntrack: table full, dropping packet` in dmesg), MTU issues (large responses hang).
9. **Client timeouts** too short vs normal response time.

Example:

```
kubectl run net --rm -it --image=nicolaka/netshoot -- bash
  curl -m 3 -v http://10.0.45.12:8080/health      # pod IP
  curl -m 3 -v http://10.100.12.34/health          # ClusterIP
  curl -m 3 -v http://api.prod.svc.cluster.local/health
  nc -zv api.prod 80 ; dig api.prod.svc.cluster.local
kubectl exec <api-pod> -- ss -lntp                 # 0.0.0.0:8080 or 127.0.0.1:8080?
kubectl get networkpolicy -n prod
kubectl logs -n kube-system -l k8s-app=kube-proxy | tail
```

What to say to the interviewer: *I localize the timeout by testing pod IP, then ClusterIP, then DNS name from a netshoot pod, which tells me if it is the app, kube-proxy and Service layer, or DNS. Then I check targetPort versus the actual listening port and bind address, NetworkPolicies and security groups that silently drop packets, kube-proxy and CNI health, conntrack, and finally whether the app itself is just slow.*

Q73. Pod A cannot reach Pod B. How do you debug it?

In simple words: Two neighbours cannot visit each other. Check the address (DNS), whether the door is open (port), whether a guard blocks the gate (NetworkPolicy/SG), and whether the road exists (CNI/routing).

Detailed answer

1. **Are both pods Running and Ready,** and what are their IPs/nodes? (`-o wide`)
2. **How is A calling B?** Pod IP, Service name, or FQDN? Across namespaces it must be `svc-b.ns-b` or `svc-b.ns-b.svc.cluster.local`.
3. **DNS:** from pod A: `nslookup svc-b.ns-b.svc.cluster.local`.
4. **Service:** selector matches B's labels, endpoints populated, port/targetPort correct.
5. **Direct pod IP test:** `curl <podB-IP>:<port>` from A.

- Fails only across nodes but works on same node -> CNI/overlay/SG between nodes.
- Fails everywhere -> app not listening or NetworkPolicy.

6. **NetworkPolicy** on B's namespace (ingress) or A's (egress, including DNS egress).
7. **CNI health:** aws-node/calico/cilium pods, IP allocation errors; `cilium connectivity test` / Hubble for flow drops.
8. **Security Groups for pods / node SGs** on EKS; host firewall.
9. **Service mesh:** mTLS STRICT mode and pod A without sidecar -> connection reset.

Example:

```
kubectl get pod -o wide -n ns-a; kubectl get pod -o wide -n ns-b
kubectl exec -n ns-a podA -- nslookup svc-b.ns-b.svc.cluster.local
kubectl exec -n ns-a podA -- curl -m 3 -sv http://svc-b.ns-b:8080/health
kubectl exec -n ns-a podA -- curl -m 3 -sv http://<podB-ip>:8080/health
kubectl get endpoints svc-b -n ns-b
kubectl get networkpolicy -n ns-b -o yaml
kubectl get pods -n kube-system -l k8s-app=aws-node
```

> ***What to say to the interviewer:*** *I confirm both pods are ready, then test from pod A by FQDN, by Service ClusterIP and by pod B's IP to isolate DNS, Service and network layers. After that I check selectors and endpoints, NetworkPolicies in both namespaces including DNS egress, CNI health and security groups between nodes, and mesh mTLS settings.*

Q74. Service discovery fails across namespaces, but DNS looks green. What is your next step, and how do you confirm the root cause?

Also asked as: A pod cannot resolve service names, yet DNS looks fine - what is your next move?

> **In simple words:** The phone directory works for people on your own floor, but when you look up someone on another floor using only their first name, you get nothing - you need their full name and floor, and the corridor door must be open.

Detailed answer

"DNS looks green" (CoreDNS pods Running, no errors) does not mean the **name the app uses** resolves or the service behind it works. Next steps:

1. **Short name vs FQDN:** a short name `payments` only resolves inside the same namespace because of `search` domains in `/etc/resolv.conf` (`<ns>.svc.cluster.local svc.cluster.local cluster.local`). Cross-namespace needs `payments.billing` or `payments.billing.svc.cluster.local`. Check the app config.
2. **Test from the failing pod itself** (not from your laptop or a random pod):

- `nslookup payments.billing.svc.cluster.local` -> returns ClusterIP? Then DNS is really fine and the issue is further along.
- NXDOMAIN on short name but FQDN works -> config issue (wrong name/namespace).

3. **Pod's resolv.conf / dnsPolicy:** `hostNetwork` pods or `dnsPolicy: Default` use node DNS, not CoreDNS; custom `dnsConfig` may override search domains.
4. **Service has endpoints?** `kubectl get endpointslices -n billing -l kubernetes.io/service-name=payments` - empty = selector mismatch or pods not Ready. DNS still resolves the ClusterIP, but connections fail.
5. **Ports:** Service port vs `targetPort` vs container port; named port mismatch.
6. **NetworkPolicy** in the target namespace allowing only same-namespace traffic (namespaceSelector missing), or egress policy in the source namespace blocking.
7. **Curl the pod IP directly** from the source pod - works? then Service/kube-proxy (iptables rules) problem; check kube-proxy logs.
8. **Mesh / headless:** headless Service returns pod IPs; ExternalName misconfigured.

Confirm root cause by reproducing: FQDN nslookup -> ClusterIP, EndpointSlice contents, curl pod IP vs ClusterIP, and a temporary NetworkPolicy allow test in a non-prod namespace.

Example:

```
kubectl exec -n orders deploy/api -- cat /etc/resolv.conf
# search orders.svc.cluster.local svc.cluster.local cluster.local
# options ndots:5
kubectl exec -n orders deploy/api -- nslookup payments                    # NXDOMAIN
kubectl exec -n orders deploy/api -- nslookup payments.billing.svc.cluster.local
kubectl get endpointslices -n billing -l kubernetes.io/service-name=payments
kubectl exec -n orders deploy/api -- curl -m 3 -sv http://payments.billing:8080/health
kubectl exec -n orders deploy/api -- curl -m 3 -sv http://<payments-pod-ip>:8080/health
kubectl get networkpolicy -n billing -o yaml | grep -A5 namespaceSelector
```

> ***What to say to the interviewer:*** *I test from the failing pod itself: the short name only resolves within its own namespace through the search domains, so cross-namespace calls need name.namespace or the full svc.cluster.local FQDN. If the FQDN resolves, I check EndpointSlices for a selector mismatch, targetPort, NetworkPolicies that block cross-namespace traffic and kube-proxy, confirming by comparing curl to the ClusterIP versus the pod IP.*

Q75. Cluster-wide DNS stops resolving randomly across nodes. What is your step-by-step debug path?

> **In simple words:** The phone directory office answers most calls, but randomly drops some. Either too few operators (under-scaled), too many pointless calls (ndots), the phone lines get crossed (conntrack race), or the head office (upstream resolver) limits how many calls it takes.

Detailed answer

1. **Confirm and measure:** run a loop test from dnsutils pods on several nodes; note failure rate, timeouts vs NXDOMAIN/SERVFAIL, and whether failures are ~5 seconds long.
2. **CoreDNS health:** pods Running, restarts, OOMKilled, CPU throttling; logs (`SERVFAIL`, `i/o timeout` to upstream); metrics `coredns_dns_request_duration_seconds`, `coredns_dns_responses_total{rcode="SERVFAIL"}`, `coredns_forward_*`.

3. **Under-scaled:** only 2 CoreDNS replicas for a big cluster -> use **cluster-proportional-autoscaler** / more replicas, spread across nodes/AZs, raise limits.
4. **ndots:5 amplification:** every external name like `api.stripe.com` is tried with each search domain first (4-5 extra queries, A and AAAA). Fix: use FQDN with a trailing dot, or set `dnsConfig.options ndots: 2` for heavy callers; enable CoreDNS `autopath`/cache.
5. **Conntrack race (the famous 5-second timeouts):** glibc sends A and AAAA queries in parallel over UDP from the same socket; a Linux conntrack race drops one, and the client retries after 5 seconds. Fixes: **NodeLocal DNSCache** (node-local cache, upstream over TCP), `options single-request-reopen` (glibc), or avoiding musl/alpine quirks.
6. **Upstream limits:** on AWS the **VPC resolver allows 1024 packets per second per ENI**; CoreDNS pods concentrated on few nodes exceed it and packets drop. Check `linklocal_allowance_exceeded` in ENA stats (`ethtool -S`). Fix: NodeLocal DNSCache, more CoreDNS replicas spread across nodes, caching.
7. **NetworkPolicy** blocking UDP/TCP 53 to kube-system for some namespaces (big responses fall back to TCP).
8. **kube-proxy health:** stale iptables rules pointing to deleted CoreDNS pod IPs after restarts; kube-proxy errors.
9. **Node-level:** conntrack table full (`dmesg | grep conntrack`), CNI issues on specific nodes.
10. Fix, then keep **alerts** on CoreDNS latency/SERVFAIL and a synthetic DNS probe.

Example:

```
kubectl run dnsutils --rm -it --image=registry.k8s.io/e2e-test-images/jessie-dnsutils:1.3 -- bash
  for i in $(seq 1 200); do
    start=$(date +%s%N); getent hosts api.prod.svc.cluster.local >/dev/null || echo FAIL
    echo $(( ($(date +%s%N)-start)/1000000 ))ms
  done | sort | uniq -c | sort -rn | head
  dig +search +stats payments.billing
kubectl get pods -n kube-system -l k8s-app=kube-dns -o wide
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=100 | grep -iE "servfail|timeout"
kubectl top pods -n kube-system -l k8s-app=kube-dns
# on a node:
ethtool -S eth0 | grep linklocal_allowance_exceeded
dmesg | grep -i conntrack
```

What to say to the interviewer: *I measure with a DNS loop test from several nodes, then check CoreDNS pods, logs and latency and SERVFAIL metrics, scaling CoreDNS if it is under-provisioned. Classic causes are ndots:5 query amplification, the UDP conntrack race that causes 5-second timeouts, and on AWS the VPC resolver limit of 1024 packets per second per ENI; NodeLocal DNSCache fixes most of these, along with checking NetworkPolicies on port 53 and kube-proxy health.*

Q76. One worker node becomes NotReady. What actions do you take?

In simple words: A branch office stopped answering the head office's calls. Is the office closed (machine dead), is the manager sick (kubelet), or is the phone line cut (network)? Meanwhile, move urgent work to other offices.

Detailed answer

1. **Confirm and scope:** one node or many? same AZ/node group? `kubectl get nodes`, `kubectl describe node` -> Conditions (Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable) and events.
2. **Protect workloads:** `kubectl cordon` to stop new pods; after ~5 minutes pods are evicted automatically (taint + toleration 300s); for faster recovery `kubectl drain --ignore-daemonsets --delete-emptydir-data`.
3. **Check the machine:** EC2 status checks, instance still running? Spot interruption? (cloud console / `aws ec2 describe-instance-status`).
4. **On the node (SSM Session Manager/SSH):**

- `systemctl status kubelet`, `journalctl -u kubelet` - crashed? certificate expired? cannot reach API endpoint?

- `systemctl status containerd` - runtime down.

- Disk full (`df -h`, images/logs), memory exhaustion (`free -m`, `dmesg | grep -i oom`), PID exhaustion.

- CNI: aws-node pod on that node failing, out of IPs.

- Network: SG/NACL change, route, DNS to API endpoint.

5. **Fix or replace:** restart kubelet/containerd, clean disk; in the cloud it is usually faster and safer to **terminate the node** and let the node group/Karpenter replace it (cattle, not pets).
6. **Root cause + prevention:** kubelet `system-reserved`/`kube-reserved`, eviction thresholds, log rotation, image GC, node problem detector, alerts on NotReady.

Example:

```
kubectl get nodes
kubectl describe node ip-10-0-2-40.ap-south-1.compute.internal | sed -n '/Conditions/,/Addresses/p'
kubectl cordon ip-10-0-2-40...
kubectl drain ip-10-0-2-40... --ignore-daemonsets --delete-emptydir-data --timeout=5m
aws ssm start-session --target i-0abc123
  sudo systemctl status kubelet containerd
  sudo journalctl -u kubelet --since "30 min ago" | tail -50
  df -h ; free -m ; dmesg | grep -iE "oom|conntrack" | tail
aws ec2 terminate-instances --instance-ids i-0abc123      # let ASG/Karpenter replace
```

> ***What to say to the interviewer:*** *I check node conditions and events, cordon and drain the node to protect workloads, then look at the instance status and, on the node, kubelet and containerd logs, disk, memory and CNI. If it is not a quick fix I terminate it and let the node group or Karpenter replace it, then add prevention like reserved resources, eviction thresholds and NotReady alerts.*

Q77. A node went down suddenly. What happens to the pods running on it?

> **In simple words:** When a shop suddenly closes, the head office waits a few minutes in case it reopens, then reassigns its staff to other shops. Staff with their own locked cabinets (StatefulSets with disks) have to wait until it is certain the old shop is truly closed.

Detailed answer

Timeline (default settings):

1. Kubelet stops sending heartbeats (Lease objects).

2. After the **node monitor grace period (~40s, 50s in newer versions)** the node controller marks it **NotReady/Unknown** and adds taints `node.kubernetes.io/unreachable:NoExecute` (or `not-ready`).
3. Pods have a default toleration of **300 seconds** for these taints. After ~5 minutes they are **evicted** (marked Terminating; the kubelet is unreachable so they cannot actually be killed).
4. **Deployment/ReplicaSet pods:** controller creates replacements on healthy nodes right away after eviction. Traffic stops going to the dead pods much earlier, because endpoints are updated when the node is NotReady (pods marked not ready).
5. **StatefulSet pods:** NOT recreated automatically while the old pod still exists (Terminating), to avoid two pods with the same identity writing to the same disk (split brain). You must confirm the node is dead, then `kubectl delete pod --force --grace-period=0` or delete the Node object; in the cloud, the node object is removed when the instance is terminated.
6. **Volumes:** EBS (RWO) must detach from the dead node before attaching elsewhere - can take several minutes (force detach after ~6 min).
7. **DaemonSet pods:** not rescheduled (they belong to that node).
8. Bare pods (no controller) are simply lost.

To recover faster: lower `tolerationSeconds` for critical pods, multiple replicas spread across nodes/AZs, PDBs, fast node replacement.

Example:

```
tolerations:
- {key: node.kubernetes.io/unreachable, operator: Exists, effect: NoExecute, tolerationSeconds: 60}
- {key: node.kubernetes.io/not-ready,   operator: Exists, effect: NoExecute, tolerationSeconds: 60}

kubectl get pods -o wide | grep <dead-node>          # Terminating / Unknown
kubectl delete pod mongo-1 --force --grace-period=0  # only after confirming node is dead
```

> ***What to say to the interviewer:*** *The node is marked NotReady after about 40 to 50 seconds and tainted, pods are removed from endpoints, and after the default 300-second toleration they are evicted and Deployments recreate them on healthy nodes. StatefulSet pods are not replaced until the node is confirmed gone, to avoid split brain, and EBS volumes need to detach first, so I tune tolerationSeconds for critical apps and always spread replicas.*

Q78. A node has reached (exhausted) its Pod capacity. What would you check first, and what would you do?

> **In simple words:** A parking lot has a fixed number of spots. Even if there is space on the ground (free CPU/memory), when all numbered spots are used, no more cars can park.

Detailed answer

What "pod capacity" means: each node has a **max pods** limit (`kubectl describe node` -> Capacity: pods). Default kubelet limit is 110; on **EKS with the VPC CNI** it is based on ENIs x IPs per ENI (e.g. t3.medium = 17 pods, m5.large = 29), because every pod needs a VPC IP.

Check first:

1. `kubectl describe node` -> Capacity/Allocatable pods vs running pods; pending pod events say `Too many pods`.

2. Is it the **pod limit** or **IP exhaustion** (subnet has no free IPs - aws-node logs `failed to assign an IP address`)?

3. What is using the slots: completed/failed pods not cleaned up, many small DaemonSets, a runaway Job/CronJob creating pods, too many tiny replicas?

Fixes:

- Short term: clean up Completed/Evicted pods; let autoscaler add nodes (Cluster Autoscaler/Karpenter respect the pod limit); cordon and spread.

- **EKS prefix delegation** (`ENABLE_PREFIX_DELEGATION=true`) - assigns /28 prefixes, raising pods per node (e.g. up to 110 on many Nitro instances); update node max-pods accordingly.

- Larger instance types (more ENIs/IPs), larger subnets or custom networking (secondary CIDR for pods).

- Set `ttlSecondsAfterFinished` for Jobs, `successfulJobsHistoryLimit` for CronJobs; review replica counts.

- Topology spread so pods do not pile onto one node.

Example:

```
kubectl describe node ip-10-0-1-12... | grep -A6 -E "Capacity|Allocatable"
#  pods: 17
kubectl get pods -A --field-selector spec.nodeName=ip-10-0-1-12... | wc -l
kubectl get pods -A --field-selector=status.phase==Succeeded -o name | xargs kubectl delete
kubectl logs -n kube-system -l k8s-app=aws-node --tail=50 | grep -i "ip address"
kubectl set env daemonset aws-node -n kube-system ENABLE_PREFIX_DELEGATION=true
```

> ***What to say to the interviewer:*** *I first check describe node for the pods capacity versus running count and whether pending pods say Too many pods or the CNI is out of subnet IPs. Short term I clean up completed pods and let the autoscaler add nodes; long term on EKS I enable prefix delegation or use bigger instances and larger subnets, and set TTLs on Jobs so finished pods do not waste slots.*

Q79. Pods are getting evicted frequently due to resource pressure (for example in EKS). How do you fix and prevent it?

> **In simple words:** When the lifeboat is overloaded, the captain throws off the passengers without tickets first. Give every important passenger a proper ticket (requests), do not overload the boat, and keep it clean.

Detailed answer

Eviction happens when a **node** is under pressure (memory, disk/ephemeral storage, PIDs). The kubelet evicts pods in order: **BestEffort** (no requests) first, then **Burstable** pods using more than their request, **Guaranteed** last. (Different from OOMKilled, which is a single container exceeding its own limit.)

Investigate:

1. `kubectl get pods -A | grep Evicted` and `kubectl describe pod` -> reason: `The node was low on resource: memory` / `ephemeral-storage`.

2. Which nodes? `kubectl describe node` conditions and allocated resources; overcommitted limits?

3. Metrics: node memory, disk usage (container logs, images, emptyDir), which pods consume most.

Fix / prevent:

- **Set requests for every pod** (requests close to actual usage), so the scheduler does not overpack nodes. Use LimitRange defaults so nothing runs as BestEffort.
- Memory limits close to requests for heavy apps; critical pods as **Guaranteed** QoS + **PriorityClass**.
- **Ephemeral storage:** set `ephemeral-storage` requests/limits, log rotation, avoid writing big files to container filesystem, bigger root volumes, image garbage collection.
- **Kubelet reservations:** `kube-reserved`, `system-reserved`, eviction thresholds (EKS AMIs set defaults; tune via bootstrap/launch template).
- Right-size node types; autoscaler to add capacity; fix memory leaks.
- Spread with topology constraints; PDBs.
- Alerts on node MemoryPressure/DiskPressure and on eviction counts.
- Clean up evicted pods (they stay as Failed objects).

Example:

```
kubectl get pods -A --field-selector=status.phase=Failed | grep Evicted
kubectl describe pod <evicted-pod> | grep -i -A2 "Reason"
kubectl describe nodes | grep -A10 "Allocated resources"
kubectl top pods -A --sort-by=memory | head

resources:
  requests: {cpu: 250m, memory: 512Mi, ephemeral-storage: 1Gi}
  limits:   {memory: 512Mi, ephemeral-storage: 2Gi}
```

> ***What to say to the interviewer:*** *Evictions mean the node itself is under memory or disk pressure, and BestEffort and over-request Burstable pods go first. I fix it by giving every pod realistic requests, making critical pods Guaranteed with a PriorityClass, setting ephemeral-storage limits and log rotation, reserving resources for kubelet and system, right-sizing nodes, and alerting on node pressure.*

Q80. You see high CPU usage in one pod (or one node group), but its logs look clean. What next?

Also asked as: CPU usage on one EKS node group is consistently high - Grafana shows the spike but logs are normal. A critical microservice in production has high CPU usage - diagnose and resolve it in real time.

> **In simple words:** A worker is sweating heavily but his diary says everything is normal. You need to watch what he is actually doing (profiling), not just read his diary.

Detailed answer

Real-time mitigation first if users are affected: scale out (`kubectl scale` or raise HPA min/max), temporarily raise the CPU limit if it is throttled, or **roll back** if a recent deploy caused it. Then diagnose:

1. **Confirm it is real and scope:** `kubectl top pod --containers`, Grafana per-container CPU; only one replica or all? If only one -> something specific to that pod (hot key, stuck request, bad node). All replicas -> load or code. For a **node group**: `kubectl top nodes`, then which

pods on those nodes use the CPU (often one workload, a DaemonSet, or pods packed onto that group by nodeSelector).

2. **What changed?** Recent deploy (`kubectl rollout history`), config change, traffic increase or new client, a batch job/CronJob landing on that node group.

3. **Is it load?** Compare request rate to that pod - uneven load balancing (long-lived keep-alive/gRPC connections pinned to one pod), a hot partition or hot key.

4. **Is it throttling?** Check `container_cpu_cfs_throttled_periods_total` and compare usage vs **requests vs limits**: usage at the limit = throttled (slow responses, probe timeouts); requests far below usage = node overcommitted.

5. **HPA status:** `kubectl get hpa` - at maxReplicas? not scaling because of missing requests or metrics? (see the HPA scenario).

6. **Look inside the process:** `kubectl exec` -> `top -H` for threads; profilers: Java `jstack`/`jcmd Thread.print` or async-profiler, Go `pprof`, Node `--inspect`, py-spy style samplers; continuous profiling (Pyroscope). GC thrash from memory near heap limit causes high CPU with clean logs.

7. **Traces/APM:** Tempo/Jaeger/Datadog showing slow endpoints.

8. **Background work:** cron-like tasks inside the app, retry storms, infinite loop after a specific input, sidecar (Envoy, log shipper) using CPU rather than the app container - check per-container CPU.

9. **Noisy neighbours / node:** another pod on the same node without CPU requests, kernel issues, instance type too small (burstable t3 out of CPU credits); move the pod (`kubectl delete pod`) to compare.

10. Fix root cause (code, config, balancing, right-sized requests/instance type) and add alerts on throttling and per-pod CPU.

Example:

```
kubectl top pod -n prod --containers | sort -k3 -rn | head
kubectl exec -it api-7d9f-abcde -n prod -- top -H -b -n1 | head -20
kubectl exec -it api-7d9f-abcde -n prod -- jcmd 1 Thread.print > threads.txt
# PromQL: per-pod CPU and throttling
sum by (pod,container) (rate(container_cpu_usage_seconds_total{namespace="prod"}[5m]))
sum by (pod) (rate(container_cpu_cfs_throttled_periods_total{namespace="prod"}[5m]))
```

> ***What to say to the interviewer:*** *If users are affected I first mitigate by scaling out, raising a throttled limit or rolling back a recent deploy. Then I check whether it is one pod, all pods or one node group, what changed, and whether request distribution explains it, such as sticky connections or a hot key. Then I profile inside the container with top -H and language tools like jstack or pprof, check GC pressure, per-container CPU in case a sidecar is the culprit, and throttling metrics; I mitigate by restarting or scaling and then fix the code or balancing issue.*

Q81. etcd becomes unavailable in a production cluster. What is your recovery strategy?

> **In simple words:** The cluster's memory notebook is missing pages. Running work continues, but no new instructions can be written. Fix the notebook if possible; if not, restore it from last night's photocopy.

Detailed answer

First, understand impact: API server errors (`etcdserver: request timed out`, `leader changed`), kubectl fails, no scheduling/healing - but **running pods keep serving**. Do not panic-restart workloads.

Managed (EKS/GKE/AKS): etcd is the provider's responsibility - check the provider health dashboard, open a high-priority support case, avoid mass changes; ensure your own recovery path (GitOps + Velero) is ready.

Self-managed:

1. **Check members:** `etcdctl endpoint health --cluster`, `endpoint status` (leader, DB size, raft index), `member list`.
2. **Common causes:** disk full or DB size exceeded quota (`mvcc: database space exceeded` -> compact + defrag + disarm alarm), slow disk (fsync latency - etcd needs fast SSD), expired certificates, network partition, clock issues.
3. **One member down, quorum intact:** fix or `member remove` the bad one and `member add` a fresh one.
4. **Quorum lost / data corrupt:** restore from the latest **snapshot**:

- stop kube-apiserver (and etcd) on all control-plane nodes,

- `etcdutl snapshot restore` (older: `etcdctl snapshot restore`) on each member with the right `--name`, `--initial-cluster`, new data dir,

- start etcd, verify health, then start API servers,

- reconcile: objects created after the snapshot are lost; GitOps re-applies them.

5. **Prevention:** scheduled snapshots (every few hours, copied off-cluster), 3/5 members across zones, monitoring (`etcd_server_has_leader`, `etcd_disk_wal_fsync_duration_seconds`, DB size), cert rotation, restore drills.

Example:

```
ETCDCTL_API=3 etcdctl --endpoints=$EPS --cacert=ca.crt --cert=etcd.crt --key=etcd.key \
  endpoint status --cluster -w table
etcdctl alarm list
etcdctl compact $(etcdctl endpoint status -w json | jq '.[0].Status.header.revision')
etcdctl defrag --cluster && etcdctl alarm disarm

etcdutl snapshot restore /backup/etcd-2026-09-24.db --name cp1 \
  --initial-cluster cp1=https://10.0.1.10:2380,cp2=https://10.0.2.10:2380,cp3=https://10.0.3.10:2380
     \
  --initial-advertise-peer-urls https://10.0.1.10:2380 --data-dir /var/lib/etcd-restored
```

> ***What to say to the interviewer:*** *Running pods keep serving, so I focus on restoring the control plane: on EKS that is AWS's responsibility and I raise a support case, while self-managed I check member health, fix disk space or quota alarms with compact and defrag, and replace a failed member if quorum still exists. If quorum is lost I restore all members from the latest snapshot and let GitOps re-apply newer changes, and I prevent it with frequent off-cluster snapshots, fast disks and etcd monitoring.*

Q82. HPA refuses to scale a critical workload even though Prometheus shows CPU above 90%. What is the root cause and fix?

In simple words: The fire alarm in the control room (Prometheus) is ringing, but the sprinkler system (HPA) reads a different sensor that says everything is fine - or the sprinkler is already at full blast, or the water pipes (nodes, quota) are blocked.

Detailed answer

Check `kubectl describe hpa` first - conditions and events tell you why. Common root causes:

1. **Different measurement:** HPA CPU utilization = usage / **request**. Prometheus may show % of node CPU or % of **limit**. Example: request 1 core, limit 2 cores, usage 1.8 cores = 90% of limit but HPA sees 180% (would scale) - or the reverse if the dashboard is node-based. Compare apples to apples.
2. **Missing requests:** if any container in the pod (including a sidecar) has no CPU request, HPA cannot compute utilization -> `FailedGetResourceMetric` / `missing request for cpu`.
3. **metrics-server broken/lagging:** targets show <unknown>, APIService unavailable.
4. **Already at maxReplicas** -> `ScalingLimited: TooManyReplicas`.
5. **Scaled, but new pods Pending:** no node capacity (autoscaler not working, max nodes reached), **ResourceQuota** exceeded (events on ReplicaSet), IP exhaustion.
6. **Wrong target:** `scaleTargetRef` points to a different Deployment name, or the Deployment is managed by another tool that resets replicas (Helm/Argo CD setting `replicas`, fighting HPA).
7. **Behavior/stabilization policies** too conservative; unready pods and CPU during startup (`cpu-initialization-period`) are ignored in the calculation.
8. **HPA based on the wrong metric** (memory or custom metric) while CPU is high.
9. Target averaged across pods: one hot pod at 90% but average 50%.

Fix accordingly: set requests on all containers, repair metrics-server, raise maxReplicas and node group max, remove `replicas` from Helm when HPA manages it, adjust quota, tune behavior.

Example:

```
kubectl describe hpa api -n prod
#  AbleToScale     True    ReadyForNewScale
#  ScalingActive   False   FailedGetResourceMetric  missing request for cpu in container "envoy"
#  ScalingLimited  True    TooManyReplicas
kubectl get hpa api -n prod -o yaml | grep -A3 scaleTargetRef
kubectl get pods -n prod -l app=api | grep Pending
kubectl describe quota -n prod
```

What to say to the interviewer: *I read describe hpa conditions first. Typical causes are a container or sidecar without a CPU request, metrics-server not serving metrics, HPA already at maxReplicas, scaled pods stuck Pending due to node capacity or ResourceQuota, a wrong scaleTargetRef, or Helm resetting replicas; I also make sure I compare HPA's usage-over-request with what Prometheus is actually graphing.*

Q83. You are on call: pods are not pulling images (ImagePullBackOff), some are getting evicted, and users are seeing errors or 503s. How do you troubleshoot, and how do you prevent it next time?

Also asked as: Pods show ImagePullBackOff in EKS right after a new image was pushed - what do you check?

In simple words: Three alarms at once. First stop the bleeding for customers, then find whether one root problem (for example a full disk on nodes) is causing all three before fixing each one.

Detailed answer

1. Stabilize and triage (first 5-10 minutes):

- Declare incident, check dashboards: which services, namespaces, nodes, AZs? Recent changes (deploys, node upgrades, registry credential rotation)?
- If a recent deploy caused it -> roll back. Keep healthy old pods serving (do not delete running pods).

2. Image pulls:

- `kubectl describe pod` events: `not found` (wrong tag), `unauthorized`/`no basic auth credentials` (ECR permissions on node role, expired imagePullSecret), `toomanyrequests` (Docker Hub rate limit), `i/o timeout` (NAT Gateway/VPC endpoint issue, registry outage).
- Often linked to evictions: **disk pressure** on nodes triggers image garbage collection and pulls fail due to no space.
- **EKS-specific checks after a new image push:**
- Does the exact **tag/digest exist** in the ECR repo and region the manifest points to? (`aws ecr describe-images`) - typo, tag not pushed yet, or tag overwritten/lifecycle-expired.
- **Node IAM role** has ECR read (`AmazonEC2ContainerRegistryReadOnly` managed policy).
- **Cross-account ECR:** the repository policy in the registry account must allow the node role of the cluster account (`ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`, `ecr:BatchCheckLayerAvailability`).
- **Private nodes without NAT:** need VPC endpoints `ecr.api`, `ecr.dkr` and the **S3 gateway endpoint** (image layers are stored in S3), with SGs allowing 443.
- **Architecture:** image built only for arm64 but nodes are amd64 (or the reverse) -> `no matching manifest for linux/amd64`; build multi-arch.

3. Evictions:

- Node conditions MemoryPressure/DiskPressure; which pods consume most; BestEffort pods without requests.
- Short term: add nodes, cordon bad nodes, clean disk, move critical workloads.

4. User errors / 503s:

- Endpoints for key services - enough Ready pods? HPA at max? Ingress/LB target health.
- Scale critical services up, prioritize with PriorityClass.

5. Prevention:

- Pull from **ECR** (pull-through cache for Docker Hub), pin image tags/digests, VPC endpoints for ECR/S3, `imagePullPolicy: IfNotPresent`.
- Requests/limits everywhere, LimitRange, PriorityClasses, PDBs, ephemeral-storage limits, larger root volumes, log rotation.
- Alerts on ImagePullBackOff, evictions, node pressure, and SLO-based user error alerts.
- Postmortem with timeline and action items.

Example:

```
kubectl get pods -A | grep -E "ImagePull|ErrImage|Evicted|CrashLoop" | awk '{print $1,$4}' | sort |
    uniq -c
kubectl describe pod <pod> | sed -n '/Events/,$p'
#  Failed to pull image ... 403 Forbidden / no space left on device
kubectl describe nodes | grep -E "Name:|DiskPressure|MemoryPressure"
kubectl get endpoints -n prod
kubectl get hpa -A
aws ecr describe-images --repository-name api --image-ids imageTag=1.4.2 --region ap-south-1
aws iam list-attached-role-policies --role-name <eks-node-role>
docker manifest inspect <acct>.dkr.ecr.ap-south-1.amazonaws.com/api:1.4.2 | grep architecture
```

What to say to the interviewer: *I stabilize first by rolling back any recent change and protecting the critical services, then look for a common root cause such as disk pressure on nodes that causes both evictions and failed image pulls. I check pull errors for the tag or digest, node role ECR permissions, cross-account repository policy, ECR and S3 VPC endpoints, image architecture and rate limits, node pressure conditions, and endpoints behind the 503s; prevention is ECR pull-through caching, pinned tags, requests and limits everywhere, priority classes, PDBs and alerts on these states.*

Q84. An app on AKS (or any managed Kubernetes) fails health checks randomly. How do you debug this end-to-end?

Also asked as: How do you configure and troubleshoot health probes in AKS?

In simple words: The doctor checks the patient's pulse every 10 seconds, but gives up if the pulse takes more than 1 second to find. If the patient is sometimes busy, the doctor wrongly declares him sick. Find out whether the patient is really sick or the check is too strict.

Detailed answer

1. **Which health check fails?** Kubernetes probes (events `Liveness/Readiness probe failed`) or the **Azure Load Balancer / Application Gateway** health probe (backend health in portal)? They are different.
2. **Pattern:** random pods or specific nodes? Correlates with traffic peaks, GC, deployments, node upgrades, time of day?
3. **Probe config:** default `timeoutSeconds: 1` is too tight for many apps; probe hitting an expensive endpoint (checks DB); liveness checking dependencies -> restarts during DB blips. Relax timeout/threshold, make probes lightweight, add startup probe.
4. **App/resource side:** CPU throttling (CPU limit too low -> probe response slow), GC pauses, thread pool exhaustion (probe waits in the same queue as user requests), memory pressure.
5. **Node side:** node pressure, kubelet busy, noisy neighbours, SNAT port exhaustion on outbound, conntrack.

6. **Load balancer probe side:** `externalTrafficPolicy: Local` makes nodes without pods fail LB probes (expected); App Gateway probe path/host header/port mismatch, TLS/SNI, probe interval vs app warm-up.
7. **Network:** NSG rules, CNI (Azure CNI IP exhaustion), DNS delays in dependency checks.
8. **Correlate** with metrics/logs (Container Insights / Prometheus): probe failures vs CPU throttling vs latency vs GC.
9. Fix, then load test to confirm.

Example:

```
kubectl get events -A --field-selector reason=Unhealthy --sort-by=.lastTimestamp | tail -20
kubectl describe pod <pod> | grep -A3 -E "Liveness|Readiness"
kubectl top pod <pod> --containers
kubectl exec <pod> -- sh -c 'time wget -qO- localhost:8080/healthz'

livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  timeoutSeconds: 3
  periodSeconds: 10
  failureThreshold: 3
```

> ***What to say to the interviewer:*** *I first separate Kubernetes probe failures from cloud load balancer probe failures, then look for a pattern by node, time and load. Most often the cause is a probe timeout that is too tight combined with CPU throttling, GC pauses or a heavy probe endpoint, so I make probes lightweight, relax timeouts, add a startup probe and fix resources, and verify LB probe settings like path, port and externalTrafficPolicy.*

Q85. In a canary deployment to production, half the traffic returns 502 while the rest succeeds. How do you troubleshoot?

> **In simple words:** Half the customers go to the new counter and get turned away, the other half go to the old counter and are served. The first suspicion is the new counter - but also check the signpost that splits the queue.

Detailed answer

1. **Stop the bleeding:** 50% errors is too much - set canary weight back to 0 / abort the rollout (Argo Rollouts abort, Istio weights 100/0). Then debug calmly.
2. **Confirm the split:** break errors down by **version/subset label**, pod and node. If all 502s come from canary pods -> the canary is broken. If spread across both -> infra/route issue.
3. **502 meaning:** the proxy (ingress/Envoy/ALB) got an **invalid or no response from upstream**: connection refused/reset, upstream closed connection, protocol mismatch.
4. **Canary pods:** Ready? Listening on the right port (targetPort changed in new version)? Crashing under load? Logs of the canary pods and of the ingress/Envoy (`upstream connect error or disconnect/reset before headers`, response flags `UF`, `UC`, `URX`).
5. **Routing config:** canary Service selector correct; Istio **DestinationRule subsets** match pod labels (`version: v2`); mTLS mode mismatch (canary pods missing sidecar injection -> resets); NGINX canary annotations pointing to right Service; ALB weighted target group health.
6. **Protocol/timeouts:** new version switched to HTTP/2, gRPC or HTTPS; app keep-alive timeout lower than proxy idle timeout (connection reuse -> 502); new version slower than proxy

timeout.

7. **Dependencies:** canary uses a new config/secret/DB migration that is failing.
8. Fix, re-run canary at 1-5% with **automated analysis** (error rate, latency) to catch this before 50%.

Example:

```
# errors by version
sum by (version) (rate(istio_requests_total{destination_app="api",response_code="502"}[1m]))
kubectl get pods -l app=api -L version -o wide
kubectl logs -l app=api,version=v2 --tail=50
kubectl logs -n istio-system deploy/istio-ingressgateway | grep " 502 " | tail
kubectl get destinationrule api -o yaml | grep -A3 subsets
kubectl argo rollouts abort api -n prod
```

> ***What to say to the interviewer:*** *I abort or roll the canary weight back to zero first, then split the 502s by version; if they all come from canary pods I check their readiness, ports, logs and new config. I also verify the routing layer, such as DestinationRule subset labels, sidecar and mTLS consistency, protocol changes and keep-alive timeouts, and then retry the canary at a small weight with automated metric analysis.*

Q86. Users report 10-second delays every 15 minutes in an app running on AKS (or any Kubernetes). No code changes happened. How do you begin RCA?

> **In simple words:** Something happens like clockwork every 15 minutes. Periodic problems come from periodic things - find everything that runs on a 15-minute timer.

Detailed answer

1. **Confirm the pattern precisely:** exact timestamps, duration, which endpoints/users/regions; is it all pods or some? Plot p99 latency at 1-minute resolution.
2. **List everything periodic** and correlate with the timestamps:

- CronJobs in the cluster (backups, reports, cleanup), DB maintenance/vacuum, cache expiry (TTL 15 min -> cache stampede).
- Token/credential refresh (managed identity, OAuth tokens, Key Vault/secret rotation, CSI driver sync interval).
- DNS TTLs / cache expiry, connection pool recycle (`maxLifetime`), idle connection timeouts (Azure Load Balancer idle timeout default 4 min, NAT idle timeout) -> first request after reconnect is slow.
- JVM full GC cycles, log rotation, metrics scrapes of heavy endpoints, HPA/autoscaler scale events, Karpenter/cluster autoscaler consolidation.
- Node-level: image GC, kernel tasks, antivirus/security agents, **SNAT port exhaustion** on outbound connections.

3. **Trace a slow request** (Application Insights / OpenTelemetry): which span takes 10 seconds - DNS lookup, TCP connect, TLS, DB query, external API?
4. **10 seconds exactly** is suspicious: typical of a timeout + retry (e.g. 5s DNS timeout x2, connect timeout), so check DNS (conntrack race), and connect timeouts.
5. Check platform events: AKS node image auto-upgrade, Azure maintenance, LB health.

6. Test hypothesis: change one variable (disable CronJob, adjust pool settings, enable NodeLocal DNSCache) and watch the next cycle.

Example:

```
kubectl get cronjobs -A
kubectl get events -A --sort-by=.lastTimestamp | grep -iE "scaled|killing|pull" | tail
# PromQL: p99 latency at 1m resolution
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket{app="api"}[1m])))
kubectl exec <pod> -- sh -c 'time nslookup db.internal'
```

> ***What to say to the interviewer:*** *A 15-minute rhythm means a periodic cause, so I line up the exact latency spikes with everything periodic: CronJobs, cache TTLs, token or secret refresh, connection pool recycling, idle timeouts, GC and autoscaler events. I trace a slow request to see which span takes the 10 seconds, and an exact 10 seconds often points to timeouts and retries such as DNS or connection timeouts, then I test one hypothesis at a time.*

Q87. A kube-proxy update rolls out in the middle of a live event (for example a big match). What is your network rollback plan to avoid packet drops?

> **In simple words:** Someone started rewiring the traffic lights city-wide during the rush hour. Stop the rewiring, put back the old lights one junction at a time, and next time only rewire at night, one junction first.

Detailed answer

Immediately:

1. **Pause the change:** stop the pipeline/add-on update; if it is a DaemonSet rollout in progress, check `kubectl rollout status ds/kube-proxy -n kube-system`.
2. **Assess impact:** packet drops/5xx/latency per node - are only updated nodes affected? Compare updated vs old nodes.
3. **Roll back kube-proxy:**

- Self-managed DaemonSet: `kubectl rollout undo ds/kube-proxy -n kube-system` (updates one node at a time with `maxUnavailable: 1`).

- EKS managed add-on: `aws eks update-addon --addon-name kube-proxy --addon-version <previous> --resolve-conflicts PRESERVE` (or pin the previous version in Terraform).

4. **Protect traffic during rollback:** cordon/drain affected nodes in the worst case so traffic shifts to healthy nodes; external LBs route away from unhealthy nodes (health checks). Keep existing connections: kube-proxy restarts do not flush conntrack by default, so established flows usually survive; avoid changing proxy **mode** (iptables <-> IPVS) mid-event since that rewrites all rules.
5. **Verify:** iptables/IPVS rules sync (`kubeproxy_sync_proxy_rules_duration_seconds`), Service connectivity tests, error rate back to normal.

Prevention:

- **Change freeze** during major events; add-on version pinned in IaC (no auto-update).
- Canary node group first, `maxUnavailable: 1`, soak time, automatic halt on error metrics.
- Test version compatibility (kube-proxy must match or be close to control plane version) in staging under load.

- Run a pre-event game day and keep a written rollback runbook.

Example:

```
kubectl -n kube-system rollout status ds/kube-proxy
kubectl -n kube-system rollout history ds/kube-proxy
kubectl -n kube-system rollout undo ds/kube-proxy
kubectl -n kube-system get ds kube-proxy -o jsonpath='{.spec.updateStrategy}'
aws eks describe-addon --cluster-name prod-eks --addon-name kube-proxy --query addon.addonVersion
aws eks update-addon --cluster-name prod-eks --addon-name kube-proxy \
  --addon-version v1.30.0-eksbuild.3 --resolve-conflicts PRESERVE
```

What to say to the interviewer: *I halt the rollout, compare updated versus not-yet-updated nodes to confirm impact, and roll kube-proxy back with rollout undo or by pinning the previous EKS add-on version, one node at a time, draining nodes if drops are severe. Established connections generally survive because conntrack is not flushed, and prevention is a change freeze for big events, pinned add-on versions and a canary node group with automatic halt.*

Q88. You need to containerize a legacy monolith and deploy it to EKS with minimal downtime. What is your approach?

In simple words: Move a shop to a new mall without closing it. Open the new shop while the old one still runs, send a few customers there first, then more and more, and only close the old shop when the new one is proven.

Detailed answer

1. Assess the monolith:

- Where is **state** kept: local disk (uploads, generated files), in-memory **sessions**, local cron jobs, config files with hard-coded hosts/secrets?
- Dependencies (DB, message queues, external IPs whitelisted), OS packages, startup time, memory needs.

2. Make it container-friendly (12-factor basics):

- **Externalize config** to env vars/ConfigMaps and secrets to Secrets Manager (External Secrets).
- **Sessions** to Redis/ElastiCache (or sticky sessions temporarily); files to **S3/EFS**.
- **Logs to stdout/stderr** instead of local files.
- Add **health endpoints** (`/health`, `/ready`) and graceful SIGTERM handling.
- Local crons -> Kubernetes CronJobs.

3. Build and deploy:

- Dockerfile (pinned base image, multi-stage if compiled, non-root), scan, push to ECR.
- Helm chart: Deployment with probes and realistic resources (monoliths are big - start generous, tune later), Service, Ingress (ALB), HPA if it is stateless now, PDB.
- Test in staging with production-like data and a load test.

4. Run in parallel and shift traffic gradually (strangler fig):

- Legacy stays live; the EKS version runs alongside, pointing to the **same database** initially (no data migration on day one).

- Shift traffic with **ALB weighted target groups** or **Route 53 weighted records**: 5% -> 25% -> 50% -> 100%, watching error rate, latency and business metrics at each step. Rollback = weight back to legacy.
- Lower DNS TTLs before starting.

5. After cut-over: keep legacy on standby for a few days, then decommission; later peel off features into microservices (strangler fig pattern) and plan the DB migration separately.

Example:

```
# ALB listener rule with weighted forward (Terraform / console)
forward {
  target_group { arn = aws_lb_target_group.legacy_ec2.arn  weight = 90 }
  target_group { arn = aws_lb_target_group.eks_ingress.arn weight = 10 }
}

# Route 53 weighted alternative
aws route53 change-resource-record-sets --hosted-zone-id Z123 --change-batch file://weights.json
kubectl rollout status deploy/monolith -n app
```

What to say to the interviewer: *I first assess where the monolith keeps state, then externalize config, sessions and files, send logs to stdout, add health endpoints and build a hardened image with a Helm chart including probes, resources and a PDB. The EKS version runs in parallel with the legacy system on the same database, and I shift traffic gradually with ALB weighted target groups or Route 53 weighted records, rolling back by weight if metrics degrade, then decommission the legacy system and peel off services over time.*

Q89. Have you faced production issues in Kubernetes? Tell me about one and how you troubleshot it.

Also asked as: Tell me about a recent production issue you handled in Kubernetes. What was the last Kubernetes outage you debugged, and what was the RCA?

In simple words: Tell a short story with a clear structure: what broke, how you noticed, how you found the cause, how you fixed it, and what you changed so it does not happen again (STAR: Situation, Task, Action, Result).

Detailed answer

Structure your answer: **symptom -> impact -> investigation steps -> root cause -> fix -> prevention**. Use a real example from your work and keep it to 2 minutes.

RCA structure interviewers like:

1. **Detection:** which alert fired (or did users report it first - then alerting is an action item).
2. **Timeline:** start, detection, mitigation, resolution times (MTTD, MTTR).
3. **Evidence:** metrics (error rate, latency, CPU/memory, restarts), logs (`kubectl logs --previous`, Loki queries), events (`kubectl get events`), traces, and the change log (deploys, node upgrades).
4. **Root cause** (5 whys - technical cause and why it was not caught) and **contributing factors**.
5. **Fix + prevention:** action items with owners (PDBs, probes, alerts, tests, runbooks).
6. **Blameless postmortem** document shared with the team, and follow-up on action items.

Sample answers you can adapt (from real EKS monitoring work):

Example 1 - logs missing for one service: A Loki/Grafana dashboard showed no application logs for one service while others were fine. I checked that the pods were Running and producing logs with `kubectl logs`, so the app was fine. Then I compared the dashboard's container label filter with the actual container name in the pod spec (`kubectl get pod -o jsonpath='{.spec.containers[*].name}'`) and found the real container name differed from what the dashboard query expected. Fix: corrected the label/query. Prevention: dashboards use variables populated from real label values instead of hard-coded names.

Example 2 - SLA breach after a node group update: A service availability dip lined up with a managed node group update. Pods were rescheduled as nodes were replaced, and one service did not have enough replicas/PDB protection, so it had a short outage. Fix: PDB with `maxUnavailable: 1`, at least 2 replicas spread across nodes, readiness probes; schedule node upgrades in low-traffic windows.

Example 3 - alerts silently not delivered: An alert pipeline looked healthy but notifications never reached Teams because a sidecar forwarder had silently died. Fix: moved to Alertmanager's native Teams receiver and added an end-to-end test alert (a "dead man's switch") so a broken alert path is itself detected.

Example:

```
kubectl get pods -n <ns> -o wide
kubectl logs <pod> -n <ns> --tail=50
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].name}'
kubectl get events -n <ns> --sort-by=.lastTimestamp | tail
kubectl get pdb -n <ns>
```

> ***What to say to the interviewer:*** *Pick one real incident and tell it as symptom, impact, how it was detected, the specific commands and evidence from metrics, logs and events you used, the root cause, the fix and the prevention you added, and mention the blameless postmortem with owned action items. Interviewers care most that you debugged systematically, communicated, and made a lasting improvement like a PDB, better probes or an end-to-end alert test.*

CI/CD, Jenkins, GitOps & Ansible

Basic

Q1. What is the difference between CI, CD and GitOps?

Also asked as: What is the difference between CI and CD? CI vs CD vs GitOps: explain all 3 with real-world examples.

In simple words: CI is like a kitchen that tastes every new ingredient the moment it arrives. CD is the waiter who is always ready to carry the finished dish to the table - either after the manager says "go" (Delivery) or automatically (Deployment). GitOps is a written menu on the wall: a robot keeps checking that what is on every table matches the menu, and to change a dish you change the menu.

Detailed answer

- **Continuous Integration (CI):** every code push is automatically built and tested. Goal: catch bugs within minutes, keep the main branch always working.
- **Continuous Delivery:** every change that passes CI is packaged and deployable to production at any time, but the final push to prod needs a human approval.
- **Continuous Deployment:** same as Delivery, but with no human gate - if all tests pass, it goes to production automatically.
- **GitOps:** a way of doing CD where **Git is the single source of truth** for what should run. It is **pull-based**: an agent inside the cluster (Argo CD / Flux) continuously compares the cluster with Git and **reconciles** differences. Deploy = merge a commit; rollback = revert a commit; manual drift is undone automatically.

Approach	What happens	Who decides prod release
CI	build, unit test, lint, scan on every push	-
Continuous Delivery	package, deploy to staging, prod-ready	Human approval
Continuous Deployment	auto deploy to prod	Pipeline (tests)
GitOps	agent pulls desired state from Git and syncs	Whoever merges the Git PR

Real-world examples (smart-metering style project):

- **CI:** a developer changes the meter-data ingestion service; the PR triggers build, unit tests for the parser of meter readings, SonarQube and Trivy - the PR cannot merge until it is green.
- **Continuous Delivery:** the merged billing (MDMS) service image is deployed to staging automatically; the production release waits for the release manager's approval because it affects customer bills.
- **Continuous Deployment:** an internal Grafana dashboard or alert-rule change goes straight to production once validation passes - low risk, no approval.
- **GitOps:** the head-end (HES) services' Helm values live in a config repo; bumping the image tag in `envs/prod/values.yaml` through a PR makes Argo CD roll it out, and a `git revert` rolls it back.

Example:

```
git push -> Jenkins: build + unit tests (CI)
         -> Docker image pushed to ECR, deployed to staging (CD)
         -> "Approve prod?" button (Continuous Delivery)
         -> or: PR bumps tag in gitops repo -> Argo CD syncs prod (GitOps)
```

> ***What to say to the interviewer:*** *CI means every commit is automatically built and tested so integration problems are caught early. Continuous Delivery keeps a deployable artifact ready with a manual approval before prod, while Continuous Deployment pushes to prod automatically when all checks pass. GitOps is a pull-based way to do CD where Git holds the desired state and an agent like Argo CD reconciles the cluster to it, so deploys and rollbacks are just Git commits.*

Q2. What triggers your pipeline, and how do you trigger a Jenkins job automatically?

Also asked as: How do you trigger a job automatically in Jenkins?

> **In simple words:** A pipeline is like a doorbell-activated machine. Something has to press the bell - a code push, a pull request, a timer, another machine, or a person clicking a button.

Detailed answer

Common triggers:

- **Webhook on push / merge** to a branch (most common) - GitHub/GitLab/Bitbucket calls Jenkins.
- **Pull request opened/updated** - runs CI checks before merge.
- **Tag creation** (e.g. `v1.4.0`) - triggers a release build.
- **Build periodically (cron)** - nightly builds, security scans, cleanup jobs. `H/5 * * * *` = every 5 minutes; `H` spreads jobs so they do not all start at the same second.
- **Poll SCM** - Jenkins checks the repo on a schedule and builds only if there are new commits (fallback when GitHub cannot reach Jenkins).
- **Upstream/downstream** - job B runs after job A succeeds (`upstream` trigger, or `build job:` step from A).
- **Remote trigger** - "Trigger builds remotely" with an authentication token, or the REST API with a user API token, e.g. called from another system.
- **Manual** - "Build with Parameters" for hotfixes or prod deploys.

Note: cron builds even if nothing changed; Poll SCM builds only on new commits.

Example:

```
pipeline {
  agent any
  triggers {
    githubPush()                              // webhook from GitHub
    cron('H 2 * * *')                         // nightly at ~2 AM
    pollSCM('H/5 * * * *')                    // check for commits every ~5 min
    upstream(upstreamProjects: 'build-lib', threshold: hudson.model.Result.SUCCESS)
  }
  stages { stage('Build') { steps { sh 'make build' } } }
}

# Remote trigger (job config: "Trigger builds remotely", token = deploy123)
curl -X POST -u priyanka:$API_TOKEN \
  "https://jenkins.example.com/job/orders/build?token=deploy123"
```

> ***What to say to the interviewer:*** *Our pipelines are mainly triggered by GitHub webhooks on push and pull requests - PRs run CI checks and merges to main trigger build and deploy to dev. Release tags trigger production builds, and we also have cron-triggered nightly scans. Prod deploys additionally need a manual approval.*

Q3. What are webhooks?

> **In simple words:** Polling is you calling the pizza shop every 5 minutes asking "is it ready?". A webhook is the shop calling you when it is ready.

Detailed answer

A webhook is an HTTP POST request that one system sends to another when an event happens. In CI/CD, GitHub sends a webhook to Jenkins when someone pushes code or opens a PR. The payload (JSON) contains the repo, branch, commit ID, and author.

How to set it up:

1. In Jenkins install the GitHub plugin; in the job enable "GitHub hook trigger for GITScm polling".
2. In GitHub repo -> Settings -> Webhooks -> add `https://jenkins.company.com/github-webhook/`, content type JSON, choose events (push, pull request).
3. Add a secret token so Jenkins can verify the request really came from GitHub.

Benefits vs polling: instant builds, less load on Git server and Jenkins.

Example:

```
Payload URL : https://jenkins.example.com/github-webhook/
Content type: application/json
Secret      : <shared secret>
Events      : Push, Pull requests

# Test from GitHub: Settings -> Webhooks -> Recent Deliveries -> Redeliver
```

> ***What to say to the interviewer:*** *A webhook is an event-driven HTTP callback - when code is pushed, GitHub immediately POSTs the event to Jenkins, which starts the build. It is faster and lighter than polling. We secure it with a shared secret and make sure the Jenkins URL is reachable from GitHub.*

Q4. What is a Jenkins agent (worker node)? Where does a pipeline run and where do you define agents?

Also asked as: Where does the Jenkins pipeline run? What is a worker node? Where do you define Jenkins worker nodes?

> **In simple words:** The Jenkins controller is the manager who hands out work. Agents are the workers who actually do the building. The manager should not be doing the heavy lifting.

Detailed answer

- **Controller (master):** stores job config, schedules builds, serves the UI. Ideally runs zero builds itself.
- **Agent (worker node):** a machine, VM, container or Kubernetes pod that connects to the controller and executes the pipeline steps. Each agent has **executors** (parallel build slots) and **labels** (e.g. `linux`, `docker`, `maven`).
- **Where defined:** Manage Jenkins -> Nodes (static agents via SSH or inbound JNLP agent), or Manage Jenkins -> Clouds (dynamic agents from Kubernetes plugin, EC2 plugin, Docker plugin).
- **In the Jenkinsfile** you choose which agent with the `agent` directive.

Example:

```
pipeline {
  agent { label 'docker-linux' }          // run on any agent with this label
  stages {
    stage('Build') { steps { sh 'mvn -B package' } }
  }
}

// Dynamic pod agent on Kubernetes (Kubernetes plugin)
pipeline {
  agent {
    kubernetes {
      yaml '''
        apiVersion: v1
        kind: Pod
        spec:
          containers:
          - name: maven
            image: maven:3.9-eclipse-temurin-17
            command: ["sleep"]
            args: ["infinity"]
      '''
    }
  }
  stages { stage('Build') { steps { container('maven') { sh 'mvn -B package' } } } }
}
```

> ***What to say to the interviewer:*** *The pipeline runs on Jenkins agents, not on the controller. Agents are defined under Manage Jenkins -> Nodes for static machines, or under Clouds for dynamic ones like Kubernetes pods or EC2 instances. In the Jenkinsfile the agent directive with a label or a pod template picks where each stage runs.*

Q5. What is the file structure of a Jenkinsfile, and how do you use it in your project?

Also asked as: How do you write pipeline scripts in Jenkins?

In simple words: A Jenkinsfile is a recipe card kept in the same box as the ingredients (the code). Anyone can read it, change it, and review changes to it.

Detailed answer

A Jenkinsfile is pipeline-as-code, stored in the root of the repository. Two styles: **Declarative** (structured, recommended) and **Scripted** (Groovy, flexible). Declarative building blocks:

- `pipeline` - the outer block.
- `agent` - where to run.
- `environment` - variables and credentials.
- `options` - timeouts, build discarder, timestamps.
- `parameters` - inputs for manual runs.
- `triggers` - cron, webhooks.
- `stages` -> `stage` -> `steps` - the actual work.
- `when` - conditional stages (e.g. only on main).
- `post` - always / success / failure actions (notify, cleanup).

Example:

```
pipeline {
  agent { label 'linux' }
  options { timeout(time: 30, unit: 'MINUTES'); timestamps() }
  environment { APP = 'orders-api' }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build')    { steps { sh 'mvn -B -DskipTests package' } }
    stage('Test')     { steps { sh 'mvn test' } }
    stage('Deploy')   {
      when { branch 'main' }
      steps { sh './deploy.sh dev' }
    }
  }
  post {
    failure { echo 'Notify Teams/Slack here' }
    always  { cleanWs() }
  }
}
```

What to say to the interviewer: *We keep a declarative Jenkinsfile in the root of each repo, so the pipeline is versioned and reviewed like code. It has agent, environment, options, stages for checkout, build, test, scan, image push and deploy, when-conditions for branch-specific stages, and a post block for notifications and cleanup.*

Q6. What is a multibranch pipeline?

In simple words: Instead of creating a new pipeline by hand for every branch, Jenkins walks through the repo, finds every branch that has a Jenkinsfile, and creates a pipeline for each automatically.

Detailed answer

- Jenkins scans the repo (on webhook or schedule) and creates one job per branch / PR that contains a Jenkinsfile.
- When a branch is deleted, its job is removed (orphan cleanup).
- Each branch uses its own version of the Jenkinsfile - great for feature branches.
- Inside the Jenkinsfile you use `when { branch 'main' }` or `env.BRANCH_NAME` so feature branches only build/test while main also deploys.

Example:

```
stage('Deploy to Dev') {
  when { branch 'develop' }
  steps { sh './deploy.sh dev' }
}
stage('Deploy to Prod') {
  when { buildingTag() }        // only for release tags like v1.2.0
  steps { sh './deploy.sh prod' }
}
// PR builds: env.CHANGE_ID is set, so we only run build + test
```

> ***What to say to the interviewer:*** *A multibranch pipeline automatically discovers every branch and pull request with a Jenkinsfile and creates a pipeline for each. Feature branches and PRs only run build and tests, while develop deploys to dev and tags deploy to prod, all controlled with when-conditions in one Jenkinsfile.*

Q7. How do you define and invoke pipelines in Jenkins?

> **In simple words:** Defining a pipeline is writing the recipe (Jenkinsfile). Invoking it is deciding what rings the kitchen bell - a code push, a timer, a button, or another recipe finishing.

Detailed answer

Defining:

- **Pipeline script from SCM (recommended):** create a Pipeline job, choose "Pipeline script from SCM", point to the repo and the path `Jenkinsfile`. The pipeline is versioned with the code.
- **Inline script:** pasted into the job UI - only for quick tests, not reviewable.
- **Multibranch Pipeline / Organization Folder:** Jenkins auto-creates jobs for every branch/PR/repo that has a Jenkinsfile.
- **Job DSL / Configuration as Code (JCasC):** create the jobs themselves from code, so even job setup is not click-ops.

Invoking:

- **Triggers:** webhook (`githubPush()`), `cron`, `pollSCM`, `upstream`.
- **Manual:** "Build Now" or "Build with Parameters" (`parameters {}` block).
- **From another pipeline:** `build job: 'deploy-app', parameters: [...]`.
- **Remotely:** REST API with a user API token, or the Jenkins CLI.

Example:

```
pipeline {
  agent any
  parameters {
    choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Target')
    string(name: 'TAG', defaultValue: '', description: 'Image tag to deploy')
  }
  triggers { githubPush() }
  stages {
    stage('Deploy') {
      steps {
        build job: 'deploy-orders', wait: true,
              parameters: [string(name: 'ENV', value: params.ENV),
                           string(name: 'TAG', value: params.TAG)]
      }
    }
  }
}

# Remote trigger via REST API
curl -X POST -u priyanka:$API_TOKEN \
  "https://jenkins.example.com/job/deploy-orders/buildWithParameters?ENV=dev&TAG=1.4.0"
```

> ***What to say to the interviewer:*** *I define pipelines as a Jenkinsfile in the repo and create a Pipeline or Multibranch job that reads it from SCM, with the jobs themselves created through Job DSL or JCasC. They are invoked by GitHub webhooks, cron triggers, manual Build with Parameters, the build job step from another pipeline, or the REST API.*

Q8. What are the types of jobs in Jenkins?

> **In simple words:** Jenkins job types are like different kinds of vehicles - a scooter (Freestyle) for quick simple trips, a truck with a written route (Pipeline), and a whole fleet that auto-assigns a truck to every branch (Multibranch).

Detailed answer

Job type	What it is	When to use
Freestyle	UI-configured steps (shell, build, post-build actions)	Simple one-off tasks; legacy jobs
Pipeline	Stages defined in a Jenkinsfile (inline or from SCM)	Standard CI/CD as code
Multibranch Pipeline	Auto-creates a Pipeline job per branch/PR with a Jenkinsfile	Feature branch + PR builds
Organization Folder	Scans a whole GitHub org/Bitbucket team, creates multibranch jobs per repo	Many repos, zero manual job creation
Folder	Container to group jobs, with folder-scoped credentials and permissions	Separate teams/environments
Multi-configuration (Matrix)	Runs the same build across combinations (OS x JDK)	Compatibility testing (Pipeline `matrix` is the modern way)

Freestyle vs Pipeline: Freestyle config lives in the Jenkins UI (not versioned, hard to review, weak at complex flows); Pipeline is code in Git, supports parallel stages, approvals, resuming and shared libraries.

Example:

```
// Modern matrix instead of a Multi-configuration job
pipeline {
  agent none
  stages {
    stage('Test') {
      matrix {
        axes {
          axis { name 'JDK'; values '11', '17' }
          axis { name 'OS';  values 'linux', 'windows' }
        }
        agent { label "${OS}" }
        stages { stage('Run') { steps { echo "Testing JDK ${JDK} on ${OS}" } } }
      }
    }
  }
}
```

> ***What to say to the interviewer:*** *The main Jenkins job types are Freestyle, Pipeline, Multibranch Pipeline, Organization Folder, Folder, and Multi-configuration or matrix jobs. We mostly use Multibranch Pipelines and Organization Folders so every repo and branch with a Jenkinsfile gets its job automatically, and Folders to scope credentials and permissions per team; Freestyle is only for legacy or trivial tasks.*

Q9. How do you configure GitHub in Jenkins for Freestyle and Pipeline jobs?

> **In simple words:** Jenkins needs two things from GitHub: a key to read the code (credentials) and a doorbell so GitHub can tell Jenkins "new code arrived" (webhook).

Detailed answer

Common setup (once):

1. Install **Git**, **GitHub** and **GitHub Branch Source** plugins.
2. Add credentials: a **GitHub App** (preferred - fine-grained, higher API limits, no personal account) or a **Personal Access Token** as Username/Password, or an SSH key for git clone.
3. Manage Jenkins -> System -> GitHub -> add GitHub Server with the credential, tick "Manage hooks" (optional, lets Jenkins create webhooks).
4. In the GitHub repo: Settings -> Webhooks -> `https://<jenkins>/github-webhook/`, JSON, push + PR events.

Freestyle job:

- Source Code Management -> Git -> repo URL + credentials + branch (`*/main`).
- Build Triggers -> tick **GitHub hook trigger for GITScm polling**.
- Build steps: Execute shell; post-build actions for reports/notifications.

Pipeline job:

- Definition -> **Pipeline script from SCM** -> Git -> repo URL + credentials -> Script Path `Jenkinsfile`.
- Tick "GitHub hook trigger for GITScm polling" (or `triggers { githubPush() }` in the Jenkinsfile).

- Multibranch: Branch Sources -> GitHub -> credential + repo; webhooks trigger re-scans automatically.

Example:

```
// Jenkinsfile checkout using the stored credential
checkout([$class: 'GitSCM',
  branches: [[name: '*/main']],
  userRemoteConfigs: [[url: 'https://github.com/org/orders-api.git',
                       credentialsId: 'github-app-creds']]])

# Verify: GitHub -> Settings -> Webhooks -> Recent Deliveries shows HTTP 200
```

What to say to the interviewer: *I install the Git, GitHub and Branch Source plugins, add a GitHub App or PAT credential, and configure a webhook to jenkins-url/github-webhook. For Freestyle I set the Git repo in Source Code Management and tick GitHub hook trigger for GITScm polling; for Pipeline I choose Pipeline script from SCM with the Jenkinsfile path and the same trigger, or use a Multibranch job with a GitHub branch source.*

Q10. How do you know the whole Jenkins pipeline actually executed successfully?

In simple words: Like tracking a courier parcel - you look at every checkpoint scan (stages), the final "delivered" status, the photo proof (reports and artifacts), and you get an SMS at the end (notification).

Detailed answer

Evidence to check:

- **Build status/result:** SUCCESS, UNSTABLE (tests failed but build continued), FAILURE, ABORTED - a green ball is not enough if stages were skipped.
- **Stage View / Blue Ocean / Pipeline Graph View:** each stage shows passed, failed, or **skipped** (grey) - a skipped deploy stage means it did not run.
- **Console Output:** full log with timestamps (`timestamps{ }` option); Pipeline Steps view shows each step and its duration.
- **Test reports:** `junit` step shows test count and failures; coverage reports.
- **Artifacts / fingerprints:** `archiveArtifacts`, image tag in ECR, Helm release revision.
- **post blocks:** `always` / `success` / `failure` / `unstable` send notifications and publish results.
- **Notifications:** Teams/Slack/email with job, build number, commit and deployed version.
- **Deployment verification:** the pipeline itself runs `kubectl rollout status` or a smoke test, so "success" means the app is really healthy, not just "command ran".
- **Audit:** build history, Audit Trail plugin, who approved the input step.

Example:

```
post {
  success {
    office365ConnectorSend webhookUrl: env.TEAMS_HOOK,
      message: "SUCCESS ${env.JOB_NAME} #${env.BUILD_NUMBER} deployed ${env.TAG}"
  }
  unstable { echo 'Tests failed - check JUnit report' }
  failure  { echo "Failed at stage: ${env.STAGE_NAME}" }
  always   { junit 'reports/*.xml'; archiveArtifacts 'reports/**' }
}
stage('Verify') { steps { sh 'kubectl -n prod rollout status deploy/orders-api --timeout=5m' } }
```

What to say to the interviewer: *I check the build result and the Stage View or Blue Ocean to confirm every stage ran and none was skipped, the console output and step timings, published JUnit and coverage reports, and archived artifacts or the pushed image tag. Post blocks send Teams or Slack notifications for success and failure, and the pipeline ends with a rollout status or smoke test so success really means the app is healthy.*

Q11. What types of artifacts are generated in your pipeline?

In simple words: An artifact is the finished product that comes off the assembly line - the thing you actually ship, not the raw code.

Detailed answer

Typical artifacts:

- **Docker images** pushed to ECR / Docker Hub / Artifactory, tagged with version + git SHA.
- **Helm charts** (packaged `.tgz`) pushed to a chart repo or OCI registry.
- **Binaries/packages:** `.jar`/`.war` (Java), `.zip` (Lambda), `.whl`, npm packages - stored in Nexus/Artifactory/S3.
- **Reports:** unit test reports (JUnit XML), code coverage, SonarQube results, Trivy scan reports, SBOM.
- **IaC outputs:** saved `terraform plan` files.

Key rule: **build once, deploy the same artifact everywhere** - dev, staging and prod all use the exact same image tag.

Example:

```
IMAGE=123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-api
TAG=1.4.0-${GIT_COMMIT:0:7}          # e.g. 1.4.0-a1b2c3d
docker build -t $IMAGE:$TAG .
docker push $IMAGE:$TAG
helm package charts/orders-api --version 1.4.0
archiveArtifacts artifacts: 'target/*.jar, reports/**', fingerprint: true
```

What to say to the interviewer: *Our main artifacts are Docker images pushed to ECR, tagged with the release version and short git commit SHA, plus Helm charts, test and coverage reports, and security scan reports. We build the artifact once and promote the same immutable tag from dev to prod.*

Q12. What is Blue-Green deployment vs Canary deployment? Explain with examples.

Also asked as: What is Blue-Green deployment? What is Canary deployment? Explain Blue-Green vs Canary with examples. Blue-green: how do you shift traffic safely at 10%, 25%?

In simple words: Blue-Green is having two identical stages in a theatre - the show runs on Blue while you set up Green, then you swing the spotlight to Green in one go. Canary is letting a few people taste a new dish first; if they are fine, you serve it to everyone.

Detailed answer

Blue-Green:

- Two full environments: Blue (current live) and Green (new version).
- Deploy and test Green, then switch all traffic (load balancer / DNS / K8s Service selector) at once.
- Rollback = switch back to Blue instantly.
- Cost: double infrastructure for the switch window.

Canary:

- Release new version to a small slice of traffic (1% -> 10% -> 50% -> 100%).
- Watch error rate, latency, CPU at each step; auto-rollback if metrics get worse.
- Cheaper, lower blast radius, but slower and needs good monitoring.

How to shift traffic safely in steps (10% -> 25% -> 50% -> 100%):

- **ALB weighted target groups:** one listener rule forwards to blue and green target groups with weights (90/10, then 75/25...). Instant, no DNS caching.
- **Route 53 weighted records:** two records with weights; simple, but clients and resolvers cache DNS for the TTL, so shifts are slower and less precise.
- **Argo Rollouts (Kubernetes):** `setWeight` steps with `pause` and an **analysis** step between them that checks Prometheus metrics.
- **CodeDeploy (ECS/Lambda):** built-in configs like `CodeDeployDefault.ECSCanary10Percent5Minutes` (10% for 5 minutes, then 100%) or `ECSLinear10PercentEvery1Minutes`.
- **Automatic rollback:** tie each step to alarms (5xx rate, p99 latency). CodeDeploy rolls back when a CloudWatch alarm fires; Argo Rollouts aborts when analysis fails. Hold at each step long enough to see real traffic.

	Blue-Green	Canary
Traffic shift	All at once	Gradual %
Rollback	Instant switch back	Shift weight to 0%
Infra cost	2x during release	Small extra
Needs	Good pre-switch tests	Strong metrics / monitoring
Best for	DB-safe big releases, fast rollback	High-traffic user-facing apps

Example:

```
# Blue-Green on Kubernetes: flip the Service selector
kubectl patch service orders -p '{"spec":{"selector":{"app":"orders","version":"green"}}}'

# Canary with AWS ALB weighted target groups (90/10)
aws elbv2 modify-listener --listener-arn $LISTENER --default-actions \
 '[{"Type":"forward","ForwardConfig":{"TargetGroups":[
   {"TargetGroupArn":"'$TG_V1'","Weight":90},{"TargetGroupArn":"'$TG_V2'","Weight":10}]}}]'

# Argo Rollouts: 10% -> 25% -> 50% -> 100% with pauses and metric analysis
strategy:
  canary:
    steps:
    - setWeight: 10
    - pause: { duration: 5m }
    - analysis: { templates: [ { templateName: error-rate } ] }
    - setWeight: 25
    - pause: { duration: 10m }
    - setWeight: 50
    - pause: { duration: 10m }
```

What to say to the interviewer: *Blue-Green runs two identical environments and switches all traffic to the new one after testing, so rollback is an instant switch back but it costs double infrastructure briefly. Canary sends a small percentage of traffic to the new version and increases it while watching error rate and latency, which limits blast radius. I use Blue-Green when I need instant rollback and Canary for high-traffic services with good monitoring.*

Q13. How do you handle build failures?

In simple words: When the oven beeps "failed", you do not just press start again - you read the error, see which step burned, fix the cause, and make sure the cook gets told right away.

Detailed answer

1. **Fail fast and notify:** pipeline stops at the failing stage; `post { failure {...} }` sends a Teams/Slack/email message with the build link and commit author.
2. **Read the console log** of the failed stage - compilation error, failed test, dependency download, docker push, timeout.
3. **Classify:** code problem (dev fixes), test flakiness (quarantine + ticket), infra problem (agent disk full, network, credentials expired - DevOps fixes).
4. **Reproduce locally** with the same Docker build image so "works on my machine" is ruled out.
5. **Protect main:** branch protection requires a green build before merge, so failures stay on feature branches.
6. **Keep evidence:** archive test reports and logs; use `retry` only for known transient network steps, never to hide real failures.

Example:

```
post {
  failure {
    office365ConnectorSend webhookUrl: env.TEAMS_HOOK,
      message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} (${env.GIT_COMMIT})"
  }
  always { junit 'target/surefire-reports/*.xml' }
}
stage('Download deps') {
  steps { retry(3) { sh 'mvn -B dependency:go-offline' } }   // transient network only
}
```

> ***What to say to the interviewer:*** *The pipeline fails fast and notifies the team with the build link. I check the failing stage log, classify it as code, test or infrastructure, and route it to the right owner. Branch protection keeps main green, and I only use retries for known transient network steps, never to hide real failures.*

Q14. What branching strategy do you follow, and how do you handle a production bug?

> **In simple words:** Branches are separate drafts of a document. Trunk-based means everyone edits one main draft in small pieces. GitFlow means several long-lived drafts (develop, release, main). A hotfix is an emergency patch made on a copy of exactly what customers have right now.

Detailed answer

	GitFlow	Trunk-based
Long-lived branches	main, develop, release/*, hotfix/*	only main
Feature branches	long-ish	short (hours to 1-2 days)
Release style	scheduled releases	continuous, feature flags hide unfinished work
Good for	versioned products, slower release cycles	microservices, frequent deploys

Rules we apply either way:

- **Protected main:** no direct push, PR with at least 1-2 reviews, required green CI checks (build, tests, scans).
- **PR checks:** lint, unit tests, coverage gate, SonarQube, secret scan.
- **Releases tagged:** `v1.4.0` tag = exactly what is in prod.

Handling a production bug (hotfix):

1. Create `hotfix/1.4.1` from the **prod tag** (`v1.4.0`), not from develop, so no unreleased code sneaks in.
2. Fix + test; PR with fast-track review; full pipeline still runs.
3. Tag `v1.4.1`, deploy to prod (via staging smoke test).
4. **Merge the fix back** to main and develop (or cherry-pick) so the bug does not return in the next release.
5. RCA and add a test that would have caught it.

Example:

```
git checkout -b hotfix/1.4.1 v1.4.0
# fix the bug, commit
git push origin hotfix/1.4.1          # PR -> review -> CI green
git tag v1.4.1 && git push origin v1.4.1   # tag triggers prod release pipeline
git checkout develop && git merge --no-ff hotfix/1.4.1
```

> ***What to say to the interviewer:*** *For microservices I prefer trunk-based development with short-lived feature branches, protected main, mandatory PR reviews and required CI checks, while GitFlow suits scheduled versioned releases. For a production bug I branch a hotfix from the production tag, fix and test it through the pipeline, tag and release it, then merge it back to main and develop so it is not lost, followed by an RCA and a regression test.*

Q15. What is the difference between git merge and git rebase? When do you use each?

Also asked as: Describe the Git merge process. How do you resolve merge conflicts?

> **In simple words:** Merge is like tying two ropes together with a visible knot - you can always see where they joined. Rebase is like cutting your rope and re-attaching it at the end of the other rope so it looks like one straight line.

Detailed answer

- **git merge:** combines two branches and creates a **merge commit**. History is preserved exactly as it happened (non-linear). Safe for shared branches.
- **git rebase:** replays your commits on top of the latest target branch, creating **new commits (new SHAs)**. History becomes linear and clean.
- **Squash merge:** all commits of a feature branch become **one commit** on main - clean history, one commit per PR, easy to revert.

History shape:

```
Before:           A---B---C  main
                       \
                        D---E  feature

After merge:      A---B---C-------M  main      (M = merge commit)
                       \         /
                        D-------E

After rebase:     A---B---C---D'---E'  feature  (D', E' are new commits)

After squash:     A---B---C---S  main          (S = D+E in one commit)
```

When to use:

- **Rebase** your own local/feature branch onto main before opening or updating a PR, to get a clean linear history and resolve conflicts yourself.
- **Merge** (or squash merge) to bring a finished PR into main.
- **Golden rule: never rebase a shared/public branch** (main, develop, a branch others pulled) - it rewrites history and breaks everyone else's copy. If you must update a rebased branch on the remote, use `git push --force-with-lease`, never plain `--force`.

Example:

```
# Update my feature branch with latest main (clean linear history)
git checkout feature/login
git fetch origin
git rebase origin/main          # fix conflicts -> git add . -> git rebase --continue
git push --force-with-lease     # only because it is MY branch

# Merge a finished branch into main
git checkout main
git merge --no-ff feature/login

# Squash merge
git merge --squash feature/login && git commit -m "Add login feature"
```

Resolving merge conflicts step by step:

1. `git merge feature/x` (or a rebase) stops with `CONFLICT (content): Merge conflict in app.yaml`.
2. `git status` lists the files marked "both modified".
3. Open each file and find the markers: `<<<<<<< HEAD` (your side), `=======`, `>>>>>>> feature/x` (their side). Edit to the correct final content and delete the markers.
4. `git add <file>` to mark it resolved, then `git commit` (or `git rebase --continue` during a rebase).
5. Stuck? `git merge --abort` or `git rebase --abort` returns to the state before you started.
6. Helpers: `git mergetool`, VS Code's merge editor, `git diff --name-only --diff-filter=U` to list unresolved files. Run the tests before pushing.

Prevention: small, short-lived branches, pull or rebase from main often, and CODEOWNERS on hot files.

```
git merge feature/retry
# CONFLICT (content): Merge conflict in config/app.yaml
git status
#   both modified:   config/app.yaml
# edit the file, remove <<<<<<< ======= >>>>>>> markers
git add config/app.yaml
git commit -m "Merge feature/retry, resolve config conflict"
```

> ***What to say to the interviewer:*** *Merge joins branches with a merge commit and keeps the true history, while rebase replays my commits on top of the target branch to give a clean linear history but rewrites commit IDs. I rebase my own feature branch onto main before a PR, and merge or squash-merge the PR into main. I never rebase shared branches, and when I update my own rebased branch I use force-with-lease.*

Q16. What is git stash used for?

> **In simple words:** Stash is a drawer where you quickly put your half-finished work so you have a clean desk for an urgent job. When you are done, you take your work back out of the drawer.

Detailed answer

- `git stash` saves uncommitted changes (staged and unstaged) and cleans the working directory, without making a commit.

- `git stash push -m "message"` - stash with a name so you remember what it is.
- `git stash list` - see all stashes (`stash@{0}` is the newest).
- `git stash apply` - restore changes but **keep** the stash (useful to apply to several branches).
- `git stash pop` - restore changes and **delete** the stash.
- `git stash -u` (`--include-untracked`) - also stash new untracked files; by default they are left behind.
- `git stash show -p stash@{1}` - see what is inside; `git stash drop` / `clear` to remove.

Real use case: I am halfway through a feature, and a production bug needs a hotfix now. I stash my work, switch to a hotfix branch, fix and push, then come back and pop my stash.

Example:

```
git stash push -u -m "WIP: login page"      # save work incl. new files
git checkout -b hotfix/1.4.1 v1.4.0          # clean tree -> do the urgent fix
# ... fix, commit, push ...
git checkout feature/login
git stash list                               # stash@{0}: On feature/login: WIP: login page
git stash pop                                # changes back, stash removed
```

> ***What to say to the interviewer:*** *Git stash temporarily saves uncommitted changes and gives me a clean working tree without committing. I use stash push with a message and -u to include untracked files, list to see stashes, apply to restore while keeping the stash, and pop to restore and remove it. A typical case is stashing half-done feature work to switch branches for an urgent hotfix, then popping it back afterwards.*

Q17. How do you store credentials securely in Jenkins?

Also asked as: How do you handle secrets in Jenkins pipelines?

> **In simple words:** Never write the house key on the door. Keep it in a locked key box (Jenkins Credentials store or a vault) and hand it to the worker only while they are working, then take it back.

Detailed answer

- Use the **Jenkins Credentials** store (Manage Jenkins -> Credentials): types like Username/Password, Secret text, SSH key, Secret file, AWS credentials. They are encrypted on disk.
- Scope them to a **folder** so only relevant jobs can use them.
- Access them in pipelines with `withCredentials` or `credentials()` - Jenkins masks the values in logs.
- Better: don't store long-lived cloud keys at all - use **IAM roles** on agents (EC2 instance profile / EKS IRSA) or fetch secrets at runtime from **HashiCorp Vault / AWS Secrets Manager** plugins.
- Restrict who can view/configure credentials with Role-Based Access (Role Strategy / Matrix Authorization).
- Never `echo` secrets, never put them in the Jenkinsfile or Git.

Example:

```
pipeline {
  agent any
  environment { SONAR_TOKEN = credentials('sonar-token') }   // masked in logs
  stages {
    stage('Login') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'nexus-creds',
                         usernameVariable: 'U', passwordVariable: 'P')]) {
          sh 'echo "$P" | docker login nexus.example.com -u "$U" --password-stdin'
        }
      }
    }
  }
}
```

> ***What to say to the interviewer:*** *I store secrets in the Jenkins Credentials store, scoped per folder, and use them via withCredentials so they are masked in logs. For AWS I avoid static keys entirely and use IAM roles on the agents, and for application secrets we pull from Vault or Secrets Manager at runtime.*

Q18. Azure DevOps: How do you enforce branch policies and PR validation in Azure Repos?

> **In simple words:** Branch policies are the rules at the door of the main branch: nobody walks in alone, someone must check your work, the robot must test it, and only a few trusted people have a key to skip the line.

Detailed answer

Set on the branch (Repos -> Branches -> main -> Branch policies), or across many repos at project level:

- **Require a minimum number of reviewers** (e.g. 2); optionally "reset votes when new changes are pushed" and "prohibit the most recent pusher from approving their own changes".
- **Check for linked work items:** every PR must link a Boards item (traceability).
- **Check for comment resolution:** all review comments must be resolved before merge.
- **Build validation:** a pipeline runs on every PR (build + tests + scans); the PR cannot complete unless it passes. Can be required or optional, with path filters, and expires after main changes.
- **Status checks:** external services (SonarQube, security scanners) post a status that must succeed.
- **Automatically included reviewers by path:** e.g. `/infra/*` requires the platform team, `/db/migrations/*` requires a DBA.
- **Limit merge types:** allow only squash merge (or rebase) to keep history clean.
- **Limit who can bypass:** the "Bypass policies when completing pull requests" and "Bypass policies when pushing" permissions should be given only to a small admin group, and their use audited. Direct pushes to main are blocked once policies are on.

Example:

```
# Build validation pipeline triggered by the PR policy (azure-pipelines-pr.yml)
trigger: none
pr: none                      # Azure Repos PRs are triggered by the branch policy
pool: { vmImage: ubuntu-latest }
steps:
- script: npm ci && npm run lint && npm test
- task: SonarQubePrepare@6
  inputs: { SonarQube: 'sonar-sc', scannerMode: 'CLI' }

# Add a required reviewer policy with the CLI
az repos policy required-reviewer create --repository-id $REPO_ID --branch main \
  --blocking true --enabled true --path-filter "/infra/*" \
  --required-reviewer-ids platform-team@company.com --message "Infra change"
```

What to say to the interviewer: *I protect main with branch policies: minimum two reviewers with votes reset on new pushes, linked work items, all comments resolved, and a required build validation pipeline that runs build, tests and scans on every PR. External status checks like SonarQube, path-based required reviewers for infra or DB changes, squash-only merges, and very limited, audited bypass permissions complete it, so nothing reaches main without review and a green pipeline.*

Q19. Azure DevOps: What is the purpose of YAML pipeline templates?

In simple words: A template is a form with blanks. The platform team designs the form once; every team fills in their own details. With an `extends` template, teams can only fill in the blanks - they cannot remove the security checks printed on the form.

Detailed answer

- **Reuse and consistency:** write build/test/deploy logic once, use it in every pipeline; fix once, fixed everywhere.
- **Types of templates: stage**, **job**, **step** and **variable** templates, included with `- template: path.yml`.
- **Parameters:** typed inputs (`string`, `boolean`, `object`, `stepList`) with defaults and allowed values, so a template can serve many services and environments.
- `extends` **templates for security:** the pipeline must extend an approved template; the template decides the structure (mandatory scans, approved tasks only) and can wrap the user's steps. Combine with the "Required template" check on environments or service connections so only pipelines extending it can deploy.
- **Central template repo:** keep templates in a separate repo, referenced via `resources: repositories` and pinned to a tag or branch (`ref: refs/tags/v2`).
- Equivalents: Jenkins **shared libraries**, GitHub Actions **reusable workflows** and composite actions, GitLab `include`.

Example:

```
# templates repo: deploy-stage.yml
parameters:
- name: env
  type: string
  values: [dev, staging, prod]
- name: service
  type: string
stages:
- stage: deploy_${{ parameters.env }}
  jobs:
  - deployment: deploy
    environment: ${{ parameters.env }}          # approvals and checks live here
    strategy:
      runOnce:
        deploy:
          steps:
          - script: helm upgrade --install ${{ parameters.service }} chart/ -f values-${{ parameters
    .env }}.yaml

# service repo: azure-pipelines.yml
resources:
  repositories:
  - repository: templates
    type: git
    name: Platform/pipeline-templates
    ref: refs/tags/v2
extends:
  template: secure-pipeline.yml@templates      # enforced structure with mandatory scans
  parameters:
    service: orders
```

What to say to the interviewer: *YAML templates let us define stages, jobs, steps and variables once and reuse them with typed parameters across all services and environments. We keep them in a central repo referenced through resources repositories and pinned to a tag, and use extends templates plus the required-template check so every production pipeline must include our security scans. It is the Azure DevOps equivalent of Jenkins shared libraries or GitHub reusable workflows.*

Q20. Ansible: What is an Ansible playbook?

In simple words: A playbook is a written checklist: "on these servers, install nginx, copy this config, start the service". Ansible reads the checklist and does it on 1 or 1000 servers.

Detailed answer

- A playbook is a **YAML** file with one or more **plays**.
- Each play targets `hosts` (from inventory) and runs a list of **tasks**.
- Each task calls a **module** (`apt`, `yum`, `copy`, `template`, `service`, `user`).
- Tasks are **idempotent** - running twice does not change anything if the system is already in the desired state.
- Ansible is **agentless**: it connects over SSH (WinRM for Windows).

Example:

```
- name: Install and start nginx
  hosts: webservers
  become: true
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
        update_cache: true
    - name: Ensure nginx is running
      service:
        name: nginx
        state: started
        enabled: true

# Run it
ansible-playbook -i inventory.ini site.yml
```

> ***What to say to the interviewer:*** *A playbook is a YAML file of plays, where each play targets a group of hosts and runs idempotent tasks using modules like apt, copy, template and service. Ansible is agentless and connects over SSH, so we can configure many servers consistently with one command.*

Q21. Ansible: What is the difference between push and pull configuration management?

> **In simple words:** Push is the teacher walking to every desk and handing out the worksheet. Pull is every student walking to the teacher's desk every 30 minutes to pick up the latest worksheet.

Detailed answer

	Push	Pull
Who starts	Central control node	Each managed node
Agent needed	No (SSH)	Yes (agent on node)
Tools	Ansible (default), Salt SSH	Puppet, Chef, ansible-pull
Timing	On demand, immediate	Periodic (e.g. every 30 min)
Good for	Ad-hoc changes, deployments, small/medium fleets	Very large fleets, self-healing drift correction

- Push gives immediate control and simple setup, but the control node must reach every server.
- Pull scales well and auto-corrects drift, but changes are not instant and you must run agents.
- Ansible can also do pull with `ansible-pull` (node clones a Git repo and applies it via cron).

Example:

```
# Push (Ansible default)
ansible-playbook -i inventory/prod site.yml

# Pull (run on each node via cron)
*/30 * * * * ansible-pull -U https://github.com/org/config-repo.git local.yml
```

> ***What to say to the interviewer:*** *In push mode a central node connects out to servers and applies config immediately - Ansible works this way over SSH with no agent. In pull mode each server runs*

an agent that periodically fetches and applies config, like Puppet or Chef, which scales well and auto-fixes drift. Ansible also supports pull via ansible-pull.

Q22. Ansible: How do you manage Ansible inventory?

In simple words: The inventory is Ansible's phone book - the list of servers, grouped by role, with any special details about each.

Detailed answer

- **Static inventory:** INI or YAML file listing hosts and groups (`[web]`, `[db]`), plus variables.
- **Separate per environment:** `inventory/dev`, `inventory/prod` - you pick with `-i`, so you never hit prod by accident.
- **group_vars / host_vars:** folders holding variables per group or host.
- **Dynamic inventory:** for cloud where IPs change - plugins like `amazon.aws.aws_ec2` query AWS and group hosts by tags.

Example:

```
# inventory/prod/hosts.ini
[web]
web1.prod.internal
web2.prod.internal
[db]
db1.prod.internal ansible_user=ubuntu

# inventory/prod/aws_ec2.yml  (dynamic)
plugin: amazon.aws.aws_ec2
regions: [ap-south-1]
filters:
  tag:Env: prod
keyed_groups:
  - key: tags.Role
    prefix: role

ansible-inventory -i inventory/prod/aws_ec2.yml --graph
```

What to say to the interviewer: *We keep separate inventories per environment with group_vars and host_vars for variables. For AWS we use the aws_ec2 dynamic inventory plugin that discovers instances by tags, so new autoscaled servers are picked up automatically without editing files.*

Q23. Ansible: How do you work with modules and roles?

In simple words: A module is a single tool (a screwdriver). A role is a complete toolkit for one job ("set up nginx") that you can reuse in any project.

Detailed answer

- **Modules** are the units of work: `apt`, `copy`, `template`, `service`, `file`, `user`, `ec2_instance`, etc. Prefer modules over `shell` because they are idempotent.
- **Roles** package tasks, handlers, templates, files, defaults and vars in a standard folder structure so they can be reused and shared (Ansible Galaxy).
- Create a skeleton with `ansible-galaxy init`.

Example:

```
ansible-galaxy init roles/nginx

roles/nginx/
  tasks/main.yml        # what to do
  handlers/main.yml     # restart nginx on change
  templates/nginx.conf.j2
  defaults/main.yml     # default variables (lowest priority)
  vars/main.yml
  files/

# site.yml
- hosts: web
  become: true
  roles:
    - common
    - nginx
```

What to say to the interviewer: *Modules are idempotent building blocks like apt, template and service, and I avoid raw shell commands where a module exists. I organize reusable logic into roles created with ansible-galaxy init, each with tasks, handlers, templates and defaults, and combine roles in a site playbook per environment.*

Q24. Ansible: How do you use handlers in a playbook?

In simple words: A handler is "only restart the machine if you actually changed its settings". If nothing changed, nobody gets disturbed.

Detailed answer

- A handler is a special task that runs only when another task **notifies** it and that task reported `changed`.
- Handlers run **once at the end of the play**, even if notified many times.
- Typical use: restart/reload a service after its config file changes.
- Use `meta: flush_handlers` if you need them to run immediately.

Example:

```
- hosts: web
  become: true
  tasks:
    - name: Deploy nginx config
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Reload nginx
  handlers:
    - name: Reload nginx
      service:
        name: nginx
        state: reloaded
```

What to say to the interviewer: *Handlers are tasks triggered by notify only when a task actually changes something, and they run once at the end of the play. I use them to reload or restart services after a config template changes, so services are not restarted unnecessarily.*

Q25. Ansible: How do you store secrets securely in Ansible?

In simple words: Ansible Vault is a locked diary - the file sits in Git, but only people with the key can read what is inside.

Detailed answer

- **Ansible Vault** encrypts files or single variables with AES256.
- Store the vault password outside Git - in a CI credential, or a password file/script with restricted permissions.
- Use separate vault IDs per environment (`dev`, `prod`).
- Add `no_log: true` to tasks that handle secrets so they do not appear in output.
- Alternative: fetch secrets at runtime from AWS Secrets Manager or HashiCorp Vault via lookup plugins.

Example:

```
ansible-vault create group_vars/prod/vault.yml
ansible-vault encrypt_string 'S3cr3t!' --name 'db_password'
ansible-playbook site.yml --vault-id prod@~/.vault_pass_prod

# Runtime lookup from AWS Secrets Manager
db_password: "{{ lookup('amazon.aws.aws_secret', 'prod/db/password') }}"

- name: Create DB user
  mysql_user: name=app password="{{ db_password }}"
  no_log: true
```

What to say to the interviewer: *I encrypt secrets with Ansible Vault, keeping the vault password in the CI credential store rather than Git, with separate vault IDs per environment. For sensitive tasks I set no_log, and where possible I look up secrets at runtime from AWS Secrets Manager or HashiCorp Vault.*

Q26. Ansible: How do you run your Ansible projects?

In simple words: Like a well-organized cookbook: recipes (roles) in one folder, the guest list (inventory) per event, and a practice run (check mode) before the real dinner.

Detailed answer

Typical flow:

1. Project layout: `ansible.cfg`, `inventory/<env>`, `group_vars`, `roles/`, `site.yml`, `requirements.yml`.
2. Install dependencies: `ansible-galaxy install -r requirements.yml`.
3. Lint: `ansible-lint`.
4. Dry run: `--check --diff` to preview changes.
5. Limit blast radius: `--limit` one host/group first, `serial` for rolling batches.
6. Run from CI (Jenkins/GitHub Actions) or AWX/Ansible Tower for audit logs and RBAC, not from someone's laptop.

Example:

```
ansible-lint site.yml
ansible-playbook -i inventory/prod site.yml --check --diff
ansible-playbook -i inventory/prod site.yml --limit web1.prod.internal
ansible-playbook -i inventory/prod site.yml --tags nginx

# Rolling update: 25% of hosts at a time
- hosts: web
  serial: "25%"
  max_fail_percentage: 10
```

What to say to the interviewer: *Our Ansible repo has per-environment inventories, group_vars and reusable roles. Changes go through ansible-lint and a check-and-diff dry run, then run first on a limited host, then in rolling batches with serial. Runs are triggered from the CI pipeline so they are logged and repeatable.*

Q27. What is the difference between Scripted and Declarative Jenkins pipelines, and when would you use each?

In simple words: Declarative is a form with fixed boxes to fill in (stages, steps, post actions), so it's easy for everyone to read. Scripted is a blank sheet where you write Groovy code, which is powerful but easy to make messy.

Detailed answer

	Declarative	Scripted
Syntax	`pipeline { agent ... stages { stage { steps } } }`	`node { stage('x') { ... } }` plain Groovy
Structure	Fixed, validated before running	Free-form code
Readability	High, easy for the whole team	Depends on the author
Built-ins	`post`, `when`, `options`, `environment`, `parallel`, `input`	Write it yourself with try/catch/finally
Flexibility	Limited, extend with `script {}` blocks	Full Groovy (loops, dynamic stages)
Restart from stage	Supported	Not supported

- **Use Declarative by default** for almost everything; it is the recommended style.
- Use a `script { }` block, or shared-library code, for the few places that need loops or complex logic.
- Use fully **Scripted** only for highly dynamic pipelines (for example stages generated from a list of services).

Example:

```
// Declarative
pipeline {
  agent { label 'docker' }
  stages {
    stage('Build') { steps { sh 'make build' } }
    stage('Test')  { when { branch 'main' } steps { sh 'make test' } }
  }
  post { failure { echo 'notify team' } }
}

// Scripted
node('docker') {
  try {
    stage('Build') { sh 'make build' }
    for (svc in ['api', 'worker']) { stage("Test ${svc}") { sh "make test-${svc}" } }
  } catch (err) { echo 'notify team'; throw err }
}
```

What to say to the interviewer: *Declarative pipelines have a fixed, validated structure with built-in* `when`*,* `post`*,* `options` *and* `parallel`*, so they're easier to read and maintain. Scripted pipelines are plain Groovy and more flexible. I use Declarative by default and drop into a* `script` *block or a shared library only where I need loops or dynamic logic.*

Q28. How do you set up Jenkins from scratch on a fresh server?

In simple words: Setting up a new factory: install the power supply (Java), install the machine (Jenkins), lock the doors (admin user, TLS), and hire workers (agents) so the manager's office isn't used as the shop floor.

Detailed answer

1. **Prerequisites:** a Linux VM (2+ vCPU, 4GB+ RAM, a separate disk for `/var/lib/jenkins`) and Java 17 or 21.
2. **Install:** from the official Jenkins apt/yum repository (LTS), or run the `jenkins/jenkins:lts` container with a persistent volume.
3. **Unlock:** read the initial admin password from `/var/lib/jenkins/secrets/initialAdminPassword`, install the suggested plugins, and create an admin user.
4. **Secure it:** put Nginx in front with TLS, set the Jenkins URL, enable security (matrix or role-based authorization, SSO if possible), and set the controller's executors to 0.
5. **Agents:** add build agents (SSH agents, Docker, or the Kubernetes plugin for ephemeral pods), so builds never run on the controller.
6. **Credentials and tooling:** add Git/registry/cloud credentials in the credentials store, and configure tools (JDK, Maven, Node) or use container agents.
7. **Configuration as Code:** capture settings in JCasC YAML and the plugin list in `plugins.txt`, so the server can be rebuilt.
8. **Backups and monitoring:** back up `JENKINS_HOME` (thinBackup or EBS snapshots), and add the Prometheus metrics plugin.

Example:

```
# Ubuntu, LTS repo
sudo apt update && sudo apt install -y fontconfig openjdk-17-jre
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list
sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword

# or Docker
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk17
```

What to say to the interviewer: *I install Java 17 and Jenkins LTS from the official repo or run the LTS container with a persistent volume, unlock it with the initial admin password, install the suggested plugins and create an admin user. Then I secure it with TLS behind Nginx and role-based access, set the controller to zero executors, and add agents. Finally I capture the setup with JCasC and a plugin list so the server can be rebuilt, and back up JENKINS_HOME.*

Q29. How do you install Jenkins plugins, and which plugins are most common in a CI/CD setup?

In simple words: Jenkins on its own is a bare phone; plugins are the apps. Install only what you need and keep them updated, because too many apps slow the phone down and break after updates.

Detailed answer

How to install:

- UI: **Manage Jenkins -> Plugins -> Available**, search, install, and restart if needed.
- CLI or immutable images: list plugins in `plugins.txt` and run `jenkins-plugin-cli --plugin-file plugins.txt` in the Dockerfile. This is repeatable and version-pinned.
- Offline: upload the `.hpi` file under Advanced settings.
- Update carefully: check compatibility, test in staging, and back up `JENKINS_HOME` first, because plugin updates are the most common cause of broken Jenkins servers.

Common plugins:

Purpose	Plugin
Pipelines	Pipeline (workflow-aggregator), Pipeline: Stage View
Source control	Git, GitHub Branch Source, Bitbucket Branch Source
Credentials	Credentials Binding, SSH Agent, AWS Credentials
Containers and K8s	Docker Pipeline, Kubernetes
Quality	JUnit, SonarQube Scanner, Warnings Next Generation
Notifications	Slack Notification, Office 365 Connector, Email Extension
Security and admin	Role-based Authorization Strategy, Configuration as Code
Usability	Timestamper, AnsiColor, Blue Ocean (legacy UI)

Example:

```
FROM jenkins/jenkins:lts-jdk17
COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt
# plugins.txt:
#   workflow-aggregator:latest
#   git:latest
#   kubernetes:latest
#   configuration-as-code:latest
```

What to say to the interviewer: *For ad-hoc installs I use Manage Jenkins -> Plugins, but for production I bake plugins into the image with* `jenkins-plugin-cli` *and a pinned* `plugins.txt`*, so the setup is repeatable. The common ones are Pipeline, Git and GitHub Branch Source, Credentials Binding, Docker Pipeline, Kubernetes, JUnit, SonarQube, Slack or Teams notifications, Role-based Authorization and JCasC. I test plugin updates in staging, because they're the most common cause of breakage.*

Q30. Git: what is the "detached HEAD" state, what causes it, and how do you fix it?

In simple words: Normally you stand on a branch, a moving walkway, and new commits move it forward. In detached HEAD you've stepped off onto one specific tile (a commit). You can still walk around and make changes, but nothing is carrying them forward, so they're easy to lose.

Detailed answer

- **Causes:** `git checkout <commit-sha>`, `git checkout v1.2.0` (a tag), `git checkout origin/main` (a remote-tracking branch), during an interactive rebase or bisect. CI systems also check out a specific commit, which is why build logs often show it.
- **It's not an error.** It's useful for inspecting old code or building a tag.
- **To keep work done there:** create a branch from it: `git switch -c fix/from-old-commit`.
- **To just go back:** `git switch main`.
- **Lost the commits after switching away?** `git reflog` shows where HEAD has been, so you can create a branch from the lost SHA.

Example:

```
git checkout v2.3.0
# You are in 'detached HEAD' state...
git status              # HEAD detached at v2.3.0

# made a hotfix commit here and want to keep it:
git switch -c hotfix/2.3.1

# already switched away and lost it?
git reflog              # find: a1b2c3d HEAD@{2}: commit: fix null check
git branch recovered-fix a1b2c3d
```

What to say to the interviewer: *Detached HEAD means HEAD points directly at a commit instead of a branch, usually because I checked out a tag, a SHA or a remote branch, or CI checked out a specific commit. It's fine for looking around, but new commits there aren't on any branch, so to keep*

them I run `git switch -c new-branch`*. If I already switched away,* `git reflog` *lets me find the commit and recover it.*

Q31. Git/GitHub: how do you give a specific team member access to a repository, and grant them write permission?

Also asked as: How do you grant write permissions to a contributor in a GitHub repository?

In simple words: Hand out keys by role, not by person. Put people in teams (developers, QA, admins) and give each team the right level of key. When someone joins or leaves, you change the team, not every lock.

Detailed answer

- **Personal repository:** Settings -> Collaborators -> Add people. A collaborator on a personal repo gets write (push) access.
- **Organization repository (the right way):** add the person to a **team**, then give the team a role on the repo in Settings -> Collaborators and teams.
- **Roles:** Read (clone, open issues), Triage (manage issues and PRs), **Write** (push, merge PRs), Maintain (manage the repo without admin rights), Admin (settings, secrets, delete).
- **Write access still goes through branch protection:** protected `main`, required PR reviews, status checks and CODEOWNERS, so write doesn't mean "can push straight to production".
- Enforce SSO/2FA at the org level, review access regularly, and prefer fine-grained tokens or GitHub Apps for automation.

Example:

```
# Add a user as collaborator with write (push) permission
gh api -X PUT repos/my-org/meter-api/collaborators/jdoe -f permission=push

# Better: give a team write access
gh api -X PUT orgs/my-org/teams/backend-devs/repos/my-org/meter-api -f permission=push

# Check what someone has
gh api repos/my-org/meter-api/collaborators/jdoe/permission --jq .permission
```

What to say to the interviewer: *For an organization repo I add the person to the right team and give that team the Write role, instead of adding individuals, so onboarding and offboarding happen in one place. Write lets them push branches and merge PRs, but branch protection still requires reviews and passing checks on main. For a personal repo I'd add them as a collaborator, and in all cases I enforce 2FA or SSO and review access periodically.*

Q32. Git/GitHub: what is the difference between public and private repositories, and when would you use each?

In simple words: A public repo is a shop window: anyone on the street can look and copy the design. A private repo is the back office: only people you give a key to can enter.

Detailed answer

	Public	Private
Who can see it	Anyone on the internet	Only invited users or teams
Forking	Anyone	Only people with access (if allowed)
Typical use	Open source, samples, docs, portfolios	Company code, infrastructure, configs
Risk	Any leaked secret is exposed instantly and scraped by bots	Lower exposure, but secrets still don't belong in Git
GitHub Actions	Free minutes on hosted runners	Uses the plan's minute quota

- **Internal** repos (GitHub Enterprise) are visible to everyone in the company but not the public, which is good for inner-source.
- Company application code, Terraform, and anything with internal hostnames or architecture should be **private**.
- Before making a repo public, scan the whole history for secrets (gitleaks/trufflehog), not just the latest commit.

Example:

```
gh repo create my-org/infra-terraform --private
gh repo edit my-org/helm-charts --visibility public --accept-visibility-change-consequences
gitleaks detect --source . --log-opts="--all"     # scan the full history before going public
```

> ***What to say to the interviewer:*** *Public repos are visible to everyone and are for open-source work, samples or portfolios. Private repos are only for invited users or teams and are the default for company code and infrastructure. Before making anything public I scan the full Git history for secrets, because bots scrape public repos within minutes, and for company-wide sharing I use internal visibility.*

Q33. Git: how does squash work, when do you use it, and which commands do you run?

> **In simple words:** You wrote ten messy draft notes while working on one feature. Squashing staples them into one clean note, "Add meter-read retry logic", before handing it to the team.

Detailed answer

- **What it does:** combines several commits into one, so the main branch history shows one meaningful commit per feature or fix.
- **When to use it:** before merging a feature branch full of "wip", "fix typo" and "try again" commits, or when team policy says one commit per PR.
- **When not to:** when individual commits matter for review, bisect or reverts, and never on branches others have already pulled without coordinating (it rewrites history).

Three ways:

1. **Interactive rebase:** `git rebase -i HEAD~4`, then mark commits as `squash` or `fixup` and edit the message.

2. **Merge with squash:** `git merge --squash feature` stages all changes as one new commit.
3. **GitHub/GitLab "Squash and merge"** button on the PR, the most common in teams.

Example:

```
git log --oneline -4
# 9f1c2aa fix lint
# 7d3b1e0 wip tests
# 4a8e9c1 wip retry
# 1c2d3e4 add retry logic

git rebase -i HEAD~4
# pick 1c2d3e4 add retry logic
# squash 4a8e9c1 wip retry
# fixup 7d3b1e0 wip tests
# fixup 9f1c2aa fix lint

git push --force-with-lease        # only on your own feature branch

# alternative:
git checkout main && git merge --squash feature/retry && git commit -m "Add meter-read retry logic"
```

> ***What to say to the interviewer:*** *Squash combines several commits into one so the main branch has one clean commit per change. I do it with* `git rebase -i` *and squash or fixup, with* `git merge --squash`*, or most often with the Squash and merge button on the PR. Because it rewrites history, I only squash my own feature branches and push with* `--force-with-lease`*.*

Q34. Git: what is the difference between Git and GitHub?

> **In simple words:** Git is the camera that takes snapshots of your code. GitHub is the online photo album where you store the snapshots, share them, and let others comment on them.

Detailed answer

	Git	GitHub
What it is	A distributed version control tool	A cloud platform that hosts Git repositories
Runs where	On your machine, works offline	On the web (or GitHub Enterprise Server)
Made by	Linus Torvalds (2005), open source	GitHub Inc. (owned by Microsoft)
Does	Commits, branches, merges, history	PRs, code review, Issues, Actions CI/CD, Packages, security scanning, access control
Alternatives	Mercurial, SVN (centralized)	GitLab, Bitbucket, Azure Repos, Gitea

- You can use Git without GitHub, but GitHub is useless without Git.
- In DevOps, GitHub is often the **trigger point**: a push or PR fires webhooks or GitHub Actions.

Example:

```
git init && git add . && git commit -m "first commit"        # Git only, local
git remote add origin git@github.com:my-org/app.git          # now connect to GitHub
git push -u origin main
gh pr create --fill                                          # GitHub feature (PR)
```

> ***What to say to the interviewer:*** *Git is the version control tool that tracks changes, branches and history, and it works fully offline on my laptop. GitHub is a hosting platform built around Git that adds pull requests, code review, Issues, Actions for CI/CD and access control. GitLab, Bitbucket and Azure Repos are alternatives to GitHub, not to Git.*

Q35. Git: what are the essential Git commands, and what does each do?

> **In simple words:** Think of three places: your desk (working directory), a tray of things ready to file (the staging area), and the filing cabinet (commits), plus a shared office cabinet (the remote). Each command moves work between them.

Detailed answer

Command	What it does
`git init`	Creates a new repo (adds a hidden `.git` folder)
`git clone <url>`	Copies a remote repo locally with full history, linked to `origin`
`git status`	Shows changed, staged and untracked files
`git add <file>` / `git add .`	Stages changes for the **next commit**
`git commit -m "msg"`	Saves the staged snapshot to **local** history
`git push`	Uploads local commits to the remote branch (`-u origin <branch>` the first time)
`git fetch`	Downloads remote changes without touching your files
`git pull`	`fetch` + merge (or `--rebase`) into your current branch
`git branch` / `git switch -c x`	Lists branches / creates and switches to a new branch
`git checkout`	Older all-in-one command, now split into `switch` (branches) and `restore` (files)
`git merge` / `git rebase`	Integrates another branch
`git log --oneline --graph`	History
`git diff` / `git diff --staged`	Unstaged / staged changes
`git restore <file>`	Discards local changes to a file
`git reset` / `git revert`	Move a branch pointer (rewrites history) / create an undo commit (safe on shared branches)
`git tag v1.0.0`	Marks a release

Correcting common misconceptions:

- `git add` does not "compare with the remote"; it only stages changes for the next commit.
- `git push` does not create a repository; the remote repo must exist. It can create a new **branch** on the remote.

- `git checkout -b` creates a branch; the modern equivalent is `git switch -c`.

Example:

```
git clone git@github.com:my-org/meter-api.git && cd meter-api
git switch -c feature/retry
# ...edit files...
git status && git add src/retry.go && git commit -m "Add retry with backoff"
git push -u origin feature/retry
git fetch origin && git rebase origin/main      # stay current with main
```

> ***What to say to the interviewer:*** *The daily flow is clone, switch -c to create a branch, edit, add to stage, commit to save locally, and push to upload to the remote. I fetch or pull to get others' changes, and use merge or rebase to integrate. I also use status, log, diff and restore constantly, and I use revert rather than reset on shared branches because it doesn't rewrite history.*

Q36. Git: what is the difference between git push, git fetch and git pull?

> **In simple words:** fetch is checking the post box and bringing the letters inside without opening them. pull is bringing them in and immediately mixing them into your own papers. push is posting your own letters out.

Detailed answer

Command	Direction	Changes your working files?
`git fetch`	Remote -> local (updates `origin/*` only)	No, it's safe anytime
`git pull`	Remote -> local, then merge/rebase into the current branch	Yes, can cause conflicts
`git push`	Local -> remote	No (updates the remote branch)

- `git fetch` then `git log HEAD..origin/main` lets you **review** incoming changes before integrating them.
- `git pull --rebase` (or `git config pull.rebase true`) avoids noisy merge commits.
- `git push` is rejected if the remote has commits you don't have, so fetch/rebase first. Use `--force-with-lease`, never plain `--force`, on your own branches.

Example:

```
   remote (GitHub)      origin/main    your local main    working files
        |  <-- push --------------------------|
        |  -- fetch -->  updated              |              unchanged
        |  -- pull  -->  updated  --merge-->  updated        updated
```

```
git fetch origin
git log --oneline HEAD..origin/main      # what's new upstream?
git pull --rebase origin main
git push origin feature/retry
```

> ***What to say to the interviewer:*** `git fetch` *downloads new commits into the remote-tracking branches without touching my work, so it's always safe.* `git pull` *is fetch plus merge or rebase into my current branch, so it changes my files and can conflict.* `git push` *sends my commits to the remote and is rejected if the remote has moved ahead, so I fetch or rebase first.*

Q37. Git: how do you recover deleted changes (lost commits, a deleted branch, discarded files)?

In simple words: Git has a CCTV recording (the reflog) of everywhere your branches have pointed recently. Even if you "deleted" something, you can rewind the tape and take it back, as long as it was committed or stashed at some point.

Detailed answer

What was lost	How to recover
Commits after a bad `reset --hard` or rebase	`git reflog`, then `git reset --hard HEAD@{n}` or `git branch rescue <sha>`
A deleted branch	`git reflog` (or the SHA from `git branch -D` output), then `git branch <name> <sha>`
A dropped stash	`git fsck --lost-found`, look for dangling commits, then `git stash apply <sha>`
A file deleted but not committed	`git restore <file>` (from the last commit)
A file deleted in an old commit	`git log --diff-filter=D -- path` to find it, then `git checkout <sha>^ -- path`
A pushed change someone reverted	`git revert <revert-sha>` (revert the revert)
Uncommitted, never-staged edits	Git can't help; try the IDE local history or editor backups

- The reflog is local and entries expire (90 days by default), so recover sooner rather than later.
- Anything that was pushed also exists on the remote and in teammates' clones.

Example:

```
git reset --hard HEAD~3          # oops
git reflog
# e5f6a7b HEAD@{0}: reset: moving to HEAD~3
# 9c8b7a6 HEAD@{1}: commit: add billing export     <- what we want
git reset --hard 9c8b7a6

git branch -D feature/export     # oops: "Deleted branch feature/export (was 3d4e5f6)"
git branch feature/export 3d4e5f6
```

What to say to the interviewer: *For anything that was committed,* `git reflog` *is my safety net: it records where HEAD and the branches pointed, so I can reset back or create a branch from the lost SHA, including after a hard reset or a deleted branch. For dropped stashes I use* `git fsck --lost-found`*, for a deleted file* `git restore`*, and for a bad change on a shared branch I use* `git revert`*. Changes that were never committed or stashed can't be recovered by Git.*

Q38. Git: which useful Git commands are often ignored?

In simple words: Most people only use the steering wheel and pedals. These are the less-known dashboard buttons: rear camera, cruise control, lane assist. They save you when things go wrong.

Detailed answer

Command	Why it's useful
`git reflog`	Recover "lost" commits and branches
`git bisect start/good/bad`	Binary-search history to find the commit that introduced a bug
`git cherry-pick <sha>`	Copy one fix onto another branch (e.g. a hotfix onto a release branch)
`git blame -w -C <file>`	Who changed each line, ignoring whitespace and moved code
`git worktree add ../hotfix main`	Second working directory, so you can fix prod without stashing
`git stash -p` / `git add -p`	Stash or stage only selected hunks
`git log --oneline --graph --all`	Visual branch history
`git commit --fixup <sha>` + `git rebase -i --autosquash`	Clean fixes into the right commit
`git restore --staged <file>`	Unstage without losing changes
`git clean -nd`	Preview which untracked files would be deleted
`git shortlog -sn`	Commits per author
`git config rerere.enabled true`	Reuse recorded conflict resolutions during repeated rebases
`git switch -`	Jump back to the previous branch

Example:

```
git bisect start
git bisect bad HEAD
git bisect good v2.1.0
git bisect run ./scripts/test_meter_parse.sh      # finds the first bad commit automatically
git bisect reset

git worktree add ../app-hotfix release/2.3        # fix prod without disturbing current work
```

> ***What to say to the interviewer:*** *The ones I rely on that people often skip are reflog for recovery, bisect to find the commit that introduced a bug, cherry-pick to move a hotfix between branches, and worktree to work on a hotfix without stashing. I also use* `add -p` *for clean commits, fixup plus autosquash to tidy history, and* `clean -n` *to preview deletions.*

Advanced

Q39. Explain your CI/CD pipeline architecture, including approvals.

Also asked as: Describe the CI/CD workflow in your project. Explain your pipeline with CodePipeline/Jenkins and approvals.

> **In simple words:** Code travels on a conveyor belt through checkpoints: build, test, security scan, package, deploy to test rooms, and finally a manager signs off before it reaches customers.

Detailed answer

End-to-end flow (Jenkins + EKS example):

1. Developer opens a PR -> webhook triggers Jenkins multibranch job.
2. **CI stage:** checkout, build, unit tests, lint, SonarQube quality gate, dependency/secret scan.
3. On merge to main: Docker build, Trivy image scan, push to ECR with tag `version-gitsha`.
4. Deploy to **dev** automatically via Helm (or update the GitOps repo for ArgoCD).
5. Automated smoke/integration tests; promote the same image to **staging**.
6. **Manual approval** (Jenkins `input` with `submitter` restricted to release managers, or CodePipeline Manual Approval action with SNS email).
7. Deploy to **prod** with rolling/canary strategy; post-deploy health checks; auto-rollback on failure.
8. Notifications to Teams/Slack at each stage; metrics and logs watched in Grafana.

AWS-native equivalent: CodeCommit/GitHub -> CodePipeline -> CodeBuild (build/test/push ECR) -> Manual approval action -> CodeDeploy / EKS deploy.

Example:

```
Git PR --webhook--> Jenkins CI (build, test, Sonar, Trivy)
  --merge--> Docker build -> ECR (orders-api:1.4.0-a1b2c3d)
  --> Helm deploy DEV -> smoke tests
  --> Helm deploy STAGING -> integration tests
  --> [Manual Approval: release-managers]
  --> Helm deploy PROD (rolling / canary) -> health check -> auto rollback if failed
```

> ***What to say to the interviewer:*** *PRs trigger CI in Jenkins - build, unit tests, SonarQube and security scans. Merges build an immutable Docker image pushed to ECR, auto-deploy to dev via Helm, run smoke tests, promote to staging, and then a restricted manual approval gates production, where we deploy with a rolling or canary strategy and automatic rollback on failed health checks.*

Q40. Explain how a typical AWS-native CI/CD pipeline works end to end.

> **In simple words:** Think of an airport: check-in (source), security scan (build and test), boarding with a gate agent (approval), take-off in small groups (canary deploy), air traffic control watching the radar (monitoring), and the plane can turn back (rollback).

Detailed answer

1. **Source:** GitHub (via CodeStar/CodeConnections) or CodeCommit; a push or PR merge triggers CodePipeline through a webhook or EventBridge rule. S3 or ECR image push can also be a source.
2. **Build (CodeBuild or Jenkins):** `buildspec.yml` compiles, runs lint and unit tests, publishes test reports, builds the Docker image and pushes it to **ECR** (tag = commit SHA), or packages a zip to **S3**. Artifacts are versioned.
3. **Test and scan:** SonarQube quality gate, SCA/SAST, Trivy image scan, **Amazon Inspector** ECR scanning; integration tests in a test environment. Fail the pipeline on critical issues.
4. **Approval:** CodePipeline **Manual Approval** action (SNS email/Teams) before prod.
5. **Deploy: CodeDeploy** for EC2/ECS/Lambda (blue/green, canary `Canary10Percent5Minutes`, linear), **CloudFormation/CDK/Terraform** for infra,

Helm/kubectl/ArgoCD for **EKS**, **SAM** for Lambda.

6. **Rollback:** versioned immutable artifacts; CodeDeploy auto-rollback on failed deployment or **CloudWatch alarm**; ECS deployment circuit breaker; Lambda alias shifting back.
7. **Monitoring:** CloudWatch metrics/logs/alarms, SNS notifications, X-Ray traces, pipeline state change events via EventBridge to Teams/Slack.
8. **Security:** each stage has its own least-privilege IAM role; secrets from **Secrets Manager / Parameter Store**; cross-account deploys via assume-role into prod account; artifacts encrypted with KMS.

Improvement over a basic answer: mention **multi-account** (tools account deploys to dev/prod accounts), **build once promote everywhere**, and **alarm-based automatic rollback**.

Example:

```
# buildspec.yml
version: 0.2
phases:
  pre_build:
    commands:
      - aws ecr get-login-password | docker login -u AWS --password-stdin $ECR
      - TAG=${CODEBUILD_RESOLVED_SOURCE_VERSION:0:7}
  build:
    commands:
      - npm ci && npm run lint && npm test
      - docker build -t $ECR/orders:$TAG .
  post_build:
    commands:
      - docker push $ECR/orders:$TAG
      - printf '[{"name":"orders","imageUri":"%s"}]' $ECR/orders:$TAG > imagedefinitions.json
reports:
  unit: { files: ['reports/junit.xml'], file-format: JUNITXML }
artifacts:
  files: [imagedefinitions.json, appspec.yaml, taskdef.json]
```

> ***What to say to the interviewer:*** *A push to GitHub triggers CodePipeline through a webhook or EventBridge; CodeBuild runs lint and unit tests, builds the image and pushes it to ECR with the commit SHA. Scans like SonarQube, Trivy and Inspector gate it, a manual approval guards prod, and CodeDeploy does blue-green or canary on ECS, EC2 or Lambda with automatic rollback on CloudWatch alarms. Each stage has least-privilege IAM roles, secrets come from Secrets Manager, and we monitor with CloudWatch, SNS and X-Ray.*

Q41. What stages do you define in Jenkins, and how do you ensure full quality checks?

> **In simple words:** Quality checks are a series of filters - each one catches a different kind of dirt. Cheap and fast filters first, expensive ones later, and nothing reaches customers unless it passes all of them.

Detailed answer

Typical stages, fast to slow:

Stage	Tool examples	Gate
Checkout	git (shallow)	-

Stage	Tool examples	Gate
Lint / format	eslint, checkstyle, hadolint, tflint	fail on error
Unit tests	JUnit, pytest, jest	all pass
Coverage	JaCoCo, Istanbul	e.g. >= 80% on new code
SAST + quality	SonarQube / Semgrep	quality gate must pass
SCA (dependencies)	OWASP Dependency-Check, Snyk, Trivy fs	no CRITICAL
Secret scan	gitleaks	no findings
Build image	docker buildx	-
Image scan	Trivy, Inspector	no CRITICAL/HIGH fixable
Deploy dev + integration tests	Helm, Postman/newman, k6	pass
Approval	input step / environment	authorized approver
Deploy prod + smoke test	Helm --atomic	healthy, else rollback

How we enforce it:

- Gates **fail the build** (`abortPipeline: true`, `--exit-code 1`), not just report.
- Branch protection requires the checks before merge.
- The stages live in a **shared library**, so no team can quietly drop a check.
- Reports published (`junit`, coverage) so trends are visible.

Example:

```
stage('Quality') {
  parallel {
    stage('Lint')    { steps { sh 'npm run lint' } }
    stage('Unit')    { steps { sh 'npm test -- --coverage' } }
    stage('Secrets') { steps { sh 'gitleaks detect --exit-code 1' } }
    stage('SCA')     { steps { sh 'trivy fs --severity CRITICAL --exit-code 1 .' } }
  }
}
stage('Sonar Gate') {
  steps {
    withSonarQubeEnv('sonar') { sh 'sonar-scanner' }
    timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true }
  }
}
```

> ***What to say to the interviewer:*** *Our stages go from fast to slow: lint, unit tests with a coverage gate, SonarQube quality gate, dependency and secret scanning, image build and Trivy scan, deploy to dev with integration tests, approval, and prod deploy with smoke tests. Every gate fails the build rather than just reporting, branch protection requires them before merge, and they live in a shared library so they cannot be skipped.*

Q42. Write a CI/CD Jenkins pipeline that you have implemented in your project.

Also asked as: Walk me through a Jenkins pipeline you have written.

In simple words: This is the full recipe card - every step the code goes through from GitHub to production on EKS.

Detailed answer

Key design points: runs on Kubernetes pod agents, AWS access via IAM role (no static keys), immutable image tags, quality gate and image scan before push, auto-deploy to dev, approval gate for prod, rollback on failure.

Example:

```
pipeline {
  agent { label 'k8s-docker' }
  options {
    timeout(time: 45, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
    disableConcurrentBuilds()
    timestamps()
  }
  environment {
    AWS_REGION = 'ap-south-1'
    ECR        = '123456789012.dkr.ecr.ap-south-1.amazonaws.com'
    APP        = 'orders-api'
    TAG        = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build & Unit Test') {
      steps { sh 'mvn -B clean verify' }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Code Quality') {
      steps {
        withSonarQubeEnv('sonar') { sh 'mvn -B sonar:sonar' }
        timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true }
      }
    }
    stage('Docker Build') { steps { sh 'docker build -t $ECR/$APP:$TAG .' } }
    stage('Image Scan') {
      steps { sh 'trivy image --exit-code 1 --severity CRITICAL,HIGH $ECR/$APP:$TAG' }
    }
    stage('Push to ECR') {
      when { branch 'main' }
      steps {
        sh '''
          aws ecr get-login-password --region $AWS_REGION |
            docker login --username AWS --password-stdin $ECR
          docker push $ECR/$APP:$TAG
        '''
      }
    }
    stage('Deploy DEV') {
      when { branch 'main' }
      steps {
        sh '''
          aws eks update-kubeconfig --name dev-eks --region $AWS_REGION
          helm upgrade --install $APP charts/$APP -n dev \
            --set image.tag=$TAG --wait --timeout 5m --atomic
        '''
      }
    }
```

```
    stage('Approve PROD') {
      when { branch 'main' }
      steps {
        timeout(time: 2, unit: 'HOURS') {
          input message: "Deploy $TAG to PROD?", submitter: 'release-managers'
        }
      }
    }
    stage('Deploy PROD') {
      when { branch 'main' }
      steps {
        sh '''
          aws eks update-kubeconfig --name prod-eks --region $AWS_REGION
          helm upgrade --install $APP charts/$APP -n prod \
            --set image.tag=$TAG --wait --timeout 10m --atomic
        '''
      }
    }
  }
  post {
    success { echo "Deployed ${env.TAG}" }
    failure { echo 'Send Teams/Slack failure alert with build URL' }
    always  { sh 'docker rmi $ECR/$APP:$TAG || true'; cleanWs() }
  }
}
```

What to say to the interviewer: *Our Jenkinsfile runs on Kubernetes agents with an IAM role for AWS. It builds and unit-tests, enforces a SonarQube quality gate, builds a Docker image tagged with build number and commit SHA, scans it with Trivy, pushes to ECR, and deploys to dev with helm upgrade --atomic. Production needs an approval from release managers and uses the same image tag, with atomic Helm rollback if health checks fail.*

Q43. What kinds of applications do you deploy through Jenkins, and with which deployment tools?

In simple words: Different parcels need different delivery vans - a container goes by Helm truck to Kubernetes, a serverless function goes by SAM courier to Lambda, a server config goes by Ansible.

Detailed answer

App type	Artifact	Deploy tool
Containerized microservices (Java/Node/Go APIs)	Docker image in ECR	Helm / kubectl to EKS, or ArgoCD (GitOps)
ECS services	Docker image + task definition	CodeDeploy blue/green, `aws ecs update-service`
Serverless functions	zip in S3	SAM / Serverless / Terraform to Lambda
Static frontends (React/Angular)	build folder	`aws s3 sync` + CloudFront invalidation
VM-based apps	jar/package	Ansible, CodeDeploy agent
Infrastructure	Terraform code / plan	`terraform plan` + approval + `apply`
Config / monitoring	dashboards, alert rules	Grafana API, Helm (kube-prometheus-stack)

Why this matters: same pipeline template, different final deploy stage. Kubernetes apps prefer Helm with `--atomic` or ArgoCD because they give rollback history and drift correction.

Example:

```
stage('Deploy Frontend') {
  steps {
    sh 'aws s3 sync build/ s3://web-prod-bucket --delete'
    sh 'aws cloudfront create-invalidation --distribution-id $CF_ID --paths "/*"'
  }
}
stage('Deploy Lambda') { steps { sh 'sam deploy --stack-name reports --no-confirm-changeset' } }
stage('Deploy EKS')    { steps { sh 'helm upgrade --install api charts/api --set image.tag=$TAG --at
    omic' } }
```

> ***What to say to the interviewer:*** *Mostly containerized microservices deployed to EKS with Helm or through ArgoCD, plus Lambda functions via SAM or Terraform, static frontends to S3 with CloudFront invalidation, and Terraform infrastructure with plan and approval. The pipeline template is shared, and only the deploy stage changes per application type.*

Q44. How do you handle the CD aspect and promote builds across environments (Dev -> Staging -> Prod)?

Also asked as: How do you handle continuous delivery in your projects? Explain multi-environment promotion flows. How do you manage dev, staging and prod environments in CI/CD?

> **In simple words:** You bake one cake, taste it in the test kitchen, then in the dress-rehearsal kitchen, and only then serve the SAME cake to guests - you never bake a new one for the guests.

Detailed answer

Principles:

- **Build once, promote the same artifact:** the image tag tested in dev/staging is exactly what goes to prod. Never rebuild for prod.

- **Environment config is separate** from the artifact: Helm values per env (`values-dev.yaml`, `values-prod.yaml`), ConfigMaps, secrets from Secrets Manager.
- **Gates between environments:** automated tests (smoke, integration, performance), quality gates, and a manual approval for prod.
- **Progressive rollout in prod:** rolling/canary with health checks and automatic rollback.
- **GitOps option:** promotion = a PR that updates the image tag in the env folder of a config repo; ArgoCD syncs it. Git history becomes the audit log.
- **Isolate the environments themselves:** separate namespaces or clusters, ideally separate cloud accounts/subscriptions for prod; per-env deploy identities and secrets, so a dev pipeline physically cannot touch prod.
- **Environment objects in the CI tool:** GitHub/GitLab environments or Azure DevOps Environments carry protection rules (approvals, wait timers, allowed branches) and env-scoped secrets; Jenkins uses folder-scoped credentials plus an `input` gate.
- **Keep environments alike:** the same Terraform modules and Helm chart for every env, only sizes and endpoints differ, so "works in staging" means something in prod.

Example:

```
gitops-repo/
  apps/orders-api/
    base/
    envs/dev/values.yaml       image.tag: 1.4.0-a1b2c3d
    envs/staging/values.yaml   image.tag: 1.4.0-a1b2c3d
    envs/prod/values.yaml      image.tag: 1.3.2-9f8e7d6   <- promote via PR

# Promote to prod = PR that changes one line, approved by release manager
yq -i '.image.tag = "1.4.0-a1b2c3d"' apps/orders-api/envs/prod/values.yaml
```

> ***What to say to the interviewer:*** *We build the artifact once and promote the same immutable image tag through dev, staging and prod, with only environment config differing through per-env Helm values. Each promotion is gated by automated tests, and prod needs an approval. With GitOps, promotion is simply a reviewed pull request that bumps the image tag in the prod folder, and ArgoCD applies it.*

Q45. How do you connect Jenkins to AWS securely and push Docker images for multiple services to ECR?

Also asked as: How do you connect multiple services in a Jenkins pipeline and push images to AWS ECR securely using secrets?

> **In simple words:** Instead of giving Jenkins a permanent key to AWS, give it a temporary visitor badge that expires by itself.

Detailed answer

Secure options, best first:

1. **Jenkins on EC2/EKS -> IAM role:** EC2 instance profile or EKS IRSA/Pod Identity for the agent pod. No keys stored anywhere; credentials rotate automatically.
2. **Cross-account:** agent role does `sts:AssumeRole` into the target account's deploy role.

3. **Jenkins outside AWS:** OIDC federation or, last resort, AWS credentials in Jenkins Credentials (AWS Credentials plugin) with least-privilege policy and regular rotation.

- The IAM policy should allow only `ecr:GetAuthorizationToken` plus push actions on specific repositories.

For many services: loop or `parallel` over a list of services (monorepo) or a shared library function `buildAndPush(service)`; only build services whose folder changed.

Example:

```
def services = ['orders', 'payments', 'users']
pipeline {
  agent { label 'docker' }          // agent has IAM role via instance profile / IRSA
  environment { ECR = '123456789012.dkr.ecr.ap-south-1.amazonaws.com'; TAG = "${GIT_COMMIT.take(7)}"
    }
  stages {
    stage('ECR Login') {
      steps { sh 'aws ecr get-login-password | docker login -u AWS --password-stdin $ECR' }
    }
    stage('Build & Push') {
      steps {
        script {
          def jobs = [:]
          services.each { s ->
            jobs[s] = {
              if (sh(script: "git diff --quiet HEAD~1 -- services/${s}", returnStatus: true) != 0) {
                sh "docker build -t $ECR/${s}:$TAG services/${s} && docker push $ECR/${s}:$TAG"
              }
            }
          }
          parallel jobs
        }
      }
    }
  }
}
```

What to say to the interviewer: *Jenkins agents run on AWS with an IAM role - instance profile or IRSA - so there are no static keys, and cross-account deploys use sts assume-role. The role only has ECR push rights on specific repos. For multiple services we build and push in parallel, only for services whose folder changed, each tagged with the commit SHA.*

Q46. How do Jenkins and a Kubernetes cluster communicate?

In simple words: Jenkins needs a pass (kubeconfig or cloud identity) to talk to the Kubernetes front desk (API server). With GitOps, Jenkins does not even enter - it just leaves a note in Git and ArgoCD inside the cluster picks it up.

Detailed answer

Two directions:

- **Jenkins uses Kubernetes for agents:** Kubernetes plugin creates build pods on demand. Configured under Manage Jenkins -> Clouds with the API server URL and a service account / credential.
- **Jenkins deploys to Kubernetes:**

- Agent has `kubectl`/`helm` and a kubeconfig. On EKS: `aws eks update-kubeconfig` using the agent's IAM role, which is mapped to Kubernetes RBAC via EKS access entries (or aws-auth ConfigMap).
- The deploy identity should have a namespace-scoped Role, not cluster-admin.
- Store kubeconfig (if used) as a Secret file credential.
- **GitOps (pull) model:** Jenkins only updates the manifest repo; ArgoCD in the cluster pulls and applies. Cluster credentials never leave the cluster - most secure.

Example:

```
# EKS: map Jenkins IAM role to a k8s group (access entries)
aws eks create-access-entry --cluster-name prod-eks \
  --principal-arn arn:aws:iam::123456789012:role/jenkins-deployer
aws eks associate-access-policy --cluster-name prod-eks \
  --principal-arn arn:aws:iam::123456789012:role/jenkins-deployer \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSEditPolicy \
  --access-scope type=namespace,namespaces=orders

# In pipeline
withCredentials([file(credentialsId: 'kubeconfig-dev', variable: 'KUBECONFIG')]) {
  sh 'kubectl -n orders rollout status deploy/orders-api'
}
```

> ***What to say to the interviewer:*** *Jenkins talks to the Kubernetes API server either through a kubeconfig credential or, on EKS, through its IAM role mapped to namespace-scoped RBAC using access entries. We also use the Kubernetes plugin so builds run in ephemeral pods. For production we prefer GitOps, where Jenkins only updates the manifest repo and ArgoCD inside the cluster pulls the change.*

Q47. How can Jenkins be integrated with other DevOps tools?

> **In simple words:** Jenkins is the conductor of an orchestra - it does not play every instrument, it tells Git, Maven, SonarQube, Docker, Trivy, Helm and Slack when to play.

Detailed answer

Integrations through plugins, CLIs and webhooks:

Area	Tools	How
Source control	GitHub, GitLab, Bitbucket	Webhooks, Git/Branch Source plugins
Build	Maven, Gradle, npm	Tools config, container agents
Code quality	SonarQube	`withSonarQubeEnv`, `waitForQualityGate`
Security	Trivy, Snyk, OWASP Dependency-Check, gitleaks	CLI steps, fail on severity
Artifacts	Nexus, Artifactory, ECR, S3	Plugins or CLI push
Containers	Docker, Kubernetes	Docker Pipeline plugin, Kubernetes plugin
Deploy	Helm, kubectl, ArgoCD, Terraform, Ansible	CLI in stages

Area	Tools	How
Secrets	Vault, AWS Secrets Manager	Vault plugin, IAM roles
Notify	Teams, Slack, email, Jira	Notification plugins, Jira plugin
Monitoring	Prometheus	Prometheus metrics plugin -> Grafana

Example:

```
stage('Terraform Plan') { steps { sh 'terraform init && terraform plan -out=tfplan' } }
stage('Ansible')        { steps { sh 'ansible-playbook -i inventory/dev site.yml' } }
stage('Jira update')    { steps { jiraComment issueKey: 'PC-123', body: "Deployed ${TAG}" } }
```

> ***What to say to the interviewer:*** *Jenkins integrates through plugins, CLIs and webhooks - GitHub webhooks to trigger, Maven or npm to build, SonarQube and Trivy for quality and security, ECR or Nexus for artifacts, Helm, Terraform and Ansible to deploy, Vault or Secrets Manager for secrets, and Teams or Slack plus Jira for notifications. We also export Jenkins metrics to Prometheus and Grafana.*

Q48. What are Jenkins Shared Libraries, and how do you use them to reduce or modularize a large Jenkinsfile?

Also asked as: How can you reduce the size of a Jenkinsfile? How do you refactor a Jenkinsfile that exceeds practical limits? How are shared libraries written, defined and integrated into Jenkinsfiles?

> **In simple words:** Instead of every team writing the same 500-line recipe, put the common recipe in a shared cookbook and each team just writes "use the standard recipe for a Java app".

Detailed answer

Why it matters: huge Jenkinsfiles are hard to maintain and can hit the JVM "Method code too large" (64KB) limit.

A **Shared Library** is a separate Git repo of reusable Groovy pipeline code. Structure:

- `vars/` - global steps; `vars/helmDeploy.groovy` with a `call()` method becomes a step `helmDeploy(...)`.
- `src/` - Groovy classes in packages (e.g. `org/company/Docker.groovy`) for complex logic.
- `resources/` - non-Groovy files (templates, scripts) loaded with `libraryResource`.

Defining: Manage Jenkins -> System -> **Global Pipeline Libraries** -> name `ci-lib`, default version (branch/tag), Git repo URL, credentials. Global libraries are trusted (run outside the sandbox), so only admins should change them; folder-level libraries are sandboxed.

Integrating: `@Library('ci-lib@v2') _` at the top of the Jenkinsfile (pin a tag for stability), or `library 'ci-lib@v2'` dynamically. Version the library with tags and test changes on a branch before moving the tag.

Techniques to shrink the Jenkinsfile:

- **Shared Libraries:** move common steps into the library. Each app Jenkinsfile becomes 5-10 lines.
- **Move logic into scripts:** long shell blocks go into `scripts/build.sh`, called from the pipeline.

- **Parameterize and loop** instead of copy-pasted stages; use `matrix` for OS/version combinations.
- **Templates:** Jenkins Templating Engine or a standard pipeline per app type (java, node, helm).
- **Split pipelines:** separate CI and CD jobs, or trigger downstream jobs.
- Keep heavy Groovy logic out of the controller; avoid giant `script {}` blocks.

Example:

```
// ci-lib repo: vars/javaServicePipeline.groovy
def call(Map cfg) {
  pipeline {
    agent { label 'docker' }
    stages {
      stage('Build')  { steps { sh 'mvn -B clean verify' } }
      stage('Image')  { steps { buildAndPushImage(cfg.app) } }
      stage('Deploy') { when { branch 'main' } steps { helmDeploy(cfg.app, 'dev') } }
    }
  }
}

// ci-lib repo: vars/helmDeploy.groovy  -> usable as step helmDeploy('orders-api', 'dev')
def call(String app, String env) {
  def values = libraryResource "helm/values-${env}.yaml"   // from resources/
  writeFile file: 'values.yaml', text: values
  sh "helm upgrade --install ${app} charts/${app} -n ${env} -f values.yaml --atomic"
}

// Application Jenkinsfile - now only 3 lines
@Library('ci-lib@v2') _
javaServicePipeline(app: 'orders-api')
```

> ***What to say to the interviewer:*** *A shared library is a separate Git repo with vars for global steps, src for Groovy classes and resources for templates, registered under Global Pipeline Libraries and loaded with @Library pinned to a version tag. I move common logic there so each app Jenkinsfile becomes a few lines calling a standard pipeline, move long shell logic into scripts, and replace duplicated stages with loops or matrix - this avoids the method-too-large limit and lets us fix the pipeline once for all services.*

Q49. What rollback strategies do you use, and how do you version artifacts?

Also asked as: How would you make a deployment rollback-safe? Your actual rollout + rollback strategy, not buzzwords.

> **In simple words:** If every box leaving the factory has a unique label, going back to "yesterday's good box" is easy. If every box is just called "latest", you cannot tell which one was good.

Detailed answer

Artifact versioning:

- Semantic version + git SHA: `1.4.0-a1b2c3d`, or build number + SHA. Never deploy `latest` to prod.
- Enable **ECR tag immutability** so a tag cannot be overwritten.
- Record which version is in which environment (GitOps repo, Helm release history, deployment dashboard).

Rollback strategies:

- **Redeploy previous version:** `helm rollback`, `kubectl rollout undo`, or revert the GitOps commit.
- **Blue-Green:** switch traffic back to the old environment.
- **Canary:** set new version weight to 0.
- **Feature flags:** turn the feature off without redeploying.
- **Automatic:** `helm --atomic`, Argo Rollouts analysis, CodeDeploy auto-rollback on CloudWatch alarm.
- **Database:** use backward-compatible (expand/contract) migrations so old code still works with the new schema - otherwise app rollback is not safe.

Rollback-safe checklist (the actual rollout + rollback strategy, not buzzwords):

1. **Immutable, versioned artifacts:** deploy by image **digest** (`@sha256:...`) or an immutable tag; the same artifact is promoted through all environments.
2. **Keep previous versions ready:** `revisionHistoryLimit` keeps the last N ReplicaSets; Helm keeps release history (`--history-max`); old images are not deleted by ECR lifecycle rules too early.
3. **Backward-compatible DB migrations (expand/contract):** the previous app version must still work against the new schema.
4. **Feature flags:** risky behaviour can be switched off without a redeploy.
5. **Config versioned together with code:** Helm values, ConfigMaps and flags live in Git with the release, so a rollback restores the matching config too.
6. **Progressive rollout with automated analysis and abort:** canary steps checked against error rate and latency (Argo Rollouts, CodeDeploy alarms), aborting automatically.
7. **One-command rollback, practised regularly:** `helm rollback`, `kubectl rollout undo`, or `git revert` in GitOps - rehearsed in game days, and documented in the runbook.
8. **No manual hotfixes that skip Git:** a `kubectl edit` in prod is lost on the next sync or rollback; every fix goes through the pipeline.

Example:

```
spec:
  revisionHistoryLimit: 10                 # keep last 10 ReplicaSets for rollout undo
  template:
    spec:
      containers:
      - name: orders-api
        image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-api@sha256:4f1c...

helm upgrade --install orders-api charts/orders-api --history-max 10 --atomic
helm history orders-api -n prod
helm rollback orders-api 41 -n prod --wait

kubectl rollout history deploy/orders-api -n prod
kubectl rollout undo deploy/orders-api -n prod --to-revision=7

aws ecr put-image-tag-mutability --repository-name orders-api \
  --image-tag-mutability IMMUTABLE
```

> ***What to say to the interviewer:*** *Every artifact gets an immutable version tag with the git SHA, with ECR tag immutability on, so we always know exactly what is running. Rollback is a Helm rollback or*

a git revert in the GitOps repo, Blue-Green switch-back or canary weight to zero, and we use feature flags and backward-compatible database migrations so rolling back the app is always safe. To make it truly rollback-safe we deploy by digest, keep the last N releases, version config with code, use canary with automatic abort, practise the one-command rollback regularly, and never hotfix production outside Git.

Q50. What are secure pipeline (DevSecOps) practices?

In simple words: The pipeline has the keys to production, so it must be guarded like the vault of a bank - checks at every door, no spare keys lying around, and cameras recording everything.

Detailed answer

- **Shift-left scanning:** SAST (SonarQube/Semgrep), dependency/SCA (OWASP Dependency-Check, Snyk), secret scanning (gitleaks), IaC scanning (Checkov/tfsec), container scanning (Trivy). Fail the build on critical findings.
- **No static secrets:** IAM roles/OIDC for cloud, Vault/Secrets Manager at runtime, masked credentials.
- **Least privilege:** deploy role scoped to one account/namespace; separate roles for dev and prod.
- **Protect the pipeline itself:** branch protection, required reviews, signed commits; pipeline definition changes reviewed; restrict who can approve prod.
- **Ephemeral isolated agents:** fresh pod per build; no untrusted PR code on agents that hold prod credentials.
- **Supply chain:** pin base image digests and action versions (by SHA), generate SBOM, sign images (cosign) and verify at deploy.
- **Audit:** keep build logs, who approved what, and deploy history.

Example:

```
stage('Security Scans') {
  parallel {
    stage('Secrets') { steps { sh 'gitleaks detect --source . --exit-code 1' } }
    stage('Deps')    { steps { sh 'trivy fs --exit-code 1 --severity CRITICAL .' } }
    stage('IaC')     { steps { sh 'checkov -d infra/ --quiet' } }
  }
}
stage('Sign image') { steps { sh 'cosign sign --key awskms:///alias/cosign $IMAGE@$DIGEST' } }
```

What to say to the interviewer: *I build security into every stage: secret, dependency, SAST, IaC and image scanning with builds failing on critical issues. Pipelines use short-lived IAM or OIDC credentials instead of stored keys, run on ephemeral agents with least-privilege deploy roles, have protected branches and restricted prod approvals, and images are pinned, scanned and signed before deploy.*

Q51. How do you secure secrets across CI/CD pipelines?

Also asked as: You are asked to secure secrets across CI/CD pipelines. What is your preferred approach?

In simple words: The best secret is one that does not exist - use temporary badges instead of permanent keys. For the secrets you must have, keep them in one central safe and let the pipeline

borrow them only for a moment.

Detailed answer

My preferred approach, in order:

1. **Eliminate long-lived cloud keys:** GitHub Actions/GitLab/Azure DevOps -> OIDC federation to AWS/Azure/GCP; Jenkins on AWS -> IAM roles/IRSA.
2. **Central secrets manager:** HashiCorp Vault or AWS Secrets Manager / Azure Key Vault; pipelines fetch at runtime with a short-lived identity.
3. **CI-native secret store** only for what remains (Jenkins Credentials, GitHub encrypted secrets, Azure DevOps variable groups linked to Key Vault), scoped per environment with approvals on prod environments.
4. **Hygiene:** mask in logs, never echo, never bake into Docker images or artifacts, `no_log` in Ansible.
5. **Detect leaks:** gitleaks pre-commit + pipeline scan, GitHub secret scanning.
6. **Rotate** regularly and immediately on any exposure.

Example:

```
# GitHub Actions -> AWS with OIDC (no stored AWS keys)
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production          # requires reviewers
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
          aws-region: ap-south-1
      - run: |
          DB_PASS=$(aws secretsmanager get-secret-value --secret-id prod/db \
            --query SecretString --output text)
          echo "::add-mask::$DB_PASS"
```

What to say to the interviewer: *My first step is removing static keys by using OIDC federation or IAM roles, so pipelines get short-lived credentials. Remaining secrets live in a central manager like Vault or Secrets Manager and are fetched at runtime, scoped per environment, masked in logs, never baked into images, and leak scanning plus regular rotation cover the rest.*

Q52. How do you ensure secure and dynamic secret rotation in Azure DevOps pipelines?

In simple words: Instead of copying the password into the pipeline (and forgetting to update it), point the pipeline at the safe (Key Vault). When the password in the safe changes, the pipeline automatically uses the new one.

Detailed answer

- Store secrets in **Azure Key Vault**, not as plain pipeline variables.

- Link a **Variable Group to Key Vault** or use the `AzureKeyVault@2` task - secrets are fetched at each run, so rotation in Key Vault needs no pipeline change.
- Connect with a **Workload Identity Federation service connection** (OIDC) - no client secret to rotate for the pipeline itself.
- Enable **Key Vault rotation policies** (for keys) or Event Grid `SecretNearExpiry` -> Azure Function to rotate app secrets (e.g. regenerate storage keys, update SQL password).
- Apps read secrets at runtime (Managed Identity, Key Vault references, CSI Secret Store driver on AKS) so they pick up new versions.
- Protect with RBAC, environment approvals and checks, and audit via Key Vault diagnostic logs.

Example:

```
variables:
- group: prod-kv-secrets          # variable group linked to Key Vault

steps:
- task: AzureKeyVault@2
  inputs:
    azureSubscription: 'sc-prod-oidc'   # workload identity federation
    KeyVaultName: 'kv-payments-prod'
    SecretsFilter: 'DbPassword,ApiKey'
    RunAsPreJob: true
- script: ./deploy.sh
  env:
    DB_PASSWORD: $(DbPassword)          # masked in logs
```

> ***What to say to the interviewer:*** *I keep secrets in Azure Key Vault and fetch them at runtime via a Key Vault-linked variable group or the AzureKeyVault task, using a workload identity federation service connection so the pipeline itself has no secret. Rotation happens in Key Vault through rotation policies or an Event Grid-triggered function, and because pipelines and apps read the latest version at runtime, no pipeline changes are needed.*

Q53. Azure DevOps: How do you secure access to artifacts and feeds (Azure Artifacts)?

> **In simple words:** A feed is a company warehouse of packages. You decide who can only take things out (Reader), who can put things in (Contributor), and who manages the warehouse (Owner). Outside goods only come in through one checked gate (upstream sources).

Detailed answer

- **Feed permissions (least privilege):** Reader (download), Collaborator (download + save packages from upstream), Contributor (publish), Owner (manage feed and permissions). Developers usually get Reader; only CI publishes.
- **Project-scoped vs organization-scoped feeds:** project-scoped feeds inherit the project's visibility and permissions and are the recommended default; org-scoped feeds are visible across the organization, so use them only for truly shared packages.
- **Pipeline identity:** pipelines authenticate as the **Project Build Service** (or Project Collection Build Service) identity - grant it Contributor only on the feeds it publishes to, and Reader elsewhere. Use `NuGetAuthenticate`/`npmAuthenticate` tasks instead of stored tokens.

Limit job authorization scope to the current project.

- **Upstream sources:** proxy npmjs, PyPI, NuGet or Maven Central through the feed so every external package is cached and auditable, and not pulled directly by developers; this also reduces dependency-confusion risk because internal package names resolve from your feed first.
- **Views:** `@local`, `@prerelease`, `@release` - promote a package to `@release` only after it passes testing; consumers and prod builds only read the `@release` view.
- **PATs:** if a PAT is needed (e.g. a developer laptop), scope it to Packaging (Read) only with a short expiry; admins can restrict full-scope PATs by policy.
- **Service connections to external systems:** use **workload identity federation** (OIDC) instead of secrets, with pipeline permissions and approvals on the connection.
- **Retention policies:** keep the last N versions per package and delete old ones (packages promoted to views are kept) - controls cost and removes stale vulnerable versions.
- **Audit:** Azure DevOps auditing streamed to Log Analytics.

Example:

```
steps:
- task: npmAuthenticate@0              # uses the build service identity, no PAT
  inputs: { workingFile: .npmrc }
- script: npm ci && npm publish

# .npmrc
registry=https://pkgs.dev.azure.com/org/project/_packaging/internal-feed/npm/registry/
always-auth=true

# Consumers of tested packages read only the release view
registry=https://pkgs.dev.azure.com/org/project/_packaging/internal-feed@release/npm/registry/
```

> ***What to say to the interviewer:*** *I use project-scoped feeds with least-privilege roles - developers read, only the pipeline's build service identity can publish - and pipelines authenticate with the npm or NuGet authenticate tasks rather than PATs. External packages come only through upstream sources, tested versions are promoted to the release view that prod builds consume, any PATs are packaging-read-only and short-lived, service connections use workload identity federation, and retention policies and audit logs keep it clean.*

Q54. How do you manage deployments and rollbacks for 100+ microservices on EKS/ECS?

In simple words: With 100 services you cannot hand-deploy each one. You write the desired state of every service in one Git notebook, and a robot (ArgoCD) keeps the cluster matching that notebook - rolling back is just undoing a page in the notebook.

Detailed answer

- **Standardize:** one shared pipeline template / Jenkins shared library and one base Helm chart for all services; each service only provides values.
- **GitOps with ArgoCD:** a config repo holds manifests per service per environment. ArgoCD `ApplicationSet` generates Applications for all services automatically. CI only builds images and bumps tags.

- **Independent deploys:** each service deploys on its own; no big-bang releases.
- **Progressive delivery:** Argo Rollouts canary with automated analysis on Prometheus metrics, auto-abort on errors. On ECS: CodeDeploy blue/green with CloudWatch alarm rollback, or ECS deployment circuit breaker.
- **Rollback:** `git revert` the tag bump -> ArgoCD syncs the old version; or `argo rollouts abort/undo`.
- **Visibility:** dashboard of versions per environment, deploy events annotated on Grafana, alerting per service.
- **Guardrails:** sync windows, PodDisruptionBudgets, per-service SLOs.

Example:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: prod-services }
spec:
  generators:
  - git:
      repoURL: https://github.com/org/gitops.git
      revision: main
      directories: [ { path: "apps/*/envs/prod" } ]
  template:
    metadata: { name: '{{path[1]}}-prod' }
    spec:
      project: prod
      source: { repoURL: https://github.com/org/gitops.git, targetRevision: main,
                path: '{{path}}' }
      destination: { server: https://kubernetes.default.svc, namespace: '{{path[1]}}' }
      syncPolicy: { automated: { prune: true, selfHeal: true } }

# Rollback one service
git revert <commit-that-bumped-orders-tag> && git push
```

> ***What to say to the interviewer:*** *At that scale I standardize on one pipeline template and one base Helm chart, and use GitOps with ArgoCD ApplicationSets so every service's desired version lives in Git. CI builds and bumps the tag, Argo Rollouts does canary with Prometheus-based automatic abort, and rollback is a git revert that ArgoCD syncs - on ECS the equivalent is CodeDeploy blue/green or the deployment circuit breaker.*

Q55. How do you ensure zero-downtime deployments? Share a real example.

> **In simple words:** Like changing the tyres of a bus while it keeps driving - you only remove an old tyre after the new one is firmly on and holding weight.

Detailed answer

Strategy-level ingredients:

- **Deployment strategy:** rolling (with maxUnavailable 0), Blue-Green, or Canary.
- **Health checks:** readiness probes / target group health checks so traffic only goes to ready instances.
- **Graceful shutdown:** connection draining (deregistration delay, `preStop` sleep, handle SIGTERM).
- **Enough capacity:** min replicas >= 2, PodDisruptionBudget, surge capacity.

- **Backward-compatible changes:** API versioning and expand/contract DB migrations, so old and new versions can run side by side.
- **Automated verification and rollback:** smoke tests, metrics-based canary analysis, `helm --atomic`.

Real example (sample - adapt to your project): an API on EKS behind an ALB. We set `maxUnavailable: 0, maxSurge: 25%`, readiness probe on `/health`, `preStop` sleep 15s, and ALB deregistration delay 30s. A DB column rename was done in 3 releases (add new column, dual-write, drop old). The pipeline ran `helm upgrade --atomic`; one release failed readiness and Helm rolled back automatically with zero user errors.

Example:

```
strategy:
  type: RollingUpdate
  rollingUpdate: { maxUnavailable: 0, maxSurge: 25% }
...
lifecycle:
  preStop:
    exec: { command: ["sh", "-c", "sleep 15"] }
readinessProbe:
  httpGet: { path: /health, port: 8080 }

helm upgrade --install api charts/api --set image.tag=$TAG --atomic --wait --timeout 10m
```

> ***What to say to the interviewer:*** *Zero downtime needs a rolling, blue-green or canary strategy, readiness checks so only healthy instances get traffic, graceful connection draining, enough replicas, and backward-compatible database changes. In practice we deploy with maxUnavailable zero, readiness probes, a preStop delay and helm atomic, so a bad release fails its checks and rolls back automatically without users noticing.*

Q56. How would you optimize a slow CI/CD pipeline (e.g. 40 minutes for a small change)?

Also asked as: How would you optimize a CI/CD pipeline? Pipeline optimization for 70%+ faster deployments. How does caching work in pipelines? / CI builds 40 Docker images and takes 18 minutes. How do you optimize it?

> **In simple words:** If a car wash takes 40 minutes, time each station first. Then stop re-washing clean parts (caching), run stations side by side (parallel), and skip stations that do not apply to this car (change detection).

Detailed answer

Step-by-step:

1. **Measure first:** stage timing view / Blue Ocean / GitHub Actions timing - find the top 2-3 slow stages.
2. **Cache:** dependency caches (`~/.m2`, `node_modules`, pip), Docker layer cache (BuildKit `--cache-from`, registry cache), Terraform plugin cache.
3. **Parallelize:** unit tests, lint, scans in `parallel`; split test suites across agents (test sharding).
4. **Build only what changed:** in monorepos run only affected services (path filters, `git diff`).
5. **Optimize Dockerfile:** copy dependency files first, multi-stage, slim base images, `.dockerignore`.

6. **Right test at right stage:** fast unit tests on PR; heavy e2e/perf tests nightly or only before prod.
7. **Faster agents:** warm pools of agents, bigger instances for build stage, avoid queue waiting; ephemeral pods with pre-pulled images.
8. **Remove waste:** redundant checkouts (shallow clone `depth 1`), duplicate builds, unnecessary sleeps; fail fast.

How pipeline caching works:

- A cache is saved under a **key**, usually built from the OS plus a hash of the lock file (`package-lock.json`, `pom.xml`, `requirements.txt`). Same lock file -> same key.
- **Cache hit:** the key matches exactly, the folder is restored and the download/install step is skipped or near-instant.
- **Cache miss:** no match, so the step runs fully and the cache is saved at the end for next time.
- **Restore keys** are fallback prefixes (e.g. `npm | Linux`): on a miss, the most recent partially matching cache is restored, so only the changed dependencies download.
- Tools: GitHub `actions/cache` (or `setup-node` with `cache: npm`), Azure DevOps `Cache@2` task, Jenkins with a persistent volume or a cache plugin, GitLab `cache:key:files`.
- **Docker layer cache:** each Dockerfile instruction is a layer; if the instruction and its inputs did not change, the layer is reused. On ephemeral agents, store it in a registry (`--cache-from/--cache-to type=registry`) or `type=gha`.
- Never cache build outputs that should be fresh, and never cache secrets.

9. **Deploy faster:** only changed services, proper readiness probe tuning instead of fixed waits.

Typical result: 40 min -> ~10-12 min (70%+ faster).

Example:

```
# GitHub Actions dependency cache
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ runner.os }}-${{ hashFiles('package-lock.json') }}
    restore-keys: npm-${{ runner.os }}-

# Azure DevOps Cache@2
- task: Cache@2
  inputs:
    key: 'npm | "$(Agent.OS)" | package-lock.json'
    restoreKeys: 'npm | "$(Agent.OS)"'
    path: $(npm_config_cache)

# Docker layer cache via registry
docker buildx build --cache-from type=registry,ref=$ECR/app:cache \
  --cache-to type=registry,ref=$ECR/app:cache,mode=max -t $ECR/app:$TAG --push .

stage('Checks') {
  parallel {
    stage('Unit')  { steps { sh 'mvn -B -o test' } }
    stage('Lint')  { steps { sh 'npm run lint' } }
    stage('Scan')  { steps { sh 'trivy fs --exit-code 1 .' } }
  }
}
checkout([$class: 'GitSCM', extensions: [[$class: 'CloneOption', shallow: true, depth: 1]]])
```

When one pipeline builds many images (for example 40 images in 18 minutes):

1. **Build only what changed:** path filters per service (`dorny/paths-filter`, Jenkins `changeset`, or a script diffing `git diff --name-only origin/main...HEAD`). Usually only 2-3 of the 40 images need rebuilding.
2. **Parallelize:** a matrix job (GitHub Actions `strategy.matrix`, Jenkins `parallel`), one image per runner, instead of 40 sequential builds.
3. **Remote layer cache:** BuildKit with `--cache-from/--cache-to type=registry` (or `type=gha`), so a fresh runner reuses layers from the last build.
4. **Cache mounts** for package managers: `RUN --mount=type=cache,target=/root/.npm npm ci`.
5. **Shared base image** with the heavy OS and dependencies, built once a week, so service images only add their own code.
6. **Order Dockerfile layers** so dependency files are copied before source code, and use a good `.dockerignore`.
7. Bigger or ARM runners, and avoid `--no-cache` in CI.

```
docker buildx build   --cache-from type=registry,ref=$REG/meter-api:buildcache   --cache-to   type=r
    egistry,ref=$REG/meter-api:buildcache,mode=max   -t $REG/meter-api:$GIT_SHA --push services/mete
    r-api
```

> ***What to say to the interviewer:*** *I start by measuring stage durations to find the bottleneck, then add dependency and Docker layer caching, run tests and scans in parallel, build only changed services, optimize the Dockerfile, and move heavy e2e tests to later stages or nightly runs. Using warm agents and shallow clones as well, this typically cuts a 40-minute pipeline to around 10 minutes, 70 percent or more faster.*

Q57. How would you implement blue/green deployment with rollback on Azure using Terraform and pipelines?

> **In simple words:** Terraform builds both stages (Blue and Green) once. The pipeline puts the new show on the idle stage, tests it, then flips the spotlight. If anything looks wrong, flip it back.

Detailed answer

Option A - Azure App Service deployment slots (simplest):

1. Terraform creates the App Service with a `staging` slot.
2. Pipeline deploys the new build to the `staging` slot.
3. Run smoke tests against the staging slot URL.
4. **Swap** staging <-> production (warm-up happens before swap, no cold start).
5. Rollback = swap again (old version is still in the staging slot).

Option B - AKS: Terraform manages infra; two Deployments (`app-blue`, `app-green`); pipeline switches Service selector or Application Gateway / Front Door backend weights.

Pipeline design (Azure DevOps multi-stage YAML): Build -> Deploy Green -> Test -> Approval (environment check) -> Swap -> Monitor -> auto swap back if health/alerts fail.

Example:

```
# Terraform
resource "azurerm_linux_web_app_slot" "staging" {
  name           = "staging"
  app_service_id = azurerm_linux_web_app.app.id
  site_config {}
}

# azure-pipelines.yml (deploy stage)
- task: AzureWebApp@1
  inputs: { azureSubscription: 'sc-prod', appName: 'orders-web',
            deployToSlotOrASE: true, resourceGroupName: 'rg-prod',
            slotName: 'staging', package: '$(Pipeline.Workspace)/drop/*.zip' }
- script: curl -fsS https://orders-web-staging.azurewebsites.net/health
- task: AzureAppServiceManage@0
  inputs: { azureSubscription: 'sc-prod', Action: 'Swap Slots', WebAppName: 'orders-web',
            ResourceGroupName: 'rg-prod', SourceSlot: 'staging' }

# Rollback = same swap task again
```

What to say to the interviewer: *I use Terraform to create the environment including a staging deployment slot, or two blue and green deployments on AKS. The pipeline deploys to the idle slot, runs smoke tests, waits for the environment approval, and swaps; if health checks or alerts fail after the swap, a rollback stage swaps back, which is instant because the old version is still warm.*

Q58. Compare deployment strategies: in-place, rolling, immutable, traffic splitting (canary) and blue/green.

In simple words: Replacing the seats in a cinema: in-place closes the whole cinema; rolling fixes one row at a time; immutable builds new rows next to the old ones; traffic splitting lets a few guests try the new seats first; blue/green builds a second cinema and moves everyone over at once.

Detailed answer

This is the umbrella comparison; Blue-Green and Canary are explained in detail in Q12.

- **In-place (all at once):** stop the app on existing servers, deploy new version, start. Cheapest, but downtime and a failure hits everyone.
- **Rolling:** update a batch of instances at a time (e.g. 25%); remaining batches keep serving. A bad version affects only the current batch until you stop.
- **Immutable:** launch a brand-new set of instances with the new version (e.g. a temporary Auto Scaling group), shift to them once healthy, terminate the old ones. Never modifies running servers.
- **Traffic splitting (canary):** new instances receive a small % of traffic first, increased if metrics are healthy.
- **Blue/green:** full parallel environment, switch all traffic at once, switch back to roll back.

Strategy	Impact if failed	Deploy time	Zero downtime	DNS change	Rollback	Code deployed to
In-place (all at once)	Downtime	1x	No	No	Redeploy	Existing instances
Rolling	Single batch impacted	2x	Yes	No	Redeploy	Existing instances
Immutable	Minimal	4x	Yes	No	Redeploy	New instances

Strategy	Impact if failed	Deploy time	Zero downtime	DNS change	Rollback	Code deployed to
Traffic splitting (canary)	Minimal	4x	Yes	No	Re-route traffic and terminate new instances	New instances
Blue/green	Minimal	4x	Yes	Yes (see note)	Switch back	New instances

Accuracy note: the "DNS change = Yes" for blue/green comes from the classic **Elastic Beanstalk "Swap environment URLs" (CNAME swap)** style. If you switch with an **ALB listener / target-group** change, weighted target groups, or a **Kubernetes Service selector**, there is **no DNS change** - and switching is faster because it avoids DNS caching/TTL delays. "Deploy time" is relative (1x = fastest), because immutable, canary and blue/green all launch new capacity.

Which one to choose:

- **In-place:** dev/test environments or tools where a short outage is acceptable and cost matters most.
- **Rolling:** default for most stateless services (Kubernetes RollingUpdate), when brief mixed versions are fine.
- **Immutable:** when you want a clean, known-good fleet every time and fast safe rollback without patching servers (AMI-based EC2 fleets).
- **Traffic splitting (canary):** high-traffic user-facing services with good metrics, where you want the smallest blast radius.
- **Blue/green:** big or risky releases needing instant full rollback and pre-switch testing, when you can afford double capacity briefly.

Example:

```
# Elastic Beanstalk deployment policy (.ebextensions/deploy.config)
option_settings:
  aws:elasticbeanstalk:command:
    DeploymentPolicy: Immutable        # AllAtOnce | Rolling | RollingWithAdditionalBatch
                                       # | Immutable | TrafficSplitting
  aws:elasticbeanstalk:trafficsplitting:
    NewVersionPercent: "10"
    EvaluationTime: "5"

# Classic EB blue/green = CNAME swap (this is the DNS change)
aws elasticbeanstalk swap-environment-cnames \
  --source-environment-name orders-blue --destination-environment-name orders-green
```

> ***What to say to the interviewer:*** *In-place is cheapest but causes downtime, rolling updates batches so only one batch is at risk, immutable launches a fresh fleet and swaps to it, traffic splitting sends a small percentage to new instances first, and blue/green runs a full parallel environment and switches all traffic. Blue/green only needs a DNS change in the classic Elastic Beanstalk CNAME-swap style - with an ALB target-group switch or a Kubernetes Service selector there is no DNS change. I use rolling as the default, canary for high-traffic services, and blue/green or immutable when I need instant, clean rollback.*

Q59. How do you implement environment-specific CI/CD pipelines for 10+ microservices, and how do you scale and manage them?

In simple words: Do not write 10 services x 3 environments = 30 separate recipes. Write one master recipe with blanks to fill in ("which service?", "which environment?"), and keep each environment's settings on its own small card.

Detailed answer

This question is about pipeline design; the deploy/rollback mechanics at 100+ service scale are covered in Q54.

1. **One pipeline definition, many services (templating):** Jenkins **shared library** step like `microservicePipeline(app: 'orders')` so each repo Jenkinsfile is 2-3 lines; GitHub Actions **reusable workflows** (`workflow_call`) in a central repo; GitLab `include:` of central templates. Change the template once -> all services get the fix. Version the template (tags) so teams upgrade safely.
2. **Environments are parameters, not copies:** the same pipeline takes `ENV=dev|staging|prod`; differences live in config (Helm values per env, Kustomize overlays, per-env secrets from Secrets Manager/Key Vault), never in separate pipeline code.
3. **Build once, promote:** CI builds and scans the image once; CD promotes the same tag through environments.
4. **GitOps for deployment:** an ArgoCD **ApplicationSet** generates one Application per service per environment from the config repo layout; per-env overlays hold replica counts, resources and URLs.
5. **Promotion = PR or tag bump:** automatic to dev on merge, PR to staging/prod folders with required reviewers, or an automated image-tag bump (Argo CD Image Updater, a pipeline step).
6. **Guardrails per environment:** GitHub **environment protection rules** (required reviewers, wait timers, branch restrictions), environment-scoped secrets, separate AWS accounts/roles per env, prod sync windows.
7. **Scale the runners:** ephemeral agents (Kubernetes pod agents, ARC runners), path filters so only changed services build.

Example:

```
# Central repo: .github/workflows/service-pipeline.yml (reusable)
on:
  workflow_call:
    inputs:
      service:     { type: string, required: true }
      environment: { type: string, required: true }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}      # protection rules apply per env
    steps:
      - uses: actions/checkout@v4
      - run: |
          helm upgrade --install ${{ inputs.service }} charts/service \
            -f envs/${{ inputs.environment }}/values.yaml --atomic

# Each service repo: a few lines
jobs:
  prod:
    uses: org/ci-templates/.github/workflows/service-pipeline.yml@v3
    with: { service: orders, environment: prod }
    secrets: inherit

# GitOps layout
apps/orders/base/   apps/orders/overlays/dev/   apps/orders/overlays/prod/
```

> ***What to say to the interviewer:*** *I keep one templated pipeline - a Jenkins shared library or a GitHub reusable workflow - that every service calls with its name and target environment, so environments are parameters and their differences live in per-env Helm values or Kustomize overlays and secrets. CI builds the image once, and deployment is GitOps with ArgoCD ApplicationSets, where promotion is a PR or image-tag bump. Environment protection rules, per-env accounts and ephemeral runners keep it safe and scalable.*

Q60. How would you design a GitOps pipeline for a microservices application? Which tools, what flow, and how do you roll back?

Also asked as: How does Argo CD work in CI/CD, and why use it? / How do you design GitOps for multiple teams with independent releases?

> **In simple words:** Git becomes the single "master plan" of what should run in the cluster. Nobody touches the cluster directly - you change the plan through a reviewed PR, and a robot inside the cluster (Argo CD) keeps making reality match the plan. To undo, you undo the plan.

Detailed answer

For deploying and rolling back 100+ services at scale see Q54; this answer focuses on the GitOps design itself.

How Argo CD works and why use it: Argo CD is a Kubernetes controller that watches Git repos. For each Application it renders the manifests (Helm, Kustomize or plain YAML), compares them with the live cluster, and marks the app Synced/OutOfSync and Healthy/Degraded. With auto-sync it applies changes itself. In CI/CD it takes over the CD half: CI only builds and pushes the image and updates Git; Argo CD deploys. Why: no cluster credentials in CI (pull model), Git is the audit trail, drift is visible and self-healed, rollback is a revert, and one UI shows every app across clusters.

Flow:

1. **App repo CI:** build, test, scan, then push the image with an **immutable tag or digest** (`1.4.0-a1b2c3d`, `@sha256:...`).
2. **Config repo update:** a CI bot opens a PR (or Argo CD Image Updater commits) that bumps the tag in the Helm values or Kustomize overlay for that environment. Dev can auto-merge; staging and prod need reviewers.
3. **Sync:** Argo CD (or Flux) running in the cluster detects the commit and applies it. **ApplicationSets** generate Applications for many services/envs; **sync waves** order things (CRDs and config first, then apps); **health checks** decide if the sync is Healthy.
4. **Progressive delivery:** Argo Rollouts canary/blue-green with analysis on Prometheus metrics, auto-abort on bad metrics.
5. **Rollback:** `git revert` the config commit - Git is the source of truth. Argo CD's "rollback to previous sync" works only if auto-sync is disabled; with auto-sync on, Argo re-applies whatever Git says, so revert in Git.

Also design in:

- **Drift self-heal:** `selfHeal: true` undoes manual `kubectl edit` changes; `prune: true` removes deleted resources.
- **Security:** Argo CD **SSO + RBAC** (AppProjects restrict which repos/namespaces/clusters a team can deploy to); CI never needs cluster credentials (pull model).
- **Secrets:** never plain secrets in Git - use **External Secrets Operator** (pulls from AWS Secrets Manager/Vault) or **SOPS**/Sealed Secrets (encrypted in Git).
- **Notifications:** Argo CD Notifications to Teams/Slack on sync failure or degraded health.

```
Dev push -> [App repo CI: build/test/scan] -> push image (immutable tag) -> ECR
                                  |
                                  v
                PR: bump tag in [Config repo: envs/dev|staging|prod]
                                  |  (review + merge)
                                  v
             [Argo CD in cluster] --pull + sync--> Kubernetes
                                  |                     |
                       self-heal drift            Argo Rollouts canary
                                  |                     |
                rollback = git revert <---- analysis fails -> auto abort
```

Example:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: orders-prod
  namespace: argocd
spec:
  project: payments-team
  source:
    repoURL: https://github.com/org/gitops-config.git
    targetRevision: main
    path: apps/orders/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: orders
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions: [CreateNamespace=true]
    retry: { limit: 3, backoff: { duration: 10s, factor: 2 } }

# Rollback
git revert 3f2a1bc && git push     # Argo CD syncs the previous version
```

GitOps for many teams with independent releases:

- **App-of-Apps:** one root Argo CD Application points to a folder of child Application manifests, so adding a service means adding one file.
- **ApplicationSets:** generate Applications automatically. A git directory generator creates one app per folder `teams/*/services/*`; a cluster generator deploys the same app to every cluster.
- **AppProject per team:** restricts which repos a team can deploy from, which namespaces or clusters it can deploy to, and which resource kinds are allowed, with RBAC mapped to SSO groups. One team cannot touch another team's namespace.
- **Repo layout:** a platform or infra repo owned by the platform team (ingress, cert-manager, monitoring), plus per-team config repos or folders. Each team merges and releases on its own schedule.
- **Guardrails:** sync windows (no prod syncs during business-critical hours), required PR reviews on prod folders, and progressive delivery with Argo Rollouts.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: team-services, namespace: argocd }
spec:
  generators:
    - git:
        repoURL: https://github.com/my-org/deploy-config.git
        revision: main
        directories: [{ path: "teams/*/services/*" }]
  template:
    metadata: { name: "{{path[1]}}-{{path.basename}}" }
    spec:
      project: "{{path[1]}}"              # AppProject per team
      source:
        repoURL: https://github.com/my-org/deploy-config.git
        targetRevision: main
        path: "{{path}}"
      destination: { server: https://kubernetes.default.svc, namespace: "{{path[1]}}" }
      syncPolicy: { automated: { prune: true, selfHeal: true } }
```

What to say to the interviewer: *The app repo CI builds, tests and scans, then pushes an image with an immutable tag, and a bot or Argo CD Image Updater bumps that tag in the config repo per environment through a PR. Argo CD in the cluster pulls and syncs using ApplicationSets, sync waves and health checks, with Argo Rollouts for canary analysis. Rollback is a git revert because Git is the source of truth, and the design includes self-heal for drift, SSO and RBAC through AppProjects, and secrets through External Secrets or SOPS rather than plain secrets in Git.*

Q61. How do you handle databases in blue-green deployments? With critical data, how do you keep the database consistent and avoid issues during switchover?

In simple words: Blue and green are two kitchens, but they usually share ONE fridge (the database). So any change to the fridge must work for both kitchens at the same time - you add a new shelf before anyone uses it, and only remove the old shelf when nobody needs it anymore.

Detailed answer

What to do when a migration step fails mid-deploy is covered in Q75; this answer is about designing the database side of blue-green.

Core idea: **blue and green normally share one database.** Only the application layer is duplicated, so the schema must support both versions during the switch and during a rollback.

1. **Backward-compatible schema changes with expand -> migrate -> contract**, spread across releases:

- Release 1 (expand): add the new column/table, nullable or with a default; old code ignores it.
- Release 2 (migrate): new code writes both old and new, backfill existing rows in batches, then read from new.
- Release 3 (contract): once no running version uses the old column, drop it.

2. **Never make destructive changes (drop/rename column, change type) in the same release** as the code that needs them - blue would break the moment the migration runs, and rollback would be impossible.

3. **Feature flags:** ship the code dark, turn the new behaviour on after the switch, and turn it off instead of rolling back.
4. **Avoid conflicting dual writers:** blue and green must not write the same data in incompatible formats (e.g. green writes JSON v2 that blue cannot parse). Keep formats versioned and readable by both.
5. **Switchover hygiene:** drain connections, let long-running transactions and background jobs finish, and make sure only one colour runs schedulers/queue consumers that must not run twice.
6. **Rollback implication:** data written by green must still be readable by blue. If it is not, switching back is not a real rollback.
7. **If you must use separate databases:** keep them in sync with replication/CDC (AWS DMS, PostgreSQL logical replication), plan a short **write freeze**, verify replication lag = 0 and row counts/checksums, then cut over; decide in advance how to handle writes if you roll back (reverse replication).
8. **RDS Blue/Green Deployments** for engine upgrades or parameter changes: AWS creates a green copy kept in sync by replication and performs a managed switchover (typically about a minute), renaming endpoints so apps do not change connection strings.

Example:

```
-- Release 1: expand (safe for blue and green)
ALTER TABLE customers ADD COLUMN email_normalized VARCHAR(255);

-- Release 2: backfill in batches while code dual-writes
UPDATE customers SET email_normalized = lower(email)
WHERE email_normalized IS NULL AND id BETWEEN 1 AND 10000;

-- Release 3: contract (only after blue with old code is gone)
ALTER TABLE customers DROP COLUMN email_old;

# RDS managed blue/green for an engine upgrade
aws rds create-blue-green-deployment --blue-green-deployment-name pg16-upgrade \
  --source arn:aws:rds:ap-south-1:123456789012:db:orders-db --target-engine-version 16.4
aws rds switchover-blue-green-deployment \
  --blue-green-deployment-identifier bgd-abc123 --switchover-timeout 300
```

What to say to the interviewer: *In blue-green both colours usually share one database, so every schema change must be backward compatible - I use expand, migrate, contract across separate releases and never drop or rename columns in the same release as the code that needs it. I use feature flags, avoid incompatible dual writes, drain connections and long transactions on switchover, and make sure data written by green is readable by blue so rollback stays safe. If separate databases are unavoidable I use replication or CDC like DMS with a write freeze and lag check at cutover, and for engine upgrades I use RDS Blue/Green Deployments with managed switchover.*

Scenario-Based

Q62. The Jenkins controller goes down while builds are running on agents. What happens and what do you do?

In simple words: If the manager faints, the workers keep their tools in hand for a little while, but nobody is recording their work or giving next instructions. When the manager wakes up, Pipeline jobs can often pick up where they left off; Freestyle jobs are simply lost.

Detailed answer

What happens:

- Agents lose their connection (remoting channel) to the controller.
- **Freestyle jobs** running on agents fail/abort - their state lived on the controller.
- **Declarative/Scripted Pipelines are durable**: state is checkpointed to disk on the controller. After restart, pipelines **resume**; an `sh` step already running on an agent can continue if the agent and workspace survive (durable task). Some steps may still fail and need a rerun.
- No new builds are scheduled; webhooks received during downtime are missed (need re-scan / redeliver).

What I do:

1. Restore the controller: check the service/pod (`systemctl status jenkins` or `kubectl get pod`), logs, disk, memory (OOM is common), then restart.
2. Check which builds resumed vs failed; rerun failed ones; redeliver missed GitHub webhooks or trigger a branch re-scan.
3. Make sure a half-finished deploy did not leave prod inconsistent (check Helm/rollout status).

Prevent:

- Run the controller on Kubernetes/EC2 with persistent `JENKINS_HOME` (EBS/EFS) so it auto-restarts; back up `JENKINS_HOME` (thinBackup / volume snapshots).
- Configuration as Code (JCasC) + jobs in Git so a new controller can be rebuilt fast.
- Monitor the controller (Prometheus plugin, heap, disk) and set pipeline `durabilityHint` appropriately.
- Deploy steps should be idempotent (`helm upgrade --install --atomic`) so a rerun is safe.

Example:

```
kubectl -n jenkins get pods
kubectl -n jenkins logs jenkins-0 --previous | tail -50   # why did it die? OOMKilled?
kubectl -n jenkins describe pod jenkins-0 | grep -i -A3 "last state"

options { durabilityHint('MAX_SURVIVABILITY') }   // strongest resume guarantees
```

What to say to the interviewer: *Agents lose their connection; Freestyle jobs are lost, but Pipelines are durable and usually resume after the controller restarts, though some steps may need a rerun. I restore the controller, check resumed versus failed builds, rerun and redeliver missed webhooks, and verify no deployment was left half-done. To prevent it we run Jenkins with persistent storage and auto-restart, JCasC for fast rebuild, backups and monitoring.*

Q63. The Jenkins master (controller) is running out of disk space. What actions would you take?

Also asked as: If Jenkins storage is full, how would you troubleshoot it?

In simple words: Jenkins keeps every old receipt, log and leftover box forever unless told not to. First throw away the obvious junk to get breathing room, then set rules so the junk does not pile up again.

Detailed answer

Immediate (get space back safely):

1. Find what is big: `du` on `JENKINS_HOME` - usually `jobs/*/builds` (old build logs + artifacts), `workspace/`, `caches`, `logs`, or Docker images if builds run on the controller.
2. Delete old builds via Script Console or job config (not by randomly deleting folders while Jenkins runs).
3. Wipe old workspaces (`workspace/`), clean `/tmp`, rotate `jenkins.log`, `docker system prune` if Docker is on the controller.
4. If still critical, expand the EBS volume (online resize) to avoid an outage.

Long term:

- `buildDiscarder(logRotator(numToKeepStr: '20', artifactNumToKeepStr: '5'))` on every job (enforce via shared library).
- **Don't build on the controller**: set controller executors to 0; builds on ephemeral agents.
- Store artifacts in S3/Nexus/Artifactory, not in Jenkins (`archiveArtifacts` only for small reports).
- `cleanWs()` in `post { always }`.
- Disk usage alert at 80% (Prometheus node exporter / CloudWatch agent).

Example:

```
df -h /var/lib/jenkins
du -sh /var/lib/jenkins/* | sort -rh | head
du -sh /var/lib/jenkins/jobs/*/builds | sort -rh | head

// Script Console: keep only last 20 builds for every job
Jenkins.instance.getAllItems(Job).each { job ->
  job.builds.drop(20).each { it.delete() }
}

options { buildDiscarder(logRotator(numToKeepStr: '20', artifactNumToKeepStr: '5')) }
```

What to say to the interviewer: *I first find what is consuming space in JENKINS_HOME - usually old builds, artifacts and workspaces - and safely delete old builds through the Script Console, clean workspaces and logs, and expand the volume if it is critical. Long term I enforce build discarders on all jobs, run zero executors on the controller, store artifacts in S3 or Nexus, clean workspaces after each build, and alert on disk at 80 percent.*

Q64. Build artifacts are not getting uploaded to the artifact repository, or Jenkins jobs fail randomly at the artifact upload step. How do you investigate?

Also asked as: Jenkins jobs are randomly failing at the artifact upload step. What layers would you check?

In simple words: Uploading a parcel can fail because of the sender (the build), the road (network), the gatekeeper (credentials/permissions), or the warehouse (the repository is full or broken). Check each layer in order.

Detailed answer

Layers to check:

1. **Build layer:** did the artifact actually get produced? Wrong path/pattern, artifact built in another stage's workspace or another agent (use `stash/unstash`).
2. **Credentials/auth:** expired token or password, ECR token expired (12h), IAM role missing `ecr:PutImage`/`s3:PutObject`, KMS permission for encrypted bucket. Intermittent -> token expiring mid-build or different agents with different roles.
3. **Network:** DNS resolution, proxy, firewall/SG, NAT gateway limits, timeouts on large files, TLS certificate errors. Random failures often = specific agents in a subnet without the right route.
4. **Repository side:** Nexus/Artifactory disk full, repo read-only, version already exists (release repo disallows redeploy, ECR tag immutability), rate limiting (429), 5xx during repo maintenance.
5. **Agent resources:** agent disk full (cannot stage upload), OOM killing the upload process.
6. **Size/timeouts:** very large artifacts hit proxy or client timeouts - use multipart upload, increase timeouts.

Fix pattern: read the exact error code (401/403 auth, 404 path, 409 conflict/exists, 413 too large, 429 rate limit, 5xx server), correlate failures with agent name and time, add `retry` with backoff only for transient 5xx/network errors, and add monitoring on the repository.

Example:

```
sh 'ls -lh target/*.jar'                                  # artifact exists?
sh 'aws sts get-caller-identity'                          # which identity is used?
sh 'curl -sv https://nexus.example.com/service/rest/v1/status 2>&1 | tail -20'

retry(3) {
  sh 'mvn -B deploy -DskipTests'                          # transient errors only
}
// Correlate: failed builds -> which agent? env.NODE_NAME
echo "Running on ${env.NODE_NAME}"
```

What to say to the interviewer: *I check layer by layer: whether the artifact was really produced at that path, then credentials and IAM permissions including expiring tokens, then network - DNS, proxy, security groups and timeouts, then the repository itself for disk, immutability conflicts, rate limits or 5xx errors, and finally agent disk and memory. For random failures I correlate them with the agent name and time, read the HTTP status code, and only add retries with backoff for genuine transient errors.*

Q65. A Jenkins pipeline fails randomly after long runs. How do you find the root cause?

In simple words: If a marathon runner collapses only after 2 hours, the problem is usually stamina (memory), running out of water (disk), or the race timer (timeouts and expiring passes) - not their shoes.

Detailed answer

Common causes of "fails only on long runs":

- **Expired credentials:** AWS STS session (1h default), ECR token, Kubernetes token, OAuth token expiring mid-build.
- **Timeouts:** pipeline `timeout`, agent connection timeouts, load balancer/proxy idle timeouts cutting a long quiet `sh` step, SSH keepalive.
- **Agent problems:** Kubernetes pod agent evicted/OOMKilled, spot instance reclaimed, agent disk filling with logs/temp files, agent disconnected (ping thread timeout).
- **Controller pressure:** heap exhaustion or GC pauses under long heavy logs.
- **Resource leaks:** test processes, Docker containers, file descriptors piling up.
- **External dependencies:** rate limits, DB connections timing out.

How to find it:

1. Collect several failed runs: same stage? same agent? same duration (e.g. always ~60 min = token expiry)?
2. Read the error at the exact failure point and the agent log (`Manage Jenkins -> Nodes -> agent -> Log`).
3. Check agent pod events (`kubectl describe pod` - OOMKilled/Evicted) or EC2 spot interruption notices.
4. Check controller logs and JVM heap/GC metrics.
5. Reproduce with a debug run; add `timestamps()` to see where time goes.

Fixes: refresh credentials per stage (assume role with longer duration or re-login), right-size agent resources, use on-demand agents for long jobs, add keepalive output for quiet steps, split long jobs into stages or parallel shards, cleanup in `post`.

Example:

```
kubectl -n jenkins get events --sort-by=.lastTimestamp | grep -i -E "oom|evict|kill"
aws sts assume-role --role-arn $ROLE --role-session-name ci --duration-seconds 14400
stage('Push') {
  steps { sh 'aws ecr get-login-password | docker login -u AWS --password-stdin $ECR' } // re-login
}
```

What to say to the interviewer: *I look for a pattern across failures - same stage, same agent, or same elapsed time, since a failure around 60 minutes usually means an expired STS or registry token. Then I check agent logs and pod events for OOMKilled, eviction or spot interruption, controller heap, and network idle timeouts. Fixes include refreshing credentials per stage, right-sizing agents, on-demand agents for long jobs, keepalive output, and splitting the job into smaller parallel stages.*

Q66. The Jenkins pipeline runs, but the build does not actually happen. What could be wrong?

In simple words: The oven turned on but no cake came out. Maybe the recipe said "only bake on Sundays" (a when condition), there was no cook available (no agent), or someone rang the wrong kitchen's bell (the webhook hit a different job).

Detailed answer

Possible causes and checks:

- `when` **conditions / branch filters skip stages:** `when { branch 'main' }` but the build is on `feature/x` or a PR (`BRANCH_NAME` is `PR-12`); stages show grey "skipped" in Stage View.
- **No agent / executors:** wrong `agent { label 'docker' }` label, agents offline, all executors busy - build sits in queue "Waiting for next available executor".
- **Webhook or polling hits the wrong job:** repo URL in job does not exactly match the webhook payload (https vs ssh, `.git`), branch spec `*/master` vs `main`, so Jenkins "sees" no changes.
- **Empty changeset:** Poll SCM found no new commits, or a path/`changeset` condition excludes the changed files; monorepo change detection skipping every service.
- **Cached/stale workspace:** old checkout, `git diff` against wrong base, build tool says "up-to-date" and skips.
- **Script approval / sandbox:** Groovy method not approved (`Scripts not permitted to use method...`) - pipeline stops early.
- **Parameters:** default parameter values like `SKIP_BUILD=true` or empty `TAG`; first run of a parameterized pipeline uses defaults.
- **Script bug:** `sh` returns 0 even though it did nothing (`set -e` missing, `|| true`), wrong working directory (`dir()`).

Debug order: open Stage View (which stages skipped) -> console log -> `echo` key variables (`BRANCH_NAME`, `CHANGE_ID`, params) -> check queue/agents -> check webhook deliveries and job SCM config -> Script Approval page.

Example:

```
stage('Debug') {
  steps {
    echo "BRANCH=${env.BRANCH_NAME} PR=${env.CHANGE_ID} NODE=${env.NODE_NAME}"
    echo "PARAMS=${params}"
    sh 'git log -1 --oneline && git status'
  }
}
sh '''
  set -euo pipefail          # fail loudly instead of silently doing nothing
  make build
'''
```

What to say to the interviewer: *I open the Stage View to see which stages were skipped - often a when branch condition, or a PR build where the branch name is different. Then I check the queue for missing agent labels or executors, the webhook and job SCM config for repo or branch mismatches, empty changesets or stale workspaces, pending script approvals, and default parameter values. Echoing the branch, parameters and node, and using set -e in shell steps, makes the cause obvious*

quickly.

Q67. Jenkins credentials were exposed accidentally. What is your immediate action plan?

In simple words: If you lose your house key, you do not first investigate how you lost it - you change the locks right now, then check if anyone entered, then figure out how it happened.

Detailed answer

Immediate (minutes):

1. **Revoke/rotate** the exposed secret at the source: deactivate the AWS access key, regenerate GitHub token, change DB password, rotate API key. Update the Jenkins credential with the new value.
2. **Contain:** if a cloud key, attach a deny-all policy or disable the IAM user; revoke active sessions (`aws iam` revoke older sessions for roles).
3. **Remove the exposure:** delete the log/build output, purge from Git history (`git filter-repo` / BFG) and force push, remove public paste/screenshot; note: Git history purging does not un-leak it - rotation is what matters.

Investigate (hours):

4. Check **CloudTrail / audit logs** for usage of the key since the exposure time (unknown IPs, new IAM users, EC2 instances in odd regions, S3 downloads).
5. Check Jenkins audit trail: who could see it, which jobs used it.
6. Clean up any attacker-created resources.

Prevent:

7. Mask secrets (`withCredentials`), never echo; enable secret scanning (gitleaks, GitHub push protection); replace static keys with IAM roles/OIDC; restrict credential permissions by folder; blameless postmortem.

Example:

```
aws iam update-access-key --user-name ci-user --access-key-id AKIA... --status Inactive
aws iam create-access-key --user-name ci-user          # only if a key is truly needed
aws cloudtrail lookup-events --lookup-attributes \
  AttributeKey=AccessKeyId,AttributeValue=AKIA... --max-results 50
git filter-repo --replace-text secrets-to-remove.txt
```

What to say to the interviewer: *First I revoke or rotate the credential at the source immediately and update Jenkins with the new one, then remove the exposure from logs and Git history, knowing rotation is what actually protects us. Next I check CloudTrail and audit logs for any misuse since the exposure and clean up anything suspicious. Finally I prevent recurrence with masking, secret scanning with push protection, IAM roles instead of static keys, and a blameless postmortem.*

Q68. CI/CD logs show flaky tests breaking the pipeline. How do you handle them?

In simple words: A flaky test is a smoke alarm that goes off randomly. If you ignore it, people stop trusting all alarms. Take it off the wall, fix it, then put it back - do not just remove the batteries forever.

Detailed answer

1. **Confirm it is flaky:** same commit passes and fails on rerun. Track failure rates per test (JUnit history, Flaky Test Handler plugin, test analytics).
2. **Quarantine:** move known flaky tests into a separate non-blocking suite/tag so the main pipeline stays trustworthy - with a ticket and an owner, and a deadline.
3. **Find the root cause** - common ones:

- timing: `sleep` instead of waiting for a condition, async code, timeouts too tight.
- shared state: tests depend on order, shared DB rows, fixed ports, parallel tests clashing.
- external dependencies: real network calls, third-party APIs -> use mocks/test containers.
- environment: time zones, random seeds, low agent resources.

4. **Fix and bring back** into the blocking suite.
5. Limited `retry` for a known unstable integration step is okay short-term, but never blanket-retry the whole test stage - it hides real bugs.
6. Make metrics visible: flaky count on a dashboard, a "no new flaky tests" rule.

Example:

```
# Run tests in random order to expose order dependence
mvn test -Dsurefire.runOrder=random
pytest -p randomly --count=20 tests/test_orders.py   # repeat to confirm flakiness

stage('Tests') {
  steps { sh 'mvn -B test -Dgroups=!quarantine' }          // blocking suite
}
stage('Quarantined tests') {
  steps { catchError(buildResult: 'SUCCESS', stageResult: 'UNSTABLE') {
    sh 'mvn -B test -Dgroups=quarantine' } }               // non-blocking
}
```

> ***What to say to the interviewer:*** *I first confirm flakiness by rerunning the same commit and tracking failure rates, then quarantine the flaky tests into a non-blocking suite with an owner and ticket so the main pipeline stays trustworthy. The root cause is usually timing, shared state, test ordering or external dependencies, which we fix with proper waits, isolation and mocks before bringing the test back. I avoid blanket retries because they hide real bugs.*

Q69. A GitHub Actions workflow is stuck in "pending" or "queued" state. How do you troubleshoot it?

> **In simple words:** The job is waiting in line for a worker. Either no worker with the right badge exists, all workers are busy, or a gate (approval, concurrency lock) is holding it back.

Detailed answer

Checks in order:

1. **GitHub status page** - Actions outage or degraded service.
2. `runs-on` **label mismatch** - job asks for `self-hosted, linux, gpu` but no runner has all those labels -> waits forever ("Waiting for a runner to pick up this job").
3. **Self-hosted runners offline/busy** - Settings -> Actions -> Runners: offline, runner service stopped, runner in a group not allowed for this repo, autoscaler (ARC on Kubernetes) not scaling

- check controller logs and pod pending reasons.

4. **Concurrency groups** - `concurrency:` with an older run in progress holds new ones pending.
5. **Environment protection rules** - job targeting `environment: production` waits for required reviewers or a wait timer.
6. **Usage limits** - concurrent job limits for the plan, spending limit reached for private repos, org policy disabling actions.
7. **Workflow approval** - first-time contributors from forks need approval to run.

Fix examples: correct labels, restart runner service, scale runner pool, cancel the stale run holding the concurrency lock, approve the deployment, raise limits.

Example:

```
jobs:
  deploy:
    runs-on: [self-hosted, linux, x64]      # must match a registered runner's labels
    environment: production                 # may wait for reviewers
    concurrency:
      group: deploy-prod
      cancel-in-progress: false             # new runs wait until current one finishes

# Self-hosted runner checks
sudo ./svc.sh status
kubectl -n arc-runners get pods; kubectl -n arc-systems logs deploy/arc-gha-rs-controller
gh run list --status queued ; gh run cancel <run-id>
```

> ***What to say to the interviewer:*** *A pending job is waiting for a runner or a gate. I check the GitHub status page, then whether the runs-on labels match an online runner, whether self-hosted runners or the ARC autoscaler are healthy and allowed for the repo, then concurrency groups holding the run, environment approvals, usage or spending limits, and fork approval rules. Usually it is a label mismatch or offline runners.*

Q70. Your team complains of deployment delays due to manual approvals. How would you automate this safely?

> **In simple words:** Instead of a manager personally checking every package, install good scanners and rules. If the scanner says everything is normal, the package goes through; the manager only looks at unusual ones.

Detailed answer

Replace human judgement with automated evidence:

- **Strong automated gates:** tests, coverage, SonarQube quality gate, security scans, policy checks (OPA/Conftest on manifests, Terraform plan checks) - if all pass, promotion is automatic.
- **Risk-based approvals:** auto-approve low-risk changes (config, small services, non-prod, standard changes) and keep a human approval only for high-risk ones (DB migrations, infra deletes, payment services).
- **Progressive delivery with automated rollback:** canary + metric analysis (Argo Rollouts, CodeDeploy alarms). If the canary looks bad it rolls back automatically, so the approval adds little.

- **Approve earlier in the flow:** code review on the PR is the approval; merge = approved to deploy (GitOps).
- **Deployment windows and change freezes** enforced by the tool, not by a person.
- **Audit trail:** every deploy logged with who merged, what changed, test results - satisfies compliance without waiting.
- **Make remaining approvals fast:** Teams/Slack approval buttons, approver groups instead of one person, timeouts with escalation.

Example:

```
stage('Policy check') { steps { sh 'conftest test k8s/ -p policy/' } }
stage('Approval') {
  when { expression { env.RISK == 'high' } }       // only risky changes wait
  steps { timeout(time: 4, unit: 'HOURS') {
    input message: 'Approve high-risk deploy?', submitter: 'release-managers,sre-leads' } }
}
stage('Canary') { steps { sh 'kubectl argo rollouts set image orders app=$IMAGE' } }
```

What to say to the interviewer: *I replace manual checks with automated evidence - tests, quality and security gates, and policy-as-code - and use canary deployments with automatic metric-based rollback. Then approvals become risk-based: low-risk changes promote automatically, and only high-risk changes like DB migrations need a human, via group approvers with Teams or Slack buttons. Everything is audited, so we meet compliance without the delay.*

Q71. Your production pipeline is blocked due to missing approvals and stakeholders are unreachable. What will you do?

In simple words: If the bank manager who signs cheques is on leave, you do not forge the signature. You follow the bank's backup rule - ask the deputy, or for a true emergency use the break-glass procedure that gets reviewed afterwards.

Detailed answer

1. **Assess urgency:** is it a normal release (can wait) or a production-fixing hotfix/security patch (cannot wait)?
2. **Normal release:** wait; communicate the delay; do not bypass the gate. Reschedule.
3. **Urgent:** follow the documented **escalation path** - backup approvers, on-call manager, delegate group. Approval should be to a group, not one person.
4. **Break-glass procedure** if defined: an emergency change path with a limited set of people, logged, and reviewed after (retro-approval / emergency CAB). Never share someone's login or edit the pipeline to remove the gate.
5. **Reduce risk of the change:** deploy the smallest possible fix, with canary and ready rollback, and monitoring watched live.
6. **Document everything:** who approved, why, timeline; inform stakeholders as soon as reachable.
7. **Fix the process:** add backup approvers, approval groups, SLA/timeout escalation, and define "standard changes" that can auto-approve.

Example:

```
input message: 'Approve PROD deploy?',
      submitter: 'release-managers,oncall-managers'   // group + backup, not one person

// Break-glass: separate job, restricted permission, forces an incident ticket
parameters { string(name: 'INCIDENT_ID', description: 'Required for emergency deploy') }
stage('Validate') { steps { script {
  if (!params.INCIDENT_ID?.trim()) { error 'Emergency deploy needs an incident ID' } } } }
```

What to say to the interviewer: *I first judge urgency - a normal release simply waits and I communicate the delay, because bypassing controls is not an option. For an urgent fix I follow the escalation path to backup approvers or use the documented break-glass process, deploy the smallest change with canary and rollback ready, and log everything for retrospective approval. Afterwards I fix the process with approval groups, backups and escalation timeouts.*

Q72. A pipeline deployment failed in production. How would you troubleshoot it step by step?

In simple words: First make sure customers are safe (is the old version still serving?), then read the error message, then walk backwards: was it the package, the delivery truck, or the destination?

Detailed answer

1. **Check user impact first:** is production still serving the old version (failed rollout usually leaves old pods running) or is it broken? If broken -> roll back immediately, debug later.
2. **Find the failing stage and exact error** in the pipeline log: build, push, auth, Helm/kubectl, health check timeout.
3. **Pipeline/infra layer:** credentials/role expired, kubeconfig/context wrong, registry unreachable, Helm release stuck in `pending-upgrade`, Terraform lock.
4. **Deployment layer (Kubernetes):** `kubectl rollout status`, `get pods`, `describe` pod events - ImagePullBackOff (wrong tag/permissions), CrashLoopBackOff (app/config), Pending (resources/quota), readiness probe failing.
5. **Application layer:** pod logs - missing env var/secret/ConfigMap, DB migration failure, dependency unreachable.
6. **Compare with the last successful deploy:** diff image tag, Helm values, config, secrets, infra changes.
7. **Fix forward or roll back**, then rerun the pipeline; verify with smoke tests and metrics.
8. **RCA + prevention:** add the missing check to the pipeline (e.g. config validation, staging parity, `helm --atomic`).

Example:

```
helm status orders-api -n prod ; helm history orders-api -n prod
kubectl -n prod rollout status deploy/orders-api
kubectl -n prod get pods -l app=orders-api
kubectl -n prod describe pod <pod> | tail -30
kubectl -n prod logs <pod> --previous
helm get values orders-api -n prod --revision 42 > old.yaml
helm get values orders-api -n prod --revision 43 > new.yaml ; diff old.yaml new.yaml
# Stuck release
helm rollback orders-api 42 -n prod
```

What to say to the interviewer: *My first step is checking user impact - if production is broken I roll back immediately. Then I find the failing stage and exact error, and check layer by layer: pipeline credentials and tooling, Kubernetes rollout and pod events for image pull, crash or pending issues, then application logs, and I diff the failed release against the last good one. After fixing or rolling back, I verify with smoke tests and add a pipeline check so it cannot recur.*

Q73. A production deployment caused downtime or an incident. How do you roll back and identify the root cause?

Also asked as: How would you handle a production incident caused by a faulty deployment?

In simple words: When a new tyre makes the car wobble on the highway, you put the old tyre back first and get the car moving. Only after that do you study why the new tyre was bad.

Detailed answer

During the incident:

1. **Declare and communicate:** incident channel, incident commander, status update to stakeholders.
2. **Correlate:** deploy time vs when errors started (deploy markers on Grafana, alert timeline). If they match, suspect the deploy.
3. **Mitigate first - roll back:** `helm rollback` / `kubectl rollout undo` / git revert in the GitOps repo / swap Blue-Green back / canary weight to 0 / turn off the feature flag. Check DB migrations are backward compatible before rolling back code.
4. **Verify recovery:** error rate, latency, and business metrics back to normal; keep watching.
5. **Freeze further deploys** for that service until understood.

After the incident (RCA):

6. **Collect evidence:** diff between versions (code, config, Helm values, dependencies), logs, traces, metrics, pipeline logs.
7. **Reproduce in staging** with the bad version.
8. **5 Whys:** why did the bug exist, and why did the pipeline not catch it (missing test, staging different from prod, no canary)?
9. **Blameless postmortem** with timeline, impact, root cause, action items with owners (add tests, canary analysis, config validation, alerts).

Example:

```
kubectl -n prod rollout undo deploy/orders-api
kubectl -n prod rollout status deploy/orders-api
git log --oneline v1.4.0..v1.4.1            # what changed between releases
git diff v1.4.0 v1.4.1 -- charts/ config/
# Grafana annotation for deploy markers
curl -X POST -H "Authorization: Bearer $GRAFANA_TOKEN" -H "Content-Type: application/json" \
  https://grafana.example.com/api/annotations -d '{"tags":["deploy","orders"],"text":"v1.4.1"}'
```

What to say to the interviewer: *I mitigate first: open an incident channel, correlate the deploy time with the error spike, and roll back through Helm, a GitOps revert, a Blue-Green switch or a feature flag, then confirm metrics recover and freeze further deploys. After that I diff the two versions, reproduce in staging, use 5 Whys to find both the bug and why the pipeline missed it, and write a blameless postmortem with owned action items like better tests or canary analysis.*

Q74. A new deployment causes a CPU spike. How will you roll back and debug?

In simple words: The new engine runs hot. Swap the old engine back in so the car keeps going, then put the new engine on a test bench to find which part overheats.

Detailed answer

Roll back / contain:

1. Confirm correlation: CPU spike starts at the deploy time; only new-version pods are hot (`kubectl top pods`, Grafana by version label).
2. If users are impacted (latency, errors, HPA maxed out, node pressure) -> **roll back immediately** (`helm rollback` / `rollout undo` / canary abort). Short-term alternative: scale out while deciding.
3. Verify CPU and latency return to baseline.

Debug the bad version (in staging or a single canary pod):

- **Diff the change:** code (new loop, regex, serialization, logging level set to DEBUG), dependency upgrade, config (thread pool, cache disabled, connection pool too small causing retries), resource limits changed (CPU throttling looks like high CPU).
- **Profile:** language profilers (async-profiler/JFR for Java, pprof for Go, py-spy), thread dumps to see hot threads; `top -H` inside the container.
- **Check traffic pattern:** retry storms, a new endpoint being called heavily, health checks too frequent.
- **Check GC:** memory leak -> constant garbage collection -> high CPU.
- **Load test** the fix before redeploying with canary.

Prevent: performance test stage in pipeline, canary analysis on CPU/latency, resource requests/limits set correctly, alerts on CPU vs baseline per version.

Example:

```
kubectl -n prod top pods -l app=orders-api --sort-by=cpu
helm rollback orders-api -n prod        # or: kubectl argo rollouts abort orders-api
kubectl -n staging exec -it <new-pod> -- top -H -b -n 1 | head -20
kubectl -n staging exec <new-pod> -- jcmd 1 Thread.print > threads.txt   # Java thread dump
# Prometheus: CPU by version
sum by (version) (rate(container_cpu_usage_seconds_total{container="orders-api"}[5m]))
```

What to say to the interviewer: *I confirm the CPU spike lines up with the deploy and only affects new-version pods, and if users are impacted I roll back or abort the canary immediately, then confirm CPU returns to baseline. I debug the new version in staging by diffing code, dependencies and config, profiling with thread dumps or a profiler, and checking for retry storms or GC pressure. Before redeploying I load-test the fix and add canary analysis on CPU and latency to the pipeline.*

Q75. A deployment to staging failed during the DB-migration step and blocked all environments. What is your rollback plan?

In simple words: A renovation stopped halfway through knocking down a wall. First stop all work so nobody makes it worse, look at exactly which bricks are already gone, then either put the wall back (restore) or finish the job carefully (fix forward). Next time, renovate one house at a time instead of making all houses wait on one.

Detailed answer

Immediate:

1. **Stop and lock:** abort the pipeline and lock promotion so nothing retries the migration or pushes this version to other environments.
2. **Check the actual DB state:** the migration tool's history table (Flyway `flyway_schema_history`, Liquibase `DATABASECHANGELOG`, Alembic `alembic_version`) - which migration failed, is it marked failed/dirty, was it partially applied? Check for leftover locks (`DATABASECHANGELOGLOCK`).
3. **Choose the recovery path:**

- Migration ran inside a transaction and rolled back cleanly -> repair the history entry, fix the script, rerun.
- Partially applied and the script is idempotent (`IF NOT EXISTS`) -> **fix forward**: correct the script and rerun.
- Partially applied and not safe -> run the **down-migration**, or **restore the snapshot** taken before the migration.

4. **Roll the app back** to the previous version only if the schema is still compatible with it.
5. **Unblock other environments:** release the lock; dev and prod should not be waiting on staging.

Prevention:

- **Expand-contract (backward-compatible) migrations:** add new columns/tables first, switch the code, remove old ones in a later release - so old and new app versions both work.
- **Automatic pre-migration snapshot/backup** step.
- **Dry-run** migrations against a prod-like copy (restored snapshot) in CI.
- **Decouple migrations into their own job/stage** per environment, run before the app deploy, with clear failure handling.
- **Per-environment pipelines and locks**, not one shared lock that blocks everything.
- Keep migrations small and transactional where the database supports it.

Example:

```
flyway -url=$STAGING_DB info            # state: Success / Failed / Pending
flyway -url=$STAGING_DB repair          # clean failed entry after fixing the script
liquibase --url=$STAGING_DB release-locks
alembic current ; alembic downgrade -1

# Pre-migration snapshot (RDS)
aws rds create-db-snapshot --db-instance-identifier staging-db \
  --db-snapshot-identifier pre-mig-$(date +%Y%m%d%H%M)

-- Idempotent, expand-style migration
ALTER TABLE orders ADD COLUMN IF NOT EXISTS customer_ref VARCHAR(64);
```

What to say to the interviewer: *I stop the pipeline and lock promotion, then check the migration history table to see exactly what was applied and whether it is marked failed or locked. If it rolled back cleanly I repair and rerun; if it is partially applied I either fix forward with an idempotent script or run the down-migration or restore the pre-migration snapshot, then release locks so other environments are unblocked. To prevent it I use expand-contract migrations, automatic snapshots, dry-runs against a prod-like copy, and a separate migration job per environment instead of one shared lock.*

Q76. During a blue-green deployment, shifting traffic to green caused a spike in errors. What do you check and fix?

In simple words: You moved the audience to the new stage and people started complaining. Move them back to the old stage right away (it is still ready), then check what the new stage was missing - lights not warmed up, wrong script, not enough seats.

Detailed answer

Immediately: **shift traffic back to blue** (listener / target group / Service selector / slot swap). Blue is still running, so this is fast. Confirm errors drop. Then investigate green without users on it:

1. **Readiness vs real warm-up:** health check passed but the app was cold - empty caches, JIT not warmed, connection pools not filled, lazy loading on first request.
2. **Health check path vs real path:** `/health` returns 200 but does not test the DB, cache or downstream APIs that real requests need.
3. **Config/secret/env diff:** compare blue vs green env vars, ConfigMaps, secrets, feature flags and endpoints (e.g. green pointing to a wrong DB or missing a secret).
4. **DB schema compatibility:** green expects a migration that was not applied, or a migration broke blue/green coexistence.
5. **Sessions:** in-memory sessions or sticky cookies bound to blue are lost -> users logged out or 401/500 errors. Use a shared session store (Redis).
6. **Capacity:** green scaled smaller than blue (fewer replicas, smaller instances, HPA min too low) -> overload at 100% traffic.
7. **Downstream limits:** third-party rate limits, IP allowlists that only include blue's egress IPs, security groups, connection storms to the DB (max connections).
8. **Logs, metrics and traces by version:** which endpoints and error codes (5xx vs 4xx) appear only on green.

Prevention:

- Do a **canary step first** (1-10% to green) before 100%.
- **Automated analysis and rollback:** Argo Rollouts blue-green with pre/post-promotion analysis, CodeDeploy with CloudWatch alarms.
- **Pre-warm** green (synthetic traffic, cache warm-up) and match its capacity to blue.
- Deep health checks, config parity checks, externalized sessions.

Example:

```
# ALB: send traffic back to blue
aws elbv2 modify-listener --listener-arn $LISTENER \
  --default-actions Type=forward,TargetGroupArn=$TG_BLUE

# K8s: flip the Service back
kubectl patch svc orders -p '{"spec":{"selector":{"app":"orders","version":"blue"}}}'

# Compare config and capacity
diff <(kubectl get deploy orders-blue -o jsonpath='{.spec.template.spec.containers[0].env}') \
     <(kubectl get deploy orders-green -o jsonpath='{.spec.template.spec.containers[0].env}')
kubectl get hpa orders-blue orders-green
```

> ***What to say to the interviewer:*** *First I shift traffic back to blue, which is still running, and confirm errors drop. Then I check green offline: whether it was really warmed up or only passed a shallow health check, config and secret differences from blue, database schema compatibility, lost sessions or sticky cookies, whether green had the same capacity, and downstream rate limits or IP allowlists. To prevent it I add a canary step before full cutover, automated analysis with Argo Rollouts or CodeDeploy alarms, pre-warming and deep health checks.*

Q77. How do you know your last deploy was actually successful? Monitoring? Business metrics? Or silence?

> **In simple words:** A green pipeline only means the parcel left the warehouse. Success means the customer received it and is happy. And "nobody complained" is not proof - maybe the phone line is broken.

Detailed answer

Pipeline green is not success. Layers of evidence, from technical to business:

1. **Rollout completed:** `kubectl rollout status`, all new pods Ready, no restarts; Helm/Argo CD status Healthy and Synced.
2. **Smoke tests and synthetic checks** right after deploy: hit key endpoints and user journeys (login, checkout) - part of the pipeline, fail = rollback.
3. **Golden signals before vs after** (and canary vs baseline): error rate, p95/p99 latency, traffic, saturation (CPU, memory, DB connections). Automate with **Argo Rollouts AnalysisTemplate**, **CodeDeploy alarms** or **Kayenta**-style canary analysis.
4. **Business KPIs:** checkouts per minute, successful logins, payments processed, orders created, meter reads processed - a deploy can be technically healthy but break a business flow.
5. **Log error-rate diff:** new error messages or exception types appearing after the deploy (Loki/ELK query by version).
6. **Deploy annotations on Grafana dashboards** so every graph shows exactly when the deploy happened.

7. **SLO / error-budget burn alerts:** a fast burn right after a deploy = likely bad deploy.
8. **Track DORA metrics:** change failure rate and time to restore tell you how often "successful" deploys really were not.

Silence is not evidence: verify the alerting path itself works - alert rules loaded, Alertmanager/contact points healthy, notification templates valid, a test alert or heartbeat (dead man's switch) arriving in Teams/Slack. A broken notification channel looks exactly like "everything is fine".

Example:

```
kubectl -n prod rollout status deploy/orders-api --timeout=5m
curl -fsS https://api.example.com/health/deep && ./smoke-tests.sh prod

# Error rate new version vs before (PromQL)
sum(rate(http_requests_total{app="orders",code=~"5.."}[5m]))
  / sum(rate(http_requests_total{app="orders"}[5m]))

# Argo Rollouts analysis (fragment)
kind: AnalysisTemplate
spec:
  metrics:
  - name: error-rate
    successCondition: result[0] < 0.01
    provider:
      prometheus:
        query: sum(rate(http_requests_total{app="orders",code=~"5.."}[5m])) /
               sum(rate(http_requests_total{app="orders"}[5m]))

# Heartbeat alert: always firing, proves the alert pipeline is alive
- alert: Watchdog
  expr: vector(1)
```

> ***What to say to the interviewer:*** *A green pipeline only proves the deploy ran, so I verify in layers: rollout status and readiness, post-deploy smoke and synthetic tests, golden signals compared before and after or canary versus baseline with automated analysis, business KPIs like logins or orders, log error diffs, deploy annotations on dashboards and error-budget burn alerts, and we track change failure rate. I never treat silence as success - I make sure the alerting path itself works with heartbeat alerts and tested notification channels.*

Q78. You notice a sudden spike in failed deployments. How would you investigate?

> **In simple words:** If suddenly many parcels are failing delivery, you do not check each parcel one by one. You sort them into piles - by city, by truck, by day - and find the one thing they have in common: a closed road, a broken truck, or a new wrong address label.

Detailed answer

1. **Quantify and categorize** the failures by **stage** (build, push, deploy, health check), **service**, **environment**, **agent/runner** and **time**. A spike across many services at the same stage means a shared cause; one service means that service's own change.
2. **Find the common change** around when it started:

- a **pipeline template / shared library** version bump;
- a new **base image** or dependency version (broken upstream release);

- an **agent/runner image update** (tool versions changed, e.g. new Helm or kubectl);
- an **expired credential**, certificate or service connection (OIDC trust, token, registry password);
- an outage of a **dependency registry** (npm, Maven Central, Docker Hub rate limits, ECR);
- cloud **quota** or API rate limits;
- **cluster capacity** (pods Pending, no nodes, IP exhaustion) or a new admission policy (OPA/Kyverno rule rejecting manifests).

3. **Read the actual errors** from a few failures in each category - often they are the same message.
4. **Check trends:** DORA **change failure rate** and deployment frequency over recent weeks - a sudden jump points to a platform change; a slow rise points to quality or test gaps.
5. **Fix the common cause** (pin or roll back the template or agent image, rotate the credential, add a registry mirror, add capacity), then rerun the failed deploys.
6. **Add guardrails:** version-pinned templates and agent images rolled out gradually (canary pipelines), alerts on credential expiry, a pull-through cache for registries, capacity alerts, and a dashboard of deploy failures by stage so the next spike is spotted in minutes.

Example:

```
# GitHub Actions: failed runs since yesterday, grouped by workflow
gh run list --status failure --created ">=2026-09-23" --limit 200 \
  --json workflowName,createdAt | jq 'group_by(.workflowName)
  | map({wf: .[0].workflowName, fails: length})'

# Kubernetes side: are deploys failing on capacity or policy?
kubectl get events -A --field-selector type=Warning --sort-by=.lastTimestamp | tail -30
kubectl get pods -A --field-selector status.phase=Pending

# Change failure rate = failed deployments / total deployments (per week)
```

> ***What to say to the interviewer:*** *I categorize the failures by stage, service, environment, agent and time to see whether it is one service or a shared cause, then look for the common change - a pipeline template or agent image update, a new base image, an expired credential or service connection, a registry outage, quota, or cluster capacity and admission policies. I confirm with the actual error messages and the change failure rate trend, fix the shared cause and rerun, then add guardrails like pinned templates and agent images, credential-expiry alerts, registry caching and a deploy-failure dashboard.*

Q79. Jenkins is failing to push a Docker image to the registry. How do you troubleshoot?

> **In simple words:** A delivery van can't drop a parcel at the warehouse. Is the van's permit expired (auth), is it at the wrong address (tag/repo name), is the road blocked (network), or is the warehouse refusing that parcel (permissions, immutable tags, size)?

Detailed answer

1. **Read the exact error in the console log.** It usually points straight at the layer:

- `unauthorized` / `denied` / `no basic auth credentials` -> authentication. ECR login tokens expire after **12 hours**, so run `aws ecr get-login-password` in the pipeline every time; the Jenkins credential or IAM role may be wrong or expired.

- `denied: ... not authorized to perform ecr:InitiateLayerUpload` -> the IAM policy is missing push actions (`ecr:PutImage`, `InitiateLayerUpload`, `UploadLayerPart`, `CompleteLayerUpload`, `BatchCheckLayerAvailability`).

- `name unknown` / `repository does not exist` -> ECR needs the repo created first (Terraform it), or the tag doesn't include the full registry prefix.

- `tag invalid: ... already exists` -> **immutable tags** are enabled, so use unique tags (the Git SHA or build number).

- `i/o timeout` / `TLS handshake timeout` -> network: proxy, DNS, security group egress, or missing VPC endpoints (`ecr.api`, `ecr.dkr`, S3) for private agents.

2. **Agent side:** is the Docker daemon running, and does the `jenkins` user have socket permissions (`permission denied ... docker.sock`)? Is there disk space for layers (`docker system df`)?

3. **Image size and timeouts:** huge layers on slow links, so optimise the image and retry.

4. **Reproduce manually** on the same agent as the same user, to separate a pipeline problem from an environment problem.

Example:

```
stage('Push') {
  steps {
    withAWS(role: 'jenkins-ecr-push', region: 'ap-south-1') {
      sh '''
        REG=123456789012.dkr.ecr.ap-south-1.amazonaws.com
        aws ecr get-login-password | docker login --username AWS --password-stdin $REG
        docker tag meter-api:${GIT_COMMIT} $REG/meter-api:${GIT_COMMIT}
        docker push $REG/meter-api:${GIT_COMMIT}
      '''
    }
  }
}
```

```
# manual checks on the agent, as the jenkins user
sudo -u jenkins docker info >/dev/null && echo "daemon ok"
aws sts get-caller-identity
aws ecr describe-repositories --repository-names meter-api
nc -zv 123456789012.dkr.ecr.ap-south-1.amazonaws.com 443
```

> ***What to say to the interviewer:*** *I read the exact push error first because it tells me the layer. Unauthorized means an expired ECR token or wrong credentials, so I log in freshly each run with the IAM role. AccessDenied on upload actions means the IAM policy. "Repository does not exist" or a tag conflict means the repo isn't created or tags are immutable. Timeouts mean proxy, DNS or missing VPC endpoints. I also check that the Docker daemon, socket permissions and disk on the agent are fine, and reproduce the push manually as the jenkins user.*

Terraform & Infrastructure as Code

Basic

Q1. What is Infrastructure as Code, and how do you manage it using Terraform or CloudFormation?

Also asked as: What are the advantages of IaC tools like Terraform?

> **In simple words:** Instead of clicking around the AWS console to build servers, you write a recipe file. Anyone can read the recipe, review it, and cook the exact same dish again and again.

Detailed answer

Infrastructure as Code (IaC) means your VPCs, servers, databases and DNS records are described in text files that live in Git. A tool reads those files and creates or changes the real cloud resources to match.

- **Terraform** (HashiCorp, or its open-source fork OpenTofu) works with almost any cloud or SaaS through "providers". It keeps a **state file** to remember what it built.
- **CloudFormation** is AWS-only. You write YAML/JSON "templates"; AWS itself stores the state inside a "stack".
- The basic Terraform workflow: `init` (download providers) -> `plan` (preview changes) -> `apply` (make changes) -> `destroy` (remove).
- Good practice: code in Git, pull requests for review, `plan` run automatically in CI, `apply` only after approval.

Advantages of IaC:

- **Repeatability**: the same code builds identical dev, stage and prod, with no snowflake setups.
- **Version control and review**: every change is a Git commit and a pull request, so you can see who, what and why, and roll back.
- **Drift detection**: `plan` shows when reality no longer matches the code.
- **Speed**: a whole environment in minutes instead of days of clicking.
- **DR rebuild**: re-create the stack in another region from code.
- **Documentation as code**: the code is the always-current description of the infrastructure.
- **Multi-cloud**: one tool and workflow for AWS, GCP, Azure and SaaS such as Datadog or GitHub.
- **Cost and policy checks in CI**: tools like Infracost, Checkov and OPA review a change before it exists.

Example:

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "logs" {
  bucket = "myapp-prod-logs"
  tags   = { Env = "prod", Owner = "devops" }
}
```

```
terraform init
terraform plan   # preview: 1 to add, 0 to change, 0 to destroy
terraform apply
```

> ***What to say to the interviewer:*** *IaC means defining infrastructure in version-controlled code instead of manual console clicks, so it is repeatable, reviewable and auditable. I use Terraform with modules and remote state, run plan in CI on every pull request, and apply only after review. CloudFormation is the AWS-native alternative where AWS manages state as stacks.*

Q2. What is the Terraform state file and why is it important?

Also asked as: Why is the state file important, and how does DynamoDB locking protect it?

Also asked as: Explain the state file and where you store it. What are the best practices for storing state?

> **In simple words:** The state file is Terraform's notebook. It writes down "I created this bucket and its real AWS ID is X". Without the notebook, Terraform forgets what it owns.

Detailed answer

`terraform.tfstate` is a JSON file that maps each resource in your code (for example `aws_instance.web`) to the real object in the cloud (for example `i-0abc123`). It also stores attributes like IPs, ARNs and dependencies.

Terraform uses it to:

- Know what already exists, so it updates instead of re-creating.
- Calculate the difference (the plan) between code, state and reality.
- Track dependencies so it deletes things in the right order.
- Speed up plans for large setups.

Important points:

- State can contain **secrets in plain text** (DB passwords, keys). Never commit it to Git.
- Store it remotely (S3, GCS, Azure Blob, HCP Terraform) with encryption, versioning and locking.
- Never hand-edit it. Use `terraform state list`, `state show`, `state mv`, `state rm`.

Example:

```
terraform state list
# aws_vpc.main
# aws_instance.web
terraform state show aws_instance.web   # shows id, private_ip, ami, ...
```

> ***What to say to the interviewer:*** *The state file is the mapping between my Terraform code and the real resources in the cloud, and Terraform uses it to compute plans and dependencies. Because it can hold secrets and is critical, I keep it in a remote encrypted and versioned backend like S3 with locking, never in Git, and I only change it using terraform state commands.*

Q3. What are the two main types of variables in Terraform? How are locals and outputs different?

In simple words: Input variables are the blanks in a form you fill in. Environment variables are another way of filling those blanks without typing. Locals are your own scratch calculations, and outputs are the answers you print at the end.

Detailed answer

1. **Input variables** - declared with a `variable` block and set via `terraform.tfvars`, `*.auto.tfvars`, `-var-file`, or `-var` on the command line. They can have a type, a default, a description, validation and `sensitive = true`.
2. **Environment variables (TF_VAR_name)** - Terraform automatically reads any env var named `TF_VAR_<variable_name>` into the matching input variable. Very useful in CI to inject secrets without writing them to files.

Other related concepts:

- **locals** - computed values inside a module (like naming prefixes). Cannot be set from outside.
- **outputs** - values a module exposes (like a VPC ID) to the user or to another module.

Precedence is explained in the next question (variables.tf vs .tfvars).

Example:

```
variable "instance_type" {
  type    = string
  default = "t3.micro"
}

variable "db_password" {
  type      = string
  sensitive = true
}

locals {
  name_prefix = "myapp-${terraform.workspace}"
}

output "web_ip" {
  value = aws_instance.web.private_ip
}
```

```
export TF_VAR_db_password="from-ci-secret-store"
terraform apply -var="instance_type=t3.small"
```

What to say to the interviewer: *The two main types are input variables declared with variable blocks and set through tfvars files or the command line, and environment variables using the TF_VAR_ prefix, which Terraform maps automatically. I use TF_VAR_ in pipelines for secrets, locals for derived values like name prefixes, and outputs to pass IDs between modules.*

Q4. What is the difference between count and for_each, and what is a for expression? When do you use each?

In simple words: count is like numbering seats 0, 1, 2 - if one person leaves, everyone shifts seats. for_each is like name tags - if one person leaves, nobody else moves.

Detailed answer

- **count** creates N copies of a resource, addressed by index: `aws_instance.web[0]`, `[1]`, `[2]`. Best for "N identical things" or an on/off switch (`count = var.enabled ? 1 : 0`).
- **The index-shift problem**: with `count = length(var.users)`, removing the first item from the list shifts every index down by one. Terraform then thinks every later item changed and plans to destroy and re-create them.
- **for_each** takes a map or a set of strings and addresses each copy by key: `aws_iam_user.u["alice"]`. Removing one key only removes that one resource. Use it whenever items have their own identity or different settings.
- **for expression** does not create resources. It transforms a list or map into a new list or map, often to build the input for for_each or an output.

Use	When
count	N identical copies, or a conditional resource
for_each	items with names or different settings (users, buckets, subnets)
for expression	reshape or filter data

Example:

```
variable "buckets" {
  default = {
    logs   = { versioning = true }
    assets = { versioning = false }
  }
}

resource "aws_s3_bucket" "b" {
  for_each = var.buckets
  bucket   = "acme-${each.key}"
  tags     = { Versioning = tostring(each.value.versioning) }
}

locals {
  names_upper = [for x in ["dev", "prod"] : upper(x)]         # ["DEV", "PROD"]
  bucket_arns = { for k, b in aws_s3_bucket.b : k => b.arn }  # map of key -> ARN
}
```

What to say to the interviewer: *count creates copies by numeric index, which is fine for identical resources or on/off switches, but removing an item shifts the indexes and can recreate resources. for_each keys each copy by a map key or set value, so changes stay isolated, and I prefer it for anything with an identity. A for expression like [for x in list : upper(x)] only transforms data, for example to build the map I pass into for_each.*

Q5. What is the difference between variables.tf and .tfvars files? What is the variable precedence order?

In simple words: variables.tf is the form with empty boxes and labels ("instance type: ____"). A .tfvars file is a filled-in copy of that form, one per environment.

Detailed answer

- **variables.tf** = **declaration**: name, type, description, default, validation, sensitive. It says what inputs exist.
- **.tfvars** = **values**: `terraform.tfvars` and `*.auto.tfvars` are loaded automatically; others like `prod.tfvars` are passed with `-var-file`.
- Same code, different tfvars per environment (dev.tfvars, prod.tfvars). Do not commit tfvars that contain secrets.

Precedence (highest wins):

Rank	Source
1 (highest)	`-var` on the command line
2	`-var-file=...`
3	`*.auto.tfvars` (alphabetical order)
4	`terraform.tfvars`
5	`TF_VAR_name` environment variable
6 (lowest)	`default` in the variable block

If a variable has no default and no value is given, Terraform asks for it interactively (or fails in CI with `-input=false`).

Example:

```
# variables.tf
variable "instance_type" {
  type        = string
  description = "EC2 size"
  default     = "t3.micro"
  validation {
    condition     = can(regex("^t3\\.", var.instance_type))
    error_message = "Only t3 family allowed."
  }
}
```

```
# prod.tfvars
instance_type = "t3.large"
```

```
terraform plan -var-file=prod.tfvars                      # t3.large
terraform plan -var-file=prod.tfvars -var="instance_type=t3.xlarge"   # -var wins
```

What to say to the interviewer: *variables.tf declares the inputs with their types, defaults and validation, while tfvars files supply the actual values, usually one file per environment. The precedence from highest to lowest is -var, then -var-file, then auto.tfvars, then terraform.tfvars, then TF_VAR_ environment variables, and finally the default.*

Q6. In which Terraform file or block can you identify the provider version (for example the GCP google provider)?

Also asked as: How do you handle provider API changes between provider versions?

In simple words: The terraform block is your shopping list saying "I need the google plugin, version 6 or newer". The lock file is the receipt saying exactly which version was actually bought.

Detailed answer

- **required_providers** inside the `terraform {}` block declares the provider source and the allowed version range. By convention it lives in `versions.tf` or `providers.tf`, but it can be in any `.tf` file.
- **.terraform.lock.hcl** records the exact version that `terraform init` selected, plus checksums. Commit this file so everyone and CI use the same version.
- The `provider "google" {}` block itself configures project/region; it should not normally contain the version.
- Commands: `terraform version` shows installed providers; `terraform providers` lists requirements; `terraform init -upgrade` moves to a newer allowed version.

Handling provider API changes between versions:

- Pin a range like `version = "~> 5.0"` so a new major version (with breaking changes) is never picked up by surprise.
- Commit `.terraform.lock.hcl` so every laptop and CI run uses the exact same build.
- Upgrade deliberately: `terraform init -upgrade` in a branch, read the provider upgrade guide and changelog, fix deprecated arguments, and run `plan` in staging before prod.
- For mixed Windows/Mac/Linux teams, record checksums for all platforms: `terraform providers lock -platform=linux_amd64 -platform=darwin_arm64 -platform=windows_amd64`.

Example:

```
# versions.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 6.0"   # any 6.x, not 7.0
    }
  }
}
```

```
# .terraform.lock.hcl (generated)
provider "registry.terraform.io/hashicorp/google" {
  version     = "6.8.0"
  constraints = "~> 6.0"
  hashes      = [ "h1:..." ]
}
```

What to say to the interviewer: *The version constraint is declared in the required_providers section of the terraform block, usually in versions.tf, and the exact version actually installed is pinned in .terraform.lock.hcl, which I commit to Git. I can also check it with terraform version or terraform providers.*

Q7. What files are in a typical Terraform folder or module, and what is each file for?

In simple words: Terraform reads every .tf file in a folder as one big file. Splitting it into main, variables, outputs and so on is just tidy labelling, like drawers in a cupboard.

Detailed answer

File / folder	Purpose
`main.tf`	The main resources (and module calls)
`variables.tf`	Input variable declarations
`outputs.tf`	Values exposed to the user or to other modules
`providers.tf` / `versions.tf`	`terraform {}` block with required_version and required_providers, plus provider config
`backend.tf`	Remote state backend (S3 / GCS / azurerm) - only in root modules
`locals.tf`	Computed local values (name prefixes, common tags)
`data.tf`	Data sources (lookups of existing AMIs, VPCs, accounts)
`terraform.tfvars` / `prod.tfvars`	Values for variables, per environment
`.terraform/`	Downloaded providers and modules (created by init, never commit)
`.terraform.lock.hcl`	Exact provider versions and checksums (commit it)
`terraform.tfstate`	Exists locally only when using the local backend - avoid in teams
`.gitignore`	Ignores `.terraform/`, `*.tfstate*`, secret tfvars, crash logs

Root module vs child module: a **root module** (what you run `terraform apply` in) has backend and provider config; a **child module** (reusable, under `modules/`) has only resources, variables and outputs, and never its own backend. The module design question in the Advanced section covers versioning and DRY patterns.

Example:

```
infra/
  modules/
    vpc/
      main.tf  variables.tf  outputs.tf  versions.tf  README.md
    eks/
      main.tf  variables.tf  outputs.tf  versions.tf
  envs/
    dev/
      main.tf  backend.tf  providers.tf  dev.tfvars  .terraform.lock.hcl
    prod/
      main.tf  backend.tf  providers.tf  prod.tfvars  .terraform.lock.hcl
  .gitignore
```

```
# .gitignore
.terraform/
*.tfstate
*.tfstate.*
crash.log
secrets.auto.tfvars
```

> ***What to say to the interviewer:*** *A typical folder has main.tf for resources, variables.tf for inputs, outputs.tf for outputs, versions.tf or providers.tf for provider requirements, backend.tf for remote state, plus locals.tf and data.tf, and tfvars files for per-environment values. .terraform is the downloaded plugin cache that I never commit, while .terraform.lock.hcl is committed, and child modules under modules/ contain only resources, variables and outputs.*

Q8. How does Terraform prevent duplicate resource creation?

> **In simple words:** Before building anything, Terraform checks its notebook (state). If the notebook says "already built this", it does not build a second one, it only fixes differences.

Detailed answer

- Every resource has a unique **address** in code, like `aws_instance.web`. State maps that address to one real ID.
- On `plan`, Terraform refreshes state against the real cloud and compares with code. If the resource exists and matches, it does nothing (idempotent).
- **State locking** stops two people running apply at the same time, which would otherwise create two copies.
- Duplicates can still happen if state is lost, if two teams use different state files for the same thing, or if a resource was created manually. Fix: `import` it instead of creating it.
- Many cloud APIs also reject duplicates (S3 bucket names, IAM role names are unique).
- `count` / `for_each` give each copy a stable key so repeated runs do not add more.

Example:

```
terraform apply    # first run: 1 to add
terraform apply    # second run:
# No changes. Your infrastructure matches the configuration.
```

> ***What to say to the interviewer:*** *Terraform is idempotent because it tracks every resource address against a real ID in the state file, so a second apply only shows differences instead of creating duplicates. Remote state with locking prevents concurrent runs, and for resources that already exist outside Terraform I use import rather than letting it create a copy.*

Q9. Write Terraform code to launch an EC2 instance.

> **In simple words:** Tell Terraform four things: which cloud and region, which operating system image, what firewall rules, and what size of machine. Then preview and build.

Detailed answer

A minimal but complete setup has: a `terraform` block (provider version), a `provider` block (region), a data source to find the latest AMI, a security group, the `aws_instance`, and an output. For a version that also looks up an existing VPC and subnets with data sources, see the "dynamically retrieve VPC details" question in the Advanced section.

Example:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd*/ubuntu-*-24.04-amd64-server-*"]
  }
}

resource "aws_security_group" "web" {
  name = "web-sg"
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web.id]
  tags                   = { Name = "web-server" }
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```

```
terraform init      # download the AWS provider
terraform plan      # preview: 2 to add
terraform apply     # create SG + instance, prints public_ip
terraform destroy   # clean up when done
```

What to say to the interviewer: *I declare the AWS provider in a terraform block, set the region, look up the latest Ubuntu AMI with a data source, create a security group, and reference both in an aws_instance resource, then output the public IP. The workflow is terraform init, plan to review, apply to create, and destroy to clean up.*

Q10. What is drift in Terraform?

In simple words: Your recipe says the cake has 2 eggs, but someone secretly added a third egg to the real cake. The difference between the recipe and the real cake is drift.

Detailed answer

Drift is when the real infrastructure no longer matches what the Terraform code and state say. It usually happens because:

- Someone changed a resource manually in the console (opened a security group port, edited a bucket policy).
- Another tool or script modified it (autoscaling, a Lambda, another team's pipeline).
- A resource was deleted outside Terraform.

Why it matters: the next `terraform apply` may silently undo the manual fix, or fail, and your code is no longer the source of truth.

How you see it: `terraform plan` (or `terraform plan -refresh-only`) shows what changed outside Terraform, marked as "Objects have changed outside of Terraform".

Example:

```
Note: Objects have changed outside of Terraform

  # aws_security_group.web has changed
  ~ ingress = [
      + { from_port = 22, to_port = 22, cidr_blocks = ["0.0.0.0/0"] },
    ]
```

What to say to the interviewer: *Drift is the gap between the desired state in code and the actual state in the cloud, usually caused by manual console changes or other automation. I detect it with terraform plan or plan -refresh-only, ideally on a schedule in CI, and then either revert it by applying code or codify the change if it was intended.*

Q11. What is the use of the terraform refresh command?

In simple words: Refresh is Terraform re-reading the real world to update its notebook, without changing anything in the real world.

Detailed answer

- `terraform refresh` reads the current settings of every managed resource from the cloud and updates the state file to match reality. It does not change infrastructure.
- It is **deprecated** because it updates state silently with no review. The modern, safe way is `terraform plan -refresh-only` (preview) and `terraform apply -refresh-only` (accept the update after you review it).

- Normal `plan` and `apply` already do a refresh automatically first. You can skip that with `-refresh=false` for speed on huge setups.
- Use case: someone changed a resource manually and you want state to reflect it (then update code too), or you want to see drift without planning any changes.

Example:

```
terraform plan -refresh-only     # show what changed outside Terraform
terraform apply -refresh-only    # accept those changes into state only
```

> ***What to say to the interviewer:*** *terraform refresh syncs the state file with the real infrastructure without modifying resources. It is deprecated now, so I use terraform plan -refresh-only to review drift and apply -refresh-only to accept it into state, because that gives a reviewable step instead of a silent state change.*

Q12. What is a null_resource in Terraform?

> **In simple words:** A null_resource is an empty box that does not create anything in the cloud. You use it as a hook to run a script at the right moment.

Detailed answer

- `null_resource` (from the `hashicorp/null` provider) creates no real infrastructure. It is used together with **provisioners** (`local-exec`, `remote-exec`) to run commands, and with `triggers` to decide when to re-run.
- Typical uses: run a DB migration script after RDS is created, call an API, run `kubectl apply` once a cluster is up.
- Since Terraform 1.4, the built-in `terraform_data` resource does the same job without an extra provider, using `triggers_replace`.
- Caution: provisioners are a last resort. They are not idempotent and are hard to debug; prefer native resources, user_data, Ansible, or CI steps.

Example:

```
resource "terraform_data" "db_migrate" {
  triggers_replace = [aws_db_instance.app.id, var.schema_version]

  provisioner "local-exec" {
    command = "./migrate.sh ${aws_db_instance.app.address}"
  }
}
```

> ***What to say to the interviewer:*** *A null_resource creates nothing in the cloud; it acts as a container for provisioners so I can run a script at a specific point in the dependency graph, re-run using triggers. In newer Terraform I use terraform_data instead, and I keep provisioners as a last resort because they are not declarative.*

Q13. What is the GCP (and Azure) equivalent of AWS CloudFormation?

> **In simple words:** Every cloud has its own native recipe language. AWS has CloudFormation, Azure has ARM/Bicep, and Google has Deployment Manager and now Infrastructure Manager.

Detailed answer

Cloud	Native IaC	Notes
AWS	CloudFormation (plus CDK on top)	YAML/JSON templates, stacks
GCP	Deployment Manager (legacy) / Infrastructure Manager	Deployment Manager is deprecated; Infrastructure Manager is a managed service that runs Terraform
Azure	ARM templates / Bicep	Bicep is the friendlier language that compiles to ARM

Why many teams still pick Terraform: one language across all clouds, huge provider ecosystem, and on GCP Google itself recommends Terraform (via Infrastructure Manager).

Example:

```
# GCP Infrastructure Manager deploying a Terraform config from Git
gcloud infra-manager deployments apply projects/my-proj/locations/us-central1/deployments/net \
  --service-account=projects/my-proj/serviceAccounts/im-sa@my-proj.iam.gserviceaccount.com \
  --git-source-repo=https://github.com/org/infra --git-source-directory=network
```

> ***What to say to the interviewer:*** *The classic GCP equivalent is Deployment Manager, but Google has deprecated it and now recommends Infrastructure Manager, which is a managed way to run Terraform. On Azure the equivalent is ARM templates or Bicep. In multi-cloud setups I prefer Terraform because it is one workflow everywhere.*

Q14. What are some non-cloud-native alternatives to Terraform?

> **In simple words:** Terraform is one brand of recipe book. Others exist - some use real programming languages, some focus on configuring servers rather than creating them.

Detailed answer

- **OpenTofu** - open-source fork of Terraform (Linux Foundation), near drop-in compatible.
- **Pulumi** - IaC using real languages (TypeScript, Go, Python) with state like Terraform.
- **Crossplane** - manages cloud resources as Kubernetes custom resources (GitOps friendly).
- **Ansible** - mainly configuration management (install packages, configure servers), but it can create cloud resources too. Agentless, push-based.
- **Chef / Puppet / SaltStack** - configuration management, agent-based, pull model.
- **Terragrunt** - a wrapper around Terraform for DRY multi-environment setups (not a replacement).
- **AWS CDK / CDKTF** - write code that generates CloudFormation or Terraform.

Key distinction: **provisioning tools** (Terraform, Pulumi, Crossplane) create infrastructure; **configuration management tools** (Ansible, Chef, Puppet) configure what runs on it.

Example:

```
Terraform  -> create VPC, EC2, RDS
Ansible    -> install nginx, push config files, restart service on those EC2s
```

What to say to the interviewer: *Alternatives include OpenTofu, Pulumi, Crossplane for provisioning, and Ansible, Chef or Puppet for configuration management. A common pattern is Terraform to provision the infrastructure and Ansible to configure the servers, or baking images with Packer so no configuration is needed at boot.*

Q15. What is immutable infrastructure, and how do Packer and Terraform help?

In simple words: Instead of repairing an old car on the road, you build a fresh identical car in the factory and swap it in. Servers are replaced, never patched in place.

Detailed answer

- **Mutable**: SSH into servers and patch/upgrade them. Over time each server becomes slightly different ("snowflake servers") and failures are hard to reproduce.
- **Immutable**: bake a new machine image with everything installed, deploy new instances from it, and destroy the old ones.
- **Packer** builds the golden image (AMI) from a template, often using shell or Ansible provisioners.
- **Terraform** then points the Launch Template to the new AMI and an Auto Scaling Group instance refresh rolls it out.
- Benefits: consistent servers, easy rollback (previous AMI), faster boot, less drift, better security patching cadence.
- Containers are the same idea: build a new image tag, never change a running container.

Example:

```
# Packer (golden.pkr.hcl)
source "amazon-ebs" "base" {
  region        = "ap-south-1"
  instance_type = "t3.micro"
  source_ami    = "ami-0abc123"
  ssh_username  = "ubuntu"
  ami_name      = "web-{{timestamp}}"
}
build {
  sources = ["source.amazon-ebs.base"]
  provisioner "shell" { inline = ["sudo apt-get update", "sudo apt-get install -y nginx"] }
}
```

```
# Terraform picks the newest baked AMI
data "aws_ami" "web" {
  most_recent = true
  owners      = ["self"]
  filter {
    name   = "name"
    values = ["web-*"]
  }
}
```

What to say to the interviewer: *Immutable infrastructure means we replace servers instead of patching them. I bake a golden AMI with Packer, reference it in Terraform through a launch template, and roll it out with an ASG instance refresh, so every server is identical and rollback is just pointing back to the previous image.*

Advanced

Q16. How do you set up remote state and state locking, especially when multiple engineers modify infrastructure at the same time?

Also asked as: How do you avoid state conflicts? How do you lock state with S3 and DynamoDB?

Also asked as: Two engineers run terraform apply at the same time. What happens and how do you prevent it?

In simple words: Keep the notebook in a shared locker, not on anyone's laptop. When one person is writing in it, the locker is locked so nobody else can scribble at the same time.

Detailed answer

Remote state:

- Store state in a shared backend: **S3** (AWS), **GCS**, **Azure Blob**, or **HCP Terraform / Terraform Cloud**.
- Turn on **encryption** (SSE-KMS), **versioning** (to roll back a bad state), block public access, and restrict IAM access to the bucket.

Locking:

- While `plan`/`apply` runs, Terraform takes a lock. A second person gets `Error acquiring the state lock` and must wait.
- S3: since Terraform 1.10, **S3 native locking** (`use_lockfile = true`) is the recommended path - Terraform writes a `.tflock` object next to the state. The older **DynamoDB table** method (`dynamodb_table`, key `LockID`) still works on existing setups but is deprecated.
- GCS and HCP Terraform lock automatically.
- If a run crashes and leaves a stale lock, confirm nobody is running, then `terraform force-unlock <LOCK_ID>`.

Team process on top of locking:

- If two engineers apply at once with locking, the second one fails fast with `Error acquiring the state lock`. Without locking, both write state and one overwrites the other -> corrupted state, duplicates or orphans.
- Only CI applies (not laptops), one pipeline queue per state.
- Split big states by component/env so teams do not block each other.
- Plan on pull request, apply after merge (or tools like Atlantis).

Example:

```
terraform {
  backend "s3" {
    bucket       = "acme-tf-state"
    key          = "prod/network/terraform.tfstate"
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true          # Terraform 1.10+ native S3 lock
    # dynamodb_table = "tf-locks" # older approach
  }
}
```

```
terraform force-unlock 6f1b2c3d-...   # only after confirming no run is active
```

What to say to the interviewer: *I keep state in an encrypted, versioned S3 bucket with locking - native S3 lockfile on newer Terraform or a DynamoDB table on older setups - so only one apply can run at a time. On top of that, only CI applies, states are split per environment and component, and force-unlock is used only after confirming the lock is stale.*

Q17. How do you manage existing (unmanaged) AWS resources in Terraform? What is the difference between terraform import and an import block?

Also asked as: How do you bring manually created resources under Terraform?

In simple words: The resource already exists; you just need to write it into Terraform's notebook and write matching code, so Terraform adopts it instead of building a duplicate.

Detailed answer

Two ways to adopt existing resources:

	terraform import (CLI)	import block (Terraform 1.5+)
Where	One-off command	Written in .tf code
Reviewable in PR / plan	No, changes state immediately	Yes, shows in plan before apply
Many resources at once	One per command	Many blocks, also works with for_each
Generates config	No, you write code by hand	Yes, `-generate-config-out`

Steps (import block way):

1. Write an `import` block with the target address and the real ID.
2. Run `terraform plan -generate-config-out=generated.tf` to auto-write resource code.
3. Clean up the generated code (remove computed/default noise, use variables).
4. Run `plan` until it shows `1 to import, 0 to add, 0 to change, 0 to destroy`.
5. Apply, then you can delete the import block.

Other adoption tools: `terraformer` or `aws2tf` for bulk reverse-engineering.

Example:

```
import {
  to = aws_s3_bucket.assets
  id = "acme-prod-assets"
}
```

```
terraform plan -generate-config-out=generated.tf
# Old CLI way:
terraform import aws_s3_bucket.assets acme-prod-assets
```

What to say to the interviewer: *To manage unmanaged resources I write matching Terraform code and import them into state. The old terraform import command changes state immediately and one resource at a time, while the import block from Terraform 1.5 is declarative, shows up in the plan for review, and can even generate config, so I prefer the import block.*

Q18. How do you dynamically retrieve VPC details from AWS to create an EC2 instance using IaC? Write the code.

In simple words: Instead of hard-coding "subnet-123" in your recipe, you ask AWS "give me the VPC tagged prod and its private subnets" at run time. That question is a data source.

Detailed answer

- **Data sources** (`data` blocks) read existing information without managing it.
- Look up the VPC by tag, then its subnets, then the latest AMI, and feed those into the EC2 resource.
- This makes the same code work in every account/environment and avoids breaking when IDs change.

Example:

```
data "aws_vpc" "main" {
  filter {
    name   = "tag:Name"
    values = ["prod-vpc"]
  }
}

data "aws_subnets" "private" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.main.id]
  }
  tags = { Tier = "private" }
}

data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "app" {
  name   = "app-sg"
  vpc_id = data.aws_vpc.main.id
}

resource "aws_instance" "app" {
  ami                    = data.aws_ami.al2023.id
  instance_type          = "t3.micro"
  subnet_id              = data.aws_subnets.private.ids[0]
  vpc_security_group_ids = [aws_security_group.app.id]
  tags                   = { Name = "app-server" }
}
```

If the VPC is managed by another Terraform stack, you can instead read its outputs with `data "terraform_remote_state"`, or better, SSM parameters.

What to say to the interviewer: *I use data sources - aws_vpc filtered by tag, aws_subnets filtered by VPC ID and tier, and aws_ami for the latest image - and pass their IDs into the aws_instance resource. That way nothing is hard-coded and the same code works across environments.*

Q19. If you want to replace (recreate) a single resource in existing infrastructure, how would you do it?

In simple words: One brick in the wall is bad. You tell Terraform "replace just this brick", not rebuild the whole wall.

Detailed answer

- Modern way: `terraform apply -replace="ADDRESS"` - forces destroy-and-recreate of only that resource in one reviewed plan.
- Old way: `terraform taint ADDRESS` then apply. `taint` is deprecated because it changes state before you see a plan.
- Only touch one resource and its dependents: `-target=ADDRESS` (for emergencies only, not daily use).
- To avoid downtime while replacing, use `lifecycle { create_before_destroy = true }`.
- To make Terraform always replace when something changes: `lifecycle { replace_triggered_by = [...] }`.

Example:

```
terraform plan  -replace="aws_instance.web[1]"
terraform apply -replace="aws_instance.web[1]"
# Plan: 1 to add, 0 to change, 1 to destroy.
```

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true
  }
}
```

What to say to the interviewer: *I use terraform apply -replace with the resource address, which forces recreation of only that resource while showing me the plan first; taint is the older, deprecated way. If downtime matters I add create_before_destroy so the new one comes up before the old one is removed.*

Q20. How do you control precisely which resources are destroyed and which are kept?

Also asked as: Scenario: a resource was removed from the config but is still live. What happens and how do you handle it?

In simple words: You want to demolish the garage but keep the house. Terraform gives you tools to point at exactly one thing, to put a "do not demolish" sign on others, or to simply stop looking after something without knocking it down.

Detailed answer

Goal	Tool
Destroy only one resource	`terraform destroy -target=ADDRESS` (or remove it from code and apply) - use carefully
Never allow a resource to be destroyed by Terraform	`lifecycle { prevent_destroy = true }` - plan errors out
Stop managing a resource but keep it in the cloud	`terraform state rm ADDRESS`, or the `removed` block (TF 1.7+) with `destroy = false`
Protect at the cloud level too	`deletion_protection = true` (RDS, ALB), S3 object lock, termination protection
Limit blast radius	split state: databases and networking in their own root modules
Review before destroy	`terraform plan -destroy -out=destroy.plan` then apply that plan

The `removed` block is the reviewable, code-based way to "forget" a resource: the plan shows it will be removed from state, not destroyed.

Resource removed from config but still live:

- Common misconception: deleting a resource block does not make Terraform "forget" it. The resource is still in state, so the next `plan` shows it as **to destroy**, and apply deletes it.
- To keep it running but stop managing it: add a `removed` block with `destroy = false` (TF 1.7+) or run `terraform state rm`.
- `terraform apply -refresh-only` never destroys anything; it only updates state to match reality.
- Truly **orphaned** resources (live in the cloud but in no state file) come from `state rm`, failed applies that did not record state, or manual creation. Find them with tag-based inventories (e.g. every managed resource tagged `ManagedBy=terraform` + stack name), AWS Config / Resource Explorer, or driftctl-style tooling, then either import them or delete them.

Example:

```
removed {
  from = aws_instance.legacy_batch
  lifecycle {
    destroy = false   # remove from state, keep the real EC2
  }
}

resource "aws_db_instance" "main" {
  # ...
  deletion_protection = true
  lifecycle {
    prevent_destroy = true
  }
}
```

```
terraform plan -destroy -target=aws_instance.test_box -out=destroy.plan
terraform apply destroy.plan
terraform state rm aws_s3_bucket.shared_logs   # CLI way to stop managing it
```

What to say to the interviewer: *To destroy a single resource I remove it from code or use destroy with -target and review the saved plan. To keep resources, I use prevent_destroy and cloud-level deletion_protection, and when I want Terraform to stop managing something without deleting it I use a removed block with destroy = false or terraform state rm. Splitting state so databases live separately limits the blast radius even further.*

Q21. Besides depends_on, how do you control the order in which resources are deployed? Can you add a time delay?

In simple words: Terraform builds things in parallel unless one thing clearly needs another. You can show the need by pointing to it, add an explicit "wait for X", or even put in a "wait 30 seconds" timer.

Detailed answer

Ways to control order, from best to last resort:

1. **Implicit dependency (preferred)**: reference an attribute, e.g. `subnet_id = aws_subnet.a.id`. Terraform builds the graph automatically.
2. **depends_on**: for hidden dependencies Terraform cannot see (an IAM policy that must be attached before an app starts).
3. **time_sleep** (hashicorp/time provider): adds a delay after a resource is created or before it is destroyed. Useful for eventual consistency (IAM, DNS).
4. **terraform_data / null_resource with triggers**: run a `local-exec` wait loop, for example poll an endpoint until it returns 200.
5. **Separate root modules / pipeline stages**: network stack -> cluster stack -> apps stack. The strongest ordering, and it reduces blast radius.
6. `-target`: emergency tool only, not for routine ordering.
7. Provisioners in general: last resort, not declarative.

Example:

```
resource "aws_iam_role_policy_attachment" "app" {
  role       = aws_iam_role.app.name
  policy_arn = aws_iam_policy.app.arn
}

resource "time_sleep" "wait_iam" {
  depends_on      = [aws_iam_role_policy_attachment.app]
  create_duration = "20s"
}

resource "aws_lambda_function" "app" {
  depends_on    = [time_sleep.wait_iam]
  function_name = "app"
  role          = aws_iam_role.app.arn      # implicit dependency on the role
  runtime       = "python3.12"
  handler       = "app.handler"
  filename      = "app.zip"
}

resource "terraform_data" "wait_for_api" {
  triggers_replace = [aws_lb.api.dns_name]
  provisioner "local-exec" {
    command = "until curl -sf https://${aws_lb.api.dns_name}/health; do sleep 10; done"
  }
}
```

What to say to the interviewer: *I rely first on implicit dependencies through attribute references, use depends_on only for hidden dependencies, and for timing issues like IAM propagation I add a time_sleep resource with a create_duration. For waiting on something to become ready I can use terraform_data with a local-exec wait loop, and for strong ordering I split infrastructure into separate stacks applied in sequence.*

Q22. How do you design Terraform modules and keep code DRY?

Also asked as: How do you create and use Terraform modules?

In simple words: A module is a reusable Lego kit. Build a "VPC kit" once, then snap it into dev, stage and prod with different sizes instead of rewriting it three times.

Detailed answer

(For the list of files in a module folder, see the Basic question on Terraform folder files.)

Good module design:

- **Small and focused**: `vpc`, `eks`, `rds`, `alb` - not one giant "everything" module.
- **Clear interface**: `variables.tf` (inputs with types, descriptions, validation), `outputs.tf`, `main.tf`, `versions.tf`, `README.md`.
- **Sensible defaults**, but no hard-coded env names, account IDs, or regions.
- **Version the modules**: publish via Git tags or a private registry and pin `?ref=v1.4.0` so prod does not change when someone edits the module.
- **Compose** modules in thin "root" configs per environment.
- Use `for_each` / `count` and `locals` to avoid copy-paste.
- Test modules: `terraform validate`, `tflint`, `terraform test`, Checkov.
- Tools like **Terragrunt** remove repeated backend/provider blocks across many envs.

Example:

```
modules/
  vpc/ (main.tf variables.tf outputs.tf versions.tf)
  rds/
envs/
  dev/main.tf
  prod/main.tf
```

```
# envs/prod/main.tf
module "vpc" {
  source     = "git::https://github.com/acme/tf-modules.git//vpc?ref=v1.4.0"
  name       = "prod"
  cidr_block = "10.10.0.0/16"
  az_count   = 3
}
```

> ***What to say to the interviewer:*** *I build small, single-purpose modules with typed inputs and clear outputs, version them with Git tags, and compose them in thin root configurations per environment. Loops, locals and tools like Terragrunt keep it DRY, and every module is validated with tflint, Checkov and terraform test in CI.*

Q23. How do you manage separate environments (Dev, Test, Prod) in Terraform? Workspaces or separate directories?

Also asked as: How do you manage Dev, Staging and Prod with Terraform?

> **In simple words:** Dev and prod should have separate notebooks and separate lockers. Workspaces give separate notebooks in the same locker; separate folders or accounts give completely separate lockers.

Detailed answer

Options:

1. **Workspaces** - same code, separate state per workspace (`terraform workspace new prod`). Easy, but all envs share the same backend and credentials, and it is easy to apply to the wrong one. Good for short-lived or identical copies.
2. **Directory per environment** (most common in production) - `envs/dev`, `envs/prod`, each with its own backend key, tfvars, and provider credentials, all calling the same versioned modules.
3. **Terragrunt** - directory layout with DRY config.
4. **Separate AWS accounts per environment** (best isolation), combined with option 2.

Best practice:

- Different state files AND ideally different cloud accounts for prod.
- Same modules, different inputs (`dev.tfvars` small instances, `prod.tfvars` multi-AZ).
- Promotion: change is tested in dev, then the same module version is bumped in stage, then prod.
- Prod applies need approval in the pipeline.

Example:

```
envs/
  dev/   backend key: dev/terraform.tfstate   -> account 111111111111
  stage/ backend key: stage/terraform.tfstate -> account 222222222222
  prod/  backend key: prod/terraform.tfstate  -> account 333333333333
```

```
cd envs/prod && terraform init && terraform plan -var-file=prod.tfvars
```

> ***What to say to the interviewer:*** *I use one directory per environment, each with its own backend state, tfvars, and ideally its own AWS account, all reusing the same versioned modules. Workspaces are fine for quick identical copies, but for prod I want strong isolation and a promotion flow where the same module version moves from dev to stage to prod with approvals.*

Q24. How would you create resources across two different AWS accounts using Terraform?

> **In simple words:** You give Terraform two keycards - one for account A and one for account B - and label each resource with which keycard to use.

Detailed answer

- Define the `aws` provider twice using an **alias**.
- Each provider **assumes an IAM role** in its target account (no long-lived keys). The CI runner has base credentials only allowed to `sts:AssumeRole` into those roles.
- Point each resource or module to the right provider with `provider = aws.xxx` (or `providers = {}` for modules).
- Typical use cases: VPC peering between accounts, Route 53 records in a shared DNS account, cross-account S3 replication.

Example:

```
provider "aws" {
  alias  = "network"
  region = "ap-south-1"
  assume_role {
    role_arn = "arn:aws:iam::111111111111:role/TerraformDeploy"
  }
}

provider "aws" {
  alias  = "app"
  region = "ap-south-1"
  assume_role {
    role_arn = "arn:aws:iam::222222222222:role/TerraformDeploy"
  }
}

resource "aws_route53_record" "api" {
  provider = aws.network
  zone_id  = var.zone_id
  name     = "api.example.com"
  type     = "CNAME"
  ttl      = 300
  records  = [module.alb.dns_name]
}

module "alb" {
  source    = "./modules/alb"
  providers = { aws = aws.app }
}
```

What to say to the interviewer: *I define two aws providers with aliases, each using assume_role into a deploy role in its account, and then set provider on each resource or pass providers into modules. The pipeline only holds credentials that can assume those roles, so there are no long-lived keys and every action is audited per account.*

Q25. How do you manage sensitive values like secrets or API keys in Terraform without hardcoding them?

Also asked as: How do you pass secrets into Terraform safely?

In simple words: Never write the password in the recipe. Keep it in a safe (Secrets Manager / Vault) and let Terraform or the app open the safe at run time.

Detailed answer

Options, from good to best:

- Mark variables `sensitive = true` so they are hidden in plan output (they are still stored in state).
- Inject via CI secrets as `TF_VAR_db_password`, never in `.tfvars` committed to Git.
- Read from a secret store with a data source: AWS Secrets Manager / SSM Parameter Store, Vault, GCP Secret Manager.
- Let the cloud generate and hold the secret: e.g. RDS `manage_master_user_password = true` stores the password in Secrets Manager and it never appears in your code.
- Use `random_password` to generate, and store it straight into Secrets Manager.

- Newer Terraform (1.10+) adds ephemeral values and write-only arguments so some secrets never land in state at all.
- Always: encrypted, access-restricted remote state; `.gitignore` for `*.tfstate` and `*.tfvars`; secret scanning (gitleaks) in CI.

Example:

```
resource "aws_db_instance" "app" {
  identifier                  = "app-db"
  engine                      = "postgres"
  instance_class              = "db.t3.medium"
  allocated_storage           = 50
  username                    = "appadmin"
  manage_master_user_password = true   # password lives in Secrets Manager
}

data "aws_secretsmanager_secret_version" "api_key" {
  secret_id = "prod/payment/api-key"
}
```

> ***What to say to the interviewer:*** *I never hardcode secrets; I either inject them from CI as TF_VAR_ variables marked sensitive, read them from Secrets Manager or Vault with data sources, or better, let AWS generate and manage them like RDS managed master passwords. Because state can still contain secrets, the state bucket is encrypted with KMS and tightly access-controlled.*

Q26. How do you upgrade a GKE node pool version using Terraform (and manually)?

> **In simple words:** First upgrade the brain (control plane), then swap the workers (nodes) a few at a time, so the shop never closes.

Detailed answer

Order: control plane first, then node pools (nodes can be up to a couple of minor versions behind the master, never ahead).

Terraform way:

1. Bump `min_master_version` on `google_container_cluster` (or let a release channel handle it) and apply.
2. Bump `version` on `google_container_node_pool`, set `upgrade_settings` for surge upgrades, and apply. GKE drains and replaces nodes gradually.
3. For big jumps, use blue/green: create a new node pool with the new version, cordon/drain the old one, then remove it from code.
4. Many teams instead set `release_channel` + `auto_upgrade = true` with a maintenance window, and add `lifecycle { ignore_changes = [version] }` so Terraform does not fight auto-upgrades.

Manual way:

```
gcloud container clusters upgrade my-cluster --master --cluster-version=1.30 --region=asia-south1
gcloud container clusters upgrade my-cluster --node-pool=default-pool \
  --cluster-version=1.30 --region=asia-south1
```

Example:

```
resource "google_container_node_pool" "apps" {
  name     = "apps"
  cluster  = google_container_cluster.main.name
  location = "asia-south1"
  version  = "1.30.5-gke.1014001"

  upgrade_settings {
    max_surge       = 1
    max_unavailable = 0
  }

  management {
    auto_repair  = true
    auto_upgrade = false
  }
}
```

What to say to the interviewer: *I upgrade the control plane first, then change the version field on the google_container_node_pool resource with surge settings of max_surge 1 and max_unavailable 0, so nodes are replaced gradually and PodDisruptionBudgets keep apps up. For risky jumps I create a new node pool and drain the old one, and if we use release channels I ignore the version field so Terraform does not fight auto-upgrades.*

Q27. How do you create an EKS cluster using Terraform, and where do Kubernetes core components come into play?

Also asked as: When an EKS cluster is deployed via Terraform, what gets created? What is Karpenter used for?

In simple words: Terraform orders the building (network, cluster, worker nodes, permissions). AWS runs the brain of Kubernetes for you; your nodes are the workers.

Detailed answer

What Terraform creates (usually with the community `terraform-aws-modules/eks` module):

- VPC with private subnets for nodes and public subnets for load balancers (tagged for EKS).
- EKS cluster -> AWS runs and patches the **control plane** (API server, etcd, scheduler, controller manager) across 3 AZs.
- Managed node groups (EC2 in an Auto Scaling Group) -> these run **kubelet, kube-proxy, container runtime**.
- Add-ons: VPC CNI, CoreDNS, kube-proxy, EBS CSI driver.
- IAM: cluster role, node role, IRSA / Pod Identity for app permissions, access entries for who can use kubectl.

Also created: the **cluster security group** (control plane <-> nodes), the **OIDC issuer** URL (plus the IAM OIDC provider used by IRSA), **launch templates** for node groups, and access entries. Locally you get `terraform.tfstate` in the remote backend with its lock, and `.terraform.lock.hcl`.

The **kubeconfig** is not created by Terraform: you generate it with `aws eks update-kubeconfig`. Then deploy apps with Helm or Argo CD (keep app deployments separate from cluster Terraform).

Karpenter (short version): a node autoscaler that watches pending pods and launches right-sized EC2 instances directly in seconds, picks from many instance types including Spot, and consolidates

under-used nodes to save cost. Cluster Autoscaler only resizes pre-defined node groups, so it is slower and less flexible. The Kubernetes chapter covers it in depth.

Example:

```
module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  version         = "~> 20.0"
  cluster_name    = "prod-eks"
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets

  eks_managed_node_groups = {
    general = {
      instance_types = ["m6i.large"]
      min_size       = 2
      max_size       = 6
      desired_size   = 3
    }
  }
}
```

```
aws eks update-kubeconfig --name prod-eks --region ap-south-1
kubectl get nodes
```

> ***What to say to the interviewer:*** *I use the terraform-aws-modules EKS module on top of a VPC module; it creates the managed control plane, which AWS runs with the API server, etcd, scheduler and controllers, plus managed node groups that run kubelet and kube-proxy, and the core add-ons like VPC CNI and CoreDNS. Application deployment is kept separate, via Helm or Argo CD.*

Q28. You wrote Terraform for an EKS cluster plus a Kubernetes service account. The IAM role must exist before the cluster, and IRSA needs the OIDC provider, which only exists after the cluster. How do you solve the ordering?

> **In simple words:** It is a chain of dominoes. Terraform knows the order automatically if each piece points at the one before it, so you do not have to force anything.

Detailed answer

The correct chain:

1. **Cluster IAM role** (trusts `eks.amazonaws.com`) + policy attachment.
2. **EKS cluster** with `role_arn = aws_iam_role.cluster.arn` -> implicit dependency, the role is created first.
3. **IAM OIDC provider** using the cluster's issuer URL `aws_eks_cluster.this.identity[0].oidc[0].issuer` -> automatically created after the cluster.
4. **IRSA role** whose trust policy allows `sts:AssumeRoleWithWebIdentity` from that OIDC provider, with a condition on `<issuer>:sub = system:serviceaccount:<ns>:<sa>` (and `:aud = sts.amazonaws.com`).
5. **kubernetes_service_account** annotated with `eks.amazonaws.com/role-arn = <IRSA role ARN>`.

Key points:

- No `depends_on` needed: attribute references create the dependencies.
- Who can use kubectl: use **EKS access entries** (`aws_eks_access_entry` + access policy) instead of editing the old `aws-auth` ConfigMap; access entries are plain AWS API resources, so there is no chicken-and-egg.
- The **kubernetes provider** must be configured from cluster outputs (endpoint, CA, token). Providers cannot fully depend on resources in the same apply, so it is recommended to apply the cluster in one stack and Kubernetes objects (service accounts, Helm) in a separate apply/stack.
- Newer alternative: **EKS Pod Identity** (`aws_eks_pod_identity_association`) avoids the OIDC provider step entirely.

Example:

```
resource "aws_eks_cluster" "this" {
  name     = "prod-eks"
  role_arn = aws_iam_role.cluster.arn              # step 1 -> 2
  vpc_config { subnet_ids = var.private_subnets }
  access_config { authentication_mode = "API" }
}

data "tls_certificate" "oidc" {
  url = aws_eks_cluster.this.identity[0].oidc[0].issuer
}

resource "aws_iam_openid_connect_provider" "eks" {   # step 3
  url             = aws_eks_cluster.this.identity[0].oidc[0].issuer
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = [data.tls_certificate.oidc.certificates[0].sha1_fingerprint]
}

locals {
  issuer = replace(aws_iam_openid_connect_provider.eks.url, "https://", "")
}

data "aws_iam_policy_document" "irsa_trust" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.eks.arn]
    }
    condition {
      test     = "StringEquals"
      variable = "${local.issuer}:sub"
      values   = ["system:serviceaccount:app:app-sa"]
    }
    condition {
      test     = "StringEquals"
      variable = "${local.issuer}:aud"
      values   = ["sts.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "app_irsa" {               # step 4
  name               = "prod-app-irsa"
  assume_role_policy = data.aws_iam_policy_document.irsa_trust.json
}

resource "kubernetes_service_account" "app" {      # step 5 (ideally separate stack)
  metadata {
    name        = "app-sa"
    namespace   = "app"
    annotations = { "eks.amazonaws.com/role-arn" = aws_iam_role.app_irsa.arn }
  }
}
```

What to say to the interviewer: *I let Terraform order it through attribute references: the cluster references the cluster role ARN, the OIDC provider references the cluster issuer URL, the IRSA role trusts that OIDC provider with a sub condition for the service account, and the service account is annotated with the role ARN. I use EKS access entries instead of aws-auth, and I configure the kubernetes provider from cluster outputs in a separate stack, or use EKS Pod Identity to skip the OIDC step.*

Q29. Write Terraform code to deploy an EKS cluster, a Helm chart and the Datadog agent, and use the agent for monitoring.

In simple words: Terraform builds the cluster, then installs the Datadog "spy" on every node using Helm, then creates an alarm rule in Datadog - all from one codebase.

Detailed answer

Pieces:

- `terraform-aws-modules/eks` creates the cluster and nodes.
- The **helm provider** authenticates to the cluster using its endpoint, CA and `aws eks get-token`.
- `helm_release` installs the `datadog/datadog` chart; the API key comes from a **sensitive** variable (fed from CI secrets / Secrets Manager) via `set_sensitive`.
- Values enable **logs, APM and the process agent**.
- The **Datadog provider** creates a `datadog_monitor` so alerts are code too.
- In real setups, put the Helm and Datadog parts in a separate stack from the cluster (same provider-ordering reason as IRSA).

Example:

```
terraform {
  required_providers {
    aws     = { source = "hashicorp/aws", version = "~> 5.0" }
    helm    = { source = "hashicorp/helm", version = "~> 2.17" }
    datadog = { source = "DataDog/datadog", version = "~> 3.0" }
  }
}

variable "datadog_api_key" {
  type      = string
  sensitive = true
}
variable "datadog_app_key" {
  type      = string
  sensitive = true
}

module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  version         = "~> 20.0"
  cluster_name    = "prod-eks"
  cluster_version = "1.30"
  vpc_id          = var.vpc_id
  subnet_ids      = var.private_subnets
  enable_cluster_creator_admin_permissions = true
  eks_managed_node_groups = {
    general = { instance_types = ["m6i.large"], min_size = 2, max_size = 5, desired_size = 3 }
  }
}

provider "helm" {
  kubernetes {   # helm provider 3.x uses: kubernetes = { ... }
    host                   = module.eks.cluster_endpoint
    cluster_ca_certificate = base64decode(module.eks.cluster_certificate_authority_data)
    exec {
      api_version = "client.authentication.k8s.io/v1beta1"
      command     = "aws"
      args        = ["eks", "get-token", "--cluster-name", module.eks.cluster_name]
    }
  }
}

resource "helm_release" "datadog" {
  name             = "datadog"
  repository       = "https://helm.datadoghq.com"
  chart            = "datadog"
  namespace        = "datadog"
  create_namespace = true

  values = [yamlencode({
```

```
    datadog = {
      site          = "datadoghq.com"
      clusterName   = module.eks.cluster_name
      logs          = { enabled = true, containerCollectAll = true }
      apm           = { portEnabled = true }
      processAgent  = { enabled = true, processCollection = true }
    }
  })]

  set_sensitive {
    name  = "datadog.apiKey"
    value = var.datadog_api_key
  }
}

provider "datadog" {
  api_key = var.datadog_api_key
  app_key = var.datadog_app_key
}

resource "datadog_monitor" "pod_restarts" {
  name    = "[prod-eks] Pods restarting"
  type    = "query alert"
  query   = "change(max(last_5m),last_5m):sum:kubernetes_state.container.restarts{kube_cluster_name:
    prod-eks} by {kube_deployment} > 3"
  message = "Pods in {{kube_deployment.name}} are restarting. @teams-devops-alerts"
  monitor_thresholds {
    critical = 3
  }
}
```

```
export TF_VAR_datadog_api_key=... TF_VAR_datadog_app_key=...   # from CI secret store
terraform init && terraform plan -out=tfplan && terraform apply tfplan
kubectl get pods -n datadog    # agent DaemonSet + cluster agent
```

> ***What to say to the interviewer:*** *I create the cluster with the EKS module, configure the helm provider from the cluster endpoint, CA and aws eks get-token, and install the datadog chart with helm_release, passing the API key as a sensitive variable with set_sensitive and enabling logs, APM and process collection in the values. Then I use the Datadog provider to create monitors like a pod-restart alert, so the monitoring itself is code, and I keep the Helm layer in a separate stack from the cluster.*

Q30. How do you detect drift in Terraform and troubleshoot it?

> **In simple words:** Every night a guard compares the recipe with the real cake and raises a flag if anything differs.

Detailed answer

Detect:

- `terraform plan -refresh-only` - shows only what changed outside Terraform.
- `terraform plan -detailed-exitcode` in a scheduled CI job: exit code `0` = no changes, 2 = drift/changes, 1 = error. Alert on 2 (Teams/Slack).
- Managed drift detection: HCP Terraform, Spacelift, env0. (driftctl was used earlier but is no longer actively developed.)

- AWS Config / CloudTrail show who changed what and when.

Troubleshoot:

1. Read the plan diff: which resource and which attribute.
2. Check CloudTrail for the API call, user, and time.
3. Decide: was it an unwanted change (revert) or an intended hotfix (codify)?
4. Revert -> `terraform apply`. Codify -> update code until plan is clean, or `apply -refresh-only` then update code.
5. If a field is legitimately changed by something else (e.g. ASG desired count by autoscaler), add `lifecycle { ignore_changes = [...] }`.
6. Prevent: restrict console write access in prod (read-only roles, SCPs), all changes through PRs.

Example:

```
terraform plan -refresh-only -detailed-exitcode -input=false
echo $?   # 2 means drift found -> send alert
```

> ***What to say to the interviewer:*** *I run terraform plan with detailed-exitcode on a schedule in CI and alert when it returns 2, and I use CloudTrail to find who made the change. Then I either revert by applying the code or codify the change, use ignore_changes for fields managed by other systems, and prevent recurrence by making prod console access read-only.*

Q31. How do you integrate Terraform with CI/CD and safely deploy infrastructure changes to production?

> **In simple words:** Nobody changes production by hand. A robot shows everyone the exact plan, a human approves it, and the robot applies exactly that approved plan.

Detailed answer

Pipeline stages:

1. **Format and lint**: `terraform fmt -check`, `terraform validate`, `tflint`.
2. **Security / policy scan**: Checkov, tfsec/Trivy, and policy-as-code (OPA/Conftest or Sentinel) - e.g. "no public S3", "no deleting RDS".
3. **Plan** on every pull request; post the plan as a PR comment. Save it with `-out=tfplan`.
4. **Review and approval**: required reviewers plus a manual approval gate for prod (Jenkins input, GitHub environments).
5. **Apply the saved plan**: `terraform apply tfplan` - guarantees what was approved is what runs.
6. **Promote**: dev -> stage -> prod, same module version.
7. **Post-apply checks**: smoke tests, monitoring, and a scheduled drift check.

Safety guardrails:

- CI uses short-lived credentials (OIDC to an IAM role), no static keys.
- Fail the pipeline if the plan contains destroys of protected resource types unless explicitly approved.

- `prevent_destroy` and cloud-side `deletion_protection` on critical resources.
- Small, frequent changes; separate states per component to reduce blast radius.
- Schedule risky changes in a maintenance window with a rollback plan.

Example:

```
terraform fmt -check && terraform validate && tflint
checkov -d .
terraform plan -out=tfplan -input=false
terraform show -json tfplan > plan.json
conftest test plan.json          # OPA policies, e.g. deny destroy of aws_db_instance
# ... manual approval ...
terraform apply -input=false tfplan
```

> ***What to say to the interviewer:*** *My Terraform pipeline runs fmt, validate, tflint and Checkov, then generates a saved plan on every pull request, checks it against OPA policies such as no destroys on databases, and requires approval for prod. After approval it applies exactly that saved plan using OIDC short-lived credentials, and changes are promoted from dev to stage to prod.*

Q32. What are RPO and RTO, and how does Terraform / IaC help you meet them in disaster recovery?

Also asked as: Have you worked on DR? What is RTO vs RPO?

> **In simple words:** RPO is how much data you can afford to lose (for example "at most 5 minutes of orders"). RTO is how fast you must be back up (for example "running again within 1 hour"). Backups protect the RPO; being able to rebuild quickly protects the RTO.

Detailed answer

- **RPO (Recovery Point Objective)** - maximum acceptable data loss, measured in time.
- **RTO (Recovery Time Objective)** - maximum acceptable downtime until service is restored.

RPO - Terraform does not back up your data. Data RPO comes from database backups, point-in-time recovery (PITR), and cross-region replication. Terraform's job is to **configure** those consistently:

- `backup_retention_period` on RDS (enables automated backups + PITR).
- `aws_backup_plan` with a cross-region `copy_action`.
- S3 Cross-Region Replication (`aws_s3_bucket_replication_configuration`).
- **State RPO**: the state file itself needs a versioned remote backend (S3 versioning, or Azure Blob versioning plus soft delete) so you can always restore it.
- Note: running backup scripts through provisioners is an anti-pattern. Use managed backup services instead.

RTO - this is where IaC shines:

- Modular, **region-parameterized** code so the same stack can be applied in the DR region just by changing a variable.
- A CI pipeline ready to rebuild the stack in the DR region.
- Pre-provisioned **pilot light / warm standby** resources (replica DB, minimal nodes) already defined in code, to cut rebuild time.
- Monitoring and alerts defined in code so DR comes up already observable.

- **Regular DR drills** where you actually time the rebuild and compare it with the RTO target.

Example:

```
resource "aws_backup_plan" "db" {
  name = "prod-db-daily"
  rule {
    rule_name         = "daily"
    target_vault_name = aws_backup_vault.primary.name
    schedule          = "cron(0 2 * * ? *)"
    lifecycle { delete_after = 35 }
    copy_action {
      destination_vault_arn = aws_backup_vault.dr.arn   # vault in the DR region
      lifecycle { delete_after = 35 }
    }
  }
}

resource "aws_db_instance" "primary" {
  identifier              = "prod-db"
  backup_retention_period = 7          # automated backups + PITR
  # ...
}

resource "aws_db_instance" "dr_replica" {
  provider            = aws.dr          # provider alias for the DR region
  identifier          = "prod-db-dr"
  replicate_source_db = aws_db_instance.primary.arn
  instance_class      = "db.r6g.large"
}
```

> ***What to say to the interviewer:*** *RPO is how much data we can afford to lose and RTO is how quickly we must recover. Terraform does not back up data, but I use it to configure backups, PITR, AWS Backup cross-region copies and replicas for RPO, and I keep state in a versioned backend. For RTO I keep region-parameterized modules, a pipeline that can rebuild in the DR region, pilot-light resources, and I run timed DR drills to prove we meet the target.*

Q33. How do you refactor Terraform code (rename resources, move them into modules) without downtime or recreation?

Also asked as: What happens if you rename a resource in Terraform? How do you use terraform state mv?

> **In simple words:** Terraform tracks every real resource by its "address" in the code, like a house tracked by its street address. If you rename the street without telling the post office, it thinks the old house was demolished and a new one must be built. A `moved` block is the change-of-address form.

Detailed answer

- If you simply rename `aws_instance.web` to `aws_instance.app`, the plan shows **destroy + create**, because the old address disappears from the config and a new one appears.
- `moved` **block (Terraform 1.1+, preferred):** declare the rename in code. It is reviewed in the PR, works for every teammate and pipeline automatically, and can be removed later.
- `terraform state mv` **(imperative):** edits the state directly, once. It's fine for a quick fix, but it is not visible in code review, and every other workspace or environment needs the same command.
- The same applies when you change a `count` resource to `for_each` (index `[0]` -> key `["web"]`), or move resources into a module.

- Always confirm the plan says **"has moved to"** and shows **0 to destroy** before applying. Keep refactor PRs separate from functional changes.

Example:

```
# Renamed a resource and moved another into a module - no recreation
moved {
  from = aws_instance.web
  to   = aws_instance.app
}

moved {
  from = aws_s3_bucket.logs
  to   = module.logging.aws_s3_bucket.this
}

moved {
  from = aws_subnet.private[0]          # count -> for_each migration
  to   = aws_subnet.private["ap-south-1a"]
}
```

```
terraform plan
#  # aws_instance.web has moved to aws_instance.app
#  Plan: 0 to add, 0 to change, 0 to destroy.

# Imperative alternative (one-off, not reviewable):
terraform state mv aws_instance.web aws_instance.app
```

> ***What to say to the interviewer:*** *Renaming a resource or moving it into a module changes its address, so without telling Terraform the plan would destroy and recreate it. I add* `moved` *blocks in the same PR so the refactor is reviewed and applied the same way everywhere, and I check that the plan shows "has moved" and zero destroys.* `terraform state mv` *is my fallback for one-off fixes.*

Q34. How do you perform zero-downtime updates with Terraform?

> **In simple words:** When you replace a bridge, you build the new one next to it, move the traffic across, and only then knock down the old one. By default Terraform knocks down first and builds second.

Detailed answer

- **Know which changes force replacement.** In the plan, `-/+` or "forces replacement" means destroy then create, which is downtime by default. Examples: changing an instance's AMI or subnet, an RDS identifier, some launch template settings.
- `lifecycle { create_before_destroy = true }`: Terraform creates the new resource, repoints dependencies, then deletes the old one. Names must be unique, so use `name_prefix` instead of a fixed `name`.
- **Auto Scaling Groups:** change the launch template version and use `instance_refresh` with `min_healthy_percentage`, so instances roll one batch at a time behind the load balancer health checks.
- **Blue-green with Terraform:** two target groups (or two ASGs) and a weighted ALB listener rule; shift the weights in steps through variables.
- **Databases:** prefer in-place modifications, keep `apply_immediately = false` for the maintenance window, and use RDS Blue/Green Deployments for engine upgrades.

- Protect critical resources with `prevent_destroy`, and review every plan for unexpected replacements.

Example:

```
resource "aws_launch_template" "app" {
  name_prefix   = "app-"
  image_id      = var.ami_id
  instance_type = "t3.medium"
  lifecycle { create_before_destroy = true }
}

resource "aws_autoscaling_group" "app" {
  name_prefix         = "app-"
  min_size            = 2
  max_size            = 6
  vpc_zone_identifier = var.private_subnet_ids
  target_group_arns   = [aws_lb_target_group.app.arn]
  health_check_type   = "ELB"

  launch_template {
    id      = aws_launch_template.app.id
    version = aws_launch_template.app.latest_version
  }

  instance_refresh {
    strategy = "Rolling"
    preferences { min_healthy_percentage = 90 }
  }
}
```

What to say to the interviewer: *First I read the plan for anything marked "forces replacement". For those I use* `create_before_destroy` *with* `name_prefix`*, so the new resource exists before the old one is removed. For compute behind a load balancer I roll through an ASG instance refresh that keeps 90% healthy, and for risky changes I do blue-green with weighted target groups. Databases get in-place changes or RDS Blue/Green.*

Q35. What are circular dependencies in Terraform, and how do you fix them?

In simple words: Two people each saying "I'll sign the contract after you sign it". Nobody can go first, so nothing happens. Terraform stops with "Error: Cycle".

Detailed answer

- Terraform builds a dependency graph from references (and `depends_on`). If A needs B and B needs A, there is no valid order, and plan fails with `Error: Cycle: aws_security_group.a, aws_security_group.b`.
- **Classic case:** two security groups with inline rules that reference each other (app allows DB, DB allows app).
- **Fix: split the relationship into separate resources.** Create both groups with no rules, then add the rules as standalone `aws_vpc_security_group_ingress_rule` resources. The groups no longer depend on each other; the rules depend on both.
- Other fixes: move a value into a variable or data source instead of a resource reference, remove an unnecessary `depends_on`, or split modules whose outputs feed each other.
- Use `terraform graph | dot -Tsvg > graph.svg` to see the loop.

Example:

```
# BAD: inline rules referencing each other -> Error: Cycle
# resource "aws_security_group" "app" { ingress { security_groups = [aws_security_group.db.id] } }
# resource "aws_security_group" "db"  { ingress { security_groups = [aws_security_group.app.id] } }

# GOOD: groups first, rules as separate resources
resource "aws_security_group" "app" {
  name_prefix = "app-"
  vpc_id      = var.vpc_id
}
resource "aws_security_group" "db" {
  name_prefix = "db-"
  vpc_id      = var.vpc_id
}

resource "aws_vpc_security_group_ingress_rule" "db_from_app" {
  security_group_id            = aws_security_group.db.id
  referenced_security_group_id = aws_security_group.app.id
  from_port                    = 5432
  to_port                      = 5432
  ip_protocol                  = "tcp"
}
```

What to say to the interviewer: *A cycle happens when two resources reference each other, so Terraform can't decide what to create first and fails with "Error: Cycle". The typical case is two security groups with inline rules pointing at each other. I fix it by creating the groups without rules and adding the rules as separate resources, which breaks the loop.* `terraform graph` *helps me see where the loop is.*

Scenario-Based

Q36. A resource was manually changed in AWS (for example someone added an S3 bucket policy to a Terraform-managed bucket), and Terraform shows drift. How do you handle it using IaC?

Also asked as: If someone manually changed an S3 bucket policy that was originally created with Terraform, how would you deal with that drift?

Also asked as: Someone changed a resource manually in the console. How do you fix the drift?

In simple words: Either put the cake back the way the recipe says, or, if the change was good, update the recipe. Never leave them different.

Detailed answer

Steps:

1. **See the drift**: `terraform plan -refresh-only` shows the policy change.
2. **Find the reason**: CloudTrail `PutBucketPolicy` event - who, when, why (maybe an urgent access fix).
3. **Decide**:

- Unwanted / insecure (e.g. made bucket public) -> run `terraform apply` to overwrite it with the code version. If the policy resource was never in code, add an `aws_s3_bucket_policy` resource with the correct policy so Terraform owns it.

- Intended change -> copy the policy into code (`aws_s3_bucket_policy` + `aws_iam_policy_document`), raise a PR, and plan until "No changes".
- If the policy object exists but is not in state -> `import` it.

4. **Prevent**: prod console is read-only, SCP or IAM deny on `s3:PutBucketPolicy` except for the Terraform role, scheduled drift detection, AWS Config rule for public buckets.

Note: in AWS provider v4+, bucket policy is a separate resource `aws_s3_bucket_policy`; do not also define it inline elsewhere or two resources will fight.

Example:

```
data "aws_iam_policy_document" "assets" {
  statement {
    actions   = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.assets.arn}/*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:aws:iam::222222222222:role/app"]
    }
  }
}

resource "aws_s3_bucket_policy" "assets" {
  bucket = aws_s3_bucket.assets.id
  policy = data.aws_iam_policy_document.assets.json
}
```

```
terraform import aws_s3_bucket_policy.assets acme-prod-assets   # if not yet managed
terraform plan   # goal: No changes
```

> ***What to say to the interviewer:*** *I first run plan -refresh-only and check CloudTrail to see who changed the policy and why. If the change was wrong I re-apply the code to revert it; if it was needed I codify it in an aws_s3_bucket_policy resource, import it if needed, and get the plan to show no changes. Then I lock down console write access and schedule drift detection so it does not happen again.*

Q37. There is an existing VPC created manually. How would you import it into Terraform without breaking anything? What are the prerequisites and how do you pass the VPC ID?

Also asked as: What are the prerequisites before importing a VPC? How do you pass arguments to a VPC with terraform import?

> **In simple words:** You are adopting a house that is already built. You measure it, draw the blueprint exactly as it is, and register it - you do not knock anything down.

Detailed answer

Prerequisites:

- Terraform and AWS provider configured with credentials that can read the VPC (and backend set up for state).
- The real IDs: VPC ID (`vpc-0abc...`), plus IDs of subnets, route tables, IGW, NAT, SGs you also want to manage.

- Know the current settings (CIDR, DNS support, tags) - get them from `aws ec2 describe-vpcs`.
- Take a snapshot of current config and decide scope (import only the VPC, or all its children).

How the ID is passed: `terraform import` takes two arguments - the resource address in code and the real ID. You do not pass CIDR etc. on the command line; those come from your code, which must match reality.

Safe steps:

1. Write an `import` block (or empty resource for the CLI method).
2. `terraform plan -generate-config-out=vpc.tf` to generate matching code.
3. Tidy the code; keep values identical to the real VPC.
4. `terraform plan` must show `1 to import, 0 to add, 0 to change, 0 to destroy`. Any "must be replaced" (for example a CIDR mismatch) means your code is wrong - fix it, never apply.
5. Apply, then import subnets, route tables, IGW the same way.
6. Add `lifecycle { prevent_destroy = true }` for safety.

Example:

```
import {
  to = aws_vpc.main
  id = "vpc-0abc123def4567890"
}

resource "aws_vpc" "main" {
  cidr_block           = "10.20.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags                 = { Name = "legacy-prod-vpc" }

  lifecycle {
    prevent_destroy = true
  }
}
```

```
# CLI alternative: address + real ID are the two arguments
terraform import aws_vpc.main vpc-0abc123def4567890
terraform plan    # must be: 0 to add, 0 to change, 0 to destroy
```

> ***What to say to the interviewer:*** *I gather the real VPC ID and its settings, write an import block or run terraform import with the resource address and the VPC ID, and generate or write matching code. I only apply once the plan shows zero changes and zero destroys, then repeat for subnets and route tables, and protect it with prevent_destroy.*

Q38. Your Terraform state file was accidentally deleted, lost, or got corrupted / out of sync. How do you recover it safely? What are the risks and how do you protect it?

Also asked as: A team member accidentally deleted the state file. What immediate steps prevent drift or resource loss?

Also asked as: Scenario: the state file got deleted. What do you do?

In simple words: If the notebook is lost, Terraform thinks nothing exists and will try to build everything again. You either restore the notebook from a backup copy or rewrite it by re-adopting each resource.

Detailed answer

Immediate: **freeze all applies first** - lock or pause the pipeline (and keep/obtain the state lock) so nobody runs an apply against an empty state and creates duplicates or orphans. Then restore the **previous S3 object version** of the state before doing anything else.

Recovery options:

1. **Restore from backend versioning** (best): S3 versioning -> restore the previous object version; GCS object versioning; HCP Terraform keeps state history.
2. **Local backup**: `terraform.tfstate.backup` or a copy from `terraform state pull` done earlier.
3. **Corrupted but present**: `terraform state pull > broken.json`, fix JSON carefully (or restore an older version), bump the serial, `terraform state push`.
4. **No backup at all**: rebuild state by importing every resource (import blocks, or tools like terraformer) until `plan` shows no changes.
5. **Out of sync**: `terraform plan -refresh-only`, then `state rm` for things that no longer exist and `import` for things missing from state.

Risks: duplicate resources, apply trying to recreate or delete prod, orphaned resources nobody tracks, secrets exposed if state leaks.

Protection:

- Remote backend with **versioning**, KMS encryption, MFA delete / object lock, bucket deletion protection.
- Locking to prevent corruption from parallel runs.
- Least-privilege IAM on the state bucket; only CI can write.
- Separate state per env/component (smaller blast radius).
- Never edit state by hand; use `state mv/rm/import`.

Example:

```
aws s3api list-object-versions --bucket acme-tf-state --prefix prod/network/terraform.tfstate
aws s3api get-object --bucket acme-tf-state --key prod/network/terraform.tfstate \
  --version-id 3HL4kqtJlcpXroDTDmJ.rmSpXd3dIbrHY restored.tfstate
terraform state push restored.tfstate
terraform plan    # verify: No changes
```

What to say to the interviewer: *First I freeze all pipelines, then restore the last good version from S3 versioning and push it, and verify with a plan that shows no changes. If there is no backup I rebuild the state by importing each resource. To protect it, the backend has versioning, encryption, locking, restricted access, and state is split per environment so any damage stays small.*

Q39. A terraform apply fails midway in production after creating some resources. What would be your next steps?

Also asked as: Scenario: terraform apply failed halfway. How do you recover?

In simple words: The builder stopped halfway. Do not panic and restart from scratch; check what got built, fix the problem, and let Terraform finish the rest.

Detailed answer

Terraform is not transactional - there is no automatic rollback. Whatever was created before the error is already recorded in state.

Steps:

1. **Do not re-run blindly.** Read the error (quota limit, IAM permission, name conflict, timeout, API throttling, dependency).
2. **Check the lock**: if the process crashed, a stale lock may remain -> confirm no run is active, then `force-unlock`.
3. **Check state vs reality**: `terraform plan` shows what is done and what is still pending. Look for resources marked tainted (created but failed during setup) - Terraform will replace them.
4. **Check for orphans**: if a resource was created in the cloud but the crash stopped state from being saved, it is not in state -> import it (otherwise the next apply fails with "already exists" or creates a duplicate).
5. **Fix the root cause**: raise quota, fix IAM, fix the config.
6. **Re-run plan, review, apply** - Terraform continues from where it stopped.
7. If the partial change is hurting production, roll back by reverting the code commit and applying the previous version (or restoring state + infra).
8. Afterwards: add smaller changes, pre-checks (quotas, permissions), and test in stage first.

Example:

```
Error: creating EC2 Instance: VcpuLimitExceeded: You have requested more vCPU capacity...
```

```
terraform plan          # 2 already created, 3 still to add
# fix quota / config, then
terraform apply
```

What to say to the interviewer: *Terraform does not roll back, so I first read the error, clear a stale lock if needed, and run plan to see exactly what was created and what is pending, importing any orphaned resources that were created but not recorded. After fixing the root cause I re-plan and apply to finish, or revert the commit and apply if the partial state is hurting production.*

Q40. terraform plan shows that critical resources will be deleted unexpectedly. What would you do?

Also asked as: During apply, production resources are about to be recreated. How do you detect and stop it before impact?

In simple words: The builder says "I will tear down your main pillar". Stop, do not approve, and find out why he thinks that.

Detailed answer

1. **Do not apply.** Stop the pipeline.
2. Read the plan: is it `destroy` or `must be replaced` (`-/+`)? The plan tells which attribute "forces replacement".

3. Common causes and fixes:

Cause	Fix
Resource renamed / moved into a module	`moved` block or `terraform state mv`
count index shifted (item removed from a list)	switch to `for_each` with stable keys + `moved`
Immutable field changed (DB identifier, AZ, CIDR, name)	revert the change or plan a proper migration
Wrong workspace / wrong backend / wrong tfvars	check `terraform workspace show` and backend key
Provider major upgrade changed behavior	pin version, read upgrade guide
Resource removed from code but still needed	restore the code; to just stop managing, use a `removed` block or `state rm`
Drift - someone changed it manually	codify or revert

4. Re-plan until destroy count is 0 for critical resources.
5. Prevent: `lifecycle { prevent_destroy = true }`, `deletion_protection = true` on RDS/ALB, policy checks in CI that fail on destroys, mandatory reviewer for prod.

Detecting and stopping it before impact:

- Read the plan symbols: `-/+` means destroy then create, `+/-` means create before destroy, and the attribute line says `# forces replacement`. The summary line shows `N to destroy`.
- Always `terraform plan -out=tfplan`, review it, and apply only that saved plan - nothing new can sneak in between review and apply.
- CI policy gate: `terraform show -json tfplan` + conftest/OPA fails the pipeline if any action contains `delete` for protected types (RDS, S3, EKS).
- lifecycle options: `prevent_destroy` makes the plan error out; `create_before_destroy` avoids downtime when replacement is truly needed; `ignore_changes` stops replacement for attributes changed by other systems.
- If you notice it mid-apply, press **Ctrl+C once**: Terraform stops starting new operations, lets in-flight ones finish, saves state and releases the lock. Pressing it a second time forces exit and can leave state inconsistent and the lock stuck, so avoid that.

Example:

```
moved {
  from = aws_db_instance.db
  to   = module.database.aws_db_instance.this
}

resource "aws_db_instance" "this" {
  # ...
  deletion_protection = true
  lifecycle {
    prevent_destroy = true
  }
}
```

> ***What to say to the interviewer:*** *I never apply; I read the plan to find which attribute forces replacement or why the resource is being destroyed, which is usually a rename, a count index shift, an immutable field change or the wrong workspace. I fix it with moved blocks or state mv until the plan shows no destroy, and I protect critical resources with prevent_destroy, deletion_protection and*

a CI policy that blocks destroys.

Q41. A Terraform script bypassed validation and deleted a production database (or other prod resources) during peak business hours. What is your action plan, and how do you contain the blast radius?

Also asked as: How would you troubleshoot and contain the blast radius if Terraform accidentally deleted production resources?

In simple words: First stop the fire from spreading, then bring the patient back from the latest backup, keep everyone informed, and afterwards fix the process so nobody can do it again.

Detailed answer

Phase 1 - Contain (first minutes):

1. Declare an incident, open a war room, assign an incident commander and a communicator (status page / stakeholders).
2. **Stop further damage**: cancel running pipelines, disable the Terraform CI job, revoke/limit the Terraform role, keep the state lock.
3. Find scope: what else was in the plan? Check apply logs and CloudTrail (`DeleteDBInstance`, who/when).

Phase 2 - Recover:

4. Restore the database:

- RDS: restore from the **final snapshot** (if `skip_final_snapshot = false`), latest automated snapshot, or **point-in-time restore** to just before deletion. Retained automated backups can exist after deletion if configured.
- Aurora: restore cluster from snapshot / backtrack if enabled.
- Or promote a cross-region read replica / AWS Backup copy.

5. Point the app to the restored endpoint (same identifier, or update DNS/secret), put the app in maintenance mode meanwhile.
6. Re-align Terraform: import the restored DB into state so the next plan is clean.
7. Validate data (row counts, last transactions), then reopen traffic and monitor.

Phase 3 - Prevent (post-mortem, blameless):

- `deletion_protection = true` and `prevent_destroy` on all stateful resources; `skip_final_snapshot = false`.
- Split state: databases in their own state, rarely touched.
- Policy as code (OPA/Sentinel) that fails any plan with destroy on data stores.
- Mandatory plan review + approval; apply only saved plans; no applies from laptops; freeze during peak hours.
- IAM guardrails: SCP or deny `rds:DeleteDBInstance` for the pipeline role except via break-glass.
- Tested backups: AWS Backup, cross-region/cross-account copies, regular restore drills.

Example:

```
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-db \
  --target-db-instance-identifier prod-db-restored \
  --restore-time 2026-09-20T10:14:00Z
terraform import aws_db_instance.prod prod-db-restored
```

```
{
  "Effect": "Deny",
  "Action": ["rds:DeleteDBInstance", "rds:DeleteDBCluster"],
  "Resource": "*",
  "Condition": { "StringNotLike": { "aws:PrincipalArn": "arn:aws:iam::*:role/BreakGlass" } }
}
```

> ***What to say to the interviewer:*** *I would first contain it by declaring an incident, stopping all pipelines and restricting the Terraform role, then restore the database from the final snapshot or point-in-time recovery, reconnect the app, and import the restored DB back into state. In the post-mortem I would add deletion_protection and prevent_destroy, move databases to their own state, block destroys with OPA policies and an IAM deny, and require approved saved plans.*

Q42. Terraform creates an IAM role and immediately uses it, but the apply fails even though the trust policy and permissions are correct. Why, and how do you fix it?

> **In simple words:** IAM is a global notice board. When you pin a new notice, it takes a few seconds before every office in the world can read it. If the next step runs too fast, it cannot see the notice yet.

Detailed answer

Cause: **IAM eventual consistency**. IAM changes are replicated globally; a new role, trust policy or policy attachment can take a few seconds to become visible to other services (Lambda, EKS, EC2 instance profiles, ECS).

Typical error:

```
Error: creating Lambda Function (app): InvalidParameterValueException:
The role defined for the function cannot be assumed by Lambda.
```

Other variants: `InvalidInstanceProfile` / "Invalid IAM Instance Profile name" on EC2, access denied right after attaching a policy.

Fixes:

1. **Re-run first**: the second apply usually succeeds - confirms it is propagation, not a config bug.
2. The AWS provider already has **built-in retries** for many of these cases; keep the provider up to date.
3. Add a **time_sleep** after the role/policy attachment and make the consumer depend on it.
4. Use **depends_on** so the consumer waits for the policy attachment, not just the role (a common real bug: the Lambda referenced the role, but not the attachment).
5. In scripts/pipelines, **retry with backoff** instead of failing immediately.
6. Longer term: create shared IAM roles in an earlier stack so they already exist when apps are deployed.

Example:

```
resource "aws_iam_role_policy_attachment" "lambda_basic" {
  role       = aws_iam_role.lambda.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

resource "time_sleep" "iam_propagation" {
  depends_on      = [aws_iam_role_policy_attachment.lambda_basic]
  create_duration = "15s"
}

resource "aws_lambda_function" "app" {
  depends_on    = [time_sleep.iam_propagation]
  function_name = "app"
  role          = aws_iam_role.lambda.arn
  runtime       = "python3.12"
  handler       = "app.handler"
  filename      = "app.zip"
}
```

> ***What to say to the interviewer:*** *That is IAM eventual consistency: the role exists but has not propagated to the service yet, which gives errors like "The role defined for the function cannot be assumed by Lambda". I confirm it by re-running, then fix it by making the consumer depend on the policy attachment and adding a short time_sleep, relying on the provider's built-in retries, and in the long run creating shared IAM roles in an earlier stack.*

Q43. Scenario: you need to create dynamic IAM roles across multiple environments. What is your module strategy?

> **In simple words:** Build one "role factory" module. Each environment hands it a shopping list of roles, and the factory stamps out correctly named, safely limited roles for that environment.

Detailed answer

Strategy:

1. **One reusable** `iam-role` **module** that takes a **map of roles** as input (name -> trusted principals, managed policies, inline policy statements).
2. Inside, `for_each` over the map, so adding or removing a role never shifts others.
3. **Env-prefixed names** (`dev-app-reader`, `prod-app-reader`) built from a `var.env` in locals, plus common tags.
4. A **permissions boundary** on every role, so even a bad policy cannot exceed the allowed maximum.
5. Policies built with **aws_iam_policy_document** (validated HCL, no hand-written JSON strings).
6. **Per-environment tfvars** (`dev.tfvars`, `prod.tfvars`) holding the role map for that env.
7. **One module call per account** using a provider alias with assume_role (dev account, prod account).
8. Guardrails: CI policy check (no `*:*`, no wildcard principals), pinned module version, review on changes.

Example:

```
# modules/iam-role/main.tf
variable "env" { type = string }
variable "permissions_boundary_arn" { type = string }
variable "roles" {
  type = map(object({
    trusted_services = list(string)
    policy_arns      = list(string)
  }))
}

data "aws_iam_policy_document" "trust" {
  for_each = var.roles
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = each.value.trusted_services
    }
  }
}

resource "aws_iam_role" "this" {
  for_each             = var.roles
  name                 = "${var.env}-${each.key}"
  assume_role_policy   = data.aws_iam_policy_document.trust[each.key].json
  permissions_boundary = var.permissions_boundary_arn
  tags                 = { Env = var.env, ManagedBy = "terraform" }
}

locals {
  attachments = merge([
    for role, cfg in var.roles : {
      for arn in cfg.policy_arns : "${role}|${arn}" => { role = role, arn = arn }
    }
  ]...)
}

resource "aws_iam_role_policy_attachment" "this" {
  for_each   = local.attachments
  role       = aws_iam_role.this[each.value.role].name
  policy_arn = each.value.arn
}

output "role_arns" {
  value = { for k, r in aws_iam_role.this : k => r.arn }
}
```

```
# envs/prod/main.tf
module "iam_roles" {
  source                   = "git::https://github.com/acme/tf-modules.git//iam-role?ref=v2.1.0"
  providers                = { aws = aws.prod }
  env                      = "prod"
  permissions_boundary_arn = "arn:aws:iam::333333333333:policy/DevOpsBoundary"
  roles                    = var.roles   # from prod.tfvars
}
```

```
# prod.tfvars
roles = {
  app-lambda = {
    trusted_services = ["lambda.amazonaws.com"]
    policy_arns      = ["arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"]
  }
  batch-ec2 = {
    trusted_services = ["ec2.amazonaws.com"]
    policy_arns      = ["arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"]
  }
}
```

> ***What to say to the interviewer:*** *I build one reusable iam-role module that takes a map of roles and uses for_each, with env-prefixed names, a mandatory permissions boundary and policies written with aws_iam_policy_document. Each environment has its own tfvars with the role map, and I call the module once per account through a provider alias, with the module version pinned and CI policy checks blocking wildcard permissions.*

Q44. Scenario: AWS API rate limits (throttling) are hit during terraform apply. How do you handle it?

> **In simple words:** Terraform is sending AWS too many requests at the same time, like 50 people shouting at one cashier. You slow the queue down, let the cashier retry calmly, and split the big order into smaller ones.

Detailed answer

How you recognize it: errors like `ThrottlingException: Rate exceeded` or `RequestLimitExceeded`, very slow applies, or the same apply succeeding on a retry. Run with `TF_LOG=DEBUG` to see the retries and which API is being throttled.

Fixes, in the order I would try:

1. **Lower parallelism**: `terraform apply -parallelism=N` (default is 10) so fewer API calls run at once.
2. **Tune provider retries**: in the `aws` provider block set `max_retries` and `retry_mode = "adaptive"`, which backs off automatically on throttling.
3. **Split into smaller root modules/stacks**: fewer resources per plan means fewer describe/refresh calls (big states also refresh hundreds of resources on every plan).
4. **Avoid huge fan-out at once**: a `for_each` creating 500 records or roles in one run hammers the same API; roll it out in batches.
5. **Request a quota increase** (Service Quotas) if a real limit is hit, and check whether other tools in the same account (CI jobs, scripts, autoscalers) share the same API budget.
6. `depends_on` / `time_sleep` to serialize calls are a crude last resort, not the main fix.

Example:

```
provider "aws" {
  region      = "ap-south-1"
  max_retries = 25
  retry_mode  = "adaptive"   # client-side rate limiting with backoff
}
```

```
TF_LOG=DEBUG terraform apply -parallelism=3 2> tf-debug.log
grep -i "throttl\|rate exceeded" tf-debug.log | head
```

What to say to the interviewer: *I confirm it is throttling from errors like ThrottlingException or Rate exceeded in TF_LOG=DEBUG output, then reduce -parallelism below the default of 10 and set max_retries with retry_mode adaptive in the AWS provider. Longer term I split large states into smaller stacks, avoid creating hundreds of resources in one run, and request quota increases where needed; time_sleep or depends_on is only a crude last resort.*

Q45. Scenario: you added a new EC2 instance to an Auto Scaling Group using Terraform, but the ASG does not register the instance. How do you troubleshoot?

In simple words: An Auto Scaling Group is a factory that builds its own cars from its own blueprint (the launch template). A car you built separately in your garage does not become part of the factory fleet just because it looks the same.

Detailed answer

Key point: an `aws_instance` resource **never joins an ASG automatically**. ASGs launch and own their instances from the launch template. So the right fix depends on what you wanted:

- **Want more instances** -> change `desired_capacity` / `min_size` on the ASG (or let a scaling policy do it).
- **Want new settings / new AMI** -> change the launch template and bump its version, then roll out with an **instance refresh**.
- **Really want to attach an existing instance** -> `aws autoscaling attach-instances` (CLI/API). Terraform has no first-class resource for attaching EC2 instances; `aws_autoscaling_attachment` is for attaching the ASG to ELBs / target groups, not instances.

If the ASG itself is not launching or keeping instances, check:

1. **Scaling activity history**: `aws autoscaling describe-scaling-activities` - it states exactly why a launch failed.
2. **max_size reached** - desired cannot go above max.
3. **Subnets / AZs**: `vpc_zone_identifier` subnets must be valid, in the right VPC, and have free IPs; an attached instance must be in one of the ASG's AZs.
4. **Launch template valid**: AMI exists in this region, instance profile exists, security groups belong to the same VPC, key pair exists.
5. **Health checks**: with `health_check_type = "ELB"` and a short grace period, new instances are marked unhealthy and terminated before the app starts.
6. **Lifecycle hooks** stuck in `Pending:Wait` until timeout.
7. **Terraform masking changes**: `lifecycle { ignore_changes = [desired_capacity] }` means your capacity change in code is silently ignored.
8. **Instance refresh** in progress or failed, or suspended processes (`Launch` suspended).
9. **Quotas / capacity**: vCPU service quota reached or `InsufficientInstanceCapacity` for that instance type in the AZ -> use multiple instance types or AZs.

Example:

```
resource "aws_autoscaling_group" "web" {
  name                = "web-asg"
  min_size            = 2
  max_size            = 6
  desired_capacity    = 3              # change this to add instances
  vpc_zone_identifier = var.private_subnets
  health_check_type         = "ELB"
  health_check_grace_period = 300

  launch_template {
    id      = aws_launch_template.web.id
    version = aws_launch_template.web.latest_version
  }

  instance_refresh {
    strategy = "Rolling"
    preferences { min_healthy_percentage = 90 }
  }
}
```

```
aws autoscaling describe-scaling-activities --auto-scaling-group-name web-asg --max-items 5
aws autoscaling attach-instances --instance-ids i-0abc123 --auto-scaling-group-name web-asg
```

> ***What to say to the interviewer:*** *An aws_instance created by Terraform never joins an ASG on its own, because the ASG launches its own instances from the launch template, so I would raise desired_capacity or update the launch template version, or use attach-instances if I really need to adopt an existing instance. If the ASG is not launching, I check the scaling activity history, max_size, subnets and AZs, launch template validity, health check grace period, lifecycle hooks, ignore_changes on desired_capacity, and capacity or quota errors.*

Q46. What is the fastest way to roll back an infrastructure change made with Terraform?

> **In simple words:** Terraform has no undo button. Your Git history is the undo button - go back to the last good recipe and cook again.

Detailed answer

Key facts:

- There is **no** `terraform rollback` **command**. Terraform only knows "make reality match the code". So rolling back = putting the old code back and applying it.
- **Fastest safe path**: `git revert` the bad commit -> pipeline runs `plan` -> review that the plan only undoes the bad change -> `apply`.
- Because modules are versioned, rolling back a module change is often just pinning `?ref=` back to the previous tag.
- `-target` lets you apply only one resource. Use it only in an emergency to fix the broken piece quickly, then run a full plan/apply afterwards so nothing is left out of sync. It is not for routine use.
- Some changes cannot be "rolled back" by code alone: a deleted database or a replaced resource with new IDs needs a restore from snapshot/backup.
- **Restoring an older state version from S3** is the last resort, only when state itself is broken - it does not change real infrastructure and can make state lie about reality.

Real example: a security group update removed the ingress rule the ALB needed, and the service started failing health checks. I reverted the Git commit, the pipeline showed a plan adding back just that one rule, I applied it, and the service recovered within minutes. The post-mortem added a policy check for removing ingress on prod SGs.

Example:

```
git revert a1b2c3d            # the faulty SG change
git push                      # pipeline runs plan
terraform plan -out=tfplan    # expect: 0 to add, 1 to change, 0 to destroy
terraform apply tfplan

# Emergency only, then do a full plan/apply afterwards:
terraform apply -target=aws_security_group.web
```

> ***What to say to the interviewer:*** *Terraform has no rollback command, so the fastest safe rollback is reverting the Git commit and running plan and apply through the pipeline, which restores the previous desired state. I use -target only in emergencies, restoring an old state version from S3 is a last resort for broken state, and for destructive changes like a deleted database I restore from backups. For example, when a security group change broke service access, reverting the commit and re-applying fixed it in minutes.*

Q47. Why might terraform plan show NO changes even though someone modified the infrastructure outside Terraform?

> **In simple words:** Terraform only compares the things written in its checklist. If someone repainted a wall that isn't on the checklist, the inspector happily says "everything matches".

Detailed answer

A common model answer says "because you didn't run terraform refresh". That is **wrong**: `terraform plan` refreshes state from the real cloud by default, so external changes to attributes Terraform manages **do** show up. When plan says "No changes", the real reasons are:

1. **The attribute isn't in your config.** Optional or computed attributes you never set are not enforced. If someone changes them, Terraform doesn't care.
2. `lifecycle { ignore_changes = [...] }` covers that attribute (common for `desired_capacity`, `tags`, `image`).
3. **The change is on a different object Terraform doesn't manage.** For example, someone added an extra security group rule outside Terraform, while your code manages the SG with separate rule resources. Or a bucket policy was added while your code never declared an `aws_s3_bucket_policy`.
4. **Plan ran with** `-refresh=false` (some CI setups do this for speed).
5. **Wrong target:** a different workspace, state file, AWS account/profile or region than the one that was modified.
6. **Provider behaviour:** some attributes aren't read back by the provider, or are normalized so the diff is hidden.

Example:

```
terraform workspace show                  # right workspace?
aws sts get-caller-identity               # right account?
terraform plan -refresh-only              # shows what changed in reality vs state
grep -rn "ignore_changes" .               # is the attribute ignored?
terraform state show aws_security_group.app   # what Terraform actually tracks
```

> ***What to say to the interviewer:*** *Plan refreshes by default, so "no changes" usually means Terraform doesn't manage what was changed: the attribute isn't set in the config, it's in* `ignore_changes`*, or the change was on a separate object like an extra rule or policy. I also check that I'm on the right workspace, account and region, and that CI isn't using* `-refresh=false`*.* `terraform plan -refresh-only` *shows exactly what drifted.*

Q48. Terraform state is huge (200MB) and plan takes 12 minutes. How do you fix it?

> **In simple words:** One giant notebook for the whole city means every small question requires reading every page. Split it into one notebook per neighbourhood, and each question gets answered in seconds.

Detailed answer

The root cause is almost always **one state managing too much**: every plan refreshes thousands of resources through cloud APIs.

1. **Split the state by lifecycle and blast radius:** network (VPC, TGW), data (RDS, S3), platform (EKS), then one stack per service or team. Things that change daily shouldn't live with things that change yearly.
2. **Connect stacks with outputs, not one big graph:** `terraform_remote_state`, or better, publish IDs to SSM Parameter Store and read them with data sources.
3. **Move resources without recreating them:** `removed { lifecycle { destroy = false } }` in the old stack plus `import` blocks in the new one (or `terraform state mv -state-out` for local migrations). Plan both sides to confirm there are 0 destroys.
4. **Don't manage high-churn, high-count objects in Terraform:** thousands of DNS records, S3 objects or IAM users generated by apps belong in app-level tooling.
5. **Quick wins while you refactor:** `-parallelism=20` (watch API throttling), a PR-time plan with `-refresh=false` and a scheduled full refresh plan for drift, avoid large `data` sources that list everything, and keep the backend in the same region as CI.
6. **Tooling for many stacks:** Terragrunt, Terramate, Spacelift or Terraform Cloud workspaces with run triggers.
7. Use `-target` only for emergencies, never as the normal workflow.

Example:

```
infra/
  00-network/     state: network.tfstate   (VPC, subnets, TGW)   -> changes yearly
  10-data/        state: data.tfstate      (RDS, S3, KMS)        -> changes monthly
  20-eks/         state: eks.tfstate       (cluster, node pools)
  30-services/
     billing/     state: billing.tfstate   -> changes daily, plan in ~20s
     meter-api/   state: meter-api.tfstate
```

```
# In 20-eks: read network IDs published by 00-network
data "aws_ssm_parameter" "private_subnets" {
  name = "/infra/network/private_subnet_ids"
}
locals {
  private_subnet_ids = split(",", data.aws_ssm_parameter.private_subnets.value)
}
```

> ***What to say to the interviewer:*** *A 200MB state means one stack is doing too much, so every plan refreshes thousands of resources. I split it into stacks by lifecycle and blast radius (network, data, cluster, per-service), connect them through SSM parameters or remote state, and move resources with removed and import blocks so nothing is recreated. Meanwhile I tune parallelism and avoid heavy data sources. After the split, most plans run in under a minute and a mistake can only affect one small stack.*

AWS

Basic

Q1. Which AWS services have you worked on, and which are you most proficient in?

Also asked as: What AWS services are you proficient in? How many AWS services have you used? Which AWS services have you used in production?

> **In simple words:** The interviewer is not asking for a shopping list. They want to hear which services you have actually run in production and what you did with each one.

Detailed answer

Group your answer by category and attach a real task to each service. Five services you truly understand beat twenty you only read about.

- **Compute:** EC2, EKS, Lambda, Auto Scaling Groups
- **Networking:** VPC, subnets, Security Groups, ALB/NLB, Route 53, VPC endpoints
- **Data:** RDS (PostgreSQL/MySQL), ElastiCache (Redis), MSK (Kafka), S3
- **Operations:** CloudWatch (metrics, alarms, logs), EventBridge, SNS, SES, Systems Manager
- **Security:** IAM roles and policies, Secrets Manager, KMS

Example:

```
"I work across about 15 services daily. EKS runs our apps, RDS/ElastiCache/MSK are the
data layer, CloudWatch + EventBridge + Lambda power our alerting and daily health
reports to Teams, and IAM roles give every workload least-privilege access."
```

> ***What to say to the interviewer:*** *I have hands-on experience with roughly 15 AWS services, mainly EC2, EKS, VPC, RDS, ElastiCache, MSK, S3, IAM, CloudWatch, Lambda and EventBridge. For example, I built Lambda functions triggered by EventBridge that collect EKS, RDS, Redis and Kafka health and post a daily report to Teams, using an IAM execution role rather than stored keys.*

Q2. Explain the main networking concepts in AWS.

> **In simple words:** Think of AWS networking as a gated housing society: the society wall (VPC), blocks (subnets), road signs (route tables), the main gate (Internet Gateway), a one-way exit (NAT), guards (SG/NACL), private tunnels to AWS shops (endpoints), bridges to other societies (peering/TGW), the address book (Route 53) and the reception desk (load balancer).

Detailed answer

Concept	What it is	Everyday analogy
VPC	Isolated private network in a region with a CIDR range	The society wall
Subnet	Slice of the VPC in one AZ; public or private	A block inside the society
Route table	Rules for where traffic goes next	Road signs
Internet Gateway (IGW)	Two-way internet door for public subnets	Main gate
NAT Gateway	Outbound-only internet for private subnets	Exit-only gate
Security Group	Stateful allow-list on each instance/ENI	Guard at each flat
NACL	Stateless allow/deny rules per subnet	Guard at each block
VPC Endpoint	Private path to AWS services (gateway: S3/DynamoDB; interface: most others)	Private tunnel to shops
Peering / Transit Gateway	Connect VPCs 1-to-1 / hub-and-spoke	Bridge / central highway
VPN / Direct Connect	Connect on-premises to AWS	Private road to the office
Route 53	Managed DNS (public and private zones)	Address book
Elastic Load Balancer	ALB (HTTP L7), NLB (TCP/UDP L4), GWLB (appliances)	Reception desk
Elastic IP	Static public IPv4 address	Permanent phone number

Example:

```
Internet -> IGW -> ALB (public subnets) -> App (private subnets, SG from ALB)
                                            |-> NAT GW -> internet (outbound only)
                                            |-> S3 gateway endpoint (private)
                                           +-> RDS (data subnets, SG from app)
```

> ***What to say to the interviewer:*** *A VPC is the private network, divided into public and private subnets per AZ, with route tables sending traffic to an Internet Gateway, NAT Gateway, endpoints, peering or Transit Gateway. Security Groups and NACLs control access, Route 53 handles DNS and load balancers spread traffic across targets.*

Q3. What is an AWS VPC and a subnet?

> **In simple words:** A VPC is your own private, fenced plot of land inside AWS. Subnets are the rooms you divide it into: some rooms have a door to the street (public), some do not (private).

Detailed answer

- **VPC (Virtual Private Cloud):** an isolated virtual network in one region, defined by a CIDR block such as `10.0.0.0/16` (65,536 IPs).

- **Subnet:** a slice of the VPC CIDR that lives in exactly **one Availability Zone**, for example `10.0.1.0/24`.
- **Public subnet:** its route table has a route `0.0.0.0/0 -> Internet Gateway`.
- **Private subnet:** no route to the IGW; outbound internet (if needed) goes through a NAT Gateway.
- AWS reserves 5 IPs in every subnet (network, router, DNS, future use, broadcast).

Example:

```
VPC 10.0.0.0/16 (ap-south-1)
 +-- public-a   10.0.1.0/24  (AZ a)  -> IGW       : ALB, NAT GW
 +-- private-a  10.0.11.0/24 (AZ a)  -> NAT GW    : EKS nodes / app servers
 +-- data-a     10.0.21.0/24 (AZ a)  -> no internet : RDS, Redis
```

> ***What to say to the interviewer:*** *A VPC is a logically isolated network in a region with its own IP range. I split it into subnets per AZ: public subnets route to an Internet Gateway for load balancers, and private subnets hold apps and databases, reaching the internet only through a NAT Gateway if needed.*

Q4. What is the purpose of a NAT Gateway?

> **In simple words:** A NAT Gateway is a one-way door for a private room: people inside can go out to the market and come back with shopping, but strangers from the street cannot walk in.

Detailed answer

- Gives instances in **private subnets outbound-only** internet access (OS updates, calling external APIs, pulling public images).
- Inbound connections started from the internet are **not allowed** through it.
- It lives in a **public subnet**, has an **Elastic IP**, and private route tables point `0.0.0.0/0 -> nat-xxxx`.
- It is **zonal**: create **one per AZ** so an AZ failure does not cut off the other AZs (and to avoid cross-AZ data charges).
- **Cost:** hourly charge plus a per-GB data processing charge (about $0.045/GB in many regions). Use free S3/DynamoDB gateway endpoints to keep that traffic off the NAT.
- Managed and scales automatically, unlike a self-managed NAT instance.
- For IPv6, the equivalent is an **Egress-Only Internet Gateway**.

Example:

```
private-a route table:  10.0.0.0/16 -> local
                        0.0.0.0/0   -> nat-0aaa (in public-a)
private-b route table:  0.0.0.0/0   -> nat-0bbb (in public-b)

aws ec2 create-nat-gateway --subnet-id subnet-public-a --allocation-id eipalloc-0abc
```

> ***What to say to the interviewer:*** *A NAT Gateway lets resources in private subnets reach the internet outbound while blocking inbound connections from the internet. I place one in a public subnet per AZ with an Elastic IP, point private route tables to it, and keep S3 and DynamoDB traffic on gateway endpoints because NAT charges per GB processed.*

Q5. What is the difference between a Security Group and a NACL?

Also asked as: NACL vs Security Group.

> **In simple words:** A Security Group is a bouncer at each person's door who remembers who they let in. A NACL is the security guard at the gate of the whole building who checks everyone, both coming in and going out, and remembers nothing.

Detailed answer

Feature	Security Group	NACL
Level	Instance / ENI	Subnet
State	Stateful (return traffic auto-allowed)	Stateless (must allow return traffic)
Rules	Allow only	Allow and Deny
Evaluation	All rules together	In number order, first match wins
Default	Deny all inbound, allow all outbound	Default NACL allows all

A common trap: with NACLs you must allow **ephemeral ports 1024-65535** outbound, or response traffic is dropped.

Example:

```
SG app-sg     : inbound 8080 from alb-sg (reference SG, not IP)
NACL private  : rule 100 ALLOW tcp 8080 from 10.0.1.0/24
                rule 110 ALLOW tcp 1024-65535 outbound (return traffic)
                rule 90  DENY  all from 203.0.113.50/32 (block a bad IP)
```

> ***What to say to the interviewer:*** *Security Groups are stateful, allow-only firewalls attached to instances, while NACLs are stateless subnet-level rules that support explicit deny and are evaluated in order. I use Security Groups for normal access control and NACLs only for broad subnet rules like blocking a malicious IP range.*

Q6. What is an IAM role?

> **In simple words:** An IAM role is like a visitor badge. Nobody owns it permanently; whoever is allowed picks it up, gets temporary access, and the badge expires automatically.

Detailed answer

- A role has **no long-term password or access keys**.
- It has two policies: a **trust policy** (who can assume it) and a **permission policy** (what they can do).
- When assumed, STS issues **temporary credentials** (default 1 hour).
- Used by EC2 (instance profile), Lambda (execution role), EKS pods (IRSA / Pod Identity), CI/CD (OIDC), and other accounts (cross-account).

Example:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

What to say to the interviewer: *An IAM role is an identity with permissions but no permanent credentials; a trusted entity like an EC2 instance, Lambda or another account assumes it and gets short-lived STS credentials. I always prefer roles over access keys because nothing is stored and the credentials rotate automatically.*

Q7. What are the different S3 storage classes and their use cases?

In simple words: Like storing things at home: the kitchen shelf (fast, expensive), the attic (cheaper, a bit slower), and a storage unit in another city (very cheap, takes hours to get things back).

Detailed answer

Class	Use case	Notes
Standard	Frequently accessed data	Default, millisecond access
Intelligent-Tiering	Unknown/changing access patterns	Auto-moves objects, small monitoring fee
Standard-IA	Infrequent access, needs fast retrieval	30-day min, retrieval fee
One Zone-IA	Re-creatable infrequent data	Single AZ, cheaper
Glacier Instant Retrieval	Archives read about once a quarter	ms access, 90-day min
Glacier Flexible Retrieval	Backups, minutes-to-hours restore	90-day min
Glacier Deep Archive	Compliance data kept 7-10 years	Cheapest, 12-48h restore, 180-day min
Express One Zone	Very low-latency, high request rate	Single AZ, directory buckets

Example:

```
App uploads (hot)        -> Standard
Logs older than 30 days  -> Standard-IA
Logs older than 90 days  -> Glacier Flexible Retrieval
Audit data (7 years)     -> Glacier Deep Archive
```

What to say to the interviewer: *S3 has classes from Standard for hot data, through Infrequent Access and Intelligent-Tiering, down to Glacier tiers for archives. I pick based on how often the data is read and how fast it must come back, and I use lifecycle rules to move data down the tiers automatically.*

Q8. What is an S3 lifecycle policy?

Also asked as: Configure a lifecycle rule to move an object to Glacier after 60 days and delete it after 90 days.

In simple words: An automatic cleaning schedule for a warehouse: after 30 days move boxes to the back room, after a year send them to off-site storage, after 3 years throw them out.

Detailed answer

Lifecycle rules act on objects (optionally filtered by prefix or tag):

- **Transition actions:** move objects to a cheaper class after N days.
- **Expiration actions:** delete objects after N days.
- **Noncurrent version rules:** clean old versions in versioned buckets.
- **Abort incomplete multipart uploads:** removes hidden half-uploaded parts that you still pay for.

Example:

```
{
  "Rules": [{
    "ID": "logs-tiering",
    "Filter": { "Prefix": "logs/" },
    "Status": "Enabled",
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 90, "StorageClass": "GLACIER" }
    ],
    "Expiration": { "Days": 1095 },
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
  }]
}
```

```
aws s3api put-bucket-lifecycle-configuration --bucket my-logs \
  --lifecycle-configuration file://lifecycle.json
```

Follow-up: "Move an object to Glacier after 60 days and delete it after 90 days."

```
{
  "Rules": [{
    "ID": "glacier-60-delete-90",
    "Filter": { "Prefix": "" },
    "Status": "Enabled",
    "Transitions": [{ "Days": 60, "StorageClass": "GLACIER" }],
    "Expiration": { "Days": 90 }
  }]
}
```

```
resource "aws_s3_bucket_lifecycle_configuration" "logs" {
  bucket = aws_s3_bucket.logs.id

  rule {
    id     = "glacier-60-delete-90"
    status = "Enabled"
    filter {}

    transition {
      days          = 60
      storage_class = "GLACIER"
    }

    expiration {
      days = 90
    }
  }
}
```

Cost gotcha (mention this in the interview):

- Glacier Flexible Retrieval (`GLACIER`) has a **90-day minimum storage duration**. An object moved on day 60 and deleted on day 90 has stayed only 30 days in Glacier, so you pay an **early-deletion fee** as if it stayed the full 90 days there. Moving it may cost more than leaving it in Standard.
- Better options: just **expire at 90 days** with no transition, or use Standard-IA (30-day minimum), or Glacier Instant Retrieval (still 90-day minimum, so same issue).
- Lifecycle transitions also have per-request costs, and by default objects **smaller than 128 KB are not transitioned** (it is not cost-effective), so millions of tiny log files may stay in Standard.

> ***What to say to the interviewer:*** *A lifecycle policy automatically transitions objects to cheaper storage classes or deletes them after a set age. For example, I move logs to Standard-IA after 30 days, Glacier after 90 and delete them after 3 years, plus abort incomplete multipart uploads so we do not pay for hidden junk. For a Glacier-at-60, delete-at-90 rule I would warn that Glacier's 90-day minimum means an early-deletion charge, so simply expiring at 90 days is often cheaper.*

Q9. What is CloudFront, and when would you use it?

> **In simple words:** Instead of every customer driving to your one warehouse, you put copies of popular items in small shops in every city. CloudFront is that network of shops (edge locations).

Detailed answer

- CloudFront is AWS's **CDN** with hundreds of edge locations worldwide.
- It caches content near users, cutting latency and load on the origin.
- Origins: S3, ALB, EC2, API Gateway, Lambda function URLs, or any HTTP server.
- Extra benefits: free TLS via ACM, AWS WAF and Shield integration, Origin Access Control to keep S3 private, CloudFront Functions / Lambda@Edge for logic at the edge.

Use it for static websites, images/videos, software downloads, and even dynamic APIs (it keeps optimized connections back to the origin).

Example:

```
User (Mumbai) -> Edge (Mumbai) --cache hit--> response in ~20 ms
                        |
                  cache miss -> Origin (S3 in us-east-1) -> cached for next user
```

What to say to the interviewer: *CloudFront is AWS's CDN that caches content at edge locations close to users. I use it in front of S3 or an ALB to reduce latency, offload the origin, terminate TLS and attach WAF, and I keep the S3 bucket private using Origin Access Control.*

Q10. What is Route 53, and how is it different from CoreDNS?

In simple words: Route 53 is the public phone book of the whole city. CoreDNS is the internal extension directory inside one office building (your Kubernetes cluster).

Detailed answer

	Route 53	CoreDNS
What	Managed AWS DNS service	DNS server running as pods in Kubernetes
Scope	Public internet and private VPC zones	Inside one cluster
Resolves	`app.example.com`, domains you own	`my-svc.my-ns.svc.cluster.local`
Extras	Domain registration, health checks, routing policies	Service discovery, plugins, forwards to upstream DNS

Route 53 routing policies: simple, weighted, latency, failover, geolocation, geoproximity, multivalue, IP-based.

Example:

```
Browser -> Route 53: api.example.com -> ALB DNS name
Pod     -> CoreDNS : orders.prod.svc.cluster.local -> 172.20.14.9 (ClusterIP)
Pod     -> CoreDNS : rds.amazonaws.com -> forwards to VPC resolver (.2)
```

What to say to the interviewer: *Route 53 is AWS's managed DNS for public and private domains, with health checks and routing policies like failover and latency. CoreDNS runs inside Kubernetes and resolves service names to ClusterIPs for pod-to-pod discovery, forwarding external names to the VPC resolver.*

Q11. What is a read replica in RDS?

In simple words: The main database is the teacher writing on the board. Read replicas are students copying the board so other people can read from them without disturbing the teacher.

Detailed answer

- A read-only copy of the primary database, updated with **asynchronous replication**.
- Offloads read-heavy traffic (reports, dashboards, search).
- Can be in the same AZ, another AZ, or **another region** (cross-region DR).
- Because replication is async, replicas can lag (check `ReplicaLag` metric).
- A replica can be **promoted** to a standalone primary.

- Different from Multi-AZ standby: a classic Multi-AZ standby is synchronous and not readable.

Example:

```
aws rds create-db-instance-read-replica \
  --db-instance-identifier orders-replica-1 \
  --source-db-instance-identifier orders-prod \
  --db-instance-class db.r6g.large
```

What to say to the interviewer: *A read replica is an asynchronously replicated, read-only copy of an RDS instance used to scale reads and for cross-region DR. It differs from Multi-AZ, which is a synchronous standby for high availability, and a replica can be promoted to a primary if needed.*

Q12. Explain the architecture of Amazon ECS.

In simple words: ECS is a restaurant manager. The task definition is the recipe, a task is one cooked dish, a service makes sure there are always N dishes on the counter, and the cluster is the kitchen.

Detailed answer

- **Cluster:** logical group of capacity.
- **Task definition:** JSON blueprint (image, CPU/memory, ports, env, IAM task role, log config).
- **Task:** a running instance of a task definition (one or more containers).
- **Service:** keeps the desired number of tasks running, replaces failed ones, integrates with ALB/NLB, does rolling or blue/green deploys.
- **Launch types / capacity:** Fargate (serverless, no servers) or EC2 (your ASG with the ECS agent), managed through **capacity providers**.
- **Networking:** `awsvpc` mode gives each task its own ENI and Security Group.
- Control plane is fully managed by AWS; Service Connect / Cloud Map handle service discovery.

Example:

```
ECS Cluster "prod"
 +-- Service "orders" (desired=4) --> ALB target group
 |     +-- Task (Fargate, ENI 10.0.11.23) -> container orders:1.4.2
 |     +-- Task (Fargate, ENI 10.0.12.41)
 +-- Capacity providers: FARGATE, FARGATE_SPOT
```

What to say to the interviewer: *ECS has clusters, task definitions that describe containers, tasks that are running copies, and services that keep a desired count running behind a load balancer. Capacity comes from Fargate or EC2 instances via capacity providers, and AWS fully manages the control plane.*

Q13. Why use Amazon EKS instead of ECS? What are the key differences?

Also asked as: ECS vs EKS.

In simple words: ECS is an automatic car from one brand: simple, just drive. EKS is a manual sports car that works the same everywhere: more control and portability, but you must know how to drive it.

Detailed answer

	ECS	EKS
Orchestrator	AWS proprietary	Upstream Kubernetes
Learning curve	Low	High
Portability	AWS only	Any cloud / on-prem
Control plane cost	Free	About $0.10/hour per cluster
Ecosystem	AWS integrations	Helm, ArgoCD, operators, service mesh, KEDA
Ops effort	Low	Higher (upgrades, add-ons, CNI)

Choose **ECS** for a small team, AWS-only, simple microservices. Choose **EKS** when you need Kubernetes tooling, multi-cloud portability, custom operators, or already have Kubernetes skills.

Example:

```
Startup, 10 services, AWS only          -> ECS on Fargate
Platform team, 100+ services, GitOps,
Helm charts, Kafka operators, hybrid     -> EKS (+ Karpenter, ArgoCD)
```

> ***What to say to the interviewer:*** *ECS is simpler and AWS-native with no control plane fee, while EKS gives standard Kubernetes with its huge ecosystem and portability at the cost of more operational work. I choose EKS when we need Helm, GitOps, operators or multi-cloud, and ECS when simplicity matters more.*

Q14. Which services can be integrated with a CDN (CloudFront)?

> **In simple words:** A CDN can sit in front of almost anything that speaks HTTP, and it can call helpers for security and small bits of logic.

Detailed answer

Origins CloudFront can serve from:

- Amazon S3 (with Origin Access Control)
- Application Load Balancer, EC2, NLB (VPC origins allow private ALB/EC2)
- API Gateway and Lambda function URLs
- AWS Elemental MediaPackage / MediaStore (video streaming)
- Any custom HTTP origin, even on-premises

Services that plug into CloudFront:

- AWS WAF (filter attacks), AWS Shield (DDoS)
- ACM (free TLS certificates, must be in us-east-1)
- CloudFront Functions and Lambda@Edge (redirects, headers, auth)
- Route 53 (alias records), CloudWatch and S3 for logs

Example:

```
Route53 (alias) -> CloudFront + WAF + ACM cert
                     |-- /static/*  -> S3 bucket (OAC)
                     |-- /api/*     -> ALB -> EKS
                     +-- /img/*     -> Lambda@Edge resize -> S3
```

What to say to the interviewer: *CloudFront integrates with S3, ALB, EC2, API Gateway, Lambda function URLs, media services and any custom HTTP origin. Around it I add WAF, Shield, ACM certificates and edge functions, with Route 53 alias records pointing the domain to the distribution.*

Q15. What is the default port for DynamoDB?

In simple words: DynamoDB is not a database server you dial into on a special number like MySQL (3306). It is a web service: you talk to it like a website, over HTTPS.

Detailed answer

- DynamoDB is a **managed HTTPS API**. Clients (SDK/CLI) call endpoints like `dynamodb.ap-south-1.amazonaws.com` on **port 443**.
- There is **no database port** to open and no connection to keep, unlike RDS (MySQL 3306, PostgreSQL 5432).
- Every request is signed with **IAM credentials (SigV4)**; access is controlled by IAM policies, not passwords.
- **DynamoDB Local** (the downloadable version for testing) listens on **port 8000** by default.
- To reach it privately from private subnets, use a **gateway VPC endpoint** for DynamoDB (free), so no NAT is needed.

Example:

```
aws dynamodb list-tables --region ap-south-1          # HTTPS 443 to AWS endpoint

docker run -p 8000:8000 amazon/dynamodb-local
aws dynamodb list-tables --endpoint-url http://localhost:8000   # local testing
```

What to say to the interviewer: *DynamoDB has no traditional database port; it is an HTTPS API reached on port 443 with IAM-signed requests. DynamoDB Local uses port 8000 for testing, and in a VPC I reach DynamoDB privately through a free gateway endpoint.*

Q16. How do you add storage to an existing EC2 instance?

In simple words: Either plug in an extra hard disk (new EBS volume) or make the existing disk bigger (resize). Both can be done while the computer keeps running.

Detailed answer

Option A - Add a new EBS volume:

1. Create an EBS volume in the **same AZ** as the instance (gp3 recommended).
2. **Attach** it to the instance.
3. On the instance: `lsblk` to find the device (for example `/dev/nvme1n1`).
4. Format it (`mkfs`) - only for a new, empty volume.

5. Mount it and add it to `/etc/fstab` using the **UUID** and the `nofail` option, so a missing disk does not block boot.

Option B - Grow an existing volume (no downtime):

1. `aws ec2 modify-volume` to increase the size.
2. `growpart` to extend the partition.
3. `resize2fs` (ext4) or `xfs_growfs` (XFS) to extend the filesystem.

Take a **snapshot first**. You can grow a volume but not shrink it, and you must wait about 6 hours between modifications of the same volume.

Example:

```
aws ec2 create-volume --availability-zone ap-south-1a --size 100 --volume-type gp3
aws ec2 attach-volume --volume-id vol-0new --instance-id i-0abc --device /dev/sdf
lsblk
sudo mkfs -t xfs /dev/nvme1n1
sudo mkdir /data && sudo mount /dev/nvme1n1 /data
echo "UUID=$(sudo blkid -s UUID -o value /dev/nvme1n1) /data xfs defaults,nofail 0 2" \
  | sudo tee -a /etc/fstab

# Grow existing root volume
aws ec2 modify-volume --volume-id vol-0root --size 50
sudo growpart /dev/nvme0n1 1
sudo xfs_growfs -d /          # or: sudo resize2fs /dev/nvme0n1p1
```

> ***What to say to the interviewer:*** *I either create a new EBS volume in the same AZ, attach it, format, mount and add it to fstab by UUID with nofail, or I grow the existing volume with modify-volume and then growpart and resize2fs or xfs_growfs. Both work without downtime, and I snapshot before changing anything.*

Q17. What is the difference between an ALB and an NLB, and when would you prefer an NLB, especially with databases?

> **In simple words:** An ALB is a smart receptionist who reads each letter and sends it to the right department. An NLB is a very fast mail chute that just forwards envelopes without opening them, at huge volume, from a fixed address.

Detailed answer

	ALB	NLB
OSI layer	Layer 7 (HTTP/HTTPS, gRPC, WebSocket)	Layer 4 (TCP, UDP, TLS)
Routing	Host, path, header, query, method rules	Port-based only
Static IP	No (DNS name only)	Yes, one static IP per AZ, can use Elastic IPs
Performance	Good, adds a few ms	Ultra-low latency, millions of requests/sec
Client IP	In `X-Forwarded-For` header	Preserved at the packet level
Extras	WAF, Cognito/OIDC auth, redirects, fixed responses	PrivateLink endpoint services, TLS passthrough
Targets	Instances, IPs, Lambda	Instances, IPs, ALB

Prefer an NLB when:

- The protocol is not HTTP (databases, MQTT, gaming, UDP, SMTP).
- A partner must **allowlist fixed IPs**.
- You need **PrivateLink**: an endpoint service must sit behind an NLB (or GWLB).
- You need extreme throughput or the original client IP.

Databases:

- Fronting a **self-managed DB cluster** on EC2 (for example PostgreSQL/Kafka/Redis nodes) with a TCP NLB.
- Exposing **RDS privately to another VPC or account via PrivateLink**: NLB with **IP targets** pointing to the RDS IP (keep it updated on failover, since the IP changes) or use RDS Proxy.
- For normal use **RDS needs no load balancer**: the endpoint DNS name plus Multi-AZ failover handles it; reads go to the reader endpoint.

GWLB (Gateway Load Balancer): Layer 3, used to insert firewalls/IDS appliances transparently (GENEVE), for example in an inspection VPC.

Example:

```
aws elbv2 create-load-balancer --name db-nlb --type network --scheme internal \
  --subnets subnet-a subnet-b
aws elbv2 create-target-group --name pg-tg --protocol TCP --port 5432 \
  --vpc-id vpc-0abc --target-type ip
aws ec2 create-vpc-endpoint-service-configuration \
  --network-load-balancer-arns <nlb-arn> --acceptance-required
```

> ***What to say to the interviewer:*** *The ALB is a layer 7 HTTP load balancer with path and host routing, WAF and auth, while the NLB is layer 4 for TCP, UDP and TLS with static IPs, very low latency and client IP preservation. I choose an NLB for non-HTTP protocols like databases, for fixed-IP allowlisting and for PrivateLink, for example to share a database with another account, though plain RDS does not need a load balancer because its endpoint and Multi-AZ failover handle routing.*

Q18. What is AWS KMS used for, and what should be stored in it?

In simple words: KMS is a bank that keeps the master keys to your safes; it never holds your valuables (passwords, data) itself. Your secrets live in a safe (Secrets Manager), and KMS holds the key that locks that safe.

Detailed answer

Common misconception: KMS stores and manages **encryption keys**, not secrets or passwords. Store secrets in **Secrets Manager** or **Parameter Store SecureString**, which KMS encrypts.

- **Key types: AWS managed keys** (`aws/s3`, `aws/ebs`, created and rotated by AWS, limited control) vs **customer managed keys (CMKs)** (you control key policy, rotation, deletion, cross-account use).
- **Key policies and grants** decide who can use or manage a key; a key policy is required and is the primary access control.
- **Rotation:** automatic yearly rotation (period configurable) for symmetric CMKs; old versions still decrypt old data.
- **Envelope encryption:** KMS generates a **data key**; the service encrypts your data locally with the data key and stores the data key encrypted by the KMS key. That is how large data is encrypted.
- **Direct Encrypt API** handles only up to **4 KB** of data, so for larger data use `GenerateDataKey`.
- **Integrations:** S3 SSE-KMS, EBS, RDS, EFS, DynamoDB, Secrets Manager, CloudWatch Logs, EKS secrets encryption.
- **Audit:** every use of the key is logged in **CloudTrail**; key deletion has a 7-30 day waiting period.

Example:

```
aws kms create-key --description "prod app data" --key-usage ENCRYPT_DECRYPT
aws kms create-alias --alias-name alias/prod-app --target-key-id <key-id>
aws kms enable-key-rotation --key-id <key-id>

aws s3api put-bucket-encryption --bucket app-data --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":
    {"SSEAlgorithm":"aws:kms","KMSMasterKeyID":"alias/prod-app"},"BucketKeyEnabled":true}]}'

aws kms generate-data-key --key-id alias/prod-app --key-spec AES_256   # envelope encryption
```

What to say to the interviewer: *KMS creates and manages encryption keys, not secrets; secrets belong in Secrets Manager or Parameter Store, which are encrypted by KMS. I use customer managed keys with tight key policies and rotation, services like S3, EBS and RDS use envelope encryption with data keys because direct Encrypt is limited to 4 KB, and CloudTrail audits every key use.*

Advanced

Q19. How do you allow one AWS account to safely access resources in another account (cross-account access)?

Also asked as: How do you secure cross-account access using IAM roles?

> **In simple words:** Account B (the house owner) makes a special key that only Account A is allowed to borrow. Account A borrows it, uses it for one hour to do only the allowed job, and the key disappears.

Detailed answer

Steps using **cross-account IAM roles**:

1. **Account B (target)** creates a role, for example `ProdS3ReadRole`.
2. Its **trust policy** names Account A (ideally a specific role, not the whole account) and can require an `ExternalId` or MFA.
3. Its **permission policy** gives least privilege, for example only `s3:GetObject` on one bucket.
4. **Account A** gives its user/role permission to call `sts:AssumeRole` on that role ARN.
5. Account A calls STS and receives temporary credentials (15 min to 12 h).

Why it is secure: no stored keys, credentials expire, least privilege, revocable instantly by editing the trust policy, and every call is logged in **CloudTrail** in both accounts.

Example:

```
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::111111111111:role/jenkins-ci" },
  "Action": "sts:AssumeRole",
  "Condition": { "StringEquals": { "sts:ExternalId": "ci-prod-2026" } }
}
```

```
aws sts assume-role \
  --role-arn arn:aws:iam::222222222222:role/ProdS3ReadRole \
  --role-session-name ci-cd --external-id ci-prod-2026
```

> ***What to say to the interviewer:*** *I create a role in the target account whose trust policy allows only a specific role in the source account, with least-privilege permissions. The source assumes it through STS to get temporary credentials, so nothing is stored, access expires automatically, and CloudTrail audits every call.*

Q20. How would you access data in an S3 bucket in Account A from an application running on EC2 in Account B? What permissions are needed on the bucket side?

Also asked as: An app in one AWS account needs to access an S3 bucket in another account - how do you set it up securely? How do you provide access to an S3 bucket?

> **In simple words:** For cross-account access, both sides must say yes: the visitor's company must allow the trip (IAM policy in B), and the building owner must allow the visitor in (bucket policy in A).

Detailed answer

Option 1 - Bucket policy + instance role (simple, direct):

1. EC2 in Account B has an **instance profile role** `app-role`.
2. `app-role` identity policy allows `s3:GetObject`/`s3:ListBucket` on the bucket ARN.
3. **Bucket policy in Account A** allows principal `arn:aws:iam::B:role/app-role`.
4. If the bucket uses **SSE-KMS**, the KMS key policy in A must also allow the role to `kms:Decrypt`.
5. For uploads, set Object Ownership to **BucketOwnerEnforced** so Account A owns objects.

Option 2 - Assume a role in Account A: Account A creates a role with S3 access trusting B's role; the app calls `sts:AssumeRole`. Better when access covers many resources in A.

Option 3 - Pre-signed URLs: someone with access in Account A generates a time-limited URL for one object. Good for one-off downloads/uploads by users or systems that have no AWS identity.

Option	Use when
Bucket policy + cross-account principal	One bucket, steady app access, simplest to audit
IAM role + AssumeRole	Many resources in Account A, or Account A wants full control of permissions
Pre-signed URL	Temporary access to a single object, no AWS identity on the client

Never copy access keys onto the instance.

Example (bucket policy in Account A):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::222222222222:role/app-role" },
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::reports-bucket", "arn:aws:s3:::reports-bucket/*"]
  }]
}
```

```
# Option 3: pre-signed URL valid for 1 hour
aws s3 presign s3://reports-bucket/2026/09/report.pdf --expires-in 3600
```

> ***What to say to the interviewer:*** *I give the EC2 instance an IAM role whose policy allows the needed S3 actions, and in Account A I add a bucket policy trusting that exact role ARN, because cross-account needs both sides to allow it. If the bucket is KMS-encrypted I also update the key policy, and I never place access keys on the server.*

Q21. How do you secure S3 buckets in a multi-account setup?

> **In simple words:** Lock every door by default, give named keys only to people who need them, and install cameras everywhere.

Detailed answer

- **Block Public Access** at the **account level** in every account (and enforce with an SCP so no one can turn it off).
- **SCPs** in AWS Organizations: deny `s3:PutBucketPolicy` that makes buckets public, deny unencrypted uploads.
- **Bucket policies** using `aws:PrincipalOrgID` so only principals from your organization can access.
- **Encryption:** SSE-KMS with customer-managed keys; key policies per account.
- **Deny non-TLS** requests with `aws:SecureTransport`.
- **Object Ownership = BucketOwnerEnforced** (disables ACLs).
- **VPC endpoint policies** and `aws:SourceVpce` conditions for private-only buckets.
- **Visibility:** CloudTrail data events, S3 server access logs, IAM Access Analyzer, AWS Config rules, Macie for sensitive data.
- Central **log-archive account** with Object Lock for audit logs.

Example:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": ["arn:aws:s3:::shared-data", "arn:aws:s3:::shared-data/*"],
  "Condition": {
    "StringNotEquals": { "aws:PrincipalOrgID": "o-abc123xyz" }
  }
}
```

> ***What to say to the interviewer:*** *I enforce Block Public Access account-wide through SCPs, restrict bucket policies to our organization with aws:PrincipalOrgID, encrypt with KMS and deny non-TLS requests. Then I monitor with CloudTrail, Access Analyzer, Config and Macie so any drift is caught quickly.*

Q22. How can you access an EC2 instance that does not have a public IP?

> **In simple words:** The house has no front door on the street, but you can still get in through a secure intercom (SSM), a guarded side gate (EC2 Instance Connect Endpoint), or a guard house (bastion).

Detailed answer

1. **SSM Session Manager (best):** needs the SSM agent, an instance role with `AmazonSSMManagedInstanceCore`, and a path to SSM (NAT or VPC interface endpoints `ssm`, `ssmmessages`, `ec2messages`). No port 22, no keys, sessions logged.
2. **EC2 Instance Connect Endpoint:** a VPC endpoint that tunnels SSH/RDP to private IPs; access controlled by IAM.
3. **Bastion host** in a public subnet (older approach; must be hardened).
4. **VPN / Direct Connect** from the office network.

Example:

```
# Session Manager
aws ssm start-session --target i-0abc123def4567890

# EC2 Instance Connect Endpoint
aws ec2-instance-connect ssh --instance-id i-0abc123def4567890 \
  --connection-type eice
```

> ***What to say to the interviewer:*** *My first choice is SSM Session Manager, which needs only the agent, an instance role and SSM endpoints, with no open SSH port and full session logging. Alternatives are EC2 Instance Connect Endpoint, a VPN, or a hardened bastion host as a last resort.*

Q23. An EC2 instance in a private subnet must download packages without a NAT Gateway or bastion host. How?

Also asked as: You have an EC2 in a private subnet and cannot use a NAT Gateway - how else can it reach the internet for updates? Which AWS services can help?

> **In simple words:** If you cannot go out to the market, have the market deliver to your door through private delivery tunnels (VPC endpoints), or keep a pantry stocked in advance (baked AMI / internal mirror).

Detailed answer

- **S3 Gateway Endpoint (free):** Amazon Linux 2/2023 repositories are hosted on S3, so `yum`/`dnf` works through a gateway endpoint with no internet.
- **Interface endpoints (PrivateLink):** ECR (`ecr.api`, `ecr.dkr`) for images, SSM for patching via **Patch Manager**, CodeArtifact for pip/npm/Maven packages proxied from public repos.
- **Internal mirror:** sync packages into your own S3 bucket or Artifactory/Nexus and point the package manager there.
- **Bake an AMI** with EC2 Image Builder or Packer so packages are already installed.
- **Proxy instance** (Squid) in a public subnet with a domain allowlist.
- **Verification:** package managers check GPG signatures; also verify checksums of downloaded files.

Example:

```
aws ec2 create-vpc-endpoint --vpc-id vpc-0abc \
  --service-name com.amazonaws.ap-south-1.s3 \
  --vpc-endpoint-type Gateway --route-table-ids rtb-0private

# then on the instance (Amazon Linux 2023)
sudo dnf update -y
```

> ***What to say to the interviewer:*** *For Amazon Linux I add a free S3 gateway endpoint because the OS repos live in S3, and I use interface endpoints for ECR, SSM Patch Manager and CodeArtifact for language packages. For other distros I use an internal mirror or bake packages into the AMI, and GPG signatures verify integrity.*

Q24. What is VPC peering and how does it work? If VPC A is peered with B, and B with C, can A talk to C?

Also asked as: How do you connect two VPCs?

> **In simple words:** You are friends with Bob, and Bob is friends with Carol. That does not make you friends with Carol. VPC peering is the same: a private road between exactly two VPCs, not transitive.

Detailed answer

How peering works:

1. VPC A sends a **peering request** to VPC B (same or different account/region); B **accepts**.
2. Add **routes on both sides**: A's route tables send B's CIDR to `pcx-xxxx`, and B's route tables send A's CIDR back.
3. Update **Security Groups** to allow the other VPC's CIDR (or referenced SGs in the same region).
4. Optionally enable **DNS resolution** across the peering so private hostnames resolve.
5. Traffic stays on the AWS backbone, no gateway or bandwidth bottleneck, and CIDRs must not overlap.

Transitivity:

- **No, A cannot reach C.** VPC peering is **non-transitive**. Traffic cannot hop through B.
- Also no edge-to-edge routing: A cannot use B's NAT Gateway, IGW, VPN or Direct Connect.

Solutions for A <-> C:

1. Create a direct **A <-> C peering** (fine for a few VPCs).
2. Use **AWS Transit Gateway** as a hub (scales to thousands of VPCs, supports route tables and segmentation).
3. **PrivateLink** if you only need to expose one service, not whole networks.

Example:

```
aws ec2 create-vpc-peering-connection --vpc-id vpc-A --peer-vpc-id vpc-B
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id pcx-0abc
aws ec2 create-route --route-table-id rtb-A --destination-cidr-block 10.20.0.0/16 \
  --vpc-peering-connection-id pcx-0abc
aws ec2 create-route --route-table-id rtb-B --destination-cidr-block 10.10.0.0/16 \
  --vpc-peering-connection-id pcx-0abc

Peering (non-transitive):    A <---> B <---> C      A -X-> C

Transit Gateway (hub):             TGW
                                 /  |  \
                                A   B   C     A -> TGW -> C  works
```

> ***What to say to the interviewer:*** *VPC peering is a private one-to-one connection: one side requests, the other accepts, and both sides add routes and security group rules for each other's non-overlapping CIDR. It is not transitive, so A cannot reach C through B; I would either add a direct A-C peering or, at scale, use a Transit Gateway as a central hub with route tables, and PrivateLink if only one service needs exposing.*

Q25. How can Instance 2 (with a static IP) communicate with Instance 1, which is in a private subnet behind a multi-AZ load balancer?

In simple words: Instance 1 lives in a back room. Instance 2 never visits it directly; it rings the reception desk (the load balancer), and reception passes the message inside.

Detailed answer

- Instance 2 always talks to the **load balancer**, never directly to Instance 1's private IP.
- If Instance 2 is outside AWS or in another VPC over the internet: use an **internet-facing ALB/NLB** in public subnets across AZs. Allow Instance 2's **Elastic IP /32** in the LB security group.
- The target (Instance 1) security group allows traffic **only from the LB security group**.
- If the partner needs a fixed IP to allowlist on their side, use an **NLB with Elastic IPs per AZ** (or Global Accelerator).
- If Instance 2 is in the same or a peered VPC / TGW, use an **internal** load balancer and route privately.
- Instance 1 replies through the LB, so it needs no public IP or NAT for this flow.

Example:

```
Instance2 (EIP 3.110.10.5)
     |  HTTPS 443
     v
Internet-facing ALB (public-a, public-b)   SG: allow 443 from 3.110.10.5/32
     |  8080
     v
Instance1 (private-a 10.0.11.20)           SG: allow 8080 from alb-sg only
```

What to say to the interviewer: *Instance 2 calls the load balancer's DNS name, and the load balancer forwards to Instance 1 in the private subnet. I allow only Instance 2's static IP on the load balancer security group and only the load balancer security group on Instance 1, using an internal load balancer instead if both are in connected VPCs.*

Q26. Your application is hosted in S3 and users are in different countries. How do you reduce latency?

In simple words: Do not make everyone fly to one city to pick up a parcel; open pickup points in every country.

Detailed answer

1. Put **CloudFront** in front of the S3 bucket; objects are cached at edge locations near users.
2. Keep the bucket private with **Origin Access Control (OAC)**.
3. Set good **Cache-Control** headers (long TTL for versioned assets like `app.3f2a.js`).
4. Enable **compression** (gzip/Brotli) and HTTP/2 or HTTP/3.
5. For uploads from far away, use **S3 Transfer Acceleration** or CloudFront PUT.
6. For truly regional data, use **S3 Cross-Region Replication** plus latency-based routing.
7. Invalidate the cache on deploy (`/index.html`) instead of short TTLs everywhere.

Example:

```
aws cloudfront create-invalidation --distribution-id E1ABC2DEF3 --paths "/index.html"

Before: user in Sydney -> S3 us-east-1        ~250 ms
After : user in Sydney -> CloudFront Sydney   ~15 ms (cache hit)
```

> ***What to say to the interviewer:*** *I put CloudFront in front of the S3 bucket with Origin Access Control, so content is cached close to users worldwide. I tune Cache-Control headers, enable compression, invalidate on deploy, and for uploads use Transfer Acceleration or cross-region replication.*

Q27. You want to serve users in different countries. How do you route traffic based on user location?

> **In simple words:** Like a call center that routes your call to the office in your own country or the one with the shortest queue.

Detailed answer

Route 53 routing policies:

- **Geolocation:** route by user country/continent (for example, EU users -> eu-west-1 for GDPR). Always add a **default** record.
- **Latency-based:** send users to the region with the lowest measured latency.
- **Geoproximity:** by distance, with a bias to shift more or less traffic to a region.
- Attach **health checks** so unhealthy regions are skipped automatically.

Other options: **CloudFront** (edge caching, geo restriction), **Global Accelerator** (anycast static IPs, routes to nearest healthy region over the AWS backbone).

Example:

```
api.example.com
  Geolocation: Europe        -> ALB eu-west-1     (health check)
  Geolocation: Asia          -> ALB ap-south-1    (health check)
  Geolocation: Default       -> ALB us-east-1
```

> ***What to say to the interviewer:*** *I use Route 53 geolocation routing when data must stay in a region, or latency-based routing for best performance, always with health checks and a default record. For non-DNS routing I use Global Accelerator, and CloudFront for cached content.*

Q28. If the primary RDS database fails, how can you promote a read replica?

> **In simple words:** The team captain is injured, so you give the armband to the vice-captain. He becomes the new captain, but he may not know the very last play (async lag).

Detailed answer

1. Check **ReplicaLag**; replication is async, so recent writes may be lost.
2. Stop writes to the old primary if it is partially alive (avoid split brain).
3. Promote: the replica reboots and becomes a **standalone, writable** instance (no longer replicating).

4. Enable **Multi-AZ** and automated backups on the new primary.
5. **Repoint applications:** update a Route 53 CNAME (for example `db.internal`), Secrets Manager endpoint, or config.
6. Create new read replicas from the new primary.
7. For **Aurora**, failover to a replica is automatic and keeps the same cluster endpoint.

Example:

```
aws rds describe-db-instances --db-instance-identifier orders-replica-1 \
  --query "DBInstances[0].StatusInfos"
aws rds promote-read-replica --db-instance-identifier orders-replica-1 \
  --backup-retention-period 7
aws rds modify-db-instance --db-instance-identifier orders-replica-1 \
  --multi-az --apply-immediately
```

> ***What to say to the interviewer:*** *I check replica lag, fence the old primary, and run promote-read-replica, which makes the replica a standalone writable database. Then I repoint apps via a DNS CNAME or secret, enable Multi-AZ and backups, and rebuild replicas; with Aurora this failover is automatic.*

Q29. How do you handle database failover in RDS Multi-AZ?

> **In simple words:** A standby goalkeeper stands next to the goal, copying every move in real time. If the main keeper falls, the standby takes over within a minute, and the scoreboard (DNS name) stays the same.

Detailed answer

- **Multi-AZ instance:** a **synchronous** standby in another AZ; no data loss on failover.
- **Automatic failover** triggers on AZ outage, primary host failure, storage failure, or reboot-with-failover. Typical time **60-120 seconds**.
- The **endpoint DNS name does not change**; RDS flips the CNAME to the standby.
- **Multi-AZ DB cluster** (2 readable standbys) fails over faster, typically under 35 seconds.

What you must do on the app side:

- Use the endpoint name, never an IP; keep **DNS TTL low** (JVM: set `networkaddress.cache.ttl`).
- **Retry logic** with backoff and connection pool validation.
- Use **RDS Proxy** to shorten failover impact and keep connections.
- Subscribe to **RDS event notifications** (SNS) and **test failover** regularly.

Example:

```
aws rds reboot-db-instance --db-instance-identifier orders-prod --force-failover

aws rds create-event-subscription --subscription-name rds-failover \
  --sns-topic-arn arn:aws:sns:ap-south-1:111111111111:db-alerts \
  --source-type db-instance --event-categories failover
```

> ***What to say to the interviewer:*** *RDS Multi-AZ keeps a synchronous standby in another AZ and fails over automatically in about one to two minutes by flipping the endpoint DNS. I make apps*

resilient with retries, low DNS TTL and RDS Proxy, subscribe to failover events, and test with reboot --force-failover.

Q30. How can a developer securely access a private RDS instance without a bastion host?

In simple words: Instead of building a guard house, give the developer a temporary, recorded, private tunnel that only their identity can open.

Detailed answer

1. **SSM Session Manager port forwarding** through any SSM-managed instance or container in the VPC (no inbound ports, no SSH keys, IAM-controlled, logged).
2. **EC2 Instance Connect Endpoint** can open a TCP tunnel to private IPs.
3. **AWS Client VPN** for teams that need network access.
4. Combine with **IAM database authentication** (15-minute tokens) or Secrets Manager credentials, so no shared static passwords.
5. Keep RDS SG allowing only the tunnel source; log sessions to CloudWatch/S3.

Example:

```
aws ssm start-session --target i-0abc123def4567890 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["orders.abc123.ap-south-1.rds.amazonaws.com"],
                 "portNumber":["5432"],"localPortNumber":["15432"]}'

TOKEN=$(aws rds generate-db-auth-token --hostname orders.abc123... \
  --port 5432 --username dev_ro)
psql "host=localhost port=15432 user=dev_ro password=$TOKEN sslmode=require"
```

What to say to the interviewer: *I use SSM Session Manager port forwarding to the RDS endpoint, so there are no open ports or SSH keys and every session is logged. Combined with IAM database authentication tokens, developers get short-lived, auditable access tied to their own identity.*

Q31. How do you configure priority-based Auto Scaling?

In simple words: When hiring extra staff, you first call your favorite agency; only if they have nobody free do you call the second agency, then the third.

Detailed answer

"Priority" in AWS Auto Scaling usually means one of these:

- **Mixed Instances Policy, On-Demand allocation strategy** `prioritized`**:** ASG launches instance types in the order you list them in overrides.
- **Spot allocation** `capacity-optimized-prioritized`**:** prefers your order but still picks pools with spare capacity.
- **Base + percentage:** for example first 2 instances On-Demand, then 30 percent On-Demand, 70 percent Spot.
- **Multiple scaling policies:** if several fire at once, the ASG applies the one that gives the **largest capacity** (safest).

- **ECS capacity providers:** `base` and `weight` decide which provider (Fargate vs Fargate Spot) gets tasks first.

Example:

```
"MixedInstancesPolicy": {
  "LaunchTemplate": {
    "LaunchTemplateSpecification": { "LaunchTemplateName": "app-lt", "Version": "$Latest" },
    "Overrides": [
      { "InstanceType": "m7g.large" },
      { "InstanceType": "m6g.large" },
      { "InstanceType": "m5.large" }
    ]
  },
  "InstancesDistribution": {
    "OnDemandAllocationStrategy": "prioritized",
    "OnDemandBaseCapacity": 2,
    "OnDemandPercentageAboveBaseCapacity": 30,
    "SpotAllocationStrategy": "capacity-optimized-prioritized"
  }
}
```

> ***What to say to the interviewer:*** *I use a mixed instances policy where the On-Demand allocation strategy is prioritized, so the ASG tries instance types in my listed order, with Spot using capacity-optimized-prioritized. I also set an On-Demand base for critical capacity and remember that when multiple scaling policies fire, AWS picks the one giving the largest capacity.*

Q32. How do you automate ECS and EKS node scaling?

> **In simple words:** Two layers of hiring: first decide how many workers (pods/tasks) you need, then automatically rent enough desks (nodes) for them.

Detailed answer

ECS:

- **Service Auto Scaling** scales task count (target tracking on CPU, memory, or ALB requests per target).
- **Capacity providers with managed scaling** scale the EC2 ASG using the `CapacityProviderReservation` metric; managed termination protection avoids killing busy instances.
- Or use **Fargate** and skip nodes entirely.

EKS:

- **HPA/KEDA** scale pods.
- **Cluster Autoscaler** adds nodes to managed node groups when pods are Pending, removes underused nodes.
- **Karpenter** (recommended) launches right-sized instances directly in seconds, supports Spot and consolidation.
- **EKS Auto Mode** lets AWS manage node scaling for you.

Example:

```
# Karpenter NodePool
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: { name: default }
spec:
  template:
    spec:
      requirements:
        - { key: karpenter.sh/capacity-type, operator: In, values: ["spot","on-demand"] }
        - { key: kubernetes.io/arch, operator: In, values: ["arm64","amd64"] }
      nodeClassRef: { group: karpenter.k8s.aws, kind: EC2NodeClass, name: default }
  limits: { cpu: "200" }
  disruption: { consolidationPolicy: WhenEmptyOrUnderutilized }
```

What to say to the interviewer: *For ECS I combine Service Auto Scaling for tasks with capacity providers using managed scaling for the EC2 fleet, or use Fargate. For EKS, HPA or KEDA scale pods and Karpenter or Cluster Autoscaler add and remove nodes based on pending pods, with Karpenter also consolidating to save cost.*

Q33. How would you perform a zero-downtime deployment in AWS?

In simple words: Change the tires of a moving bus one at a time, or bring a second bus alongside and move passengers over only when it runs fine.

Detailed answer

Building blocks:

- An **ALB** in front, with **health checks** and a **deregistration delay** (connection draining) so in-flight requests finish.
- At least **2 instances/tasks across AZs**.

Strategies:

1. **Rolling:** ASG **instance refresh** or ECS rolling update with `minimumHealthyPercent=100`, `maximumPercent=200`.
2. **Blue/Green:** CodeDeploy for ECS/EC2 creates a green fleet, shifts ALB traffic (all at once, linear, or canary), and rolls back on CloudWatch alarms.
3. **Canary for Lambda:** aliases with weighted traffic shifting.
4. **Deployment circuit breaker** in ECS for automatic rollback.
5. Database changes must be **backward-compatible** (expand-contract).

Example:

```
aws autoscaling start-instance-refresh --auto-scaling-group-name web-asg \
  --preferences '{"MinHealthyPercentage":90,"InstanceWarmup":120,
                  "AutoRollback":true}'

aws ecs update-service --cluster prod --service orders \
  --deployment-configuration "deploymentCircuitBreaker={enable=true,rollback=true},
  maximumPercent=200,minimumHealthyPercent=100"
```

What to say to the interviewer: *I keep at least two targets behind an ALB with health checks and connection draining, then use rolling updates or CodeDeploy blue/green that shifts traffic gradually and rolls back on CloudWatch alarms. Database changes are made backward-compatible so old and*

new versions can run together.

Q34. How do you implement API Gateway with Lambda for microservices?

In simple words: API Gateway is the reception desk that checks visitors and sends each one to the right department (Lambda function). Departments only open when someone arrives.

Detailed answer

Design:

- **One Lambda per bounded context** (orders, users, payments), or one per route for small functions.
- **HTTP API** (cheaper, faster) or **REST API** (usage plans, API keys, request validation, WAF).
- **Auth:** Cognito / JWT authorizer or Lambda authorizer; IAM auth for service-to-service.
- Each function has its **own IAM role** (least privilege) and env config from SSM/Secrets Manager.
- **Data:** DynamoDB per service; async work through SQS/EventBridge.
- **Stages** (dev/prod) and **Lambda aliases** for safe deploys.
- **Observability:** X-Ray tracing, structured logs, CloudWatch alarms on 5XX and latency.
- Deploy with SAM, CDK, or Terraform.

Example:

```
Client -> API Gateway (JWT authorizer, throttling, WAF)
            |-- GET  /orders/{id}  -> Lambda orders-read  -> DynamoDB orders
            |-- POST /orders       -> Lambda orders-write -> DynamoDB + EventBridge
            +-- POST /payments     -> Lambda payments     -> SQS -> Lambda worker
```

What to say to the interviewer: *I front microservices with API Gateway routes mapped to separate Lambda functions, each with its own least-privilege role and data store, using JWT or Lambda authorizers for auth. Async flows go through SQS or EventBridge, and I use stages, aliases and X-Ray for safe deploys and tracing.*

Q35. How do you handle rate limiting and throttling in API Gateway?

In simple words: A tap with a flow limiter. It lets a steady stream through (rate) and allows a small splash at once (burst). Extra water is politely refused with "try again later" (HTTP 429).

Detailed answer

- API Gateway uses a **token bucket**: **rate** (requests/second) and **burst** (max at once).
- **Account-level** default per region: 10,000 rps steady, 5,000 burst (soft limit).
- **Stage and method throttling:** protect expensive endpoints, for example `POST /reports` at 20 rps.
- **Usage plans + API keys** (REST API): per-customer quotas like 1,000 requests/day.
- **AWS WAF rate-based rules** block abusive IPs.
- Protect the backend too: **Lambda reserved concurrency**, SQS buffering, DynamoDB on-demand.

- Clients should retry with **exponential backoff and jitter** on 429.
- Monitor the `Count`, `4XXError`, and Lambda `Throttles` metrics.

Example:

```
aws apigateway update-stage --rest-api-id a1b2c3 --stage-name prod \
  --patch-operations \
  op=replace,path=/*/*/throttling/rateLimit,value=500 \
  op=replace,path=/*/*/throttling/burstLimit,value=1000

aws lambda put-function-concurrency --function-name reports \
  --reserved-concurrent-executions 50
```

> ***What to say to the interviewer:*** *API Gateway throttles with a token bucket at account, stage and method levels, and REST APIs add usage plans with per-key quotas that return 429 when exceeded. I also add WAF rate-based rules, reserved concurrency on Lambda, and make clients retry with exponential backoff.*

Q36. What are best practices for AWS serverless services (Lambda, API Gateway, DynamoDB)?

> **In simple words:** Serverless is like taking taxis instead of owning cars. Cheap and easy, but plan for traffic jams (throttles), short trips (timeouts), and not leaving the meter running.

Detailed answer

Lambda:

- Small, single-purpose functions; initialize SDK clients **outside the handler** to reuse connections.
- Right-size memory (also gives more CPU); use **Graviton (arm64)**.
- Set sensible timeouts, **reserved concurrency** to protect downstreams, **provisioned concurrency** or SnapStart for cold-start-sensitive APIs.
- Make handlers **idempotent**; use **DLQs / on-failure destinations**.
- Least-privilege role per function; secrets from Secrets Manager, not plain env vars.

API Gateway: throttling, request validation, authorizers, caching, WAF.

DynamoDB: design around access patterns and a good partition key (avoid hot keys), on-demand for spiky traffic, TTL for expiring data, PITR backups, GSIs carefully.

Ops: IaC (SAM/CDK/Terraform), X-Ray, structured logs, alarms on Errors, Throttles, Duration.

Example:

```
import boto3
ddb = boto3.resource("dynamodb").Table("orders")   # created once, reused

def handler(event, context):
    ddb.put_item(Item={"pk": event["id"], "status": "NEW"},
                 ConditionExpression="attribute_not_exists(pk)")   # idempotent
```

> ***What to say to the interviewer:*** *I keep Lambdas small and idempotent, reuse connections outside the handler, right-size memory, and use reserved or provisioned concurrency and DLQs. API Gateway gets throttling and authorizers, and DynamoDB tables are designed around access*

patterns with good partition keys, TTL and PITR.

Q37. How would you design separate environments (Dev, Test, Prod) in AWS?

In simple words: Give each environment its own house rather than separate rooms in one house, so a fire in the test kitchen cannot reach production.

Detailed answer

- **Separate AWS accounts per environment** under **AWS Organizations** (strongest isolation of blast radius, billing and limits).
- **OU structure:** Security (log-archive, audit), Infrastructure (network, shared services), Workloads (dev, test, prod).
- **AWS Control Tower** or Account Factory for guardrails and account vending.
- **SCPs:** for example deny leaving approved regions, deny disabling CloudTrail, block public S3.
- **IAM Identity Center** for SSO; developers get broad access in dev, read-only in prod.
- **Non-overlapping CIDRs**, connected via Transit Gateway if needed.
- **Same IaC code**, different variables per env; promote the same artifact dev -> test -> prod.
- Tag everything (`env`, `owner`, `cost-center`); budgets per account.

Example:

```
Root (Organizations, SCPs)
 +-- Security OU      : log-archive, audit
 +-- Infrastructure OU: network (TGW), shared-services (ECR, CI)
 +-- Workloads OU
      +-- dev   (10.10.0.0/16)  developers: PowerUser
      +-- test  (10.20.0.0/16)
      +-- prod  (10.30.0.0/16)  developers: ReadOnly, deploy only via CI role
```

What to say to the interviewer: *I use a separate AWS account per environment under Organizations with OUs, SCP guardrails and SSO through IAM Identity Center. The same Terraform code with per-environment variables builds each one, CIDRs do not overlap, and only the CI/CD role can deploy to prod.*

Q38. What are best practices for VPC design at enterprise scale?

In simple words: Plan a city before building it: reserve non-overlapping street numbers, build roads in every district, and put one central highway interchange instead of private bridges between every pair of towns.

Detailed answer

- **IP planning:** non-overlapping CIDRs across all accounts and on-prem; use **VPC IPAM**; leave room for growth (EKS pods consume many IPs).
- **3 AZs**, tiered subnets: public (LB, NAT), private app, isolated data.
- **One NAT Gateway per AZ** for resilience (or a centralized egress VPC to save cost).
- **Hub-and-spoke with Transit Gateway**; separate TGW route tables for prod and non-prod.
- **Shared VPCs via AWS RAM** so a network team owns networking and app teams deploy into it.

- **VPC endpoints** (S3/DynamoDB gateway, others via interface) to cut NAT cost and keep traffic private.
- **Security:** SG-to-SG references, NACLs for coarse rules, Network Firewall in an inspection VPC.
- **Visibility:** VPC Flow Logs, Route 53 Resolver query logs.
- **Hybrid:** Direct Connect with VPN backup; Route 53 Resolver endpoints for hybrid DNS.
- Everything in **IaC**.

Example:

```
         On-prem ==DX/VPN==+
                           |
Inspection VPC <---> Transit Gateway <---> Egress VPC (NAT x3)
                      /     |      \
                prod VPC  test VPC  shared-services VPC
                3 AZ x (public | app | data) subnets
```

> ***What to say to the interviewer:*** *I start with an IPAM-managed, non-overlapping CIDR plan, build VPCs across three AZs with public, app and data tiers, and connect them hub-and-spoke through Transit Gateway with separate route tables. I add VPC endpoints, centralized egress and inspection, flow logs, and manage it all with Terraform.*

Q39. How do you design a highly available architecture across Multi-AZ and Multi-Region?

Also asked as: Design a highly available backend on AWS.

> **In simple words:** Multi-AZ is having two kitchens in the same city so one fire does not stop dinner. Multi-Region is having a kitchen in another city in case the whole city loses power.

Detailed answer

Multi-AZ (default for production):

- ALB across 3 AZs; ASG/EKS nodes spread across AZs.
- RDS Multi-AZ, ElastiCache with replicas and auto-failover, MSK across 3 AZs.
- NAT Gateway per AZ; stateless apps; sessions in Redis/DynamoDB.

Multi-Region (for DR or global users):

- **Data replication:** Aurora Global Database, DynamoDB Global Tables, S3 Cross-Region Replication, cross-region RDS replicas.
- **Traffic:** Route 53 failover/latency records with health checks, or Global Accelerator.
- **Infra parity** through the same IaC in both regions; images replicated in ECR.
- Choose **active-passive** (cheaper) or **active-active** (complex, needs conflict handling).
- Regularly **test failover** (game days).

Example:

```
          Route 53 / Global Accelerator (health checks)
             /                                  \
ap-south-1 (primary)                    ap-southeast-1 (DR)
ALB -> EKS (3 AZ)                        ALB -> EKS (scaled down)
Aurora writer ======= Global DB =======> Aurora reader (promote on DR)
S3 bucket ============ CRR ============> S3 bucket
```

What to say to the interviewer: *Within a region I spread every layer across three AZs: ALB, autoscaled compute, Multi-AZ databases and per-AZ NAT, keeping apps stateless. For multi-region I replicate data with Aurora Global Database, DynamoDB Global Tables and S3 CRR, deploy the same IaC in both regions, and fail over with Route 53 health checks or Global Accelerator.*

Q40. What is your strategy for Disaster Recovery and Backup in AWS?

Also asked as: Do you have disaster recovery experience? What is your disaster recovery plan for a regional outage?

In simple words: Decide two numbers first: how much data you can afford to lose (RPO) and how long you can be down (RTO). Then pick the cheapest plan that meets them.

Detailed answer

- **RPO (Recovery Point Objective):** the maximum data loss you accept, measured in time (for example "the last 5 minutes of orders").
- **RTO (Recovery Time Objective):** the maximum time the service may be down before it is back.

Strategy	RPO / RTO	Cost	How
Backup and Restore	hours / hours	Lowest	Snapshots copied to another region, rebuild with IaC
Pilot Light	minutes / tens of minutes	Low	Data replicated live; core infra off or minimal
Warm Standby	seconds-minutes / minutes	Medium	Scaled-down full copy running
Active-Active	near zero / near zero	Highest	Both regions serve traffic

Backup practices:

- **AWS Backup** plans for EBS, RDS, DynamoDB, EFS, S3; cross-region and **cross-account copies**.
- **Vault Lock / Object Lock** to protect against ransomware or accidental deletion.
- RDS automated backups + PITR; Velero for Kubernetes objects and volumes.
- Everything rebuildable from **IaC** (see the Terraform chapter for multi-region modules, remote state and state backups).
- **Test restores regularly** and verify data (checksums, row counts). A backup never restored is only a hope.

Regional outage runbook (warm standby / pilot light):

1. **Declare the incident:** confirm via AWS Health Dashboard and your own health checks; the incident commander decides to fail over using pre-agreed criteria.

2. **Fail over data:** promote the **Aurora Global Database** secondary (or the RDS cross-region read replica); DynamoDB Global Tables are already active in both regions; S3 CRR buckets already hold the data.
3. **Bring up compute:** scale up the warm standby (ASG/EKS node pools) or apply the same IaC in the DR region for pilot light.
4. **Shift traffic:** Route 53 failover records with health checks, **Route 53 ARC routing controls** (data-plane switches built for this), or Global Accelerator traffic dials.
5. **Verify:** smoke tests, synthetic checks, dashboards, error rates.
6. **Communicate:** status page and stakeholder updates at fixed intervals.
7. **Fail back later** in a calm window after re-syncing data to the primary region.

Static stability: failover must not depend on the failed region's control plane (you should not need to edit resources in the broken region). Use data-plane mechanisms like ARC routing controls and pre-provisioned capacity. Run **game-day drills** regularly (for example quarterly) and record the real RTO/RPO achieved.

Example:

```
aws backup create-backup-plan --backup-plan '{
  "BackupPlanName": "prod-daily",
  "Rules": [{ "RuleName": "daily", "TargetBackupVaultName": "prod-vault",
    "ScheduleExpression": "cron(0 20 * * ? *)",
    "Lifecycle": { "DeleteAfterDays": 35 },
    "CopyActions": [{ "DestinationBackupVaultArn":
      "arn:aws:backup:ap-southeast-1:333333333333:backup-vault:dr-vault" }] }]}'
```

> ***What to say to the interviewer:*** *I start from the business RPO, how much data we can lose, and RTO, how long we can be down, and choose between backup-restore, pilot light, warm standby or active-active. I use AWS Backup with cross-region and cross-account copies and IaC to rebuild, and for a regional outage my runbook promotes the Aurora Global Database secondary, scales up the standby, shifts traffic with ARC routing controls or Route 53 failover, verifies and communicates, and we rehearse it in regular game days.*

Q41. How do you handle secret rotation using Secrets Manager or Parameter Store?

> **In simple words:** Change the locks on a schedule automatically, and make sure everyone always asks the key cabinet for the current key instead of keeping an old copy in their pocket.

Detailed answer

Secrets Manager (built for rotation):

- **Managed rotation** for RDS master passwords (`--manage-master-user-password`).
- **Rotation Lambda** for other secrets: steps `createSecret -> setSecret -> testSecret -> finishSecret`, using staging labels `AWSPENDING` and `AWSCURRENT`.
- **Alternating-users strategy** so connections using the old user keep working during rotation.
- Schedule, for example every 30 days.

Parameter Store: no native rotation; use EventBridge schedule + Lambda, or migrate rotating secrets to Secrets Manager.

Apps must read secrets at runtime (SDK with caching) or via External Secrets Operator / Secrets Store CSI on EKS, and reload on change.

Example:

```
aws secretsmanager rotate-secret --secret-id prod/orders/db \
  --rotation-lambda-arn arn:aws:lambda:ap-south-1:111111111111:function:pg-rotator \
  --rotation-rules '{"ScheduleExpression":"rate(30 days)"}'

aws rds modify-db-instance --db-instance-identifier orders-prod \
  --manage-master-user-password --apply-immediately
```

> ***What to say to the interviewer:*** *I use Secrets Manager rotation, managed for RDS and via a rotation Lambda with the alternating-users strategy for other databases, on a fixed schedule. Applications fetch secrets at runtime with caching, or through External Secrets Operator on EKS, so rotations happen without redeploys.*

Q42. What is your approach for centralized secrets management for 100+ microservices?

> **In simple words:** One secure bank vault with a separate locker per service. Each service's badge opens only its own locker, and the vault logs every opening.

Detailed answer

- **Single source of truth:** AWS Secrets Manager (or HashiCorp Vault) with a naming convention `/<env>/<service>/<name>`.
- **Identity-based access:** each service gets its own IAM role (IRSA or EKS Pod Identity, ECS task role) allowed only `secretsmanager:GetSecretValue` on its own path.
- **Delivery into apps:** External Secrets Operator syncs to Kubernetes Secrets, or Secrets Store CSI driver mounts them; ECS injects via `secrets` in the task definition.
- **Encryption** with a customer-managed KMS key per environment.
- **Rotation** automated; apps reload.
- **Cross-account** sharing via resource policies when needed.
- **Audit** with CloudTrail; alert on unusual access. Never store secrets in Git (use SOPS or sealed secrets for GitOps if needed).
- Enforce with policy (OPA/Kyverno: no plain env secrets).

Example:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: orders-db, namespace: orders }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: aws-secrets-manager, kind: ClusterSecretStore }
  target: { name: orders-db }
  data:
    - secretKey: password
      remoteRef: { key: prod/orders/db, property: password }
```

What to say to the interviewer: *I store all secrets centrally in Secrets Manager with a per-environment, per-service path and give each service its own IAM role scoped to its path through IRSA or task roles. External Secrets Operator delivers them into pods, KMS encrypts them, rotation is automated and CloudTrail audits access.*

Q43. What strategy do you use to rotate access keys automatically?

In simple words: The best key is no key at all. If you must have one, replace it on a timer and throw the old one away after everyone has the new one.

Detailed answer

1. **Eliminate long-lived keys first:** IAM roles for EC2/Lambda, IRSA/Pod Identity for EKS, **OIDC federation** for GitHub Actions/GitLab/Jenkins, IAM Identity Center for humans.
2. For unavoidable keys (legacy third-party tools):

- An **EventBridge scheduled Lambda** creates a new key (IAM allows 2 per user), stores it in **Secrets Manager**, lets consumers pick it up, then **deactivates** the old key after a grace period and **deletes** it later.
- Or use Secrets Manager rotation with a custom Lambda for IAM keys.

3. **Detect** with AWS Config rule `access-keys-rotated` (for example max 90 days) and IAM credential report; alert on keys unused for 90 days.
4. **SCP** can deny `iam:CreateAccessKey` except for approved roles.

Example:

```
aws iam create-access-key --user-name legacy-tool
aws secretsmanager put-secret-value --secret-id legacy-tool/keys --secret-string file://k.json
aws iam update-access-key --user-name legacy-tool --access-key-id AKIAOLD... --status Inactive
aws iam delete-access-key --user-name legacy-tool --access-key-id AKIAOLD...

aws configservice put-config-rule --config-rule '{"ConfigRuleName":"keys-rotated",
  "Source":{"Owner":"AWS","SourceIdentifier":"ACCESS_KEYS_ROTATED"},
  "InputParameters":"{\"maxAccessKeyAge\":\"90\"}"}'
```

What to say to the interviewer: *My main strategy is to remove access keys entirely using roles, IRSA and OIDC for CI/CD. For the few that must exist, a scheduled Lambda creates a new key, stores it in Secrets Manager, deactivates and later deletes the old one, and an AWS Config rule flags any key older than 90 days.*

Q44. What metrics do you monitor for EC2, ECS, and RDS using CloudWatch?

Also asked as: How do you monitor AWS resources with CloudWatch?

In simple words: Like a car dashboard: speed (CPU), fuel (memory/disk), engine warnings (status checks), and for the database, how many passengers (connections) and how slow the doors open (latency).

Detailed answer

Service	Key metrics
EC2	CPUUtilization, CPUCreditBalance (T types), StatusCheckFailed_System/Instance, NetworkIn/Out, EBS VolumeQueueLength, BurstBalance; memory and disk via CloudWatch Agent
ECS	Service CPUUtilization, MemoryUtilization, RunningTaskCount vs desired, Container Insights (restarts, task-level), ALB TargetResponseTime, HTTPCode_Target_5XX, UnHealthyHostCount
RDS	CPUUtilization, DatabaseConnections, FreeableMemory, FreeStorageSpace, Read/WriteLatency, Read/WriteIOPS, DiskQueueDepth, ReplicaLag, SwapUsage, BurstBalance; Performance Insights for top SQL

CloudWatch building blocks:

- **Metrics:** free default metrics per service (EC2 every 5 min, 1 min with detailed monitoring); custom metrics via `put-metric-data`.
- **CloudWatch Agent:** needed for **memory and disk** usage on EC2 (not sent by default) and to ship log files.
- **Alarms:** threshold or anomaly detection, composite alarms to reduce noise, actions to SNS, Auto Scaling, or EC2 recover.
- **Logs:** log groups with retention settings; **metric filters** turn log lines (for example "ERROR") into metrics.
- **Logs Insights:** query language to search and aggregate logs quickly.
- **Dashboards:** one view per service or environment; **Container Insights** for ECS/EKS, **Synthetics** canaries for URLs.

Good alarm practice: alarm on **symptoms** (latency, errors) first, use **percentiles (p95/p99)**, require several datapoints to avoid noise, and send to SNS -> Teams/Slack/pager.

Example:

```
# Logs Insights: top 5XX paths in the last hour
fields @timestamp, path, status
| filter status >= 500
| stats count(*) as errors by path
| sort errors desc
| limit 10
```

```
aws cloudwatch put-metric-alarm --alarm-name rds-orders-low-storage \
  --namespace AWS/RDS --metric-name FreeStorageSpace \
  --dimensions Name=DBInstanceIdentifier,Value=orders-prod \
  --statistic Average --period 300 --evaluation-periods 3 \
  --threshold 10737418240 --comparison-operator LessThanThreshold \
  --alarm-actions arn:aws:sns:ap-south-1:111111111111:db-alerts
```

> ***What to say to the interviewer:*** *For EC2 I watch CPU, credit balance, status checks and memory/disk through the CloudWatch agent; for ECS, service CPU and memory, running vs desired tasks and ALB errors and latency; for RDS, CPU, connections, free storage and memory, latency, IOPS and replica lag. I combine metrics, log groups with metric filters, Logs Insights queries and*

dashboards, and alarms use multiple datapoints and route through SNS to our Teams channels.

Q45. How do you optimize AWS cost for EC2, NAT, and S3?

Also asked as: What approaches do you use to reduce AWS costs?

In simple words: Stop paying for lights in empty rooms, buy in bulk what you always use, use discount seats for flexible trips, and move old boxes to cheaper storage.

Detailed answer

EC2:

- **Rightsize** with Compute Optimizer; move to **Graviton** (about 20 percent cheaper).
- **Savings Plans / Reserved Instances** for steady baseline; **Spot** for stateless, batch, CI, and EKS worker pools.
- **Schedule** dev/test off nights and weekends; delete idle instances, unattached EBS, old snapshots, unused EIPs; gp2 -> **gp3**.

NAT Gateway (charged per hour + per GB processed):

- Add **S3 and DynamoDB gateway endpoints** (free) so that traffic skips NAT.
- Interface endpoints for ECR, CloudWatch Logs when volume is high.
- Keep traffic in the same AZ (avoid cross-AZ data charges); find top talkers with VPC Flow Logs.

S3:

- Lifecycle to IA/Glacier, **Intelligent-Tiering** for unknown patterns, expire noncurrent versions, abort incomplete multipart uploads; use **Storage Lens**.

Governance: tagging, Cost Explorer, Budgets, **Cost Anomaly Detection**.

Example:

```
aws ec2 describe-volumes --filters Name=status,Values=available \
  --query "Volumes[].{ID:VolumeId,GB:Size}" --output table   # orphaned EBS

aws ce get-cost-and-usage --time-period Start=2026-09-01,End=2026-09-24 \
  --granularity MONTHLY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=USAGE_TYPE
```

What to say to the interviewer: *For EC2 I rightsize, move to Graviton, cover the baseline with Savings Plans and use Spot for flexible workloads, while shutting down non-prod off-hours. For NAT I add free S3 and DynamoDB gateway endpoints, and for S3 I use lifecycle rules and Intelligent-Tiering, all tracked with tags, Budgets and Cost Anomaly Detection.*

Q46. What are best practices to protect an AWS-hosted web application from attacks?

In simple words: Layers like a castle: a moat (Shield), guards at the gate checking IDs (WAF), locked inner doors (private subnets and security groups), and cameras everywhere (GuardDuty, logs).

Detailed answer

1. **Edge:** CloudFront + **AWS WAF** (managed rules for OWASP Top 10, SQL injection, XSS, bad bots, rate-based rules, geo blocking) + **Shield** Standard (free) or Advanced (DDoS response team and cost protection).
2. **Network:** ALB in public subnets, app and DB in private subnets; SGs allow only what is needed; no SSH from internet (use SSM).
3. **TLS everywhere** with ACM; HSTS; redirect HTTP -> HTTPS.
4. **Identity:** least-privilege IAM roles, MFA, no root usage, no long-lived keys.
5. **Data:** KMS encryption at rest, Secrets Manager for credentials, private S3 with OAC.
6. **Patching and images:** hardened AMIs, Inspector scanning, ECR image scanning.
7. **Detection:** GuardDuty, Security Hub, CloudTrail, Config, VPC Flow Logs, WAF logs.
8. **App level:** input validation, authentication (Cognito), security headers, dependency scanning.
9. **Resilience:** autoscaling to absorb load, backups, incident runbooks.

Example:

```
User -> Route53 -> CloudFront (Shield) -> WAF (OWASP, rate 2000/5min/IP)
     -> ALB (443 only, ACM) -> App in private subnets (SG from ALB only)
     -> RDS in isolated subnets (SG from app only, KMS encrypted)
GuardDuty + Security Hub + CloudTrail watching all accounts
```

> ***What to say to the interviewer:*** *I use defense in depth: CloudFront with WAF managed and rate-based rules and Shield at the edge, private subnets with tight security groups, TLS everywhere, least-privilege IAM and encrypted data. Then I detect with GuardDuty, Security Hub and CloudTrail and keep images patched and scanned.*

Q47. Can you write a custom IAM policy? Explain its structure and how AWS evaluates policies.

> **In simple words:** An IAM policy is a permission slip: who may do which actions, on which things, under which conditions. If any slip anywhere says "NO", the answer is no.

Detailed answer

Structure of a statement:

- **Effect:** `Allow` or `Deny`.
- **Action:** API calls, for example `s3:GetObject`.
- **Resource:** ARNs the actions apply to (scope as narrowly as possible).
- **Condition:** extra rules, for example source VPC endpoint, MFA present, tag match, TLS only.
- **Principal:** only in resource-based policies (bucket policy, trust policy).

Managed vs inline:

- **AWS managed:** written by AWS, broad, convenient (for example `ReadOnlyAccess`).
- **Customer managed:** your reusable, versioned policies (preferred).
- **Inline:** embedded in one user/role, deleted with it; use for strict one-to-one cases.

Evaluation logic:

1. Everything starts as **implicit deny**.
2. An **explicit Deny** anywhere (SCP, permission boundary, identity, resource, session policy) **always wins**.
3. Otherwise an **Allow** is needed, and it must be allowed by every applicable layer (SCP and permission boundary must also allow).
4. Cross-account: both the identity policy and the resource policy must allow.

Example (least privilege: read/write only one prefix, only through our VPC endpoint):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListOwnPrefix",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::app-data",
      "Condition": { "StringLike": { "s3:prefix": ["orders/*"] } }
    },
    {
      "Sid": "ReadWriteOwnPrefix",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::app-data/orders/*",
      "Condition": { "StringEquals": { "aws:SourceVpce": "vpce-0abc123" } }
    }
  ]
}
```

```
aws iam create-policy --policy-name orders-s3-rw --policy-document file://policy.json
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::111111111111:role/orders \
  --action-names s3:DeleteObject --resource-arns arn:aws:s3:::app-data/orders/x
```

> ***What to say to the interviewer:*** *Yes, a policy has statements with Effect, Action, Resource and optional Condition, and I write customer-managed policies scoped to exact ARNs and prefixes with conditions like source VPC endpoint. AWS starts from implicit deny, an explicit deny anywhere always wins, and an action is allowed only if every layer such as SCPs and boundaries also allows it; I test with the IAM policy simulator.*

Q48. Two employees work different shifts (10 AM-5 PM and 6 PM-2 AM). How do you give them AWS access based on time?

> **In simple words:** IAM can say "this pass is valid until 5 PM on 3rd October", but it cannot say "valid every day from 10 to 5". For a daily schedule you need a small robot that hands out and takes back the pass on time.

Detailed answer

Important limitation: the condition keys `aws:CurrentTime` and `aws:EpochTime` compare against an **absolute date-time**. IAM has **no native recurring daily time-window** condition, so a single static policy cannot express "10:00-17:00 every day".

Practical options:

1. **Scheduled automation:** EventBridge Scheduler triggers a Lambda at shift start to attach a policy (or create an IAM Identity Center **permission-set assignment**) and at shift end to detach/remove it. Handle time zones carefully (IST 10:00 = 04:30 UTC).
2. **Time-bounded policy renewed daily:** the Lambda rewrites the policy each day with that day's `DateGreaterThan`/`DateLessThan` window.
3. **Temporary elevated access** (for example AWS TEAM with IAM Identity Center): users request access for their shift, it is approved and auto-expires.
4. **Short session duration** on the permission set / role (for example 7-8 hours) so sessions do not outlive the shift. Note that revoking a policy does not kill an already-issued session, so short sessions matter.
5. Audit with CloudTrail; alert on off-shift activity.

Example (one day's window for the 10 AM-5 PM IST user, written in UTC):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["ec2:Describe*", "cloudwatch:Get*", "logs:*"],
    "Resource": "*",
    "Condition": {
      "DateGreaterThan": { "aws:CurrentTime": "2026-10-01T04:30:00Z" },
      "DateLessThan":    { "aws:CurrentTime": "2026-10-01T11:30:00Z" }
    }
  }]
}
```

```
# EventBridge Scheduler (IST) calling a Lambda that grants/revokes access
aws scheduler create-schedule --name grant-shift-a \
  --schedule-expression "cron(0 10 * * ? *)" \
  --schedule-expression-timezone Asia/Kolkata \
  --flexible-time-window Mode=OFF \
  --target file://grant-shift-a-target.json
```

> ***What to say to the interviewer:*** *IAM's aws:CurrentTime condition only compares against fixed dates, so there is no built-in daily recurring time window. I would use EventBridge Scheduler with a Lambda to grant and revoke the permission-set assignment at each shift boundary, or a just-in-time access tool like TEAM, with short session durations and CloudTrail alerts for off-shift activity.*

Q49. How do you connect AWS to an on-premises data center?

> **In simple words:** Either build an encrypted tunnel over the public internet (VPN - quick and cheap), or lay a private dedicated cable into AWS (Direct Connect - fast and steady).

Detailed answer

Option	What	When
Site-to-Site VPN	Two IPsec tunnels over the internet to a VGW or TGW	Quick setup, backup link, lower bandwidth
Direct Connect (DX)	Dedicated private link 1/10/100 Gbps via a DX location	Steady high bandwidth, low latency, lower data-out cost
DX + VPN backup	Primary DX, VPN failover	Production hybrid
Transit Gateway	Hub attaching VPN/DX to many VPCs	Many VPCs/accounts
Client VPN	Per-user OpenVPN access	Remote engineers

Other pieces:

- **BGP** for dynamic routing; non-overlapping CIDRs between on-prem and VPCs.
- **Hybrid DNS:** Route 53 Resolver **inbound endpoints** (on-prem resolves AWS private zones) and **outbound endpoints + forwarding rules** (AWS resolves `corp.local`).
- Security Groups/NACLs allow on-prem ranges; firewall rules on-prem.
- For migrations: Storage Gateway, DataSync, DMS, Application Migration Service.

Example:

```
On-prem DC (192.168.0.0/16, BGP router)
   ||== Direct Connect (primary) ==||
   ||-- Site-to-Site VPN (backup) --||
               Transit Gateway
            /        |         \
        prod VPC   test VPC   shared VPC (Route 53 Resolver in/out endpoints)
```

> ***What to say to the interviewer:*** *I connect on-premises through a Site-to-Site VPN for quick or backup connectivity and Direct Connect for dedicated high bandwidth, usually terminated on a Transit Gateway so all VPCs can use it with BGP routing. For DNS I use Route 53 Resolver inbound and outbound endpoints, and Client VPN for individual users.*

Q50. How do you manage and connect to services like EC2, RDS, EKS and ECS?

> **In simple words:** Every service has its own secure front door, and all of them open with your AWS identity instead of shared passwords or open ports.

Detailed answer

- **EC2:** SSM Session Manager (no SSH port, no keys, logged).
- **EKS:** `aws eks update-kubeconfig` writes a kubeconfig that authenticates with your IAM identity (mapped via EKS access entries); then use `kubectl`.
- **ECS: ECS Exec** opens a shell in a running container. Needs `enableExecuteCommand` on the service/task, the SSM agent (ECS injects it automatically), and the **task role** allowing the `ssmmessages` channel actions.
- **RDS:** SSM port forwarding through an instance in the VPC, then a normal DB client to `localhost`.
- Everything else managed through **IaC** (Terraform) and the CLI, with IAM Identity Center for human access.

Example:

```
# EC2
aws ssm start-session --target i-0abc123def4567890

# EKS
aws eks update-kubeconfig --name prod-eks --region ap-south-1
kubectl get nodes

# ECS Exec
aws ecs update-service --cluster prod --service orders \
  --enable-execute-command --force-new-deployment
aws ecs execute-command --cluster prod --task 0f1e2d3c4b5a \
  --container orders --interactive --command "/bin/sh"

# RDS through SSM port forwarding
aws ssm start-session --target i-0abc123def4567890 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["orders.abc123.ap-south-1.rds.amazonaws.com"],
                 "portNumber":["5432"],"localPortNumber":["15432"]}'
psql -h localhost -p 15432 -U app_ro orders
```

What to say to the interviewer: *I use IAM-based access everywhere: SSM Session Manager for EC2, aws eks update-kubeconfig plus kubectl for EKS, ECS Exec with enableExecuteCommand and the right task role for containers, and SSM port forwarding to reach private RDS. Nothing needs public IPs, open SSH ports or shared keys, and all sessions are logged.*

Q51. How do you establish database connections from your applications in deployments?

In simple words: The app looks up the database address and password in a safe (Secrets Manager), travels only on private roads (private subnets, security groups), and shares a few taxis (a connection pool) instead of calling a new one for every trip.

Detailed answer

1. **Endpoint and credentials** come from config/Secrets Manager (via External Secrets on EKS or ECS task `secrets`), never hard-coded in the image or Git.
2. **Network:** DB in private/data subnets, no public access; the DB security group allows 5432/3306 **only from the app security group** (SG-to-SG reference).
3. **Connection pooling** in the app (HikariCP, pgbouncer, SQLAlchemy pool) and **RDS Proxy** for spiky or Lambda workloads.
4. **Authentication:** Secrets Manager with rotation, or **IAM database authentication** tokens via IRSA/task role.
5. **TLS** in transit (`sslmode=require` or `verify-full` with the RDS CA bundle); KMS encryption at rest.
6. **Resilience:** use the DNS endpoint (not IP), retries with backoff, a separate reader endpoint for read traffic.

Example:

```
# EKS Deployment snippet
env:
  - name: DB_HOST
    value: orders-proxy.proxy-abc123.ap-south-1.rds.amazonaws.com
  - name: DB_PASSWORD
    valueFrom: { secretKeyRef: { name: orders-db, key: password } }
  - name: DB_SSLMODE
    value: verify-full
```

```
aws ec2 authorize-security-group-ingress --group-id sg-rds \
  --protocol tcp --port 5432 --source-group sg-app
```

> ***What to say to the interviewer:*** *The app gets the DB endpoint and credentials from Secrets Manager, connects over TLS from private subnets, and the database security group only accepts traffic from the app's security group. I use connection pooling or RDS Proxy, rotate credentials or use IAM auth tokens, and always connect by DNS endpoint with retries.*

Q52. How do you create AWS Lambda functions and manage their deployment artifacts?

> **In simple words:** Package your code like a parcel (a zip or a container image), store it in a warehouse (S3 or ECR), label every version, and use a pointer (alias) that you can slowly move to the new parcel and move back if it breaks.

Detailed answer

Packaging:

- **Zip package** (50 MB direct upload, 250 MB unzipped) uploaded to **S3** with a versioned key, for example `lambda/orders/1.4.2.zip`.
- **Container image** (up to 10 GB) pushed to **ECR**, tagged with the git SHA.
- **Layers** for shared dependencies/libraries across functions.

Tooling: AWS SAM, Serverless Framework, CDK or Terraform; CI builds, tests and uploads the artifact, then updates the function.

Versions and aliases:

- Publishing creates an immutable **version**.
- An **alias** (`live`, `prod`) points to a version; clients invoke the alias.
- **Canary with CodeDeploy:** weighted alias shifts, for example 10 percent for 5 minutes then 100 percent, with automatic rollback on CloudWatch alarms (SAM `DeploymentPreference`).

Example:

```
aws s3 cp build/orders-1.4.2.zip s3://artifacts-111111111111/lambda/orders/1.4.2.zip
aws lambda update-function-code --function-name orders \
  --s3-bucket artifacts-111111111111 --s3-key lambda/orders/1.4.2.zip --publish
aws lambda update-alias --function-name orders --name live --function-version 7 \
  --routing-config '{"AdditionalVersionWeights":{"8":0.1}}'
```

```
# SAM template: automatic canary + rollback
OrdersFunction:
  Type: AWS::Serverless::Function
  Properties:
    CodeUri: src/
    Handler: app.handler
    Runtime: python3.12
    AutoPublishAlias: live
    DeploymentPreference:
      Type: Canary10Percent5Minutes
      Alarms: [!Ref OrdersErrorsAlarm]
```

What to say to the interviewer: *CI builds the function as a versioned zip in S3 or a container image in ECR, with shared libraries in layers, and deploys it with SAM or Terraform. Each release publishes an immutable version, clients call an alias, and CodeDeploy shifts the alias gradually, for example 10 percent for 5 minutes, rolling back automatically on alarms.*

Q53. What is the hub-and-spoke network model?

In simple words: Like an airline hub: instead of direct flights between every pair of small cities, all flights go through one big airport. Fewer routes, one place for security checks.

Detailed answer

- **Hub:** an **AWS Transit Gateway** (a regional router) that every VPC and on-prem link attaches to.
- **Spokes:** workload VPCs (prod, test, per-team) that only connect to the hub, not to each other directly.
- **Shared VPCs on the hub:**
- **Shared services VPC:** CI runners, AD/DNS, artifact repos, monitoring.
- **Egress VPC: centralized NAT Gateways** (and proxy) so spokes do not each pay for NAT.
- **Inspection VPC:** AWS Network Firewall or third-party appliances (via Gateway Load Balancer) for east-west and north-south traffic.
- **TGW route tables** create segmentation, for example prod spokes cannot reach dev spokes, but both reach shared services.
- **Versus a peering mesh:** N VPCs need N x (N-1) / 2 peerings (10 VPCs = 45), with no transitive routing and no central inspection. Hub-and-spoke scales to thousands of VPCs and one VPN/DX serves all.
- **Trade-offs:** TGW charges per attachment-hour and per GB, and adds a hop.
- It is the same idea as **Azure hub-spoke** (hub VNet with firewall/VPN gateway, peered spoke VNets) and GCP hub-and-spoke with Network Connectivity Center.

Example:

```
                On-prem (VPN / Direct Connect)
                          |
   Egress VPC (NAT) -- Transit Gateway -- Inspection VPC (Network Firewall)
                     /      |       \
               prod VPC  dev VPC  shared-services VPC
   TGW RT "prod": prod -> shared, egress, on-prem     (no route to dev)
   TGW RT "dev" : dev  -> shared, egress              (no route to prod)
```

What to say to the interviewer: *Hub-and-spoke puts a Transit Gateway at the center with workload VPCs as spokes, plus shared services, centralized egress NAT and an inspection VPC attached to the hub. TGW route tables segment prod from dev, and it scales far better than a peering mesh, which needs N times N minus 1 over 2 connections and has no transitive routing.*

Q54. If you do not want to use Auto Scaling policies, how else can you scale EC2?

In simple words: Instead of an automatic thermostat, you can use a timer (schedules), a helper who reacts to alarms (Lambda), a forecast (predictive scaling), or just turn the knob yourself (manual).

Detailed answer

1. **Scheduled actions** on the ASG: set min/max/desired at fixed times (for example scale to 20 before 9 AM, back to 4 at 9 PM).
2. **Predictive scaling:** ML forecasts daily/weekly patterns and scales ahead of time (can run in forecast-only mode first).
3. **CloudWatch alarm -> SNS/EventBridge -> Lambda:** your own logic calls `set-desired-capacity`, or launches/starts/stops standalone instances.
4. **EventBridge Scheduler** calling EC2 start/stop APIs directly (great for dev/test office hours).
5. **Manual:** `set-desired-capacity` from CLI/console or a CI job before an event.
6. **Vertical scaling:** stop, change instance type, start (causes downtime for that instance).

Important: **CloudWatch alarm EC2 actions** can only **stop, terminate, reboot or recover** an instance. They cannot add capacity, so scaling out always needs an ASG action, a Lambda, or you.

Example:

```
aws autoscaling put-scheduled-update-group-action --auto-scaling-group-name web-asg \
  --scheduled-action-name office-hours-up --recurrence "30 3 * * MON-FRI" \
  --min-size 6 --desired-capacity 10 --max-size 20          # 09:00 IST = 03:30 UTC

aws autoscaling set-desired-capacity --auto-scaling-group-name web-asg \
  --desired-capacity 15 --honor-cooldown

aws scheduler create-schedule --name stop-dev-nightly \
  --schedule-expression "cron(0 21 * * ? *)" --schedule-expression-timezone Asia/Kolkata \
  --flexible-time-window Mode=OFF --target file://stop-dev-target.json
```

What to say to the interviewer: *Without dynamic policies I can use ASG scheduled actions, predictive scaling, or a CloudWatch alarm that triggers a Lambda through SNS or EventBridge to call set-desired-capacity, plus EventBridge Scheduler to start and stop instances. Alarm EC2 actions can only stop, terminate, reboot or recover, so they cannot add capacity by themselves.*

Scenario-Based

Q55. An application behind an ALB is returning 503 errors. How would you troubleshoot it?

In simple words: 503 means the receptionist (ALB) is at the desk but has nobody to pass your request to. So first check whether any healthy staff (targets) are sitting behind the desk.

Detailed answer

First, who produced the error? Compare `HTTPCode_ELB_5XX_Count` (ALB generated it) with `HTTPCode_Target_5XX_Count` (your app returned it).

If the ALB returned 503:

1. **Target group has no registered targets** (ASG scaled to 0, ECS tasks all stopped, wrong target group on the listener rule).
2. **Targets unhealthy:** check `describe-target-health` reasons (`Target.Timeout`, `Target.ResponseCodeMismatch`, `Target.FailedHealthChecks`). Note: if all targets are unhealthy, ALB "fails open" and still sends traffic, so errors then look like 502/504 too.
3. **Health check misconfig:** wrong path/port, app returns 301/401 on `/health`, too short timeout.
4. **Security groups:** target SG does not allow the ALB SG on the app port.
5. **Listener rule** with a fixed-response 503 or forwarding to an empty group; Lambda target throttled.
6. **Deployments:** all old targets draining at once.

If the app returned 503: app overloaded, dependency (DB/Redis) down, thread pool exhausted - check app logs and ALB access logs (`target_status_code`).

Example:

```
aws elbv2 describe-target-health --target-group-arn arn:aws:elasticloadbalancing:...:tg/web/abc
# {"State":"unhealthy","Reason":"Target.ResponseCodeMismatch","Description":"...[404]"}

aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB \
  --metric-name HTTPCode_ELB_5XX_Count --dimensions Name=LoadBalancer,Value=app/web/123 \
  --start-time 2026-09-24T09:00:00Z --end-time 2026-09-24T10:00:00Z \
  --period 60 --statistics Sum
```

> ***What to say to the interviewer:*** *I first check whether the 503 comes from the ALB or the targets using the ELB vs Target 5XX metrics. For ALB 503s I check that the target group has registered, healthy targets, the health check path and security groups are correct and the listener rules point to the right group; for app 503s I go to application logs and dependencies.*

Q56. An EC2 instance behind an ALB becomes unreachable. How do you troubleshoot it in an existing infrastructure, and which metrics, tools and CLI commands do you use?

Also asked as: What metrics would you monitor for this setup? Which CLI commands would you run? An EC2 instance is unreachable and it is NOT a security group issue - what else do you check?

> **In simple words:** Walk the path the request takes: reception (ALB) -> corridor doors (security groups, NACLs) -> the office (instance) -> the person at the desk (the app). Find the first place where it stops.

Detailed answer

Step-by-step:

1. **Target group health:** `describe-target-health` gives the reason code:

- `Target.Timeout`: no answer - usually security group, NACL, or the app not listening.

- `Target.FailedHealthChecks`: connects but the check fails.
- `Target.ResponseCodeMismatch`: app returns, for example, 301/404/500 on the health path.
- `Target.NotInUse` / `Target.NotRegistered`: wrong AZ or not registered.

2. **EC2 status checks:** `describe-instance-status` - system check failed = AWS hardware (stop/start); instance check failed = OS (kernel, disk, memory).
3. **Security group chain:** the ALB SG must allow outbound to the instance, and the **instance SG must allow the app port from the ALB SG**.
4. **NACLs** on both subnets: app port inbound and **ephemeral ports 1024-65535** for return traffic.
5. **Health check config:** path, port, success codes (matcher), timeout and thresholds match the app.
6. **Inside the instance** (via SSM, no SSH needed): is the service running (`systemctl`), listening on the right port and interface (`ss -tlnp`, not bound only to 127.0.0.1 when using a different port), does `curl localhost:8080/health` work, is the **disk full** (`df -h`) or was there an **OOM kill** (`dmesg`)?
7. **What changed?** Recent deploy, new AMI, config or SG change (CloudTrail), certificate expiry.
8. **ASG behaviour:** the ASG may be replacing it (health check type ELB) - check scaling activities.
9. **ALB access logs:** `target_status_code`, `-` means no response; `elb_status_code` 502/504.

If the security groups are fine, check these next:

- **Status checks:** system check failed = AWS host or network problem (stop/start moves it to healthy hardware); instance check failed = OS problem.
- **Look without logging in:** `get-console-output` (system log) and `get-console-screenshot` show a kernel panic, an OOM kill, fsck waiting at boot, or a bad `/etc/fstab` entry.
- **Network path:** NACL rules (inbound port and outbound ephemeral ports), the subnet **route table** (a deleted or changed route to the IGW/NAT/TGW), and a changed **Elastic IP or ENI** (EIP moved to another instance, secondary ENI detached, private IP changed so the target registration is stale).
- **Resource exhaustion:** a full root disk stops services and logins; memory exhaustion causes OOM kills or a hang.
- **SSM:** if the SSM agent still responds, use Session Manager or Run Command to inspect it without any SSH; if SSM is also offline, the OS is probably hung, so reboot, or detach the root volume and fix it on a rescue instance.

Metrics to monitor for this setup:

Where	Metrics
ALB / target group	HealthyHostCount, UnHealthyHostCount, HTTPCode_ELB_5XX_Count vs HTTPCode_Target_5XX_Count, TargetResponseTime (p99), RequestCount, RejectedConnectionCount

Where	Metrics
EC2	CPUUtilization, StatusCheckFailed_System / _Instance, NetworkIn/Out, CPUCreditBalance
CloudWatch Agent	mem_used_percent, disk_used_percent, process counts
Logs	App logs and ALB access logs in CloudWatch Logs / S3 (Athena)

Example:

```
aws elbv2 describe-target-health --target-group-arn <tg-arn> \
  --query "TargetHealthDescriptions[].{id:Target.Id,state:TargetHealth.State,
  reason:TargetHealth.Reason}"
aws ec2 describe-instance-status --instance-ids i-0abc --include-all-instances
aws ec2 describe-security-groups --group-ids sg-app \
  --query "SecurityGroups[0].IpPermissions"
aws elbv2 describe-target-groups --target-group-arns <tg-arn> \
  --query "TargetGroups[0].{path:HealthCheckPath,port:HealthCheckPort,codes:Matcher}"
aws ssm start-session --target i-0abc
# on the instance:
sudo systemctl status myapp
sudo ss -tlnp | grep 8080
curl -i http://localhost:8080/health
df -h ; free -m ; sudo dmesg | grep -i -E "killed process|out of memory"
```

What to say to the interviewer: *I start with describe-target-health to get the reason code, then check the EC2 status checks, the security group chain from the ALB SG to the instance port, NACLs and the health check settings. Then I get in with SSM to confirm the service is running and listening, curl the health path locally and check disk and OOM, while correlating with recent deploys; I monitor HealthyHostCount, ELB versus target 5XX, TargetResponseTime, status checks and agent memory and disk metrics.*

Q57. What is the typical latency of a load balancer, and how do you troubleshoot high latency on an ALB?

Also asked as: Explain how you would troubleshoot high latency in an ALB.

In simple words: The receptionist itself is quick (a few milliseconds). If the queue is slow, it is almost always the staff behind the desk (your targets) or the road to them.

Detailed answer

- ALB itself normally adds only **single-digit milliseconds**. High latency almost always comes from targets or dependencies.
- `TargetResponseTime` = time from ALB sending the request to the target starting to respond. Look at **p95/p99**, not only average.

Triage order:

1. **CloudWatch:** TargetResponseTime (p99), RequestCount, HTTPCode_Target_5XX, ActiveConnectionCount, RejectedConnectionCount, UnHealthyHostCount. Is it all targets or one AZ/target?
2. **ALB access logs:** split `request_processing_time`, `target_processing_time`, `response_processing_time`. High target time -> app; high request/response time -> ALB/client network.

3. **Targets:** CPU, memory, CPU credits (T instances), GC pauses, thread/connection pools, too few targets (scaling lag).
4. **Dependencies:** RDS latency/connections, Redis, external APIs; use **X-Ray/OpenTelemetry** traces to find the slow hop.
5. **Config:** cross-zone disabled with uneven targets, sticky sessions overloading one target, slow start, keep-alive mismatch (idle timeout).
6. **Client side:** far-away users -> add CloudFront.

Example:

```
# Athena query on ALB access logs: slowest targets in the last hour
SELECT target_ip, avg(target_processing_time) AS avg_s,
       approx_percentile(target_processing_time, 0.99) AS p99_s, count(*) AS reqs
FROM alb_logs
WHERE time > date_add('hour', -1, now())
GROUP BY target_ip ORDER BY p99_s DESC LIMIT 10;
```

> ***What to say to the interviewer:*** *The ALB itself adds only a few milliseconds, so I look at TargetResponseTime p99 and the access log fields to separate ALB, target and network time. Then I check target CPU, memory and pools, dependencies like RDS through traces, and configuration issues like sticky sessions or disabled cross-zone balancing.*

Q58. Users are reporting slow application performance. Which AWS services and metrics would you check first?

Also asked as: A service is slow - how do you debug performance issues in AWS?

> **In simple words:** Follow the request's journey from the user's door to the database and back, and time every stop. The slow stop is your culprit.

Detailed answer

Step-by-step, outside-in:

1. **Scope:** all users or some? All endpoints or one? When did it start? Any deployment or config change?
2. **Edge/CDN:** CloudFront cache hit ratio, origin latency.
3. **Load balancer:** ALB `TargetResponseTime` p99, 5XX, request count (traffic spike?).
4. **Compute:** EC2/ECS/EKS CPU, memory, **CPUCreditBalance** (T-series throttling is a classic), network, pod restarts, throttling from CPU limits.
5. **Storage:** EBS `VolumeQueueLength`, `BurstBalance` (gp2 burst exhaustion), IOPS limits.
6. **Database:** RDS CPU, `DatabaseConnections`, Read/WriteLatency, `DiskQueueDepth`, locks; **Performance Insights** shows top SQL and wait events.
7. **Cache/queues:** ElastiCache CPU, evictions, latency; SQS age of oldest message; MSK consumer lag.
8. **Traces:** X-Ray / OpenTelemetry to see which service or call is slow.
9. **Logs:** CloudWatch Logs Insights for slow requests and errors.
10. **External:** third-party API latency, AWS Health Dashboard.

Example:

```
Deploy at 14:05 -> p99 latency 180 ms -> 2.4 s
Performance Insights: top wait = IO:DataFileRead, query "SELECT ... FROM orders WHERE email=?"
Root cause: new release dropped an index in migration. Fix: recreate index, p99 back to 190 ms.
```

> ***What to say to the interviewer:*** *I scope the problem first, then go outside-in: CloudFront, ALB TargetResponseTime, compute CPU, memory and credit balance, EBS burst balance, then RDS with Performance Insights and caches, using X-Ray traces to pinpoint the slow hop. I always correlate the start time with recent deployments or config changes.*

Q59. Two users access the same application: one sees low latency and the other sees high latency. How do you troubleshoot?

> **In simple words:** Same shop, two customers, different waiting times. Maybe one lives far away, came by a jammed road, got sent to a slow counter, or ordered something much bigger.

Detailed answer

Compare the two users on each layer:

1. **Geography and edge:** where are they? Which **CloudFront edge** served them (`x-amz-cf-pop` header)? Cache hit (`x-cache: Hit from cloudfront`) for one and miss for the other?
2. **DNS routing:** Route 53 latency/geolocation rules may send them to different regions; check what `dig` returns for each.
3. **Network path / ISP:** `traceroute`/`mtr` from the slow user; packet loss, VPN or corporate proxy, Wi-Fi.
4. **Which target/AZ:** ALB access logs by client IP show `target_ip` and `target_processing_time`. **Sticky sessions** may pin the slow user to an overloaded or unhealthy target.
5. **User-specific data:** bigger account, more records, heavier queries, different feature flags or permissions (for example an admin dashboard loading everything).
6. **Client device/browser:** old device, extensions, browser dev tools timing (DNS, TLS, TTFB, download).
7. **Traces:** filter X-Ray/OTel by user ID to see the slow span.

Example:

```
curl -s -o /dev/null \
  -w "dns=%{time_namelookup} tls=%{time_appconnect} ttfb=%{time_starttransfer}\n" \
  https://app.example.com/api/dashboard
curl -sI https://app.example.com/ | grep -i -E "x-cache|x-amz-cf-pop"
mtr -rwc 50 app.example.com
```

> ***What to say to the interviewer:*** *I compare the two users layer by layer: their location and CloudFront edge or cache status, where Route 53 sends them, and their network path with traceroute or mtr. Then I use ALB access logs and traces filtered by their client IP or user ID to see whether sticky sessions pinned them to a slow target or whether their data makes queries heavier.*

Q60. An Auto Scaling Group is not launching new instances during traffic spikes. How would you investigate?

Also asked as: Explain a real scenario where Auto Scaling did not scale properly. What was the root cause?

In simple words: The manager wants to hire but nobody shows up. Either the manager never got the message (alarm), the hiring budget is maxed out (max size/quotas), or the job ad is broken (launch template).

Detailed answer

Check in this order:

1. **Scaling activity history:** `describe-scaling-activities` shows the exact failure message.
2. **Capacity limits:** desired already equals **MaxSize**.
3. **Alarm/policy:** did the CloudWatch alarm fire? Wrong metric, too long period, or scaling on CPU when the bottleneck is memory or requests. Cooldown/instance warmup delaying action.
4. **Suspended processes:** `Launch` or `AlarmNotification` suspended (someone forgot to resume after maintenance).
5. **Launch template errors:** deleted AMI, missing instance profile/KMS permission, invalid SG/subnet, wrong key pair.
6. **Capacity and quotas:** `InsufficientInstanceCapacity` in the AZ, **EC2 vCPU service quota** reached, Spot capacity gone, subnet out of IP addresses.
7. **Health checks:** new instances launch but fail ELB health check and get terminated (launch-terminate loop).

Real example: during a sale, the ASG hit its vCPU quota for the instance family; activities showed `VcpuLimitExceeded`. Fix: raised the quota, added multiple instance types across 3 AZs in a mixed instances policy, and moved to target tracking on `ALBRequestCountPerTarget` with scheduled/predictive scaling before known peaks.

Example:

```
aws autoscaling describe-scaling-activities --auto-scaling-group-name web-asg --max-items 5
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names web-asg \
  --query "AutoScalingGroups[0].{Min:MinSize,Max:MaxSize,Desired:DesiredCapacity,
  Suspended:SuspendedProcesses}"
aws service-quotas get-service-quota --service-code ec2 --quota-code L-1216C47A
```

What to say to the interviewer: *I start with the ASG scaling activity history, which usually states the reason, then check max size, whether the alarm and policy actually fired, suspended processes, launch template errors, AZ capacity, vCPU quotas and subnet IPs. In one case we hit the vCPU quota, so we raised it, diversified instance types and added predictive scaling on requests per target.*

Q61. Your EC2 instances scale up but never scale down. How would you debug and fix it?

Also asked as: An Auto Scaling Group scales out but not in.

In simple words: Extra staff were hired for the rush but nobody was sent home afterwards. Either the "it is quiet now" signal never comes, or someone blocked the exit door.

Detailed answer

Common causes:

1. **Metric never drops:** the scale-in alarm threshold is too low, or background jobs keep CPU up. Memory-based scaling rarely drops because apps keep memory allocated.
2. **Target tracking scale-in is conservative** (needs a sustained low period) or `DisableScaleIn` is true.
3. **Only a scale-out policy exists** (step scaling without a scale-in policy).
4. **MinSize or desired manually raised** (MinSize = current count).
5. **Scale-in protection** on instances, or the `Terminate` process suspended.
6. **Lifecycle hooks** stuck in `Terminating:Wait` (hook never completes).
7. **Conflicting policies** (a scheduled action resets desired capacity; multiple policies where scale-out keeps winning).
8. On ECS/EKS: the node cannot be drained (PodDisruptionBudgets, pods with local storage, `do-not-disrupt` annotations).

Fix: correct metric and thresholds (requests per target is better than CPU for web), ensure a scale-in path exists, remove unneeded protection, complete lifecycle hooks with timeouts, and alarm on "desired > baseline for 6 hours".

Example:

```
aws autoscaling describe-policies --auto-scaling-group-name web-asg
aws autoscaling describe-auto-scaling-instances \
  --query "AutoScalingInstances[].{ID:InstanceId,Protected:ProtectedFromScaleIn,
  State:LifecycleState}"
aws autoscaling set-instance-protection --auto-scaling-group-name web-asg \
  --instance-ids i-0abc --no-protected-from-scale-in
```

> ***What to say to the interviewer:*** *I check whether the scale-in metric ever crosses its threshold, whether a scale-in policy even exists or target tracking has scale-in disabled, and whether MinSize, scale-in protection, suspended Terminate processes or stuck lifecycle hooks are blocking it. Then I fix the metric choice and thresholds and add an alarm if capacity stays above baseline for hours.*

Q62. An RDS database becomes unavailable during business hours. What is your recovery approach?

> **In simple words:** The shop's cash counter stops. First, open a backup counter fast (restore service), keep customers informed, and only after the rush find out why it broke.

Detailed answer

1. Triage (first 5 minutes):

- `describe-db-instances` status: `storage-full`, `rebooting`, `failing-over`, `maintenance`, `incompatible-parameters`, `inaccessible-encryption-credentials`?
- **RDS Events** and the **AWS Health Dashboard** (AZ issue?).
- CloudWatch: FreeStorageSpace, CPU, DatabaseConnections (max connections reached looks like "down" to apps), FreeableMemory.

- Is it the DB or the network (SG change, DNS, secret rotated)?

2. Restore service:

- **Multi-AZ:** failover is automatic; if hung, `reboot --force-failover`.
- **storage-full:** increase allocated storage (enable storage autoscaling).
- **Connections exhausted:** kill idle sessions, scale app pools down, add RDS Proxy.
- **Single-AZ and broken:** promote a read replica or **point-in-time restore** to a new instance, then repoint the app (DNS CNAME / secret).

3. Communicate status every 15-30 minutes; put the app in maintenance mode if needed.

4. After: RCA, then prevent: Multi-AZ, storage autoscaling, alarms on storage/connections, maintenance windows outside business hours, tested restore runbook.

Example:

```
aws rds describe-events --source-identifier orders-prod --source-type db-instance \
  --duration 120
aws rds modify-db-instance --db-instance-identifier orders-prod \
  --allocated-storage 500 --max-allocated-storage 1000 --apply-immediately
aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier orders-prod \
  --target-db-instance-identifier orders-restored --restore-time 2026-09-24T09:40:00Z
```

> ***What to say to the interviewer:*** *I check the instance status, RDS events and key metrics like free storage and connections to find why it is down, then restore service fastest: failover for Multi-AZ, add storage if full, or promote a replica or do a point-in-time restore and repoint the app. I keep stakeholders updated and follow with an RCA and preventive alarms, Multi-AZ and storage autoscaling.*

Q63. An RDS instance is hitting max connections during peak traffic. How would you handle it?

Also asked as: RDS connections suddenly spike - how do you monitor and protect the database without downtime?

> **In simple words:** The restaurant has 100 chairs and every waiter is holding chairs for guests who left. Share chairs better (pooling) before building a bigger restaurant.

Detailed answer

Monitor: CloudWatch `DatabaseConnections` (alarm at 80 percent of `max_connections`), **Performance Insights** (sessions by wait event, top SQL, top hosts/users), and the app's pool metrics. A sudden jump right after a deploy usually means a connection leak or a bigger pool setting.

Immediate (no downtime):

1. Find who holds connections: `pg_stat_activity` / `SHOW PROCESSLIST` grouped by user, application and client address to find the **leaking client**; kill long-idle sessions.
2. Reduce per-pod/instance **pool sizes**; many small pods each with a pool of 50 add up fast (20 pods x 50 = 1,000).

Short term:

- **RDS Proxy** pools and multiplexes connections, and helps greatly with Lambda.
- **pgbouncer** in transaction mode for PostgreSQL.

- Route reads to **read replicas** (reader endpoint).
- Add caching (ElastiCache) for hot reads.

Longer term:

- `max_connections` is derived from instance memory; raising it or moving to a bigger class helps but costs memory per connection.
- Fix connection leaks, set idle timeouts, cap autoscaling so pods x pool <= DB limit.
- Alarm at 80 percent of `DatabaseConnections`.

Example:

```
SELECT usename, application_name, client_addr, state, count(*)
FROM pg_stat_activity GROUP BY 1,2,3,4 ORDER BY 5 DESC;

SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE state = 'idle' AND state_change < now() - interval '10 minutes';
```

> ***What to say to the interviewer:*** *I watch DatabaseConnections and Performance Insights, and when it spikes I use pg_stat_activity grouped by application and client to find the leaking client, then trim idle sessions and oversized pools, then put RDS Proxy or pgbouncer in front to multiplex connections and move reads to replicas and cache. Long term I size pools so pods times pool size stays under the limit, fix leaks, and alarm at 80 percent of max connections.*

Q64. Your ECS service is stuck in PROVISIONING. What steps would you take?

> **In simple words:** The dish is ordered but there is no free stove to cook it on. ECS is waiting for capacity or a network slot before the task can start.

Detailed answer

`PROVISIONING` means ECS is waiting for resources before the task can run: capacity to be added, or the network interface to be attached.

Checks:

1. **Service events:** `describe-services` -> `events` often says "unable to place a task" and why.
2. **EC2 capacity provider:** managed scaling must scale the ASG; check ASG **max size**, scaling activities, and `CapacityProviderReservation` metric. The ASG may be at max or failing to launch.
3. **Resources:** no instance has enough free CPU/memory/ports for the task size; placement constraints too strict.
4. **awsvpc networking:** subnets out of **free IP addresses**; ENI limit per instance reached (enable ENI trunking).
5. **Fargate:** capacity shortage in the AZ (spread across subnets in several AZs), Fargate Spot unavailable, Fargate vCPU quota.
6. **Task launch errors** after provisioning: image pull (ECR permissions, no NAT/endpoints), secrets fetch failure -> check stopped task `stoppedReason`.

Example:

```
aws ecs describe-services --cluster prod --services orders \
  --query "services[0].events[:5].message"
aws ecs describe-tasks --cluster prod --tasks <task-id> \
  --query "tasks[0].{status:lastStatus,reason:stoppedReason}"
aws ec2 describe-subnets --subnet-ids subnet-a subnet-b \
  --query "Subnets[].{id:SubnetId,free:AvailableIpAddressCount}"
```

What to say to the interviewer: *PROVISIONING means ECS is waiting for capacity or networking, so I read the service events first, then check the capacity provider's ASG max size and scaling activity, free CPU and memory, subnet IPs and ENI limits for awsvpc. For Fargate I check AZ capacity and quotas, and if tasks then stop I look at stoppedReason for image pull or secrets issues.*

Q65. Describe an incident where an EC2 instance crashed or became unresponsive. What did you do?

In simple words: When a machine stops answering, first find out whether it is the building (AWS host) or the machine itself (your OS). Get it working again fast, then do the post-mortem.

Detailed answer

Structured approach:

1. **Status checks:** `StatusCheckFailed_System` = AWS host problem -> stop/start moves it to new hardware. `StatusCheckFailed_Instance` = OS problem (kernel panic, OOM, disk full, bad fstab).
2. **Look without logging in:** `get-console-output` and **instance screenshot** for kernel panic/OOM messages.
3. **Metrics before the crash:** CPU, memory and disk from the CloudWatch agent, CPU credits.
4. **Recover:** reboot; if it will not boot, stop, detach the root volume, attach to a rescue instance, fix (fstab, full disk), reattach. Or restore from the latest snapshot/AMI.
5. **Prevent:** EC2 auto-recovery alarm, run behind an ASG so it self-heals, disk/memory alarms, log rotation.

Sample real story (adapt to yours): "Our Zabbix monitoring server's web UI started returning 504. The EC2 was hung at OS level, so SSM and SSH did not respond. I checked the console output, rebooted, verified the web server and the `mysqld` service came back, and confirmed monitoring resumed. RCA found memory and disk pressure on a small instance, so we tuned PHP-FPM workers, vacuumed the systemd journal from 91 percent to 82 percent disk, and added disk and memory alarms."

Example:

```
aws ec2 describe-instance-status --instance-ids i-0abc --include-all-instances
aws ec2 get-console-output --instance-id i-0abc --latest --output text | tail -50
aws ec2 get-console-screenshot --instance-id i-0abc --query ImageData --output text \
  | base64 -d > screen.jpg
aws cloudwatch put-metric-alarm --alarm-name ec2-autorecover-i-0abc \
  --namespace AWS/EC2 --metric-name StatusCheckFailed_System --statistic Maximum \
  --dimensions Name=InstanceId,Value=i-0abc --period 60 --evaluation-periods 2 \
  --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions arn:aws:automate:ap-south-1:ec2:recover
```

What to say to the interviewer: *I check system versus instance status checks to know if it is AWS hardware or our OS, read the console output and screenshot, then recover by stop/start, reboot or fixing the root volume on a rescue instance. In our case a monitoring server hung from memory and disk pressure; I rebooted it, verified the services, then tuned PHP-FPM, cleaned the journal and added alarms.*

Q66. An S3 bucket used for artifact storage was accidentally made public. What is your fix?

Also asked as: An S3 bucket was made public by mistake - how do you secure and audit it?

In simple words: The warehouse door was left open. Shut it immediately, check the CCTV for who came in, then fit a lock that cannot be left open again.

Detailed answer

Contain (minutes):

1. Turn on **Block Public Access** on the bucket (and account).
2. Remove the public bucket policy statement / public ACLs; set Object Ownership to BucketOwnerEnforced.

Assess:

3. **CloudTrail:** who changed the policy (`PutBucketPolicy`, `PutBucketAcl`, `PutPublicAccessBlock`) and when.
4. **S3 server access logs / CloudTrail data events:** which objects were downloaded by unknown IPs during the exposure window.
5. Check whether artifacts contained **secrets** (config files, keys). If yes, **rotate them all** and treat as a security incident.
6. Verify artifact integrity (checksums/signatures) in case someone **wrote** to it.
7. **Amazon Macie** scan of the bucket to find sensitive data (PII, credentials) that may have been exposed.
8. **IAM Access Analyzer for S3** to list any other buckets shared publicly or with outside accounts.

Prevent:

- **Account-level Block Public Access**, protected by an **SCP** that denies turning it off; AWS Config rule `s3-bucket-public-read-prohibited` with auto-remediation; IAM Access Analyzer alerts; bucket managed in Terraform so drift is visible; artifacts served via pre-signed URLs or VPC endpoint only.

Example:

```
aws s3api put-public-access-block --bucket build-artifacts \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api delete-bucket-policy --bucket build-artifacts
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=PutBucketPolicy --max-results 5
```

What to say to the interviewer: *I immediately enable Block Public Access and remove the public policy, then use CloudTrail to find who changed it and access logs to see what was downloaded,*

rotating any secrets that were in the artifacts. To prevent a repeat I enforce Block Public Access with an SCP, add a Config rule with auto-remediation and manage the bucket in Terraform.

Q67. NAT Gateway costs doubled in 24 hours during a live event, with no infrastructure changes. What could be causing it?

In simple words: The toll bridge bill doubled although you built no new roads. Something started driving over the bridge a lot more: find which car is making all the trips.

Detailed answer

NAT charges per GB processed, so something is sending much more data **through** it.

Likely silent causes:

- **Image pulls:** pods CrashLooping or nodes churning (Spot, autoscaling for the event) pulling large images from ECR/Docker Hub via NAT again and again.
- **S3/DynamoDB traffic via NAT** because there is no gateway endpoint (event traffic = more S3 reads/writes).
- **Logs and metrics shipping** to CloudWatch or a SaaS (debug logging enabled, traffic spike = more log volume).
- **Retry storms** to an external API or a dependency that is timing out.
- **Cross-AZ:** instances using a NAT in another AZ (extra inter-AZ charges).
- **Scaling itself:** more pods/nodes = more outbound calls (third-party CDN/DRM/analytics calls per viewer).
- Security: compromised instance exfiltrating data or mining crypto.

How to find it:

1. Cost Explorer: usage type `NatGateway-Bytes` by day/hour.
2. CloudWatch NAT metrics `BytesOutToDestination`, `BytesInFromDestination` per NAT.
3. **VPC Flow Logs** on the NAT ENI + Athena: top source IPs and destinations.

Fix: S3/DynamoDB gateway endpoints, ECR/CloudWatch interface endpoints, pull-through cache, fix crashloops, reduce log verbosity, NAT per AZ.

Example:

```
SELECT srcaddr, dstaddr, sum(bytes)/1024/1024/1024 AS gb
FROM vpc_flow_logs
WHERE interface_id = 'eni-0natgw' AND start > to_unixtime(now() - interval '24' hour)
GROUP BY srcaddr, dstaddr ORDER BY gb DESC LIMIT 20;
```

What to say to the interviewer: *NAT bills per GB, so I look for what started pushing more data through it, such as repeated image pulls from crashlooping pods or node churn, S3 traffic without a gateway endpoint, log shipping or retry storms. I confirm with NAT CloudWatch metrics and VPC Flow Logs in Athena, then fix with VPC endpoints, fixing the crashloops and reducing log volume.*

Q68. Your team sees a spike in cloud costs. How do you find out why and cut costs without hurting performance?

Also asked as: You suddenly see a 5x spike in cloud billing and the infra looks fine. Where do you look first, and how do you trace the bleed? Your monthly AWS bill increased unexpectedly - what tools and reports do you use? / Reduce cloud cost by 40% without impacting performance. Where do you start?

Tools and reports at a glance: Billing console Bills page (charges per service and region), Cost Explorer (filters, grouping, rightsizing recommendations), Cost Anomaly Detection, AWS Budgets, Cost and Usage Report (CUR) queried with Athena, Trusted Advisor cost checks, Compute Optimizer, and CloudTrail for who created what.

> **In simple words:** Your electricity bill jumped 5x. Check the meter room by room (service), then appliance by appliance (usage type, resource), and ask who plugged in something new. Then switch off the waste, not the fridge.

Detailed answer

Trace the bleed (top-down):

1. **Cost Explorer** with **hourly/daily granularity**: group by **service -> usage type -> resource**, and by linked account, region and tag. Find when it started.
2. **Cost Anomaly Detection** alerts usually point straight at the service and account.
3. **CUR (Cost and Usage Report) + Athena** for resource-level lines when Cost Explorer is not detailed enough.
4. **CloudTrail:** who created what (`RunInstances`, `CreateNatGateway`), especially in **regions you never use**. Compromised access keys launching **GPU crypto-mining instances** is a classic 5x cause: check all regions, disable the keys, terminate, open an AWS support case.

Usual silent culprits:

- **Data transfer:** NAT processing, cross-AZ traffic, internet egress.
- **Log ingestion:** CloudWatch `PutLogEvents` from someone leaving **debug logging** on.
- **S3 request costs:** a job doing millions of LIST/GET calls, or lifecycle transitions of tiny objects.
- **Lambda recursion loops** (Lambda writes to S3 which triggers the same Lambda); AWS now detects some loops, but check invocations.
- Forgotten large instances, unattached EBS, snapshots, idle load balancers, provisioned IOPS.

Cut without hurting performance: rightsizing from real p95 usage (Compute Optimizer), Savings Plans for baseline, Spot for stateless, Graviton, VPC endpoints, log levels and retention, S3 lifecycle/Intelligent-Tiering, schedule non-prod. Change one thing at a time and watch latency/error SLOs.

Prevent: AWS Budgets with alerts at 80/100 percent, anomaly detection subscriptions, tagging policy, SCPs denying unused regions.

Example:

```
aws ce get-cost-and-usage --time-period Start=2026-09-20,End=2026-09-24 \
  --granularity DAILY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE Type=DIMENSION,Key=USAGE_TYPE

for r in $(aws ec2 describe-regions --query "Regions[].RegionName" --output text); do
  echo "$r: $(aws ec2 describe-instances --region $r \
    --query 'length(Reservations[].Instances[])')"
done
```

A 40% reduction target, in order of impact:

1. **Visibility first:** cost allocation tags, Cost Explorer by service, account and tag, and the top 10 line items. Usually 3-4 items are most of the bill.
2. **Remove waste:** idle or unattached EBS volumes, old snapshots, unused Elastic IPs, idle load balancers, forgotten dev environments. Schedule non-prod to stop at night and on weekends (about 65% off those hours).
3. **Rightsize:** Compute Optimizer and CloudWatch data, over-provisioned instances and RDS, K8s requests vs actual usage, and move to Graviton where possible.
4. **Pricing models:** Savings Plans or RIs for the steady baseline, Spot for stateless, batch and CI workloads, and Karpenter consolidation.
5. **Data transfer and NAT:** VPC endpoints for S3 and ECR, keep traffic in the same AZ, and use CloudFront.
6. **Storage tiering:** S3 lifecycle to IA or Glacier, gp2 to gp3, and log retention limits.
7. **Guard performance:** agree SLOs (p95 latency, error rate) first, change in small steps, compare before and after, and keep budgets and anomaly alerts so costs don't creep back.

What to say to the interviewer: *I break the bill down in Cost Explorer by service, usage type and resource at hourly granularity, check Cost Anomaly Detection and CUR in Athena, and use CloudTrail to see who created what, including in unused regions where compromised keys may be mining crypto. Common silent culprits are NAT and data transfer, debug log ingestion, S3 requests and Lambda loops; I cut waste with rightsizing, Savings Plans, Spot and endpoints while watching SLOs, and prevent repeats with budgets and anomaly alerts.*

Q69. How would you handle a sudden traffic spike from 10K to 200K users?

In simple words: Twenty times more guests are coming. Serve snacks at the door (cache), open more counters (autoscale), queue orders instead of dropping them, and turn away only the extras politely if you must.

Detailed answer

If you know in advance (event, sale):

1. **Load test** at 1.5-2x the expected peak.
2. **Pre-scale:** scheduled scaling / raise min capacity, warm pools; pre-provision Lambda concurrency, DynamoDB capacity; check **service quotas** early (vCPU, Lambda concurrency, API Gateway).
3. For very sudden huge jumps, ALB scales automatically but you can use **LCU capacity reservation**; NLB and CloudFront handle bursts well.

Architecture for the spike:

- **CloudFront** caching for static and cacheable API responses (biggest win).
- **Stateless app tier** autoscaling on requests per target; EKS with Karpenter for fast nodes.
- **Database protection:** read replicas, ElastiCache, RDS Proxy; Aurora auto-scaling replicas.
- **Queues (SQS/Kafka)** to absorb writes asynchronously.
- **Rate limiting** (WAF, API Gateway) and **graceful degradation**: turn off non-critical features with feature flags.
- Dashboards and alarms on latency/errors; a war room during the event.

Example:

```
aws autoscaling put-scheduled-update-group-action --auto-scaling-group-name web-asg \
  --scheduled-action-name sale-prewarm --start-time 2026-10-02T12:30:00Z \
  --min-size 40 --max-size 200 --desired-capacity 60

User -> CloudFront (80% cache hit) -> WAF rate limit -> ALB -> ASG/EKS (autoscale)
                                                          -> ElastiCache -> RDS + replicas
                                                          -> SQS -> workers (async writes)
```

> ***What to say to the interviewer:*** *For a planned spike I load test, check quotas and pre-scale with scheduled actions, then rely on CloudFront caching, a stateless autoscaling tier, caches and read replicas to protect the database, and queues for writes. I add rate limiting and feature flags for graceful degradation and watch latency and error dashboards throughout.*

Q70. How do you design a fault-tolerant system that keeps running when one Availability Zone is down?

> **In simple words:** Keep three kitchens in three buildings, each able to handle the load if one burns down, and do not keep the only recipe book in one of them.

Detailed answer

- **Spread everything across 3 AZs:** ALB subnets, ASG/EKS nodes, ECS tasks, MSK brokers, ElastiCache replicas.
- **Static stability:** run enough capacity so 2 AZs can carry 100 percent of load without needing to launch anything during the failure (for example 3 AZs x 50 percent). Scaling during an AZ event may be slow or impossible.
- **Data:** RDS/Aurora Multi-AZ, S3 (regional, multi-AZ by default), DynamoDB (multi-AZ by default), EFS regional.
- **NAT Gateway per AZ** with per-AZ route tables.
- **Health checks** remove bad targets; ALB cross-zone load balancing on.
- **Kubernetes:** topology spread constraints across zones, PodDisruptionBudgets, zone-aware storage (EBS is zonal: use StatefulSets with a replica per AZ).
- **Avoid single-AZ dependencies:** one EC2 cron server, a single Redis node, a self-managed DB in one AZ.
- **Test:** use **ARC zonal shift** or FIS to simulate an AZ failure.

Example:

```
            ALB (AZ-a, AZ-b, AZ-c)
        /            |            \
  AZ-a: 3 nodes  AZ-b: 3 nodes  AZ-c: 3 nodes   (2 AZs alone = 100% load)
  NAT-a          NAT-b          NAT-c
  Aurora writer  Aurora reader  Aurora reader   (auto failover)

aws arc-zonal-shift start-zonal-shift --resource-identifier <alb-arn> \
  --away-from aps1-az1 --expires-in 1h --comment "AZ failure drill"
```

What to say to the interviewer: *I deploy every tier across three AZs with per-AZ NAT, Multi-AZ databases and topology spread for pods, and I keep enough capacity that two AZs can carry full load without scaling during the failure, which is static stability. I remove single-AZ dependencies and regularly test with ARC zonal shift or Fault Injection Service.*

Q71. You are asked to migrate workloads from on-premises to AWS with zero downtime. How would you approach it?

In simple words: Move house without ever leaving the family homeless: build the new house, keep both homes synced, move people one room at a time, and keep the old house ready in case the new one leaks.

Detailed answer

1. **Discover and plan:** inventory apps and dependencies (Migration Hub / Application Discovery Service), pick a strategy per app (rehost, replatform, refactor), define success criteria and rollback.
2. **Connectivity:** Site-to-Site VPN or Direct Connect, hybrid DNS with Route 53 Resolver.
3. **Build the landing zone** with IaC: accounts, VPCs, IAM, monitoring.
4. **Replicate continuously:**

- Servers: **AWS Application Migration Service (MGN)** block-level continuous replication.
- Databases: **DMS full load + CDC** (or native replication) so AWS stays in sync.
- Files: DataSync.

5. **Test** the AWS copy with non-disruptive test launches and load tests.
6. **Cutover gradually:** lower DNS TTL days before, shift traffic with Route 53 **weighted records** (1 -> 10 -> 50 -> 100 percent), keep the database single-writer at any moment.
7. **Rollback plan:** keep on-prem running and reverse replication until stable.
8. **Decommission** after a hypercare period.

Example:

```
Day -7 : Route 53 TTL 3600 -> 60
Day  0 : MGN + DMS CDC in sync, test instances pass
Day  1 : weight on-prem 90 / AWS 10  (read-only traffic first)
Day  3 : DB cutover in low traffic window (stop writes 1-2 min, CDC catch-up, switch)
Day  4 : weight 0 / 100, reverse replication to on-prem kept for rollback
Day 30 : decommission on-prem
```

What to say to the interviewer: *I set up hybrid connectivity and the AWS landing zone with IaC, then replicate continuously with Application Migration Service for servers and DMS CDC for databases. After testing, I shift traffic gradually using low-TTL weighted Route 53 records, keep the*

database single-writer during a short cutover, and keep on-prem running with reverse replication for rollback.

Q72. What strategies do you use for database migration with zero downtime?

Also asked as: How do you handle zero-downtime database migrations in a distributed application? How do you migrate an RDS database with minimal downtime?

In simple words: Build a new road next to the old one, let cars use both for a while, then close the old road only when everyone is on the new one.

Detailed answer

Moving to a new database/engine/version:

- **DMS full load + CDC:** copy existing data, then stream ongoing changes until cutover; validate with DMS data validation.
- **RDS Blue/Green Deployments:** RDS creates a synced green copy (for upgrades or parameter changes); switchover typically takes under a minute and keeps endpoint names.
- **Native replication** (logical replication for PostgreSQL, binlog replication for MySQL).
- Cutover: stop writes briefly, confirm lag = 0, switch the endpoint (RDS Proxy / DNS / secret), keep the old DB for rollback.
- A common wrong answer is "enable Multi-AZ". Multi-AZ gives high availability (a standby for failover) inside the same database; it does not move data to a new instance, engine, version or account, so it is not a migration method.

Schema changes in a live app (expand-contract):

1. **Expand:** add new column/table (nullable, no locks); deploy code writing to both old and new.
2. **Backfill** data in small batches.
3. Switch reads to the new structure.
4. **Contract:** remove the old column in a later release.

- Never rename or drop in the same release; use online tools (`gh-ost`, `pg_repack`, `CREATE INDEX CONCURRENTLY`).
- Migrations run in CI/CD as a separate, versioned step (Flyway/Liquibase) and must be backward-compatible.

Example:

```
aws rds create-blue-green-deployment --blue-green-deployment-name orders-pg16 \
  --source arn:aws:rds:ap-south-1:111111111111:db:orders-prod \
  --target-engine-version 16.4
aws rds switchover-blue-green-deployment \
  --blue-green-deployment-identifier bgd-abc123 --switchover-timeout 300

-- expand step (PostgreSQL)
ALTER TABLE users ADD COLUMN email_verified boolean;
CREATE INDEX CONCURRENTLY idx_users_email ON users(email);
```

What to say to the interviewer: *For moving databases I use DMS full load plus CDC or native replication, or RDS Blue/Green Deployments for upgrades, cutting over when lag is zero and keeping the old database for rollback. For schema changes I follow expand-contract with backward-compatible, versioned migrations so old and new app versions work at the same time.*

Q73. You must ship multi-region failover for live events in 2 weeks, and DNS-based routing is not allowed. What is your plan?

In simple words: Give users one permanent phone number that rings in two cities. If one office stops answering, the phone system rings the other one automatically, without printing a new phone book.

Detailed answer

Traffic layer (no DNS changes):

- **AWS Global Accelerator:** two static anycast IPs; endpoint groups in two regions (ALB/NLB) with health checks; failover in seconds; traffic dials to shift load.
- For HTTP content, **CloudFront origin groups** give origin failover (for GET/HEAD/OPTIONS requests).

2-week plan:

1. **Days 1-2:** agree RTO/RPO, scope only the critical live path (playback/API), pick the secondary region.
2. **Days 3-6:** deploy the same IaC stack in region 2 (warm standby, pre-scaled for the event); replicate images (ECR replication), secrets and config.
3. **Days 5-8:** data: Aurora Global Database / DynamoDB Global Tables / S3 CRR; for live streams, ingest to both regions.
4. **Days 7-9:** Global Accelerator with health checks, traffic dials 100/0.
5. **Days 10-12:** failover drills, load tests in region 2, runbook with clear decision owner.
6. **Days 13-14:** freeze, dashboards, war room.

Example:

```
Clients -> Global Accelerator (static IPs 75.2.x.x, 99.83.x.x)
             |-- endpoint group ap-south-1     dial 100%  (ALB, health check /healthz)
             +-- endpoint group ap-southeast-1 dial 0%    (warm, pre-scaled)

aws globalaccelerator update-endpoint-group --endpoint-group-arn <arn-primary> \
  --traffic-dial-percentage 0      # manual failover drill
```

What to say to the interviewer: *I would use Global Accelerator, whose static anycast IPs route to healthy regional endpoints and fail over in seconds without any DNS change, with CloudFront origin groups for cacheable content. In two weeks I would scope only the critical path, deploy a warm, pre-scaled copy via IaC in the second region, replicate data with global databases, and spend the final days on failover drills and a clear runbook.*

Q74. Application latency suddenly increased. The app runs in a private subnet behind a NAT Gateway. How do you investigate?

In simple words: If every trip out of the building goes through one exit gate, a jam at that gate slows everyone. First check whether the slowness is really on trips going out, then check the gate.

Detailed answer

1. **Is it outbound?** Traces (X-Ray/OTel) or app timing: is the slow span a call to an external API, AWS API or S3 through the NAT, or is it internal (DB, CPU)? If internal, NAT is not the issue.
2. **NAT Gateway CloudWatch metrics:**

- `ErrorPortAllocation` > 0: NAT ran out of source ports. A NAT supports about **55,000 simultaneous connections to each unique destination** (IP + port + protocol); fix by adding secondary private IPs to the NAT, spreading destinations, or reusing connections.
- `PacketsDropCount`: drops at the NAT.
- `ActiveConnectionCount`, `ConnectionAttemptCount` vs `ConnectionEstablishedCount`, `IdleTimeoutCount`.
- `BytesOutToDestination`: a sudden traffic jump (bandwidth scales up to 100 Gbps but bursts can queue).

3. **Cross-AZ NAT:** instances in AZ-b using a NAT in AZ-a add latency and cost; use one NAT per AZ with per-AZ route tables.
4. **Skip the NAT for AWS services:** S3/DynamoDB gateway endpoints, interface endpoints for STS, Secrets Manager, ECR, CloudWatch.
5. **Connection behaviour:** no keep-alive, a new TLS handshake per request, connection pool too small, idle connections dropped after 350 seconds of NAT idle timeout causing retries.
6. **The destination itself:** third-party API slow or rate limiting (429s), their region/latency, test from another network.
7. **DNS:** slow or failing lookups (Route 53 Resolver limits, missing caching).

Example:

```
aws cloudwatch get-metric-statistics --namespace AWS/NATGateway \
  --metric-name ErrorPortAllocation --dimensions Name=NatGatewayId,Value=nat-0abc \
  --start-time 2026-09-24T08:00:00Z --end-time 2026-09-24T10:00:00Z \
  --period 60 --statistics Sum

aws ec2 assign-private-nat-gateway-address --nat-gateway-id nat-0abc \
  --private-ip-address-count 2          # more IPs = more ports per destination
```

> ***What to say to the interviewer:*** *I first confirm with traces that the slow part is outbound traffic through the NAT, then check NAT metrics like ErrorPortAllocation for port exhaustion, PacketsDropCount and connection counts, plus cross-AZ NAT usage. Fixes include VPC endpoints so AWS traffic skips the NAT, connection reuse and keep-alive, extra NAT IPs, one NAT per AZ, and checking whether the third-party endpoint itself is slow.*

Q75. How would you expose a backend API securely through CloudFront and WAF, allowing only a certain IP range?

> **In simple words:** Put a guarded gate (CloudFront + WAF) in front with a guest list (IP set), and wall off the back door (origin) so the only way in is through that gate.

Detailed answer

Edge (CloudFront + WAF):

1. Create an **IP set** with the allowed CIDRs.
2. **Web ACL** (scope CLOUDFRONT, created in us-east-1) with **default action Block** and a rule **Allow if source IP in IP set**.
3. Add AWS **managed rule groups** (Core rule set, Known bad inputs, SQLi) and a **rate-based rule**, evaluated before the allow rule so allowed IPs still get inspected.
4. Attach the web ACL to the distribution; HTTPS only with an ACM certificate.

Lock the origin so nobody bypasses CloudFront:

- Best: **CloudFront VPC origins** pointing to an **internal ALB** in private subnets (no public exposure at all).
- Or public ALB with its security group allowing only the **CloudFront managed prefix list** (`com.amazonaws.global.cloudfront.origin-facing`), plus a **secret custom header** added by CloudFront and checked by an ALB listener rule (others get 403). Rotate the header value via Secrets Manager.
- For S3 origins use **OAC**.

Also: TLS end to end (CloudFront -> ALB on HTTPS), disable caching for the API or cache only safe GETs, forward only needed headers, and enable **WAF logs, CloudFront logs and ALB access logs**.

Example:

```
aws wafv2 create-ip-set --name partner-ips --scope CLOUDFRONT --region us-east-1 \
  --ip-address-version IPV4 --addresses 203.0.113.0/24 198.51.100.10/32

# ALB listener rule: forward only if the secret header matches, else fixed 403
aws elbv2 create-rule --listener-arn <listener-arn> --priority 1 \
  --conditions '[{"Field":"http-header","HttpHeaderConfig":
     {"HttpHeaderName":"X-Origin-Verify","Values":["<secret-value>"]}}]' \
  --actions Type=forward,TargetGroupArn=<tg-arn>
```

> ***What to say to the interviewer:*** *I attach a WAF web ACL to CloudFront with an IP set allow rule, a default block action, managed rule groups and rate limiting. Then I lock the origin using CloudFront VPC origins to a private ALB, or the CloudFront managed prefix list in the ALB security group plus a secret header checked by a listener rule, with TLS end to end and WAF, CloudFront and ALB logging.*

Q76. CloudWatch alerts are firing for Lambda timeout errors. How do you debug?

> **In simple words:** The worker keeps running out of time. Find out whether he is waiting for someone else (a slow call), stuck at a locked door (network), or just too slow for the job (not enough memory/CPU).

Detailed answer

1. **Compare `Duration` with the configured timeout:** are invocations hitting the limit exactly (hung) or slowly rising (growing load/data)?
2. **Logs Insights:** search `Task timed out after` and see which requests or inputs time out; look at the `REPORT` lines for init duration and memory used.
3. **X-Ray traces** show which downstream call (DB, HTTP API, S3) consumes the time.

4. **VPC Lambda without internet path:** a function in private subnets with no NAT or VPC endpoint **hangs** on AWS API calls (Secrets Manager, S3, SQS) until the timeout. Add endpoints or a NAT.
5. **Cold starts/init:** heavy init code, large packages; use provisioned concurrency or SnapStart where supported.
6. **Memory:** more memory also means more CPU; check `Max Memory Used` and try higher memory.
7. **Database connections:** exhausted connections make calls wait; use **RDS Proxy** and reuse connections outside the handler.
8. **SDK and HTTP client timeouts/retries** must be **shorter than the Lambda timeout**, or retries silently eat the whole budget.
9. **Throttles and concurrency:** `Throttles` metric, reserved concurrency, downstream rate limits.
10. **SQS triggers:** the queue **visibility timeout** should be at least 6x the function timeout, otherwise messages reappear and are processed twice.

Example:

```
fields @timestamp, @requestId, @message
| filter @message like /Task timed out/
| stats count(*) as timeouts by bin(5m)

fields @timestamp, @duration, @maxMemoryUsed, @initDuration
| filter @type = "REPORT"
| sort @duration desc | limit 20
```

> ***What to say to the interviewer:*** *I compare Duration against the timeout, search Logs Insights for Task timed out and use X-Ray to find the slow downstream call. Common causes are a VPC Lambda with no NAT or endpoint hanging on AWS APIs, cold starts, too little memory, exhausted DB connections, SDK timeouts longer than the Lambda timeout, throttling, and an SQS visibility timeout that is too short.*

Q77. You want to auto-remediate high CPU on EC2 by rebooting it through Lambda. What is the logic?

> **In simple words:** Rebooting is like turning a laptop off and on: it often works, but you learn nothing about why it was slow. Take a quick photo of the problem first, and never let the robot reboot the same machine in an endless loop.

Detailed answer

First, you may not need Lambda: a CloudWatch alarm can reboot natively using the EC2 **reboot alarm action** (`arn:aws:automate:<region>:ec2:reboot`).

If you need logic (allowlists, diagnostics, notifications), the Lambda flow is:

1. CloudWatch alarm (CPU > 95 percent for 10 minutes) -> **EventBridge** "Alarm State Change" rule (or SNS) -> Lambda.
2. **Parse the instance ID** from the alarm dimensions.

3. **Guardrails:** only act on instances tagged `auto-remediate=true`; skip databases and stateful nodes; enforce a **cooldown** (for example no second reboot within 30 minutes, tracked in a tag or DynamoDB) to avoid reboot loops.
4. **Capture diagnostics** first with **SSM Run Command** (`top`, `ps`, logs) saved to S3.
5. Call **RebootInstances**.
6. **Notify** Teams/Slack with instance, time and diagnostics link; escalate if it happens again.
7. **IAM least privilege:** `ec2:RebootInstances` only on tagged instances, `ssm:SendCommand`, `ec2:CreateTags`.

Warning: rebooting **hides the root cause**. Use it as a temporary safety net and still do RCA (memory leak, runaway cron, traffic).

Example (the decision steps as CLI):

```
ID=i-0abc123def4567890
TAG=$(aws ec2 describe-tags --filters Name=resource-id,Values=$ID \
  Name=key,Values=auto-remediate --query "Tags[0].Value" --output text)
[ "$TAG" != "true" ] && echo "not allowlisted, notify only" && exit 0

LAST=$(aws ec2 describe-tags --filters Name=resource-id,Values=$ID \
  Name=key,Values=last-auto-reboot --query "Tags[0].Value" --output text)
# if LAST is within 30 minutes -> skip reboot and escalate to on-call

aws ssm send-command --instance-ids $ID --document-name AWS-RunShellScript \
  --parameters 'commands=["top -b -n1 | head -30","ps aux --sort=-%cpu | head"]' \
  --output-s3-bucket-name ops-diagnostics
aws ec2 reboot-instances --instance-ids $ID
aws ec2 create-tags --resources $ID --tags Key=last-auto-reboot,Value=$(date -u +%s)
```

> ***What to say to the interviewer:*** *If a plain reboot is enough, a CloudWatch alarm can reboot the instance natively without Lambda. With Lambda, the alarm goes through EventBridge, the function checks an allowlist tag and a cooldown to avoid reboot loops, captures diagnostics with SSM Run Command, reboots, and notifies the team, while I still do the RCA because a reboot only hides the cause.*

Q78. An automation job that starts and stops EMR clusters suddenly fails with IAM errors. How do you debug and fix it?

> **In simple words:** The robot's key stopped opening the door. Read exactly which door refused it, check which key the robot is actually carrying, and see who changed the lock.

Detailed answer

1. **Read the exact error:** which **action** and **resource** were denied, and whether the message says **explicit deny**, **service control policy** or **permissions boundary** (newer AccessDenied messages include this).
2. **Who am I?** `aws sts get-caller-identity` from the job's environment. The job may now run as a different role (profile changed, instance role replaced, expired or rotated keys, assume-role failing).
3. **CloudTrail:** filter by `errorCode = AccessDenied` / `UnauthorizedOperation` for the exact event and principal.
4. **Encoded messages:** decode with `aws sts decode-authorization-message`.

5. **IAM Policy Simulator** for the role against `elasticmapreduce:RunJobFlow`, `TerminateJobFlows`, and so on.
6. **Classic EMR gap:** `iam:PassRole`**.** Starting a cluster passes the EMR service role and the EC2 instance profile role; the caller needs `iam:PassRole` on those role ARNs.
7. **What changed?** CloudTrail for `PutRolePolicy`, `DetachRolePolicy`, SCP updates, new permission boundaries, tag-based conditions (cluster tags missing), KMS key policy changes for encrypted clusters.
8. **Fix** with the smallest least-privilege change, test with the simulator, commit it to IaC so it does not drift again, and add an alarm on repeated failures.

Example:

```
aws sts get-caller-identity
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::111111111111:role/emr-scheduler \
  --action-names elasticmapreduce:RunJobFlow iam:PassRole \
  --resource-arns "*"
```

```
{
  "Effect": "Allow",
  "Action": "iam:PassRole",
  "Resource": [
    "arn:aws:iam::111111111111:role/EMR_DefaultRole",
    "arn:aws:iam::111111111111:role/EMR_EC2_DefaultRole"
  ]
}
```

> ***What to say to the interviewer:*** *I read the exact denied action, resource and whether an explicit deny, SCP or boundary is involved, confirm the job's identity with sts get-caller-identity, and find the event in CloudTrail and test in the Policy Simulator. For EMR the usual gap is iam:PassRole on the service and instance-profile roles; I then check what changed, apply a least-privilege fix and commit it to IaC.*

Q79. An EMR cluster runs 24x7 but is used only 4 hours a day. What automation do you implement?

> **In simple words:** Do not keep a bus running all day for a 4-hour route. Call a bus when passengers arrive and send it home when the trip ends.

Detailed answer

1. **Transient clusters:** launch a cluster per job run, submit steps, and **auto-terminate after the last step**. Orchestrate with **Step Functions**, Airflow (MWAA), or **EventBridge Scheduler**.
2. **Auto-termination policy:** for interactive clusters, terminate after an **idle timeout** (for example 1 hour).
3. **Keep data in S3** (EMRFS), not HDFS, so clusters are disposable; keep the Hive/Glue Data Catalog external.
4. **EMR managed scaling:** set min/max units so the cluster shrinks when idle.
5. **Spot for task nodes** (and instance fleets with several instance types); On-Demand only for the primary/core nodes.
6. **EMR Serverless:** pay only while the job runs; no cluster to manage at all.

7. **Graviton instances** and right-sizing; tag and track cost per job.

Savings: roughly 20 hours a day of idle cost (about 80 percent) disappear, before Spot savings.

Example:

```
aws emr create-cluster --name nightly-etl --release-label emr-7.2.0 \
  --applications Name=Spark --use-default-roles \
  --instance-groups InstanceGroupType=MASTER,InstanceCount=1,InstanceType=m7g.xlarge \
    InstanceGroupType=CORE,InstanceCount=2,InstanceType=r7g.2xlarge \
    InstanceGroupType=TASK,InstanceCount=4,InstanceType=r7g.2xlarge,BidPrice=OnDemandPrice \
  --steps Type=Spark,Name=etl,ActionOnFailure=TERMINATE_CLUSTER,Args=[s3://jobs/etl.jar] \
  --auto-terminate

aws emr put-auto-termination-policy --cluster-id j-ABC123 \
  --auto-termination-policy IdleTimeout=3600
```

> ***What to say to the interviewer:*** *I would switch to transient clusters started per job by Step Functions or EventBridge Scheduler that auto-terminate after their steps, keep data in S3 so clusters are disposable, and add an idle auto-termination policy for any interactive cluster. Managed scaling, Spot task nodes and possibly EMR Serverless cut cost further, removing about 80 percent of the idle spend.*

Q80. How do you identify underutilized EC2 instances across multiple AWS accounts?

> **In simple words:** Walk through every office in the company with one checklist and find the computers that are switched on but nobody really uses.

Detailed answer

Find them (organization-wide):

1. **AWS Compute Optimizer** enabled at the **organization level** (management or delegated admin account): rightsizing recommendations for EC2, ASGs, EBS, Lambda across all accounts. Install the **CloudWatch agent** so it also uses memory metrics.
2. **Cost Explorer rightsizing recommendations** (from the management account) show savings in money.
3. **Trusted Advisor** "Low Utilization Amazon EC2 Instances" check (needs Business support or higher), viewable across the org.
4. **CloudWatch cross-account observability:** one monitoring account with dashboards of CPU, network and memory for all accounts (for example p95 CPU < 10 percent for 14 days).
5. **Tag hygiene:** owner, env and cost-center tags so each finding goes to the right team; untagged resources are reported.

Act:

- **Rightsize** (smaller type or Graviton), **stop or schedule** idle dev/test instances, terminate orphans after owner confirmation.
- Move steady baseline to **Savings Plans** after rightsizing (not before).
- Repeat monthly; track savings.

Example:

```
aws compute-optimizer update-enrollment-status --status Active --include-member-accounts

aws compute-optimizer get-ec2-instance-recommendations \
  --filters name=Finding,values=Overprovisioned \
  --query "instanceRecommendations[].{acct:accountId,id:instanceArn,now:currentInstanceType,
  rec:recommendationOptions[0].instanceType}" --output table
```

What to say to the interviewer: *I enable Compute Optimizer at the organization level with the CloudWatch agent for memory data, and combine it with Cost Explorer rightsizing, Trusted Advisor low-utilization checks and cross-account CloudWatch dashboards. Using owner tags, teams then rightsize, schedule or terminate the instances, and only after rightsizing do we buy Savings Plans for the baseline.*

Q81. Some APIs behind API Gateway are unsecured. How would you secure them?

In simple words: Some doors of the building have no lock. First walk around and list every unlocked door, then fit the right lock on each (ID check, badge, guest list), put a guard at the entrance, and install cameras.

Detailed answer

1. **Inventory:** list every route/method and find the ones with `authorizationType = NONE`. Also check for backend URLs (ALB, Lambda function URLs) reachable directly, bypassing API Gateway.
2. **Authentication per caller type:**

- End users: **Cognito user pool authorizer** (REST) or **JWT authorizer** (HTTP API) with any OIDC provider.
- Custom logic (tokens, headers): **Lambda authorizer**, with result caching.
- Service-to-service: **IAM auth** (`AWS_IAM`), callers sign with **SigV4** using their role.

3. **API keys + usage plans:** for throttling, quotas and metering per client. They are **not security** (keys are easy to leak); always pair them with a real authorizer.
4. **Resource policies:** allow only certain IP ranges, accounts or VPC endpoints (`aws:SourceIp`, `aws:SourceVpce`); use **private APIs** for internal-only traffic.
5. **AWS WAF** on the stage: managed rule groups (common, SQLi, known bad inputs) and **rate-based rules**.
6. **mTLS on a custom domain** for partners/B2B: clients must present a certificate from your truststore; disable the default `execute-api` endpoint so the custom domain is the only way in.
7. **Throttling** per stage and per method to protect backends.
8. **Request validation** (models/schemas for body, required params and headers) to reject bad input early.
9. **TLS 1.2+** security policy on custom domains.
10. **Logging and alerting:** CloudWatch access logs (caller, IP, status, latency), execution logs while debugging, alarms on 4XX/5XX spikes, CloudTrail for config changes.
11. **Do not expose backends directly:** internal ALB via VPC link, Lambda permission only for API Gateway, and IaC so the fix does not drift back.

Example:

```
# Find unauthenticated methods (REST API)
aws apigateway get-resources --rest-api-id a1b2c3 --embed methods \
  --query "items[].{path:path,methods:resourceMethods}" --output json \
  | grep -B3 '"authorizationType": "NONE"'

# Attach a Cognito authorizer to a method
aws apigateway update-method --rest-api-id a1b2c3 --resource-id r1s2t3 \
  --http-method GET --patch-operations \
  op=replace,path=/authorizationType,value=COGNITO_USER_POOLS \
  op=replace,path=/authorizerId,value=auth123
aws apigateway create-deployment --rest-api-id a1b2c3 --stage-name prod
```

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Principal": "*", "Action": "execute-api:Invoke",
      "Resource": "execute-api:/*" },
    { "Effect": "Deny", "Principal": "*", "Action": "execute-api:Invoke",
      "Resource": "execute-api:/*",
      "Condition": { "NotIpAddress": { "aws:SourceIp": ["203.0.113.0/24"] } } }
  ]
}
```

> ***What to say to the interviewer:*** *I first inventory every route with authorizationType NONE and any backend reachable directly, then add the right authentication: Cognito or JWT authorizers for users, Lambda authorizers for custom tokens and IAM SigV4 for service-to-service calls. I layer resource policies or private APIs, WAF managed and rate-based rules, mTLS for partners, throttling, request validation and TLS 1.2, and log and alarm on access; API keys and usage plans are only for metering, not security.*

GCP, Azure & DevSecOps

Basic

Q1. GCP: What is Cloud Armor, and how does it protect applications?

In simple words: Cloud Armor is the security guard standing at the front gate of your Google load balancer. It checks every visitor against a list of rules and turns away the troublemakers before they reach your house.

Detailed answer

Cloud Armor is Google Cloud's **WAF (Web Application Firewall) and DDoS protection** service. It is attached to a **backend service** behind the external Application Load Balancer (or Cloud CDN, and some other load balancer types), so traffic is filtered at Google's edge before it reaches your VMs or GKE pods.

What it gives you:

- **DDoS protection** at network level (always on) and application level (Layer 7 floods).
- **Preconfigured WAF rules** based on OWASP Top 10, for example SQL injection (`sqli-v33-stable`) and cross-site scripting (`xss-v33-stable`).
- **IP allow/deny lists** and **geo-blocking** (block or allow by country).
- **Rate limiting** - throttle or ban clients that send too many requests.
- **Adaptive Protection** - machine learning that detects unusual traffic and suggests rules.
- **Bot management** with reCAPTCHA integration.
- Rules can run in **preview mode** first, so you see what would be blocked without breaking users.

Example:

```
gcloud compute security-policies create web-policy --description "Prod WAF"

# Block SQL injection using Google's preconfigured OWASP rule
gcloud compute security-policies rules create 1000 --security-policy web-policy \
  --expression "evaluatePreconfiguredExpr('sqli-v33-stable')" --action deny-403

# Attach the policy to the backend service behind the load balancer
gcloud compute backend-services update web-backend --global --security-policy web-policy
```

What to say to the interviewer: *Cloud Armor is GCP's WAF and DDoS service that sits on the external HTTP(S) load balancer. I use it for OWASP rules like SQLi and XSS, IP and geo filtering, and rate limiting, and I always roll new rules out in preview mode first and watch the logs before enforcing.*

Q2. GCP: What network services are available in GCP?

In simple words: Think of GCP networking as the roads, gates, toll booths and signboards of a city - private roads (VPC), gates to the outside (NAT, load balancers), signboards (DNS) and express highways to your office (Interconnect/VPN).

Detailed answer

Main GCP networking services:

- **VPC** - a global private network; subnets are regional. Includes firewall rules and routes.
- **Shared VPC** and **VPC Network Peering** - share or connect networks between projects.
- **Cloud Load Balancing** - global external Application LB, regional/internal LBs, Network (TCP/UDP) LB.
- **Cloud NAT** - lets private VMs reach the internet without public IPs.
- **Cloud DNS** - managed public and private DNS zones.
- **Cloud CDN** - caches content at Google's edge.
- **Cloud VPN** and **Cloud Interconnect** - connect on-prem to GCP (encrypted tunnel vs dedicated fibre).
- **Cloud Router** - dynamic BGP routing for VPN/Interconnect.
- **Private Service Connect / Private Google Access** - reach Google APIs and services privately.
- **Cloud Armor** - WAF/DDoS; **Network Connectivity Center** - hub-and-spoke connectivity.

Need	GCP	AWS	Azure
Private network	VPC (global)	VPC (regional)	VNet
Outbound for private hosts	Cloud NAT	NAT Gateway	NAT Gateway
DNS	Cloud DNS	Route 53	Azure DNS
CDN	Cloud CDN	CloudFront	Azure Front Door / CDN
Hybrid link	Interconnect / VPN	Direct Connect / VPN	ExpressRoute / VPN Gateway

Example:

```
gcloud compute networks create prod-vpc --subnet-mode custom
gcloud compute networks subnets create app-subnet --network prod-vpc \
  --region asia-south1 --range 10.10.0.0/20
gcloud compute routers create nat-router --network prod-vpc --region asia-south1
gcloud compute routers nats create nat-cfg --router nat-router --region asia-south1 \
  --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges
```

> ***What to say to the interviewer:*** *The core ones are VPC with firewall rules, Cloud Load Balancing, Cloud NAT, Cloud DNS, Cloud CDN, Cloud VPN and Interconnect with Cloud Router, plus Private Service Connect and Cloud Armor. A key difference from AWS is that a GCP VPC is global while subnets are regional.*

Q3. GCP: What are the GCP security services other than Cloud Armor and VPC?

> **In simple words:** If Cloud Armor is the gate guard, these are the ID card office, the locker room, the CCTV and the safety inspector of the building.

Detailed answer

- **Cloud IAM** - who can do what; roles, service accounts, conditions.

- **Security Command Center (SCC)** - central dashboard of misconfigurations, vulnerabilities and threats.
- **Cloud KMS / Cloud HSM** - manage encryption keys, including customer-managed keys (CMEK).
- **Secret Manager** - store and version passwords, API keys, certificates.
- **Identity-Aware Proxy (IAP)** - log in to apps and VMs (SSH) without a VPN or public IP.
- **VPC Service Controls** - a perimeter that stops data being copied out of services like BigQuery/GCS.
- **Binary Authorization** - only allow signed/approved container images to deploy on GKE.
- **Artifact Analysis** - vulnerability scanning of container images in Artifact Registry.
- **Cloud Audit Logs** - who did what, where and when.
- **Organization Policy Service** - guardrails such as "no public IPs" or "only these regions".
- **Sensitive Data Protection (Cloud DLP)**, **reCAPTCHA Enterprise**, **Certificate Manager**, **Chronicle/Google SecOps** (SIEM).

Example:

```
# Create a secret and read it
echo -n "S3cr3tP@ss" | gcloud secrets create db-password --data-file=-
gcloud secrets versions access latest --secret db-password

# Org policy: forbid external IPs on VMs across the organization
gcloud resource-manager org-policies enable-enforce \
  compute.vmExternalIpAccess --organization 123456789012
```

> ***What to say to the interviewer:*** *Beyond Cloud Armor and VPC firewalls, I would use IAM with least privilege, Secret Manager and Cloud KMS for secrets and keys, Security Command Center for posture, IAP for access without bastions, VPC Service Controls against data exfiltration, Binary Authorization for GKE, and Audit Logs plus Organization Policies for governance.*

Q4. GCP: What is the GCP equivalent of CloudFormation?

> **In simple words:** CloudFormation is AWS's own "blueprint" tool. Google had its own blueprint tool too, but now it basically hands you Terraform and runs it for you.

Detailed answer

- **Cloud Deployment Manager** was GCP's native IaC tool (YAML/Jinja/Python templates), the direct equivalent of CloudFormation. It has been **deprecated** and Google has moved customers off it.
- **Infrastructure Manager (Infra Manager)** is the current Google-managed option. It runs **Terraform** for you, stores state and keeps a history of deployments - similar to a managed Terraform service.
- **Config Connector** lets you manage GCP resources as Kubernetes objects (YAML applied to a GKE cluster).
- In practice most teams simply use **Terraform** with the `google` provider and a GCS bucket for remote state.

Cloud	Native IaC
AWS	CloudFormation (and CDK)
Azure	ARM templates / Bicep
GCP	Deployment Manager (deprecated) -> Infrastructure Manager (Terraform)

Example:

```
gcloud infra-manager deployments apply projects/my-proj/locations/us-central1/deployments/web \
  --service-account projects/my-proj/serviceAccounts/infra@my-proj.iam.gserviceaccount.com \
  --git-source-repo https://github.com/org/infra --git-source-directory envs/prod
```

> ***What to say to the interviewer:*** *The historical equivalent was Cloud Deployment Manager, which is now deprecated; Google's current managed option is Infrastructure Manager, which runs Terraform for you. In real projects I use Terraform with the google provider and a GCS backend for state.*

Q5. GCP: What is the CloudWatch equivalent in GCP?

> **In simple words:** CloudWatch is AWS's health monitor. In GCP the same job is done by a family called "Google Cloud Observability" - one part watches numbers, another keeps the diary (logs).

Detailed answer

The equivalent is **Google Cloud Observability** (formerly called Stackdriver):

- **Cloud Monitoring** - metrics, dashboards, uptime checks, alerting policies (like CloudWatch Metrics + Alarms).
- **Cloud Logging** - central logs, Log Explorer, log-based metrics, log sinks to BigQuery/GCS/Pub/Sub (like CloudWatch Logs).
- **Cloud Trace** - distributed tracing (like X-Ray).
- **Cloud Profiler** and **Error Reporting**.
- **Google Cloud Managed Service for Prometheus** - run PromQL against GKE metrics without running Prometheus storage yourself.

AWS	GCP
CloudWatch Metrics / Alarms	Cloud Monitoring / Alerting policies
CloudWatch Logs	Cloud Logging
X-Ray	Cloud Trace
CloudTrail	Cloud Audit Logs

Example:

```
# Read recent error logs from GKE containers
gcloud logging read 'resource.type="k8s_container" AND severity>=ERROR' \
  --limit 20 --freshness 1h

# Send logs older-term to a storage bucket (cheap retention)
gcloud logging sinks create archive-sink storage.googleapis.com/my-log-archive \
  --log-filter 'severity>=WARNING'
```

What to say to the interviewer: *The CloudWatch equivalent is Google Cloud Observability - Cloud Monitoring for metrics and alerts, Cloud Logging for logs, and Cloud Trace for tracing. CloudTrail's equivalent is Cloud Audit Logs.*

Q6. GCP: What is a CDN, and how does GCP Cloud CDN work?

In simple words: A CDN is like keeping copies of a popular book in libraries in every city, so readers don't all travel to the one central library. Users get the nearest copy, faster.

Detailed answer

A **CDN (Content Delivery Network)** caches content (images, JS, CSS, videos, even API responses) at edge locations close to users. Benefits: lower latency, less load on the origin, lower egress cost, and some DDoS absorption.

How **Cloud CDN** works:

1. It is enabled on a **backend service or backend bucket** of the global external Application Load Balancer.
2. A user request hits the nearest Google edge (anycast IP).
3. **Cache hit** -> served straight from the edge. **Cache miss** -> fetched from the origin (GCS bucket, VMs, GKE, or an external origin), then cached.
4. Caching follows `Cache-Control` headers or a **cache mode** (`CACHE_ALL_STATIC`, `USE_ORIGIN_HEADERS`, `FORCE_CACHE_ALL`) with a TTL.
5. You can **invalidate** paths after a deploy, and use **signed URLs/cookies** for private content.
6. Cloud Armor edge policies can be combined with it.

Example:

```
gcloud compute backend-buckets create static-bucket --gcs-bucket-name my-static-site \
  --enable-cdn --cache-mode CACHE_ALL_STATIC --default-ttl 3600

# After a release, clear the old files
gcloud compute url-maps invalidate-cdn-cache web-map --path "/assets/*"
```

What to say to the interviewer: *A CDN caches content at edge locations near users to cut latency and origin load. In GCP, Cloud CDN is switched on per backend service or backend bucket of the global HTTP(S) load balancer; it caches based on Cache-Control or cache mode, and I invalidate paths after each deployment.*

Q7. GCP: What is the difference between a public cluster and a private cluster in GKE?

In simple words: A public cluster is a house where every room has a door onto the street. A private cluster is a gated building: rooms have no street doors, and even the manager's office (control plane) can be reached only through the gate you choose.

Detailed answer

	Public cluster	Private cluster
Node IPs	Nodes get public (external) IPs	Nodes have only internal IPs
Control plane endpoint	Public endpoint	Private endpoint; public endpoint optional and can be restricted
Outbound internet from nodes	Directly	Needs Cloud NAT
Pulling from Google APIs/Artifact Registry	Direct	Via Private Google Access
Security	Larger attack surface	Recommended for production

Key points for private clusters:

- You give the control plane its own small range (for example `--master-ipv4-cidr 172.16.0.0/28`).
- Use **authorized networks** to allow only specific CIDRs (office VPN, CI runners) to reach the API server.
- Admin access usually via a bastion with IAP, a VPN, or the private endpoint from inside the VPC.
- Newer GKE versions describe this as "control plane access" plus "private nodes" options, but the idea is the same.

Example:

```
gcloud container clusters create prod-gke --region asia-south1 \
  --enable-ip-alias --enable-private-nodes --master-ipv4-cidr 172.16.0.0/28 \
  --enable-master-authorized-networks --master-authorized-networks 203.0.113.10/32
```

> ***What to say to the interviewer:*** *In a public GKE cluster nodes have external IPs and the API endpoint is public; in a private cluster nodes only have internal IPs, outbound goes through Cloud NAT, and the control plane is private or restricted with authorized networks. For production I always use private nodes.*

Q8. Security: How do you manage secrets securely? Are Kubernetes Secrets encrypted?

Also asked as: Kubernetes Secrets aren't encrypted - what do you use instead? Where do you keep API keys and DB passwords?

> **In simple words:** A Kubernetes Secret is like writing your password in a sealed envelope - base64 just folds the paper, anyone who opens the envelope can read it. Real security is a bank locker (Vault/KMS) with a key you control, where only specific people can open it and every access is recorded.

Detailed answer

Facts to state clearly:

- A Kubernetes Secret is only **base64-encoded, not encrypted**. `echo <value> | base64 -d` reveals it.

- By default it is stored in **etcd** without extra encryption unless you enable **encryption at rest** (EncryptionConfiguration with a KMS provider; on EKS/GKE/AKS this is "envelope encryption with KMS").
- Anyone with `get secrets` RBAC permission in the namespace can read them.

Good practice:

- Keep the source of truth in a **secret manager**: HashiCorp Vault, AWS Secrets Manager / SSM Parameter Store, GCP Secret Manager, Azure Key Vault.
- Sync into pods with **External Secrets Operator** or the **Secrets Store CSI Driver**, or inject via Vault Agent.
- For GitOps, encrypt secrets in Git with **SOPS** (KMS-backed keys) or **Sealed Secrets** - never plain YAML in Git.
- Use workload identity (IRSA/EKS Pod Identity, GKE Workload Identity, Azure Workload Identity) so pods get cloud access without static keys.
- Tight RBAC, rotation, and audit logging on secret reads.

Example:

```
# ExternalSecret: pull db password from AWS Secrets Manager into a K8s Secret
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-creds
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets
    kind: ClusterSecretStore
  target:
    name: db-creds
  data:
    - secretKey: password
      remoteRef:
        key: prod/app/db
        property: password
```

> ***What to say to the interviewer:*** *Kubernetes Secrets are only base64 encoded, so I enable KMS encryption at rest for etcd, restrict secret access with RBAC, and keep the real secrets in a manager like Vault or AWS Secrets Manager synced via External Secrets Operator. In Git I only store SOPS or Sealed Secrets encrypted files, and pods get cloud access via workload identity, not static keys.*

Q9. Security: What methods do you use to check code vulnerabilities?

Also asked as: How do you do vulnerability scanning in the pipeline? How do you detect and measure security vulnerabilities for our company?

> **In simple words:** It is like checking a car at several stations - inspect the design drawings (SAST), check the spare parts you bought from others (SCA), inspect the finished car body (image scan), then take it for a test drive to see if the brakes fail (DAST).

Detailed answer

Type	What it checks	Tools
SAST (static)	Your source code for bugs like SQLi, hardcoded secrets	SonarQube, Semgrep, CodeQL, Checkmarx
SCA (dependencies)	Known CVEs in libraries you import	Snyk, OWASP Dependency-Check, Dependabot, Trivy
Secret scanning	Passwords/keys committed to Git	Gitleaks, TruffleHog, GitHub secret scanning
Container image scan	OS packages and libs inside the image	Trivy, Grype, ECR/Artifact Registry scanning
IaC scan	Misconfigured Terraform/K8s YAML	Checkov, tfsec/Trivy config, KICS
DAST (dynamic)	The running app, attacked from outside	OWASP ZAP, Burp Suite

How I wire it in CI/CD:

1. Pre-commit hooks for secrets (Gitleaks).
2. On pull request: SAST + SCA + IaC scan; results shown on the PR.
3. After image build: Trivy scan; **fail the build on CRITICAL/HIGH** with a fix available.
4. After deploy to staging: OWASP ZAP baseline scan.
5. Continuous: registry re-scans and Dependabot PRs for new CVEs.

How I **measure** it for the company (what you can't measure, you can't improve):

- **Open vulnerabilities by severity** (Critical/High/Medium/Low), trended weekly per team or service.
- **MTTR to patch** (mean time from detection to fix) and an **SLA per severity**, for example Critical within 7 days, High within 30 and Medium within 90, reported as the % fixed within SLA.
- **Coverage**: % of repos with SAST/SCA and % of running images scanned.
- **Posture scores**: Microsoft Defender for Cloud **secure score**, AWS **Security Hub** score, GCP Security Command Center findings.
- **External validation**: periodic **pen tests**, and a **bug bounty** or responsible-disclosure program.
- Monthly dashboard to leadership, so the trend is visible and owned.

Example:

```
# Fail the pipeline if the image has fixable HIGH/CRITICAL CVEs
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com/app:1.4.2

# SonarQube scan with quality gate
sonar-scanner -Dsonar.projectKey=app -Dsonar.qualitygate.wait=true
```

> ***What to say to the interviewer:*** *I layer the checks: secret scanning and SAST with SonarQube on every PR, SCA with Snyk or Trivy for dependency CVEs, Checkov for IaC, Trivy image scans that fail the build on fixable critical issues, and an OWASP ZAP DAST scan in staging. Quality gates make the pipeline block rather than just report.*

Q10. Security: What is the Shared Responsibility Model (in Azure, and how does it apply to AWS/GCP)?

In simple words: Renting a flat. The landlord looks after the building, the walls and the plumbing. You look after who you give keys to and what you keep inside. The more "furnished" the flat (PaaS/SaaS), the more the landlord does, but your valuables and keys are always your job.

Detailed answer

Security is split between the cloud provider and the customer. The split moves with the service model:

Responsibility	On-prem	IaaS (VMs)	PaaS (App Service, Azure SQL)	SaaS (M365)
Physical datacenter, hosts, network	Customer	Provider	Provider	Provider
Operating system and patching	Customer	Customer	Provider	Provider
Network controls (NSG, firewall rules)	Customer	Customer	Shared	Provider
Application code and config	Customer	Customer	Customer	Provider
Identity and access (who can log in)	Customer	Customer	Customer	Customer
Data (classification, encryption choices, backups)	Customer	Customer	Customer	Customer
Devices / endpoints	Customer	Customer	Customer	Customer

Key points:

- **You always own your data, identities, access management and devices**, whatever the service.
- AWS describes it as the provider securing **"of the cloud"** and the customer securing **"in the cloud"**. GCP calls it **shared fate** and adds secure defaults and blueprints, but the idea is the same.
- Most cloud breaches are customer-side problems: a public bucket, a leaked key, no MFA or an open NSG. None of these is the provider's failure.
- For example, with managed Kubernetes (AKS/EKS/GKE) the provider runs the control plane, but you own node patching (or turning on auto-upgrade), RBAC, network policies and your images.

Example:

```
# Customer-side checks you always own: MFA/Conditional Access, public exposure, encryption
az ad signed-in-user show --query userPrincipalName
az storage account list --query "[].{name:name,publicBlob:allowBlobPublicAccess,tls:minimumTlsVersio
    n}" -o table
az security secure-score-controls list -o table
```

> ***What to say to the interviewer:*** *The Shared Responsibility Model splits security between the provider, which secures the physical infrastructure and the more of the stack the more managed the service, and the customer. The customer always owns data, identities, access and endpoints; with IaaS they also own the OS and network controls, while PaaS and SaaS shift more to the provider. AWS calls it security of versus in the cloud, and GCP calls it shared fate.*

Q11. Azure: What is the difference between Azure Key Vault and Managed Identity, and how do they work together?

> **In simple words:** Key Vault is a bank locker that holds your secrets. Managed Identity is the app's own ID card, issued and renewed by Azure automatically. The app shows its ID card at the locker, and the locker opens only the drawers that ID is allowed to open. No password is written down anywhere.

Detailed answer

	Azure Key Vault	Managed Identity
What it is	A store for **secrets, keys and certificates**	An **identity** (a service principal in Entra ID) for an Azure resource
Solves	"Where do I keep the DB password / TLS cert / encryption key?"	"How does my app log in to Azure without a stored credential?"
Credentials	Holds them, with versioning, rotation and HSM-backed keys	None to manage; Azure issues and rotates tokens
Types	Standard / Premium (HSM), Managed HSM	System-assigned (lives and dies with the resource) or user-assigned (reusable)

How they work together:

1. Enable a managed identity on the App Service, VM, Function or AKS workload (AKS uses Workload Identity).
2. Grant that identity access on the vault: the **Key Vault Secrets User** RBAC role (recommended) or a legacy access policy.
3. The app asks the local endpoint (IMDS) for a token, and Azure SDKs do this through `DefaultAzureCredential`.
4. It calls Key Vault with the token and gets the secret. No password or connection string sits in config.
5. App Service can even use **Key Vault references** in app settings: `@Microsoft.KeyVault(SecretUri=...)`.

The **AWS analogy** is an IAM role attached to EC2/Lambda (identity) plus Secrets Manager (store). In GCP it is a service account with Workload Identity plus Secret Manager.

Example:

```
az webapp identity assign -g prod-rg -n pay-api
PRINCIPAL=$(az webapp identity show -g prod-rg -n pay-api --query principalId -o tsv)
az role assignment create --assignee $PRINCIPAL --role "Key Vault Secrets User" \
  --scope $(az keyvault show -n pay-kv --query id -o tsv)
az webapp config appsettings set -g prod-rg -n pay-api --settings \
  DB_PASSWORD="@Microsoft.KeyVault(SecretUri=https://pay-kv.vault.azure.net/secrets/db-pass/)"
```

What to say to the interviewer: *Key Vault stores secrets, keys and certificates, while Managed Identity gives the app an Entra ID identity with no credentials to manage. I enable a managed identity on the app, grant it the Key Vault Secrets User role, and the app gets a token and reads the secret at runtime, so nothing sensitive is in config. It's the same pattern as an IAM role plus Secrets Manager in AWS.*

Q12. Azure: What is Azure Monitor Logs, and how does it differ from Azure Monitor Metrics?

In simple words: Metrics are the car's speedometer and fuel gauge: small numbers, updated every minute, instant to read. Logs are the detailed trip diary: rich and searchable, but bigger, slower and more expensive to keep.

Detailed answer

	Azure Monitor Metrics	Azure Monitor Logs
Data	Numeric **time series** (CPU %, requests/sec)	Rich **records**: app logs, activity logs, resource logs, traces
Storage	Time-series metrics database	**Log Analytics workspace**
Query	Metrics Explorer, simple aggregations	**KQL (Kusto Query Language)**, joins, parsing
Latency	Near real time (about 1 minute)	Usually a few minutes of ingestion delay
Retention	Platform metrics kept 93 days	Configurable: interactive 30 days to 2 years, long-term archive up to 12 years
Cost	Platform metrics are free	Paid **per GB ingested** plus retention; the main cost driver
Alerts	Metric alerts (fast, cheap)	Log search alerts (flexible, slower)

When to use which:

- **Metrics** for fast health alerts, such as CPU > 85% for 5 minutes or HTTP 5xx rate.
- **Logs** for investigation and complex conditions, such as the top failing URLs by client IP or joining app errors with deployments.
- Send resource logs to a workspace with **Diagnostic settings**. Control cost with the Basic logs tier, daily caps, sampling and archive.
- Application Insights (workspace-based) stores its telemetry in the same workspace.

Example:

```
// KQL: 5xx errors per 5 minutes from App Service HTTP logs, last 24 hours
AppServiceHTTPLogs
| where TimeGenerated > ago(24h) and ScStatus >= 500
| summarize errors = count() by bin(TimeGenerated, 5m), CsUriStem
| order by errors desc

// Slowest dependencies seen by Application Insights
AppDependencies
| summarize p95 = percentile(DurationMs, 95) by Target
| top 5 by p95
```

What to say to the interviewer: *Azure Monitor Metrics is a near-real-time time-series store for numeric data like CPU, good for fast and cheap alerts, with 93 days of retention. Azure Monitor Logs stores rich records in a Log Analytics workspace that you query with KQL, with configurable retention and pay-per-GB ingestion. I use metric alerts for health and log queries for investigation and complex alerts.*

Advanced

Q13. GCP: How many IP ranges are required for a VPC-native GKE cluster, and what is each range used for?

In simple words: Imagine a housing society. One set of numbers for the buildings (nodes), another set for the flats inside them (pods), and a third set for shared services like the club house phone numbers (Services). They must never clash.

Detailed answer

A VPC-native (alias IP) GKE cluster needs **three ranges**:

Range	Type	Used for	Example
Primary subnet range	Subnet primary	Node IPs (VM NICs), internal LBs	10.10.0.0/20
Pod range	Secondary range	Pod IPs; each node gets a slice (default /24 = up to 110 pods)	10.20.0.0/14
Service range	Secondary range	ClusterIP addresses of Kubernetes Services	10.30.0.0/20

Extra points interviewers like:

- A **private cluster** also needs a small **/28 for the control plane** (`master-ipv4-cidr`) - not from your subnet.
- Pod IPs are real VPC IPs, so pods are routable directly from the VPC and peered networks - no overlay.
- **Size the pod range carefully**: it limits max nodes. With /24 per node, a /14 pod range gives 1024 nodes. You can reduce per-node pod IPs with `--default-max-pods-per-node`.
- Ranges must not overlap with on-prem or peered VPCs.
- Extra pod ranges can be added later (discontiguous multi-Pod CIDR) if you run out.

Example:

```
gcloud compute networks subnets create gke-subnet --network prod-vpc --region asia-south1 \
  --range 10.10.0.0/20 \
  --secondary-range pods=10.20.0.0/14,services=10.30.0.0/20

gcloud container clusters create prod-gke --region asia-south1 --enable-ip-alias \
  --subnetwork gke-subnet --cluster-secondary-range-name pods \
  --services-secondary-range-name services
```

What to say to the interviewer: *Three ranges: the subnet primary range for nodes, a secondary range for pods, and a secondary range for Services; a private cluster also needs a /28 for the control*

plane. I size the pod range for future node count because each node reserves a /24 by default.

Q14. Security: How do you secure a Kubernetes / GKE cluster at network, pod, and identity level?

Also asked as: How would you secure a Kubernetes/GKE cluster?

In simple words: Securing a cluster is like securing an office building - locked outer gates (network), rules for what each employee can do inside their room (pod), and ID badges that decide which doors open (identity). Plus CCTV (audit logs) and only trusted furniture allowed in (image policy).

Detailed answer

Infrastructure / cluster level

- **Private cluster** (private nodes), control plane restricted with **authorized networks**.
- **Shielded GKE Nodes** (secure boot, integrity monitoring), Container-Optimized OS, auto-upgrades via release channels (surge or blue-green node pool upgrades - steps are in the Terraform chapter).
- Encrypt Secrets in etcd with **Cloud KMS (application-layer secrets encryption)**.

Network level

- **NetworkPolicies** with default-deny, then allow only needed flows (GKE Dataplane V2 enforces them).
- Cloud Armor + HTTPS load balancer in front; no NodePort to the internet.
- Service mesh (Istio/Cloud Service Mesh) for **mTLS** between services if needed.

Pod / workload level

- **Pod Security Standards** (Restricted): non-root, no privileged, drop capabilities, read-only root filesystem.
- Resource limits, and GKE Sandbox (gVisor) for untrusted workloads.
- **Binary Authorization**: only images signed by our CI and scanned in Artifact Registry can run.

Identity level

- **Workload Identity Federation for GKE**: pods act as a Google service account without JSON keys.
- **RBAC** with least privilege; no `cluster-admin` for humans day to day; use Google Groups for RBAC.
- Don't use the default Compute Engine service account for nodes; create a minimal node SA.

Visibility: Cloud Audit Logs, GKE security posture dashboard, Security Command Center.

Example:

```
gcloud container clusters create prod-gke --region asia-south1 \
  --enable-private-nodes --master-ipv4-cidr 172.16.0.0/28 \
  --enable-master-authorized-networks --master-authorized-networks 10.0.0.0/8 \
  --enable-shielded-nodes --workload-pool my-proj.svc.id.goog \
  --binauthz-evaluation-mode PROJECT_SINGLETON_POLICY_ENFORCE \
  --database-encryption-key projects/my-proj/locations/asia-south1/keyRings/gke/cryptoKeys/etcd \
  --service-account gke-nodes@my-proj.iam.gserviceaccount.com --enable-dataplane-v2
```

What to say to the interviewer: *I secure it in layers: private nodes with authorized networks and shielded nodes; default-deny NetworkPolicies and Cloud Armor at the edge; Pod Security Standards and Binary Authorization for workloads; and Workload Identity plus least-privilege RBAC for identity, with KMS encryption of secrets and audit logs for visibility.*

Q15. Security: How do you design IAM with least privilege and handle permissions across AWS accounts and GCP projects?

Also asked as: How do you handle permissioning across AWS/GCP projects? (Don't say "give admin to service accounts".)

In simple words: Give people keys only to the rooms they actually work in, only for as long as they need, and keep a register of every key. In a big company you also build walls between departments (accounts/projects), so a lost key opens one department, not the whole building.

Detailed answer

Principles (both clouds)

- **Least privilege**: start from zero and grant only the actions and resources needed. No ***:***, and no admin roles on service accounts.
- Humans log in with **SSO** and get temporary access. **No long-lived keys.**
- **Grant to groups, not individuals**, so joiners and leavers are handled in one place.
- **Just-in-time (JIT)** elevated access with approval and expiry, for example for prod admin.
- **Periodic access reviews**, plus unused-permission cleanup.
- **Everything in Terraform**: IAM changes go through PRs, review and audit, never console clicks.

AWS

- **AWS Organizations** with **OUs** (Security, Prod, NonProd, Sandbox) and **SCPs** as the maximum-allowed guardrail, for example deny leaving the org, deny disabling CloudTrail, or allow only approved regions.
- **IAM Identity Center permission sets** assigned per account (ReadOnly everywhere, Developer in dev, a narrow Deployer in prod).
- **Cross-account roles** with trust policies (and an ExternalId for third parties); CI assumes a deploy role in each account.
- **Permission boundaries** so teams can create roles without escalating privilege.
- **IAM Access Analyzer**: find external access, generate policies from CloudTrail activity, flag unused permissions.
- Workloads use instance profiles and IRSA/EKS Pod Identity.

GCP

- The **org -> folders -> projects** hierarchy. Grant at the **lowest level** that works, because permissions inherit downward.
- **Google Groups** as principals. **Predefined roles** first, **custom roles** when predefined ones are too broad; avoid basic roles (Owner/Editor).
- **IAM Conditions**: limit a role by resource name prefix, tag or time.

- **Service account impersonation** (`roles/iam.serviceAccountTokenCreator`) instead of downloading keys; block keys with the org policy `iam.disableServiceAccountKeyCreation`.
- **Workload Identity Federation** for CI (GitHub/GitLab OIDC) and cross-cloud (an AWS role -> GCP), plus GKE Workload Identity for pods.
- Use the **IAM Recommender** to trim excess grants.

Example:

```
# AWS SCP: deny actions outside ap-south-1 (global services exempt); CloudTrail can't be stopped
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Deny", "NotAction": ["iam:*", "sts:*", "support:*"], "Resource": "*",
     "Condition": {"StringNotEquals": {"aws:RequestedRegion": "ap-south-1"}}},
    {"Effect": "Deny", "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail"],
     "Resource": "*"}
  ]
}

# GCP: group gets a narrow role on one project with a condition; CI impersonates an SA
gcloud projects add-iam-policy-binding prod-app --member group:devs@corp.com \
  --role roles/storage.objectViewer \
  --condition=expression='resource.name.startsWith("projects/_/buckets/app-logs")',title=logs-only
gcloud storage ls gs://app-logs \
  --impersonate-service-account deployer@prod-app.iam.gserviceaccount.com
```

What to say to the interviewer: *In AWS I use Organizations with OUs and SCPs as guardrails, Identity Center permission sets per account, cross-account deploy roles, permission boundaries and Access Analyzer. In GCP I use the org-folder-project hierarchy with Google Groups, predefined or custom roles at the lowest level with IAM Conditions, service account impersonation instead of keys, and Workload Identity Federation for CI and cross-cloud. In both, access is just-in-time, reviewed periodically and managed entirely in Terraform.*

Q16. Security: What policies and tools are used for policy compliance checks, and how do you enforce policy as code across environments?

Also asked as: How do you enforce policy as code across environments? OPA? Sentinel? Pulumi? What are OPA/Gatekeeper, IAM policies, Organization policies and Pod Security Standards?

In simple words: Company rules like "no one enters without a badge" only work if a guard checks every time. Policy as code turns the rule book into code, and robot guards check it at three doors: when a change is planned (CI), when it enters the cluster (admission), and all the time in the cloud (guardrails).

Detailed answer

1. Pre-deploy, in CI (cheapest place to catch problems)

- **Conftest / OPA** on the Terraform plan: `terraform show -json plan.out | conftest test -`.
- **Checkov / tfsec (Trivy config)** static scans of Terraform, Helm and K8s YAML.
- **Sentinel** in Terraform Cloud/Enterprise, with levels advisory, soft-mandatory and hard-mandatory.
- **Pulumi CrossGuard** policy packs if the team uses Pulumi.

2. Admission, in the cluster

- **OPA Gatekeeper** (ConstraintTemplates written in Rego, plus Constraints) or **Kyverno** (ClusterPolicies in YAML).
- Typical rules: no privileged pods, images only from our registry, limits required, labels required.
- **Pod Security Standards** (Baseline/Restricted) through the built-in Pod Security Admission.
- Start in **audit/warn** mode (Gatekeeper `dryrun`/`warn`, Kyverno `Audit`), then switch to **enforce/deny**.

3. Runtime and cloud guardrails (catch anything that bypassed CI)

- **AWS**: SCPs in Organizations (deny wrong regions, deny disabling CloudTrail) plus **AWS Config rules** and Security Hub.
- **GCP**: **Organization Policies**, for example `iam.disableServiceAccountKeyCreation` or no external IPs.
- **Azure**: **Azure Policy** with deny/audit effects and initiatives (CIS, PCI).

How I run it across environments

- One **policy repo**, versioned (git tags) and tested with `opa test` / `kyverno test` in its own CI.
- Promote **dev -> stage -> prod**: new rules run in **warn** mode first, we fix the violations, then **enforce**.
- **Exceptions** are explicit: a label or annotation such as `policy-exempt=reason` with an **expiry date** and an owner, reviewed regularly. Never a silent bypass.
- Policy results go to dashboards, so compliance is measurable.

Example:

```
# policy/s3.rego - run with: conftest test plan.json
package main

deny[msg] {
  r := input.resource_changes[_]
  r.type == "aws_s3_bucket_public_access_block"
  r.change.after.block_public_acls == false
  msg := sprintf("%s: public ACLs must be blocked", [r.address])
}

deny[msg] {
  r := input.resource_changes[_]
  r.change.actions[_] == "create"
  not r.change.after.tags.owner
  msg := sprintf("%s: missing required tag 'owner'", [r.address])
}

# CI steps
terraform plan -out plan.out && terraform show -json plan.out > plan.json
conftest test plan.json --policy policy/
opa test policy/ -v
```

> ***What to say to the interviewer:*** *I enforce policy at three points: in CI with Conftest/OPA on the Terraform plan JSON, Checkov, or Sentinel in Terraform Cloud; at cluster admission with Gatekeeper or Kyverno plus Pod Security Standards; and at runtime with SCPs and AWS Config, GCP Organization Policies or Azure Policy. The policies live in one versioned, tested repo, get promoted dev to prod in warn-then-enforce mode, and exceptions are labelled with an owner and an*

expiry date.

Q17. Security: What audit logging and encryption strategies do you follow?

In simple words: Encryption is putting valuables in a locked box - when stored and when moving in the van. Audit logs are the CCTV recording who opened which box and when, stored somewhere the thief can't delete.

Detailed answer

Encryption

- **At rest**: enable encryption for disks, databases, buckets, backups and etcd. Default provider keys are fine for most data; use **customer-managed keys (KMS / Cloud KMS / Key Vault)** for sensitive data so you control rotation and can revoke access.
- **In transit**: TLS 1.2+ everywhere - on the load balancer, and ideally end-to-end to the backend; mTLS between services with a mesh.
- **Key management**: automatic key rotation, separate keys per environment/data class, least-privilege on who can use (`kms:Decrypt`) vs manage keys.
- **Secrets**: in a secret manager, never in code or images.

Audit logs

- Turn on **AWS CloudTrail (all regions, org trail)**, **GCP Cloud Audit Logs (incl. Data Access for sensitive services)**, **Azure Activity Log**, and **Kubernetes API audit logs**.
- Ship to a **separate, locked-down log account/project**; enable immutability (S3 Object Lock, bucket retention lock).
- Set retention per compliance (e.g., 1 year hot, 7 years archive for banking).
- Alert on high-risk events: root login, IAM policy changes, security group opened to 0.0.0.0/0, KMS key deletion; feed into a SIEM (Security Hub, Sentinel, Chronicle).

Example:

```
aws cloudtrail create-trail --name org-trail --s3-bucket-name org-audit-logs \
  --is-multi-region-trail --is-organization-trail --enable-log-file-validation
aws kms enable-key-rotation --key-id alias/prod-data
```

What to say to the interviewer: *I encrypt everything at rest and in transit, using customer-managed KMS keys with rotation for sensitive data, and TLS 1.2 plus mTLS inside the mesh. All control-plane activity goes to organization-wide CloudTrail or Cloud Audit Logs stored immutably in a separate security account, with alerts on risky actions and a SIEM on top.*

Q18. Azure: How would you use Azure Application Gateway with WAF for a sensitive banking application?

In simple words: Application Gateway is the bank's front desk that decides which counter you go to; WAF is the security scanner at the door. For a bank you add metal detectors, ID checks, CCTV and a locked vault behind them.

Detailed answer

Design:

1. **Application Gateway WAF_v2 SKU**, **zone-redundant**, with autoscaling (min instances set for peak).
2. **WAF policy in Prevention mode** with the Microsoft **Default Rule Set (DRS 2.x)** / OWASP CRS, plus **Bot Manager** rules. Tune false positives with exclusions after running in Detection mode on staging.
3. **Custom rules**: geo-filter (e.g., allow only India), rate limiting, block known bad IPs, allow admin paths only from office IPs.
4. **TLS**: HTTPS only, TLS 1.2+ with a strong predefined SSL policy; certificates stored in **Azure Key Vault**, pulled by App Gateway via **managed identity**; **end-to-end TLS** to the backends.
5. **Backends are private**: App Service/AKS behind **private endpoints** or internal IPs; backend access restricted so traffic can only come through App Gateway.
6. **Network**: App Gateway in its own subnet with NSG, **Azure DDoS Network Protection** on the VNet. For global users, put **Azure Front Door Premium** (with WAF) in front.
7. **Monitoring**: WAF and access logs to **Log Analytics** and **Microsoft Sentinel**; alerts on blocked-request spikes.
8. **Compliance**: Azure Policy to enforce WAF enabled and TLS versions (PCI-DSS, RBI guidelines).

Example:

```
az network application-gateway waf-policy create -g bank-rg -n bank-waf \
  --type Microsoft_DefaultRuleSet --version 2.1
az network application-gateway waf-policy policy-setting update -g bank-rg \
  --policy-name bank-waf --mode Prevention --state Enabled
az network application-gateway waf-policy custom-rule create -g bank-rg \
  --policy-name bank-waf -n geoBlock --priority 10 --rule-type MatchRule --action Block
```

> ***What to say to the interviewer:*** *I would deploy a zone-redundant App Gateway WAF_v2 with the Microsoft Default Rule Set in Prevention mode, custom geo and rate-limit rules, TLS 1.2 end-to-end with certificates from Key Vault, and private backends reachable only through the gateway. DDoS Protection, WAF logs into Sentinel and Azure Policy for compliance complete the design.*

Q19. Azure: Explain the difference in scaling strategies for compute-intensive vs I/O-intensive workloads.

> **In simple words:** A compute-heavy job is like a kitchen short on chefs - hire more chefs or faster chefs. An I/O-heavy job is a kitchen where chefs wait for ingredients from the store - hiring chefs won't help; you need a faster supply line or a fridge nearby (cache).

Detailed answer

	Compute-intensive (video encoding, ML, reports)	I/O-intensive (DB-heavy APIs, file processing, queues)
Bottleneck	CPU/GPU	Disk, network, DB, downstream APIs
Scale signal	CPU utilization	Queue length, request latency, IOPS, connections
VM choice (Azure)	Compute-optimized (F/Fsv2), HPC, GPU (NC/ND)	Memory/storage optimized (E, L-series), Premium SSD v2 / Ultra Disk
Scale method	Scale out on CPU (VMSS autoscale, HPA), batch parallelism, Spot VMs for batch	Scale on backlog (KEDA, Functions queue triggers), cache (Azure Cache for Redis), async processing
Watch out	Noisy neighbours, CPU throttling limits in K8s	Adding instances can overload the DB; connection limits, disk throughput caps

Key points:

- For I/O workloads, **scaling out can make things worse** if the database is the bottleneck - fix with caching, read replicas, connection pooling, batching, or bigger disk throughput.
- Azure specifics: **VM Scale Sets** autoscale on CPU for compute; **Azure Batch** for big parallel compute; **Functions/Container Apps with KEDA** scaling on Service Bus/Event Hub backlog for I/O; disk performance scales with disk size/tier.

Example:

```
# KEDA scaler: scale I/O workers on Service Bus queue depth, not CPU
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: order-worker
spec:
  scaleTargetRef:
    name: order-worker
  minReplicaCount: 1
  maxReplicaCount: 30
  triggers:
    - type: azure-servicebus
      metadata:
        queueName: orders
        messageCount: "50"
      authenticationRef:
        name: sb-auth
```

> ***What to say to the interviewer:*** *Compute-bound workloads scale on CPU using compute-optimized or GPU VMs, scale sets or HPA, and Spot for batch. I/O-bound workloads scale on backlog or latency with KEDA and need faster storage, caching and connection pooling, because simply adding instances can overload the database.*

Q20. Security: What security measures would you implement while hosting a production web application?

Also asked as: What are best practices to protect a hosted web application? (AWS-specific version is in the AWS chapter.)

> **In simple words:** Defense in depth - like a bank with a guard, locked doors, a vault, cameras and an alarm. If one layer fails, the next one still stops the thief.

Detailed answer

Layer by layer:

- **Edge**: CDN + **WAF** (OWASP rules, rate limiting, bot protection) + **DDoS protection**; HTTPS only with HSTS.
- **Network**: app servers in **private subnets**, only the load balancer is public; tight security groups/firewall rules; no SSH from the internet (use SSM Session Manager / IAP / Bastion).
- **Application**: input validation, parameterized queries, secure headers (CSP, X-Frame-Options), authentication with MFA/OAuth, session security, CORS limited.
- **Secrets & identity**: secrets manager, no keys in code; least-privilege IAM roles for the app.
- **Data**: encryption at rest (KMS) and in transit (TLS 1.2+); private DB endpoints; backups encrypted and tested.
- **Supply chain**: SAST/SCA/image scanning in CI, signed images, minimal non-root containers, patching.
- **Monitoring & response**: audit logs, WAF logs, SIEM alerts, runbooks, regular pentests and DR drills.

Example:

```
# Nginx security headers on the web tier
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Content-Security-Policy "default-src 'self'" always;
ssl_protocols TLSv1.2 TLSv1.3;
```

> ***What to say to the interviewer:*** *I use defense in depth: CDN with WAF and DDoS protection at the edge, private subnets with only the load balancer exposed, least-privilege IAM and a secrets manager, encryption at rest and in transit, scanned and signed images in CI, and centralized logging with alerting and regular pentests.*

Q21. Security: What is email/commit signing and Helm chart signing? Which tools do you use to sign Helm charts?

> **In simple words:** Signing is a tamper-proof seal on a medicine bottle. If the seal is intact and carries the company's stamp, you know who packed it and that nobody opened it on the way. A signature proves who made something and that it was not changed.

Detailed answer

How signing works in one line: the author signs with a **private key**; anyone can check it with the matching **public key**. If even one byte changes, verification fails.

- **Email signing**: **S/MIME** (certificate-based, common in Outlook) or **PGP/GPG** signs emails so the receiver knows the sender is real and the text was not altered.
- **Git commit signing**: sign commits and tags with **GPG**, **SSH keys** or **Sigstore gitsign** (`git commit -S`). GitHub/GitLab show a **Verified** badge, and branch protection can **require signed commits**, so nobody can push code pretending to be someone else.
- **Helm chart signing (classic)**: `helm package --sign` uses a **GPG key** and creates a `.prov` **provenance file** next to the `.tgz` (it holds the chart's hash plus signature). Users run `helm verify` or `helm install --verify` with the public keyring.
- **Helm charts in OCI registries (modern)**: charts pushed to ECR/Artifact Registry/Harbor are signed with **Sigstore cosign** (or **Notation**/Notary v2), the same way as container images; keyless signing uses the CI identity via OIDC.
- **Container image signing**: **cosign** or **Notation** in CI after the image scan passes; then **enforce** at deploy time with **Kyverno verifyImages**, **Gatekeeper + Ratify**, or **GKE Binary Authorization** - unsigned images are rejected.

Why it matters: stops **supply-chain attacks** (a tampered chart or image from a compromised registry) and proves who approved what for audits.

Example:

```
# Git: sign commits with GPG
git config --global user.signingkey 3AA5C34371567BD2
git commit -S -m "Add ingress config"

# Classic Helm provenance with GPG
helm package ./webapp --sign --key "DevOps Team" --keyring ~/.gnupg/secring.gpg
helm verify webapp-1.2.0.tgz --keyring ~/.gnupg/pubring.gpg

# OCI chart + image signed with cosign
helm push webapp-1.2.0.tgz oci://123456789012.dkr.ecr.ap-south-1.amazonaws.com/charts
cosign sign --key awskms:///alias/cosign 123456789012.dkr.ecr.ap-south-1.amazonaws.com/charts/webapp
    :1.2.0
cosign verify --key cosign.pub 123456789012.dkr.ecr.ap-south-1.amazonaws.com/app:1.4.2
```

> ***What to say to the interviewer:*** *Signing is a tamper-proof seal - a private-key signature that proves who produced an email, commit, chart or image and that it wasn't modified. I sign commits with GPG and enforce verified commits, sign classic Helm charts with helm package --sign and verify the .prov file, and for OCI charts and images I use cosign with a KMS key, enforcing only signed images at admission with Kyverno or Binary Authorization.*

Q22. Azure: How do you implement high availability for a multi-region application?

> **In simple words:** Don't keep all your shops in one mall. First spread tills across several floors (zones) so one broken escalator doesn't close the shop. Then open a second mall in another city (region) with a signboard on the highway (Front Door) that sends customers to whichever mall is open.

Detailed answer

Layer 1 - inside a region: Availability Zones

- Zone-redundant services: **App Service** (zone redundancy, 3+ instances), **AKS** node pools across zones 1/2/3, zone-redundant Application Gateway, and **ZRS** storage.
- Zones protect against one datacenter failing, with no data loss.

Layer 2 - across regions (usually Azure paired regions, e.g. Central India + South India)

- **Global entry point**:
- **Azure Front Door**: Layer 7, global anycast, WAF, TLS, caching, **health probes**, failover in seconds, path-based routing.
- **Traffic Manager**: **DNS-based**, works for any protocol, but failover depends on the DNS TTL.
- **Data tier** (the hard part):
- **Azure SQL auto-failover groups**: geo-replication with a single listener endpoint that follows the primary.
- **Cosmos DB** multi-region writes, or a single write region with automatic failover.
- **Storage**: GRS/GZRS, or **RA-GZRS** if you need to read the secondary.
- **Redis**: geo-replication (Premium/Enterprise).
- **Patterns**:
- **Active-active**: both regions serve traffic. Best RTO, but needs data consistency handling.
- **Active-passive (warm standby)**: the secondary is scaled down and scaled up on failover. Cheaper.

Make it real

- Everything is in IaC, so the second region is identical.
- Deep health endpoints are used as probes.
- Define **RTO/RPO**, write failover runbooks, and run **regular DR drills**.

Example:

```
az sql failover-group create -g prod-rg --server sql-cin --partner-server sql-sin \
  -n pay-fog --add-db paydb --failover-policy Automatic --grace-period 1
az afd origin create -g prod-rg --profile-name pay-fd --origin-group-name web \
  --origin-name cin --host-name pay-cin.azurewebsites.net --priority 1 --weight 1000 \
  --enabled-state Enabled --origin-host-header pay-cin.azurewebsites.net
az afd origin create -g prod-rg --profile-name pay-fd --origin-group-name web \
  --origin-name sin --host-name pay-sin.azurewebsites.net --priority 2 --weight 1000 \
  --enabled-state Enabled --origin-host-header pay-sin.azurewebsites.net
```

> ***What to say to the interviewer:*** *I use Availability Zones inside each region, with zone-redundant App Service or AKS and ZRS storage, and then a paired secondary region behind Azure Front Door with health probes. Front Door is my default because it gives layer 7 failover in seconds plus WAF; Traffic Manager is the option for DNS-based or non-HTTP traffic. Data uses SQL auto-failover groups, Cosmos DB multi-region or RA-GZRS storage, and I choose active-active or active-passive by the RTO/RPO and budget, then prove it with DR drills.*

Q23. Azure: How do you set up automated scaling for Azure App Service?

> **In simple words:** Scaling up is buying a bigger oven. Scaling out is adding more ovens. Autoscale is a manager who adds ovens when the orders queue grows and removes them when it's quiet, with a short wait in between so it doesn't keep switching them on and off.

Detailed answer

- **Scale up** = a bigger App Service plan tier/size (more CPU/RAM per instance). This is manual and causes a restart.
- **Scale out** = more instances of the plan. This is what autoscale does.

Autoscale (Azure Monitor autoscale settings):

- Needs **Standard tier or higher** (Premium v3 recommended); Basic/Free can't autoscale.
- **Rules** on metrics such as **CPU %**, **memory %**, **HTTP queue length**, or a Service Bus queue for workers.
- Always create a **scale-out rule and a matching scale-in rule**, with **cooldowns** (e.g. 5-10 minutes) to avoid flapping.
- Set **min/max/default instances**. Keep min >= 2 or 3 for HA (and for zone redundancy).
- **Scheduled profiles**: for example, more instances during business hours or before a known sale event.
- Autoscale settings are per **plan**, so all apps on the plan scale together.

Automatic scaling (the newer Premium v2/v3 feature): App Service scales on **HTTP traffic** with pre-warmed instances. You set a maximum burst and optional always-ready instances, with no rules to write.

Watch out: scaling the web tier doesn't help if the database is the bottleneck, and slow app startup delays scale-out, so use warm-up and health check settings.

Example:

```
az monitor autoscale create -g prod-rg --resource pay-plan \
  --resource-type Microsoft.Web/serverfarms --name pay-autoscale \
  --min-count 2 --max-count 10 --count 2

az monitor autoscale rule create -g prod-rg --autoscale-name pay-autoscale \
  --condition "CpuPercentage > 70 avg 10m" --scale out 2 --cooldown 5
az monitor autoscale rule create -g prod-rg --autoscale-name pay-autoscale \
  --condition "CpuPercentage < 30 avg 15m" --scale in 1 --cooldown 10
az monitor autoscale rule create -g prod-rg --autoscale-name pay-autoscale \
  --condition "HttpQueueLength > 100 avg 5m" --scale out 2 --cooldown 5
```

> ***What to say to the interviewer:*** *I keep the plan on Premium v3 and create autoscale settings with scale-out and scale-in rule pairs on CPU, memory or HTTP queue length, with cooldowns, a minimum of 2-3 instances for HA and a sensible maximum. I add scheduled profiles for known peaks, or use the newer automatic scaling for HTTP-driven apps. Scaling up to a bigger SKU is separate, and I check that the database won't become the next bottleneck.*

Q24. Azure: How do you use Microsoft Defender for Cloud for workload protection?

Also asked as: What is Azure Defender / Azure Security Center?

In simple words: Defender for Cloud is both a building inspector and a security guard. The inspector gives you a checklist and a score for what's unlocked or misconfigured (posture). The guard watches your servers, containers and databases live and raises the alarm when someone breaks in (threat protection).

Detailed answer

Two halves:

- **CSPM (Cloud Security Posture Management)**
- **Secure score** plus prioritized **recommendations**, such as "enable MFA", "storage allows public access" or "VMs missing updates".
- **Regulatory compliance dashboards**: Microsoft Cloud Security Benchmark, CIS, PCI-DSS, ISO 27001.
- Works multi-cloud by connecting AWS and GCP accounts.
- Defender CSPM (the paid plan) adds attack path analysis and agentless scanning.
- **CWP (Cloud Workload Protection) - Defender plans per resource type**
- **Servers**: vulnerability assessment, EDR (Defender for Endpoint), file integrity monitoring, **just-in-time VM access**, which opens RDP/SSH ports only on request, for a limited time and from your IP.
- **Containers**: **image vulnerability scanning** in ACR, **runtime threat detection** on AKS (e.g. crypto-miners, privileged container launch) and Kubernetes posture checks.
- **SQL, Storage** (malware scanning, anomalous access), **Key Vault** (unusual secret access), **App Service**, **Resource Manager** and **DNS**.

How I roll it out:

1. Enable it at subscription or management-group level, via Azure Policy or Terraform so it's consistent.
2. Turn on the Defender plans for the resource types we actually run.
3. Work down the recommendations by secure-score impact; exempt with a justification where needed.
4. Stream alerts to **Microsoft Sentinel** (SIEM) or Teams through workflow automation; use continuous export to Log Analytics.
5. Track secure score and compliance percentages monthly as KPIs.

Example:

```
az security pricing create -n VirtualMachines --tier Standard
az security pricing create -n Containers --tier Standard
az security pricing create -n KeyVaults --tier Standard
az security secure-score-controls list -o table
az security assessment list --query "[?status.code=='Unhealthy'].displayName" -o tsv
```

> ***What to say to the interviewer:*** *Defender for Cloud does two jobs: CSPM, with a secure score, recommendations and regulatory compliance dashboards, and workload protection plans for servers, containers, SQL, storage and Key Vault. Those plans cover image scanning in ACR, AKS runtime threat detection and just-in-time VM access. I enable it org-wide through policy, fix recommendations by score impact, and route alerts into Sentinel.*

Scenario-Based

Q25. GCP: How do you restrict and protect a web application from cyber attacks on GCP?

In simple words: Put the app behind Google's big front gate (global load balancer), station the guard there (Cloud Armor), lock all the back doors (private backends, firewall), and keep the keys in a safe (Secret Manager, IAM).

Detailed answer

Step by step:

1. **Global external Application Load Balancer** as the only entry point, with **Google-managed SSL certificates** and an SSL policy enforcing TLS 1.2+; HTTP -> HTTPS redirect.
2. **Cloud Armor** policy on the backend: OWASP preconfigured rules (SQLi, XSS, LFI, RCE), **rate limiting / rate-based ban**, geo restriction, **Adaptive Protection** for L7 DDoS, reCAPTCHA Enterprise for bots. Start in preview mode.
3. **Private backends**: GKE private nodes or MIGs with no external IPs; firewall allows port 443/80 only from Google LB and health-check ranges (`35.191.0.0/16`, `130.211.0.0/22`).
4. **Admin access** through **Identity-Aware Proxy (IAP)** - no public SSH, no bastion with open ports.
5. **Cloud CDN** for static content to absorb traffic.
6. **Data protection**: Cloud SQL with private IP, CMEK via Cloud KMS, Secret Manager for credentials, **VPC Service Controls** around sensitive data.
7. **Supply chain**: Artifact Analysis scanning + Binary Authorization.
8. **Detect**: Security Command Center, Cloud Armor logs in Cloud Logging, alerts on spikes of 403/429.

Example:

```
# Allow LB and health-check traffic only; nothing else reaches the backends
gcloud compute firewall-rules create allow-glb --network prod-vpc --direction INGRESS \
  --allow tcp:80,tcp:443 --source-ranges 35.191.0.0/16,130.211.0.0/22 --target-tags web

# Rate-based ban: >100 req/min per IP -> banned for 10 minutes
gcloud compute security-policies rules create 900 --security-policy web-policy \
  --src-ip-ranges "*" --action rate-based-ban --rate-limit-threshold-count 100 \
  --rate-limit-threshold-interval-sec 60 --ban-duration-sec 600 \
  --conform-action allow --exceed-action deny-429 --enforce-on-key IP
```

What to say to the interviewer: *I expose the app only through the global HTTPS load balancer with Cloud Armor OWASP rules, rate-based bans and Adaptive Protection, keep backends private with firewall rules allowing only Google LB ranges, and use IAP for admin access. Data is protected with private Cloud SQL, KMS and Secret Manager, and Security Command Center plus Armor logs give detection.*

Q26. Azure: During an Azure deployment you get intermittent DNS resolution issues. What can be the causes?

In simple words: DNS is the phone book. "Sometimes it works" means some phone books are outdated, some operators are overloaded, or some calls are routed to the wrong exchange.

Detailed answer

Likely causes and checks:

1. **Custom DNS servers on the VNet** (e.g., on-prem AD DNS) that don't forward to Azure DNS **168.63.129.16** - private endpoints and private zones won't resolve consistently. Fix: conditional forwarders or **Azure DNS Private Resolver**.
2. **Private DNS zone not linked** to every VNet (hub/spoke), or duplicate zones with different records -> some VMs get public IP, others private.
3. **Stale caching / TTL**: during deployments resources are recreated (new IPs, slot swaps), and clients or intermediate resolvers keep old answers until TTL expires.
4. **One of multiple DNS servers is unhealthy** - clients rotate between them, so failures look random.
5. **AKS-specific**: CoreDNS pods overloaded or scaled too low; `ndots:5` causing many extra lookups; the known **UDP conntrack race** (fix with NodeLocal DNSCache); SNAT port exhaustion on outbound.
6. **Throttling**: Azure DNS limits queries per VM (around 1000 qps per VM to 168.63.129.16) - chatty apps hit it.
7. **NSG/firewall** blocking UDP/TCP 53 to some DNS servers, or Azure Firewall DNS proxy misconfigured.
8. **Propagation delays** after creating/changing public DNS records.

How to debug:

- Test from the failing host repeatedly: `nslookup` / `dig` against each DNS server; compare results.
- Check which DNS server the VNet/VM uses; check private zone links.
- In AKS, check CoreDNS logs and metrics.

Example:

```
for i in $(seq 1 20); do dig +short myapp.privatelink.database.windows.net; done
az network vnet show -g net-rg -n spoke-vnet --query dhcpOptions.dnsServers
az network private-dns link vnet list -g dns-rg -z privatelink.database.windows.net -o table
kubectl -n kube-system logs -l k8s-app=kube-dns --tail 50
```

What to say to the interviewer: *I would check whether custom DNS servers forward to 168.63.129.16, whether the private DNS zones are linked to every VNet, TTL caching after resources were recreated, an unhealthy DNS server in the rotation, and in AKS CoreDNS load, ndots and the conntrack race. I prove it by querying each DNS server repeatedly from the failing host and comparing answers.*

Q27. Azure: An Azure Function is being throttled. How will you detect and fix it?

Also asked as: An Azure Function has frequent timeouts. How do you troubleshoot it?

In simple words: The function is a toll booth. Throttling means too many cars are arriving - either the booth itself limits cars, or the road after it (a database or API) is jammed and sends cars back.

Detailed answer

Detect

- **Application Insights**: HTTP **429** responses, failed invocations, increased duration, dependency failures.
- **Azure Monitor metrics**: function execution count, instance count, queue length growing (for queue/Event Hub triggers).
- Host logs showing the host is at max concurrency, or downstream **429 from Cosmos DB/Storage/external API**.
- Check **plan limits**: Consumption plan has cold starts and a maximum instance count; one instance handles limited concurrency.

Fix (depending on the cause)

1. **Plan limits**: move to **Premium (Elastic Premium)** or **Flex Consumption** for pre-warmed instances, higher scale-out and VNet support; or raise `functionAppScaleLimit` if it was set low.
2. **Concurrency tuning** in `host.json`: `maxConcurrentRequests` for HTTP, `batchSize`/`maxConcurrentCalls` for queue/Service Bus - to either use capacity better or protect downstream.
3. **Downstream throttling**: add retries with exponential backoff, increase Cosmos DB RU/s or enable autoscale, cache results, batch writes.
4. **Buffer bursts**: put a queue (Service Bus/Storage Queue) in front so spikes are absorbed and processed at a steady rate.
5. **Front door rate limits**: if API Management is in front, check its rate-limit policy.
6. Reduce execution time (reuse HTTP/DB clients, avoid static-per-call connections).

If the symptom is timeouts rather than 429s:

- **functionTimeout per plan**: Consumption defaults to 5 minutes (max 10), while Premium and Dedicated default to 30 minutes and can be unbounded. Set it in `host.json`.
- **HTTP triggers have a hard limit of about 230 seconds** (the front-end load balancer idle timeout), whatever functionTimeout says. For long work, return 202 quickly and use the **Durable Functions async HTTP pattern** or hand off to a queue.
- **Cold starts** on Consumption: use Premium/Flex always-ready instances, or keep the package small.
- **Outbound connection / SNAT port exhaustion**: creating a new `HttpClient` or DB connection per call exhausts ports, so reuse static clients and use connection pooling.
- **Slow downstream**: find it with **Application Insights dependency tracking** (duration per dependency, failed calls), then add timeouts, retries and caching.

Example:

```
{
  "version": "2.0",
  "functionTimeout": "00:10:00",
  "extensions": {
    "http": { "maxConcurrentRequests": 100, "maxOutstandingRequests": 200 },
    "serviceBus": { "maxConcurrentCalls": 32 }
  },
  "retry": { "strategy": "exponentialBackoff", "maxRetryCount": 5,
             "minimumInterval": "00:00:02", "maximumInterval": "00:00:30" }
}
```

What to say to the interviewer: *I detect it in Application Insights and Azure Monitor - 429s, rising duration, growing queue length - and then identify whether the limit is the plan, host concurrency or a throttled downstream like Cosmos DB. Fixes are moving to Premium or Flex Consumption, tuning host.json concurrency, backoff retries, more downstream capacity, and a queue to buffer bursts. For timeouts I check functionTimeout for the plan, the 230-second HTTP limit (moving long work to Durable Functions), cold starts, SNAT exhaustion from not reusing HttpClient, and slow dependencies in Application Insights.*

Q28. Azure: Design a cost-optimized cloud architecture for an internal reporting app that runs every night and stores logs for 3 years.

In simple words: Don't rent a full-time taxi for a trip you take once a night. Book a cab only for the ride (serverless/spot), and put old files in a cheap basement storeroom (archive tier) instead of the expensive front shelf.

Detailed answer

Design principles: **pay only while running**, **tier storage by age**, **no always-on servers**.

Piece	Azure	AWS equivalent
Nightly trigger	Logic Apps / Function timer / Container Apps Job (cron)	EventBridge Scheduler
Compute for the job	Container Apps Job or Azure Batch on **Spot VMs**	Fargate Spot / AWS Batch on Spot / Lambda
Source data	Read replica or export, so prod DB is not loaded	RDS read replica / snapshot export
Report output	Blob Storage (Hot for 30 days)	S3 Standard
Log retention 3 yrs	Log Analytics short retention (30-90 days) + export to Blob; lifecycle Cool -> Cold -> Archive	CloudWatch 30-90 days -> S3; lifecycle to Glacier Deep Archive
Access for users	Power BI / static page with Entra ID auth	QuickSight / S3 + CloudFront

Cost levers:

1. Compute scales to **zero** between runs; Spot for batch (retry if evicted).
2. **Lifecycle management**: Hot 30 days -> Cool 90 days -> Archive until 3 years -> delete. Archive tier is a tiny fraction of hot storage cost (remember retrieval takes hours - fine for audit logs).

3. Keep **Log Analytics** interactive retention short; long-term logs as compressed files in storage, not in the expensive analytics store.
4. Run in one region with **LRS/ZRS** (not GRS) unless compliance requires otherwise; use **immutability policy** if logs are for audit.
5. Budgets and cost alerts; tag everything.

Example:

```
{
  "rules": [{
    "name": "logs-3yr",
    "enabled": true,
    "type": "Lifecycle",
    "definition": {
      "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["logs/"] },
      "actions": { "baseBlob": {
        "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
        "tierToArchive": { "daysAfterModificationGreaterThan": 120 },
        "delete":        { "daysAfterModificationGreaterThan": 1095 } } }
    }
  }]
}
```

> ***What to say to the interviewer:*** *I would run the nightly job as a scheduled Container Apps Job or Batch on Spot that scales to zero, read from a replica, write reports to Blob, and keep logs short-term in Log Analytics while exporting to Blob with a lifecycle policy moving to Cool, then Archive, and deleting after 3 years. On AWS that is EventBridge plus Fargate Spot and S3 lifecycle to Glacier Deep Archive.*

Q29. Azure: The production app works fine for internal users but external users get 403 errors. How will you isolate the issue?

> **In simple words:** Staff enter through the back door and walk straight in; visitors come through the front door where the guard is turning them away. So look at the front-door guard, not the building.

Detailed answer

403 means the request **reached something that refused it** (not a network timeout). The difference between internal and external users is the **path**, so compare paths:

1. **Map both paths**: internal users may hit the private IP / internal DNS directly; external users go through **Front Door / App Gateway WAF / API Management / public DNS**.
2. **Check the WAF logs first** - the most common cause. A managed rule (e.g., SQLi false positive), **geo-filter**, bot rule, or IP rule blocking. Query `AzureDiagnostics` for `ApplicationGatewayFirewallLog` / Front Door WAF logs with `action == Block`.
3. **App Service Access Restrictions** / Azure Functions IP restrictions allowing only internal ranges, or requiring the Front Door header `X-Azure-FDID` - requests bypassing Front Door get 403.
4. **Storage/Key Vault firewall** or **private endpoint only** settings - external calls to a public endpoint get 403.
5. **Authentication**: Entra ID **Conditional Access** (trusted locations/compliant device), Easy Auth, or the app rejecting tokens from external tenants.

6. **Host header / CORS / hostname** issues: external domain not in allowed hosts, or origin checks in the app.
7. Reproduce: `curl -v` from outside (mobile hotspot or cloud shell) vs inside, compare response headers (Front Door, App Gateway and App Service return different 403 pages/headers) to pinpoint the layer.

Fix: tune/exclude the WAF rule (never just disable the WAF), correct access restriction rules, or update Conditional Access - then add a synthetic external check so it is caught earlier.

Example:

```
AzureDiagnostics
| where Category == "ApplicationGatewayFirewallLog" and action_s == "Blocked"
| summarize count() by ruleId_s, Message, clientIp_s, requestUri_s
| order by count_ desc

az webapp config access-restriction show -g prod-rg -n bank-web
```

> ***What to say to the interviewer:*** *Since internal users succeed, I compare the request paths and look at the layers only external traffic passes through - first the WAF logs on Front Door or App Gateway, then App Service access restrictions, storage or Key Vault firewalls and Conditional Access. I reproduce with curl from outside, read which layer returned the 403 from the headers, and fix that rule rather than disabling protection.*

Q30. Security: A service account key/credential leaked in a public container image. How fast can you isolate and rotate it, and how do you prevent it from happening again?

> **In simple words:** Your house key was left in a box posted on the internet. Taking the box down doesn't help - someone may already have copied the key. Change the lock immediately, check the CCTV for who came in, then stop putting keys in boxes.

Detailed answer

Golden rule: **assume the credential is compromised forever**. Deleting the image from the registry does not un-leak it - anyone could have pulled or cached it.

1. **Contain (first 5-15 minutes)**: disable or delete the leaked key right away - GCP service account key, AWS access key, or K8s service account token. Don't wait until a replacement is ready; a short outage is better than an attacker with access.
2. **Remove the exposure**: make the repository or image tag private, delete the affected tags and layers, and ask any mirror or cache to purge them.
3. **Rotate and restore**: create a new credential (ideally switch to keyless, see step 6), store it in a secret manager, and redeploy the consumers so they read it from there.
4. **Investigate**: search **Cloud Audit Logs / CloudTrail** for any use of the leaked key ID - which source IPs, which API calls, and any new resources (VMs, keys, IAM bindings, crypto miners). Check the **blast radius** of that identity's permissions and revoke anything an attacker created, such as new keys or users.
5. **Hunt for more leaks**: scan other images and the git history - `trivy image --scanners secret`, `gitleaks`, `trufflehog`. Rewrite git history if needed, but still rotate.
6. **Prevent**:

- Go keyless: **GKE Workload Identity**, **IRSA/EKS Pod Identity**, and **OIDC federation for CI** (GitHub Actions to AWS/GCP), so there is no key file to leak.
- Keep secrets out of the build: BuildKit `--mount=type=secret`, a proper `.dockerignore`, multi-stage builds.
- Add secret scanning in pre-commit and CI that blocks the push or build.
- Set the org policy `iam.disableServiceAccountKeyCreation` (and an SCP or alerts on `CreateAccessKey` in AWS).
- Least privilege, so the next leak has a small blast radius.

7. Write a blameless postmortem with owners and dates for each action.

Time	Action
0-15 min	Disable/delete key, make image private, alert security
15-60 min	New credential in secret manager, redeploy consumers
1-4 hrs	Audit log review, revoke attacker-created resources
Same day	Scan all images and repos, purge history
1-2 weeks	Keyless identity, CI secret scanning, org policy, postmortem

Example:

```
# GCP: disable then delete the leaked key
gcloud iam service-accounts keys disable KEY_ID --iam-account app@my-proj.iam.gserviceaccount.com
gcloud iam service-accounts keys delete KEY_ID --iam-account app@my-proj.iam.gserviceaccount.com
gcloud logging read 'protoPayload.authenticationInfo.serviceAccountKeyName:"KEY_ID"' --freshness 30d

# AWS: deactivate the key and see where it was used
aws iam update-access-key --user-name ci-bot --access-key-id AKIA... --status Inactive
aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIA...

# Hunt for other leaks
trivy image --scanners secret myorg/app:latest
gitleaks detect --source . --log-opts="--all"

# Build without baking secrets into layers
docker build --secret id=npmrc,src=$HOME/.npmrc .   # Dockerfile: RUN --mount=type=secret,id=npmrc .
    ..
```

> ***What to say to the interviewer:*** *Within minutes I disable the leaked key without waiting for a replacement, make the image private and treat the key as permanently compromised. Then I rotate to a new credential held in a secret manager, check CloudTrail or Cloud Audit Logs for any misuse and revoke what the attacker created, and scan other images and repos. To prevent a repeat I move to keyless workload identity and OIDC in CI, use BuildKit secret mounts, block pushes with secret scanning, and disable service account key creation by org policy.*

Q31. Azure: How would you reduce cost in an over-provisioned Azure environment?

> **In simple words:** You're paying for a 10-room house but live in 3 rooms with all the lights on. Find the empty rooms, switch off lights when you leave, and sign a yearly lease for the rooms you do use to get a discount.

Detailed answer

Step by step:

1. **See the spend**:

- **Cost Management + Billing**: cost by resource group, service and tag.
- **Azure Advisor** cost recommendations.
- Enforce **tags** (owner, env, app) with Azure Policy so every cost has an owner.

2. **Delete waste (quick wins)**:

- Unattached **managed disks**, old snapshots, idle **public IPs**, orphaned **NICs**.
- Empty App Service plans, idle load balancers/gateways, forgotten test resources.

3. **Rightsize**:

- Downsize VMs and App Service plans with low CPU/memory (Advisor flags under ~5% CPU).
- Move to newer, cheaper VM series.
- Scale SQL/Cosmos RU/s to the real load.

4. **Scale with demand**:

- Autoscale App Service/VMSS; set the minimum to what HA needs, not peak.
- **AKS**: cluster autoscaler per node pool, a **Spot node pool** for batch/stateless workloads, and rightsized pod requests.

5. **Switch off when unused**: **auto-shutdown** or start/stop schedules for dev/test VMs, and scale non-prod AKS to zero at night.

6. **Commit for steady usage**: **Reservations** (1 or 3 years) or **Savings Plans** for the baseline, **Azure Hybrid Benefit** for existing Windows/SQL licenses, and **Spot VMs** for interruptible work.

7. **Storage and logs**:

- Lifecycle policies Hot -> Cool -> Cold -> Archive.
- Right redundancy: LRS for non-critical data instead of GRS.
- **Log Analytics**: daily ingestion caps, the Basic logs tier for noisy tables, shorter interactive retention, and only the needed diagnostic categories.

8. **Govern**: **budgets with alerts** per subscription/team, a monthly FinOps review, and cost shown in dashboards so teams see their own numbers.

Always check performance after each change (latency, errors). Saving money must not break SLOs.

Example:

```
az advisor recommendation list --category Cost -o table
az disk list --query "[?diskState=='Unattached'].{name:name,rg:resourceGroup,gb:diskSizeGb}" -o tabl
    e
az network public-ip list --query "[?ipConfiguration==null].{name:name,rg:resourceGroup}" -o table
az vm auto-shutdown -g dev-rg -n dev-vm01 --time 1930
az consumption budget create --budget-name team-a --amount 2000 --time-grain Monthly \
  --category Cost --start-date 2026-10-01 --end-date 2027-09-30
```

> ***What to say to the interviewer:*** *I start with Cost Management and Advisor, with tags enforced so every cost has an owner, then delete orphaned disks, IPs and NICs, rightsize VMs and App Service plans, and add autoscale and AKS spot node pools. Dev/test gets auto-shutdown, the steady baseline gets reservations or savings plans, storage is tiered with lifecycle rules, and Log Analytics ingestion is capped. Budgets and monthly reviews keep it from creeping back, and I verify SLOs*

after each change.

Q32. Security: An image passed all vulnerability scans but still got exploited in production. What did you miss?

In simple words: A scanner is like checking a house against a list of known faulty locks. It can't tell you the window was left open, that a new lock flaw was discovered yesterday, or that the burglar was let in by a guest. A clean scan means "no known bad parts", not "safe".

Detailed answer

Why scans aren't enough: image scanners (Trivy, Grype, ECR/ACR scanning) only match **known CVEs in packages**. They miss **zero-days**, **application logic flaws** (auth bypass, SSRF, injection in your own code), **misconfigurations** and often **secrets**.

Common gaps that let the attack succeed:

1. **Over-privileged pod**: running as root, `privileged: true`, `hostPath` mounts, added capabilities (`NET_ADMIN`, `SYS_ADMIN`), or `hostNetwork`. The attacker can break out to the node.
2. **Default service account token automounted** while the RBAC is too broad. The attacker calls the Kubernetes API, reads secrets or creates pods.
3. **Writable root filesystem**. The attacker downloads and runs a dropper or crypto-miner.
4. **No seccomp/AppArmor profile**, so every syscall is allowed.
5. **Secrets in env vars or image layers**, which the attacker reads straight away.
6. **Open network**: no NetworkPolicy and no egress filtering, so the malware calls back to its C2 server and data leaves freely.
7. **Dependencies pulled at runtime** (`pip install` or `curl | sh` at startup), so the scanned image isn't what actually runs.
8. **Stale scan results**: the image was clean on build day, and a CVE was published later. Rescan the registry and running images continuously.
9. **Unsigned images or mutable tags**: `:latest` was overwritten, so what's running isn't what was scanned.
10. **No runtime detection**: nothing alerted on a shell spawned in a container.

Incident response:

1. **Isolate**: apply a deny-all NetworkPolicy to the pod, cordon the node, and don't delete anything yet.
2. **Capture forensics**: pod logs, `kubectl describe`, a container filesystem snapshot or node disk snapshot, K8s audit logs, cloud audit logs.
3. **Rotate every credential** the pod could reach: SA tokens, DB passwords, cloud keys.
4. **Rebuild** from a clean base, patch the root cause, redeploy with hardened settings, and replace the node.
5. Hold a postmortem and add the missing controls below.

Layer	Control	Tools
Code	SAST, dependency pinning, DAST	SonarQube, Semgrep, OWASP ZAP

Layer	Control	Tools
Build	Image scan, secret scan, minimal/distroless base	Trivy, Grype, Gitleaks
Supply chain	Sign images, deploy by digest, verify at admission	cosign, Kyverno, Binary Authorization
Admission	Restricted Pod Security Standard, policy as code	Pod Security Admission, Gatekeeper, Kyverno
Pod runtime	Non-root, read-only root FS, drop ALL caps, seccomp RuntimeDefault, no SA token	securityContext, automountServiceAccountToken: false
Network	Default-deny ingress and egress, egress allowlist	NetworkPolicy, Cilium/Calico, egress firewall
Detection	Runtime threat detection, continuous rescans	Falco, GuardDuty EKS Runtime, Defender for Containers
Identity	Least-privilege RBAC and workload identity	RBAC, IRSA, Workload Identity

Example:

```
# Hardened pod settings the scan could never have checked
spec:
  automountServiceAccountToken: false
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: api
      image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/api@sha256:3f1c...
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]

# Contain first: cut the pod's network, keep it for forensics
kubectl label pod api-7d9f compromised=true -n prod
kubectl apply -f deny-all-for-compromised.yaml
kubectl cordon ip-10-0-3-17.ap-south-1.compute.internal
```

> ***What to say to the interviewer:*** *A clean scan only means no known CVEs in packages; it doesn't cover zero-days, app logic flaws, misconfiguration or CVEs published after the scan. The usual gaps are a root or privileged pod with a broad default service account token, a writable filesystem, no seccomp, secrets in env vars, no egress NetworkPolicy, and no runtime detection like Falco or GuardDuty. I'd isolate the pod and node, capture forensics, rotate every reachable credential and rebuild, then close the gaps with signed digests, the restricted Pod Security Standard, default-deny networking and continuous rescans.*

Monitoring, Observability, SRE & Scale Design

Basic

Q1. Which monitoring and logging tools have you worked with (Prometheus, Grafana, ELK, Loki, Datadog)?

Also asked as: What monitoring tools have you used? Have you worked with Prometheus, Grafana or ELK Stack?

In simple words: Monitoring tools are the dashboard and warning lights of a car. Prometheus measures the numbers (speed, fuel), Loki/ELK keeps the diary of what happened, and Grafana is the screen where you look at all of it.

Detailed answer

Group the tools by the job they do, so the answer sounds organized instead of a list of names:

Job	Common tools	What it gives you
Metrics	Prometheus, CloudWatch, Zabbix, Datadog	Numbers over time: CPU, latency, error rate
Logs	Loki, ELK/EFK (Elasticsearch + Fluentd/Fluent Bit + Kibana), CloudWatch Logs	Text events from apps and systems
Traces	Tempo, Jaeger, OpenTelemetry, X-Ray	The path of one request across services
Visualization	Grafana, Kibana	Dashboards and exploration
Alerting	Alertmanager, Grafana Alerting, Zabbix triggers, CloudWatch Alarms	Notifications to Teams/Slack/email/pager

- **Prometheus** pulls (scrapes) metrics from `/metrics` endpoints every 15-30 seconds and stores them as time series.
- **Loki** stores logs cheaply by indexing only labels (namespace, pod, app), not the full text. ELK indexes everything, so it searches faster but costs more.
- **Grafana** can read all of them in one place.
- **Docker containers** (outside Kubernetes) are monitored with **cAdvisor**, which exposes per-container CPU, memory, network and filesystem metrics on `/metrics` for Prometheus to scrape. `docker stats` gives a quick live view. Inside Kubernetes, cAdvisor is already built into the kubelet.

What to say to the interviewer: *I have worked with Prometheus and Alertmanager for Kubernetes metrics and alerts, Zabbix for host, service and SLA monitoring, Loki for centralized logs, and Grafana as the single dashboard layer over all of them. I think of it as metrics for "is something wrong", logs for "what exactly happened" and traces for "where in the request path it happened".*

Q2. What is observability, and how is it different from monitoring?

Also asked as: What does observability mean to you, and how do you achieve it? Logs are not observability - can you debug user pain?

In simple words: Monitoring is a smoke alarm - it tells you there is a fire in the kitchen. Observability is being able to walk in and figure out which pan caught fire and why, even if nobody predicted that exact fire.

Detailed answer

- **Monitoring** = watching known things with pre-defined checks and thresholds. "Is CPU above 90%?" "Is the site returning 200?" It answers questions you already thought of.
- **Observability** = how well you can understand the internal state of a system from the data it produces, so you can **ask new questions without shipping new code**. "Why are only Android users in Mumbai on app version 5.2 seeing slow checkout since 10:05?"
- **Having logs is not observability.** A pile of unlinked logs cannot tell you which users are hurting or where time is spent. Observability means you can go from a user's pain to the exact cause.

What makes a system observable:

- **High-cardinality context** - every signal carries labels or attributes such as tenant, region, app version, endpoint, and (in traces/logs, not metric labels) user or request ID, so you can slice data many ways.
- **Correlation across signals** - metrics -> traces through **exemplars** (a latency point links to a real trace), traces -> logs through a shared `trace_id`, and the same service/namespace labels everywhere.
- **User-perspective SLOs** - measure what users feel with real-user monitoring (RUM) and synthetic checks, not only server CPU. "99% of checkouts complete in under 2s" beats "pods are Running".

How I would get there:

1. **Instrument with OpenTelemetry** - one standard for traces, metrics and logs, with auto-instrumentation where possible.
2. **Structured JSON logs** that include `trace_id`, service, version and tenant.
3. **A unified stack** - for example Prometheus/Mimir for metrics, Loki for logs, Tempo for traces, all in Grafana with exemplars and trace-to-logs links turned on.
4. **SLOs and dashboards per user journey**, with burn-rate alerts.

Example: A monitoring alert says "p99 latency > 2s". With observability you click the Grafana latency panel -> open the exemplar trace -> see the slow span is a DB query -> jump to the logs for that `trace_id` and see a lock timeout -> filter by `tenant` and find that only one large customer is affected.

What to say to the interviewer: *Monitoring tells me when something known is broken; observability lets me ask new questions about why, without shipping new code. For me that means high-cardinality context on every signal, metrics, traces and logs correlated through exemplars and trace IDs, and SLOs measured from the user's side with RUM and synthetics. I get there with OpenTelemetry instrumentation, structured logs and one Grafana stack over Prometheus, Loki and Tempo, so I can go from user pain to the exact failing request.*

Q3. What is the difference between logs, metrics and traces?

In simple words: Metrics are your heart rate on a fitness watch, logs are a diary of everything you did today, and a trace is a GPS route of one single trip from home to office showing where you got stuck.

Detailed answer

	Metrics	Logs	Traces
What	Numbers over time	Timestamped text events	Path of one request across services
Example	`http_requests_total`, CPU 72%	`ERROR payment failed: timeout`	API -> auth -> orders -> DB, 1.8s total
Cost	Cheap, small	Expensive at volume	Medium, usually sampled
Best for	Alerting, trends, dashboards	Root-cause details	Finding which hop is slow
Tools	Prometheus, CloudWatch	Loki, ELK	Tempo, Jaeger, X-Ray

A common line: "Metrics show trends, logs tell stories, traces reveal the truth of a single request."

Example:

```
Metric: rate(http_requests_total{status=~"5.."}[5m])  -> 5xx rate jumped
Log:    {app="orders"} |= "ERROR"                        -> "DB connection refused"
Trace:  trace_id=ab12 -> orders span 4.9s -> db.query span 4.8s
```

What to say to the interviewer: *Metrics are aggregated numbers I alert on, logs are detailed events I read to understand what happened, and traces follow one request across microservices to show which hop is slow. I usually start from a metric alert, use traces to find the slow service, and logs to find the exact error.*

Q4. What is log aggregation, and why do we need it?

In simple words: Instead of visiting 200 shops to read each one's complaint book, you photocopy every complaint book into one central library where you can search them all at once.

Detailed answer

In Kubernetes, pods are temporary. When a pod dies, its local logs go with it, and with 100 pods you cannot `kubectl logs` each one. Log aggregation means:

1. **Collect** - an agent on every node (Fluent Bit, Promtail, Grafana Alloy, Fluentd, Vector) tails container logs from `/var/log/containers`.
2. **Enrich** - adds labels like namespace, pod, container, node.
3. **Ship** - sends to a central store (Loki, Elasticsearch, CloudWatch Logs).
4. **Store & retain** - with retention rules (e.g. 7 days hot, 90 days in S3).
5. **Search & alert** - query in Grafana/Kibana, alert on error patterns.

Example: LogQL query in Grafana for errors in one namespace:

```
{namespace="payments", app="checkout"} |= "ERROR" | json | status >= 500
```

What to say to the interviewer: *Log aggregation collects logs from every server and pod into one central, searchable place so logs survive pod restarts and I can search across all services at once. A typical setup is a node-level agent like Fluent Bit or Promtail shipping into Loki or Elasticsearch, viewed in Grafana or Kibana.*

Q5. What are SLI, SLO, SLA and error budget?

In simple words: A pizza shop measures how long delivery takes (SLI), sets an internal goal of 95% under 30 minutes (SLO), and promises customers a free pizza if it is over 45 minutes (SLA). The 5% of "late allowed" deliveries is the error budget.

Detailed answer

- **SLI (Service Level Indicator)** - the actual measurement. Example: percentage of requests that return success in under 300 ms.
- **SLO (Service Level Objective)** - the internal target for the SLI. Example: 99.9% of requests successful over 30 days.
- **SLA (Service Level Agreement)** - the external contract with penalties. Usually looser than the SLO so you get warned before breaking a contract.
- **Error budget** = 100% - SLO. For 99.9% over 30 days that is about 43 minutes of allowed "badness". If the budget is burning fast, you slow down risky releases and focus on reliability.

Availability	Downtime per 30 days
99%	~7.2 hours
99.9%	~43 minutes
99.99%	~4.3 minutes

Example: PromQL SLI for availability:

```
sum(rate(http_requests_total{status!~"5.."}[30d]))
/ sum(rate(http_requests_total[30d]))
```

What to say to the interviewer: *The SLI is what I measure, the SLO is the internal target, and the SLA is the customer contract with penalties. The error budget is the allowed failure left over, and it becomes the rule for when we ship fast versus when we freeze features and fix reliability.*

Q6. How does Prometheus collect data from applications in Kubernetes?

In simple words: Prometheus is a meter reader who walks door to door every 30 seconds and reads each house's meter. Kubernetes gives it the updated address list, so new houses get read automatically.

Detailed answer

1. Apps expose metrics in text format on an HTTP endpoint, usually `/metrics` (via a client library, or an **exporter** for things that cannot, e.g. node-exporter for hosts, postgres-exporter, kafka-exporter).
2. Prometheus uses **Kubernetes service discovery** to find pods/services/endpoints through the API server.

3. It **pulls (scrapes)** each target on an interval and stores samples in its time-series DB.
4. With the Prometheus Operator, you declare targets using a **ServiceMonitor** or **PodMonitor** instead of editing config files.
5. Short-lived jobs that end before being scraped push to a **Pushgateway**.
6. kube-state-metrics exposes object state (deployment replicas, pod phase), and cAdvisor in the kubelet exposes container CPU/memory.

Example:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: orders
  labels: { release: kps }      # must match the Prometheus selector
spec:
  selector:
    matchLabels: { app: orders }
  endpoints:
    - port: http-metrics
      path: /metrics
      interval: 30s
```

What to say to the interviewer: *Prometheus uses a pull model - it discovers targets through the Kubernetes API and scrapes their /metrics endpoints on an interval. With the Prometheus Operator I just create a ServiceMonitor that selects the service by label, and exporters like node-exporter and kube-state-metrics cover infrastructure and cluster state.*

Q7. How do you monitor Kubernetes clusters and applications, and how do you integrate Prometheus and Grafana with Kubernetes?

Also asked as: How do you integrate monitoring tools like Prometheus and Grafana with Kubernetes? How do you monitor applications with Prometheus + Grafana (basic setup)?

In simple words: Instead of building a hospital monitoring room piece by piece, you install a ready-made kit that comes with the heart monitors, the screens and the alarm bells already wired together.

Detailed answer

The standard way is the **kube-prometheus-stack** Helm chart. One install gives you:

- Prometheus Operator + Prometheus server
- Alertmanager
- Grafana with ready dashboards
- node-exporter (node CPU/mem/disk) and kube-state-metrics (object state)
- Default alert rules (KubePodCrashLooping, KubeNodeNotReady, etc.)

What to monitor, in layers:

- **Cluster/nodes** - CPU, memory, disk, pressure conditions, NotReady nodes.
- **Kubernetes objects** - restarts, pending pods, failed deployments, HPA at max.
- **Applications** - the RED method: Rate, Errors, Duration per service.
- **Logs** - Loki + Promtail/Alloy; **traces** - Tempo/Jaeger with OpenTelemetry.

Example:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kps prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
kubectl -n monitoring port-forward svc/kps-grafana 3000:80
```

Basic setup end to end, in 5 steps:

1. **Install the stack** - `helm install kps ...` as above (Prometheus, Alertmanager, Grafana ready).
2. **App exposes metrics** - add a Prometheus client library so the app serves `/metrics` (e.g. `http_requests_total`, `http_request_duration_seconds`), and name the port in the Service (e.g. `http-metrics`).
3. **Tell Prometheus to scrape it** - create a ServiceMonitor (see Q6) with the `release: kps` label; confirm the target is UP under Status -> Targets in the Prometheus UI.
4. **Grafana** - the Prometheus data source is pre-configured by the chart; build a RED dashboard (rate, errors, duration) or import a community dashboard by ID.
5. **Alert rule** - add a PrometheusRule; Alertmanager routes it to Teams/Slack/email.

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: orders-alerts
  labels: { release: kps }
spec:
  groups:
    - name: orders
      rules:
        - alert: OrdersHigh5xx
          expr: |
            sum(rate(http_requests_total{app="orders",status=~"5.."}[5m]))
            / sum(rate(http_requests_total{app="orders"}[5m])) > 0.05
          for: 5m
          labels: { severity: critical }
          annotations: { summary: "orders 5xx above 5% for 5 minutes" }
```

What to say to the interviewer: *I install kube-prometheus-stack with Helm, which brings Prometheus, Alertmanager, Grafana, node-exporter and kube-state-metrics with default dashboards and rules. Then I add ServiceMonitors for each app, Loki for logs, and route Alertmanager to Teams or Slack by severity.*

Q8. What is a runbook, and what does on-call ownership mean?

In simple words: A runbook is the "what to do if the fire alarm rings" card on the wall, so even a new person at 3 AM knows the exact steps without guessing.

Detailed answer

- A **runbook** is a step-by-step guide linked to an alert: what the alert means, how to check impact, commands to diagnose, how to fix or mitigate, and who to escalate to.
- Every page-worthy alert should have a runbook link in its annotations. If an alert has no clear action, it probably should not page anyone.

- **On-call ownership** means a rotating engineer is responsible for acknowledging alerts within an agreed time, mitigating, communicating and handing over cleanly. Good on-call has: a rotation schedule, escalation policy, primary + secondary, and a handover note.

Example: Alert rule with a runbook link:

```
- alert: KafkaConsumerLagHigh
  expr: sum by (group) (kafka_consumergroup_lag) > 50000
  for: 10m
  labels: { severity: critical }
  annotations:
    summary: "Consumer group {{ $labels.group }} lag is high"
    runbook_url: "https://wiki.example.com/runbooks/kafka-lag"
```

> ***What to say to the interviewer:*** *A runbook is a documented, step-by-step response for a specific alert, and I link it directly in the alert annotation. On-call ownership means someone is clearly accountable to acknowledge, mitigate, communicate and hand over, backed by a rotation and escalation policy.*

Q9. What is FinOps? What are the basics of cloud cost optimization?

> **In simple words:** FinOps is running cloud spending like a household budget: everyone can see the bill, each family member knows what they spend, and you stop paying for the gym you never visit.

Detailed answer

FinOps = shared responsibility for cloud cost between engineering, finance and business. The basic loop is **Inform -> Optimize -> Operate**.

- **Visibility** - tag every resource (team, env, project), use Cost Explorer / budgets / cost anomaly alerts.
- **Rightsizing** - match instance size and pod requests to real usage.
- **Pricing models** - Savings Plans/Reserved Instances for steady load, Spot for fault-tolerant load.
- **Waste removal** - unattached volumes, orphaned PVCs, idle load balancers, old snapshots, forgotten S3 buckets.
- **Storage tiering** - S3 lifecycle to Infrequent Access / Glacier; shorter log retention.
- **Data transfer** - NAT gateway and cross-AZ traffic are often the hidden cost; VPC endpoints help.

> ***What to say to the interviewer:*** *FinOps means making cloud cost visible and owned by the teams who create it, then continuously optimizing. My basics are tagging and budgets for visibility, rightsizing and Savings Plans or Spot for compute, lifecycle policies for storage, and regular audits for orphaned resources like unattached volumes and PVCs.*

Advanced

Q10. How do you implement centralized logging for microservices?

> **In simple words:** Every microservice writes in the same format and drops its letters into one shared post box, so anyone can search all letters by sender, date or tracking number.

Detailed answer

Design points:

1. **Standard log format** - JSON logs to stdout with fields: `timestamp`, `level`, `service`, `trace_id`, `request_id`, `message`. Do not write to files inside containers.
2. **Collector per node** - Fluent Bit / Promtail / Grafana Alloy as a DaemonSet reads `/var/log/containers/*.log` and adds Kubernetes labels.
3. **Central store** - Loki (cheap, label-indexed, object storage like S3) or Elasticsearch/OpenSearch (full-text search, heavier). CloudWatch Logs if you want fully managed on AWS.
4. **Keep labels low-cardinality** - namespace, app, env, level. Never put user IDs or request IDs as Loki labels; keep them inside the log line.
5. **Correlation** - include `trace_id` so Grafana can jump log -> trace.
6. **Retention & cost** - e.g. 7-30 days hot, then archive to S3; drop noisy debug logs at the agent.
7. **Security** - mask PII/secrets before shipping; RBAC on who can query which namespace.
8. **Alerting on logs** - Loki ruler or ElastAlert for patterns like sudden "OutOfMemory" or 5xx bursts.

Example:

```
Pods (stdout JSON) -> Fluent Bit DaemonSet -> Loki (S3 backend) -> Grafana
                                          \-> Loki ruler -> Alertmanager -> Teams
```

> ***What to say to the interviewer:*** *I standardize JSON logs to stdout with a trace ID, run a collector like Fluent Bit or Promtail as a DaemonSet, and ship into Loki backed by S3 or into OpenSearch. I keep labels low-cardinality, set retention and PII masking, and link logs to traces in Grafana.*

Q11. Design a highly available logging system for 100+ microservices across 3 regions.

> **In simple words:** Each region has its own local post office so mail never waits on a long-distance line, and there is one central search counter that can look into all three post offices.

Detailed answer

Architecture:

```
Region A/B/C:
  Pods -> Fluent Bit (DaemonSet, disk buffer)
       -> Kafka / Kinesis (regional buffer, optional)
       -> Loki (distributed mode, 3 AZs, replication_factor=3)
       -> S3 bucket in same region (chunks + index)
Global:
  Grafana (HA, 2+ replicas) with 3 Loki data sources
  or query federation -> single "all regions" view
```

Key decisions:

- **Regional ingestion** - logs stay in-region (lower latency, lower transfer cost, data residency rules).
- **Buffering** - agents with disk buffers; optionally Kafka in front so a Loki outage does not drop logs.
- **HA inside region** - Loki ingesters spread across 3 AZs with replication factor 3; stateless queriers/distributors behind a load balancer; object storage (S3) is durable by design.
- **Cross-region DR** - S3 cross-region replication of critical buckets.
- **Scale** - horizontal scaling of distributors/ingesters/queriers; rate limits per tenant so one noisy service cannot flood the cluster.
- **Multi-tenancy** - tenant per team or environment (`X-Scope-OrgID`).
- **Retention tiers** - short hot retention, long cold in S3 (with compliance needs).
- **Monitor the monitor** - alert on dropped logs, ingestion rate, agent errors.

Alternatives: OpenSearch with cross-cluster search, or a managed option (CloudWatch Logs, Datadog) if the team is small.

> ***What to say to the interviewer:*** *I would run a regional logging stack per region - Fluent Bit with disk buffers, optionally Kafka, then Loki in distributed mode across three AZs with S3 storage - and put a global HA Grafana on top that queries all three regions. That keeps ingestion local and resilient, uses tenant limits to stop noisy services, and adds cross-region replication only for logs we must keep for DR or compliance.*

Q12. How do you implement and monitor distributed tracing in microservices?

> **In simple words:** Tracing is like putting a tracking number on a parcel. Every warehouse it passes through scans it, so later you can see exactly which warehouse held it for three hours.

Detailed answer

1. **Instrument** services with **OpenTelemetry** SDKs or auto-instrumentation (Java agent, Python/Node auto-instrumentation) or eBPF tools like Grafana Beyla. Each request gets a `trace_id`; each hop is a **span**.
2. **Propagate context** through HTTP headers (W3C `traceparent`) and message headers (Kafka), so the trace does not break between services.
3. **Collect** with an **OpenTelemetry Collector** (agent per node + gateway) that batches, samples and exports.
4. **Store & view** in Tempo, Jaeger, or AWS X-Ray; view in Grafana.
5. **Sampling** - head sampling (keep 5-10%) or tail sampling (keep all errors and slow traces, sample the rest) to control cost.

6. **Correlate** - put `trace_id` in logs; enable exemplars so a latency metric links to a real trace.
7. **Monitor tracing health** - spans received vs dropped in the collector; alert if span rate falls to zero (a silently broken pipeline is common).

Example:

```
# OTel Collector tail sampling: keep errors + slow requests
processors:
  tail_sampling:
    policies:
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      - name: slow
        type: latency
        latency: { threshold_ms: 1000 }
```

> ***What to say to the interviewer:*** *I instrument services with OpenTelemetry, propagate the trace context through HTTP and Kafka headers, and send spans through an OTel Collector to Tempo or Jaeger viewed in Grafana. I use tail sampling to keep all errors and slow traces, put the trace ID in logs, and alert if span ingestion drops, because a silently broken trace pipeline is a common failure.*

Q13. How would you design an observability stack using CloudWatch, Prometheus and Grafana?

> **In simple words:** CloudWatch already watches the building AWS owns (RDS, ALB, Lambda), Prometheus watches what you run inside your own rooms (pods, apps), and Grafana is the single security desk showing both camera feeds.

Detailed answer

Split responsibilities:

- **CloudWatch** - AWS managed services: ALB (TargetResponseTime, HTTPCode_ELB_5XX), RDS (CPU, connections, FreeStorageSpace, replica lag), Lambda (errors, throttles, duration), MSK, ElastiCache, billing. Also CloudWatch Logs for Lambda/VPC flow logs.
- **Prometheus** - Kubernetes and apps: kube-state-metrics, node-exporter, app `/metrics`, exporters. Use Amazon Managed Prometheus or self-managed with remote_write to long-term storage (Thanos/Mimir) if retention is needed.
- **Grafana** - one place: CloudWatch data source (via IAM role, not access keys), Prometheus data source, Loki for logs, Tempo/X-Ray for traces.
- **Alerting** - Alertmanager for cluster/app alerts, CloudWatch Alarms -> SNS -> Lambda/chat for AWS services; or unify with Grafana Alerting. Route by severity and team.
- **Dashboards** - per-service RED dashboard, per-cluster USE dashboard (Utilization, Saturation, Errors), business/SLO dashboard.

Example:

```
EKS pods --scrape--> Prometheus --remote_write--> long-term store
AWS services --------> CloudWatch metrics/alarms --SNS--> Lambda -> Teams
Grafana <---- Prometheus + CloudWatch + Loki + Tempo data sources
```

What to say to the interviewer: *I use CloudWatch for AWS-managed services and Prometheus for Kubernetes and application metrics, and bring both into Grafana using an IAM role for the CloudWatch data source. Alerts come from Alertmanager and CloudWatch Alarms, routed by severity, with RED dashboards per service and an SLO dashboard on top.*

Q14. How would you monitor HPA scaling decisions in real time and detect if the metrics server is lagging?

In simple words: The HPA is a thermostat. If the thermometer on the wall is stuck showing an old temperature, the thermostat will make wrong decisions, so you watch both the thermostat's actions and the thermometer's freshness.

Detailed answer

What to watch:

- **HPA state** from kube-state-metrics: `kube_horizontalpodautoscaler_status_current_replicas`, `..._status_desired_replicas`, `..._spec_max_replicas`, and the `ScalingLimited` / `AbleToScale` conditions.
- **Events**: `kubectl describe hpa <name>` shows "FailedGetResourceMetric" or "unable to fetch metrics".
- **Metrics pipeline health**: `kubectl get apiservice v1beta1.metrics.k8s.io` must be Available; metrics-server pod restarts and its scrape latency; for custom metrics, prometheus-adapter or KEDA errors.
- **Freshness**: compare metric timestamps; metrics-server scrapes kubelets every ~15s by default. A lag appears as desired replicas not changing while real CPU climbs.

Example:

```
# Alert: HPA pinned at max for 15m (cannot scale further)
kube_horizontalpodautoscaler_status_current_replicas
  >= kube_horizontalpodautoscaler_spec_max_replicas

# Alert: desired != current for too long (scaling stuck)
kube_horizontalpodautoscaler_status_desired_replicas
  != kube_horizontalpodautoscaler_status_current_replicas

kubectl get --raw /apis/metrics.k8s.io/v1beta1/namespaces/live/pods | head
```

What to say to the interviewer: *I graph HPA current versus desired versus max replicas from kube-state-metrics and alert when it is pinned at max or when desired and current differ too long. For metrics-server lag I watch the metrics APIService availability, FailedGetResourceMetric events and metrics-server health, and for critical workloads I scale on faster signals like request rate or queue lag through KEDA.*

Q15. Your monitoring shows constant false alerts. How do you tune alerts and reduce alert fatigue?

Also asked as: How do you reduce alert noise? Reducing alert fatigue. How do you implement SLO-based alerting without alert fatigue (error budget, burn-rate alerts, multi-window)?

In simple words: If a car alarm goes off every time a leaf falls, people stop looking. You adjust it to ring only when someone actually tries to break in.

Detailed answer

Steps:

1. **Measure the noise** - list the top 10 alerts by count last month and how many led to action. Anything never actioned is a candidate to delete or downgrade.
2. **Alert on symptoms, not causes** - page on user impact (error rate, latency SLO burn, site down), not "CPU 80%" which might be normal.
3. **Add duration** - `for: 5m` or 10m so brief spikes do not fire.
4. **Better thresholds** - base them on historical percentiles, not guesses; use SLO burn-rate alerts (fast burn pages, slow burn tickets).
5. **Deduplicate and group** - Alertmanager `group_by`, inhibition rules (if the node is down, suppress all pod alerts on it).
6. **Severity routing** - critical -> page/phone; warning -> chat channel; info -> dashboard only.
7. **Silence during maintenance** windows.
8. **Every page must have a runbook and an owner**; review alerts in a weekly or monthly ops review.
9. **Check the alerting path too** - false negatives (alerts silently not delivered) are as dangerous as false positives.

SLO-based alerting with multi-window, multi-burn-rate rules - burn rate is how fast you are using the error budget (1x = the budget lasts exactly the 30-day window). Each rule needs both a long window (proves it is significant) and a short window (proves it is still happening, so the alert clears quickly after recovery). Recommended values for a 99.9% SLO:

Burn rate	Long window	Short window	Budget used	Action
14.4x	1h	5m	2%	Page
6x	6h	30m	5%	Page
1x	3d	6h	10%	Ticket

This alerts on a symptom (users getting errors), not on causes like CPU, and it sends fast burns to the pager and slow burns to a ticket queue. For the PromQL of the 14.4x rule and how it fits SLA reporting, see Q17.

Example:

```
route:
  group_by: [alertname, cluster, namespace]
  group_wait: 30s
  repeat_interval: 4h
inhibit_rules:
  - source_matchers: [alertname="KubeNodeNotReady"]
    target_matchers: [severity="warning"]
    equal: [node]
```

What to say to the interviewer: *I first measure which alerts fire most and which ones actually led to action, then delete or downgrade the noise. I alert on user-facing symptoms with a for-duration, use SLO burn-rate alerts, group and inhibit related alerts in Alertmanager, route by severity, and make*

sure every paging alert has a runbook and an owner.

Q16. You are asked to design an alerting system for high availability. What is your approach?

In simple words: A fire alarm system that itself can fail silently is useless, so you install two alarm panels, two phone lines, and a test button you press regularly.

Detailed answer

Design:

- **Redundant alert evaluation** - two Prometheus replicas scraping the same targets (HA pair), or a managed service.
- **Alertmanager cluster** - 3 replicas in gossip mode; they deduplicate so you get one notification, not three.
- **Multiple notification channels** - primary (pager/phone via on-call tool) plus secondary (Teams/Slack, email). Critical alerts should not depend on a single webhook.
- **Watchdog / dead man's switch** - an always-firing alert (`vector(1)`) sent to an external service; if it stops arriving, the external service alerts you that your monitoring is dead.
- **External synthetic checks** - uptime checks from outside your cloud (another region/account/provider), so a total outage of your cluster still alerts.
- **Escalation policy** - primary -> secondary -> manager, with acknowledge timeouts.
- **Test end-to-end regularly** - send test alerts and confirm they arrive in the real channel; verify templates render (a broken message template can silently block delivery).
- **Alert as code** - rules in Git, reviewed and deployed through CI.

Example:

```
- alert: Watchdog
  expr: vector(1)
  labels: { severity: none }
  annotations:
    summary: "Heartbeat - if this stops, the alerting pipeline is broken"
```

What to say to the interviewer: *I make every layer redundant - HA Prometheus, a three-node Alertmanager cluster and more than one notification channel - and add a dead man's switch plus external synthetic checks so we find out if monitoring itself dies. Alert rules live in Git, escalation is defined, and I test delivery end to end regularly because a silently broken notification path is worse than a noisy one.*

Q17. How would you monitor the end-to-end SLA for services involved in a payments pipeline?

Also asked as: How would you monitor SLA compliance for a business-critical service?

In simple words: A customer does not care that each counter in the bank was fast; they care how long it took from walking in to getting the receipt. So you time the whole journey, not just each counter.

Detailed answer

1. **Define the user journey** - e.g. "payment initiated -> authorized -> captured -> confirmation shown". The SLI is end-to-end success and latency of that journey.
2. **SLIs per step and overall**: success rate, p95/p99 latency, and for async steps the **freshness/lag** (time from event produced to processed).
3. **Distributed tracing** across gateway, payment service, fraud check, bank adapter, queue consumers - with the same trace ID through Kafka headers.
4. **Business metrics** - payments per minute, success ratio by bank/method; a sudden drop in successful payments is often the best alert.
5. **Synthetic transactions** - a test card payment every minute from outside.
6. **Third-party dependency monitoring** - track bank/PSP response time and error codes separately so you know if it is "us or them".
7. **SLO + error budget + burn-rate alerts** on the journey SLI; dashboards showing budget remaining.
8. **Reconciliation checks** - count of initiated vs completed; stuck payments older than X minutes.

For SLA compliance of any business-critical service, add:

- **Multi-window, multi-burn-rate SLO alerts** - page when the error budget burns fast over both a long and a short window (for example 14.4x over 1h and 5m), and open a ticket for a slow burn (for example 6x over 6h and 30m). This catches real incidents quickly without paging for tiny blips.
- **Synthetic probes from outside** your cloud and region (for example the Blackbox exporter or an external uptime checker), so you measure what customers see even if your whole cluster is down.
- **Monthly SLA reports** - availability percentage, downtime minutes per service, and the root cause of each breach, compared with the contract target.
- **Error budget policy** - agreed in writing: when the budget is spent, risky releases freeze and reliability work comes first.

```
# Fast-burn page for a 99.9% SLO (budget = 0.001)
( sum(rate(http_requests_total{job="payments",status=~"5.."}[1h]))
  / sum(rate(http_requests_total{job="payments"}[1h])) > 14.4 * 0.001 )
and
( sum(rate(http_requests_total{job="payments",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{job="payments"}[5m])) > 14.4 * 0.001 )
```

Example:

```
# End-to-end success ratio (5m)
sum(rate(payment_completed_total{status="success"}[5m]))
/ sum(rate(payment_initiated_total[5m]))
# Alert if stuck payments > 0 for 10m
payments_pending_older_than_5m > 0
```

> ***What to say to the interviewer:*** *I define the SLI on the whole payment journey, not individual services, and measure it with business metrics, distributed traces through every hop including Kafka, and synthetic test payments. I alert on SLO burn rate and on stuck or unreconciled payments, and track third-party banks separately so we know quickly whether the problem is ours or theirs.*

Q18. How would you design auto-scaling for 50M+ concurrent viewers across multiple Kubernetes clusters without over-provisioning?

In simple words: For a huge cricket match, a stadium does not keep 50 lakh seats open all year. It knows the match schedule, opens extra gates just before the crowd arrives, and closes them after, while always keeping a small safety margin.

Detailed answer

Layers of scaling:

1. **Push most traffic to the CDN** - video segments are served from CDN edges; Kubernetes serves manifests, auth, session, entitlement, ads APIs. Offloading is the biggest cost saver.
2. **Scale on the right signals** - not only CPU. Use requests per second, concurrent sessions, or Kafka/queue lag via **KEDA** or custom metrics HPA.
3. **Predictive/scheduled scaling** - events are known in advance; use KEDA cron scaler or scheduled scaling to raise `minReplicas` before the toss, based on last event's peak + growth forecast.
4. **Fast node provisioning** - Karpenter (or Cluster Autoscaler) with multiple instance types and Spot for stateless tiers, On-Demand for critical tiers.
5. **Headroom without waste** - low-priority "placeholder" (overprovisioning) pods that reserve capacity and get evicted instantly when real pods need room.
6. **Multi-cluster distribution** - several regional clusters behind global load balancing; each cluster scales independently, and a cell-based design limits blast radius.
7. **Protect the system** - rate limits, load shedding, graceful degradation (drop to lower bitrate, disable non-critical features).
8. **Scale down carefully** - stabilization windows so it does not flap during innings breaks.

Example:

```
# KEDA: pre-scale before match + scale on concurrent sessions
triggers:
  - type: cron
    metadata: { timezone: Asia/Kolkata, start: "0 18 * * *", end: "0 1 * * *",
                desiredReplicas: "400" }
  - type: prometheus
    metadata:
      serverAddress: http://prometheus.monitoring:9090
      query: sum(active_sessions{svc="playback-api"})
      threshold: "5000"
```

What to say to the interviewer: *I push video delivery to the CDN, then scale the Kubernetes APIs on business signals like concurrent sessions using KEDA, combined with scheduled pre-scaling before known events. Karpenter with mixed Spot and On-Demand adds nodes quickly, low-priority placeholder pods give instant headroom, and multiple regional clusters, rate limits and graceful degradation protect us without paying for peak capacity all year.*

Q19. During an IPL final a new region needs to spin up instantly. How would you pre-warm nodes and scale workloads with zero cold-start impact?

In simple words: Before a big wedding, the caterer starts the stoves and cooks some food in advance, so when guests arrive they do not wait for the gas to be lit.

Detailed answer

Cold starts come from: new nodes booting, image pulls, app warm-up (JVM, caches), empty caches, and load balancers/CDN not warmed. Remove each one:

1. **Infrastructure ready in advance** - the region is already built by Terraform (same modules), cluster running at a small baseline. Standing up from nothing on the day is too risky.
2. **Pre-warm nodes** - raise the node group minimum or use Karpenter with placeholder (pause) pods at low priority 1-2 hours before; request capacity reservations for critical instance types.
3. **Pre-pull images** - a DaemonSet that pulls the big images, or bake images into the AMI; keep images small; use a registry replica in that region.
4. **Pre-scale pods** - raise `minReplicas` on HPA/KEDA ahead of time.
5. **App warm-up** - startup probes and readiness probes so pods only receive traffic after warm-up; warm caches (Redis) with popular content/metadata.
6. **Warm the edge** - CDN pre-fetch of manifests/assets; ask AWS to pre-warm load balancers for massive expected jumps if needed.
7. **Shift traffic gradually** - weighted routing (5% -> 25% -> 100%) while watching SLOs.
8. **Rehearse** - load test with k6/Locust at expected peak in the days before.

What to say to the interviewer: *I would never build the region on the day - it is pre-created from the same Terraform modules and running at a baseline. Before the match I pre-scale nodes with placeholder pods or capacity reservations, pre-pull images, raise minReplicas, warm caches and CDN, gate traffic on startup and readiness probes, and shift traffic in weighted steps after a rehearsed load test.*

Q20. How would you use Envoy and Istio to route low-latency live streams differently from VOD without service restarts?

In simple words: At a toll plaza, the traffic police can change which lane ambulances use just by changing the signboards, without closing the road or rebuilding the plaza.

Detailed answer

- Istio puts an **Envoy sidecar** (or ambient-mode proxy) next to each service. Envoy receives its routing config dynamically from **istiod** over the xDS API, so changing routes does not restart pods.
- Use a **VirtualService** to match live vs VOD requests (by path, header, or host) and send them to different subsets or services.
- Use a **DestinationRule** to give each subset different traffic policies: tight timeouts, fewer retries (retries add latency for live), connection pool limits, outlier detection, and locality-aware load balancing for live; more generous timeouts and caching-friendly settings for VOD.

Example:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: playback }
spec:
  hosts: [playback.svc.cluster.local]
  http:
    - match: [{ uri: { prefix: /live/ } }]
      route: [{ destination: { host: playback-live } }]
      timeout: 2s
      retries: { attempts: 1, perTryTimeout: 1s }
    - route: [{ destination: { host: playback-vod } }]
      timeout: 10s
      retries: { attempts: 3, perTryTimeout: 3s }
```

Apply with `kubectl apply`; Envoy picks it up in seconds. Roll out changes safely with weights (90/10) and watch latency.

> ***What to say to the interviewer:*** *Envoy gets its config dynamically from istiod over xDS, so I can change routing just by applying Istio resources with no restarts. A VirtualService matches live versus VOD by path or header and sends them to different backends, and DestinationRules give live traffic tight timeouts, minimal retries, locality-aware balancing and outlier detection, while VOD gets more relaxed settings.*

Q21. How would you configure readiness and liveness probes to catch buffering or lag in stream-processing services before users notice?

> **In simple words:** Do not just check if the worker is breathing (liveness); check if they are keeping up with the work (readiness). A worker who is alive but far behind should stop getting new work.

Detailed answer

(Probe basics are covered in the Kubernetes chapter; this is the streaming angle.)

- **Readiness** should reflect "can I serve good quality right now": check internal lag (e.g. consumer lag or segment processing delay below a threshold), dependency reachability (Redis, origin). If lag is high, return 503 so the pod leaves the load balancer and traffic goes to healthy pods.
- **Liveness** should be simple and only detect a truly stuck process (event loop hung, no progress for N minutes). Do not put dependency checks in liveness or you will restart every pod when Redis blips.
- **Startup probe** protects slow warm-up.
- Probes are not enough - also alert on business signals: rebuffering ratio, segment generation delay, consumer lag.

Example:

```
readinessProbe:
  httpGet: { path: /ready, port: 8080 }   # returns 503 if lag > 2s
  periodSeconds: 5
  failureThreshold: 2
livenessProbe:
  httpGet: { path: /live, port: 8080 }    # 500 only if no progress for 60s
  periodSeconds: 10
  failureThreshold: 6
```

What to say to the interviewer: *I make readiness reflect real serving quality, like processing lag under a threshold, so a lagging pod is taken out of rotation before users buffer, while liveness only detects a truly stuck process so we do not cause restart storms. Alongside probes I alert on rebuffering ratio and consumer lag, because probes only protect one pod at a time.*

Q22. How do you build a culture where latency SLOs are enforced like uptime SLAs?

In simple words: A restaurant that only checks "is the door open" will miss that customers wait 40 minutes for food. You make "food served within 15 minutes" a goal everyone is measured on, not just "open for business".

Detailed answer

1. **Make latency a first-class SLO** - e.g. "99% of play starts under 2 seconds" and "rebuffer ratio under 0.5%", agreed with product owners, because slow is the new down for streaming.
2. **Visible dashboards** - SLO and error budget shown on team dashboards and in weekly reviews.
3. **Error budget policy** - written and agreed: if the latency budget is exhausted, feature releases pause and reliability work gets priority.
4. **Alerts on burn rate** of latency SLOs, not just on up/down.
5. **Shift left** - performance tests in CI, latency budgets per service in design reviews, canary analysis that rolls back if p99 regresses.
6. **Ownership** - each service team owns its SLO; postmortems for latency incidents just like outages.
7. **Celebrate improvements** and keep it blameless.

What to say to the interviewer: *I define latency SLOs with product teams, show them and their error budgets on shared dashboards, and alert on burn rate exactly like availability. A written error budget policy pauses feature work when the budget is exhausted, canary analysis rolls back latency regressions automatically, and latency incidents get the same blameless postmortems as outages.*

Q23. How would you simulate chaos in a streaming pipeline without risking real user impact?

In simple words: Fire drills are done with a plan, a small area first, and a way to stop immediately - not by setting the real building on fire during a wedding.

Detailed answer

1. **Start with a hypothesis** - "If one Kafka broker dies, consumer lag stays below 30s and no playback errors".
2. **Start in staging/pre-prod** with production-like load (replayed or synthetic traffic).
3. **In production, limit blast radius** - one pod, one AZ, a canary cell, or only synthetic/test users; never during a live event.
4. **Guardrails / abort conditions** - automatic stop if SLO metrics cross a threshold.
5. **Tools** - Chaos Mesh or LitmusChaos in Kubernetes (pod kill, network delay, packet loss), AWS Fault Injection Service for AZ/instance failures, Istio fault injection for latency on specific

routes or headers only.

6. **Shadow traffic** - mirror real traffic to a copy of the pipeline and break the copy.
7. **Game days** - scheduled, announced, with the on-call team; document findings and fix gaps.

Example:

```
# Istio fault injection only for requests with a test header
http:
  - match: [{ headers: { x-chaos-test: { exact: "true" } } }]
    fault: { delay: { percentage: { value: 100 }, fixedDelay: 3s } }
    route: [{ destination: { host: playback-live } }]
```

What to say to the interviewer: *I run chaos as controlled experiments with a clear hypothesis, starting in staging with production-like load, then in production only on a small blast radius such as one canary cell or synthetic test traffic, never during live events. I use tools like Chaos Mesh, AWS FIS or Istio fault injection with automatic abort conditions on SLO metrics, and turn every finding into a fix.*

Q24. How do you justify infrastructure costs for pre-warmed scaling capacity to executives before a major sports event?

In simple words: You explain it like insurance: "We will spend 10 lakh for these three days so we do not risk losing 5 crore in ads and subscribers if the app crashes in the final over."

Detailed answer

Talk in business terms, not CPU:

1. **Cost of failure** - revenue per minute of the event (ads, subscriptions, sponsors), churn risk, brand damage, SLA penalties. Use past incidents as evidence.
2. **Data-backed forecast** - last year's peak concurrency, growth trend, and the gap between how fast autoscaling reacts (minutes) vs how fast traffic jumps (seconds at toss/final over).
3. **Exact, time-boxed ask** - pre-warm only for event windows, with a scale-down plan; show the cost per hour.
4. **Options table** - no pre-warm (cheapest, highest risk), partial pre-warm, full pre-warm - with cost and risk for each.
5. **Cost reducers** - Spot for stateless tiers, capacity reservations only for peak hours, CDN offload, Savings Plans for the baseline.
6. **Show results after** - report actual peak, cost, and SLOs met, which builds trust for the next ask.

Option	Extra cost (event days)	Risk
Reactive autoscaling only	Lowest	High - cold start at toss
Partial pre-warm (60% of peak)	Medium	Medium
Full pre-warm + Spot for stateless	Higher	Low

What to say to the interviewer: *I compare the time-boxed cost of pre-warming against the business cost of failure - lost ad revenue, churn and penalties per minute of downtime - using last event's real peak data. I present two or three options with cost and risk, reduce the bill with Spot, reservations*

only for the event window and CDN offload, and report actual results afterwards.

Scenario-Based

Q25. "Production is down. Users are impacted. What will you do?"

Also asked as: How do you debug a production issue? What is your general troubleshooting approach?

In simple words: When a patient arrives bleeding, the doctor first stops the bleeding, then finds the cause. You restore service first, investigate deeply later.

Detailed answer

1. **Acknowledge and declare** - take ownership, open an incident channel/bridge, assign roles: Incident Commander, Tech lead, Comms. Set severity.
2. **Assess impact** - which users, regions, features? Check SLO dashboards, error rate, synthetic checks, status of dependencies (cloud status page).
3. **Ask "what changed?"** - recent deploys, config changes, infra changes, feature flags, certificate expiry, traffic spike. Most incidents follow a change.
4. **Mitigate first** - rollback the last deploy, disable a feature flag, fail over, scale up, restart a stuck component, block abusive traffic. Stop the bleeding before root cause.
5. **Diagnose in layers** - edge (DNS, CDN, LB) -> ingress -> service -> dependencies (DB, cache, queue) using metrics, then traces, then logs.
6. **Communicate** regularly - every 15-30 minutes to stakeholders and status page, even if "still investigating".
7. **Verify recovery** - metrics back to normal, synthetic checks green, users confirm.
8. **Postmortem** - blameless RCA, action items with owners and dates.

General troubleshooting methodology (the 10 rules I follow for any issue):

1. **Check logs first** - application, pod events, system logs; the error message often names the cause.
2. **Trace the request flow** - user -> DNS -> CDN/LB -> ingress -> service -> pod -> DB/cache; find the first hop that fails.
3. **Process of elimination** - test one layer at a time (curl the service directly, then the pod, then the DB) and cross off what works.
4. **Infra vs app** - are all services affected (likely infra: node, network, DNS) or only one (likely app: code, config)?
5. **Validate external dependencies** - third-party APIs, cloud provider status, certificates, DNS, managed DB/cache.
6. **Check resource limits** - CPU throttling, OOMKilled, disk full, connection pools, file descriptors, quotas.
7. **Reproduce in a test environment** - confirm the cause and test the fix safely before touching production.
8. **Keep a known-issues doc / runbook** - check it first, and add every new issue and fix to it afterwards.

9. **Use health checks** - readiness/liveness probes, LB target health and synthetic checks show where it breaks.
10. **Know when to escalate** - if impact is growing or the issue is outside your area, pull in the owner early; time-box solo debugging.

Example: First 5 commands in a Kubernetes incident:

```
kubectl get pods -A | grep -v Running
kubectl get events -A --sort-by=.lastTimestamp | tail -30
kubectl rollout history deploy/checkout -n prod
kubectl logs deploy/checkout -n prod --since=15m | grep -i error | tail
kubectl rollout undo deploy/checkout -n prod     # if the last deploy is the cause
```

> ***What to say to the interviewer:*** *I declare an incident, set up roles and a channel, and quickly assess the blast radius. I check what changed recently and mitigate first - usually a rollback, feature flag or failover - then diagnose layer by layer with metrics, traces and logs, keep stakeholders updated on a fixed cadence, confirm recovery, and follow up with a blameless postmortem.*

Q26. Monitoring shows increased response time, but infrastructure (CPU, memory, pods) looks healthy. How would you investigate?

> **In simple words:** The kitchen staff are not tired and the stoves are working, but food is still late. Maybe the delivery boy is stuck in traffic, or they are waiting for a supplier - the problem is outside the kitchen.

Detailed answer

Healthy CPU/memory usually means the service is **waiting**, not working. Look for:

1. **Where is the time spent?** - open a slow trace: which span is slow - DB, cache, external API, another service?
2. **Dependencies** - DB slow queries, lock waits, connection pool exhaustion (threads waiting for a connection), Redis latency, third-party API latency.
3. **Saturation that is not CPU** - thread pools, connection pools, file descriptors, network bandwidth, disk IOPS/EBS burst credits, CPU throttling from low limits (`container_cpu_cfs_throttled_seconds_total`) even when node CPU looks fine.
4. **Scope it** - one endpoint, region, AZ, customer or version? Compare p50 vs p99 (only tail = contention/GC/noisy neighbor).
5. **Network** - DNS lookup delays, TLS handshakes, cross-AZ hops, retries/timeouts, load balancer queueing.
6. **Recent changes** - deploys, config, a new feature causing N+1 queries, bigger payloads.
7. **GC pauses** in JVM/Node apps.
8. **Time pattern and traffic level** - if the slowness is periodic or happens off-peak, suspect scheduled work (cron, backups, snapshots, DB maintenance) or cold starts after scale-down (empty caches, closed connections, cold Lambdas). At low traffic a few slow requests can distort p99, so check the request count before trusting the percentile.

Example:

```
# Is the pod CPU-throttled even though node CPU is fine?
rate(container_cpu_cfs_throttled_periods_total{pod=~"orders.*"}[5m])
/ rate(container_cpu_cfs_periods_total{pod=~"orders.*"}[5m])

# p99 by endpoint to scope the problem
histogram_quantile(0.99, sum by (le, route)
  (rate(http_request_duration_seconds_bucket{app="orders"}[5m])))
```

What to say to the interviewer: *If CPU and memory are fine, the service is usually waiting on something, so I open slow traces to see which dependency or hop the time is spent in. I check database slow queries and connection pools, cache and third-party latency, CPU throttling from limits, thread pools, DNS and network, scope it by endpoint or region, and correlate it with recent changes.*

Q27. It is 2 AM. Users report the app is slow. There are no errors and no logs. What is your first move?

Also asked as: After a deploy, latency spikes for 30% of users - no errors, no logs. What now? You get paged at 2 AM for high latency across services - what is your move? (tests calm under pressure)

In simple words: A customer calls at night saying the shop is slow, but the cashier's notebook is empty. Either the customers never reached the counter, or the notebook itself stopped working - so first check the door and the notebook, then the kitchen.

Detailed answer

1. **Confirm and scope** - synthetic checks and real-user monitoring (RUM): is it real, all users or some, one region/endpoint? Compare latency **p50 vs p95/p99** against the error rate (slow with zero errors = something is waiting, not crashing).
2. **Golden signals / RED per service** - rate, errors, duration, saturation. Which service's latency moved first?
3. **Traces** - open a slow trace (or a latency exemplar) and find the slow span.
4. **Dependencies** - DB slow queries, locks, connection pool saturation, cache hit ratio drop, third-party API latency.
5. **What runs at 2 AM?** - backups, cron jobs, batch ETL, DB autovacuum, log rotation, EBS/RDS snapshots. Night-time slowness is very often scheduled work.
6. **Noisy neighbours and GC** - another workload on the same node or DB, burst credits exhausted, long GC pauses.
7. **"No logs" is itself a clue:**

- the **log pipeline may be down** (agent crashed, Loki/ELK ingestion stuck, disk full), so check whether any service is logging right now; or
- **requests are not reaching the app at all**, so look before the app: load balancer/CDN metrics and queueing, DNS resolution, TLS handshake or certificate problems.

8. **Mitigate first** - scale out, fail over, pause or reschedule the batch job, then confirm latency recovers.
9. **Communicate** - short update to stakeholders or the status page, then a proper RCA the next day.

The "paged at 2 AM, latency high across many services" variant (calm under pressure):

- **Acknowledge the page** right away so nobody else is left wondering, then **set severity** based on user impact.
- **Open an incident channel** and write down what you see and when; a calm, written process beats panicked clicking.
- **Many services slow at once = look for one shared dependency** - a database, DNS/CoreDNS, the network or a NAT/cross-AZ path, a shared cache (Redis), the ingress/load balancer, or a noisy node pool where all these pods run together.
- **Mitigate before root-causing** - fail over, scale out, cordon the bad nodes, pause the batch job.
- **Post updates on a fixed cadence** (for example every 15-30 minutes), even if it is only "still investigating".
- **Escalate early** to the owner of the shared dependency; escalating early is a strength, not a weakness.

The post-deploy variant (30% of users slow after a release):

- **Correlate with the deploy timestamp** - Grafana deploy annotations make this obvious on the latency graph.
- **30% is a strong clue** - it usually means a subset: canary pods on the new version, one AZ, one node group, or one ingress path/route.
- **Compare latency by label** - `version`, `pod`, `zone`, `route`; open trace exemplars from the slow series to find the slow span in the new code (new query, missing index, bigger payload, lower resource limits causing CPU throttling).
- **Roll back first if the SLO is burning** - `kubectl rollout undo` / `helm rollback`, or shift canary weight to 0, then debug the new version off the critical path.

Example:

```
# Tail-only or everyone? Compare percentiles
histogram_quantile(0.50, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
# Post-deploy: p99 split by version (old vs new pods)
histogram_quantile(0.99, sum by (le, version)
  (rate(http_request_duration_seconds_bucket{app="checkout"}[5m])))
# Are logs flowing at all? (LogQL)
sum by (namespace) (count_over_time({namespace=~".+"}[5m]))
kubectl get cronjobs -A          # anything scheduled at 2 AM?
```

> ***What to say to the interviewer:*** *First I confirm and scope it with synthetic checks, RUM and p50 versus p99, then follow RED metrics and traces to the slow span, and check dependencies and anything scheduled at that hour like backups, ETL or vacuum. No logs is itself a clue - either the log pipeline is broken or requests are not reaching the app, so I check the LB, CDN, DNS and TLS. If it started after a deploy and hits about 30% of users, I compare latency by version, pod and AZ and roll back first if the SLO is burning.*

Q28. Application metrics are not being scraped by Prometheus. What could cause it, and how do you validate scraping?

In simple words: The meter reader is not recording your house. Either he does not have your address, he is going to the wrong door, the gate is locked, or there is no meter on the wall. Check each one in order.

Detailed answer

Common causes:

- **Target not discovered** - ServiceMonitor labels do not match the Prometheus `serviceMonitorSelector` (for example a missing `release: kps` label), or its `namespaceSelector` does not include the app's namespace.
- **Selector mismatch** - the ServiceMonitor `selector.matchLabels` does not match the Service's labels.
- **Port name mismatch** - the ServiceMonitor `port:` must be the named port in the Service (for example `http-metrics`), not the number.
- **Wrong path or scheme** - the app serves `/actuator/prometheus` or HTTPS, but the scrape uses `/metrics` over HTTP -> `server returned HTTP status 404`.
- **Auth** - `401/403` because the endpoint needs a token or basic auth.
- **Network** - a NetworkPolicy blocks the monitoring namespace, or the app binds to `127.0.0.1` so it is unreachable from outside the pod.
- **App does not expose metrics** at all (no client library or exporter).
- **Prometheus RBAC** - no permission to list/watch endpoints or pods in that namespace.
- **relabel_configs** dropping the target, or `scrape_timeout` too low -> `context deadline exceeded`.

How to validate, in order:

1. Prometheus UI -> **Status -> Service Discovery**: is the target discovered or dropped?
2. **Status -> Targets**: is it up or down, and what is the error text?
3. Query `up{job="orders"}` - 1 = scraped OK, 0 = scrape failing, no result = not discovered.
4. Test the endpoint directly from outside the pod.

Example:

```
kubectl -n prod port-forward svc/orders 8080:8080
curl -s localhost:8080/metrics | head          # does the app expose metrics?
kubectl -n prod get svc orders -o yaml | grep -A5 ports    # check the port NAME
kubectl -n monitoring get prometheus -o yaml | grep -A3 serviceMonitorSelector
kubectl -n prod get servicemonitor orders -o yaml
# PromQL
up{job="orders"}
```

What to say to the interviewer: *I check the Prometheus Service Discovery and Targets pages and the up metric to see whether the target is undiscovered or failing, and read the exact error. The usual causes are ServiceMonitor labels or namespace not matching the Prometheus selector, a port-name mismatch, a wrong path or scheme, auth errors, a NetworkPolicy, the app binding to localhost, RBAC or relabeling. I confirm the fix with a port-forward and curl on /metrics, then check that up returns 1.*

Q29. Playback failures spike for only 3% of users in the APAC region. CPU, memory and pods look fine. Walk through your triage plan.

In simple words: If only some customers in one city complain about a shop, the shop itself is probably fine - the problem is one road, one delivery van, or one type of customer.

Detailed answer

A small, regional slice means **something shared by that 3%**, not a general capacity issue.

1. **Confirm and size it** - error rate and play-start failures by region, ISP, CDN edge/PoP, device, OS, app version, content ID, DRM type. Real-user monitoring (RUM) / client telemetry is key because servers may look fine.
2. **Find the common factor** - one CDN PoP, one ISP, one AZ, one app version, one codec, one content package?
3. **Edge and network** - CDN edge health and cache hit ratio in APAC, origin errors from that PoP, TLS certificate chain issues on older devices, DNS resolution for that ISP, BGP/routing issues.
4. **Application paths** - DRM/license server errors, entitlement/geo-blocking rules, token expiry or clock skew, a canary or feature flag rolled out only to APAC or to 3% of users (a 3% canary is a big clue).
5. **Recent changes** - CDN config, new app release on one platform, certificate rotation.
6. **Mitigate** - route that region/ISP to another CDN or PoP, roll back the canary/flag, purge bad cache objects.
7. **Follow up** - add alerts sliced by region/ISP/version so a 3% issue is not hidden inside a global average.

What to say to the interviewer: *When only 3% of one region fails while servers look healthy, I look for what those users share, slicing client telemetry by CDN PoP, ISP, device, app version, content and DRM type. Typical culprits are one CDN edge, an ISP routing or DNS problem, a certificate issue on old devices, or a canary rolled out to a small percentage, and I mitigate by shifting CDN or rolling back, then add region and version-sliced alerts.*

Q30. Your Kafka ingestion pipeline lags by 2 minutes during a traffic surge. Producers are fine, consumers are idle. What is your debug path?

In simple words: Parcels are piling up at the warehouse, and the delivery boys are sitting idle. So they are either not assigned any parcels, cannot reach the parcels, or are waiting for permission - the problem is in the assignment, not the workload.

Detailed answer

"Lag high + consumers idle" means consumers are not actually fetching or processing. Check in order:

1. **Consumer group state** - is it stuck **rebalancing**? Frequent rebalances (members joining/leaving, `max.poll.interval.ms` exceeded, pod restarts, HPA scaling consumers up and down) stop all consumption.
2. **Partition assignment** - more consumers than partitions leaves extras idle; all lag may be on a few **hot partitions** (bad key choice) assigned to one consumer while others sit idle.

3. **Consumer blocked downstream** - waiting on DB/Redis/API calls, so the thread is blocked (looks idle on CPU) - check traces and connection pools.
4. **Fetch configuration** - `fetch.min.bytes`, `fetch.max.wait.ms`, `max.poll.records` too small; consumer paused due to backpressure.
5. **Broker side** - broker CPU/disk/network, under-replicated partitions, leader imbalance, throttling quotas, broker disk full.
6. **Connectivity/auth** - consumers cannot reach new partition leaders, ACL/cert errors in consumer logs.
7. **Autoscaling** - scale consumers on lag with KEDA, but only up to the partition count; add partitions carefully (changes key ordering).

Example:

```
kafka-consumer-groups.sh --bootstrap-server $BROKER --describe --group ingest-svc
# Shows per-partition CURRENT-OFFSET, LOG-END-OFFSET, LAG, CONSUMER-ID
# Empty CONSUMER-ID with high LAG  -> partition not assigned
# All lag on 2 partitions          -> hot key / skew
kafka-consumer-groups.sh --bootstrap-server $BROKER --describe --group ingest-svc --state
# State: PreparingRebalance / CompletingRebalance -> rebalance storm
```

> ***What to say to the interviewer:*** *Idle consumers with growing lag tells me they are not fetching, so I describe the consumer group to check for rebalance storms, unassigned partitions, hot partitions from key skew, and more consumers than partitions. Then I check whether consumers are blocked on a downstream dependency, their fetch and poll settings, and broker health, and I scale consumers on lag with KEDA only up to the partition count.*

Q31. Sudden tail latency on the Redis-based stream session store during a Champions League match. How do you find and fix the bottleneck?

> **In simple words:** Redis is one very fast cashier who serves one customer at a time. If one customer asks to count every coin in the shop, everyone behind them waits - that is tail latency.

Detailed answer

Redis executes commands on a **single thread**, so one slow command or hot key hurts everyone's p99.

1. **Confirm it is Redis** - traces show slow Redis spans; compare client-side vs server-side latency (network vs Redis itself).
2. **Slow commands** - `SLOWLOG GET`; look for `KEYS *`, large `HGETALL`, `SMEMBERS`, `LRANGE 0 -1`, big `DEL` on huge keys, Lua scripts.
3. **Hot keys / big keys** - `redis-cli --hotkeys` (with LFU policy) and `--bigkeys`; one session key or counter hammered by millions.
4. **Resources** - CPU of the main thread near 100%, memory near `maxmemory` causing evictions, network bandwidth limit of the node, connection count spikes (new connections are expensive - use pooling).
5. **Persistence and replication** - RDB fork / AOF rewrite pauses, replica full resync.
6. **Client side** - connection pool too small, timeouts and retries causing retry storms.

Fixes: replace `KEYS` with `SCAN`, split big keys, cache hot keys locally in the app for a second, use pipelining, spread keys with Redis Cluster/more shards, read replicas for read-heavy data, bigger node or more network bandwidth, schedule persistence off the replicas, use `UNLINK` instead of `DEL`.

Example:

```
redis-cli -h $HOST --latency-history
redis-cli -h $HOST SLOWLOG GET 20
redis-cli -h $HOST --bigkeys
redis-cli -h $HOST INFO stats | grep -E "evicted_keys|instantaneous_ops"
redis-cli -h $HOST INFO clients | grep connected_clients
```

What to say to the interviewer: *Because Redis is single-threaded, I look for anything blocking that thread: slow commands in SLOWLOG, big keys, hot keys, fork pauses from persistence, evictions near maxmemory, and network or connection limits. Fixes are replacing KEYS with SCAN, splitting big keys, local caching of hot keys, pipelining and connection pooling, and sharding with Redis Cluster or read replicas.*

Q32. After an outage, how would you conduct a proper post-incident analysis (RCA / blameless postmortem)?

Also asked as: How do you write RCA documentation? Postmortems.

In simple words: After a plane incident, investigators do not just blame the pilot; they look at the whole system - checklists, training, instruments - so the same thing never happens again to any pilot.

Detailed answer

1. **Blameless** - focus on how the system allowed the mistake, not who made it. People share more honestly.
2. **Within 2-5 days**, while memory is fresh; include everyone involved.
3. **Timeline** - detection, escalation, mitigation, resolution, with timestamps from chat, alerts and deploy logs.
4. **Impact** - duration, users/regions affected, SLO budget consumed, revenue/SLA effect.
5. **Root cause and contributing factors** - use "5 Whys" or a fishbone; there is usually more than one cause (a bug + a missing alert + a slow rollback). Verify each cause with evidence, do not assume.
6. **What went well / what went badly / where we got lucky.**
7. **Action items** - specific, owned, dated, tracked in Jira: prevent (fix the bug, add test), detect (new alert), mitigate faster (runbook, automated rollback).
8. **Share and follow up** - publish, review action items until closed.

Example: Structure:

```
Summary | Impact | Timeline | Root cause | Contributing factors
Detection (how/when did we find out?) | What went well | What went wrong
Action items: [ID] [Action] [Owner] [Due date] [Type: prevent/detect/mitigate]
```

What to say to the interviewer: *I run a blameless postmortem within a few days, build an evidence-based timeline, quantify the impact, and use 5 Whys to find the root cause and contributing factors instead of blaming a person. The key output is a small set of owned, dated action items for*

prevention, detection and faster mitigation, tracked until they are closed.

Behavioral & Project Experience

Basic

Q1. Tell me about yourself.

Also asked as: Introduce yourself.

> **In simple words:** The interviewer wants a 60-90 second story: who you are, what you work on now, one or two results you are proud of, and why you want this role. It is not your life history.

Detailed answer

Use the formula **Present -> Proof -> Future**:

- **Present** - your role and the platform you support.
- **Proof** - two concrete achievements with results.
- **Future** - what you want next and why this role fits.

Example: Sample answer - adapt to your own words:

```
I have around 5 years in IT, and currently I work on the operations and monitoring
side of a smart-metering platform at a smart-energy technology company. The platform runs on Amazon
    EKS
across four production environments, with services like HES, MDMS and WFM, backed
by RDS, ElastiCache and Kafka (MSK).

My focus is observability and reliability: Zabbix, Prometheus/Alertmanager, Grafana
and Loki, SLA dashboards and reports, and alert routing to Microsoft Teams. For
example, I built a zero-cost Teams alerting path with Lambda relays when our planned
on-call tool became unavailable, and I found and fixed a bug where new alert media types
had empty message templates, which had silently blocked alert delivery.

Now I want to grow into a full DevOps role - CI/CD, Terraform and Kubernetes
operations - and this role is a good fit because it combines that with production
ownership.
```

> ***What to say to the interviewer:*** *Keep it to about 90 seconds: current role and platform, two proof points with results, and why you want this specific role. End on something that invites a follow-up question you are ready for.*

Q2. What are your daily responsibilities as a DevOps engineer?

Also asked as: Hiring-manager round - describe your day-to-day responsibilities. What are your daily tasks?

> **In simple words:** They want to see that your day is about keeping production healthy and improving it, not just doing tickets. Show a rhythm: check health, fix issues, improve something, communicate.

Detailed answer

Structure it as a routine:

- **Morning health check** - review overnight alerts, SLA dashboards, daily health reports for all environments.

- **Incident and alert handling** - triage alerts, find root cause, fix or escalate, and verify that alerts actually reach the Teams channels.
- **Monitoring improvements** - new dashboards, alert rules, thresholds, reducing noise, segregating alerts by category.
- **Automation** - scheduled Lambda reports, health probes, scripts that remove manual checks.
- **Reporting** - daily and monthly SLA reports with downtime and root causes.
- **Tickets and collaboration** - Jira stories and subtasks, working with developers and the platform team.
- **Cost and hygiene** - finding orphaned volumes, unused buckets, over-sized resources.

Example: Sample answer - adapt to your own words:

```
My day starts with checking overnight alerts and the daily health reports for our
four environments. Then I handle any open incidents - for example a disk or memory
alert on a monitoring server. The rest of the day is improvement work from Jira:
building or fixing alert rules, Grafana dashboards, automation for reports, and
preparing SLA reports that explain every minute of downtime with a root cause.
```

> ***What to say to the interviewer:*** *Describe a routine - health checks, incident handling, improvement work, automation and reporting - with one real example for each. Interviewers remember the example, not the list.*

Q3. Walk me through your current project architecture and your role in it.

> **In simple words:** They want to see if you understand the whole system, not only your piece. Draw it from the user's request inward, then say which boxes you own.

Detailed answer

Explain in layers, high level, no internal IDs:

- **Business** - smart-metering platform for power utilities: meters send readings, the platform collects, stores, bills and manages field work.
- **Applications** - HES (Head End System: talks to meters, collects data), MDMS (Meter Data Management: validates, stores, processes readings), WFM (Workforce Management: field jobs), running as microservices on **Amazon EKS**.
- **Data layer** - Amazon RDS (PostgreSQL) for databases, ElastiCache (Redis) for caching/sessions, Amazon MSK (Kafka) for streaming meter events. One environment runs self-managed PostgreSQL/Redis/Kafka on EC2.
- **Environments** - four production environments for different utility projects, each in its own AWS account or setup.
- **Monitoring layers** - Zabbix for host, service and SLA monitoring; Prometheus + Alertmanager inside the clusters; Loki for logs; Grafana dashboards; CloudWatch alarms for AWS services; alerts delivered to Microsoft Teams channels.

Example:

```
Meters -> HES (EKS) -> Kafka (MSK) -> MDMS (EKS) -> RDS PostgreSQL
                           |                |
                      Redis cache      WFM / billing / APIs -> ALB -> users
Monitoring: Zabbix + Prometheus/Alertmanager + Loki + CloudWatch -> Grafana -> Teams
```

My role: I own the observability and alerting layer across all four environments - dashboards, alert rules, SLA reporting, report automation, health checks - and I am the first responder for monitoring and infrastructure alerts.

> ***What to say to the interviewer:*** *Walk from the meter to the user through HES, Kafka, MDMS and the databases, then describe the monitoring layer, and finish with a clear statement of what you own. Be ready to go one level deeper on any box you mention.*

Q4. Which DevOps tools have you worked with in the last 2 years?

Also asked as: Which IaC tools have you used - Terraform, Ansible, Puppet? Tell me about your cloud project work.

> **In simple words:** They are checking depth, not the length of the list. Group tools by category and say honestly where you are hands-on versus learning.

Detailed answer

Group the answer:

Category	Hands-on in work	Practising / learning
Monitoring & logs	Zabbix, Prometheus, Alertmanager, Grafana (incl. API-built dashboards), Loki	OpenTelemetry, Tempo
Cloud (AWS)	EKS, EC2, Lambda, EventBridge, CloudWatch, SNS, RDS, ElastiCache, MSK, S3, IAM	Route 53, VPC design
Containers & K8s	kubectl for troubleshooting, reading pod logs/events	Helm, writing manifests
IaC & CI/CD	-	Terraform, Jenkins, GitHub Actions, Argo CD
Config management	-	Ansible (Puppet and Chef: concepts only)
Scripting & OS	Linux, Bash, cron/scheduled jobs	Ansible
Collaboration	Jira, Confluence, Git	-

For the IaC question, be clear on the difference: Terraform provisions infrastructure (declarative, keeps state), while Ansible, Puppet and Chef configure servers (Ansible is agentless and push-based; Puppet uses agents and pulls). Then say honestly which ones you used in production and which in labs. For cloud project work, point to the AWS project in Q3.

Tip: never list a tool you cannot answer two follow-up questions on. If a tool is lab-only, say "I have used it in hands-on labs" - honesty builds more trust than a long list.

> ***What to say to the interviewer:*** *Answer by category, give one real example for your strongest tools, and clearly separate production experience from lab experience. That honesty makes your strong areas more believable.*

Q5. Which AWS services are you proficient in, and how many have you worked with?

In simple words: Nobody knows all 200+ AWS services. They want to hear the core ones you actually used, and what you did with them.

Detailed answer

Pick about 8-12 and attach a verb to each:

- **EKS / EC2** - run and troubleshoot workloads and monitoring servers.
- **Lambda + EventBridge** - scheduled daily health and SLA reports, alert relays, external health checks.
- **CloudWatch + SNS** - alarms for RDS, ElastiCache, MSK; routing alerts.
- **RDS, ElastiCache, MSK** - monitor health, disk, connections, lag.
- **IAM** - roles for Lambda and EC2 instead of access keys (for example replacing an IAM user with an instance role).
- **S3** - storage, lifecycle, cost cleanup.
- **VPC / Security Groups** - understanding connectivity when checks fail.

Example: Sample answer - adapt to your own words:

```
I have worked with around 12 AWS services. Day to day I use EKS, EC2, CloudWatch,
Lambda and EventBridge - for example our daily infrastructure reports are Lambda
functions on an EventBridge schedule that check nodes, RDS, Redis and MSK and post
to Teams. I also monitor RDS, ElastiCache and MSK, and I moved one monitoring server
from stored IAM user keys to an instance role for security.
```

What to say to the interviewer: *Name a realistic number, then give real usage for the top four or five. Mentioning a security improvement like replacing access keys with IAM roles shows maturity.*

Q6. What have you achieved in your career so far?

In simple words: They want proof of impact, not a job description. Pick 3-4 achievements, and for each say the problem, what you did, and the result.

Detailed answer

Sample answer - adapt to your own words (short STAR for each):

1. **Zero-cost alerting that did not depend on a blocked tool** - S/T: the planned on-call tool became unavailable, so alert delivery was at risk. A: built Teams-direct alerting with small Lambda relays for CloudWatch alarms, and pointed Alertmanager's native Teams integration at the channels, with a backup of the old routes. R: alerts kept flowing across projects at no extra cost.
2. **Found and fixed a silent alert-delivery failure** - S/T: new category-based alert channels looked configured but alerts were not arriving. A: tested each channel with a real test message and found empty message templates (and some disabled channels). R: delivery restored and verified on all four environments. The same habit later caught a broken template on the API error alert contact points.

3. **Observability and cost audit** - S/T: logging, metrics and tracing were reviewed across five environments. A: documented each stack and found orphaned volumes, PVCs and frozen buckets. R: identified recurring monthly waste (on the order of tens of dollars per environment) for the owners to clean up, plus real gaps such as broken tracing.
4. **SLA dashboards and reporting across 4 projects** - A: built live Grafana SLA dashboards from the Zabbix SLA API and a daily and monthly SLA reporting routine that explains every minute of downtime with a specific root cause. R: management and clients get consistent, evidence-based availability numbers.
5. **Fixed the daily report automation** - S/T: the scheduled daily report Lambdas were posting the same report three times. A: traced it to automatic Lambda retries on long runs and fixed the retry setting, timeout and message pacing. R: one clean report per day.

What to say to the interviewer: *Pick three or four achievements and give each one a problem, an action and a result. Reliability, cost savings and automation results are what interviewers remember.*

Q7. How much experience do you have handling databases?

In simple words: They want to know if you can keep a database healthy in production - monitoring, backups, failover, connections - not whether you are a DBA who tunes every query. Be honest about which one you are.

Detailed answer

Honest framing (adapt to your real experience): "I have operational and monitoring experience with databases, not DBA-level experience."

- **Engines I work around** - Amazon RDS PostgreSQL; self-managed PostgreSQL on EC2 in one environment; ElastiCache Redis; Kafka on Amazon MSK (and a self-managed Kafka).
- **Monitoring** - CPU, memory, free storage, connections, replica lag, disk alarms in CloudWatch and Zabbix; DB health probes that check the database is actually reachable and answering queries, not just that the host is up.
- **Incidents** - disk-usage alerts and threshold tuning (for example the MSK disk incident), and investigating DB-related downtime for SLA reports.
- **Backup and recovery concepts** - automated RDS backups and snapshots, point-in-time restore, Multi-AZ failover, read replicas for read scaling and how a replica can be promoted.
- **Basics I can do** - connect with `psql`, check active connections and long-running queries, check sizes.

Example:

```
psql -h $DB_HOST -U app -d appdb -c "select count(*) from pg_stat_activity;"
psql -h $DB_HOST -U app -d appdb -c "select pid, now()-query_start as age, state, query
  from pg_stat_activity where state <> 'idle' order by age desc limit 5;"
```

What to say to the interviewer: *I have operational experience with databases from the DevOps side - RDS and self-managed PostgreSQL, Redis and Kafka - covering monitoring, alarms, health probes, backups and failover concepts, but I am not a DBA. I can check connections and slow queries and keep the database healthy and recoverable, and I work with the developers on schema and query tuning.*

Q8. How do you use Linux in your DevOps work, and which distributions have you used?

Also asked as: How much Linux do you know? Which version of Linux do you use?

> **In simple words:** Linux is the ground every DevOps tool stands on. They want to hear that you are comfortable fixing a real server from the command line.

Detailed answer

- **Distributions** - mainly Ubuntu and Amazon Linux (RHEL family) on EC2; container base images like Alpine and Debian slim.
- **Daily use** - SSH into servers, check services with `systemctl`, read logs with `journalctl` and `tail`, check disk/memory/CPU (`df -h`, `du`, `free -m`, `top`), manage cron jobs, and edit configs.
- **Real troubleshooting** - disk filling up on a monitoring server, memory pressure from PHP-FPM worker counts, a hung OS causing a 504 on the web UI.

Example:

```
df -h                                  # which filesystem is full?
sudo du -xh / --max-depth=2 | sort -h | tail
sudo journalctl --disk-usage
sudo journalctl --vacuum-size=500M     # safely shrink old system journals
systemctl status mysqld zabbix-server
```

Sample answer: "When our Zabbix server disk hit about 91%, I found old systemd journals taking large space and reduced usage to about 82% with `journalctl --vacuum-size`, then flagged a large stale data folder for a decision instead of deleting it blindly."

> ***What to say to the interviewer:*** *Name the distributions you use, the everyday commands, and one real Linux incident you solved. Emphasize that you check before deleting anything on a production server.*

Q9. Can you describe the CI/CD workflow in your project?

Also asked as: How do you handle the continuous delivery (CD) aspect in your projects?

> **In simple words:** They want the journey of a code change from a developer's laptop to production, and your part in it. If your role is not the pipeline owner, say so honestly and describe how you support and monitor it.

Detailed answer

Describe the flow clearly (a typical EKS flow - confirm it matches your project before saying it):

1. Developer pushes to Git -> pull request with review.
2. CI (e.g. Jenkins / GitHub Actions) builds, runs tests and code scans, builds the Docker image, tags it with the commit SHA, pushes to Amazon ECR.
3. CD updates the Helm values or manifest with the new tag; deploys to staging first, then to production after approval (via Helm or GitOps with Argo CD).
4. Kubernetes does a rolling update; readiness probes gate traffic.

5. Monitoring after deploy - error rate, latency, pod restarts; rollback with `helm rollback` or `kubectl rollout undo` if needed.

Example: Honest framing - adapt to your own words:

```
In my project the platform team owns the build pipeline, and my part is the
production side: after a deployment, our monitoring shows whether services and SLAs
are healthy, and alerts reach Teams if something breaks. I have also built pipelines
myself in hands-on labs - Jenkins building a Docker image, pushing to ECR and
deploying to Kubernetes with Helm - so I understand each stage end to end.
```

> ***What to say to the interviewer:*** *Walk through code -> build -> test -> image -> registry -> staging -> approval -> production -> monitoring -> rollback, and be precise about which parts you personally own. Honest scoping plus lab experience is far safer than claiming a pipeline you cannot explain in depth.*

Scenario-Based

Q10. Tell me about a recent outage or production incident you handled. What did you learn from it?

Also asked as: Share a real outage incident - what did you learn? Describe an incident and how you responded. Troubleshooting real incidents during on-call. How did you handle outages, communication and postmortems?

> **In simple words:** They are testing how you behave under pressure: do you stay calm, find the real cause with evidence, fix it, and make sure it never happens again?

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - We had built new category-based alert channels (media types) for Teams across all four environments as part of an alert segregation project. Alerts looked configured, but the team was not receiving some expected alerts.
- **Task** - Find out why alerts were not arriving and restore delivery everywhere, fast, because no alerts means we are blind in production.
- **Action** - Instead of assuming the network or Teams was at fault, I tested each media type with the built-in test button and checked the action logs. I found that the new media types had empty message templates, which silently blocked delivery from the day they were created. In one environment some media types were also disabled. I added the templates, enabled them, and verified delivery with live tests in every environment.
- **Result** - Alert delivery was restored on all four environments and verified end to end.
- **Learning** - "Configured" is not the same as "working". Now every new alert path is tested with a real message before it is marked done, and I think about a heartbeat check for the alerting path itself.

Communication strategy during an outage (they often ask this separately):

- **One incident commander** runs the call and decides; others investigate. This avoids five people changing things at once.
- **Status updates at a fixed cadence** (for example every 15-30 minutes) in one channel and on the status page, even when the update is "no change, still investigating".

- **Stakeholder-friendly language** - impact, what we are doing and the next update time, not internal jargon. Technical detail stays in the engineering channel.
- **Postmortem within 48 hours** - blameless, with a timeline, root cause and owned action items, shared with everyone affected.

What to say to the interviewer: *Tell one incident in STAR form with a clear evidence-based root cause, your specific actions, a verified result and a lasting process change. The learning is what separates a senior answer from a junior one.*

Q11. Tell me about a mistake you made in a DevOps project and what you learned from it.

In simple words: Everyone makes mistakes. They want to see that you own it, fix it quickly and change your process - not that you are perfect.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - We moved alerting away from an older on-call tool setup, and I reported that the old setup was no longer active.
- **Task** - Make sure there was exactly one reliable alert path to Microsoft Teams.
- **Action** - Later I noticed duplicate alerts in Teams. When I investigated, I found the old setup was still live, with webhooks still posting alongside the new Lambda relay. My mistake was assuming it was decommissioned without checking every remaining integration. I disabled the old webhooks, fixed a forwarding bug, confirmed only one path remained, and corrected my earlier report to the team.
- **Result** - Duplicate alerts stopped, and the team had an accurate picture again.
- **Learning** - I now verify decommissioning with evidence (list every integration or webhook and confirm zero traffic) before calling anything "removed", and I correct the record openly as soon as I find an error.

What to say to the interviewer: *Pick a real but recoverable mistake, own it plainly, show how you fixed it and corrected the record, and end with a concrete process change. Avoid fake mistakes like "I work too hard".*

Q12. What is the hardest technical issue you have faced, and how did you solve it?

In simple words: They want to see your problem-solving method: how you narrow down an unclear problem step by step until you prove the cause.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - In one of our staging environments we had deployed Grafana Beyla (eBPF auto-instrumentation) for distributed tracing, but the tracing backend was receiving no spans. Nothing crashed, it just silently produced no data.
- **Task** - Find why tracing was not working so we could use it as a model for production.

- **Action** - I checked each hop in order: were the Beyla pods running, were they discovering services, could they reach the collector endpoint. The logs pointed to the exporter endpoint. The cluster is IPv6, and the collector address was not written in the bracketed IPv6 format that URLs require, so the export silently failed. I fixed the endpoint format and redeployed.
- **Result** - Spans started flowing, and I confirmed thousands of spans (about 8,000) arriving in the backend.
- **Learning** - For silent failures, walk the data path hop by hop and check each boundary; and IPv6 clusters have their own gotchas, like brackets in URLs.

What to say to the interviewer: *Choose a problem where the cause was not obvious, then show the hop-by-hop method you used to isolate it and the proof that your fix worked. The method matters more than the tool.*

Q13. Tell me about a time you had to deliver something quickly without all the time or resources you needed.

Also asked as: How do you handle tight deadlines?

In simple words: They want to see prioritization: you found the smallest solution that solved the real risk, delivered it, and did not cut corners on safety.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - Our planned migration to a new on-call service was delayed, which meant our phone/paging path was not dependable. We still needed production alerts reaching the team immediately.
- **Task** - Keep critical alerts flowing without new paid tools or waiting for the migration.
- **Action** - I first checked what we already had, and found we could send alerts straight to Microsoft Teams. I built small Lambda relay functions for CloudWatch alarms and pointed Alertmanager's Teams configuration directly at the channels. I kept the Lambdas on minimal permissions, took a backup of the old routes so we could restore later, and tested every path.
- **Result** - Built and tested quickly, at no extra cost, with no dependency on the delayed tool.
- **Learning** - Under time pressure, reuse what already exists, deliver the simplest thing that covers the risk, and keep a rollback path.

What to say to the interviewer: *Show that you prioritized the real risk, reused existing tools, delivered a simple tested solution with a rollback plan, and communicated trade-offs. Speed without testing is not the story they want.*

Q14. What would you do if you had to work on a tool or technology you have never used? Share a time you picked up a new tool quickly.

Also asked as: How do you learn a new tool quickly?

In simple words: Technology changes every year. They want to see a learning method: official docs, a small lab, then a real but low-risk task, and asking for help early.

Detailed answer

My approach:

1. Understand the problem the tool solves and the basic concepts (official docs, quickstart).
2. Build a tiny working lab in a non-production place.
3. Apply it to a real small task, and compare my work with best practices.
4. Ask experienced teammates early; document what I learned for the team.

Sample answer - adapt to your own words (STAR):

- **Situation** - We needed monthly SLA dashboards in Grafana with data from the Zabbix SLA API, which our Grafana did not show natively.
- **Task** - Learn a way to pull that API data into Grafana.
- **Action** - I learned the Grafana Infinity data source (querying JSON APIs), tested queries on one small panel, understood its parser options and field mapping gotchas, then built the dashboards through the Grafana API and saved the build scripts so they can be regenerated.
- **Result** - Live SLA dashboards for the environments, repeatable instead of hand-made.

What to say to the interviewer: *Describe your learning loop - concepts, small lab, real low-risk task, ask and document - and back it with one real example where you became productive quickly. That shows you can handle any tool they use.*

Q15. How would you convince your team or manager to adopt a new tool or process?

Also asked as: How do you convince stakeholders?

In simple words: People adopt change when they see their own problem solved with low risk. Bring data, a small proof, and a way back if it fails.

Detailed answer

Steps:

1. **Start with the problem, not the tool** - with numbers: time wasted, incidents missed, money spent.
2. **Show options** - including "do nothing", with cost and effort for each.
3. **Small proof of concept** - on one environment or one team, with success criteria agreed in advance.
4. **Address risks** - cost, learning curve, lock-in, rollback plan.
5. **Share results and roll out gradually**, with documentation.

Sample answer - adapt to your own words (STAR):

- **Situation** - Alert delivery depended on a paid external on-call tool that had become unreliable for us.
- **Action** - I compared options and proposed a Teams-direct design at no extra cost using Lambda relays and Alertmanager's native Teams integration, built it, and showed test alerts arriving in the real channels, with a backup of the old configuration for rollback.
- **Result** - The team accepted it because it solved the immediate problem, cost nothing and was reversible. Similarly, my observability audit presented monthly cost waste with exact items, which made the cleanup decision easy for the owners.

What to say to the interviewer: *I lead with the problem and data, offer options with costs, prove the idea on a small scale with agreed success criteria, and include a rollback plan. People say yes to low-risk, proven improvements.*

Q16. How do you handle conflict or disagreement within your team?

Also asked as: Googliness / culture fit questions.

In simple words: They want to know you disagree respectfully, focus on facts and the shared goal, and can commit to a decision even if it was not your idea.

Detailed answer

Approach:

- Talk directly and privately first, and listen to understand their reasons.
- Move from opinions to evidence: logs, metrics, a quick test.
- Focus on the shared goal (production stability, the customer, the deadline).
- If still stuck, agree on criteria and escalate together to the lead, not around each other.
- Once decided, "disagree and commit" and support the decision fully.

About "Googliness" and culture-fit rounds: they test humility (admitting mistakes, giving credit), collaboration (helping others succeed), doing the right thing when nobody is watching (for example refusing to use pasted credentials or skip a check), and comfort with ambiguity (making progress when requirements are unclear). Answer with real examples, not adjectives.

Example: Sample answer - adapt to your own words:

```
When several SLA breaches happened on the same day, the first explanation given was
a single common cause for all of them. I felt some did not fit, so instead of
arguing, I checked each breach against the activity logs and timelines. It turned
out most matched the common cause, but one was a separate new issue. Presenting
evidence per breach made it a fact discussion rather than a disagreement, and the
report ended up more accurate.
```

What to say to the interviewer: *I keep disagreements private, respectful and evidence-based, anchored on the shared goal, and escalate together only if needed. Once a decision is made I commit to it fully.*

Q17. How do you work across teams (developers, QA, platform, business)?

In simple words: DevOps is a bridge. They want to hear that you communicate clearly, speak each team's language, and make shared information easy to find.

Detailed answer

- **Shared visibility** - dashboards and reports that every team can read (for example SLA dashboards per project).
- **Clear tickets** - Jira stories with context, evidence and expected outcome, assigned to the right owner.
- **Speak their language** - developers want logs, traces and exact errors; managers want impact, downtime minutes and risk; clients want availability and root causes.

- **Proactive communication** - during incidents, regular short updates; after, a shared RCA.
- **Make it easy** - small improvements for other teams, such as adding a pod filter dropdown to the shared Loki logs dashboard so developers can find their pod's logs quickly.

> ***What to say to the interviewer:*** *I make information shared and self-service through dashboards and clear tickets, tailor the message to each audience, and communicate early during incidents. Small improvements that save other teams time build trust quickly.*

Q18. Give an example of taking ownership beyond your assigned task.

> **In simple words:** Ownership means you do not stop at "my part is done". If you see a risk, you raise it or fix it, and you follow through until it is closed.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - A ticket said a database health probe had been deployed to all four environments.
- **Task** - My task was only a related monitoring item, but I noticed alerts that should exist for two environments were never firing.
- **Action** - I checked the servers themselves and found the probe was deployed only on two environments; the other two had nothing, including the environment whose outage had created the ticket in the first place. I documented the evidence and raised it so the gap could be closed.
- **Result** - The team got an accurate picture of coverage instead of a false sense of safety.

Other small ownership habits: verifying that alerts really arrive, flagging cost waste found during audits, and correcting records when something turns out different than reported.

> ***What to say to the interviewer:*** *Share a time you went beyond the ticket to verify reality, found a real gap, and followed through until it was fixed or clearly handed over. Ownership is about outcomes, not task boundaries.*

Q19. Can you design, secure, scale, troubleshoot and own production systems?

> **In simple words:** This is a confidence question. Answer with a short, structured "yes, here is how" across each word, backed by one real example each - and be honest about what you are still growing.

Detailed answer

Structure by the five words:

- **Design** - I think about failure first: redundancy, alert paths that do not depend on one tool, and simple, low-cost designs (for example a Teams-direct alerting architecture).
- **Secure** - least privilege IAM, roles instead of stored keys, not pasting credentials in chat or code, secrets in a manager, and flagging open security groups.
- **Scale** - automation over manual work (scheduled reports and checks across four environments), and understanding autoscaling with HPA/KEDA and Karpenter.
- **Troubleshoot** - hop-by-hop, evidence-based root cause (the empty alert template issue, the IPv6 tracing endpoint bug, a hung OS causing a 504).

- **Own** - on-call style ownership of monitoring for four production environments, SLA reports explaining every minute of downtime, and follow-up until issues are closed.

Be honest about the gap: "I am strongest in observability and incident response; I am actively building depth in Terraform and CI/CD with hands-on labs."

> ***What to say to the interviewer:*** *Yes - and I would answer each word with a real example: failure-first design, least-privilege security, automation for scale, evidence-based troubleshooting and end-to-end ownership of monitoring for four production environments. I would also say clearly which area I am still deepening, because that honesty builds trust.*

Shell Scripting: 24 Real Interview Scripts

Basic

Q1. Write a script to clean files in /tmp that are older than 7 days.

> **In simple words:** /tmp is like a desk that everyone dumps papers on. This script throws away papers nobody has touched for a week, so the desk (disk) never overflows.

Detailed answer

- `find` walks the directory and selects only regular files whose modification time is older than 7 days (`-mtime +7`).
- A **dry run** mode prints what would be deleted before anything is removed - interviewers love this.
- `-xdev` stops find from crossing into other mounted filesystems; `-mindepth 1` protects the /tmp directory itself.
- Empty directories left behind are removed in a second pass.

Example:

```
#!/usr/bin/env bash
# clean_tmp.sh - delete files older than N days (default 7) from a directory
# Usage: clean_tmp.sh [-n] [dir] [days]     -n = dry run
set -euo pipefail

DRY_RUN=0
if [[ "${1:-}" == "-n" ]]; then DRY_RUN=1; shift; fi
DIR="${1:-/tmp}"
DAYS="${2:-7}"

[[ -d "$DIR" ]] || { echo "ERROR: $DIR is not a directory" >&2; exit 1; }
[[ "$DAYS" =~ ^[0-9]+$ ]] || { echo "ERROR: days must be an integer" >&2; exit 1; }
[[ "$DIR" == "/" ]] && { echo "ERROR: refusing to clean /" >&2; exit 1; }

if (( DRY_RUN )); then
  echo "Dry run - files that would be deleted:"
  find "$DIR" -xdev -mindepth 1 -type f -mtime +"$DAYS" -print
else
  count=$(find "$DIR" -xdev -mindepth 1 -type f -mtime +"$DAYS" -print -delete | wc -l)
  # remove empty directories left behind (but never DIR itself)
  find "$DIR" -xdev -mindepth 1 -type d -empty -mtime +"$DAYS" -delete
  logger -t clean_tmp "Deleted $count files older than $DAYS days from $DIR"
  echo "Deleted $count files"
fi
```

```
# cron: every day at 03:00
0 3 * * * /usr/local/bin/clean_tmp.sh /tmp 7 >/dev/null 2>&1
```

Watch out: Files still held open by a running process will vanish from the directory but not free space until the process closes them. On systemd machines, `systemd-tmpfiles` (`/etc/tmpfiles.d/`) already does this job - mention that as the built-in alternative.

> ***What to say to the interviewer:*** *I use* `find` *with* `-type f -mtime +7 -delete`*, add* `-xdev` *so it never crosses into other mounts, and always support a dry-run flag so I can see what will be*

removed first. I validate the inputs, refuse dangerous paths like `/`, log the count, and schedule it with cron - or just use systemd-tmpfiles, which is built for this.

Q2. Write a script to validate the IP addresses in a text file.

In simple words: Like checking that every phone number in a list has the right number of digits. An IPv4 address must be four numbers from 0 to 255 separated by dots - anything else is flagged.

Detailed answer

- A regex alone (`[0-9]{1,3}`) is not enough: it accepts `999.1.1.1`. So we check the **shape** with a regex, then check each **octet** is 0-255.
- Read the file line by line with `while IFS= read -r`, skipping blank lines and comments.
- Leading zeros like `010` are rejected, because some tools read them as octal.
- Exit code is non-zero if any invalid line was found, so the script can gate a CI job.

Example:

```
#!/usr/bin/env bash
# validate_ips.sh - report valid/invalid IPv4 addresses, one per line
set -uo pipefail

FILE="${1:?Usage: $0 <file>}"
[[ -r "$FILE" ]] || { echo "ERROR: cannot read $FILE" >&2; exit 2; }

is_valid_ip() {
  local ip="$1" octet
  [[ "$ip" =~ ^([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})\.([0-9]{1,3})$ ]] || return 1
  for octet in "${BASH_REMATCH[@]:1}"; do
    [[ "$octet" =~ ^(0|[1-9][0-9]*)$ ]] || return 1    # no leading zeros
    (( octet <= 255 )) || return 1
  done
  return 0
}

invalid=0; lineno=0
while IFS= read -r line || [[ -n "$line" ]]; do
  lineno=$((lineno + 1))
  line="${line//[[:space:]]/}"                         # trim spaces / CR
  [[ -z "$line" || "$line" == \#* ]] && continue
  if is_valid_ip "$line"; then
    echo "VALID    $line"
  else
    echo "INVALID  $line (line $lineno)"
    invalid=$((invalid + 1))
  fi
done < "$FILE"

echo "Invalid entries: $invalid"
(( invalid == 0 ))
```

```
$ ./validate_ips.sh ips.txt
VALID    10.0.1.15
INVALID  256.1.1.1 (line 2)
INVALID  192.168.1 (line 3)
VALID    8.8.8.8
Invalid entries: 2
```

Watch out: `|| [[ -n "$line" ]]` handles a last line with no trailing newline. Windows files carry `\r` - the whitespace strip removes it. For IPv6 or CIDR, mention `ipcalc` or a proper library rather than a monster regex.

> ***What to say to the interviewer:*** *I read the file line by line, match the four-octet shape with a Bash regex, then check every octet is between 0 and 255 with no leading zeros, because a regex alone would accept 999. The script prints valid and invalid lines with line numbers and exits non-zero if anything is invalid, so it can fail a pipeline.*

Q3. Write a script that reads server IPs from a file and pings each one.

> **In simple words:** A roll call for servers - read the list of names and shout each one; whoever answers is present (UP), whoever stays silent is marked absent (DOWN).

Detailed answer

- `ping -c 2 -W 2` sends 2 packets and waits at most 2 seconds, so a dead host does not hang the script.
- The sequential loop is simple; the **parallel** version uses `xargs -P 20` to ping 20 hosts at once - huge time saver with 500 servers.
- Results go to stdout and a summary; exit code = number of down hosts (capped), useful for monitoring.

Example:

```
#!/usr/bin/env bash
# ping_hosts.sh - ping every host listed in a file (sequential or parallel)
# Usage: ping_hosts.sh hosts.txt [parallelism]
set -uo pipefail

FILE="${1:?Usage: $0 <hosts_file> [parallel]}"
PARALLEL="${2:-1}"
[[ -r "$FILE" ]] || { echo "ERROR: cannot read $FILE" >&2; exit 2; }

check_host() {
  local host="$1"
  if ping -c 2 -W 2 "$host" >/dev/null 2>&1; then
    echo "UP    $host"
  else
    echo "DOWN  $host"
  fi
}
export -f check_host

grep -Ev '^[[:space:]]*(#|$)' "$FILE" \
  | xargs -r -n1 -P "$PARALLEL" bash -c 'check_host "$1"' _ \
  | tee /tmp/ping_results.txt

down=$(grep -c '^DOWN' /tmp/ping_results.txt || true)
echo "Summary: $down host(s) down"
exit $(( down > 125 ? 125 : down ))
```

```
$ ./ping_hosts.sh servers.txt 20
UP    10.0.1.10
DOWN  10.0.1.11
UP    10.0.2.20
Summary: 1 host(s) down
```

Watch out: Many cloud hosts block ICMP in the security group, so "no ping" does not always mean "down" - for real health use a TCP check like `nc -z -w2 host 22`. Also `-W` is Linux ping syntax; macOS uses different flags.

> ***What to say to the interviewer:*** *I read the file skipping comments and blanks, ping each host with a count and a timeout so nothing hangs, and print UP or DOWN. For big lists I run it in parallel with* `xargs -P`*, and I point out that ICMP can be blocked, so for real health checks I would use a TCP port check with* `nc`*.*

Q4. Write a script to parse a log file and count failed login attempts.

> **In simple words:** Like a security guard reading the visitor book and counting how many times each stranger tried the wrong key - the top names are probably attackers.

Detailed answer

- SSH failures appear in `/var/log/auth.log` (Debian/Ubuntu) or `/var/log/secure` (RHEL) as `Failed password for ... from <IP>`.
- `grep` finds the lines, awk/`grep -oE` extracts the IP, then `sort | uniq -c | sort -nr` counts and ranks them.
- Also report the targeted usernames, and flag IPs above a threshold (these are candidates for fail2ban or a WAF/SG block).

Example:

```
#!/usr/bin/env bash
# failed_logins.sh - count failed SSH logins per IP and per user
set -euo pipefail

LOG="${1:-}"
if [[ -z "$LOG" ]]; then
  [[ -f /var/log/auth.log ]] && LOG=/var/log/auth.log || LOG=/var/log/secure
fi
THRESHOLD="${2:-10}"
[[ -r "$LOG" ]] || { echo "ERROR: cannot read $LOG (need sudo?)" >&2; exit 1; }

total=$(grep -c 'Failed password' "$LOG" || true)
echo "Total failed password attempts: $total"

echo; echo "Top source IPs:"
grep 'Failed password' "$LOG" \
  | grep -oE 'from [0-9]{1,3}(\.[0-9]{1,3}){3}' | awk '{print $2}' \
  | sort | uniq -c | sort -nr | head -10

echo; echo "Top targeted users:"
grep 'Failed password' "$LOG" \
  | sed -E 's/.*Failed password for (invalid user )?([^ ]+) from.*/\2/' \
  | sort | uniq -c | sort -nr | head -10

echo; echo "IPs over threshold ($THRESHOLD):"
grep 'Failed password' "$LOG" \
  | grep -oE 'from [0-9]{1,3}(\.[0-9]{1,3}){3}' | awk '{print $2}' \
  | sort | uniq -c | awk -v t="$THRESHOLD" '$1 > t {print $2, "(" $1 " attempts)"}'
```

```
# systemd-only systems keep logs in the journal instead of a file:
journalctl -u ssh --since "24 hours ago" | grep 'Failed password' | wc -l
```

Watch out: Rotated logs (`auth.log.1`, `auth.log.2.gz`) hold older attempts - use `zgrep` to include them. The sed handles the "invalid user" variant, which otherwise breaks naive `awk '{print $9}'` column counting.

> ***What to say to the interviewer:*** *I grep for "Failed password" in auth.log or secure, extract the source IP with* `grep -o`*, and use* `sort | uniq -c | sort -nr` *to rank attackers, plus a second pass for targeted usernames. Anything over a threshold gets flagged for fail2ban or a firewall block, and I use zgrep or journalctl when logs are rotated or in the journal.*

Q5. Write a script to report all users with UID greater than 1000.

> **In simple words:** Linux keeps a guest list in `/etc/passwd`. System accounts get low numbers; real people usually start at 1000. This script prints the real people.

Detailed answer

- `/etc/passwd` is colon-separated: `name:x:UID:GID:comment:home:shell`, so `awk -F:` reads field 3 as the UID.
- The question says "> 1000", but normal users start **at** 1000 on most distros (`UID_MIN` in `/etc/login.defs`), so use `>= 1000` and say why.
- Exclude `nobody` (UID 65534), which is a system account despite the big number.
- `getent passwd` is better than reading the file because it also includes LDAP/SSSD users.

Example:

```
#!/usr/bin/env bash
# human_users.sh - list regular (human) user accounts
set -euo pipefail

MIN_UID=$(awk '/^UID_MIN/ {print $2}' /etc/login.defs 2>/dev/null || true)
MIN_UID="${MIN_UID:-1000}"

printf "%-15s %-7s %-25s %s\n" "USER" "UID" "HOME" "SHELL"
getent passwd | awk -F: -v min="$MIN_UID" \
  '$3 >= min && $3 != 65534 { printf "%-15s %-7s %-25s %s\n", $1, $3, $6, $7 }' \
  | sort -k2 -n
```

```
$ ./human_users.sh
USER            UID     HOME                      SHELL
priyanka        1000    /home/priyanka            /bin/bash
deploy          1001    /home/deploy              /bin/bash

# The literal one-liner an interviewer expects:
awk -F: '$3 > 1000 {print $1, $3}' /etc/passwd
```

Watch out: Mention the > vs >= detail - it shows you know UID_MIN. Accounts with `/usr/sbin/nologin` shells are service users even with high UIDs; you can filter on the shell field too.

> ***What to say to the interviewer:*** *The quick answer is* `awk -F: '$3 > 1000' /etc/passwd`*, but I use* `getent passwd` *so directory users are included, read UID_MIN from login.defs because human users start at 1000, and exclude nobody at 65534. I print user, UID, home and shell in a table sorted by UID.*

Q6. Write a script to compare two directories and show the differences.

In simple words: Like comparing two photo albums side by side - which photos are only in the left album, which only in the right, and which have the same name but a different picture.

Detailed answer

- `diff -rq dir1 dir2` is the one-liner: recursive, quiet (names only).
- `rsync -rcn --delete --itemize-changes src/ dst/` is a dry run that shows exactly what would change if you synced them - `-c` compares checksums, not just size and time.
- For large trees, a checksum manifest (`sha256sum`) is reliable and can be compared across servers.

Example:

```
#!/usr/bin/env bash
# dircompare.sh - compare two directory trees by content
set -euo pipefail

A="${1:?Usage: $0 <dir1> <dir2>}"
B="${2:?Usage: $0 <dir1> <dir2>}"
for d in "$A" "$B"; do [[ -d "$d" ]] || { echo "ERROR: $d not a dir" >&2; exit 2; }; done

manifest() {   # relative path + sha256 for every file, NUL-safe
  (cd "$1" && find . -type f -print0 | sort -z | xargs -0 -r sha256sum)
}

tmpA=$(mktemp); tmpB=$(mktemp)
trap 'rm -f "$tmpA" "$tmpB"' EXIT
manifest "$A" > "$tmpA"
manifest "$B" > "$tmpB"

echo "== Only in $A =="
comm -23 <(awk '{print $2}' "$tmpA") <(awk '{print $2}' "$tmpB")
echo "== Only in $B =="
comm -13 <(awk '{print $2}' "$tmpA") <(awk '{print $2}' "$tmpB")
echo "== Same name, different content =="
join -j 2 <(sort -k2 "$tmpA") <(sort -k2 "$tmpB") | awk '$2 != $3 {print $1}'

if cmp -s "$tmpA" "$tmpB"; then echo "RESULT: identical"; exit 0; fi
echo "RESULT: different"; exit 1
```

```
# Quick alternatives
diff -rq /srv/app/releases/v1 /srv/app/releases/v2
rsync -rcn --delete --itemize-changes /data/src/ backup-host:/data/dst/
```

Watch out: The manifest uses `awk '{print $2}'`, which breaks on filenames containing spaces - for those, rely on `diff -rq` or `rsync -n`. Always put the trailing slash on rsync source dirs (`src/`) or you compare the wrong level.

What to say to the interviewer: *For a quick check I use* `diff -rq` *or an* `rsync -rcn --delete` *dry run, which compares by checksum. For a full report I build a sha256 manifest of each tree and use* `comm` *and* `join` *to list files only on one side and files whose content differs, and the exit code tells a pipeline whether they match.*

Q7. Write a script to validate YAML and JSON files (for example in a CI pipeline).

In simple words: A spell-checker for config files. One missing bracket or a bad indent in YAML can take down a deployment, so we check every file before it ships.

Detailed answer

- JSON: `jq empty file.json` parses the file and prints nothing if it is valid; non-zero exit if broken.
- YAML: `yq eval '.' file.yaml >/dev/null` checks syntax; `yamllint` also checks style (indentation, duplicate keys, trailing spaces).
- Loop over all matching files with `find -print0`, collect failures, exit non-zero if any failed - this makes it a CI gate.

Example:

```
#!/usr/bin/env bash
# validate_configs.sh - syntax-check all JSON and YAML files under a directory
set -uo pipefail

ROOT="${1:-.}"
fail=0; checked=0

for tool in jq yq; do
  command -v "$tool" >/dev/null || { echo "ERROR: $tool not installed" >&2; exit 2; }
done

while IFS= read -r -d '' f; do
  checked=$((checked + 1))
  case "$f" in
    *.json)       jq empty "$f" 2>/tmp/val_err ;;
    *.yaml|*.yml) yq eval '.' "$f" >/dev/null 2>/tmp/val_err ;;
  esac
  if [[ $? -ne 0 ]]; then
    echo "FAIL  $f: $(head -1 /tmp/val_err)"
    fail=$((fail + 1))
  else
    echo "OK    $f"
  fi
done < <(find "$ROOT" -type f \( -name '*.json' -o -name '*.yaml' -o -name '*.yml' \) \
          -not -path '*/node_modules/*' -not -path '*/.git/*' -print0)

if command -v yamllint >/dev/null; then
  yamllint -d relaxed "$ROOT" || fail=$((fail + 1))
fi

echo "Checked $checked files, $fail failure(s)"
(( fail == 0 ))
```

```
# GitHub Actions step
- name: Validate configs
  run: ./scripts/validate_configs.sh k8s/
```

Watch out: Valid YAML is not the same as a valid Kubernetes manifest - add `kubeconform` or `kubectl apply --dry-run=server` for schema checks. Helm templates (`{{ }}`) are not valid YAML until rendered, so run `helm template` first or exclude chart templates.

What to say to the interviewer: *I loop over every JSON and YAML file with a NUL-safe find, validate JSON with* `jq empty` *and YAML with* `yq`*, optionally run yamllint for style, and exit non-zero*

if anything fails so the CI job blocks the merge. For Kubernetes I go one step further with kubeconform, because syntactically valid YAML can still be an invalid manifest.

Advanced

Q8. Write a script to back up a MySQL database daily with a timestamp.

In simple words: A daily photograph of the database, labelled with the date, kept for a set number of days - if something breaks, you restore yesterday's photo.

Detailed answer

- `mysqldump --single-transaction` takes a consistent snapshot of InnoDB tables **without locking** them - important for production.
- Credentials come from a protected `~/.my.cnf` (mode 600), **never** `-pPassword` on the command line, which shows up in `ps` and shell history.
- Output is piped to `gzip` with a timestamped filename; `set -o pipefail` makes the script fail if mysqldump fails mid-pipe.
- Verify the backup (gzip integrity + dump completion marker), apply retention, and ideally copy it off-server (S3).

Example:

```
#!/usr/bin/env bash
# mysql_backup.sh - daily compressed MySQL backup with retention
set -euo pipefail

DB="${1:?Usage: $0 <database> [retention_days]}"
RETENTION="${2:-7}"
BACKUP_DIR="/var/backups/mysql"
TS=$(date +%Y%m%d_%H%M%S)
OUT="$BACKUP_DIR/${DB}_${TS}.sql.gz"
DEFAULTS="/root/.my.cnf"            # [client] user=... password=... (chmod 600)

mkdir -p "$BACKUP_DIR"
umask 077                           # backups readable by owner only

log() { logger -t mysql_backup "$*"; echo "$(date '+%F %T') $*"; }

log "Starting backup of $DB -> $OUT"
mysqldump --defaults-extra-file="$DEFAULTS" \
  --single-transaction --quick --routines --triggers --events \
  "$DB" | gzip > "$OUT"

# verify: archive not corrupt and dump finished
gzip -t "$OUT"
zcat "$OUT" | tail -1 | grep -q 'Dump completed' \
  || { log "ERROR: dump incomplete"; rm -f "$OUT"; exit 1; }

log "Backup OK: $(du -h "$OUT" | cut -f1)"

# optional off-site copy (EC2 IAM role provides credentials)
# aws s3 cp "$OUT" "s3://my-db-backups/$DB/" --sse aws:kms

find "$BACKUP_DIR" -name "${DB}_*.sql.gz" -type f -mtime +"$RETENTION" -delete
log "Removed backups older than $RETENTION days"
```

```
# cron: every day at 01:30
30 1 * * * /usr/local/bin/mysql_backup.sh appdb 7 >> /var/log/mysql_backup.log 2>&1

# restore
zcat /var/backups/mysql/appdb_20260924_013000.sql.gz | mysql appdb
```

Watch out: A backup you have never restored is not a backup - test restores regularly. `--single-transaction` only gives consistency for InnoDB; MyISAM tables still need locks. On RDS, prefer automated snapshots and PITR, and use a dump only for logical exports.

> ***What to say to the interviewer:*** *I use* `mysqldump --single-transaction` *so InnoDB is dumped consistently without locking, read credentials from a chmod-600 .my.cnf instead of the command line, and pipe to gzip with a timestamped filename under pipefail. Then I verify the archive, apply retention with find, copy it to S3 with encryption, and schedule it in cron - and I test restores regularly.*

Q9. Write a script to SSH into multiple servers and run a command on each.

> **In simple words:** Instead of logging into 50 servers one by one to type the same command, the script knocks on every door for you and collects the answers in one place.

Detailed answer

- `-o BatchMode=yes` makes SSH fail immediately instead of prompting for a password - essential for automation.
- `-o ConnectTimeout=5` stops dead hosts from hanging the run; `StrictHostKeyChecking=accept-new` accepts new hosts but still rejects changed keys.
- Run in parallel with background jobs or `xargs -P`, prefix every output line with the hostname, and collect a per-host exit status.
- Mention that for real fleets **Ansible ad-hoc** (`ansible all -m shell -a "uptime"`) is the better tool.

Example:

```
#!/usr/bin/env bash
# multi_ssh.sh - run a command on many hosts in parallel
# Usage: multi_ssh.sh hosts.txt "uptime" [parallel]
set -uo pipefail

HOSTS="${1:?Usage: $0 <hosts_file> <command> [parallel]}"
CMD="${2:?Usage: $0 <hosts_file> <command> [parallel]}"
PARALLEL="${3:-10}"
SSH_USER="${SSH_USER:-ec2-user}"
LOGDIR=$(mktemp -d)

run_on_host() {
  local host="$1"
  if ssh -n -o BatchMode=yes -o ConnectTimeout=5 \
         -o StrictHostKeyChecking=accept-new \
         "$SSH_USER@$host" "$CMD" > "$LOGDIR/$host.out" 2>&1; then
    echo "OK $host" >> "$LOGDIR/status"
  else
    echo "FAIL $host (exit $?)" >> "$LOGDIR/status"
  fi
  sed "s/^/[$host] /" "$LOGDIR/$host.out"
}
export -f run_on_host
export CMD SSH_USER LOGDIR

grep -Ev '^[[:space:]]*(#|$)' "$HOSTS" | xargs -r -n1 -P "$PARALLEL" bash -c 'run_on_host "$1"' _

echo "---- Summary ----"
sort "$LOGDIR/status"
failed=$(grep -c '^FAIL' "$LOGDIR/status" || true)
rm -rf "$LOGDIR"
exit $(( failed > 0 ? 1 : 0 ))
```

```
$ SSH_USER=ubuntu ./multi_ssh.sh web_servers.txt "df -h / | tail -1" 20
[10.0.1.10] /dev/nvme0n1p1   30G   21G  9.0G  70% /
[10.0.1.11] /dev/nvme0n1p1   30G   28G  2.0G  94% /
---- Summary ----
OK 10.0.1.10
OK 10.0.1.11

# The better tool for fleets:
ansible webservers -i inventory.ini -m shell -a "df -h /" -f 20
```

Watch out: `ssh -n` stops SSH from eating the rest of the host list from stdin - a classic bug in `while read` loops. Never run destructive commands across all hosts without a dry run or a canary host first. On AWS, SSM Run Command does this without opening port 22 at all.

> ***What to say to the interviewer:*** *I read the host list, run SSH with BatchMode and ConnectTimeout so it never prompts or hangs, execute in parallel with* `xargs -P`*, prefix each output line with the hostname and print a per-host OK/FAIL summary. For a real fleet I would use Ansible ad-hoc commands or AWS SSM Run Command, which handle inventory, parallelism and auditing for me.*

Q10. Write a script to archive application logs to S3.

> **In simple words:** Old logs are like old files in an office drawer - you pack them into a labelled box, send the box to cheap long-term storage (S3), and only then clear the drawer.

Detailed answer

- Find log files older than N days, bundle them into a timestamped `tar.gz`.
- Upload with `aws s3 cp --sse aws:kms` under a date/host prefix, e.g. `s3://bucket/app/2026/09/24/host/`.
- **Delete local files only after the upload succeeds** (check the exit code, optionally compare sizes).
- Credentials come from the **EC2 IAM instance role**, not access keys in the script. An S3 lifecycle rule moves old archives to Glacier.

Example:

```
#!/usr/bin/env bash
# archive_logs_s3.sh - compress logs older than N days and ship them to S3
set -euo pipefail

LOG_DIR="${1:-/var/log/myapp}"
BUCKET="${2:?Usage: $0 <log_dir> <s3_bucket> [days]}"
DAYS="${3:-3}"
HOST=$(hostname -s)
TS=$(date +%Y%m%d_%H%M%S)
PREFIX="myapp/$(date +%Y/%m/%d)/$HOST"
ARCHIVE="/tmp/${HOST}_logs_${TS}.tar.gz"
LIST=$(mktemp)
trap 'rm -f "$LIST" "$ARCHIVE"' EXIT

find "$LOG_DIR" -type f -name '*.log*' -mtime +"$DAYS" -print0 > "$LIST"
if [[ ! -s "$LIST" ]]; then echo "Nothing to archive"; exit 0; fi

tar -czf "$ARCHIVE" --null -T "$LIST"
echo "Created $ARCHIVE ($(du -h "$ARCHIVE" | cut -f1))"

if aws s3 cp "$ARCHIVE" "s3://$BUCKET/$PREFIX/" --sse aws:kms --only-show-errors; then
  # confirm the object exists before deleting anything locally
  aws s3api head-object --bucket "$BUCKET" --key "$PREFIX/$(basename "$ARCHIVE")" >/dev/null
  xargs -0 -r rm -f < "$LIST"
  logger -t archive_logs "Archived and removed $(tr -cd '\0' < "$LIST" | wc -c) files"
else
  echo "ERROR: upload failed - local logs kept" >&2
  exit 1
fi
```

```
# cron: daily at 02:00
0 2 * * * /usr/local/bin/archive_logs_s3.sh /var/log/myapp my-log-archive 3
```

```
{ "Effect": "Allow", "Action": ["s3:PutObject"],
  "Resource": "arn:aws:s3:::my-log-archive/myapp/*" }
```

Watch out: Do not archive the file the app is currently writing to - coordinate with logrotate (rotate first, archive rotated files). The IAM role needs only `s3:PutObject` on that prefix, plus `kms:GenerateDataKey` for SSE-KMS. For continuous shipping, a log agent (Fluent Bit / CloudWatch agent) is better than a cron script.

> ***What to say to the interviewer:*** *I find logs older than N days with a NUL-safe find, tar and gzip them with a host and timestamp name, upload with* `aws s3 cp --sse aws:kms` *under a date/host prefix using the instance role, and delete the local files only after I confirm the object exists in S3. A lifecycle rule then tiers the archives to Glacier to keep cost low.*

Q11. Write a script to monitor SSL certificate expiry and alert before it expires.

> **In simple words:** Like a reminder for your passport's expiry date - the script checks each website's certificate and warns you 30 days before it runs out, so customers never see a "Not Secure" page.

Detailed answer

- `openssl s_client -connect host:443 -servername host` fetches the live certificate (`-servername` sends SNI, required for most modern hosts).
- `openssl x509 -noout -enddate` prints `notAfter=...`; convert it to epoch seconds with `date -d` and compute days left.
- Loop over a list of domains; warn under 30 days, critical under 7, and post to a Teams/Slack webhook.

Example:

```
#!/usr/bin/env bash
# ssl_expiry.sh - check certificate expiry for a list of domains
# Usage: ssl_expiry.sh domains.txt   (lines: domain[:port])
set -uo pipefail

FILE="${1:?Usage: $0 <domains_file>}"
WARN_DAYS="${WARN_DAYS:-30}"
CRIT_DAYS="${CRIT_DAYS:-7}"
WEBHOOK="${WEBHOOK_URL:-}"          # Teams/Slack incoming webhook (optional)
status=0

alert() {
  echo "$1"
  [[ -n "$WEBHOOK" ]] && curl -s -m 10 -H 'Content-Type: application/json' \
    -d "{\"text\": \"$1\"}" "$WEBHOOK" >/dev/null
}

while IFS= read -r entry; do
  [[ -z "$entry" || "$entry" == \#* ]] && continue
  host="${entry%%:*}"; port="${entry##*:}"; [[ "$port" == "$host" ]] && port=443

  end=$(echo | timeout 10 openssl s_client -connect "$host:$port" -servername "$host" \
        2>/dev/null | openssl x509 -noout -enddate 2>/dev/null | cut -d= -f2)
  if [[ -z "$end" ]]; then
    alert "UNKNOWN: could not read certificate for $host:$port"; status=2; continue
  fi

  days=$(( ( $(date -d "$end" +%s) - $(date +%s) ) / 86400 ))
  if (( days < CRIT_DAYS )); then
    alert "CRITICAL: $host cert expires in $days days ($end)"; status=2
  elif (( days < WARN_DAYS )); then
    alert "WARNING: $host cert expires in $days days ($end)"; (( status < 1 )) && status=1
  else
    echo "OK: $host - $days days left"
  fi
done < "$FILE"
exit "$status"
```

```
$ ./ssl_expiry.sh domains.txt
OK: api.example.com - 74 days left
WARNING: portal.example.com cert expires in 18 days (Oct 12 23:59:59 2026 GMT)

# daily at 09:00
0 9 * * * WEBHOOK_URL=https://... /usr/local/bin/ssl_expiry.sh /etc/ssl_domains.txt
```

Watch out: Without `-servername` you may get the default certificate of the load balancer, not the site's. `date -d` is GNU; on macOS use `date -j -f`. ACM certificates auto-renew, but imported certificates do not - this script catches exactly those. Blackbox exporter (`probe_ssl_earliest_cert_expiry`) is the Prometheus way.

> ***What to say to the interviewer:*** *I pull the live certificate with* `openssl s_client` *using SNI, read notAfter with* `openssl x509 -enddate`*, convert it to epoch to get days left, and alert to Teams under 30 days and critically under 7, with Nagios-style exit codes. In a Prometheus setup I would use the blackbox exporter's cert expiry metric instead of a cron script.*

Q12. Write a script to create Linux users from a CSV file.

> **In simple words:** HR hands you a spreadsheet of new joiners. Instead of creating 40 accounts by hand, the script reads each row and creates the user, their group and a temporary password - and skips anyone who already exists.

Detailed answer

- CSV format: `username,full_name,group,shell`. Skip the header, validate each field.
- **Idempotent**: if the user already exists, skip it (running the script twice must not break anything).
- `useradd -m -c -s -G` creates home, comment, shell and supplementary group; `chpasswd` sets a random temporary password; `chage -d 0` forces a change at first login.
- Must run as root; log every action; write the temporary passwords to a root-only file (better: SSH keys only, no passwords).

Example:

```
#!/usr/bin/env bash
# create_users.sh - bulk create users from CSV: username,full_name,group,shell
set -euo pipefail

CSV="${1:?Usage: $0 <users.csv>}"
[[ $EUID -eq 0 ]] || { echo "ERROR: run as root" >&2; exit 1; }
[[ -r "$CSV" ]] || { echo "ERROR: cannot read $CSV" >&2; exit 1; }

CREDS="/root/new_user_passwords_$(date +%F).txt"
umask 077
created=0; skipped=0

while IFS=, read -r user name group shell || [[ -n "$user" ]]; do
  user=$(echo "$user" | tr -d '[:space:]')
  [[ -z "$user" || "$user" == "username" || "$user" == \#* ]] && continue
  shell="${shell//$'\r'/}"; shell="${shell:-/bin/bash}"

  if [[ ! "$user" =~ ^[a-z_][a-z0-9_-]{0,31}$ ]]; then
    echo "SKIP invalid username: $user"; skipped=$((skipped + 1)); continue
  fi
  if id "$user" &>/dev/null; then
    echo "SKIP exists: $user"; skipped=$((skipped + 1)); continue
  fi
  if [[ -n "$group" ]] && ! getent group "$group" >/dev/null; then
    groupadd "$group" && echo "Created group $group"
  fi

  useradd -m -c "$name" -s "$shell" ${group:+-G "$group"} "$user"
  pass=$(openssl rand -base64 12)
  echo "$user:$pass" | chpasswd
  chage -d 0 "$user"                 # force password change at first login
  echo "$user,$pass" >> "$CREDS"
  echo "CREATED $user (group: ${group:-none})"
  created=$((created + 1))
done < "$CSV"

echo "Done: $created created, $skipped skipped. Temp passwords in $CREDS"
```

```
$ cat users.csv
username,full_name,group,shell
asha,Asha Rao,devops,/bin/bash
ravi,Ravi Kumar,developers,/bin/bash
$ sudo ./create_users.sh users.csv
Created group devops
CREATED asha (group: devops)
SKIP exists: ravi
```

Watch out: Never echo passwords to the terminal or logs, and delete the credentials file after handing them over. Commas inside names break a naive CSV split - keep the format simple or quote-aware. At scale, users should come from LDAP/SSO or Ansible's `user` module, not a local script.

> ***What to say to the interviewer:*** *I read the CSV with* `IFS=,` *, skip the header, validate the username, and make it idempotent by skipping users that already exist. I create missing groups, run* `useradd -m` *with the shell and group, set a random temporary password with chpasswd and force a change at first login with* `chage -d 0`*, and log everything. In a real company I would push this into Ansible or SSO rather than local accounts.*

Q13. Write a script to automate git pull and restart a service.

In simple words: A simple self-updating server: it checks whether new code has arrived on the main branch, and only if it has, it pulls it and restarts the app - and puts the old version back if the new one fails its health check.

Detailed answer

- `git fetch` then compare local `HEAD` with `origin/main`; restart **only when there is a change**.
- `git reset --hard origin/main` (instead of `git pull`) avoids merge conflicts from local edits on the server.
- `flock` prevents two cron runs from overlapping.
- After the restart, run a health check; on failure, reset to the previous commit and restart again (rollback).

Example:

```
#!/usr/bin/env bash
# git_deploy.sh - pull latest main and restart service only when code changed
set -euo pipefail

APP_DIR="/opt/myapp"
BRANCH="main"
SERVICE="myapp"
HEALTH_URL="http://localhost:8080/health"

exec 9>/var/lock/git_deploy.lock
flock -n 9 || { echo "Another deploy is running"; exit 0; }

log() { logger -t git_deploy "$*"; echo "$(date '+%F %T') $*"; }
cd "$APP_DIR"

git fetch --quiet origin "$BRANCH"
LOCAL=$(git rev-parse HEAD)
REMOTE=$(git rev-parse "origin/$BRANCH")
if [[ "$LOCAL" == "$REMOTE" ]]; then
  log "No changes ($LOCAL)"; exit 0
fi

log "Updating $LOCAL -> $REMOTE"
git reset --hard "origin/$BRANCH"
systemctl restart "$SERVICE"

for i in {1..10}; do
  if curl -fsS -m 3 "$HEALTH_URL" >/dev/null; then
    log "Deploy OK at $REMOTE"; exit 0
  fi
  sleep 3
done

log "Health check failed - rolling back to $LOCAL"
git reset --hard "$LOCAL"
systemctl restart "$SERVICE"
exit 1
```

```
# check for new commits every 5 minutes
*/5 * * * * /usr/local/bin/git_deploy.sh >> /var/log/git_deploy.log 2>&1
```

Watch out: `reset --hard` destroys any local edits on the server - that is intentional (servers should be read-only copies), but say it out loud. Use a deploy key with read-only access. This is fine for a small internal app; for production the better pattern is a CI/CD pipeline that builds an immutable artifact, or GitOps with Argo CD.

> ***What to say to the interviewer:*** *I fetch from origin, compare HEAD with origin/main and exit early if nothing changed, then* `reset --hard` *to avoid merge conflicts, restart the service and poll a health endpoint. If the health check fails I reset to the previous commit and restart again, and I wrap the whole thing in flock so cron runs never overlap. For real production I would move this to a pipeline or GitOps.*

Q14. Write a script to generate a dynamic inventory file for Ansible.

> **In simple words:** In the cloud, servers appear and disappear all day, so a hand-written list of servers goes stale. This script asks AWS "which servers are running right now with this tag?" and builds Ansible's list fresh every time.

Detailed answer

- `aws ec2 describe-instances` filtered by tag and state `running`, with a `--query` to pull private IP and Role tag.
- Ansible dynamic inventory scripts must print JSON when called with `--list` (and handle `--host`).
- Group hosts by their `Role` tag, e.g. `web`, `db`.
- Mention the **proper way**: the `amazon.aws.aws_ec2` inventory plugin does this natively with caching and keyed groups.

Example:

```
#!/usr/bin/env bash
# ec2_inventory.sh - Ansible dynamic inventory from EC2 tags
# Usage: ansible-playbook -i ec2_inventory.sh site.yml
set -euo pipefail

ENV_TAG="${INV_ENV:-prod}"
REGION="${AWS_REGION:-ap-south-1}"

case "${1:-}" in
  --host) echo '{}'; exit 0 ;;           # hostvars are provided in _meta
  --list) ;;
  *) echo "Usage: $0 --list | --host <name>" >&2; exit 1 ;;
esac

aws ec2 describe-instances --region "$REGION" \
  --filters "Name=tag:Environment,Values=$ENV_TAG" "Name=instance-state-name,Values=running" \
  --query 'Reservations[].Instances[].{ip:PrivateIpAddress, role:Tags[?Key==`Role`]|[0].Value}' \
  --output json \
| jq '
    map(select(.ip != null) | .role = (.role // "ungrouped"))
    | reduce .[] as $i ({"_meta": {"hostvars": {}}};
        .[$i.role].hosts += [$i.ip]
        | ._meta.hostvars[$i.ip] = {"ansible_user": "ec2-user", "role": $i.role})
  '
```

```
$ ./ec2_inventory.sh --list
{
  "_meta": { "hostvars": { "10.0.1.10": { "ansible_user": "ec2-user", "role": "web" } } },
  "web": { "hosts": ["10.0.1.10", "10.0.1.11"] },
  "db":  { "hosts": ["10.0.2.20"] }
}
```

```
# The proper way - aws_ec2.yml inventory plugin
plugin: amazon.aws.aws_ec2
regions: [ap-south-1]
filters:
  tag:Environment: prod
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
compose:
  ansible_host: private_ip_address
```

Watch out: The script must be executable and output only JSON (no echo debugging on stdout). Use the instance role or an SSO profile for AWS credentials. Private IPs need network reachability from the Ansible control node (VPN/bastion/SSM).

> ***What to say to the interviewer:*** *My script calls* `aws ec2 describe-instances` *filtered on the Environment tag and running state, then uses jq to group private IPs by their Role tag into the JSON format Ansible expects for* `--list`*, with hostvars in* `_meta`*. In practice I prefer the* `amazon.aws.aws_ec2` *inventory plugin, which does the same with keyed groups and caching and no custom code.*

Q15. Write a script that checks Jenkins job status through the API and alerts if the build failed.

> **In simple words:** Instead of refreshing the Jenkins page, the script asks Jenkins directly "how did the last build go?" and sends a message to the team channel if the answer is FAILURE or UNSTABLE.

Detailed answer

- Every Jenkins page has a JSON API: `$JENKINS_URL/job/<name>/lastBuild/api/json`.
- Authenticate with **username + API token** (not the password). A CSRF crumb is **not** needed for GET requests, only for POSTs.
- `jq -r .result` gives `SUCCESS`, `FAILURE`, `UNSTABLE`, `ABORTED`, or `null` while still building.
- Loop over several jobs; alert to a Teams/Slack webhook with the build URL; exit non-zero on failure.

Example:

```
#!/usr/bin/env bash
# jenkins_status.sh - alert when the last build of any listed job failed
# Usage: JENKINS_USER=.. JENKINS_TOKEN=.. jenkins_status.sh job1 folder/job/job2
set -uo pipefail

JENKINS_URL="${JENKINS_URL:-https://jenkins.example.com}"
: "${JENKINS_USER:?set JENKINS_USER}" "${JENKINS_TOKEN:?set JENKINS_TOKEN}"
WEBHOOK="${WEBHOOK_URL:-}"
[[ $# -gt 0 ]] || { echo "Usage: $0 <job> [job...]" >&2; exit 2; }
failed=0

notify() {
  echo "$1"
  [[ -n "$WEBHOOK" ]] && curl -s -m 10 -H 'Content-Type: application/json' \
    -d "$(jq -n --arg t "$1" '{text: $t}')" "$WEBHOOK" >/dev/null
}

for job in "$@"; do
  json=$(curl -fsS -m 15 -u "$JENKINS_USER:$JENKINS_TOKEN" \
         "$JENKINS_URL/job/$job/lastBuild/api/json?tree=result,number,url,building") \
    || { notify "ERROR: cannot query Jenkins job $job"; failed=1; continue; }

  result=$(jq -r '.result' <<< "$json")
  number=$(jq -r '.number' <<< "$json")
  url=$(jq -r '.url' <<< "$json")

  case "$result" in
    SUCCESS) echo "OK      $job #$number" ;;
    null)    echo "RUNNING $job #$number" ;;
    FAILURE|UNSTABLE)
      notify "Jenkins $job #$number is $result - $url"; failed=1 ;;
    *)       echo "INFO    $job #$number result=$result" ;;
  esac
done
exit "$failed"
```

```
$ ./jenkins_status.sh api-service ui-service
OK      api-service #412
Jenkins ui-service #97 is FAILURE - https://jenkins.example.com/job/ui-service/97/
```

Watch out: Jobs inside folders use the path `job/folder/job/name`. Keep the API token in a secret store or credentials file, not in the crontab. Using `?tree=` limits the JSON size, which matters on big Jenkins instances. Native alternatives: the `post { failure { ... } }` block in the Jenkinsfile, or the Office 365/Slack notification plugins.

> ***What to say to the interviewer:*** *I call the Jenkins JSON API at* `job/<name>/lastBuild/api/json` *with a user and API token, use jq to read the result, number and URL, and send a Teams or Slack message when the result is FAILURE or UNSTABLE. GET calls do not need a CSRF crumb, I use the tree parameter to keep responses small, and I note that a* `post { failure }` *block in the Jenkinsfile is the native way to do the same.*

Scenario-Based

Q16. Production needs an alert when any disk goes above 80%. Write the script.

> **In simple words:** A fuel gauge warning light for every disk. The script checks each filesystem and shouts on Teams or Slack before the tank is empty.

Detailed answer

- `df -P` gives POSIX output with one line per filesystem, so it never wraps and is safe to parse with awk.
- Skip pseudo filesystems (tmpfs, devtmpfs, overlay, squashfs) so you don't get false alarms.
- Strip the `%` sign and compare as an integer.
- Send one message listing every disk that crossed the threshold, and exit non-zero so cron or monitoring can see the failure.

Example:

```
#!/usr/bin/env bash
# disk_alert.sh - alert when any filesystem crosses a usage threshold
set -euo pipefail

THRESHOLD="${1:-80}"
WEBHOOK_URL="${WEBHOOK_URL:?set WEBHOOK_URL (Teams/Slack incoming webhook)}"
HOST="$(hostname -f)"

[[ "$THRESHOLD" =~ ^[0-9]+$ ]] || { echo "usage: $0 [threshold%]" >&2; exit 2; }

alerts="$(df -P -x tmpfs -x devtmpfs -x overlay -x squashfs \
  | awk -v t="$THRESHOLD" 'NR>1 { gsub("%","",$5); if ($5+0 >= t) \
      printf "%s is %s%% full (mount %s)\\n", $1, $5, $6 }')"

if [[ -n "$alerts" ]]; then
  msg="Disk alert on ${HOST} (threshold ${THRESHOLD}%):\n${alerts}"
  curl -fsS -m 10 -H 'Content-Type: application/json' \
       -d "{\"text\": \"${msg}\"}" "$WEBHOOK_URL" >/dev/null
  echo -e "$msg" | logger -t disk_alert
  exit 1
fi
echo "All filesystems below ${THRESHOLD}%"
```

```
# cron: every 10 minutes
*/10 * * * * WEBHOOK_URL=https://example.webhook/abc /opt/scripts/disk_alert.sh 80
```

Watch out: Also check inodes (`df -Pi`), because a disk can be "full" with free space left. Without some state, the script re-alerts every run, so keep a small marker file for cooldown. In real production this belongs in Prometheus/Zabbix (node_exporter `node_filesystem_avail_bytes`), and the script is only a fallback.

> ***What to say to the interviewer:*** *I parse `df -P`, skip tmpfs and overlay, strip the percent sign, and compare each filesystem against a threshold. If any cross it, I post one message to a Teams or Slack webhook and exit non-zero. I run it from cron every 10 minutes and I'd also check inode usage. In a mature setup this is a Prometheus or Zabbix alert rule, and the script is only a backup.*

Q17. Write a script to monitor a web service and alert on failure.

> **In simple words:** A robot that knocks on the website's door every minute. If nobody answers a few times in a row, it calls the on-call person.

Detailed answer

- Use `curl` with a hard timeout, and capture the HTTP status code and the response time.
- Retry a few times before alerting, so one network blip doesn't page anyone.

- Keep a state file so you alert once when it goes down and once when it recovers, not every minute.
- Related: restarting a service automatically when it is down is covered in the Linux chapter.

Example:

```
#!/usr/bin/env bash
# web_check.sh - check a URL, alert on failure and on recovery
set -uo pipefail

URL="${1:?usage: $0 <url> [expected_code]}"
EXPECTED="${2:-200}"
WEBHOOK_URL="${WEBHOOK_URL:?set WEBHOOK_URL}"
STATE="/var/tmp/web_check_$(echo "$URL" | md5sum | cut -c1-8).state"
RETRIES=3

notify() {
  curl -fsS -m 10 -H 'Content-Type: application/json' \
       -d "{\"text\": \"$1\"}" "$WEBHOOK_URL" >/dev/null || true
}

ok=false
for i in $(seq 1 "$RETRIES"); do
  read -r code ttime < <(curl -s -o /dev/null -m 10 \
      -w '%{http_code} %{time_total}' "$URL" || echo "000 0")
  if [[ "$code" == "$EXPECTED" ]]; then ok=true; break; fi
  sleep 5
done

if $ok; then
  [[ -f "$STATE" ]] && { notify "RECOVERED: $URL is back (HTTP $code, ${ttime}s)"; rm -f "$STATE"; }
  exit 0
fi

if [[ ! -f "$STATE" ]]; then
  notify "DOWN: $URL returned HTTP $code after $RETRIES attempts"
  date > "$STATE"
fi
exit 1
```

Watch out: A 200 status can still hide a broken page, so optionally grep the body for a keyword or check a `/health` endpoint that tests dependencies. Run the check from outside the server's own network too, or you'll miss DNS, TLS and load balancer problems. `curl` code `000` means connection or DNS failure, not an HTTP error.

> ***What to say to the interviewer:*** *I use curl with a 10-second timeout to capture the status code and latency, retry three times to avoid false alarms, and keep a state file so the team gets one DOWN message and one RECOVERED message. I also run the same check from outside our network, because that is what users experience. Blackbox exporter or synthetic monitoring is the production-grade version.*

Q18. Write a script to rotate logs and keep only the last 5 files.

> **In simple words:** Like a stack of 5 notebooks. When the current one is full you put it on top of the stack, the oldest one at the bottom gets thrown away, and you start a fresh notebook.

Detailed answer

- Shift `app.log.4 -> app.log.5` ... `app.log.1 -> app.log.2`, then `app.log -> app.log.1`, and delete anything beyond 5.
- Use copy-then-truncate if the app keeps the file open. Otherwise it keeps writing to the renamed file.
- Compress old copies to save space.
- In real life, `logrotate` does exactly this, so show both.

Example:

```
#!/usr/bin/env bash
# rotate_log.sh - keep the last N rotations of a log file
set -euo pipefail

LOG="${1:?usage: $0 <logfile> [keep]}"
KEEP="${2:-5}"
[[ -f "$LOG" ]] || { echo "no such file: $LOG" >&2; exit 1; }

rm -f "${LOG}.${KEEP}.gz"                       # drop the oldest
for ((i=KEEP-1; i>=1; i--)); do
  [[ -f "${LOG}.${i}.gz" ]] && mv "${LOG}.${i}.gz" "${LOG}.$((i+1)).gz"
done
cp -p "$LOG" "${LOG}.1" && : > "$LOG"           # copytruncate: app keeps its fd
gzip -f "${LOG}.1"
echo "rotated $LOG, keeping $KEEP files"
```

```
# The logrotate equivalent: /etc/logrotate.d/myapp
/var/log/myapp/app.log {
    daily
    rotate 5
    compress
    missingok
    notifempty
    copytruncate
}
```

Watch out: With `copytruncate`, lines written between the copy and the truncate can be lost, so a signal-based reopen (`postrotate` + `kill -HUP` or `systemctl reload`) is cleaner if the app supports it. Test with `logrotate -d` for a dry run. In containers, don't rotate inside the container; log to stdout and let the runtime rotate.

> ***What to say to the interviewer:*** *I shift the numbered files up by one, delete the one beyond the keep count, copy the live log to* `.1`*, truncate the original so the app keeps writing, and gzip the copy. In production I'd use a logrotate config with* `rotate 5`*,* `compress` *and* `copytruncate`*, or better, a postrotate reload so no lines are lost.*

Q19. Write a script to fetch the top CPU-consuming processes and email them.

> **In simple words:** A snapshot of the "who is eating the CPU" leaderboard, sent to your inbox, so when someone asks "why was the server slow at 3 AM?" you have evidence.

Detailed answer

- `ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head -n 11` gives the header plus the top 10.

- Add a load average and uptime line for context.
- Send it with `mail` (mailx) or `sendmail`, or post it to Teams/Slack as an alternative.
- Trigger it from cron, or better, only when load goes above a threshold.

Example:

```
#!/usr/bin/env bash
# top_cpu_report.sh - email the top CPU consumers, optionally only above a load threshold
set -euo pipefail

TO="${1:?usage: $0 <email> [load_threshold]}"
LOAD_LIMIT="${2:-0}"
HOST="$(hostname -f)"
load1="$(cut -d' ' -f1 /proc/loadavg)"

# only report when 1-min load is above the limit (0 = always)
if awk -v l="$load1" -v t="$LOAD_LIMIT" 'BEGIN{exit !(l < t)}'; then
  exit 0
fi

report="$(
  echo "Host: $HOST   Time: $(date '+%F %T')   Load: $(cut -d' ' -f1-3 /proc/loadavg)"
  echo "CPUs: $(nproc)"
  echo
  ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%cpu | head -n 11 | cut -c1-150
)"

echo "$report" | mail -s "[${HOST}] Top CPU processes (load ${load1})" "$TO"
echo "report sent to $TO"
```

Watch out: `%cpu` in `ps` is the average over the process lifetime, not the current usage. For a "right now" view, use `top -b -n 2 -d 1` and take the second iteration. `mail` needs a working MTA or relay (postfix/SES SMTP). Compare the load against `nproc`: a load of 8 is fine on 16 cores and bad on 2.

> ***What to say to the interviewer:*** *I use* `ps` *sorted by CPU and take the top 10 with pid, user and command, add the load average and core count for context, and email it with mailx. I trigger it only when the load crosses a threshold, and I mention that* `ps %cpu` *is a lifetime average, so for a live picture I use* `top -b` *in batch mode.*

Q20. Write a script to tail multiple log files in parallel.

> **In simple words:** Watching several CCTV screens on one monitor, with each line labelled by which camera it came from.

Detailed answer

- `tail -F file1 file2` already follows several files and prints `==> file <==` headers. `-F` also survives log rotation.
- For a cleaner view, run one `tail` per file in the background and prefix each line with the file name.
- `trap` kills all the background tails when you press Ctrl+C.
- `multitail` or `lnav` are nicer interactive tools, and `journalctl -f -u a -u b` does the same for systemd services.

Example:

```
#!/usr/bin/env bash
# multitail.sh - follow several logs at once with a per-file prefix
set -euo pipefail
(( $# >= 1 )) || { echo "usage: $0 <log1> [log2 ...]" >&2; exit 2; }

pids=()
trap 'kill "${pids[@]}" 2>/dev/null; exit 0' INT TERM

for f in "$@"; do
  name="$(basename "$f")"
  tail -n 0 -F "$f" 2>/dev/null | sed -u "s/^/[${name}] /" &
  pids+=("$!")
done
wait
```

```
./multitail.sh /var/log/nginx/access.log /var/log/nginx/error.log /var/log/app/app.log | grep -i err
    or
```

Watch out: Use `sed -u` (unbuffered) or `stdbuf -oL`, otherwise output comes out in delayed chunks. `-F` vs `-f`: `-f` stops following after rotation. A file name containing `/` or `&` would break the sed expression, so I use the basename. For many servers, use a central log system (Loki/ELK) instead of tailing by hand.

> ***What to say to the interviewer:*** *For a quick look,* `tail -F` *with several files is enough. For a readable stream, I start one* `tail -F` *per file in the background, prefix each line with the file name using unbuffered sed, and trap Ctrl+C to kill all of them. Across many servers I'd use Loki or journalctl rather than tailing each one.*

Q21. Write a script to find zombie processes and clean them up.

> **In simple words:** A zombie is a finished worker whose manager never signed the exit paperwork. You can't fire someone who has already left; you have to nudge the manager (the parent) to do the paperwork, or replace the manager.

Detailed answer

- Zombies have state `Z` in `ps`. They use no CPU or memory, only a PID slot.
- **You cannot kill a zombie**; it is already dead. `kill -9` on it does nothing.
- Fix: send `SIGCHLD` to the parent so it calls `wait()`. If the parent is buggy, restart or kill the parent; the zombies are then re-parented to init/systemd, which reaps them.
- See the Linux chapter's zombie and parent-child process questions for the theory.

Example:

```
#!/usr/bin/env bash
# zombies.sh - list zombies and their parents; optionally nudge or kill the parents
set -euo pipefail
ACTION="${1:-report}"          # report | sigchld | kill-parent

mapfile -t rows < <(ps -eo pid=,ppid=,stat=,comm= | awk '$3 ~ /^Z/')
if (( ${#rows[@]} == 0 )); then echo "no zombie processes"; exit 0; fi

printf '%-8s %-8s %-6s %s\n' PID PPID STAT CMD
printf '%s\n' "${rows[@]}"

mapfile -t parents < <(printf '%s\n' "${rows[@]}" | awk '{print $2}' | sort -u)
for p in "${parents[@]}"; do
  echo "parent $p: $(ps -o comm= -p "$p") has $(printf '%s\n' "${rows[@]}" \
       | awk -v p="$p" '$2==p' | wc -l) zombie(s)"
  case "$ACTION" in
    sigchld)     kill -s SIGCHLD "$p" && echo "  sent SIGCHLD to $p" ;;
    kill-parent) [[ "$p" -ne 1 ]] && kill -TERM "$p" && echo "  terminated parent $p" ;;
  esac
done
```

Watch out: Never kill PID 1. Killing the parent kills the real service, so do it in a maintenance window or restart via systemd. A few short-lived zombies are normal. Thousands of them mean a parent is leaking and can exhaust the PID limit (`kernel.pid_max`), so fix the code or run with an init like `tini` in containers.

> ***What to say to the interviewer:*** *I list processes in state Z with their parent PIDs. Since a zombie is already dead, I first send SIGCHLD to the parent so it reaps its children. If that doesn't work, I restart or kill the parent and let init reap them. The real fix is in the parent's code, or using tini as PID 1 in containers.*

Q22. Write a script to monitor network bandwidth in real time.

> **In simple words:** Reading the electricity meter twice, one second apart. The difference tells you how fast the power is being used right now.

Detailed answer

- The kernel keeps byte counters per interface in `/sys/class/net/<if>/statistics/rx_bytes` and `tx_bytes`.
- Read them, sleep 1 second, read them again; the difference is bytes per second. Multiply by 8 for bits.
- No packages are needed, so it works on minimal servers and inside containers.
- Tools like `iftop` (per connection), `nload`, `bmon`, `sar -n DEV 1` and `vnstat` (history) do this with more detail.

Example:

```
#!/usr/bin/env bash
# bw.sh - live RX/TX rate for an interface, refreshed every second
set -euo pipefail

IFACE="${1:-$(ip route show default | awk '{print $5; exit}')}"
INTERVAL="${2:-1}"
S="/sys/class/net/${IFACE}/statistics"
[[ -d "$S" ]] || { echo "interface not found: $IFACE" >&2; exit 1; }

human() { numfmt --to=iec --suffix=B/s "$1"; }

rx1=$(<"$S/rx_bytes"); tx1=$(<"$S/tx_bytes")
echo "Monitoring $IFACE every ${INTERVAL}s (Ctrl+C to stop)"
while sleep "$INTERVAL"; do
  rx2=$(<"$S/rx_bytes"); tx2=$(<"$S/tx_bytes")
  rx=$(( (rx2 - rx1) / INTERVAL )); tx=$(( (tx2 - tx1) / INTERVAL ))
  printf '%s  RX %-12s TX %-12s (%d / %d Mbit/s)\n' "$(date +%T)" \
    "$(human "$rx")" "$(human "$tx")" $(( rx*8/1000000 )) $(( tx*8/1000000 ))
  rx1=$rx2; tx1=$tx2
done
```

```
Monitoring eth0 every 1s (Ctrl+C to stop)
10:42:01  RX 1.2MB/s      TX 350KB/s      (10 / 2 Mbit/s)
```

Watch out: Know the instance's bandwidth limit (on AWS it depends on instance size; watch `bw_in_allowance_exceeded` via `ethtool -S`). Bytes vs bits: network links are sold in bits. Counters are per interface, so bonded or VLAN interfaces and the docker0 bridge each have their own.

> ***What to say to the interviewer:*** *I read the rx and tx byte counters from sysfs, sleep a second, read them again, and print the difference as bytes and megabits per second. It needs no extra packages. For per-connection detail I use iftop, and for history sar or vnstat. In production, node_exporter exposes the same counters to Prometheus.*

Q23. Write a script that alerts if a Docker container is not running.

> **In simple words:** A headcount at the start of every shift. If a named container is missing or unhealthy, the supervisor gets a message.

Detailed answer

- `docker inspect -f '{{.State.Status}}'` returns `running`, `exited`, `restarting` and so on. It errors if the container doesn't exist.
- Also read the health status (`{{.State.Health.Status}}`) when the image has a HEALTHCHECK, because "running" is not the same as "healthy".
- Alert with the exit code, restart count and last log lines to speed up triage.
- Optionally try `docker start` once, but still alert so a human looks at the root cause.

Example:

```
#!/usr/bin/env bash
# container_check.sh - alert when containers are missing, stopped or unhealthy
set -uo pipefail

WEBHOOK_URL="${WEBHOOK_URL:?set WEBHOOK_URL}"
(( $# >= 1 )) || { echo "usage: $0 <container> [container ...]" >&2; exit 2; }
problems=()

for c in "$@"; do
  if ! status=$(docker inspect -f '{{.State.Status}}' "$c" 2>/dev/null); then
    problems+=("$c: NOT FOUND"); continue
  fi
  health=$(docker inspect -f '{{if .State.Health}}{{.State.Health.Status}}{{end}}' "$c")
  if [[ "$status" != "running" || "$health" == "unhealthy" ]]; then
    code=$(docker inspect -f '{{.State.ExitCode}}' "$c")
    restarts=$(docker inspect -f '{{.RestartCount}}' "$c")
    last=$(docker logs --tail 3 "$c" 2>&1 | tr '\n"' ' _' | cut -c1-200)
    problems+=("$c: status=$status health=${health:-n/a} exit=$code restarts=$restarts | $last")
  fi
done

if (( ${#problems[@]} )); then
  msg="Container alert on $(hostname): $(printf '%s; ' "${problems[@]}")"
  curl -fsS -m 10 -H 'Content-Type: application/json' -d "{\"text\": \"$msg\"}" \
       "$WEBHOOK_URL" >/dev/null
  echo "$msg"; exit 1
fi
echo "all containers running"
```

Watch out: The user running it needs Docker socket access, and membership of the `docker` group is effectively root, so run it as a dedicated service user. A container in a restart loop shows `restarting` or keeps a rising RestartCount, so alert on that too. For many hosts use cAdvisor + Prometheus (`time() - container_last_seen`) instead of scripts.

> ***What to say to the interviewer:*** *For each expected container I use docker inspect to read the status and health, and if it's missing, stopped or unhealthy I send one webhook message with the exit code, restart count and last log lines. A restart policy handles recovery, and the script makes sure a human knows. At scale I'd rely on cAdvisor metrics and Prometheus alerts.*

Q24. Write a script to auto-deploy a static website using rsync.

> **In simple words:** Instead of repainting the shop while customers are inside, you set up the new display in a back room and then swap the sign on the door in one second.

Detailed answer

- Build locally (or in CI), then rsync to a new timestamped release folder on the server.
- Switch a `current` symlink atomically to the new release, so users never see half-uploaded files.
- Keep the last few releases for instant rollback.
- Use `--dry-run` first, and use `--delete` so removed files disappear from the release.

Example:

```
#!/usr/bin/env bash
# deploy_static.sh - atomic static site deploy with rsync + symlink switch
set -euo pipefail

SRC="${1:?usage: $0 <build_dir> <user@host> [base_dir]}"
TARGET="${2:?missing user@host}"
BASE="${3:-/var/www/site}"
KEEP=5
REL="$(date +%Y%m%d%H%M%S)"
SSH="ssh -o BatchMode=yes -o ConnectTimeout=10"

[[ -f "$SRC/index.html" ]] || { echo "build dir looks wrong: $SRC" >&2; exit 1; }

echo "Dry run:"; rsync -azn --delete -e "$SSH" "$SRC"/ "$TARGET:$BASE/releases/$REL/" | tail -5

$SSH "$TARGET" "mkdir -p '$BASE/releases/$REL'"
# --link-dest reuses unchanged files from the current release (fast, saves space)
rsync -az --delete --link-dest="$BASE/current/" -e "$SSH" \
      "$SRC"/ "$TARGET:$BASE/releases/$REL/"

$SSH "$TARGET" "ln -sfn '$BASE/releases/$REL' '$BASE/current.tmp' \
  && mv -T '$BASE/current.tmp' '$BASE/current' \
  && ls -1dt '$BASE'/releases/* | tail -n +$((KEEP+1)) | xargs -r rm -rf"

echo "Deployed release $REL"
# Rollback: ln -sfn $BASE/releases/<previous> $BASE/current.tmp && mv -T ... current
```

Watch out: `mv -T` of a symlink is atomic; `ln -sfn` alone on the live link has a tiny window. Point the nginx root at `$BASE/current`. Use a deploy-only SSH key restricted in `authorized_keys`. A trailing slash on the rsync source means "copy the contents", not the folder itself. With a CDN in front, invalidate the cache after the switch.

> ***What to say to the interviewer:*** *I rsync the build output into a new timestamped release directory over SSH, using --link-dest so unchanged files are hard-linked, then switch a current symlink atomically with mv -T, so users never see a half-deployed site. I keep the last five releases, so rollback is just pointing the symlink back, and I run a dry run first.*